

Custom Compiler™ Technology and Display Resource File Reference Manual

Version W-2024.09, September 2024



Copyright and Proprietary Information Notice

© 2024 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>. All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Contents

New in This Release	18
Related Products, Publications, and Trademarks	18
Conventions	18
Customer Support	19
Statement on Inclusivity and Diversity	20

1. Technology and Display Resource Files	21
Technology File	21
Display Resource File	23
Managing Technology Data and Display Resource Data	24
Tools for Managing the Technology and Display Resource Data	25
About the Technology Manager	25
Reference Technologies Between Libraries	26
Export Technology Data to an ASCII File	27
Edit Technology Data	27
Import Technology Data Into a Library	28
Delete Technology Data	30
Revert Technology Data	30
Incremental Technology Data	30
Export Incremental Technology Data	30
Importing Incremental Technology Data	31
Using Tcl Commands	32

2. Technology File Contents	33
System- and Process-Specific Technology Files	33
System-specific Technology File	34
Process-specific Technology Files	34
File Header	34
Controls Section	35
techArray	36
techParams	36
techPermissions	37

Contents

viewTypeUnits	38
mfgGridResolution	38
leMPPDefinition	39
leMPPControls	39
leMPPDefinition	41
leMPPRingObject	42
leMPPMasterPath	44
leMPPEnclosedPath	45
leMPPOffsetPath	47
leMPPSubRect	49
leMPPEnclosedShape	52
colors	53
vdrc	54
Layer Definitions Section	54
techPurposes	55
techLayers	59
techLayerPurposePriorities	62
techDisplays	63
techLayerProperties	64
derivedLayerOpDefs	65
techDerivedLayers	66
Layer Rules Section	69
functions	70
mfgResolutions	72
routingDirections	73
currentDensities	73
cutClasses	76
trackPatternDefs	77
colorClasses	78
trackDefs	80
grids()	82
tracks()	83
groups()	86
regions()	88
boundaryTrackData()	92
boundaryTracks()	94
boundaryTrackRegion()	96
trackDefs Example	97
wideAlignments	98
streamLayerMapping	112

Contents

pinTextLayers	114
voltageTextLayers	115
Via Definitions Section	116
standardViaDefs	117
standardViaVariants	119
customViaDefs	120
customViaVariants	122
bridgeVias	123
Constraint Groups Section	126
constraintGroups()	126
foundry	128
drcRuleDefs	128
Region Based Rules	130
lppSpec	130
constraintGroup	132
Site Definitions Section	133
3. Custom Compiler Technology Data and OpenAccess	136
How Custom Compiler Technology Data is Represented in OpenAccess	136
Attributes or Properties on OpenAccess Objects	137
Getting the Technology Object from a Library or a Design	137
Getting the Manufacturing Grid and Other Parameters Information	137
Getting the Layer Number and Layer Name Information	138
Getting the Required Layer Constraints	138
Getting the Required Layer Pair Constraints	139
Getting the Via Names and Via Definitions	139
4. Constraints Overview	140
How the Custom Compiler Components Interact with Constraints	141
Interactions with the Technology File	142
Changing Rules for an Existing Library	142
Understanding and Interpreting 1-D, 2-D, and 3-D Table Values	143
Range Data Syntax	148
cutClass and cutClassList	149
Boolean Values	150

Contents

Common Options	151
Constraint Types	152
Voltage Dependent Design Rules	152
Forbidden End-to-End Rules	153
5. drcRule Definition	156
6. Simple Constraints	158
cumulativeMetalAntenna	159
cumulativeViaAntenna	161
default135RouteGridOffset	162
default135RouteGridPitch	163
default45RouteGridOffset	165
default45RouteGridPitch	166
defaultHorizontalRouteGridOffset	167
defaultHorizontalRouteGridPitch	168
defaultVerticalRouteGridOffset	169
defaultVerticalRouteGridPitch	170
excludeRoutingVias	172
forbiddenOffTrackSpacing	172
horizontalPlacementGridOffset	173
horizontalPlacementGridPitch	175
leConnectingLayers	176
maxFloatingArea	176
minUncutEndToEndSpacing	178
shapeAngle	179
taperHalo	179
validBridgeVias	180
validRoutingLayers	181
validRoutingVias	183
verticalPlacementGridOffset	184
verticalPlacementGridPitch	185

viaStackLimit	186
7. Layer Constraints	190
135RouteGridOffset	197
135RouteGridPitch	198
45RouteGridOffset	199
45RouteGridPitch	201
alignedWireSpacing	202
alignToOtherLayerTracks	203
allowedCutClass	204
allowedCutSpacing	206
allowedLengthRange	211
allowedEndToEndSpacing	212
allowedEndToSideSpacing	214
allowedNeighborViaGrid	216
allowedParallelSpanSpacing	218
allowedSpacingRange	221
allowedSpanLengthRanges	227
allowedViaGroupSpacing	228
allowedWidthRange	229
allowedWireSpacing	230
antenna	235
areaCapacitance	237
commonNeighborViaCount	237
concaveCornerKeepout	239
cornerViaClusterSpacing	240
criticalEndKeepout	245
cumulativeLayerAntenna	247
cutClass	248
cutMetalAvoidanceKeepout	249
defWidth	250
diagonalShapesAllowed	251

Contents

dualEndOfLineKeepout	251
edgeCapacitance	253
enclosureMergeUnderErasing	254
endOfLineKeepout	255
endOfLineNeighborKeepout	265
endOfLineProjectionSpacing	267
erasingLayer	269
erasingShapeNeededThreshold	271
errorLayer	272
excludeSignalTypeRouting	272
extendedNotchSpacing	273
forbiddenPitch	275
forbiddenSpacing	276
forbiddenTwoSidedViaSpacing	279
forbiddenWidthTransition	284
gateClearance	287
gateOrientation	288
generalCutClassSpacing	289
generalSpanSpacing	296
generalViaClusterSpacing	298
horizontalRouteGridOffset	303
horizontalRouteGridPitch	305
keepAlignedShapeAndBoundary	306
layerShapeAngle	307
maxACCurrentDensity	308
maxClusterDiagonalNumCuts	309
maxDCCurrentDensity	311
maxDensity	312
maxDiagonalEdgeLength	313
maxDiffDensity	314
maxInlineEndOfLineProjections	316
maxInlineNumCuts	317

Contents

maxLength	320
maxNeighborCuts	321
maxNumWiresEndOfLineKeepout	326
maxRoutingDistance	328
maxTapSpacing	329
maxUnalignedViaNeighbors	330
maxViaClusterNumCuts	331
maxWidth	335
maxWireWidthCount	336
mergeSpaceAllowed	337
minAdjacentEndOfLineProjectionSpacing	339
minAdjacentViaSpacing	340
minArea	344
minAreaEdgeLength	345
minBoundaryExtension	347
minBoundaryExteriorHalo	348
minBoundaryInteriorHalo	350
minCenterToCenterSpacing	351
minClusterDiagonalSpacing	352
minConcaveEdgeLength	353
minConstrainedViaSpacing	355
minConvexEdgeLength	356
minCornerToCornerDistance	358
minCutClassSpacing	359
minDensity	364
minDiagonalEdgeLength	366
minDiagonalSpacing	367
minDiagonalWidth	369
minDiffPotentialSpacing	370
minEdgeAdjacentDistance	372
minEdgeAdjacentLength	373
minEdgeByType	376

Contents

minEdgeLength	377
minEdgeMaxCount	379
minEnclosedArea	380
minEnclosedSpacing	382
minEndOfLineAdjacentToStep	384
minEndOfLineDirectionSpacing	385
minEndOfLineExtensionSpacing	387
minEndOfLineJogDistance	388
minEndOfLineNotchSpacing	389
minEndOfLineOrthoSpacing	391
minEndOfLinePerpSpacing	392
minEndOfLineSpacing	394
minEndOfLineToConcaveCornerSpacing	403
minEndOfLineViaNeighborSpacing	405
minEndOfNotchSpacing	408
minFillPatternSpacing	410
minFillToFillSpacing	411
minGeneralViaSpacing	412
minInnerConcaveCornerDistance	415
minLargeViaArrayCutSpacing	416
minLargeViaArraySpacing	418
minLargeViaArrayWidth	419
minLength	420
minLengthAlignedProximity	422
minMaskArea	423
minNeighborRegionParallelSpacing	424
minNeighborViaSpacing	426
minNotchSpacing	428
minNotchSpanSpacing	430
minNumCut	431
minOppositeSpanSpacing	433
minOrthogonalViaSpacing	436

Contents

minParallelSpanSpacing	438
minParallelViaSpacing	439
minParallelWithinViaSpacing	441
minPerimeter	443
minProtrudedProximitySpacing	443
minProtrusionLength	445
minProtrusionNumCut	447
minProtrusionSpacing	449
minProtrusionWidth	450
minProximitySpacing	452
minRectArea	454
minRectAreaProximity	455
minRedundantViaSetback	456
minSameMetalSharedEdgeViaSpacing	457
minSameNetSpacing	459
minSeparatedViaSpacing	461
minSideSpacing	462
minSize	464
minSpacing	465
minViaRegionClusterSpacing	469
minViaSpacing	471
minVoltageSpacing	475
minWidth	476
minWireExtension	477
multiEndOfLineEdgeKeepout	479
multiPatternCutSpacing	481
multiPatternSpacing	484
nonIsolatedViaCornerSpacing	489
nonSelfAlignedViaSpacing	492
onlyLargestEnclosedCutClassAllowed	493
onlyRectanglesAllowed	494
onTrackRoutingOnly	495

Contents

oppositeEndOfLineSpacing	496
parallelEndOfLineKeepout	497
preferredRoutingDirection	499
preferredRoutingOnly	501
rectangularLargeViaArraysAllowed	501
rectangularShapeDirection	503
selfAlignedViaClusterSpacing	504
selfAlignedViaDiagMinSpacing	507
selfAlignedViaMaxAlignedCuts	508
selfAlignedViaSpacing	509
selfAlignedViaZigzagMinSpacing	510
sheetResistance	512
stoppingLayer	513
threeWireForbiddenSpacing	515
twoSidedParallelMinSpacing	516
unalignedEndToEndKeepout	517
verticalRouteGridOffset	520
verticalRouteGridPitch	521
8. Layer Pair Constraints	523
allowedClearanceRange	525
allowedEnclosedViaProximitySpacing	528
allowedViaClusters	530
concaveCornerToCutSpacing	532
convexEndOfLineExtension	534
denseEndOfLineExtensionSpacing	536
forbiddenTwoSidedViaClearance	538
generalCutClassClearance	540
illegalHGatePattern	543
keepAlignedShapes	544
maxClearance	546
maxExtension	547

Contents

maxInteractCutKeepout	548
maxTouchingDirectionLength	550
metalJointViaExtension	551
minCenterlineExtension	552
minClearance	556
minConcaveConvexEdgeExtension	559
minConcaveCornerExtension	561
minConcaveCornerOverlap	563
minConstrainedViaExtension	564
minCrossMetalViaGroupExtension	566
minCutClassClearance	569
minCutMetalOverlap	574
minCutRoutingClearance	575
minCutToMetalCornerLength	580
minCutToMetalSpacing	582
minDualEndOfLineExtension	584
minDualExtension	589
minEndOfLineEdgeExtension	594
minEndOfLineExtension	597
minExtension	599
minExtensionAlignedProximity	601
minExtensionEdge	603
minExtensionToCorner	605
minGateExtension	606
minOuterVertexSpacing	608
minOverlap	609
minSameNetClearance	610
minSideExtension	611
minSparseDenseExtension	613
minSparseDenseViaSpacing	617
minTotalEnclosure	623
minUpperMetalExtension	624

minViaClearance	626
minViaExtension	631
minViaJointExtension	635
minVoltageClearance	637
nonSelfAlignedViaClearance	638
nonSelfAlignedViaEdgeToMetalSpacing	639
selfAlignedViaEdgeClearance	641
selfAlignedViaExtension	642
selfAlignedViaMetalClearance	646
vertexInsideNotAllowed	647
viaStackingAllowed	648
viaSurroundingMetalKeepout	649

9. Layer Array Constraints	657
dummyPolyExtension	658
interLayerExtension	659
maxPolyLength	661
minClearanceOverLayer	662
minInnerVertexSpacing	663
minOneDArrayStructure	663
minParallelViaClearance	665
minTouchingDirectionClearance	666
minTouchingDirectionExtension	667
validRoutingLPPs	669

10. DRD Assist Constraints	670
preferredClearance	670
preferredExtension	671
preferredNotchSpacing	672
preferredSpacing	672
preferredSpanLength	673

11. Design Rule Verification Tcl	674
Commands	674
vr::acute	677
vr::and	678
vr::angle	678
vr::area	679
vr::attach	680
vr::coinEdge	680
vr::connect	681
vr::convexEdge	681
vr::copy	682
vr::cut	683
vr::density	684
vr::donut	686
vr::dfmText	686
vr::dpt	687
vr::exclude	688
vr::extent	688
vr::extents	689
vr::extentCell	689
vr::extendDrawn	690
vr::expandEdge	690
vr::expandText	691
vr::enclose	692
vr::encloseRectangle	693
vr::enclosure	693
vr::external	694
vr::group	694
vr::grow	695
vr::holes	696
vr::include	696
vr::input	697
vr::inside	698
vr::insideCell	698
vr::insideEdge	699
vr::interact	699
vr::internal	700
vr::layer	701

Contents

vr::layerMap	701
vr::length	702
vr::netAreaRatio	703
vr::not	703
vr::offGrid	704
vr::or	705
vr::orEdge	706
vr::outside	706
vr::outsideEdge	707
vr::pathLength	707
vr::perimeter	708
vr::polygon	708
vr::renameText	709
vr::rectangle	709
vr::rectangles	710
vr::rectEnclosure	711
vr::ruleSet	712
vr::sconnect	712
vr::selectRule	713
vr::selectRulebyLayer	713
vr::set	714
vr::shift	716
vr::shrink	717
vr::size	718
vr::skew	719
vr::stamp	719
vr::textLayer	720
vr::textOrder	720
vr::touch	721
vr::touchEdge	722
vr::tpt	722
vr::unselectRule	723
vr::unselectRuleByLayer	724
vr::vertex	724
vr::virtualConnect	725
vr::withCopy	725
vr::withEdge	726
vr::withNeighbor	726
vr::withNet	728

Contents

vr::withProperty	729
vr::withSpace	730
vr::withText	731
vr::withWidth	732
vr::xor	732
Dimension Check Filters	733
Edge Directed	733
Metric	734
Polygon Containment	734
Polygon Filter	735
Connectivity Filter	735
Orientation Filter	736
Projection Filter	737
Angled Filter	737
Corner Filter	737
Intersection Filter	738
Exclude False	739
Output Filter	739
Reversal Filter	740
Information, Error, and Warning Messages	740
A. Sample Technology File	742
System Technology File Sample	742
B. Sample Display Resource File	766
Display Resource File Sample	766
C. Sample Rule Deck File	801
Rule Deck Sample	801
Glossary	803

About This User Guide

This manual describes the contents of the Custom Compiler™ technology file and display resource file and how they interact with your design data.

This preface includes the following sections:

- [New in This Release](#)
- [Related Products, Publications, and Trademarks](#)
- [Conventions](#)
- [Customer Support](#)
- [Statement on Inclusivity and Diversity](#)

New in This Release

Information about new features, enhancements, and changes, known limitations, and resolved Synopsys Technical Action Requests (STARs) is available in the Custom Compiler Release Notes in the SolvNetPlus site.

Related Products, Publications, and Trademarks

For additional information about the Custom Compiler tool, see the documentation on the Synopsys SolvNetPlus support site at the following address:

<https://solvnetplus.synopsys.com>

You might also want to see the documentation for the following related Synopsys products:

Synopsys® Custom Compiler™

OASIS® (Open Artwork System Interchange Standard)

Conventions

The following conventions are used in Synopsys documentation.

Convention	Description
Courier	Indicates syntax, such as <code>write_file</code> .

Convention	Description
<i>Courier italic</i>	Indicates a user-defined value in syntax, such as <code>write_file design_list</code>
Courier bold	Indicates user input—text you type verbatim—in examples, such as <code>prompt> write_file top</code>
Purple	• Within an example, indicates information of special interest. • Within a command-syntax section, indicates a default, such as <code>include_enclosing = true false</code>
[]	Denotes optional arguments in syntax, such as <code>write_file [-format fmt]</code>
...	Indicates that arguments can be repeated as many times as needed, such as <code>pin1 pin2 ... pinN.</code>
	Indicates a choice among alternatives, such as <code>low medium high</code>
\	Indicates a continuation of a command line.
/	Indicates levels of directory structure.
Bold	Indicates a graphical user interface (GUI) element that has an action associated with it.
Edit > Copy	Indicates a path to a menu command, such as opening the Edit menu and choosing Copy .
Ctrl+C	Indicates a keyboard combination, such as holding down the Ctrl key and pressing C.

Customer Support

Customer support is available through SolvNetPlus.

Accessing SolvNetPlus

The SolvNetPlus site includes a knowledge base of technical articles and answers to frequently asked questions about Synopsys tools. The SolvNetPlus site also gives you access to a wide range of Synopsys online services including software downloads, documentation, and technical support.

To access the SolvNetPlus site, go to the following address:

<https://solvnetplus.synopsys.com>

If prompted, enter your user name and password. If you do not have a Synopsys user name and password, follow the instructions to sign up for an account.

If you need help using the SolvNetPlus site, click REGISTRATION HELP in the top-right menu bar.

Contacting Customer Support

To contact Customer Support, go to <https://solvnetplus.synopsys.com>.

Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

1

Technology and Display Resource Files

Describes the purpose of the Custom Compiler technology file and display resource file. Explains how to create, edit, import, and export the files in the Custom Compiler environment.

This section covers the following topics:

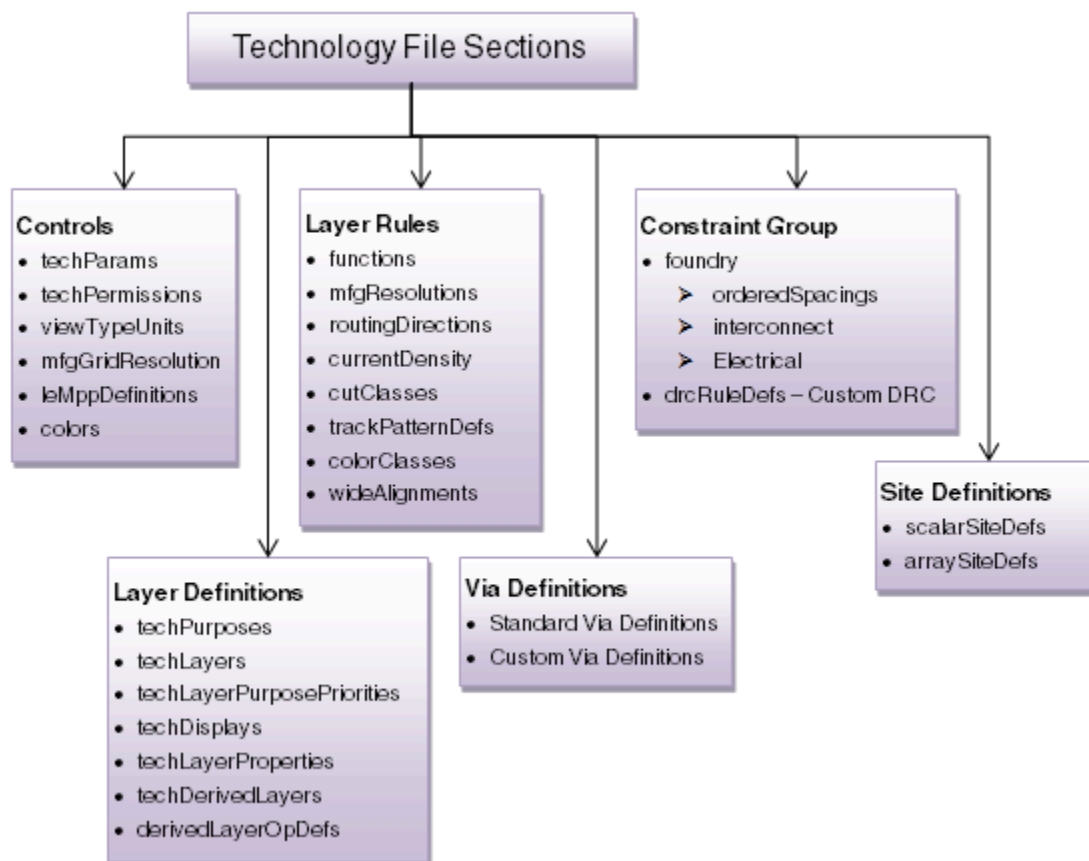
- [Technology File](#)
- [Display Resource File](#)
- [Managing Technology Data and Display Resource Data](#)

Technology File

Technology data represents attributes of a design process, for use by the Custom Compiler environment or other Electronic Design Automation tools.

Technology data is stored in binary format within the Custom Compiler tool. However, you can export and edit the data in ASCII form, and load the data back into your library. The default extension for technology data files is `.tf`, but you might specify any file extension. You can export the display resource file as ASCII data either in Tcl format (`.tcl`) or Display Resource Format (`.drf`).

Custom Compiler technology data includes both a *technology file* and a *display resource file*.



When exported in ASCII format, the technology file consists of several sections and subsections. Each section or subsection begins with a keyword followed by an open parenthesis (. The section attribute statements or subsections follow. Each section ends with a close parenthesis). Each attribute statement within a section consists of a sequence of attributes enclosed in parenthesis. Attributes are separated with one or more spaces.

The following table lists the major sections of the technology file, and refers to the sections in this manual that describe their syntax.

Section	Description
Controls Section	Controls to define: • The default global manufacturing grid • User units for each viewType • DBUperUU (Database Units Per User Unit) for each viewType • Technology parameters • Multi-part path (MPP) definitions • Color control for double / multiple patterning

Section	Description
Layer Definitions Section	Defines: • Tech Purposes - defines the purposes that can be assigned to layer-purpose pairs. • Tech Layers - defines the layers that can be used to define a layer-purpose pair. • Tech Layer Purpose Priorities (LPP) - lists the layer-purpose pairs in priority order. • Tech Displays - defines the display attributes of a layer. • Tech Layer Properties - defines properties for specific layer-purpose pairs such as sheet resistance, edge and capacitance. • Tech Derived Layers - defines the layers derived from other layers.
Layer Rules Section	Defines: • Layer functions • Manufacturing resolutions • Routing directions for layers • Current densities • Cut classes • Track pattern definitions • Color classes
Via Definitions Section	Definitions of standard and custom vias
Constraint Groups Section	Constraint groups that control: • Design rules for layers • Interconnect rules defining the valid routing layers • Electrical Design rules defining the resistances and capacitances for the layers • LEF Default Route Specification Rules • Custom constraints - DRC Rule Definitions Note: You can also view the constraint groups using the Constraint Manager.
Site Definitions Section	Defines a site where you can place cells in a row: • scalarSiteDefs • arraySiteDefs

Display Resource File

The purpose of the display resource file in the Custom Compiler tool is to control how the layers and layer-purpose pairs are displayed on the canvas.

Display resources can be created or edited in one of the two ways from within the Custom Compiler tool:

- Interactively, using the Display Resources Editor tool as described in the *Custom Compiler Environment User Guide*.
- Using the Tcl command-based display resource file. See the Display Resource Tcl chapter in the *Custom Compiler Tcl Reference Manual*.

The display resource file is a text file containing a set of Tcl commands that are interpreted by the Custom Compiler tool.

The display resource file begins with a header that creates the display object or objects to which the rest of the settings in the file apply. You can create multiple display objects with different names, to set the color, stipple, and line style values to whatever limits are appropriate for the kind of display you are defining. Displays can include computer monitors and printers.

In the following example, a display named “display” is created with the maximum number of colors, stipple patterns, and line styles each set to 256, an appropriate setting for a standard computer monitor. A variable named `display_object` is defined, so that it can be referenced in the subsequent sections of the display resource file. In this way, the file can be reused for different displays, and only the name of the display in the Header definitions and any individual color, stipple, line style, or display packet definitions appropriate for that display need to be changed.

Syntax:

```
#DisplayResourceTcl
set display_object[
    db::getNext [dr::getDisplay -filter "%name == {display}"]]
if {$display_object == ""} {
    set display_object[dr::createDisplay display -maxColors 256
        -maxStipples 256 -maxLineStyles 256]}
```

See [Sample Display Resource File](#).

Managing Technology Data and Display Resource Data

This topic describes the following:

- [Tools for Managing the Technology and Display Resource Data](#)
- [About the Technology Manager](#)
- [Reference Technologies Between Libraries](#)
- [Export Technology Data to an ASCII File](#)

- [Edit Technology Data](#)
- [Import Technology Data Into a Library](#)
- [Delete Technology Data](#)
- [Revert Technology Data](#)
- [Incremental Technology Data](#)
- [Export Incremental Technology Data](#)
- [Importing Incremental Technology Data](#)
- [Using Tcl Commands](#)

Tools for Managing the Technology and Display Resource Data

You can manage technology data, assign technology files, and display resource files to libraries, and load or export technology data using the Custom Compiler user interface. You can also enable connectivity and define design rules. You can use the following Custom Compiler tools to control technology data:

Tool	Tasks
Library Manager	Create, open, or close a design library. Attach, detach, or move a technology.
About the Technology Manager	Load a technology file you created. Revert a technology from a previous version.
Constraint Manager	Display and modify constraints and design rules.
Connectivity Engine	Enable or disable connectivity.

The Custom Compiler environment interacts with, and might modify, technology data whenever you use these tools. In addition, it might interact with or change technology data when you exit the Custom Compiler environment. The Custom Compiler environment updates the technology information when you start the Connectivity Engine if connectivity information is missing from the current library.

About the Technology Manager

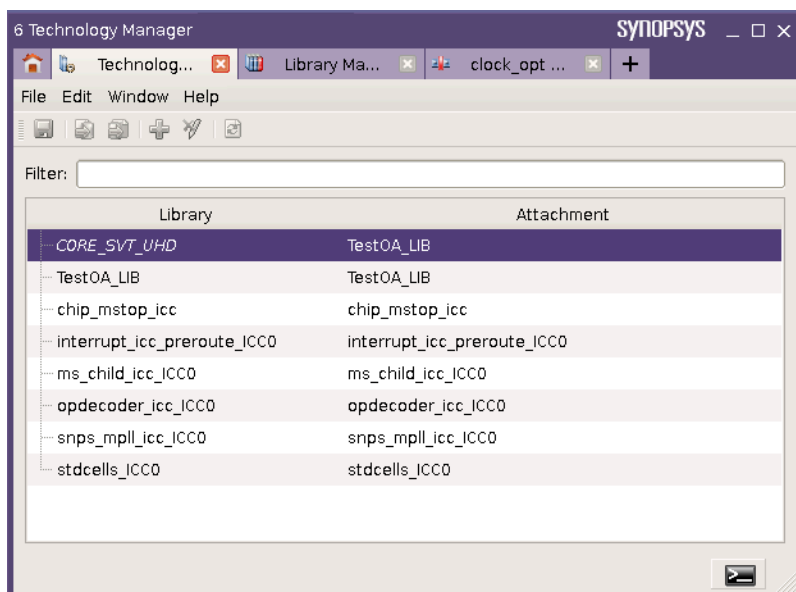
The technology database is located either inside the design library or referenced through inheritance from another library that has the technology you want to use. If referenced, the referenced library must exist first, otherwise, the technology is considered unbound.

The Technology Manager lets you set up access to the correct technology database for the design libraries. The Technology Manager also lets you import, export, modify, and save technology databases.

This topic provides guidance for managing technology data, in context of the Technology Manager. For detailed instructions on using the Technology Manager, see the *Custom Compiler Environment User Guide*.

Reference Technologies Between Libraries

You can reference a technology database when you create a library with the Library Manager. The Technology Manager lets you view which technology libraries are referenced by the current open library. From within the Technology Manager, you can change the technology database reference. For more information, see the *Custom Compiler Environment User Guide*.



In the example shown, the *test* library is referenced to the technology of the reference90RF library.

The property used to identify referenced libraries is `techLibName` in the `oaLibDMDData`. This string specifies the name of another library that either contains the `oaTech` database or that also has a `techLibName` property that refers to another library.

If you refer to another technology file and have an existing local technology, the local technology is backed up and kept in ASCII format in a file named `dbTechDataBackup.tf` attached to the library.

When you import a technology file into a library, if the library's technology reference has an attached local technology library, the local technology library is backed up as `dbTechDataBackup.tf`.

When you have modified but not yet saved a technology file, the Custom Compiler tool creates two backup files: one of them keeps the original version of the technology, the other the latest:

- `dbTechDataBackup.tf` - Latest version of technology data
- `dbTechDataBackupOriginal.tf` - Original version of technology data

When you move a referenced technology library, you are prompted to choose whether:

- Libraries that reference this library change their references to the new library name. If a technology object was opened for any of those libraries, it is reopened from the new (moved) location.
- Libraries that reference this library keep their references, which then become invalid. Any design that was open from such libraries is closed.

Export Technology Data to an ASCII File

When you create a library, the library has a `tech.db` file which has all the technology information associated with the library. If you want to edit the technology data of this library, then you must export the technology file in ASCII format.

You can export technology data in ASCII format using one of the following:

- The Technology Manager

In the Technology Manager, select the library, and then click **File > Export**. The tech data of the library is exported in ASCII format. For more information, see the *Custom Compiler Environment User Guide*.

- The `db::exportTechFile` command.

In the Console, type `db::exportTechFile` and specify the file to which you want to export the data and the library name as arguments. The tech data is exported in ASCII format. See the *Custom Compiler Tcl Reference Manual* for details about this command.

Edit Technology Data

After the technology data is exported to a file as described in [Export Technology Data to an ASCII File](#), you can edit the technology information using any text editor. Edit the constraints group section of the technology file to add new constraints or modify existing constraints.

See [Constraint Groups Section](#) to learn more about specifying constraints.

Import Technology Data Into a Library

After you have edited the technology as described in [Edit Technology Data](#), for those technology changes to be applied to the design in your library, you must import the technology file into the library.

You can replace the current technology data in the library by importing a new technology file using either of the following methods:

- The Technology Manager

In the Technology Manager, select the library, and then click **File > Import**. The tech data is imported into the library.

See the *Custom Compiler Environment User Guide* for details about how to use the Technology Manager.

- The `db::importTechFile` command

In the Custom Compiler console type `db::importTechFile` and specify the edited technology file and the library name as arguments. The tech data is imported into the library.

See the *Custom Compiler Tcl Reference Manual* for details about this command.

You can use the Technology Manager multiple times or call the `db::importTechFile` again on the same or a different file to overwrite or merge data based on the arguments you choose. This way, you can define the technology in multiple files and use these tools multiple times to merge different technologies as appropriate.

Caution:

After you load a technology file, you cannot undo the load. The Custom Compiler tool prompts you to save old technology files before loading a new one. After the library has been saved with the new version, you cannot revert the changes.

Overriding Data During an Import

When you import data with the *override* option, the contents of the local technology file are replaced by the data being imported. Note that if this technology file references other technology data, the contents of hierarchically referenced data is kept intact.

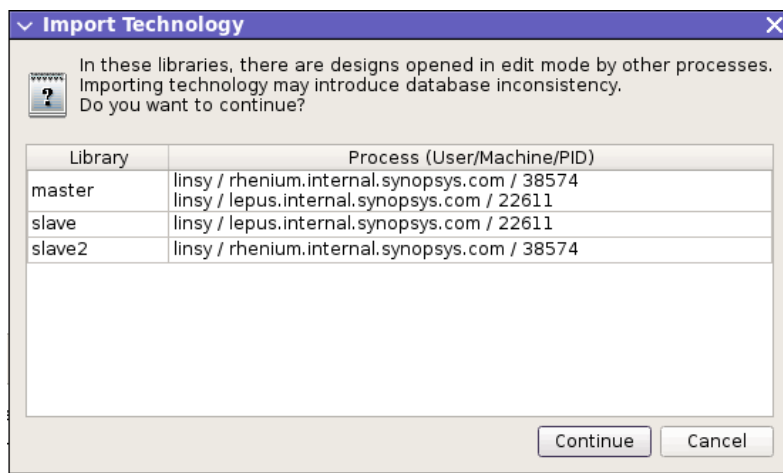
Import Failures

An import can fail in the following cases:

- If there is a conflict due to missing system-reserved layer or layer purpose information, the Custom Compiler tool attempts to create the missing reserved data as low in the technology hierarchy as possible without creating conflicts. If adding this data is not possible, import fails.
- If there is a conflict due to other missing data, for example if a layer is referenced in a constraint but does not exist in the technology hierarchy, import fails.
- If the local technology data conflicts at some point in its hierarchy with the data being imported, import fails.
- If there are conflicting layer-purpose pair (LPP) definitions in the OpenAccess extension technology data, import fails.
- If there is a missing hierarchical reference within incremental technology files, import fails.

Checking Multiple Users

To prevent inconsistent database between processes, the tool checks whether the target library and the libraries that are attached to the target library have designs opened in edit mode by the other process before importing technology.



Note:

The checking mechanism is off by default. Users need to set environment variable `SYNOPTSYS_CUSTOM_IMPORT_TECHNOLOGY_MULTI_USER_CHECK` to enable the checking. The checking happens only when users import Technology File through Technology Manager.

Delete Technology Data

There is no user interface in the Custom Compiler tool to delete technology data from a library. You can only delete technology data from a library using the standard OpenAccess Tcl command `oa::TechDestroy`.

See the OpenAccess documentation tarball from Si2 included in your Custom Compiler installation for details about how to use this command.

When deleting a technology file, the associated `oaTech` data is removed from the database. If the file you are deleting references another technology library, only the reference is removed; the other library remains in the technology database.

Revert Technology Data

There are two ways to revert technology data, either by using the Technology Manager as described in the *Custom Compiler Environment User Guide* or by using the `oa::revert` Tcl command.

If any default technology information is missing, the Custom Compiler tool restores it from the default technology file, merging it into your reverted data.

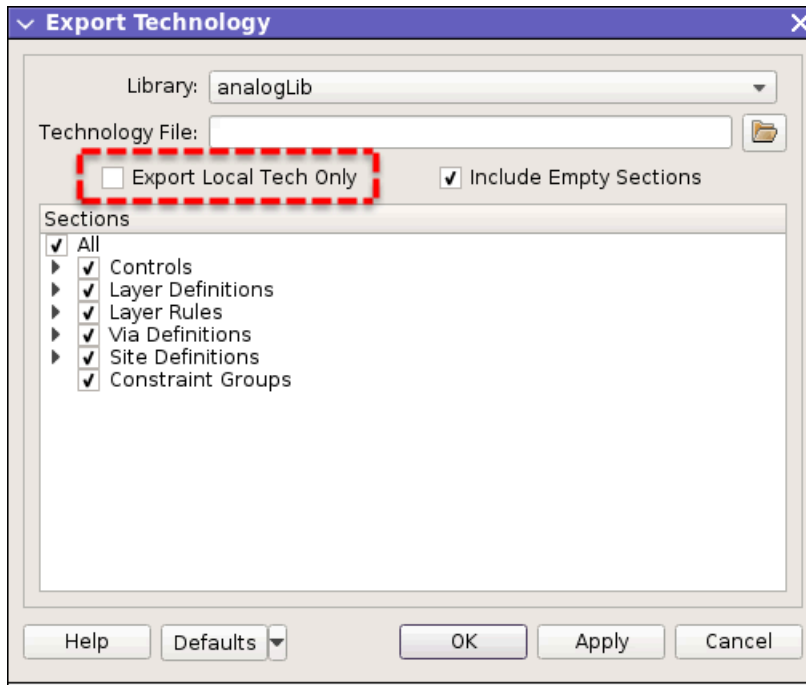
Incremental Technology Data

You can define technology files incrementally, so that one file inherits information from another. You might want to do this to create, for example, site-specific files for different groups in your company that share some technology data and differ in other areas.

You create an incremental technology data by setting up hierarchical definitions in a source ASCII file, then import it into the Custom Compiler tool.

Export Incremental Technology Data

Technology data is always exported in a flat, non-hierarchical format. If you created an incremental technology file, you have the option of exporting only the incremental technology data by selecting the **Export Local Tech Only** option. Alternatively, you can use the `-local` option for the `db::exportTechFile` command.



The difference between a local and non-local export is as follows:

- ☒ **Export Local Tech Only**
Exports only the local (incremental) technology data to a `.tf` file. The reference technology data (base technology) is not included in the export.
Note that the *names* (only) of hierarchically referenced technology libraries are also exported in LISP-like format into the same ASCII technology file. Only the first level techArray names (referenced technology files) are exported, under the techArray keyword in the Controls section of the technology file.
- ☐ **Export Local Tech Only**
Exports all data contained in the technology hierarchy tree to a single `.tf` format. Details about which portions of the technology data belongs to a particular library in the technology hierarchy are discarded.

Importing Incremental Technology Data

If you created an incremental technology data in a source ASCII file, you can import it into the Custom Compiler tool as you would import any other ASCII technology file.

The technology you modify might have a techArray in it. A techArray is an incremental technology file, created when you reference another technology file.

TechArray data is handled in an import as follows:

- If the technology file you are importing has a `techArray`, any `techArray` data in the local file is *replaced* by the one in the imported file.
- If the technology file you are importing does not specify a `techArray`, and you use the override option, any `techArray` data in the local file is discarded.
- If the `techArray` construct in the imported file is empty, the in-memory `techArray` is *removed* if the override option is specified.

It is a good idea, however, to store an original copy of your hierarchical source data file. When you export the data, it is in a *flattened* format.

See [Export Incremental Technology Data](#) for more information.

Using Tcl Commands

The following Custom Compiler Tcl commands let you manage technology data. See the *Custom Compiler Tcl Reference Manual* for syntax of all Tcl commands.

Following is a list of commonly used Tcl commands:

- `db::importTechFile` and `db::exportTechFile` let you import and export technology data.
- `db::getTechReference` lets you set a technology file to be referenced by another, and get the value of the referenced file.
- `db::showManageTechnology` opens the Technology Manager.

In addition, the following OpenAccess Tcl commands can be used to save or delete technology data:

- `oa::TechDestroy` lets you delete technology data.
- `oaTech::save` lets you save technology data.

2

Technology File Contents

Describes the content of the Custom Compiler technology file. You can use the information to create or edit the technology file according to your requirements. A sample technology file is provided in the appendix and is referenced throughout the section.

This chapter covers the following topics:

- [System- and Process-Specific Technology Files](#)
- [File Header](#)
- [Controls Section](#)
- [Layer Definitions Section](#)
- [Layer Rules Section](#)
- [Via Definitions Section](#)
- [Constraint Groups Section](#)
- [Region Based Rules](#)
- [Site Definitions Section](#)

System- and Process-Specific Technology Files

The Custom Compiler tool provides both system-specific and process-specific technology files, as described in the following sections. Normally, you do not need to create a system-specific file, because the Custom Compiler tool imports this information for you when you create a design library. Process-specific files define the information needed by different technology processes. You can create a process-specific technology file yourself, or import one provided to you by your foundry.

The following sections discuss:

- [System-specific Technology File](#)
- [Process-specific Technology Files](#)

System-specific Technology File

The system technology file contains the minimum technology information needed to run the Custom Compiler tool. Custom Compiler automatically loads the default system technology file whenever you create a new design library, so that you can use the Custom Compiler tool even if you do not have a foundry technology file for your current design process.

The default system-specific technology file is available at `install_dir/auxx/tech/cdesigner_1.0.tf`.

See [Sample Technology File](#) to view a sample system technology file.

Process-specific Technology Files

Technology information that describes a particular technology process, such as mask layer names and their properties or design rules, cannot be provided in the system technology file. Separate process-specific technology files store this information.

You should receive your process-specific technology information from your foundry.

File Header

The technology file header consists of comment lines that provide the following information:

- Date and time that the technology data was exported
- Custom Compiler version number
- Library name
- Export condition

The export condition shows whether all of the technology data was exported (full) or only technology data contained in a selected node of the technology tree was exported (local).

The following example file header shows that the technology data was exported on March 9, 2018 at 3:50 PM. The Custom Compiler version number is O-2018.09. The library associated with the technology is Default_1. Only the data in the selected node was exported.

```
;-----  
; Fri Mar  9 15:50:39 2018  
; Custom Compiler Version: O-2018.09  
; Library Name: Default_1  
; Export Condition: Local  
;-----
```

Controls Section

The controls section contains the following subsections, presented in the order in which they appear in the technology file:

Subsection	Purpose
techArray	Lists technology libraries that the current technology library should reference.
techParams	Defines standard parameters such as the mask grid and DRC grid.
techPermissions	Defines user permissions.
viewTypeUnits	Defines the standard units and database units per user unit for each design view.
mfgGridResolution	Defines the resolution of the manufacturing grid.
leMPPDefinition	Opens the section defining multi-part paths.
colors	Defines color control for double / multiple patterning for a mask.
vdrc	Specifies voltage-dependent rule checking (VDRC) parameters.

techArray

The `techArray` section lists technology libraries that the current technology library should reference. This permits the current technology library to inherit information from the referenced technology libraries, thus incrementally building the technology database.

You might want to reference multiple technology libraries to apply design rules and other technology information from multiple sources, such as the foundry, IP provider, or designer.

Syntax:

```
techArray(
    <library_name>
    <library_name>
    ...
)
```

Example 1 Sample techArray section

```
controls (
    ...
    techArray(
        lppLib
        deviceLib
    )
    ...
)
```

For an example technology file, see [Sample Technology File](#).

techParams

The `techParams` section is used to define the technology parameters that are used in the design. In this subclass, you can specify and assign a value to a parameter, then use that parameter throughout the technology file instead of specifying the value. Whenever the parameter is encountered, the software evaluates it to the value assigned to it in this section.

Syntax:

```
techParams (
    ; (Parameter          value      )
    ; (-----)
    (<parameter1>        <value1> )
    (<parameter2>        <value2> )
    ...
);techParams
```


Note:

Any line preceded by a semicolon “;” is treated as a comment.

Example 2 *Sample techParams section*

```
techParams (
; (Parameter          Value )
; (-----)
  ( maskGrid          0.005 )
  ( cadGrid            0.005 )
  ( drcGrid            0.005 )
  ( mfgGrid            0.005 )
  ( scale              1      )
);techParams
```

For an example technology file, see [Sample Technology File](#).

techPermissions

The `techPermissions` section is used to set the read/write permissions on the technology subclasses to specific users. If the section is empty, all users have read/write permissions to all the subclasses of the technology file.

Syntax:

```
techPermissions (
; ( class      (read-only users)      (read & write users))
; (-----)
  ( class      ("user1" "user2")      ("user3" "user4"  ) )
  ...
);techPermissions
```

Argument	Description
Class	The subclass for which you want to set the permissions
read-only users	The list of users (UNIX usernames) for whom you want to set the permissions to be read-only.
read & write users	The list of users (UNIX usernames) who have full read and write permissions on the class.

Note:

Any line preceded by a semicolon “;” is treated as a comment.

Example 3 *Sample techPermissions section*

```
techPermissions (
; ( class      (read-only users)      (read & write users) )
; (-----)
)
```

```
( Devices ("revraj" "vkanchi") ("haplij" "liyi" )
);techPermissions
```

For an example technology file, see [Sample Technology File](#).

viewTypeUnits

The `viewTypeUnits` section is used to define the units for each design view type. This is where you can control the number of database units for each user unit, thereby controlling the look and feel of the design.

Syntax:

```
viewTypeUnits (
    ; ( View Type      User Unit      dbuperUU      )
    ; ( -----      -----      ----- )
    ( <viewType>      <userUnit>      <dbuperuu> )
    ...
);viewTypeUnits
```

Argument	Description
ViewType	The view type such as maskLayout, schematic, or netlist.
UserUnit	The user unit to be used for the specified view type.
dbuperUU	The database to be used per user unit.

Note:

Any line preceded by a semicolon “;” is treated as a comment.

Example 4 *Sample viewTypeUnits section*

```
viewTypeUnits (
    ; ( View Type      User Unit      dbuperUU      )
    ; ( -----      -----      ----- )
    ( maskLayout      "micron"      1000      )
    ( schematic      "inch"      160      )
    ( schematicSymbol "inch"      160      )
    ( netlist      "inch"      160      )
);viewTypeUnits
```

For an example technology file, see [Sample Technology File](#).

mfgGridResolution

The `mfgGridResolution` section gives the resolution for the manufacturing grid in user units.

Syntax:

```
mfgGridResolution(  
    (<value of the manufacturing grid resolution in user units>)  
);mfgGridResolution
```

Note:

Any line preceded by a semicolon “;” is treated as a comment.

Example 5 *Sample mfgGridResolution section*

```
mfgGridResolution(  
    ( 0.005 )  
);mfgGridResolution
```

Assuming that you specified the viewType as “maskLayout” in the previous section, this sets the manufacturing grid resolution for the maskLayout to 0.005 um.

For an example technology file, see [Sample Technology File](#).

leMPPDefinition

This section covers the following multi-part path (MPP) objects:

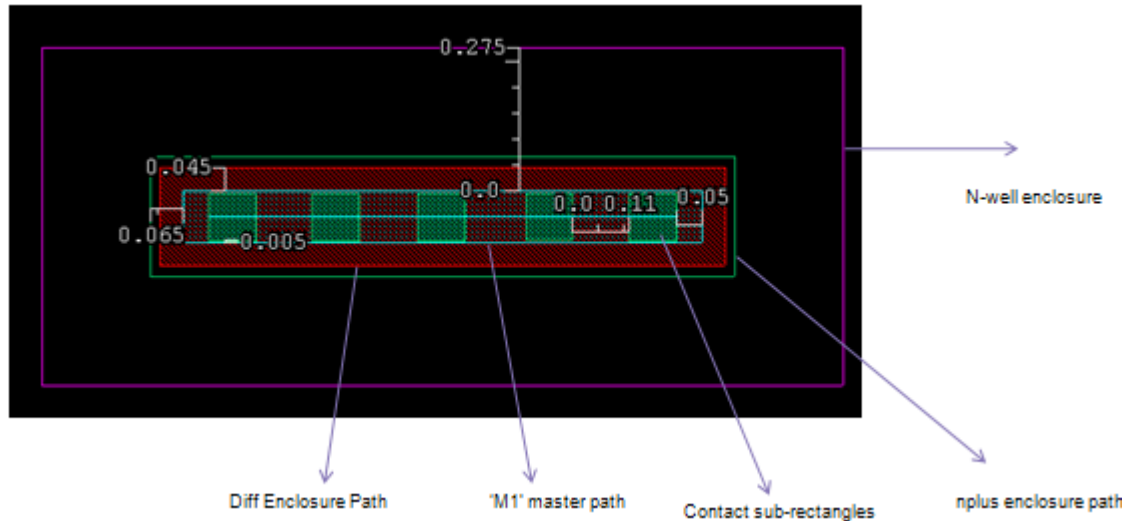
- [leMPPControls](#)
- [leMPPDefinition](#)
- [leMPPRingObject](#)
- [leMPPMasterPath](#)
- [leMPPEnclosedPath](#)
- [leMPPOffsetPath](#)
- [leMPPSubRect](#)
- [leMPPEnclosedShape](#)

leMPPControls

The `leMPPControls` section defines multi-part paths (MPPs) that are used to create guard rings and other complex design objects based upon paths.

MPP is complex object that is formed based on the commands specified under the leMPPDefinition section. The leMPPDefinition section consists of:

- A centerline, which is a poly-line without self intersections
- Path objects constructed based on the MPP centerline and provided values from the MPP Definition
- Rectangle objects



To construct a MPP object, identify the master ring object, such that the whole MPP object is constructed relative to the master ring object.

Note:

The centerline is drawn within the master ring object dividing it into two equal parts.

Syntax:

```
leMPPControls(
;MPP Definition:defines the MPP
  leMPPDefinition(...)
;MPP Ring Object:defines the objects that make up the MPP
  leMPPRingObject(...)
;MPP Master Path:defines the master path for this MPP
  leMPPMasterPath(...)
;MPP Enclosed Path:defines the enclosed path for the MPP, if any
  leMPPEnclosedPath(...)
;MPP Sub-rectangles:defines the contacts that are used in the MPP
  leMPPSubRect(...)
;MPP Enclosed Shape:defines the enclosure shapes for the MPP
  leMPPEnclosedShape(...)
);leMPPControls
```

Note:

Any line preceded by a semicolon “;” is treated as a comment.

For an example technology file, see [Sample Technology File](#).

leMPPDefinition

The MPP definition section contains information about the structure of the MPP objects that are created based on the provided centerline.

To define a MPP definition, you must:

- Define the objects that make up the MPP
- Define the objects that make up the MPP ring object
- Assemble ring object definitions into the MPP definition and mark one of the ring object definitions as the Master Ring Object

There are five types of atomic object definitions:

Atomic Object	Description
Master Path	• Each ring object must contain 1 master path object. • All other atomic objects of the ring object are defined relative to master path of the ring.
Enclosed Path	• Shares the same point list of the MPP. • Width is expressed as enclosure of master path.
Offset Path	• Point list is offset from the master path. • Width is independently defined.
Sub Rectangles	• Must be relative to a Master Path object (parent). • User might define a Spacing Style which defines the way oaRect objects are placed.
Enclosed Shape	• Must be relative to a path object (parent). • Shape defined by enclosure of parent object.

You can create properties on each object of the MPP that your users can modify as they draw or place multi-part paths. You “expose” these properties to the user by setting an attribute on the object called “Exposed Parameters,” and listing the property values in this attribute.

Syntax:

```
leMPPDefinition(  
  <string>objectName  
  <List of strings>ringObjectList
```

```
<List of floats>spacings
<int>masterIndex
);leMPPDefinition
```

Argument	Description
objectName	Defines the name of the Object.
ringObjectList	Ordered list of <i>Ring Objects</i> .
spacings	Spacing between <i>Ring Objects</i> in microns.
masterIndex	Index of Master <i>Ring Object</i> .

Note:

Any line preceded by a semicolon “;” is treated as a comment.

Example 6 Sample leMPPDefinition section

```
leMPPDefinition(
  nwellGR (ntypeGuardRing) (1) 0)
);leMPPDefinition
```

This defines a MPP with the object name `nwellGR` having a ring object `ntypeGuardRing`. The spacing between the spacing between this MPP and another MPP is 1 micron.

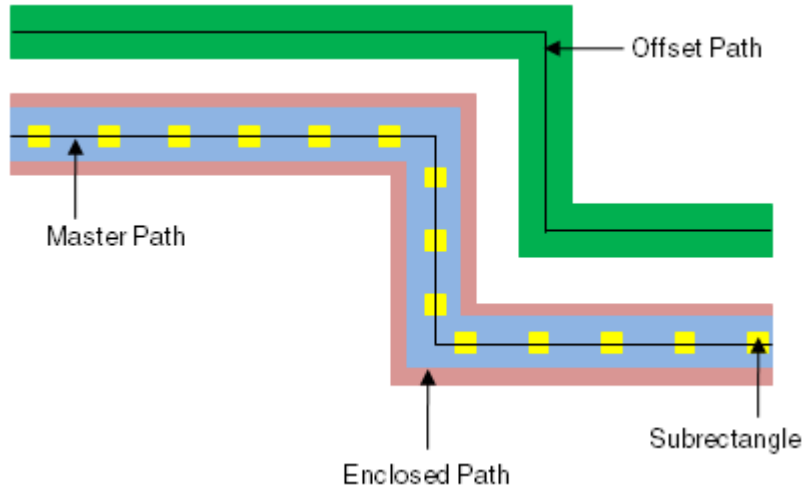
For an example technology file, see [Sample Technology File](#).

leMPPRingObject

After you define the MPP, you define the ring object that you referenced in the `leMPPDefinition`. The ring object definition contains the following:

- Object Name
- Exactly one Master Path
- Number of Enclosed Paths
- Number of Offset Paths
- Number of sub-rectangles
- Number of Enclosed Shapes
- Net Name (default is empty string)
- Exposed Parameters (default is `netName`) - this is a list of enumeration values (full list is specified below)

- name - name of the ring object
- netName - exposed ring object net name



Note:

You can define any number of Enclosed Paths, Offset Paths, and Subrectangles per Master Path.

Syntax:

```
leMPPRingObject (
  <string>objectName
  <string>masterPath
  <string>enclosedPaths
  <string>offsetPaths
  <string>subRects
  <string>enclosedShapes
  <string>netName
  [<List of strings>exposedParams]
);leMPPRingObject
```

Argument	Description
objectName	Ring Object Name used in the MPP Definition section.
masterPath	Name of the master path for the Ring Object.
enclosedPaths	Names of the enclosed paths in the Ring Object.
offsetPaths	Names of the offset paths in the Ring Object.
subRects	Names of any sub-rectangles used in the Ring Object.
enclosedShapes	Names of any enclosure shapes used in the Ring Object.

Argument	Description
netName	The net of the ring object.
exposedParams	The list of parameters that you want to expose to the user, so that the user can modify them as he places the MPP in a design.

Note:

Any line preceded by a semicolon “;” is treated as a comment.

Example 7 *Sample leMPPRingObject section*

```
leMPPRingObject (
  (ntypeGuardRing "defaultPath" (diffEncPath nplusEncPath
    (nwellEncPath) nil (defaultContacts) nil userParams
    (name netName)
  );leMPPRingObject
```

In this example, a N-type MPP ring is created with the name `ntypeGuardRing` referenced by the previous `nwellGR` MPP definition. The master path used to draw the guard ring is `defaultPath`. The enclosed paths are defined by `diffEnclPath`, `nplusEncPath` and `nwellEncPath`.

The first `nil` specifies that there are no offset paths, and the sub-rectangles in the multi-part path are the `defaultContacts`. The second `nil` specifies that no net name is assigned to the ring object. The exposed parameters associated with the MPP ring are the `name` and `netName`.

For an example technology file, see [Sample Technology File](#).

leMPPMasterPath

Each ring object must contain one master path object. All other atomic objects of a ring object are defined relative to the master path of the ring.

Syntax:

```
leMPPMasterPath (
  <string>name
  <string>layer
  <string>purpose
  <float>width
  <string>pathStyle
  <bool>conn
  <bool>chop
  <string>extension
  <List of strings>exposedParams
);leMPPMasterPath
```


Argument	Description
<code>name</code>	Defines the name of the master path.
<code>layer</code>	Defines the layer on which the master path must be created.
<code>purpose</code>	Defines the layer purpose on which the master path must be created. The default value for this is 'drawing'.
<code>width</code>	Defines the width of the master path. The default value would be the minimum width of the LPP used to create the master path.
<code>pathStyle</code>	Defines the path style used for the master path. The default value is <i>truncate</i> . Valid values: <i>truncate</i> and <i>extend</i> .
<code>conn</code>	Defines if connectivity must be enabled. The default value is <i>true</i> .
<code>chop</code>	Defines if master path can be chopped. The default value is <i>nil</i> .
<code>extension</code>	Defines the begin and end extension style for the path.
<code>exposedParams</code>	Defines the list of parameters to expose to the user. 'name', 'width', 'layer', 'purpose', 'conn', 'chop' are the possible parameters to expose.

Note:

Any line preceded by a semicolon “;” is treated as a comment.

Example 8 Sample `leMPPMasterPath` section

```
leMPPMasterPath (
  defaultPath m1 drawing 0.1 extend t t
  userParams(name width layer purpose conn chop)
);leMPPMasterPath
```

This example creates a master path called `defaultPath` on the `m1` drawing layer. The path width is `0.1` um, the path end style is `extend`, connectivity is `true` (`t`), and the layer can be chopped (`t`). The `userParams` associated with the master path are `name`, `width`, `layer`, `purpose`, `conn`, and `chop`.

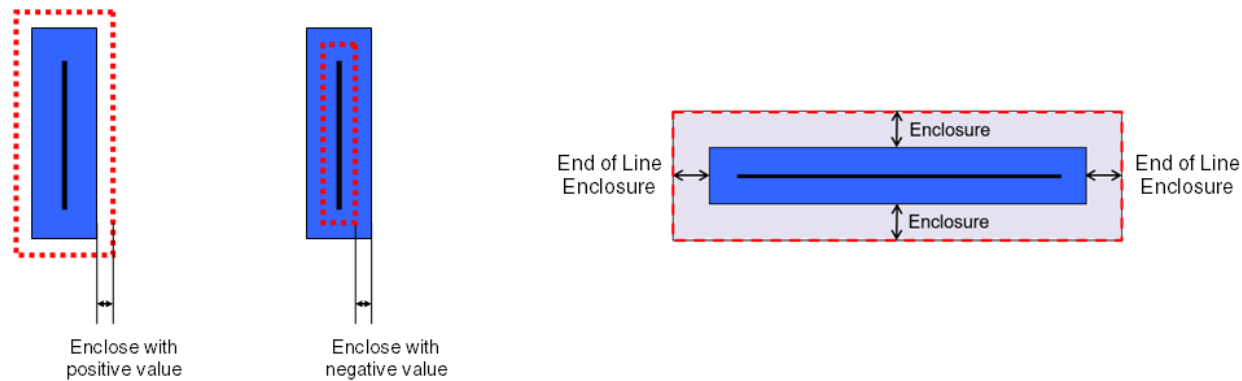
For an example technology file, see [Sample Technology File](#).

leMPPEnclosedPath

After the master path is created, you define the enclosure paths for the MPP, if any. The `leMPPEnclosedPath` statement creates an enclosed path relative to the master path. The `leMPPEnclosedPath` shares the same point list as the master path.

The width of this path is:

$$EnclosedPathWidth = (MasterPathWidth + (2 \times Enclosure))$$



Syntax:

```
leMPPEnclosedPath(  
  <string>name  
  <string>layer  
  <string>purpose  
  <float>enclosure  
  <string>pathStyle  
  <bool>conn  
  <bool>chop  
  <string>extension  
  <float>beginOffset  
  <float>endOffset  
  <List of strings>exposedParams  
  [-eolEnclosure <int | float>]  
) ;leMPPEnclosedPath
```

Argument	Description
name	Defines the name of the enclosed path.
layer	Layer used for creating the enclosed path.
purpose	Layer purpose used for creating the enclosed path.
enclosure	Distance between the edge of the path and the boundary of the enclosure. The default value is the minimum extension of the layer purpose pair (LPP) used to create the enclosure path over the LPP used for the master path.
pathStyle	Style of the master path. The default value is <i>truncate</i> .
conn	Enables connectivity. The default value is <i>true</i> .
chop	Specifies whether the enclosed path can be chopped. The default value is <i>nil</i> .
extension	Defines the begin and end extension style for the path.
beginOffset	The begin of line offset.
endOffset	The end of line offset.

Argument	Description
<code>exposedParams</code>	List of parameters to expose to the user. Possible parameters you can expose to the user are <code>name</code> , <code>layer</code> , <code>purpose</code> , <code>enclosure</code> , <code>conn</code> , and <code>chop</code> .
<code>-eolEnclosure</code>	Distance between the end of the master path and the boundary of the enclosure path. Default: same as the enclosure value.

Note:

Any line preceded by a semicolon “;” is treated as a comment.

Example 9 Sample `leMPPEnclosedPath` section

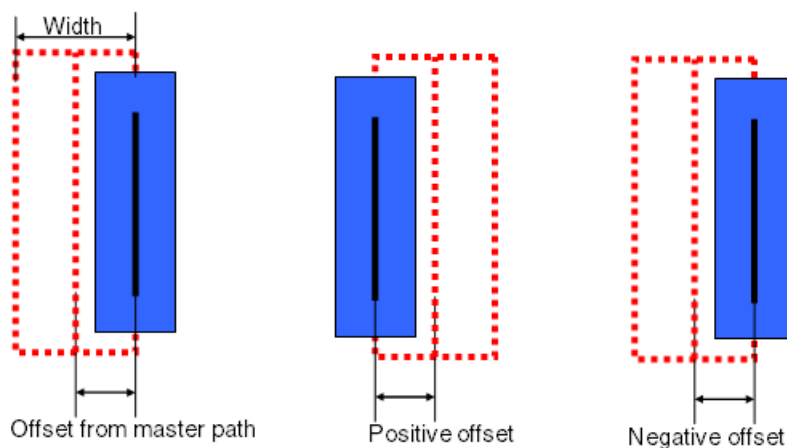
```
leMPPEnclosedPath(
  diffEncPath diff drawing 0.045 extent t nil
  userParams (name layer purpose enclosure conn chop)
  -eolEnclosure 8
);leMPPEnclosedPath
```

The sample `leMPPEnclosedPath` creates an enclosed path on the `diff` drawing layer called `diffEncPath`. The enclosure dimension is 0.045 μm , the path end style is `extend`, connectivity is true (`t`), and the layer cannot be chopped (`nil`). The `userParams` associated with the master path are `name`, `layer`, `purpose`, `enclosure`, `conn`, and `chop`. The end-of-line enclosure distance is 8 μm .

For an example technology file, see [Sample Technology File](#).

leMPPOffsetPath

After the master path is defined, you define the offset paths. This is an optional step. The point list of an offset path is offset from the master path and the width is independently defined. For a positive value, the offset is to the right with respect to the master path and for a negative value, the offset is to the left with respect to the master path.



Syntax:

```
leMPPOffsetPath (
  <string>name
  <string>layer
  <string>purpose
  <float>width
  <float>offset
  <string>pathStyle
  <bool>conn
  <bool>chop
  <float>beginOffset
  <float>endOffset
  <List of strings>exposedParams
);leMPPOffsetPath
```

Argument	Description
name	Defines the name of the offset path.
layer	Defines the layer on which the offset path is created.
purpose	Defines the layer purpose on which the offset path is created. The default value for this is 'drawing'.
width	Defines the width of the offset path. The default value is the minimum width of the layer purpose pair (LPP) used to create the offset path.
offset	Defines the offset value for this path from the master path.
pathStyle	Defines the path style used for the offset path. The default value is <i>truncate</i> .
conn	Defines if connectivity must be enabled. The default value is <i>true</i> .
chop	Defines if the master path can be chopped. The default value is <i>nil</i> .
beginOffset	The begin of line offset.
endOffset	The end of line offset.
exposedParams	Defines the list of parameters to expose to the user. 'name', 'width', 'offset', 'layer', 'purpose', 'conn', 'chop' are the possible parameters to expose.

Note:

Any line preceded by a semicolon “;” is treated as a comment.

Example 10 Sample leMPPOffsetPath section

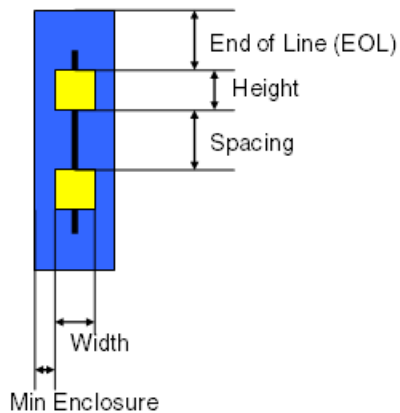
```
leMPPOffsetPath (
  offsetPath1 m2 drawing 0.12 0.01 extend t nil
  userParams(name width offset layer purpose conn chop)
);leMPPOffsetPath
```

This example defines an offset path of width 0.12 μm on layer `m2` with drawing purpose that is offset from the master path by 0.01. The connectivity is true (`t`) and the path cannot be chopped (`nil`).

For an example technology file, see [Sample Technology File](#).

leMPPSubRect

The `leMPPSubRect` creates the sub-rectangles that define the contacts for the defined MPPs. The `leMPPSubRect` statement defines the contacts on the drawing layer.



Note:

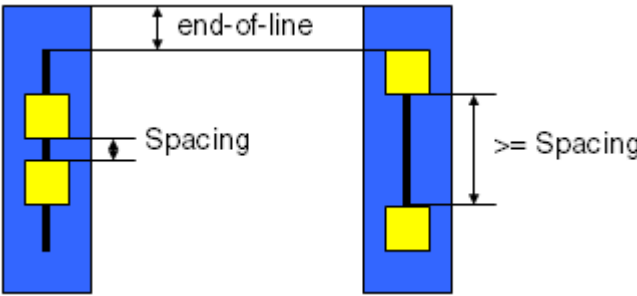
Sub-rectangles must be relative to the master path boundary and not to its centerline. You can define “Spacing Style”, which defines the way the rectangles are placed.

Syntax:

```
leMPPSubRect (
  <string>name
  <string>layer
  <string>purpose
  <float>width
  <float>height
  <bool>conn
  <bool>chop
  <float>minEnclosure
  <float>endOfLine
  <float>spacing
  <float>offset
  <float>beginOffset
  <float>endOffset
  <string>spacingType
  <int>numRows
```

```
<List of strings>exposedParams  
) ;leMPPSubRect
```

Argument	Description
name	Defines the name of the sub-rectangles.
layer	Defines the layer on which the sub-rectangles are created.
purpose	Defines the layer purpose for creating the sub-rectangles.
width	Defines the width of the sub rectangle. The default value is the minimum width of the layer purpose pair (LPP) used to create the sub-rectangles.
height	Defines the height of the sub rectangle. The default value is the minimum width of the LPP used to create the sub-rectangles.
conn	Defines the connectivity for the sub rectangle. The default value is <i>true</i> .
chop	Defines if the sub rectangle can be chopped. The default value is <i>nil</i> .
minEnclosure	Defines the enclosure of the master path over the sub-rectangles. The default value is the minimum extension of the master path LPP over the sub rectangle LPP.
endOfLine	Defines the end-of-line enclosure of the master path over the sub-rectangles. The default value is the minimum extension of the master path LPP over the sub rectangle LPP.
spacing	Defines the spacing between the sub-rectangles. The default value is the minimum spacing value for the LPP used to create the sub-rectangles.
offset	The offset of this sub-rectangle centerline with respect to the master path.
beginOffset	The begin of line offset.
endOffset	The end of line offset.

Argument	Description
<code>spacingType</code>	Defines the spacing type to be used while creating the sub-rectangles. The default value is <i>fit</i>. There are two possible values for the spacing type: • min: Uses minimum spacing from the middle of path segment, based on defined path segment points. • fit: Creates a sub-rectangle at the first and last point, then maximize the fill in the middle, based on defined path segment points. 
<code>numRows</code>	Defines the number of rows of sub-rectangles to be created. The valid value is an integer greater than or equal to -2. The default value is 1. When the value is -1 or -2, this means that it should create as many rows as possible, however Min Enclosure and EOL constraints should be met. For row direction of distribution, -1 uses <i>fit</i> <code>spaceType</code>, other values use <i>min</i> <code>spaceType</code>.
<code>exposedParams</code>	Defines the list of parameters to expose to the user. 'name', 'layer', 'purpose', 'width', 'height', 'conn', 'chop', 'minEnclosure', 'eol', 'spacing', 'spaceType', 'numRows' are the possible parameters to expose.

Note:

Any line preceded by a semicolon “;” is treated as a comment.

Example 11 *Sample leMPPSubRect section*

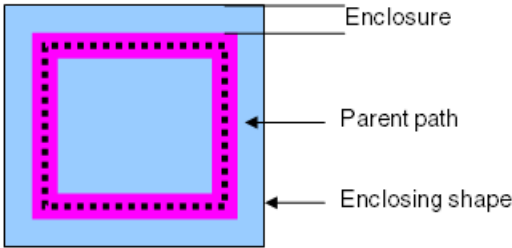
```
leMPPSubRect(
  defaultContacts cont drawing 0.09 0.09 t t 0.005 0.005
  0.1 fit 1 userParams (name layer purpose width height
  conn chop minEnclosure eol spacing spaceType numRows)
);leMPPSubRect
```

The sample `leMPPSubRect` creates the sub-rectangles that define the contacts for the previously-defined guard rings. The `leMPPSubRect` statement defines the contact, named `defaultContacts`. The contacts are on the `cont` drawing layer, with a width and height of 0.09 μm . The layer has connectivity as true (`t`), and it can be chopped (`t`). The enclosure for the contact is 0.005 μm . The spacing is 0.1 μm , with a spacing type (`spaType`) of *fit*. The number of rows of contacts in this MPP guard ring is 1.

For an example technology file, see [Sample Technology File](#).

leMPPEnclosedShape

The `leMPPEnclosedShape` defines the shape for the enclosure of the parent master path.



Syntax:

```
leMPPEnclosedShape (  
  <string>name  
  <string>layer  
  <string>purpose  
  <float>enclosure  
  <bool>conn  
  <List of strings>exposedParams  
) ;leMPPEnclosedShape
```

Argument	Description
name	Defines the name of the enclosing shape.
layer	Defines the layer on which the enclosing shape is created.
purpose	Defines the layer purpose on which the enclosing shape is created.
enclosure	Defines the enclosure of this shape over the master path. The default value is the minimum enclosure of the layer purpose pair (LPP) used for this shape over the LPP used to create the master path.
conn	Defines if connectivity must be enabled. The default value is <i>true</i> .
exposedParams	Defines the list of parameters to expose to the user. 'name', 'layer', 'purpose', 'enclosure', and 'conn' are the possible parameters to expose. The default is ''.

Note:

Any line preceded by a semicolon “;” is treated as a comment.

Example 12 Sample `leMPPEnclosedShape` section

```
leMPPEnclosedShape (encNWELL nwell drawing 0.275 t)
```


The sample `leMPPEnclosedShape` defines the shape for the enclosure. Its name is `encNWell`, and it is on the `nwell` drawing layer. The enclosure is `0.275` um, and the connectivity flag is `(t)`. There are no exposed parameters listed.

For an example technology file, see [Sample Technology File](#).

colors

The `colors` defines the mask color as a string in the ASCII technology file. In OA technology data, it is stored in the ordered `oaGroup` named as “snpsColors” and its `oaGroupMember` is `oaStringProp` with color name. The position of color in the `oaGroup` is mapped to the internal color numbers which starts from 1. Therefore, the position of `colorName` in the string list in the colors section is mapped to the color number used in the Custom Compiler tool internally which starts from 1.

Syntax:

```
colors (
    (<string>colorName1    <string>colorName2)
    (...                  <string>colorNameN)
);colors
```

Argument	Description
<code>colorName</code>	Defines the name of mask color to be used.

Note:

Any line preceded by a semicolon “;” is treated as a comment.

Example 13 Sample colors section

```
colors ((red blue))
```

The sample `colors` section defines the internal number of the color red as 1 and blue as 2.

Note:

Following are some special cases to consider:

- In the tech importing merge mode, if there is an `oaGroup` of colors existing in the technology database, the new colors defined in the merged ASCII technology file replaces the existing `oaGroupMember` in the `oaGroup` of colors by the position of `colorName` in the order. Therefore, the first `colorName` in the string list in ASCII technology file would replace the `oaString` of the first `oaGroupMember` in a `oaGroup` of “snpsColors”.

- In the tech importing merge mode, If there is no oaGroup of colors existing in the technology database or its oaGroupMember is empty, a new one would be created and added into the oaGroup of colors in the order.
- In the tech importing replace mode, if the oaGroup of colors is existing and not empty, then the oaGroup is emptied and creates a new oaGroupMember of oaString for each color in the replaced ASCII technology file in the order.

For an example technology file, see [Sample Technology File](#).

vdrc

The `vdrc` subsection specifies the default minimum and maximum voltage values. If you do not define the default values, the DRD tool will assume a default minimum voltage of 10 and a default maximum voltage of -10. (The voltage difference would be -20.)

During in-design DRC, the DRD tool will use these default values for shapes without voltage annotations. Because the DRD tool checks only positive voltage differences, shapes without voltage annotation will be ignored.

Syntax:

```
vdrc (  
    (<string>vdrcParamName <float>vdrcParamValue)  
    (...)  
);vdrc
```

Argument	Description
vdrcParamName	Name of the VDRC parameter.
vdrcParamValue	Floating value assigned to the VDRC parameter.

Example:

```
vdrc (  
    (minDefaultVoltage 0.0)  
    (maxDefaultVoltage 1.2)  
)
```

For an example technology file, see [Sample Technology File](#).

Layer Definitions Section

The `layerDefinitions` section defines the Layer Purpose Pairs (LPP) that are used to create the designs in the Custom Compiler tool. There are a set of attributes that determine how a LPP would appear on a display device.

A LPP definition requires the following:

- The layer must be defined in the *techLayers* class
- The purpose must be defined in the *techPurposes* class
- The LPP must be listed in priority order in the *techLayerPurposePriorities* class
- The LPPs display definition must be defined in the *techDisplays* class

Syntax:

```
layerDefinitions (  
    <all the subclasses needed to define the layers>  
); layerDefinitions
```

Note:

Any line preceded by a semicolon “;” is treated as a comment.

The following defines the sections inside the `layerDefinitions` section, in the order in which they appear in the technology file.

Subsection	Purpose
techPurposes	Defines the purpose for the layers; includes both user-defined and system-reserved purposes.
techLayers	Lists layer names with numbers; includes both user-defined and system-reserved layer names.
techLayerPurposePriorities	Associates each layer name with a layer purpose, and lists them in priority order. The priorities control which layers appear on top of others in the display (either a computer monitor or printer); the layers listed last appear on top of layers listed earlier.
techDisplays	Associates a display packet (defined in the display resource file) with a layer purpose pair, and assigns that pair some default display properties such as whether the layer is visible in the Object Layer Panel (OLP).
techLayerProperties	Assigns various properties to the layers, such as sheetResistance, edgeCapacitance, and sheetCapacitance.
techDerivedLayers	These are used to define the layers that are derived from other layers,

techPurposes

The `techPurposes` section defines the purposes used to define an LPP object.

The purposes can be of two types:

- User-defined purpose - those required by your design technology.
- System-reserved purpose - those required for the Custom Compiler tool.

The following layer purpose numbers are reserved in the Custom Compiler tool and always have the following reserved names.

Layer Number	Purpose Name
-8	redundant
-7	gapFill
-6	annotation
-5	OPCAntiSerif
-4	OPCSerif
-3	slot
-2	fill
-1	drawing
200	substrate
201	control
202	softBdry
203	tool
204	object
223	fatal
224	critical
225	soCritical
226	soError
227	ackWarn
228	info
229	track
230	blockage
231	grid
234	warning

Layer Number	Purpose Name
235	tool1
236	tool0
237	label
238	flight
239	error
240	annotate
241	drawing1
242	drawing2
243	drawing3
244	drawing4
245	drawing5
246	drawing6
247	drawing7
248	drawing8
249	drawing9
250	boundary
251	pin
253	net
254	cell
255	all
-842194943	steiner
-842194946	dielectricFill
-842194947	activeFill
-842194949	object

Syntax:

```
techPurposes (  
  (<string>purposeName  
   <int>purposeNumber
```

```
<string>purposeAbbreviation)
);techPurposes
```

Note:

Any line preceded by a semicolon “;” is treated as a comment.

Argument	Description
purposeName	The name of the purpose. It should be a string and the name should not conflict with any of the system-reserved purpose names.
purposeNumber	The number of the purpose. This number should be a unique integer, 1 through 128 and 256 through 2 ³² -1026.
purposeAbbreviation	The abbreviation to be used for the purpose. This value should be any string of seven characters or less.

Note:

Some of the applications that display the purpose names do not always have room to display the entire name, so the abbreviation field gives you control over what is displayed in such cases.

The application that displays the purpose name follows the following precedence order when determining how much of the name to display:

The complete purpose name

The purpose abbreviation

The first and last letters of the purpose name

Example 14 *Sample techPurposes section*

```
techPurposes (
; ( PurposeName           Purpose#   Abbreviation )
; ( -----             - - - - -   - - - - - )
;User-defined purpose:
( dmy0                   0           dmy0       )
...
;System-Reserved Purposes:
( redundant              -8          red         )
( gapFill                -7          gap         )
( annotation             -6          ann         )
...
);techPurposes
```

For an example technology file, see [Sample Technology File](#).

techLayers

The `techLayers` section defines the layers used to define an LPP. This section maps the layer names to the corresponding layer numbers for all the layers that are used in the design.

There are two types of layers:

- User-defined Layers
- System-reserved Layers

The following layer numbers are reserved in the Custom Compiler tool and always have the following reserved names.

Layer Number	Layer Name
200	Unrouted
201	Row
202	Group
203	Cannotoccupy
204	Canplace
205	hardFence
206	softFence
207	y0
208	y1
209	y2
210	y3
211	y4
212	y5
213	y6
214	y7
215	y8
216	y9
217	designFlow
218	stretch

Layer Number	Layer Name
219	edgeLayer
220	changedLayer
221	unset
222	unknown
223	spike
224	hiz
225	resist
226	drive
227	supply
228	wire
229	pin
230	text
231	device
232	border
233	snap
234	align
235	prBoundary
236	instance
237	annotate
238	marker
239	select
251	grid
252	axis
253	hilite
254	background
215801850	snpsTrunk
215801851	snpsTapoff

Syntax:

```
techLayers (  
  (<string>layerName  
   <int>layerNumber  
   [<string>layerAbbreviation])  
  ...  
);techLayers
```

Note:

Any line preceded by a semicolon “;” is treated as a comment.

Argument	Description
layerName	The name of the layer. Must be a string and the name should not conflict with any of the system-reserved layer names.
layerNumber	The number to be assigned to the layer. The valid values are 0 through 2³² (4294967296) where layer numbers from 195 through 254 are for system-reserved layers. Note: The numbers 195 to 199 are unused and unassigned system-reserved layers.
layerAbbreviation (Optional)	The abbreviation to be used for the layer. This value should be any string of seven characters or less.

Note:

Some of the applications that display the layer names do not always have room to display the entire name, so the abbreviation field gives you control over what is displayed in such cases.

The application that displays the layer name follows the following precedence order when determining how much of the name to display:

- The complete layer name
- The truncated layer name (if the abbreviation is not specified)
- The layer abbreviation
- The truncated abbreviation

Example 15 Sample techLayers section

```
techLayers (  
  ; ( LayerName           Layer#       Abbreviation )  
  ; ( -----           -)           -)           )  
  ;User-Defined Layers:  
  ( nwell                 1           nwell         )  
  ( diff                  2           diff          )
```

```
( poly                3          poly          )
...
;System-Reserved Layers:
( Unrouted           200        Unroute        )
( Row                201        Row            )
( Group              202        Group          )
...
);techLayers
```

For an example technology file, see [Sample Technology File](#).

techLayerPurposePriorities

The `techLayerPurposePriorities` section groups each layer name with a purpose, and lists the LPPs in order of priority. The order of priority determines the order in which they appear in Custom Compiler Layout Editor (LE) views and in the Object Layer Panel (OLP). LPPs at the beginning of the list have the lowest priority, and appear underneath LPPs at the end of the list.

Syntax:

```
techLayerPurposePriorities (
  <string>layerName
  <string>layerPurpose
  ...
);techLayerPurposePriorities
```

Note:

Any line preceded by a semicolon “;” is treated as a comment.

Argument	Description
layerName	Name of the layer.
layerPurpose	Name of the layer purpose.

Example 16 *Sample techLayerPurposePriorities section*

```
techLayerPurposePriorities (
  ;layers are ordered from lowest to highest priority
  ; ( LayerName          Purpose          )
  ; ( -----          -)
  ( background          drawing          )
  ( track               boundary         )
  ( grid               drawing          )
  ( grid               drawing1         )
  ...
);techLayerPurposePriorities
```

For an example technology file, see [Sample Technology File](#).

techDisplays

The `techDisplays` section defines the display attributes of a layer. The attributes include the display packet used, selectability of the layer, visibility of the layer.

Syntax:

```
techDisplays (
  (<string>layerName
   <string>purposeName
   <string>packet
   <bool>vis
   <bool>sel
   <bool>con2ChgLy
   <bool>drgEnbl
   <bool>valid)
  [<bool>visibleOnly]
  [<bool>static])
);techDisplays
```

Note:

Any line preceded by a semicolon “;” is treated as a comment.

Argument	Description
<code>layerName</code>	Name of the layer.
<code>layerPurpose</code>	Name of the layer purpose.
<code>packet</code>	The display packet that needs to be used. This is defined in the display resource file. For more information see Display Resource File .
<code>vis</code>	Sets whether the layer-purpose pair (LPP) is visible (<code>t</code>) or not visible (<code>nil</code>) in the display device.
<code>sel</code>	Specifies that the objects drawn with the LPP are selectable (<code>t</code>) or not selectable (<code>nil</code>).
<code>con2ChgLy</code>	Specifies that the LPP contributes to changed layer (<code>t</code>) or does not contribute (<code>nil</code>) to the changed layer. The Custom Compiler tool uses this flag to track objects that have been added or edited.
<code>DrgEnbl</code>	Specifies that the shape created on the LPP can be dragged (<code>t</code>) or cannot be dragged (<code>nil</code>) in the Custom Compiler Layout Editor.
<code>valid</code>	Specifies that the LPP appears in the Object Layer Panel (<code>t</code>) or not (<code>nil</code>).
<code>visibleOnly</code>	Specifies that the LPP is visible (<code>t</code>) or not visible (<code>nil</code>) in the Object/Layer Panel. Default: false
<code>static</code>	Specifies that the LPP is static (<code>t</code>) or dynamic (<code>nil</code>) in the Object/Layer Panel. Default: nil

Example 17 Sample techDisplays section

```
techDisplays(  
  ;( LayerName Purpose Packet Vis Sel Con2ChgLy DrgEnbl Valid visibleOnly  
  static)  
  ;( -----  
  -----)  
  ( background drawing background t nil t nil nil )  
  ( grid drawing grid t nil t nil nil )  
  ( grid drawing1 grid1 t nil t nil nil )  
  ( track boundary track t t nil nil nil )  
  ( boundary placement boundary_plc t t t t nil t )  
  ...  
);techDisplays
```

For an example technology file, see [Sample Technology File](#).

techLayerProperties

The `techLayerProperties` section assigns user-defined properties for specific LPP objects. Some of these properties are used by the Custom Compiler tool when you turn on dynamic measurement turned on. For example, the Custom Compiler tool displays the `sheetResistance`, `areaCapacitance`, and `edgeCapacitance` of shapes that are being drawn when you define these properties for this layer in the technology file and have the dynamic measurement turned on.

Note:

To turn-on the dynamic measurement feature, from the Custom Compiler Layout Editor, choose **Options > General**, select **Layout > Cursor** in the navigation pane, and then select **Dynamic** in the Measurement area.

Syntax:

```
techLayerProperties(  
  (<string>propName <string>layerName |  
  (<string>layerName <string>purposeName) |  
  (<string>layerName1 <string>layerName2) |  
  ((<string>layerName1 <string>purposeName1)  
  (<string>layerName2 <string>purposeName2))  
  <float>propValue)  
  ...  
);techLayerProperties
```

Note:

Any line preceded by a semicolon “;” is treated as a comment.

Argument	Description
propName	Name of the layer.

Argument	Description
layerName (layerName purposeName) (layerName1 layerName2) ((layerName1 purposeName1) (layerName2 purposeName2))	Specify the layer for which you want to define the property.
propValue	The value to be used for the property defined.

Example 18 *Sample techLayerProperties section*

```
techLayerProperties (
  ; ( PropName          Layer1      [Layer2]      PropValue )
  ; ( -----          -----      -----      ----- )
  ( sheetResistance     nwell        550.0        )
  ( sheetResistance     poly         6.34         )
  ( sheetResistance     diff         5.525        )
  ...
) ;techLayerProperties
```

For an example technology file, see [Sample Technology File](#).

derivedLayerOpDefs

This section is used to define layers that are derived from one or two `oaPhysicalLayers` or other `oaDerivedLayers` objects, by performing some user-defined layer operations.

Each user-defined layer operation is stored as a separate string and can contain references to layers and/or parameter values.

You can reference a layer using the string `$layer<int>`, where `<int>` is an integer starting at 1. The integer number corresponds to the input (parent layers) of the derived layer. For example, `$layer2` references the second layer of the derived layer. The same is true for parameters in which an array of parameter values are defined as a parameter of the derived layer. Parameters are referenced using the string `$value<int>` in the same way as layers.

Syntax:

```
derivedLayerOpDefs (
  <string>layerOpName
  <int>operation
) ;techDerivedLayers
```

Note:

Any line preceded by a semicolon “;” is treated as a comment.

Argument	Description
<code>layerOpName</code>	Name of the user defined layer operation definition.
<code>operation</code>	Array of values to be applied to the operator expressions defined in the rule definition.

Note:

Some important points:

- For each value, the Custom Compiler tool neither interprets nor converts to/from DBU.
- Each string value must be enclosed in double quotes "".
- More than one `layerName` must be enclosed in parentheses ().

Example 19 *Sample derivedLayerOpDefs section*

```
derivedLayerOpDefs(  
  ;(layerOpName operation)  
  ;(-----)  
    (CHECK_LINEEND $layer1 $layer2 enclosure1 $value1 $value2 something  
      $value3 text $value4)  
    (GROW_EDGE $layer1 value by $value1 growth factor $value2)  
);derivedLayerOpDefs
```

For an example technology file, see [Sample Technology File](#).

techDerivedLayers

This section is used to define the layers that are derived from one or two `oaPhysicalLayers` or other `oaDerivedLayers` objects, by performing some operation on the parent layers.

Custom Compiler performs the following checks when processing a derived layer definition. It checks:

- Whether any parent layer or layers exists.
- Whether any built-in operator(s) are defined or any user-defined operator(s) are defined.
- Whether any built-in parameters (if mentioned) or any `userDefinedLayerOp` and `userDefinedLayerOpValues` parameters are defined.

Syntax:

```
techDerivedLayers(  
  <string>derivedLayerName
```

```
<int>layerNumber
  (<string>layerOP
   <string>parentLayer1
   [<string>parentLayer2]
   [<string>derivedLayerParams])
);techDerivedLayers
```

Note:
Any line preceded by a semicolon “;” is treated as a comment.

Argument	Description
derivedLayerName	The name of the derived layer.
layerNumber	The number that you want to assign for the derived layer.
layerOP	The boolean expression to be used to show how the layer is derived from other primary / derived layers. The valid Boolean expressions are: • AND / andOR / or • NOT / not • XOR / xor • TOUCHING / touching • BUTTONLY / buttOnly • inside outside • overlapping • straddling • avoiding • butting • coincident coincidentOnly • enclosing • buttingOrCoincident buttingOrOverlapping • area • grow shrink • growVertical growHorizontal • shrinkVertical shrinkHorizontal • select • color • userDefined The color operator is supported when the oaDerivedLayerDef name is cdnColorSpec and numLayers = 1.
parentLayer1	An oaPhysicalLayer or other oaDerivedLayer.
parentLayer2 (Optional)	An oaPhysicalLayer or other oaDerivedLayer.

Argument	Description
derivedLayerParams (Optional)	The parameters that can be applied for deriving the layer: • -connectivityType sameNet differentNet ignore • -exclusive true false • -range [(lowerBound upperBound)] • -selectShapesInRange true false • -selectShapesWithPurpose purpose_name • -areaRange [(lowerBound upperBound)] • -distance float_value • -distance float_value • -color string_value int_value • -colorLocked true false • [-userDefinedLayerOpValues layerOpValues <valueArray>] The color name must be defined in the Controls Section.

Example 20 Sample techDerivedLayers section

```
techDerivedLayers(
  ( gate 10000 ( AND poly diff ) )
);techLayerProperties
```

In this example, `gate` is a derived layer that is assigned layer number 10000, it is derived by using `AND`, `poly` and the `diff` layers.

Example 21 Sample techDerivedLayers section with derivedLayerOpDefs

```
derivedLayerOpDefs(
  ;(layerOpName operation)
  ;(-----)
  (CHECK_LINEEND $layer1 $layer2 enclosure1 $value1 $value2 something
   $value3 text $value4)
  (GROW_EDGE $layer1 value by $value1 growth factor $value2)
  (WITH_LENGTH (
    ( length -in $layer1 -value {$value1} -out local_layer )
    ( withEdge -in {$layer1 local_layer} )
  )
)
);derivedLayerOpDefs

techDerivedLayers(
  ;( DerivedLayerName # composition )
  ;( ----- )
  (myFirstDerivedLayer 102 (userDefined m1 m2 -userDefinedLayerOp
    CHECK_LINEEND -userDefinedLayerOpValues (0.5 0.7 2 "Here is value"))
  (mySecondDerivedLayer 105 (userDefined m4 -userDefinedLayerOp GROW_EDGE
    -userDefinedLayerOpValues (0.2 0.5)))
  (m1Color1DerivedLayer 108 (color m1 -color red -colorLocked true))
  ( FINAL LAYER 11 ( WITH_LENGTH M1 -userDefineLayerOpValues (0.5) ) )
);techDerivedLayers
```

In this example, in a third-party technology library, a NULL layer1 is defined for a derived layer, then “NULL” is output to represent layer1 of the derived layer. In importing, if layer1 is specified with NULL in a derived layer, the derived layer is not created as OA does not allow to create a derived layer with NULL layer1 object. A warning message is displayed.

For an example technology file, see [Sample Technology File](#).

Layer Rules Section

The `layerRules` section defines custom rules that apply for the layers. These include properties such as manufacturing grid resolutions for specific layers, defining equivalent layers in the technology, or defining the preferred routing directions for specific layer.

Syntax:

```
layerRules (
  functions
  mfgResolutions
  routingDirections
  currentDensities
  cutClasses
  trackPatternDefs
  colorClasses
  trackDefs
  wideAlignments
  streamLayerMapping
  pinTextLayers
  voltageTextLayers
); layerRules
```

Note:

Any line preceded by a semicolon “;” is treated as a comment.

The following table defines the sections inside the `layerRules` section, in the order in which they appear in the technology file.

Subsection	Purpose
functions	Defines layer functions and their corresponding mask numbers.
mfgResolutions	Defines the manufacturing resolution for specific layers.
routingDirections	Defines a preferred routing direction for a given layer.
currentDensities	Defines a current density for a given layer.
cutClasses	Defines a group of cut classes for a specific cut layer.
trackPatternDefs	Defines a repeating pattern for individual wiring tracks.
colorClasses	Defines a constraintGroup called <code>colorClasses</code> that has the color models.
wideAlignments	Defines a constraintGroup called <code>wideAlignments</code> and adds it as <code>constraintGroupMem</code> into <code>oaTech</code> foundry constraint group.

Subsection	Purpose
trackDefs	Defines track regions, track groups that belong in the region, and individual tracks in a track group.
pinTextLayers	Specifies layer mapping information.

functions

The `functions` section defines connectivity functions for certain layers and optionally assigns mask numbers to these layers. For example, if you create a layer called `m1` that must be interpreted by the Custom Compiler tool as a metal layer, you must assign it a function of `metal`.

Syntax:

```
functions (
  (layerName function [maskNumber [numberOfColorMasks defaultColor]])
  [-fullColor [-recognitionPurposes (purposeName ...)]]
  [-PGRecognition (PGLayerName PGPurposeName)]
) ; functions
```

Note:

Any line preceded by a semicolon “;” is treated as a comment.

Argument	Description
<code>layerName</code>	Is the layer or the LPP.
<code>function</code>	The function of the layer or the LPP. The valid values for function are defined in Table 1 below.
<code>maskNumber</code>	The mask number for the layer. The valid value is any integer value. Note: If the <code>maskNumber</code> is not defined but the <code>numberOfColorMasks</code> is defined, then the <code>maskNumber</code> is exported as “nil” when exporting the technology data to an ASCII file.
<code>numberOfColorMasks</code>	The number of color masks for the physical layer. The valid value can only be 1, 2, 3, or 4.
<code>defaultColor</code>	The default color for the physical layer. Note: The <code>defaultColor</code> must be listed in the colors section. This option is valid only when <code>numberOfColorMasks</code> is greater than 1.

Argument	Description
-PGRecognition	Defines the PG marker layer mapping in technology file functions section for each layer. The tool stores this LPP as two integer type layer properties.

Note:

The mask number defines the layer's order in the layer stack, and is optional in the technology file. If it is not present, then the Custom Compiler tool uses the layer number, from the layerDefinition, instead of the mask number.

The following table shows how the Custom Compiler tool interprets the layer functions for connectivity. Each layer can be considered propagating or non-propagating, and can belong either to the resistive layer list in the `highlyResistiveLayers` constraint group or the routable layer list in the `validRoutingLayers` constraint group.

Table 1 *How the Connectivity Engine Interprets Layer Functions*

Layer function name	Propagating or non-propagating	Resistive/Routing layer list
nWell	Non-propagating	highlyResistiveLayers
pWell	Non-propagating	highlyResistiveLayers
nDiff	Non-propagating	highlyResistiveLayers
pDiff	Non-propagating	highlyResistiveLayers
nImplant	Non-propagating	highlyResistiveLayers
pImplant	Non-propagating	highlyResistiveLayers
poly	Propagating	validRoutingLayers
cut	Allows propagating between adjacent layers	validRoutingLayers
metal	Propagating	validRoutingLayers
contactlessMetal	Propagating	validRoutingLayers
diffusion	Non-propagating	highlyResistiveLayers
recognition	Ignored	Ignored
other	Ignored	Ignored
mimCut	Ignored	Ignored
mimMetal	Ignored	Ignored

Example 22 Sample functions section

```
functions(  
; ( layer      function      [maskNumber])  
; ( -----  
  ( psub      "pWell"        1      )  
  ( nwell     "nWell"        2      )  
  ( diff      "diffusion"    3      )  
  ( m1        "metal"        8      )  
  ( m0poly    "contactlessMetal 9      )  
  ...  
);functions
```

Example 23 Sample functions section with -PGRecognition

```
functions(  
; ( layer      function      [maskNumber [numberOfColorMasks [defaultColor]]  
  [-PGRecognition (PGLayerName PGPurposeName)])  
; ( -----  
  ( M1 metal   100    2    -PGRecognition (PG_markter info))  
  ...  
);functions
```

For an example technology file, see [Sample Technology File](#).

mfgResolutions

The `mfgResolutions` section defines the manufacturing resolution for specific layers, in microns. This is the smallest resolution for that layer and shapes drawn on that layer are snapped to the `mfgResolution` for that layer.

Syntax:

```
mfgResolutions(  
  (layer value)  
  ...  
);mfgResolutions
```

Note:

Any line preceded by a semicolon “;” is treated as a comment.

Argument	Description
layer	The layer name or the layer number.
value	The manufacturing grid resolution that should be applied for the layer. This value must be greater than or equal to the default manufacturing grid value.

Example 24 Sample mfgResolutions section

```
mfgResolutions (  
; ( layer      mfgResolution )  
; ( -----  
  )
```

```
( dnw          0.005      )  
( nwell        0.005      )  
( diff         0.005      )  
...  
);mfgResolutions
```

In the preceding example, layers have been set to a resolution of 0.005 um (the unit for this technology file).

For an example technology file, see [Sample Technology File](#).

routingDirections

The `routingDirections` section defines preferred routing directions for specific layers. This information is be used by the Custom Compiler routing tools.

Syntax:

```
routingDirections (  
  (layer    direction)  
  ...  
);routingDirections
```

Note:

Any line preceded by a semicolon “;” is treated as a comment.

Argument	Description
layer	The layer name or the layer number.
direction	The preferred routing direction for the layer. The valid values are 'horizontal', 'vertical', 'leftDiag', 'rightDiag', 'none'

Example 25 *Sample routingDirections section*

```
routingDirections (  
  ; ( layer    direction  )  
  ; ( -----  - - - - - )  
  ( m1        horizontal )  
  ( m2        vertical   )  
  ( m3        horizontal )  
  ( m4        vertical   )  
);routingDirections
```

In the preceding example, the metal layer `m1` has a preferred routing direction of horizontal.

currentDensities

The `currentDensity` section defines the current density for a specified layers.

Syntax:

```
currentDensities (
  (<string>currentDensityName
   <string|LPP>layerName
   <current|frequency_1DTable|
    width_1DTable|frequency_width_2DTable>value
   [-iType <string>interValue]
   [-eLowerType <string>exterValue]
   [-eUpperType <string>exterValue]
   [-iColType <string>interValue]
   [-iRowType <string>interValue]
   [-eRowLowerType <string>exterValue]
   [-eRowUpperType <string>exterValue]
   [-eColLowerType <string>exterValue]
   [-eColUpperType <string>exterValue]
   [-isHard true|false]
   [-comment [<string rule_id>] <string rule_desc>]
);currentDensities
```

Note:

Any line preceded by a semicolon “;” is treated as a comment.

Argument	Description
currentDensityName	Specify the current density of the layer. The following types of current density can be specified for a layer: • averageACCurrentDensity: Specifies the average AC current in mA/um. • averageDCCurrentDensity: Specifies the average DC current in mA/um. • peakACCurrentDensity: Specifies the peak AC current in mA/um. • rmsACCurrentDensity: Specifies the average RMS current in mA/um.
layerName	layerName can be defined as a list of strings or a LPPs.
value	Specify the current value depending upon the selected currentDensityName. The following value types can be specified: • <float>current • frequency_1DTable: (<float>frequency <float>current) • width_1DTable: (<float> width <float> current) • frequency_width_2DTable: ((<float> frequency <float>width) <float>current)
-iType	Specifies the interpolation type. The valid values are: • SnapDown (Default) • SnapUp • Linear • InclusiveSnapDown • InclusiveSnapUp

Argument	Description
-eLowerType	Specifies the lower extrapolation type. The valid values are: • SnapDown • SnapUp (Default) • Linear
-eUpperType	Specifies the upper extrapolation type. The valid values are: • SnapDown (Default) • SnapUp • Linear
-iColType	Specifies the column interpolation type. The valid values are: • SnapDown (Default) • SnapUp • Linear • InclusiveSnapDown • InclusiveSnapUp
-iRowType	Specifies the row interpolation type. The valid values are: • SnapDown (Default) • SnapUp • Linear • InclusiveSnapDown • InclusiveSnapUp
-eRowLowerType	Specifies the row lower extrapolation type. The valid values are: • SnapDown • SnapUp (Default) • Linear
-eRowUpperType	Specifies the row upper extrapolation type. The valid values are: • SnapDown (Default) • SnapUp • Linear
-eColLowerType	Specifies the column lower extrapolation type. The valid values are: • SnapDown • SnapUp (Default) • Linear
-eColUpperType	Specifies the column upper extrapolation type. The valid values are: • SnapDown (Default) • SnapUp • Linear

Example 26 Sample currentDensities section

```
currentDensities(
    (averageACCurrentDensity m1 25.5)
    (averageDCCurrentDensity m1 ((0.2 5.4)
                                   (0.4 12.6)
                                   (0.8 25.6)))
)
```

```
(peakACCurrentDensity      m1 (((1.0 0.2) 9.36775)
                               ((1.0 0.4) 19.55009)
                               ((1.0 0.8) 39.91476)
                               ((1.0 1.6) 80.64411)
                               ((1.0 5.0) 253.7439)))
(rmsACCurrentDensity       m1 (((1.0 0.2) 1.207059)
                               ((1.0 0.4) 2.155149)
                               ((1.0 0.8) 3.930906)
                               ((1.0 1.6) 7.394722)
                               ((1.0 5.0) 21.98146))
                               -eRowLowerType SnapDown
                               -eColLowerType SnapDown)

);currentDensities
```

cutClasses

The `cutClasses` section defines a group of cut classes for a specific cut layer. All of these constraints are put into a constraint group named as `cutClass_LibName`. Add this constraint group as a constraint group member into `oaTech` foundry constraint group.

If local mode in tech exporter is off, the `cutClass_LibName` constraint groups in reference technology databases are also output into `layerRules/cutClasses` in the order from the top to the bottom.

Syntax:

```
cutClasses (
; (layerName      ; (className [-numCuts numCuts] [-minWidth]
                    [-minLength] (width length)
                    [-comment [<string rule_id>] <string rule_desc>] ));
cutClasses
```

Note:

Any line preceded by a semicolon “;” is treated as a comment.

Argument	Description
layerName	Specify the layer name.
className	Specifies the name of the cutClass.
numCuts	Specifies the number of equivalent cuts or numCuts for a given cut class.
minWidth	If set to false, width of the via has to be exactly the same as width value; otherwise, width value is the minimum allowed width of the via. Default is false.
minLength	If set to false, length of the via has to be exactly the same as length value; otherwise, length value is the minimum allowed length of the via. Default is false.

Argument	Description
width	The width of via.
length	The length of via.

Example 27 Sample cutClasses section

```
cutClasses (
  (V0
    (V0 -numCuts 1 (0.032 0.032) -comment "rule1")
    (V0BAR -numCuts 2 (0.032 0.064) -comment "rule2")
  )
  (AZ
    (AZ -numCuts 1 -minWidth -minLength (0.032 0.032))
    (AZBAR -numCuts 2 -minWidth -minLength (0.032 0.064))
    (AZLRG -numCuts 4 -minWidth -minLength (0.064 0.064))
  )
); cutClasses
```

For an example technology file, see [Sample Technology File](#).

trackPatternDefs

The `trackPatternDefs` section defines a repeating pattern for individual wiring tracks. Tracks have a layer, type, color, orientation, and per track spacings. This is called a block. If the number of tracks is not specified, the tracks span the entire design and there is only one block. If the number of tracks is specified, there is a per block spacing defining the repeating pattern for blocks of tracks. Each block defines the tracks repeated at a spacing interval. The blocks can be placed at a different spacing interval.

Syntax:

```
trackPatternDefs (
  ;trackPatternDef(name layer|(layer purpose)
  type orientation trackPitch[
    width[(numTracks blockPitch) [
      offset[visible[
        active[color[
          autoGenerateTrack]]]]]]]]);trackPatternDefs
```

Note:

Any line preceded by a semicolon “;” is treated as a comment.

Argument	Description
name	Specifies a unique name for the track pattern definition.
layer	Specifies the layer name.

Argument	Description
purpose	Specifies the purpose name of oaPurpose or “oaAny” for all purposes.
type	Specifies the pattern type: default, reserved, nonReserved.
orientation	Specifies the orientation of pattern definition: horizontal or vertical.
trackPitch	Specifies the distance between tracks in a block in microns.
Width	Specifies the width of pattern in microns, or zero if undefined. Default is 0.
numTracks	Specifies the number of tracks in each block. Zero means infinite. Default is 0.
blockPitch	Specifies the distance between each block of tracks in microns. Zero means there are no blocks and the number of tracks per block is repeated uniformly once. Default is 0.
Offset	Specifies the offset of the first track of the pattern definition in microns. For horizontal definitions, offset is along the X axis and for vertical, the Y axis. Tracks extend to infinity in both directions from this initial offset. Default is 0.
Visible	Specifies if the patterns created from this definition default to being visible. Default is true.
Active	Specifies if the patterns created from this definition default to being active. Default is false.
Color	If specified it must be one of the valid colors for the layer.
autoGenerateTracks	Specifies if the tracks should be auto-generated. Default is <i>true</i> .

Example 28 Sample trackPatternDefs section

```
trackPatternDefs(
;define track pattern with width , offset as nil
trackPatternDef(m1DefHor m1 default horizontal 0.05 nil (50 3.0) nil t t)

;define track pattern with numTrack blockPitch as nil
trackPatternDef(m1DefHor m1 nonReserved horizontal 0.05 0.25 nil 0.06 t t nil)

;define track pattern
trackPatternDef(m1ResHor m1 reserved horizontal 0.06 0.25 (25 2.0) 0.06 nil t t blue)
); trackPatternDefs
```

For an example technology file, see [Sample Technology File](#).

colorClasses

The `colorClasses` defines a `constraintGroup` called `colorClasses` that has the color models which is put into `oaTech` foundry constraint group. One specific color model is represented as `OA` layer constraint, `colorClass` constraint.

You can choose one of the following model to assign shapes on a color mask:

- **Recognition Layer Model:** The shapes on the actual mask layer needs to be copied to the recognition layer, if these shapes are assigned to a specific color mask.
- **Layer Model:** The specific layer is defined on which shapes would be assigned to the corresponding mask color.
- **Purpose Model:** The specific purpose is defined with the mask layer on which shapes would be assigned to the corresponding mask color. If the shapes are assigned on other purposes, the shapes are uncolored.

Syntax:

```
colorClasses (
    ;(layer [derived]
    ; (color colorLayer| (colorLayer colorPurpose))
);colorClasses
```

Note:

Any line preceded by a semicolon “;” is treated as a comment.

Argument	Description
layer	Specifies the layer name.
derived	Specifies if the color model is a recognition model or not. If true, it is recognition model which needs to copy shapes from layer to colorLayer. The default is false.
color	Specifies the color name or number. This value is a color mask identifier, zero meaning uncolored.
colorLayer	Specifies the color layer name. The value is a layer number to assign to the color mask specified by the constraint value.
colorPurpose	Specifies the color purpose name. The value is a purpose number to assign to the color mask specified by the constraint value.

Recognition Model Example:

```
layerRules(
    colorClasses(
        (M1 true
            (red (M1_recognition purpose1))
            (blue (M1_recognition purpose2))
        )
        (M2 true
            (red (M2_recognition purpose1))
            (blue (M2_recognition purpose2))
        )
    )
)
```

```
);colorClasses  
);layer Rules
```

Layer Model Example:

```
layerRules(  
  colorClasses(  
    (M1  
      (red M1Red)  
      (blue M1Blue)  
    )  
    (M2  
      (red M2Red)  
      (blue M2Blue)  
    )  
  );colorClasses  
);layerRules
```

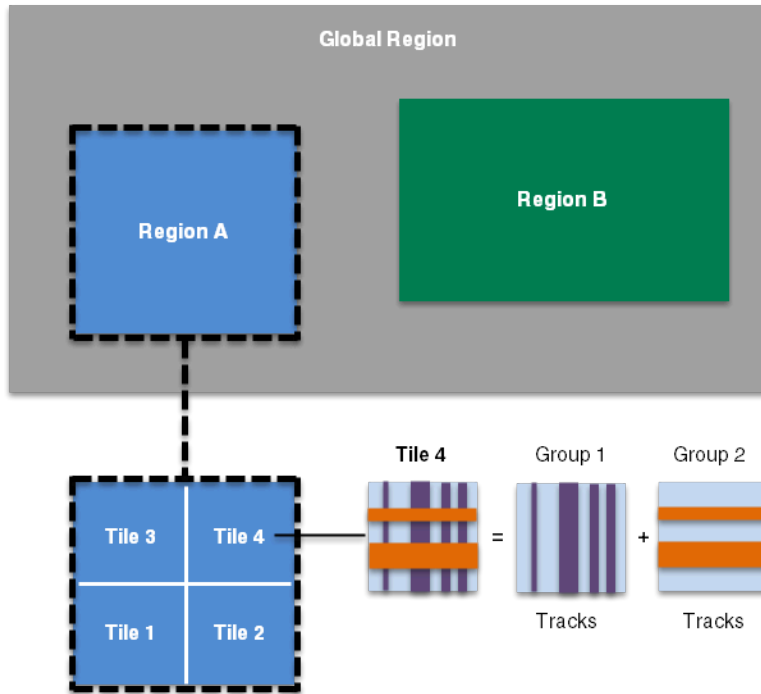
Purpose Model Example:

```
layerRules(  
  colorClasses(  
    (M1  
      (red (M1 purpose1))  
      (blue (M1 purpose2))  
    )  
    (M2  
      (red (M2 purpose1))  
      (blue (M2 purpose2))  
    )  
  );colorClasses  
);layerRules
```

For an example technology file, see [Sample Technology File](#).

trackDefs

Tracks in a design help ensure that routes and `snapTargets` in instances are properly placed and are correct by construction. Advanced node designs (such as those containing FinFETs) can have multiple track regions to accommodate different requirements within the same design. Each track region consists of one or more tiles. Each tile contains one or more groups of tracks.



Syntax:

```
trackDefs (
;   tracks()
;   groups()
;   regions()
;   grids()
;   boundaryTrackData()
;   boundaryTracks()
;   boundaryTrackRegion()
); trackDefs
```

Note:

Any line preceded by a semicolon (;) is treated as a comment.

You can use the `trackDefs` Layer Rule to define track regions. When designers create track regions in the Layout Editor, they can use one of the following methods:

Track Creation Method	Description
Track Region Assistant	The assistant provides commands that let designers create and manage track regions. Define <code>tracks()</code> , <code>groups()</code> , <code>regions()</code> , and <code>grids()</code> for this method.

Track Creation Method	Description
Direct track region creation (no assistant or special procedure required)	Designers can use the Layout Editor Create > Rectangle command to draw and edit track regions on layer-purpose pairs (LPPs) defined by the boundaryTrack layer rules. Define boundaryTrackData() , boundaryTracks() , and boundaryTrackRegion() for this method.

For a complete example of `trackDefs`, see [trackDefs Example on page 97](#).

grids()

You use `grids()` to specify the definition for an extension.

Syntax:

```
trackDefs (
  grids (
    (gridName layerName
      -pitch <float>
      [-width <float>]
      [-color <string>]
      [-active <bool>]
      [-visible <bool>]
      [-pointsLow (<float>...)]
      [-pointsHigh (<float>...)]
      [-pairs ((<float> <float>)...)]
    )
    ...
  )
)
```

Note the following points when you use the `grids()` functions:

- You do not have to use the `-pointsLow` and `-pointsHigh` option. When you use the `-pairs` option. The points specified with the `-pairs` option is also used for the `-pointsLow` and `-pointsHigh` options, as well.
- If you use the `-pairs`, the `-pointsLow` option is specified, and the `-pointsHigh` option is not specified, the `-pairs` fill the `-pointsHigh` option. When the `-pointsHigh` option is specified, and the `-pointsLow` option is not specified, the `-pairs` fill the `-pointsLow` option.
- Each pair contains one low point and one high point. These pairs must exist in the list specified with the `-pointsLow` and `-pointsHigh` options. Different pairs can have the same start or stop points.
- A smaller or equal value, and high value compared to a low value in the pair is the stop point from the next pitch.

Argument	Description
<code>gridName</code>	The unique name of the grid. You can provide a unique name or use the system-generated unique name.
<code>-layerName</code>	The layer name of the grid.
<code>-width</code>	The width of the grid.
<code>-color</code>	The color of the grid. To specify a color, make sure the color name is defined in the <code>Controls</code> section of the technology file.
<code>-visible</code>	The default visibility state of the track. Default: false
<code>-active</code>	The default active state of the track. Default: true
<code>-pointsLow</code>	List of starting points of the grid.
<code>-pointsHigh</code>	List of ending points of the grid.
<code>-pairs</code>	List of pair of starting and ending points of the grid.
<code>-pitch</code>	Pitch of the grid. All the points in <code>pointsLow</code> , <code>pointsHigh</code> , and <code>pairs</code> must not be higher than the pitch value.

Example 29

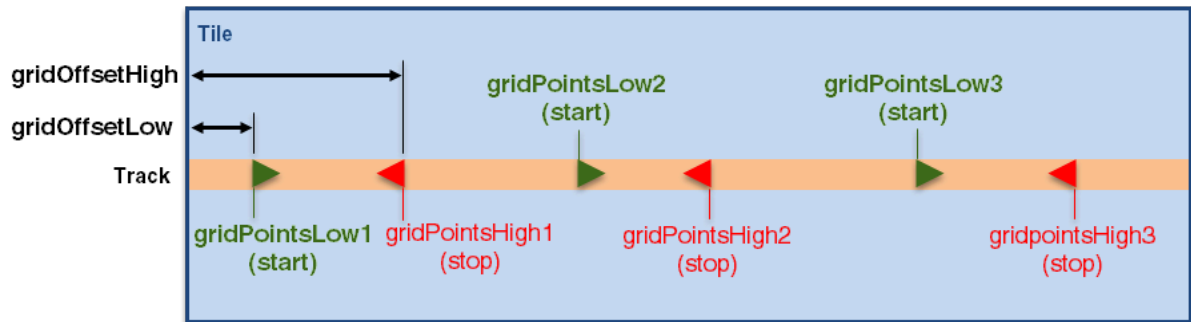
```
trackDefs (
  grids (
    (mygrid M1 -pointsLow (0.034 0.056 0.078)
      -pointsHigh (0.012 0.045 0.034)
      -visible true -active false -pairs ((0.034 0.012) (0.11
0.13))
      -color green -pitch 0.023)
    ); grids
  tracks (...)
  groups (...)
  regions (...)
); trackDefs
```

tracks()

You use `tracks()` to define individual tracks in a group.



You can use the `-grid*` arguments to specify the permissible start and stop points of interconnects and routes on the tracks. The offsets are relative to the left and bottom of the tile boundary.



Syntax:

```
tracks (
; (trackName -type <enum> -offset <float> [-width <float>]
;     [-color <string>] [-sigType <enum>] [-visible <bool>]
;     [-active <bool>] [-gridOffsetLow <float>]
;     [-gridOffsetHigh <float>]
;     [-gridPointsLow (<float> ...)]
;     [-gridPointsHigh (<float> ...)]
); tracks
```

Note:

Any line preceded by a semicolon (;) is treated as a comment.

Argument	Description
<code>trackName</code>	The unique name of the track. You can provide a unique name, or you can use the system-generated unique name.

Argument	Description
<code>-type</code>	The track type. Valid values are: • <code>default</code> • <code>nonReserved</code> • <code>reserved</code> <code>default</code> tracks have no width requirements, and wires of any width can route on them, with the wider wires consuming (or blocking) more available area. <code>nonReserved</code> tracks have a width parameter, and any wire placed on the tracks must comply with the width of the tracks. <code>reserved</code> tracks have a width parameter. When the preference <code>leReservedTrackWidthCheck</code> is set to <code>false</code>, the track operates like a default track, and any wire can route on the tracks. The wire's width does not have to match the track's width. When the preference is set to <code>true</code>, only wires with non-default-rule nets can route on the tracks, and the wires must match the width of the reserved tracks.
<code>-offset</code>	The distance between the bottom edge of the tile and the center of the track. The Layout Editor ignores offset values that are greater than the <code>groups() -pitch</code> value. For information about the <code>groups() -pitch</code> argument, see groups() on page 86.
<code>-width</code>	(Optional) The width of the track. Default: 0 Note: If <code>-type</code> is <code>reserved</code> or <code>nonReserved</code>, <code>-width</code> is required.
<code>-color</code>	(Optional) The color of the track. To specify a color, make sure the color name is defined in the <code>Controls</code> section of the technology file.
<code>-sigType</code>	(Optional) The signal type that the track will support. Valid values are: • <code>all</code> (default) • <code>power</code> • <code>ground</code> • <code>clock</code> • <code>signal</code> • <code>tieoff</code> • <code>tieHi</code> • <code>tieLo</code> • <code>analog</code> • <code>scan</code> • <code>reset</code>
<code>-visible</code>	(Optional) The default Visibility state of the track. Default: <code>false</code>
<code>-active</code>	(Optional) The default Active state of the track. Default: <code>true</code>
<code>-gridOffsetLow</code>	(Optional) Offset distance for grid starting points. Use this argument with <code>-gridPointsLow</code>. Default: 0.0
<code>-gridOffsetHigh</code>	(Optional) Offset distance for grid ending points. Use this argument with <code>-gridPointsHigh</code>. Default: 0.0

Argument	Description
-gridPointsLow	(Optional) List of starting points of the grid. You use -gridPointsLow and -gridPointsHigh to control where interconnects start and stop when routing on tracks. The -gridOffsetLow values are cycled (continues to the first value when the last value is reached). For example, if -gridOffsetLow is 1.5, -gridPointsLow are {1 2}, and the tile width is 10, then the track grid starting points are {1.5 2.5 4.5 5.5 7.5 8.5}. Default: { }
-gridPointsHigh	(Optional) List of ending points of the grid. You use -gridPointsLow and -gridPointsHigh to control where interconnects start and stop when routing on tracks. The -gridOffsetHigh values are cycled (continues to the first value when the last value is reached). For example, if -gridOffsetHigh is 3.5, -gridPointsHigh are {3 4}, and the tile width is 20, then the track grid stopping points are {6.5 10.5 13.5 17.5}. Default: { }

Example 30

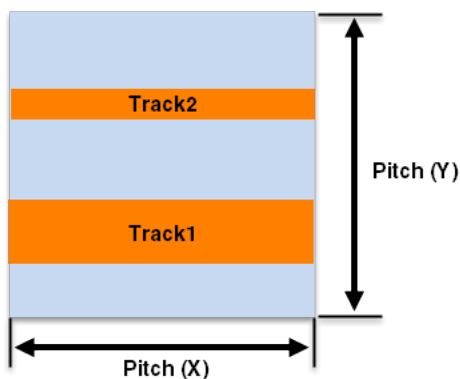
```

layerRules(
  trackDefs(
    tracks(
      ( myTrack1 -type reserved -offset 0.05 -width 0.02 -color red -visible
true -active false)
      ( myTrack2 -type reserved -offset 0.03 -color green -active false )
      ( myTrack3 -type nonReserved -offset 0.04 -width 0.1 -sigType power)
    ); tracks
  ); trackDefs
); layerRules

```

groups()

You use `groups()` to define a set of tracks that are grouped together.



Syntax:

```
layerRules
  trackDefs (
    tracks (...)
    groups (
      ; (name layer (trackName...))
      ; -direction <enum> -pitch (<float> <float>)
      ; [-allowCrossBoundary <bool>]
      ; [-renderPurpose <string>]
      ; [-LEGShapeSnapPurposes (purposeName...)]
    )
    ; groups
  ) ; tracks
); layerRules
```

Note:

Any line preceded by a semicolon (;) is treated as a comment.

Argument	Description
name	The unique name of the track group.
layer	The layer that is assigned to all the tracks in the group.
trackName	The list of tracks that belong in the group.
direction	The direction of the tracks in the group. Valid values are: - horizontal - vertical
pitch	The size of the group, specified by x (horizontal) and y (vertical) values. You can specify 0 for X if the track direction is horizontal. You can specify 0 for Y if the track direction is vertical.
allowCross Boundary	Optional. Determines if interconnect on tracks from group can cross region boundary. Default is true.
renderPurpose	Optional. Purpose used to render tracks. Default is "" in which case 'drawing' purpose is used.
LEGShape SnapPurposes	Optional. Only shapes with the purposes defined can snap to track LEGs. Default is "" which means all purposes are allowed.

Example 31

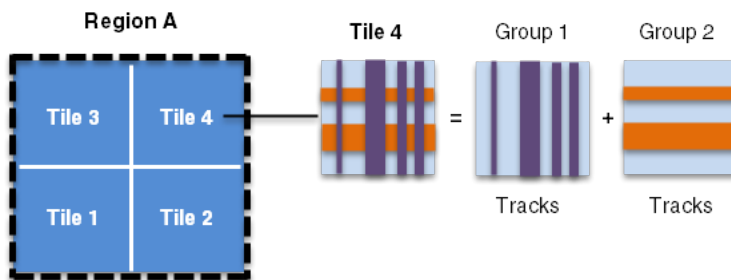
```
layerRules (
  trackDefs (
    groups (
      (myGroup1 m1 (myTrack1 myTrack2) -direction horizontal -pitch (0.02
0.06))
      (myGroup2 m2 (myTrack1 myTrack3) -direction vertical -pitch (0.25
0.15))
    )
  )
)
```

```
(myGroup3 m3 (myTrack1 myTrack2 ) -direction horizontal -pitch (0.02
0.06) -renderPurpose xlabel)
(myGroup4 m4 (myTrack1 myTrack3 ) -direction vertical -pitch (0.35
0.15) -LEGShapeSnapPurposes (drawing label pin))
); groups
); trackDefs
); layerRules
```

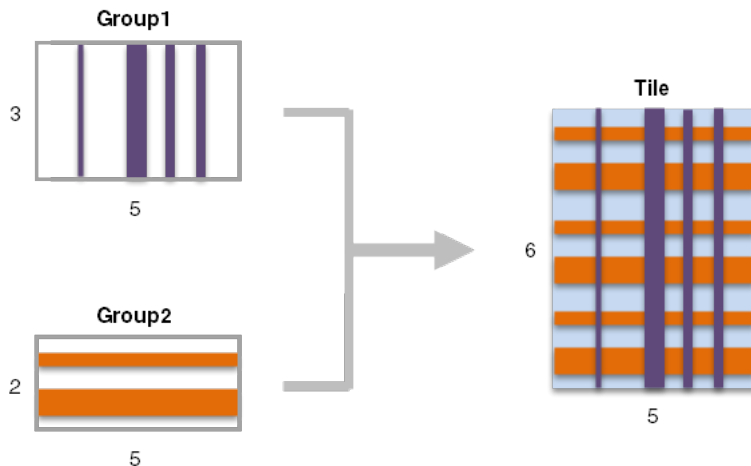
regions()

You use `regions()` to create a track region definition. This definition lets designers create and edit track regions using the Track Region Assistant.

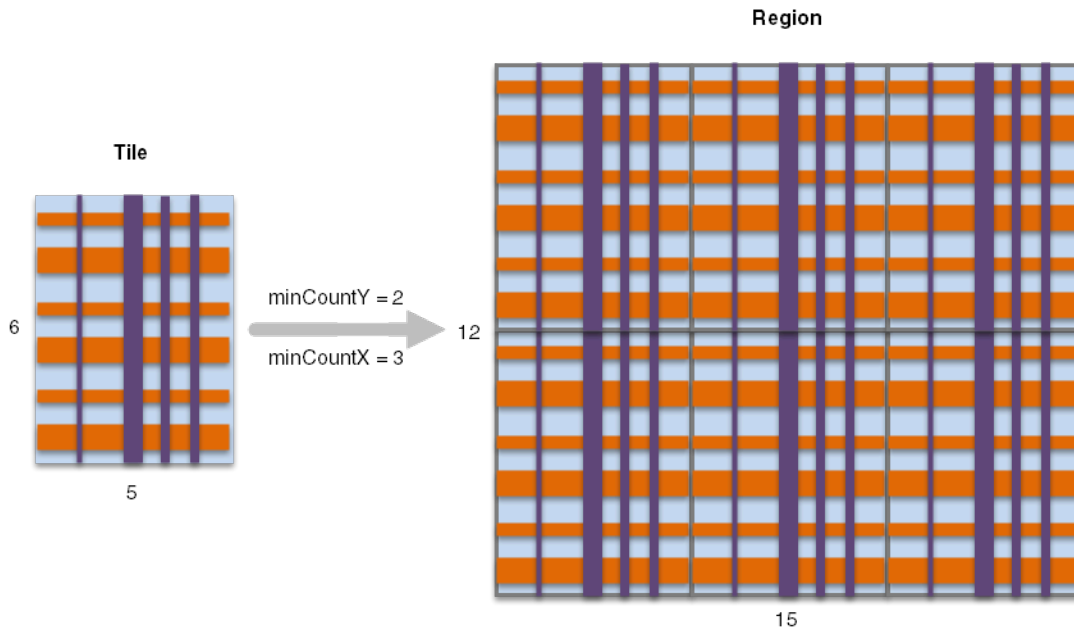
Within the track region definition, you can define the track groups that make up the tiles in the region:



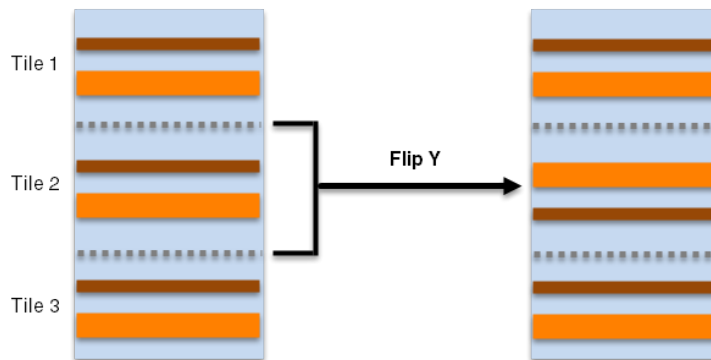
When track groups of different X and Y pitches form a tile, the tile dimensions are based on the least common multiple of the track group pitches. In the following example, track group 1 is 5 x 3, and track group 2 is 5 x 2. When the two groups form a tile, the tile is 5 x 6.



You can specify the minimum number of tiles that are in a region. For example, for a tile that is 5 x 6, if you specify that the minimum number of tiles in the X direction is 3 and the minimum number of tiles in the Y direction is 2, then the minimum region size is 15 x 12.



Within a region, you can specify that alternating tiles flip along the X or Y direction. In the following example, the same tile is repeated 3 times in the vertical direction. Also, the second tile is flipped vertically (in the Y direction).



By default, track regions automatically snap to grid as you create or modify the regions. However, you can use the `-snapGrid` and `-snapOffset` arguments to override the default grid.

Syntax:

```
regions (
    ;(name (groupName ...))
    ; [-flip (<bool> <bool>) | -flipGroups (groupName ...)]
    [-flipGroupsFirst (groupName ...)] [-groupFlipTarget tile | region]]
    ; [-minCount (<int> <int>)]
    ; [-maxCount (<int> <int>)]
```

```
; [-autoCreateGlobalRegion <bool>]
; [-foundryConstraintGroups <string>]
; [-snapGrid (<float> <float>)]
; [-snapOffset (<float> <float>)]
; [-defaultTracksVisible default | false | true]
; [-defaultTracksActive default | false | true]
; [-groupEnclosure (<float> <float>)]
; [-enclosedGroups (groupName ...)]
; [-flipTilesX <bool> [-flipTilesXFirst <bool>]] ; [-flipTilesY
<bool> [-flipTilesYFirst <bool>]]
; [-gridGroups ((groupName... (gridName))...)]
); regions
```

Note:

Any line preceded by a semicolon (;) is treated as a comment.

Argument	Description
name	The unique name of the track region.
groupName	The list of track groups that form tiles in the region.
-flip	Flips alternating tiles in the X and Y directions. Default: (false false)
-flipGroups	The list of track groups that will be flipped in the tile. Use this argument to flip only the specified groups.
-flipGroupsFirst	Specifies the leTrackGroupDef collection in the region definition that should flip first within the region. The flipGroupsFirst should be a subset of -flipGroups or match the setting of -flip. For example, when -flip is set to (true false), the -flipGroupsFirst should be a subset of all the vertical groups.
-groupFlipTarget	Specifies if group flipping is bound by region tile or the region. The enum values: tile = 0 and region = 1. When groupFlipTarget is set to region, flipTilesX, flipTilesY, flipTilesXFirst, and flipTilesYFirst cannot be set to true.
-minCount (X Y)	The minimum number of tiles in the X and Y directions. Default: (1, 1)
-maxCount (X Y)	The maximum number of tiles in the X and Y directions. Default: (-1, -1), where -1 represents infinity
-autoCreateGlobalRegion	Creates a global region using this regions definition when the design is first opened. Default: false If multiple track regions specify autoGenerateGlobalRegion = true, only the first track region definition is used to create a global region.

Argument	Description
<code>foundryConstraintGroups</code>	(Optional) Specifies the definitions associated with one or more Alternate Foundry Constraint Group (AFCG). The valid values of the attribute are <code>foundry</code> , <code>foundryAlias</code> , or an AFCG name (Only constraint groups with <code>-alternate true</code> are considered as AFCG).
<code>-snapGrid (X Y)</code>	Overrides the default snapping grid. The valid value for either X or Y is: • 0 uses the automatically computed X or Y tile dimension value. • -1 uses the <code>leSnapSpacing</code> X or Y value. • An integer greater than 0 overrides the computed X or Y dimension value. Default: (0 0) By default, snapping grids are based on the automatically computed tile size. However, you can override the default grid by specifying -1 or an integer greater than 0 for (X Y). For example, <code>-snapGrid (-1 -1)</code> uses the <code>leSnapSpacing</code> X and Y values, and <code>-snapGrid (5.8 10.7)</code> uses (5.8 10.7) instead of the computed X and Y values.
<code>-snapOffset (X Y)</code>	Specifies the offset distance from the starting snapping location.
<code>-defaultTracksVisible</code>	Specifies whether the tracks are visible: • default: Uses the track's visibility value. • false: Turns off visibility for all tracks. • true: Turns on visibility for all tracks.
<code>-defaultTracksActive</code>	Specifies whether the tracks are active: • default: Uses the track's active state value. • false: Turns off the active state of all tracks. • true: Turns on active state of all tracks.
<code>-groupEnclosure</code>	Specifies enclosure values for tracks found in <code>enclosedGroup</code> .
<code>-enclosedGroups</code>	Specifies the <code>leTrackGroupDef</code> collection abided by <code>groupEnclosure</code> value.
<code>-flipTilesX</code>	Specifies default value for <code>flipX</code> when creating <code>leTrackRegion</code> or <code>leGlobalTrackRegion</code> .
<code>-flipTilesXFirst</code>	Specifies default value for <code>flipXFirst</code> when creating <code>leTrackRegion</code> or <code>leGlobalTrackRegion</code> .
<code>-flipTilesY</code>	Specifies default value for <code>flipY</code> when creating <code>leTrackRegion</code> or <code>leGlobalTrackRegion</code> .
<code>-flipTilesYFirst</code>	Specifies default value for <code>flipYFirst</code> when creating <code>leTrackRegion</code> or <code>leGlobalTrackRegion</code> .
<code>-gridGroups</code>	Associates track grids with track groups in track region definitions. Specified groups layer and grids layer must be the same.

Example 32

```
layerRules(
  trackDefs(
    tracks(...)
    groups(...)
    regions(
      (myRegion1 (myGroup1 myGroup2) -flip (true false) -minCount (2 3)
      -maxCount (3 4))
      (myRegion2 (myGroup1) -maxCount (4 5) -autoCreateGlobalRegion true
      -snapGrid(0.05 0.07) -snapOffset(0.15 0.75))
      (myRegion3 (myGroup2) -defaultTracksVisible true -defaultTracksActive
      false -groupEnclosure (0.03 0.02) -enclosedGroups (myGroup2) )
      (myRegion4 (myGroup1 myGroup2) flipGroups (myGroup2) minCount (2 6)
      maxCount (2 4)
      (myRegion5 (myGroup1 myGroup2) -minCount (2 6) -autoCreateGlobalRegion
      true -flipTilesX true)
      (myRegion6 (myGroup1) -maxCount (2 4)
      -autoCreateGlobalRegion true -flipTilesY true -flipTilesYFirst true)
      (myRegion7 (myGroup1 myGroup2) -flipGroups (myGroup1 myGroup2) -flipGroupsFirst
      (myGroup1) -groupFlipTarget region)
      (myRegion8 (myGroup1 myGroup2)
      -flip (true false) -flipGroupsFirst (myGroup2) -groupFlipTarget tile -gridGroups
      ((myGroup1 myGroup2 (myGrid))))
    ); regions
  ); trackDefs
);layerRules
```

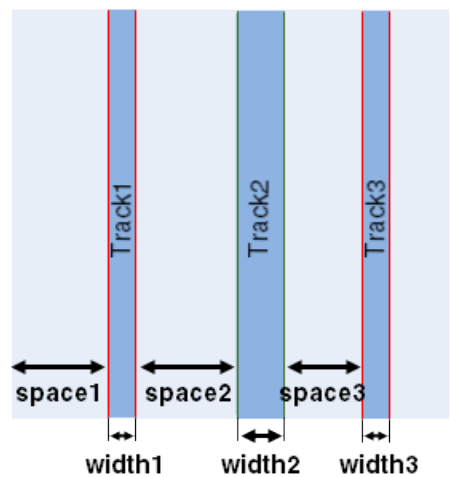
boundaryTrackData()

You use `boundaryTrackData()` to describe a track in [boundaryTrackRegion\(\)](#)s. For each track, you can specify the following:

- Width
- Type (default, reserved, non-reserved)
- Color
- Spacing to the left of vertical tracks or below horizontal tracks

The following diagram shows three tracks: track1, track2, and track3. Each track has its own color, width, spacing, and type (not shown).

Boundary Track Data



Syntax:

```
boundaryTrackData (  
    ; (<string>name  
    ;   -width <float>  
    ;   -space <float>  
    ;   -type <default | reserved | nonReserved>  
    ;   [-color <string>])  
); boundaryTrackData
```

Note:

Any line preceded by a semicolon (;) is treated as a comment.

Argument	Description
name	Unique name of the boundary track data in the OA database.
-width	Track width.
-space	Edge-to-edge spacing of tracks. The spacing is always to the left of vertical tracks or below horizontal tracks.

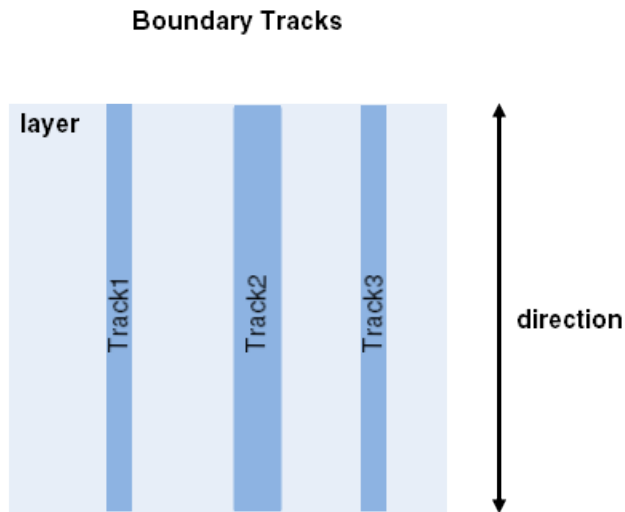
Argument	Description
-type	Track type: • default • nonReserved • reserved <code>default</code> tracks have no width requirements, and wires of any width can route on them, with the wider wires consuming (or blocking) more available area. <code>nonReserved</code> tracks have a width parameter, and any wire placed on the tracks must comply with the width of the tracks. <code>reserved</code> tracks have a width parameter. When the preference <code>leReservedTrackWidthCheck</code> is set to <code>false</code>, the track operates like a default track, and any wire can route on the tracks. The wire's width does not have to match the track's width. When the preference is set to <code>true</code>, only wires with non-default-rule nets can route on the tracks, and the wires must match the width of the reserved tracks.
-color	Track color. To specify a color, make sure the color name is defined in the Controls Section of the technology file. If you do not specify a color, the default is uncolored, regardless of whether the layer is colorable.

Example:

```
layerRules (
  trackDefs (
    boundaryTrackData (
      (myBTrackData1 -width 0.2 -space 0.5 -type default -color red)
      (myBTrackData2 -width 0.4 -space 0.6 -type nonReserved -color green)
      (myBTrackData3 -width 0.2 -space 0.5 -type default -color red) )
    ); boundaryTrackData
  ); trackDefs
);layerRules
```

boundaryTracks()

`boundaryTracks()` specifies the layer and direction of a group of tracks in a [boundaryTrackRegion\(\)](#). For example, you can specify that a group of tracks (`track1`, `track2`, and `track3`) all run vertically on layer `m1`.



Syntax:

```
boundaryTracks (
    ; (<string>name <string>layer (<string>boundaryTrackData)
    ; -direction <horizontal | vertical>
); boundaryTracks
```

Note:

Any line preceded by a semicolon (;) is treated as a comment.

Argument	Description
name	Unique name that identifies the group of tracks that are on the specified layer and direction.
layer	Track layer.
boundaryTrackData	List of tracks that belong in this track group. boundaryTrackData() describes each track.
-direction	Track direction: • horizontal • vertical

Example:

```
layerRules (
    trackDefs (
        boundaryTrackData (
            (myBTrackData1 -width 0.2 -space 0.5 -type default -color red)
            (myBTrackData2 -width 0.4 -space 0.6 -type nonReserved -color green)
            (myBTrackData3 -width 0.2 -space 0.5 -type default -color red) )
        ); boundaryTrackData
    )
```

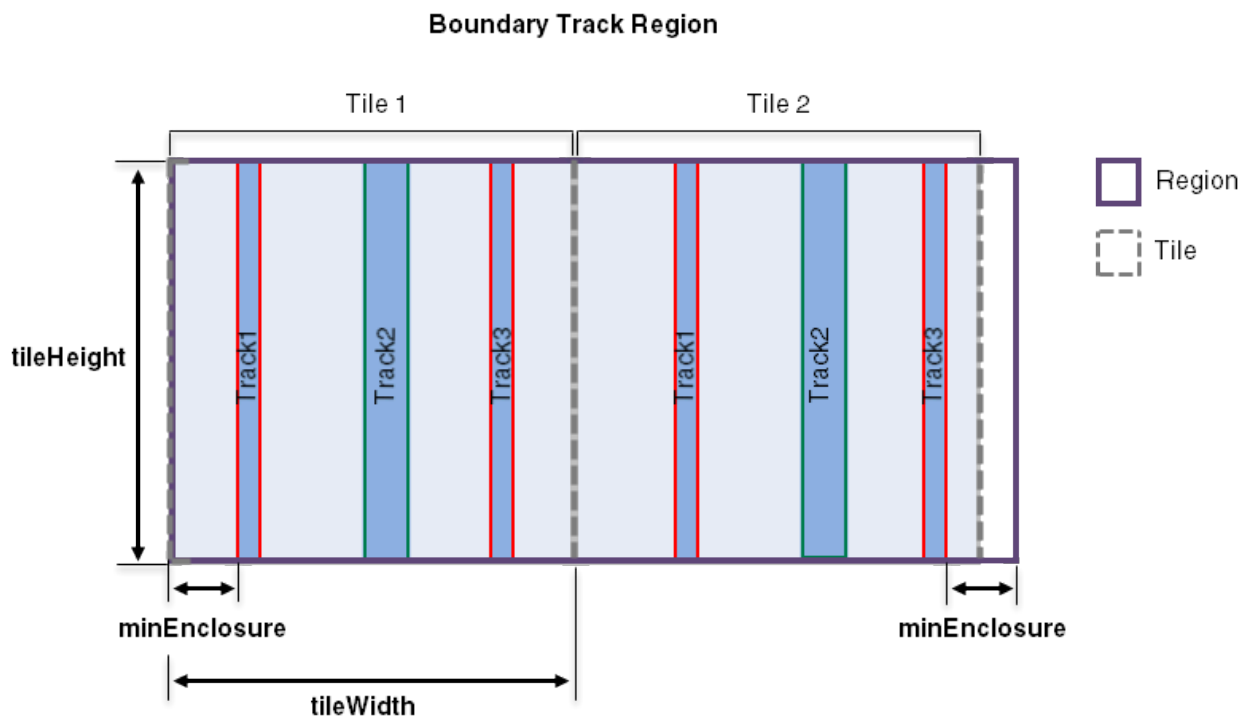
```
boundaryTracks (
  (myBTrack1 m1 (myBTrackData1 myBTrackData2 myBTrackData3) -direction
  vertical)
); boundaryTracks
); trackDefs
); layerRules
```

boundaryTrackRegion()

You use `boundaryTrackRegion()` to create a boundary track region definition. This definition lets designers create and edit track regions directly using the **Create > Rectangle** command. See [Creating and Editing Track Region PCells](#).

A boundary track region consists of a set of tiles. Tiling always starts at the left and bottom of a region. Each tile consists of a group of tracks. The individual tracks are defined by [boundaryTrackData\(\)](#), and each grouping of tracks are defined by [boundaryTracks\(\)](#).

In the following example, the boundary track region consists of two tiles. Each tile contains three tracks.



Syntax:

```
boundaryTrackRegion (
  ; (<string>name (<string>boundaryTrackName)
  ; -boundaryLPP (<string><string>)
  ; [-tileWidth <float>
  ; [-tileHeight <float>
```

```

; [-minNumTiles <int><int>
; -minEnclosure <float><float>)
); boundaryTrackRegions

```

Note:

Any line preceded by a semicolon (;) is treated as a comment.

Argument	Description
name	Unique name of the boundary track region in the OA database.
boundaryTrackName	List of boundaryTracks() that are in the region.
-boundaryLPP	Region layer and purpose.
-tileWidth	Track region width.
-tileHeight	Track region height.
-minNumTiles	Minimum number of tiles in the region.
-minEnclosure	Minimum X and Y distance between the region boundary and the first and last track in the region.

Example:

```

layerRules(
  trackDefs(
    tracks(...)
    groups(...)
    regions(...)
    boundaryTrackData(
      (myBTrackData1 -width 0.2 -space 0.5 -type default -color red)
      (myBTrackData2 -width 0.4 -space 0.6 -type nonReserved -color green)
      (myBTrackData3 -width 0.2 -space 0.5 -type default -color red) )
    ); boundaryTrackData
    boundaryTracks(
      (myBTrack1 m1 (myBTrackData1 myBTrackData2 myBTrackData3) -direction
      vertical)
    ); boundaryTracks
    boundaryTrackRegions(
      (myBTrackRegion (myBTrack1) -boundaryLPP (layer1 drawing1)
      -tileWidth 2.6 -minEnclosure (0.5 0) )
    ); boundaryTrackRegions
  ); trackDefs
); layerRules

```

trackDefs Example

The following code shows fully specified `trackDefs`:

Example 33

```

layerRules(
  trackDefs(

```

```

tracks(
  ( myTrack1 -type reserved -offset 0.05 -width 0.02 -color red -visible
true -active false)
  ( myTrack2 -type reserved -offset 0.03 -color green -active false )
  ( myTrack3 -type nonReserved -offset 0.04 -width 0.1 -sigType power)
); tracks

groups(
  ( myGroup1 m1 (myTrack1 myTrack2)
    -direction horizontal -pitch (0.02 0.06))
  ( myGroup2 m2 (myTrack1 myTrack3)
    -direction vertical -pitch (0.25 0.15))
); groups

regions(
  ( myRegion1 (myGroup1 myGroup2) -flip (true false) -minCount (2 3)
-maxCount (3 4) )
  ( myRegion2 (myGroup1) -maxCount (4 5) -autoCreateGlobalRegion true
-snapGrid (0.05 0.07) -snapOffset(0.15 0.75))
  ( myRegion3 (myGroup2) -defaultTracksVisible true -defaultTracksActive
false -groupEnclosure (0.03 0.02) -enclosedGroups (myGroup2))
  ( myRegion4 (myGroup1 myGroup2) -flipGroups (myGroup2) -minCount (2 6)
-maxCount (2 4))
  (myRegion5 (myGroup1 myGroup2) -minCount (2 6) -autoCreateGlobalRegion
true -flipTilesX true )      (myRegion6 (myGroup1) -maxCount (2 4)
-autoCreateGlobalRegion true -flipTilesY true -flipTilesYFirst true)
  (myRegion7 (myGroup1 myGroup2) -flipGroups (myGroup1 myGroup2) -flipGroupsFirst
(myGroup1) -groupFlipTarget region)      (myRegion8 (myGroup1 myGroup2) -flip
(true false) -flipGroupsFirst (myGroup2) -groupFlipTarget tile)
); regions

boundaryTrackData(
  (myBTrackData1 -width 0.2 -space 0.5 -type default -color red)
  (myBTrackData2 -width 0.4 -space 0.6 -type nonReserved -color green)
  (myBTrackData3 -width 0.2 -space 0.5 -type default -color red) )
); boundaryTrackData

boundaryTracks(
  (myBTrack1 m1 (myBTrackData1 myBTrackData2 myBTrackData3) -direction
vertical)
); boundaryTracks

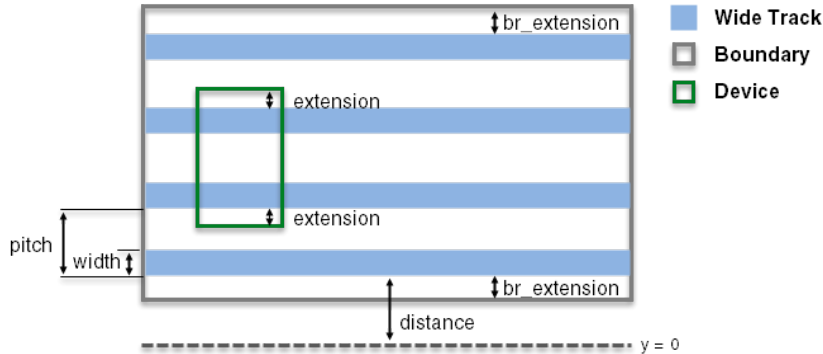
boundaryTrackRegions(
  (myBTrackRegion (myBTrack1) -boundaryLPP (layer1 drawing1)
    -tileWidth 2.6 -minEnclosure (0.5 0) )
); boundaryTrackRegions
); trackDefs
); layerRules

```

See also [Sample Technology File](#).

wideAlignments

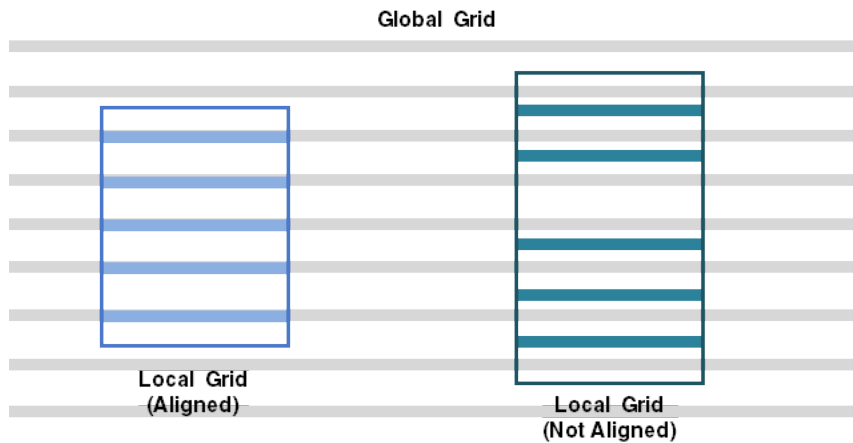
To ensure that devices (such as FinFETs) are placed uniformly and the fins are positioned correctly in a design, the devices are snapped to wide tracks within a wide alignment grid. You can use the `wideAlignments` rule to define the grids and create alignment boundaries.



You can define two types of wide alignment grids:

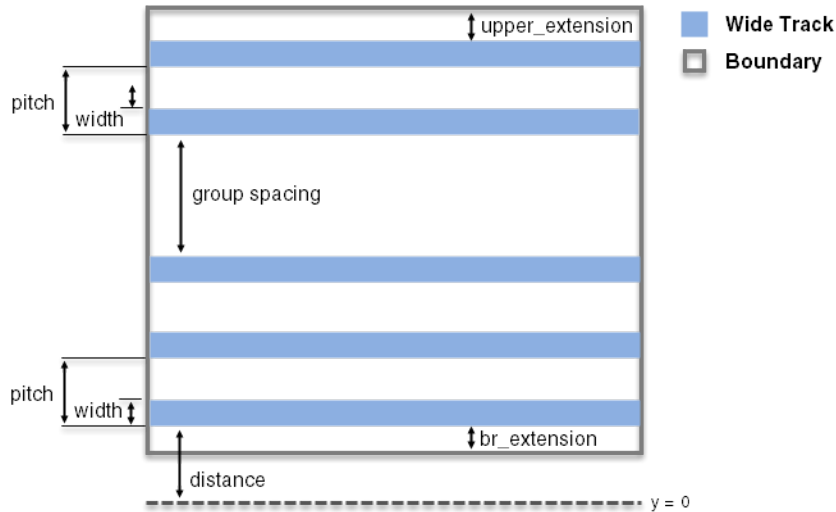
- **Global:** A wide alignment grid that can be rendered without any boundary within the cell. To define a global wide alignment grid, set the `global` argument to `true`.
- **Local:** A wide alignment grid that is rendered with a local boundary within the cell. To define a local wide alignment grid, set the `global` argument to `false`.

You can define multiple local wide alignment grids. The local grids do not have to align with the global grid or with each other. Having multiple local grids enables the design to accommodate different rules and permit devices to be placed closer together.



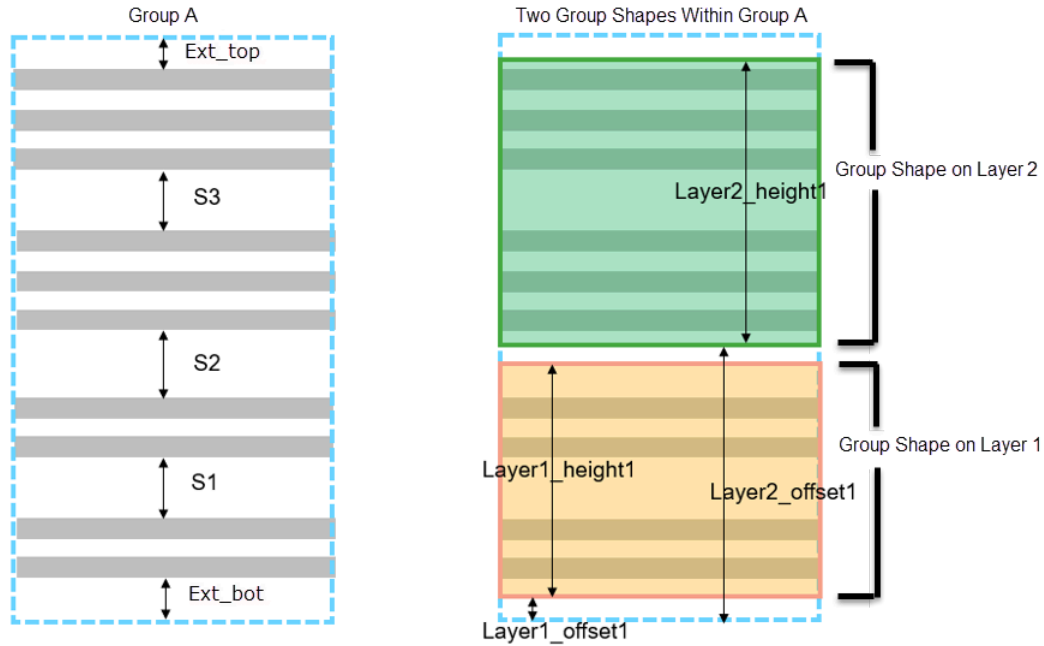
Within a local wide alignment boundary, you can define grid groups using the `-groups` argument. The width of the wide tracks and the distance between adjacent tracks (pitch) must be uniform across the groups.

In the example shown, the local wide alignment boundary on the right contains two grid groups. The first group contains two fin tracks. The second group contains three fin tracks. The track width and pitch are the same in both groups.



The foundries require some associated shapes to be created together with groups. Such shapes could be on different layers.

In below example, the group specification has four groups defined, and there are associated shapes on two layers. The location of each group shape is defined by an offset and a height, referring to the bottom edge of the group specification. Constraint parameters `groupShapeLayers` and `groupShapePoints` are used to define the group shapes, both of which are array values and have the same size. `groupShapeLayers` is an array of `oaLayerValue`, while `groupShapePoints` is an array of `oaDualInValue` with the two values representing offset and height of a shape. Each pair of `groupShapeLayers` and `groupShapePoints` elements with the same index represents one group shape. There could be multiple group shapes on the same layer. A parameter `groupShapePurposes` can be used to specify the purpose of each group shapes. If not specified, drawing purpose will be used.

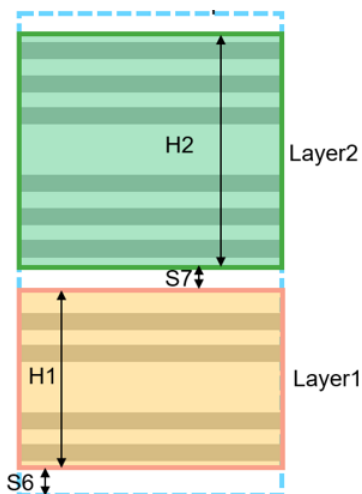


The syntax lets you specify group shapes one by one from bottom to top within one group specification.

For each group shape, you need to specify the following:

- Layer/Lpp.
- Spacing to the previous group shape, or the bottom edge of group specification if the current group shape is the first one. For example, S6 and S7 in the following figure.
- Height of the group shape. For example, H1 and H2 in the following figure.

Two Group Shapes Within Group A



A group specification can continue if more than two groups are needed. If a group specification is given for a wideAlignment constraint, that wide alignment boundary must have a height that is a multiple of the completed group specification including the lower and upper extension values. The height of one group includes all the tracks defined in the group specification plus the lower and upper extension values. The group associated shapes also continue in the same manner as group specification.

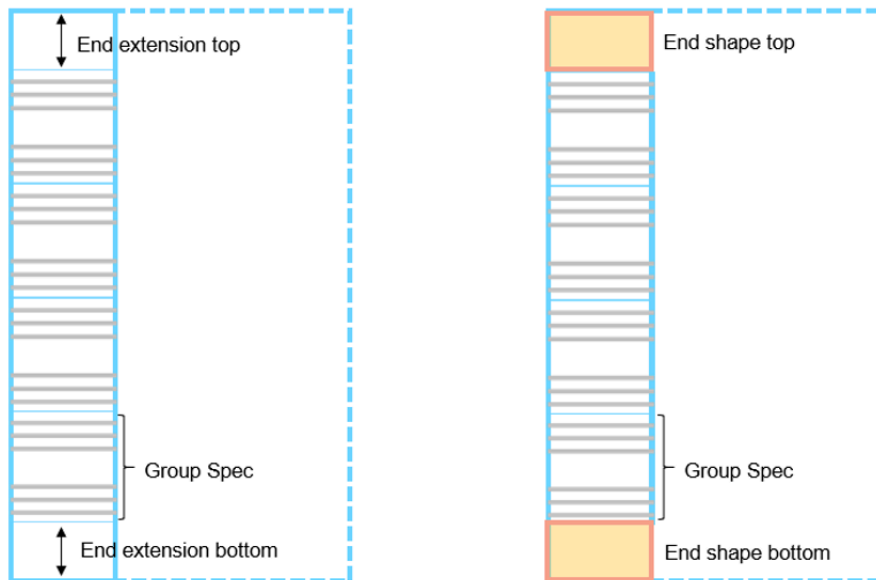
The foundries also require how big a step should be when the height of the boundary object is being changed. By default, the boundary height could be changed by an increment of one fin track (if no group specification is defined) or one group (if group specification is defined). It is possible that for some certain boundary layers, the increment step should be multiple fin tracks or groups. Constraint parameter `layerStep` is introduced for this purpose.

The newer processes require shapes on certain layers to be created at the top and bottom of the boundary, which should also be covered by the boundary layer shape.

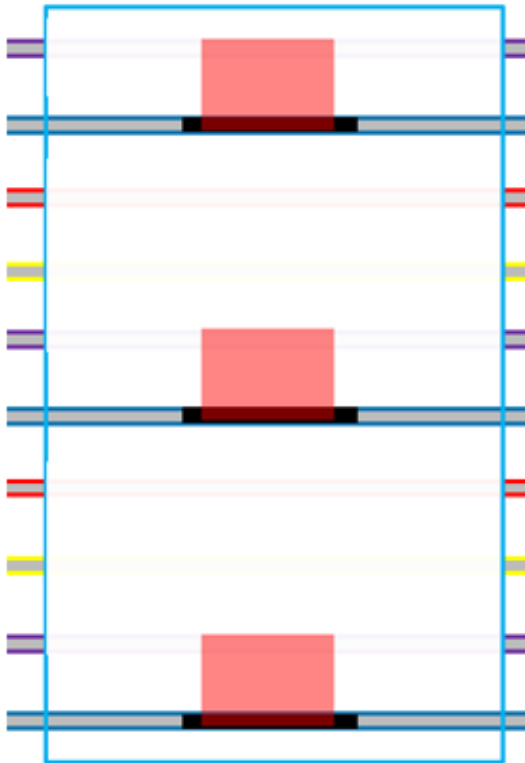
As shown in the example below, two end shapes need to be created on top and bottom of the boundary, and between them the groups repeat in the same way as defined in the group specification. Extra end extensions on top and bottom are needed to cover

these two end shapes. Constraint parameters `endExtensions` and `endShapeLayer` are introduced to define the end shapes.

Alignment boundary with end shapes at top and bottom



For certain processes, to display different colors for global fin tracks with a pattern, you can distinguish different fin tracks for better device matching purpose. To support this, an optional display pattern can be defined only for global wide alignments. Now only two colors are supported, whose display packets are defined internally by the tool, and the patterns can be defined using A and B for different colors. For example, ABBA means that a repeat pattern contains four fin tracks and the colors of these four fin tracks should be color1, color2, color2, and color1.



The global wide alignments also have restrictions on where the bottom edge of the boundary shape can start. An optional parameter `validBoundarySnapBottomPattern` defines which fins are valid and which fins are invalid for the bottom edge of boundary shape can snap to. This parameter can only be defined for global wide alignments whose `boundaryOptional` are false. A pattern can be defined using a list of 1 and 0, 1 representing valid and 0 representing invalid. For example, a value of (1 1 0 0) means that only the first and second fins of every 4 fins can be used for the boundary shape's bottom edge to locate. This pattern is repeated throughout the whole boundary and the first fin is counted from the bottom.

To ensure that devices snap to wide tracks, you must select **Wide Alignment** in the **Layout Design Options** dialog box. For more information, see the *Custom Compiler Layout Editor User Guide*.

When wide alignment grids are defined and the **Wide Alignment** option is selected in the **Layout Design Options** dialog box, devices will always snap to tracks in local wide alignment grids, if they exist. Otherwise, devices will snap to tracks in the global grid.

To display the wide alignment grids on the canvas, you must select the **Global Wide Alignment > Visibility** option in the Object/Layer Panel. For instructions, see the *Custom Compiler Environment User Guide*.

The `wideAlignments` rule defines a constraint group called `wideAlignments` and adds it as a `constraintGroupMem` into the `oaTech` foundry constraint group. One `wideAlignment` definition represents an OA layer array constraint.

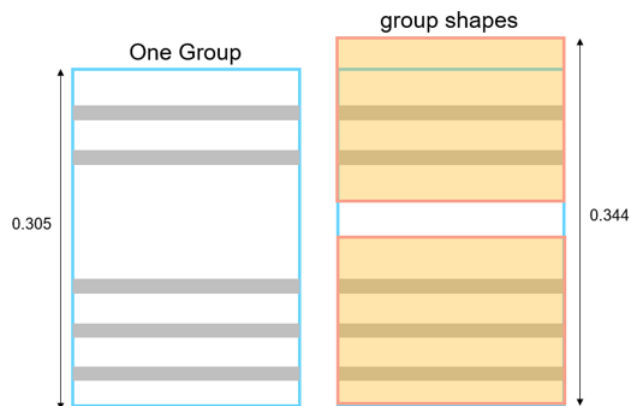
The `wideAlignments` rule supports an array of layers and an array of layer-purpose pairs.

- (Optional) The first element of the array is the layer or layer-purpose pair for the wide track shapes. If it is set to `nil`, the Layout Editor will render the wide tracks using an annotation layer, whose default display packet is `FinGrid`.
- (Optional) The second element of the array is the boundary layer or layer-purpose pair. If it is not specified, the system reserved-layer of the `prBoundary` is set to `layer`, and system reserved-purpose of the boundary is set to `purpose`.
- The remaining elements are associated layers or layer-purpose pairs.

Note:

The tool will report errors and stop importing when the height of the specified group shapes exceed (>) the height of one group.

Example of invalid settings:



```
wideAlignments(
  (0.012 0.036 -groups (3 0.07 2)
  -groupShapes (FKB_B1 (0 0.17) (FKB_B2 drawing) (0.004 0.17))
  -stepY 2
  nil
  (FKB_6 (0.05 0.053) 36)
  (OD_ 0.004 2)
); wideAlignments
```

The height of one group:

$$0.05 + (3-1) * 0.036 + 0.012 + 0.07 + (2-1) * 0.036 + 0.012 + 0.053 = 0.305$$

The total height of group shapes: $0 + 0.17 + 0.004 + 0.17 = 0.344$

Syntax:

```
wideAlignments (
    ;(width pitch [distance [global]]
    ; [-groups(<array>)][-groupShapes(shapeLayer1\
        (shapeLayer1 shapePurpose1)(spacing height)...)]
    ; [-endShapes(endLayer|(endLayer endPurpose)end-extension|\
        (bottom-extension top-extension) ) ]
    ; [-stepY step]
    ; [-colorDisplayPattern (colorPattern)]
    ; [-validFinSnapBottomPattern boolArray]
    ; [-validBoundarySnapBottomPattern boolArray]
    ; [wideTrackLayer|(wideTrackLayer wideTrackPurpose)]
    ; [(boundaryLayer|(boundaryLayer boundaryPurpose)
        br-extension|(br-extension upper-extension) br-minNumTrack)]
    ; (deviceLayer|(deviceLayer devicePurpose)
        extension minNumTrack)*
); wideAlignments
```

Note:

Any line preceded by a semicolon (;) is treated as a comment.

Argument	Description
width	The width of the wide track.
pitch	The distance between adjacent wide tracks.
distance	(Optional) The vertical offset distance from the origin. If the distance is not specified, then the boundary shapes can be placed in any position. The <code>distance</code> argument is required if <code>global</code> is <code>true</code> .
global	(Optional) The wide alignment grid is global. Valid values are <code>true</code> and <code>false</code> . Default: <code>false</code> If <code>true</code> , the <code>distance</code> argument defines the location of the grid position. If <code>false</code> , grid is created relative to the lower left boundary box coordinate. Objects are placed on the global grid only if the <code>leWideAlignmentSnapping</code> Tcl preference is set to <code>true</code> .
-groups	(Optional) The wide alignment groups. The first element specifies the number of tracks in the first group. The remaining elements must be paired and each pair specifies the spacing between the previous group and the current group and the number of tracks in the current group. For example, <code>-groups(3 0.3 2 0.5 4)</code> specifies that within a local boundary, the first group contains 3 tracks, the second group contains 2 tracks, and the last group contains 4 tracks. The spacing between the first group and the second group is 0.3. The spacing between the second group and the third group is 0.5.
boundaryOptional	(Optional) The wide alignment boundary is optional for the wide alignment grid. Valid values are <code>true</code> and <code>false</code> . Default: <code>false</code> If <code>true</code> , the boundary is optional. If <code>false</code> , the boundary is required.

Argument	Description
<code>wideTrackLayer</code>	(Optional) The wide track layer is used for the wide track shapes. Setting <code>wideTrackLayer</code> to <code>nil</code> uses the default FinGrid packet when the grid tracks are rendered.
<code>wideTrackPurpose</code>	(Optional) The wide track purpose name. Setting <code>wideTrackPurpose</code> to <code>nil</code> uses the default FinGrid packet when the grid tracks are rendered.
<code>boundaryLayer</code>	(Optional) The layer is used for the wide alignment boundary.
<code>boundaryPurpose</code>	(Optional) The boundary purpose name.
<code>br-extension</code>	The distance between the boundary and the outermost wide tracks. This is the first element in an <code>oaIntValue</code> objects array for the <code>layerExtensions</code> parameter. The array size must match the number of layers defined for the constraint minus one. The index of the LPP in the layer array minus one is used to access the extension value in this array.
<code>br-extension upper-extension</code>	(Optional) The distance between the top-most track and the boundary. If you do not supply this value, the <code>br-extension</code> value is used.
<code>br-minNumTrack</code>	The minimum number of wide tracks in the wide alignment grid. This is the first element in an <code>oaIntValue</code> objects array for the <code>layerMinTracks</code> parameter. The array size must match the number of layers defined for the constraint minus one. The index of the LPP in the layer array minus one is used to access the number of tracks in this array.
<code>deviceLayer</code>	The layer on which the device is placed within the Wide Alignment grid.
<code>devicePurpose</code>	The device purpose name.
<code>minNumTrack</code>	The minimum number of wide tracks to which the device layer shapes are snapped during within the wide alignment boundary. This is the second element in an <code>oaIntValue</code> objects array for the <code>layerMinTracks</code> parameter. The array size must match the number of layers defined for the constraint minus one. The index of the LPP in the layer array minus one is used to access the number of tracks in this array.
<code>extension</code>	The distance the device overhangs from the wide tracks. This is the second element in an <code>oaIntValue</code> objects array for the <code>layerExtensions</code> parameter. The array size must match the number of layers defined for the constraint minus one. The index of the LPP in the layer array minus one is used to access the extension value in this array.
<code>groupShapeLayer</code>	Defines the layer for group shape. It is the element in <code>oaLayerArrayValue</code> for parameter <code>groupShapeLayers</code> . The index of each layer is used to access corresponding shape data in the parameter <code>groupShapePoints</code> .
<code>groupShapePurpose</code>	Defines the purpose for group shape. It is the element in <code>oaValueArrayValue</code> for parameter <code>groupShapePurposes</code> . Each element of the array is <code>oaPurposeValue</code> .

Argument	Description
<code>spacing</code>	Defines the spacing to previous group shape or the bottom edge of group specification. It will be translated to the offset from bottom edge of group specification and stored to the first value of the element of parameter <code>groupShapePoints</code> .
<code>height</code>	Defines the height of group shape. It will be stored to the second value of the element of parameter <code>groupShapePoints</code> .
<code>end-extension</code>	Defines the extra extension at top and bottom of the boundary to cover end shapes.
<code>top-extension</code>	Defines the extra extension at top of the boundary to cover end shapes.
<code>bottom-extension</code>	Defines the extra extension at bottom of the boundary to cover end shapes.
<code>endShapeLayer</code>	Defines the layer for end shapes at top and bottom of the boundary.
<code>endShapePurpose</code>	Defines the purpose for end shapes at top and bottom of the boundary.
<code>step</code>	Defines the vertical direction steps of boundary shape when interactively increasing or decreasing the boundary size. For definitions without group specs, the step value means number of fins. For definitions with group specs, the step value means number of groups.
<code>comment</code>	(Optional) Description of the constraint. The first string specifies the rule ID, and the second string describes the constraint.
<code>colorDisplayPattern</code>	(Optional) Defines the color display pattern for global fin tracks (global = true). This argument should only be defined for global wide alignments. An array of <code>oalntValue</code> with valid values including 0 and 1 (only two colors are supported now).
<code>validFinSnapBottomPattern</code>	(Optional) Defines valid fins for snapping for global fin tracks. This argument should only be defined for global wide alignments whose <code>boundaryOptional</code> is false (global = false). An array of <code>oalntValue</code> with valid values including 0 and 1, 0 representing invalid and 1 representing valid for snapping. If a fin is invalid, it should not even be shown on the canvas. For example, an array of (1 0 0 0) means with a group of 4 fins, only the first fin is valid for snapping while the other 3 are invalid. This pattern is repeated throughout the whole boundary, with the first fin counted from the bottom.
<code>validBoundarySnapBottomPattern</code>	(Optional) Defines valid fins for the boundary shape's bottom edge to snap to. This argument should only be defined for global wide alignments whose <code>boundaryOptional</code> is false. An array of <code>oalntValue</code> with valid values including 0 and 1, 0 representing invalid and 1 representing valid for snapping. For example, an array of (1 1 0 0) means with a group of 4 fins, only the first and second fins are valid for snapping for boundary shape's bottom edge. This pattern is repeated throughout the whole boundary, with the first fin counted from the bottom.

Example 34 *FIN groups specified with upper extension*

```
layerRules(  
  wideAlignments(  
    (0.06 0.016 -groups(3 0.3 2 0.5 4)  
      (fin_track drawing)  
      ((m_boundary drawing) (0.06 0.02) 9)  
      ((od_a drawing) 0.04 2)  
      ((od_b drawing) 0.02 1) -comment "rule 3.2" "This is an example")  
    ); wideAlignments  
  ); layerRules
```

Example 35 *FIN groups specified without unique upper extension*

```
layerRules(  
  wideAlignments(  
    (0.06 0.016 -groups(3 0.3 2 0.5 4)  
      (fin_track drawing)  
      ((m_boundary drawing) 0.06 9)  
      ((od_a drawing) 0.04 2)  
      ((od_b drawing) 0.02 1) -comment "rule 3.2" "This is an example")  
    ); wideAlignments  
  ); layerRules
```

Example 36 *FIN grids specified*

```
layerRules(  
  wideAlignments(  
    (0.02 0.05 0.20  
      (FIN drawing)  
      ((FINBOUNDARY drawing) 0.05 2)  
      ((DIFF drawing) 0.04 2)  
    ); wideAlignments  
  ); layerRules
```

Example 37 *FIN grids specified*

```
layerRules(  
  wideAlignments(  
    (0.06 0.16 0.20  
      (fin_track drawing)  
      ((m_boundary drawing) 0.06 2)  
      ((od_a drawing) 0.04 2)  
      ((od_b drawing) 0.02 1)  
    ); wideAlignments  
  ); layerRules
```

Example 38 *No wide track (FIN) is specified*

```
layerRules(  
  wideAlignments(  
    (0.06 0.16 0.20 true  
      nil  
      ((m_boundary drawing) 0.06 2)  
      ((od_a drawing) 0.04 2)  
      ((od_b drawing) 0.02 1)  
    ); wideAlignments  
  ); layerRules
```

Example 39 *No wide track and boundary are specified*

```
layerRules(  
  wideAlignments(  
    (0.06 0.16 0.20  
      nil  
      nil  
      ((od_a drawing) 0.04 2)  
      ((od_b drawing) 0.02) 1)  
    ); wideAlignments  
  ); layerRules
```

Example 40 *Two group shapes are specified for each group specification, -stepY is 2 groups*

```
layerRules(  
  wideAlignments(  
    (0.012 0.036 -groups(3 0.07 3) \  
      -groupShapes(FKB_B1 (0 0.17) (FKB_B2 drawing) (0.004 0.17))  
      -stepY 2 nil (FKB_6 0.053 36) (OD 0.004 2)  
    ); wideAlignments  
  ); layerRules
```

Example 41 *End shapes (bottom-extension is 0.2, top-extension is 0.3) are specified, -stepY is 2 groups*

```
layerRules(  
  wideAlignments(  
    (0.012 0.036 -groups(3 0.07 3) \  
      -endShapes (BLK3 (0.2 0.3))  
      -stepY 2 nil (FKB_6 0.053 36) (OD 0.004 2)  
    ); wideAlignments  
  ); layerRules
```

Example 42 *-stepY is 3 wide tracks*

```
layerRules(  
  wideAlignments(  
    (0.012 0.036  
      -stepY 3 nil (FKB1 0.018 3) (OD 0.004 1)  
    ); wideAlignments  
  ); layerRules
```

Example 43 *-colorDisplayPattern is ABBA*

```
layerRules(  
  wideAlignments(  
    (0.012 0.036  
      -stepY 3  
      -colorDisplayPattern (A B B A)  
      -colorDisplayBoundaries (MATCHING_GRID)  
      nil (FKB1 0.018 3) (OD 0.004 1)  
    ); wideAlignments  
  ); layerRules
```

Example 44 *validFinSnapBottomPattern*

```
layerRules(  
  wideAlignments(  
    (0.012 0.036 false  
      -stepY 3  
      -validFinSnapBottomPattern (1 0 0 0)  
    ); wideAlignments  
  ); layerRules
```

```
-validBoundarySnapBottomPattern (0 1 0 0)
nil (FKB1 0.018 3) (OD 0.004 1)
); wideAlignments
); layerRules
```

To see an example technology file, see [Sample Technology File](#).

streamLayerMapping

The Custom Compiler environment uses layer mapping files during design import and export operations. The files map OA layer-purpose pairs to equivalent GDS layer-purpose pairs. See Import and Export.

Instead of maintaining separate layer mapping files, you have the option of creating a `streamLayerMapping` subsection to store the layer mapping information. This method can be advantageous, because the data is stored centrally with other technology data.

When you import a technology file that contains the `streamLayerMapping` subsection:

- A `.layermap` file is created in the library. Data in the `streamLayerMapping` subsection of the technology file is transferred into the `.layermap` file. If a `.layermap` file with the same name already exists, the technology import process will use the import mode you specify (**Full Overwrite**, or **Partial Replace**).
- The remaining technology data is transferred into a new or existing technology database, depending on the import options you set.

See Importing Technology Data.

When you export the technology data, the Stream layer mapping information is added in the `streamLayerMapping` section of the technology file.

- If you select the **Local** export option, the export process will look for the `.layermap` file associated with the specified library only.
- If you do not select **Local**, the export process also obtains layer mapping information from hierarchical libraries if the specified library has attachments or references other technology databases.

See Exporting Technology Data.

Syntax:

```
streamLayerMapping (  
; (<layerName> <purposeName> <gdsLayer> <gdsDataType> [color:<colorName>  
  [locked | unlocked]] [CUTSIZE:<size>:<size>])  
  ...  
); streamLayerMapping
```

Note:

Any line preceded by a semicolon (;) is treated as a comment.

Argument	Description
layerName	(Required) The name of the layer in the OA database. For more information, see OA, GDSII, and OASIS Mappings.

Argument	Description
purposeName	(Required) The purpose of the OA layer. For more information, see OA, GDSII, and OASIS Mappings.
gdsLayer	(Required) The layer number in the GDS or OASIS. For more information, see OA, GDSII, and OASIS Mappings.
gdsDataType	(Required) The data type number in the GDS or OASIS. The GDS or OASIS data type number can be a single number or a range of numbers. For example, the following are valid values for gdsDataType#: 1, 2, 5, or 0-255. For more information, see OA, GDSII, and OASIS Mappings.
color:<colorName>	(Optional) A mask color. The colorName must be preceded by the keyword <code>color</code> , followed by a colon (for example, <code>color:red</code>). The color names you supply must be defined in the technology file, or they must have the value <code>same</code> . If no color information is supplied in the file, the translator assumes that the shape on the layer has no color. For more information, see Mask Color and Color Lock Attributes.
locked unlocked	(Optional) The color lock status, which is only used if a colorName is specified, but it is not required. Valid values for colorLock are <code>locked</code> and <code>unlocked</code> . For more information, see Mask Color and Color Lock Attributes.
CUTSIZE:<size>:<size>	(Optional) The dimensions of the rectangular cut, <dimension1>:<dimension2>. The dimension values must be preceded by the keyword <code>CUTSIZE</code> , followed by a colon (for example, <code>CUTSIZE:0.032:0.032</code>). The <code>Material</code> field can be used to specify different mappings for cut rectangles of different sizes. If the <code>Material</code> value is <code>cut</code> , the cut dimensions can follow in the same field. For more information, see Cut Size Attribute.

Example:

```
streamLayerMapping (    ( M1 drawing 16 30 )    ( M1 drawing 16 31
    color:E1 locked )    ( M1 drawing 16 32 color:E1 unlocked )    ( M1
    drawing 16 33 color:E2 locked )    ( M1 drawing 16 34 color:E2
    unlocked )) ;streamLayerMapping
```

pinTextLayers

The Layout Editor uses layer mapping files during certain operations, such as pin and label generation. The layer mapping files contain the shape, text, and pin layer-purpose pair information. See [Generating Multiple Pins Simultaneously](#) and [Generating Labels](#).

Instead of maintaining separate layer mapping files, you have the option of creating a `pinTextLayers` subsection to store the layer mapping information in the technology database. This method can be advantageous, because the data is stored centrally with other technology data.

Syntax:

```
pinTextLayers (
; ((shapeLPP) (textLPP) (pinLPP))
...
); pinTextLayers
```

Note:

Any line preceded by a semicolon (;) is treated as a comment.

Argument	Description
shapeLPP	Layer-purpose pair specification for the shape layer.
textLayerLPP	Layer-purpose pair specification for the text layer.
pinLayerLPP	Layer-purpose pair specification for the pin layer.

Example:

```
pinTextLayers
; ((shapeLpp) (textLpp) (pinLpp))
((m1 drawing) (m1text drawing) (m1 pin))
((m2 drawing) (m2text drawing) (m2 pin))
((m3 drawing) (m3text drawing) (m3 pin))...
```

voltageTextLayers

The `voltageTextLayers` subsection stores layer-purpose pair (LPP) mappings that are used for creating physical voltage annotations. The order in which the mappings appear in the subsection is also the order in which the LPPs are searched during the annotation process.

In case the `voltageTextLayers` subsection is missing or is incorrect, the LPP mappings specified by the `leVDRCPHysicalAnnotationLabelTextLPPs` Tcl preference will be used.

Syntax:

```
voltageTextLayers (
; ((objectLPP) (minVoltageLPP) (maxVoltageLPP))
...
); voltageTextLayers
```

Note:

Any line preceded by a semicolon (;) is treated as a comment.

Argument	Description
<code>objectLPP</code>	Layer-purpose pair of the wire object.
<code>minVoltageLPP</code>	Layer-purpose pair of the minimum voltage label.
<code>maxVoltageLPP</code>	Layer-purpose pair of the maximum voltage label.

If any of the LPPs you specified cannot be found in the technology database, the **Create Voltage Annotations** command will skip the particular mapping.

For incremental technology:

- The reference (base) technology and the technology you are importing both can contain their own `voltageTextLayers` sections.
- When exporting a technology database, and ☒ Export Local Tech Only :
 - If the current technology database contains `voltageTextLayers`, it will be exported.
 - If the current technology does not have `voltageTextLayers`, and the reference (base) technology contains `voltageTextLayers`, the exported `voltageTextLayers` section will be empty (will not contain any data).

- When exporting a technology database, and ☐ Export Local Tech Only :
- If the current technology contains `voltageTextLayers`, only the current technology's `voltageTextLayers` will be exported.
- If the current technology does not have `voltageTextLayers`, and the reference (base) technology contains `voltageTextLayers`, the `voltageTextLayers` in the reference (base) technology will be exported.

Example:

```
voltageTextLayers ( ; ( (objectLPP) (minVoltageLPP) (maxVoltageLPP) )  
  ( (m1 drawing) (m1 loV) (m1 hiV) ) ( (m2 drawing) (m2 loV) (m2 hiV) )  
  ( (m3 drawing) (m3 loV) (m3 hiV) ) ... )
```

Via Definitions Section

The `viaDefs` section in the technology file defines via types that are used for the design. The definitions specify the formation of the via, the layers that are connected when the via is used, and the rules associated with the via, such as width, spacing, and enclosure.

You can specify the definitions for the following via types:

- **Standard:** Has a predefined set of via parameters that can be tailored to make the proper connection in a variety of situations. Because standard vias do not have a corresponding master design, they do not appear in the Library Manager. When you place a standard via, you can optionally create an array of vias.
- **Custom:** References a master design in a library, and appears in the Library Manager. When you place a custom via, the via parameters are set to their defaults. You can only place instances of custom vias. You cannot create custom via arrays.

For each via type, you can define variations. The variations are based on the original via definition (standard or custom), but they use different parameters.

Syntax:

```
viaDefs (  
  standardViaDefs  
  standardViaVariants  
  customViaDefs  
  customViaVariants  
  bridgeVias  
) ; viaDefs
```

Note:

Any line preceded by a semicolon “;” is treated as a comment.

The following table defines the sections inside the `viaDefs` section, in the order in which they appear in the technology file.

Subsection	Purpose
<code>standardViaDefs</code>	Defines the standard vias used in your design technology.
<code>standardViaVariants</code>	The variants of the specified standard vias.
<code>customViaDefs</code>	Identifies vias that are defined in separate cellviews in a design library.
<code>customViaVariants</code>	The variants of the specified custom vias.

standardViaDefs

The `standardViaDefs` defines each type of via in your technology. These are the default vias that are used when you use the **Create > Vias** command in the Layout Editor. The dimensions are all given in microns.

Syntax:

```
standardViaDefs (
  (<string>viaDefName
   <string>layer1
   <string>layer2
   (<string>cutLayer
    <float>cutWidth
    <float>cutHeight
    <float>[resistancePerCut])
   (<int>cutRows
    <int>cutCol
    (<float>cutSpace)
    (<dualFloat>layer1Enc)
    (<dualFloat>layer2Enc)
    (<dualFloat>layer1Offset)
    (<dualFloat>layer2Offset)
    (<dualFloat>origOffset)
    [<string|LPP>implant1
     (<dualFloat>implant1Enc)
     [<string|LPP>implant 2
      (<dualFloat>implant2Enc) ]])
  );standardViaDefs
```

Note:

Any line preceded by a semicolon “;” is treated as a comment.

Argument	Description
<code>viaDefName</code>	The name of the via.

Argument	Description
layer1	The lower layer.
layer2	The upper layer.
cutLayer	The contact / via cut being used.
cutWidth	The width of the cut layer being used.
cutHeight	The height of the cut layer being used.
resistancePerCut	The resistance per cut.
cutRows	The number of cut rows.
cutCol	The number of cut columns.
cutSpace	The space between the cuts. This is specified as a list. The first value is the spacing in the horizontal direction and the second value is the spacing in the vertical direction.
layer1Enc	The enclosure of layer1 over the cut layer.
layer2Enc	The enclosure of layer2 over the cut layer.
layer1Offset	The offset of layer1.
layer2Offset	The offset of layer2.
origOffset	The offset of the origin.
implant1	The implant1 layer.
implant1Enc	The enclosure of implant1 over the cut layer.
implant2	The implant2 layer.
implant2Enc	The enclosure of implant2 over the cut layer.

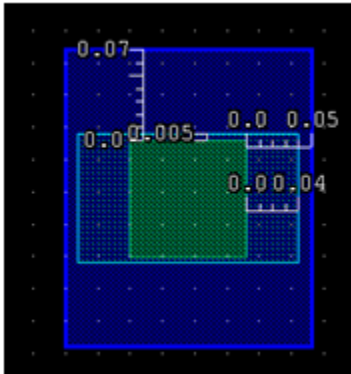
Example 45 Sample standardViaDefs section

```

standardViaDefs(
  ( polyCont poly m1 ("cont" 0.09 0.09) (1 1 (0.1 0.1))
    (0.05 0.07) (0.04 0.005) (0.0 0.0) (0.0 0.0) (0.0 0.0) )
  ( pdiff diff m1 ("cont" 0.09 0.09) (1 1 (0.1 0.1))
    (0.05 0.05) (0.005 0.04) (0.0 0.0) (0.0 0.0) (0.0 0.0)
    pplus (0.02 0.02) )
  ( diffWellCont diff m1 ("cont" 0.09 0.09) (1 1 (0.1 0.1))
    (0.05 0.05) (0.005 0.04) (0.0 0.0) (0.0 0.0) (0.0 0.0)
    nplus (0.02 0.02) nwell (0.275 0.24)
  )
  ...
);standardViaDefs

```

This example outputs the following standard via on the poly, metal 1, and contact layers:



In this example, the `polyCont` is a standard via that is defined on `m1/poly/cont` layers with a contact dimension of `0.09` and a spacing of `0.1`. The poly enclosure over the cont in the X direction is `0.05` and in the Y direction it is `0.07`, the `m1` extension over the cont in the X direction is `0.04` and in the Y direction it is `0.005`.

standardViaVariants

`standardViaVariant()` defines the variant of the standard via and must be used with `standardViaDefs()`. The designer can choose the standard via variant when using the **Create > Vias** command in the Layout Editor.

Syntax:

```
standardViaVariants (
    (viaVariantName standardViaDefName
    [-cutLayer <string>]
    [-cutWidth <float>]
    [-cutHeight <float>]
    [-cutRows <int>]
    [-cutColumns <int>]
    [-cutSpacing (<float><float>)]
    [-layer1Enc (<float><float>)]
    [-layer2Enc (<float><float>)]
    [-layer1Offset (<float><float>)]
    [-layer2Offset (<float><float>)]
    [-originOffset (<float><float>)]
    [-implant1Enc (<float><float>)]
    [-implant2Enc (<float><float>)])
    ...
);standardViaVariants
```

Note:

Any line preceded by a semicolon “;” is treated as a comment.

Argument	Description
viaVariantName	The name of the standard via variant.
standardViaDefName	The name of the standard via on which the variant is based.
-cutLayer	The layer on which the via is placed.
-cutWidth	The width of the cut layer.
-cutHeight	The height of the cut layer.
-cutRows	The number of cut rows.
-cutColumns	The number of cut columns.
-cutSpacing	The space between the cuts. This is specified as a list. The first value is the spacing in the horizontal direction and the second value is the spacing in the vertical direction.
-layer1Enc	The enclosure of layer1 above the cut layer.
-layer2Enc	The enclosure of layer2 below the cut layer.
-layer1Offset	The offset of layer1.
-layer2Offset	The offset of layer2.
-originOffset	The offset of the origin.
-implant1Enc	The enclosure of implant1 over the cut layer.
-implant2Enc	The enclosure of implant2 over the cut layer.

Example 46 *standardViaVariants* section

```
standardViaVariants(
  (myStdViaVar_1 stdViaDef_1 -cutLayer VIA1 -cutWidth 0.02 -cutHeight 0.03
    -cutRows 2 -cutColumns 3 -cutSpacing (0.05 0.05) -layer1Enc (0.14 -0.14)
    -layer2Enc(-0.12 0.12)
  )
  (myStdViaVar_2 stdViaDef_2 -cutLayer VIA2 -cutWidth 0.01 -cutHeight 0.04
    -cutRows 3 -cutColumns 4 -cutSpacing (0.05 0.05) -layer1Enc (0.14 0.14)
    -layer2Enc(0 0) -layer1Offset (0.12 0.12) -layer2Offset (0.03 0.03)
    -originOffset(0 0) -implant1Enc (-0.01 0.02) -implant2Enc ( 0 0 )
  )
);standardViaVariants
```

customViaDefs

The `customViaDefs` section defines custom vias that are defined as separate library cells. The custom via is similar to a PCell in that you can change the various parameters to modify the via layout. The default values for the parameters are based on the rules that are entered in the technology file. The definition calls out the library, cell, and view name for each via, and assigns each a via definition name.

Syntax:

```
customViaDefs (  
  (<string>viaDefName  
   <string>libName  
   <string>cellName  
   <string>viewName  
   <string>layer1  
   <string> layer2  
   <float>resistancePerCut)  
  ...  
);customViaDefs
```

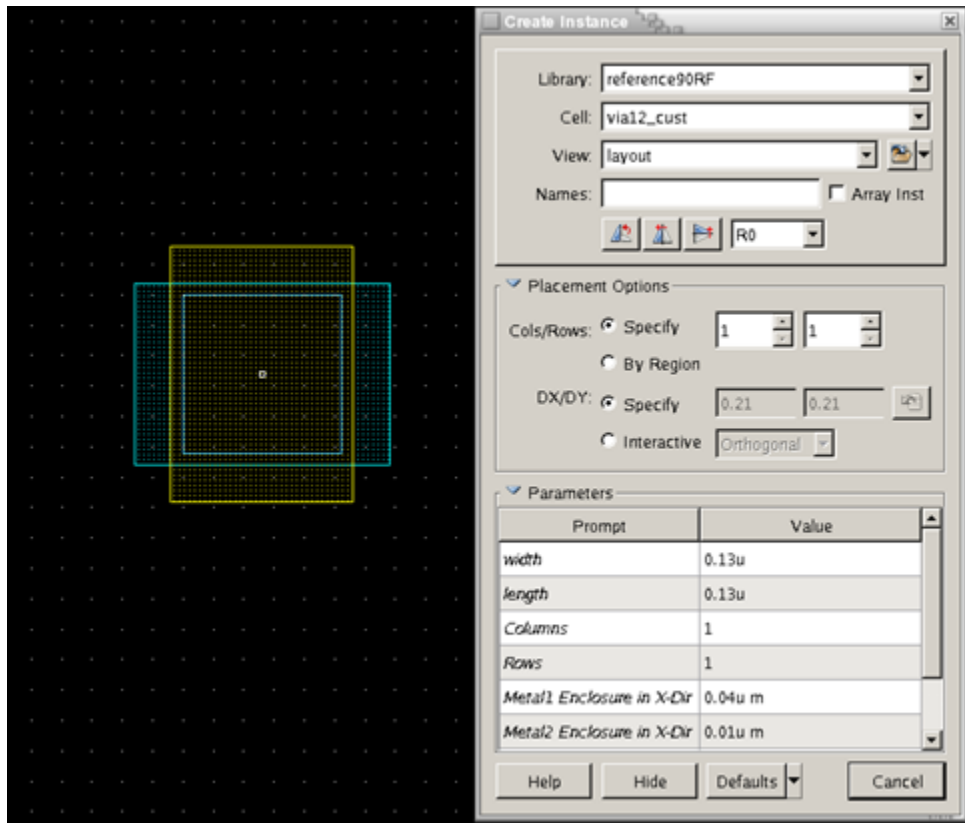
Note:

Any line preceded by a semicolon “;” is treated as a comment.

Argument	Description
viaDefName	The name of the custom via.
libName	The library that contains the via.
cellName	The cell name of the custom via.
viewName	The view name of the cell.
layer1	The name of layer1.
layer2	The name of layer2.
resistancePerCut	The resistance of a single cut.

Example 47 *Sample customViaDefs section*

```
customViaDefs (  
  ( via12_cust reference90RF via12_cust layout m1 m2 1.0) ( via23_cust  
  reference90RF via23_cust layout m2 m3 1.0) ( via34_cust reference90RF  
  via34_cust layout m3 m4 1.0)  
  ...  
);customViaDefs
```



In this example, `via12_cust` is a custom via in the `reference90RF` library and is between layers `m1` and `m2` with resistance per cut being `1.0`. It is similar to a PCell where you can change the properties of the device and re-generate shapes for the device based on the properties. The Create Instance form in the figure shows the parameters that can be set when this via is placed.

customViaVariants

`customViaDefs` defines the variant of the custom via and must be used with `customViaDefs()`. The designer can choose the custom via variant when using the **Create > Vias** command in the Layout Editor.

Syntax:

```
customViaVariants (
    (viaVariantName <string>
    customViaDefName <string>
    (paramArray <string int|float|bool|string>))
);customViaDefs
```

Note:

Any line preceded by a semicolon “;” is treated as a comment.

Argument	Description
viaVariantName	The name of the custom via variant.
customViaDefName	The name of the <code>customViaDefs</code> on which the variant is based.
paramArray	The list of parameters associated with the custom via.

Example 48 *Sample customViaVariants section*

```
customViaVariants(
  (myCusViaVar_1 cusViaDef_1
    ( (param1 3)
      (param2 0.12)
      (param3 "customViaVariant1")
      (param4 true)
    )
    (myCusViaVar_2 cusViaDef_2
      ( (myParam 0.05))
    )
  )
);customViaVariants
```

bridgeVias

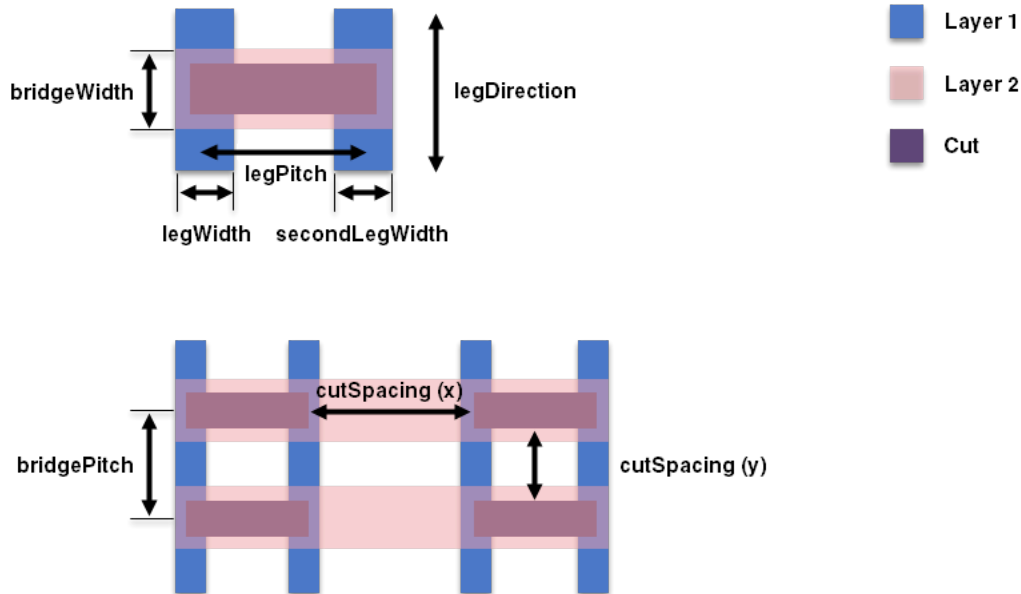
Instead of using two vias and a rectangle to bridge and connect two wires on adjacent tracks, you can use `bridgeVias` to define and create a single via for the same purpose.

To create the bridging via, you can use one of two methods:

- **Standard Via:** You can configure a standard via to have the correct cut size and enclosure. However, the via would have only a single point of contact to the connecting layer.
- **Custom Via:** You can use a custom via and a PCell master to programmatically create the correct vias. This method permits multiple shapes on the top and bottom layers.

Because custom vias are preconfigured to a specific context, you need to use `bridgeVias` to dynamically pass contextual data between the PCell and the custom via to build the correct geometries. Within `bridgeVias`, you can define a group of vias (standard or custom) that can be used for bridging.

The following figure shows the required data (arguments) that are part of the bridging context. Optional arguments are available to further fine-tune the specifications.



Having `bridgeVia` definitions available in the technology database allows designers to choose the desired bridge via when using the **Create Interconnect** command or when setting up **Auto Via** specifications. For more information, see the *Custom Compiler Layout Editor User Guide*.

Syntax:

```
bridgeVias (
; (viaDefName | viaVariantName
  -legWidth <float>
  [-secondLegWidth <float>]
  -legPitch <float>
  -legDirection <vertical | horizontal>
  -bridgeWidth <float | rangeValue>
  [-bridgeWidthParamName <string>]
  [-bridgePitch <float>]
  [-bridgePitchParamName <string>]
  [-cutSpacing <float> <float>]
  [-cutRowsParamName <string>]
  [-cutColumnsParamName <string>]
  [-comment ["rule_id"] "string"])
);bridgeVias
```

Note:

Any line preceded by a semicolon “;” is treated as a comment.

Argument	Description
viaDefName viaVariantName	Name of the standard via, standard via variant, custom via, or custom via variant. The via definitions must be defined in standardViaDefs , standardViaVariants , standardViaVariants , and standardViaVariants .
-legWidth	Width of the layer 1 leg.
-secondLegWidth	(Optional) Width of the second leg if it is different from <code>legWidth</code> .
-legPitch	Center-to-center distance between two adjacent legs.
-legDirection	Direction of the wires that are being bridged.
-bridgeWidth	Width of the layer 2 bridging shape. You can supply a single value or a range of values. See Range Data Syntax .
-bridgeWidthParamName	Name of the custom via PCell parameter that you want to use to determine the bridge width.
-bridgePitch	(Optional) Center-to-center distance between two bridging shapes.
-bridgePitchParamName	Name of the custom via PCell parameter that you want to use to determine the bridge pitch.
-cutSpacing	(Optional) X and Y distances between cut shapes in an array of bridges constructed by a single custom via.
-cutRowsParamName	(Optional) Name of the custom via PCell parameter that specifies the number of cut rows.
-cutColumnsParamName	(Optional) Name of the custom via PCell parameter that specifies the number of cut columns.
-comment	(Optional) Constraint description. The first optional string specifies the rule ID, the second string describes the constraint.

Example 49 *Example bridgeVias section using a range of bridge width values*

```
bridgeVias (
  (myCusViaVar -legWidth 0.7 -legPitch 0.6 -legDirection horizontal
    -bridgeWidth ((>=0.1 <=0.2) (0.5 0.6) (0.7 <=0.9)) )
) ;bridgeVias
```

Example 50 *Sample bridgeVias section*

```
bridgeVias (
  (myCusViaDef -legWidth 0.5 -legPitch 0.4 -legDirection vertical -bridgeWidth 0.5
    -bridgePitch 0.3 -cutSpacing (0.7 0.4) -cutRowsParamName cutRow2
    -cutColumnsParamName cutCol4 -comment "s.21" "This is my bridge via
    constraint")
  (myCusViaVar -legWidth 0.7 -legPitch 0.6 -legDirection horizontal
    -bridgeWidth 0.8)
) ;bridgeVias
```

Constraint Groups Section

This topic describes the following:

- [constraintGroups\(\)](#)
- [foundry](#)
- [drcRuleDefs](#)

constraintGroups()

Constraint groups are global design rules or *constraints* that define the physical interaction between the layer-purpose pairs (LPPs) in this technology. You can specify a constraint group with a set of constraints and constraint group references. Each constraint group can be nested hierarchically inside other constraint groups.

Note:

Although constraint groups are stored in the technology database, you can use the Constraint Manager to view and manage them.

Syntax:

```
constraintGroups (
  (<string>groupName [-operator and|or|precedence] [-alternate
  true|false] [-override true|false] [-lppSpec (layerName1| (layerName1
  PurposeName1) [AND|NOT] [layerName2| (layerName2 PurposeName2) ] )])
  (<string>constraintName constraintArguments
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
);constraintName
);groupname
);constraintGroups
```

Note:

Any line preceded by a semicolon “;” is treated as a comment.

Argument	Description
groupName	Name of the constraint group.

Argument	Description
<code>-operator</code>	Determines how the constraint group is applied to an object: • and: All constraints in the constraint group are applied to associated objects. Other constraint groups may not be nested in the AND constraint group. • or: At least one of the constraints in the constraint group is applied to associated objects. If a hard constraint cannot be met, other constraints will be considered. Other constraint groups may not be included in the OR constraint group. • precedence: The constraints are searched in order, and only one constraint will be applied. The following search rules are used: When the search finds the first hard constraint, the constraint is applied, and the search stops. When the search finds the first soft constraint, it is applied, if possible. If it cannot be applied, the search continues and evaluates the next constraint.
<code>-alternate</code>	If true, the constraint group belongs to the alternate foundry.
<code>-override</code>	The constraints in the constraint group will override stricter constraints of the same type that are lower in the hierarchical search order. If you do not want the constraint group to override other constraints, set the argument to <code>false</code>. This argument is defined in the OpenAccess database, but it is not used by the Custom Compiler environment.
<code>-lppSpec</code>	Only allow two layers. See Region Based Rules .
<code>-isHard</code>	See Common Options .
<code>-comment</code>	See Common Options .

In the following example, two constraint groups are defined: `foundry` and `leCEConstraintGroup`.

```
constraintGroups (
  (foundry
    ( minWidth M1 0.1 )
    ( minSpacing M1 0.2 )
    ( minArea M1 0.04 )
  )
  (leCEConstraintGroup
    ( validRoutingLayers ( M1 M2 M3 M4 M5 ) )
  )
)
```

foundry

The `foundry` constraint group defines the following design rules:

- **Physical rules:** Specifies design rules that is used in the design. For example, the foundry constraint group might define the min/max widths for layers, spacing between layers, width dependent spacings, or via stack limits.
- **Electrical rules:** Specifies electrical characteristics of the layers. For example, capacitance, density, resistance, and area capacitance.
- **Interconnect rules:** Specifies the valid routing layers in the technology.

See [Constraints Overview](#), for details about the syntax.

Syntax:

```
(group    [operator]
  Constraint 1
  Constraint 2
  ...
); foundry constraint group
```

Note:

Any line preceded by a semicolon “;” is treated as a comment.

Example 51 Sample foundry constraint groups section

```
constraintGroups(
  ( foundry nil
    ( minWidth "nwell" 0.55 -comment "Rule 7.1"
      "Rule 7.1: minWidth nwell 0.55um")      ( minSpacing m1 0.09 (((0.1
0.12)0.11)((0.12 0.14) 0.14)) -comment
      "Rule 7.2" "Rule 7.2: M1-M1 Spacing 0.09)
    ...
  );foundry
);constraintGroups
```

For an example technology file, see [Sample Technology File](#).

drcRuleDefs

The `drcRuleDefs` section is used to define a custom rule definition that associates with an array of layers whose order is defined by the technology file syntax. A rule definition is a sequence of generic, well-defined, DRC operators that model the new process rule.

Each operator is stored as a separate string and can contain references to layers and/or parameter values

You can reference layer using the string `$layer<int>`, where `<int>` is an integer starting at 1. The integer number corresponds to the index into the layer array, which starts at 1

(the first element of the array is at index 1). For example, `$layer2` references the second layer in the layer array from the specific constraint. The same is true for parameters in which an array of parameter values is allowed to be stored on the `drcRule` constraint. Parameters are referenced using the string `$value<int>` in the same way as layers.

Note:

The `drcRuleDef` parsing and processing is limited to first order checking of each `layerOp` string for correctness. If no `drcRuleDefs` are defined in the technology, TechIO does not export any references to this section.

Syntax:

```
drcRuleDefs (
    ; (ruleDefName      (operators))
    (<string>ruleDefName (<string>operators))
);drcRuleDefs
```

Note:

Any line preceded by a semicolon “;” is treated as a comment.

Argument	Description
<code>ruleDefName</code>	Name of the DRC rule definition.
<code>operators</code>	Array of values to be applied to the operator expressions defined in the rule definition.

Note:

Some important points:

- For each value, the Custom Compiler tool neither interprets nor converts to/from DBU.
- Each string value must be enclosed in double quotes “”.
- `layerName` more than one must be enclosed in parenthesis ().

Example 52 Sample `drcRuleDefs` section

```
drcRuleDefs(
    ;(ruleDefName (operators))
    (mycheck ((grow -in $layer1 -rightBy $value1 -out g1)
              (inside -in {g1 $layer2} -out t1)
              (internal -in t1 -value $value2 -out error))
    )
    (otherCheck ( ... ) )
    (nextCheck ( ... ) )
)
```

For information on specifying design rule checking definitions, see [drcRule Definition](#).

For an example technology file, see [Sample Technology File](#).

Region Based Rules

This section introduces the following attributes:

- [lppSpec](#)
- [constraintGroup](#)

lppSpec

To support Region Based Rules, associating a constraint group with a region by the `-lppSpec` attribute can include layer/LPP names and operators (INSIDE|NOTINSIDE|INTERACT|NOTINTERACT).

- inside: fully inside
- interact: fully inside, partially overlap or edge touch
- not inside: partially overlap, touch (edge or point) or outside (no overlap, no touch)
- not interact: outside (no overlap, no touch) or point touch

The `-lppSpec` syntax should always consist of keyword and layer/lpp pairs.

The general syntax is:

```
lppSpec    ([INSIDE / NOTINSIDE/ INTERACT/ NOTINTERACT]
            [-involvedObjects any | all]
            [-pointTouchAsInteract true | false]
            )
```

Argument	Description
<code>lppSpec</code> 2 options: • <code>pointTouchAsInteract</code> • <code>involvedObjects</code>	Specifies a constraint group with a region.
<code>layerName1</code>	Specifies one layer for which you want to define the region.
<code>INSIDE NOTINSIDE INTERACT NOTINTERACT</code>	Specifies the operation to define the region with specified layers.
<code>layerName2</code>	Specifies another layer for which you want to define the region.

The following are the several combinations of layer/LPP names and operators:

- INTERACT Layer name: interact with this layer
- NOTINTERACT Layer name: not interact with this layer
- Layer1 name INTERACT Layer2 name: inside layer1, interact with layer2
- Layer1 name NOTINTERACT Layer2 name: inside layer1, not interact with layer2
- INTERACT Layer1 name NOT Layer2 name: interact with layer1, not inside layer2
- NOTINTERACT Layer1 name NOT Layer2 name: not interact with layer1, not inside layer2
- NOT Layer1 name INTERACT Layer2 name: not inside layer1, interact with layer2
- NOT Layer1 name NOTINTERACT Layer2 name: not inside layer1, not interact with layer2
- INTERACT Layer1 name INTERACT Layer2 name: interact with layer1, interact with layer2
- INTERACT Layer1 name NOTINTERACT Layer2 name: interact with layer1, not interact with layer2
- NOTINTERACT Layer1 name INTERACT Layer2 name: not interact with layer1, interact with layer2
- NOTINTERACT Layer1 name NOTINTERACT Layer2 name: not interact with layer1, not interact with layer2

Examples

```
CG_N_R1_I_R2 lppSpec (INSIDE regionID1 INTERACT regionID2 NOTINSIDE
regionID3 NOTINTERACT regionID4)
    -involvedObjects all
    -pointTouchAsInteract False)
```

The Unified Constraint Manager (UCM) maintains a list of layers/LPPs (`-lppSpec`) that defines the region-based rule constraints. Shapes interactive/inside the region-based LPP shape should apply the corresponding constraints rather than active constraint group.

The applied rule value is decided by the UCM, which is based on region-based constraint group. Applications adopt region-based LPP shapes based on stop level.

Only the following features support region-based rule constraints:

- Create interconnect/Bus
- Create via
- Routers

constraintGroup

Support the `-constraintGroup` attribute on constraints for specifying a constraint group name. If a constraint needs to be shared among multiple constraint groups, set the constraint into a constraint group which is included by the others.

Tech Import adds each individual constraints into corresponding constraint groups.

Tech Export pulls the constraints out of constraint groups and puts them back to their original locations in the tech file.

The general syntax is:

```
ConstraintName <string|LPP>layer
                    <float|bool>value
                    [-constraintGroup <string>]
                    [-isHard true|false]
                    [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or a LPP.
value	Simple layer constraints take a single value or a preferred value. Complex layer constraint can take a range of values.
-constraintGroup	Specifies constraint group name.

Examples

```
(foundry
 (constraintGroup CG_finBoundary_not_PRBoundary)
 (constraintGroup CG_finBoundary)
 (constraintGroup minExtensionM1VIA12)
 (constraintGroup minViaSpacingVIA12)
 (minWidth M1 0.1)
 (minWidth M2 0.12)
 (minWidth M1 0.12 -constraintGroup CG_finBoundary)
 (minWidth M2 0.14 -constraintGroup CG_finBoundary)
);
(minExtensionM1VIA12 -operator or
 (minDualExtension M1 VIA12 0.01 0.015)
```



```
(minDualExtension M1 VIA12 0.012 0.014)
(minDualExtension M1 VIA12 0.08 0.015
  -constraintGroup minExtensionM1VIA12_finBoundary)
);
(minViaSpacingVIA12
  (minViaSpacing VIA12 0.02 -parallelRunLength 0.01)
  (minViaSpacing VIA12 0.03)
  (minViaSpacing VIA12 0.04
    -constraintGroup minViaSpacingVIA12_finBoundary_NOT_PRBoundary)
);
//Define the structure of region-based constraint groups, no individual
constraints needed
(minExtensionM1VIA12_finBoundary -operator or);
(minViaSpacingVIA12_finBoundary_NOT_PRBoundary);

(CG_finBoundary -lppSpec (finBoundary)
  (constraintGroup minExtensionM1M2_finBoundary)
);
(CG_finBoundary_not_PRBoundary -lppSpec (finBoundary NOT PRBoundary)
  (constraintGroup minViaSpacingVIA12_finBoundary_NOT_PRBoundary)
);
```

Site Definitions Section

Site definition defines how a standard cell must be placed in the layout. A site has a type, symmetry, and other information allowing for the specification of placeable objects within a row. The site can be either a simple scalarSiteDef, which has width and height information (in microns), or an arraySiteDef, which has a pattern of scalarSiteDefs.

Syntax:

```
siteDefs (
  scalarSiteDefs (
    (<string>siteDefName
      <string>siteDefType
      <float>width
      <float>height
      <bool>symInX
      <bool>symInY
      <bool>symInR90)
  ); scalarSiteDefs
  arraySiteDefs (
    (<string>siteDefName
      <string>siteDefType (
        (<string>siteName
          <float>dx
          <float>dy R0|R90|R180|R270|My|MYR90|MX|MXR90) *)
      [<bool>symInX]
      [<bool>symInY]
      [<bool>symInR90])
  ); arraySiteDefs
);
```

```
);arraySiteDefs  
);siteDefs
```

Note:

Any line preceded by a semicolon “;” is treated as a comment.

Argument	Description
siteDefName	The site definition name.
siteDefType	The type of site definition. Applicable values are pad core.
width	The width of the site definition.
height	The height of the site definition.
symInX	Is the site symmetrical to X axis. Applicable values are true false. Default is <i>false</i> .
symInY	Is the site symmetrical to Y axis. Applicable values are true false. Default is false.
symInR90	Is the site symmetrical at 90 degrees. Applicable values are true false. Default is <i>false</i> .
siteName	The name of the site definition.
dx	The delta X offset.
dy	The delta X offset.
orientation	The orientation type. Applicable values are: • R0: Represents no change in orientation. • R90: Represents a 90 degree rotation. • R180: Represents a 180 degree rotation. • R270: Represents a 270 degree rotation. • MY: Represents mirroring about the Y axis. • MYR90: Represents mirroring about the Y axis then a 90 degree rotation. • MX: Represents mirroring about the X axis. • MXR90: Represents mirroring about the X axis then a 90 degree rotation.

Example 53 Sample siteDefs section

```
siteDefs(  
  scalarSiteDefs(  
    (AUT core 100 200 true true false)  
    (BUT core 40 60 true false false)  
  );scalarSiteDefs  
  arraySiteDefs(  
    (BUF pad ((buf1 2.4 1.2 R90 )  
              (buf2 1.4 1.8 R180)  
              (buf3 0.8 1.0 MX )) false false true)  
  )  
)
```

```
);arraySiteDefs  
);siteDefs
```

3

Custom Compiler Technology Data and OpenAccess

Describes how the technology data used by the Custom Compiler tool relates to standard OpenAccess technology data.

This section covers the following topics:

- [How Custom Compiler Technology Data is Represented in OpenAccess](#)
- [Attributes or Properties on OpenAccess Objects](#)

How Custom Compiler Technology Data is Represented in OpenAccess

The Custom Compiler tool uses standard OpenAccess constructs as much as possible in its technology file. It uses OpenAccess extensions for all unique Custom Compiler needs. It accepts third-party extensions, and simply ignores them if they are not used in the Custom Compiler tool. You should not require any changes in your current OpenAccess technology file to use it with the Custom Compiler tool.

Each technology parameter contained in the controls section of the technology file is stored as a property (`oaProp`) attached to a binary technology file (an `oaTech` object). The `oaProp` objects are owned by `oaGroup` objects which in turn are owned by `oaTech`.

The view type units are stored directly in the technology file. Standard OpenAccess Tcl procedures can be used to access that information from Tcl scripts.

Both Custom Compiler Tcl commands and C++ parameterized cells (PCells) access technology data using the standard OpenAccess API. OpenAccess extensions are added only for data types that are needed by C++ client modules. All information needed by PCells is stored in generic OpenAccess data structures.

Attributes or Properties on OpenAccess Objects

This topic describes data in the technology file that are stored as attributes or properties on objects in OpenAccess, and explains how you can get the values for these objects from OpenAccess. For more details about OpenAccess objects, their attributes, and OpenAccess C++ and Tcl functions, see the OpenAccess documentation included as a tarball in your Custom Compiler installation.

The values of the following objects are accessible through OpenAccess Tcl commands.

Note:

The `techParams` section of the technology file might contain arbitrary name-value pairs, with the only restriction that the names must be unique. `techParams` is the name of an `oaGroup` which contains a collection of `oaProp` objects. The pairs are stored as a property of the `oaTech` object.

Following are the commonly used Tcl commands to access OpenAccess objects:

- [Getting the Technology Object from a Library or a Design](#)
- [Getting the Manufacturing Grid and Other Parameters Information](#)
- [Getting the Layer Number and Layer Name Information](#)
- [Getting the Required Layer Constraints](#)
- [Getting the Required Layer Pair Constraints](#)
- [Getting the Via Names and Via Definitions](#)

Getting the Technology Object from a Library or a Design

The `set oaTech [...]` command returns the technology ID. For example,

```
set oaTech [oa::TechFind <library Name>]
```

Getting the Manufacturing Grid and Other Parameters Information

The `set group [...]` command returns the properties in the `techParams` section of the technology file. For example,

```
set group [oa::getNext [oa::getGroupsByName $oaTech techParams]]
```

The `set cadGrid [...]` command returns the `cadGrid` value defined in the technology file. For example,

```
set cadGrid [oa::PropFind $group cadGrid]
```

Note:

The `cadGrid` OA property might or might not reside in an `oaGroup techParams` section in the technology file as it is not a built-in attribute in the technology file.

The `set mfgGrid [...]` command returns the manufacturing grid value defined in the technology file. For example,

```
set mfgGrid [oa::PropFind $group mfgGrid]
```

The `set dbu [...]` command returns the database units for performing mathematical conversions. For example,

```
set dbu [oa::TechGetDefaultDBUPerUU [oa::ViewTypeGet maskLayout]]
```

Getting the Layer Number and Layer Name Information

```
proc getLayerNumber {techId layerName}{  
  return [oa::getNumber [oa::LayerFind $techId $layerName]]  
}
```

Use the following command to get the layer number of a specific layer. For example:

```
set layerNum [getLayerNumber ...]
```

Getting the Required Layer Constraints

```
proc getLayerConstraint{const Layer tf}{  
  set fc [oa::getFoundryRules $tf]  
  set layerNo [oa::getNumber [oa::LayerFind $tf $Layer]]  
  set c_ref [oa::LayerConstraintFind $fc $layerNo  
    [oa::LayerConstraintDefGet  
    [oa::LayerConstraintType $const]]]  
  set c_val [oa::get [oa::getValue $c_ref]]  
  set dbu [oa::TechGetDefaultDBUPerUU  
    [oa::ViewTypeGet maskLayout]]  
  if {$const=="minArea"||$const=="minEnclosedArea"}{  
    set value [expr ($c_val/double($dbu*$dbu))]  
  }else{  
    set value [expr ($c_val/double($dbu))]  
  }  
  return $value  
}
```

Use the following command to get a layer constraint of a specific layer. For example:

```
set tech(polyMinWidth) [getLayerConstraint minWidth  
    $layer(poly)  
    $oaTech]
```

Getting the Required Layer Pair Constraints

```
proc getLayerPairConstraint(const layer1 layer2 tf){
  set fc      [oa::getFoundryRules $tf]
  set c_def   [oa::ConstraintDefFind $const]
  set layer1  [oa::getNumber
               [oa::LayerFind $tf $layer1]]
  set layer2  [oa::getNumber
               [oa::LayerFind $tf $layer2]]
  set c_ref   [oa::LayerPairConstraintFind $fc $layer1
               $layer2 $c_def]
  set c_val   [oa::get [oa::getValue $c_ref]]
  set dbu     [oa::TechGetDefaultDBUPerUU
               [oa::ViewTypeGet maskLayout]]
  if {$const=="minEnclosure" || $const=="endOfLineEnclosure"}{
    set value [expr round($c_val*$dbu)/double($dbu*$dbu)]
  }elseif {$const=="minOverlap"}{
    set value [expr ($c_val/double($dbu))]
  }else {
    set value [expr ($c_val/double($dbu))]
  }
  return $value
}
```

Use the following command to get a layer constraint of a specific layer. For example:

```
set tech(polyContEnclose) [getLayerPairConstraint
                           minEnclosure
                           $layer(poly)
                           $layer(cont)
                           $oaTech]
```

Getting the Via Names and Via Definitions

```
set viaNames {}
db::foreach viaDef [oa::getViaDefs $techId] {
  lappend viaNames [db::getattr name -of $viaDef]
}
puts $viaNames
```

4

Constraints Overview

Describes how constraints are used to define design rules and provides the syntax with examples for all the supported constraints.

Design rules in the Custom Compiler tool – and in OpenAccess – are expressed as *constraints* to which your designs should comply. The constraints are either global (process-level) or design-specific. Process constraints are defined in the technology database. Design-specific constraints are created and managed in the Constraint Editor.

You can define two types of process constraints:

- **Hard constraint:** To check the Design Rule Checking (DRC).
- **Soft constraint:** To be used for Design for Manufacturing (DFM) purposes.

You can define process constraints in groups, and create hierarchical groups of constraints with a child group adding more constraints to those defined by its parent.

The Custom Compiler environment supports the constraints for most of the design rules defined for 28 nm and above process nodes.

Although the constraints are stored in the technology database, you can manage constraints and constraint groups using the Constraint Manager. See the *Custom Compiler Environment User Guide* for instructions.

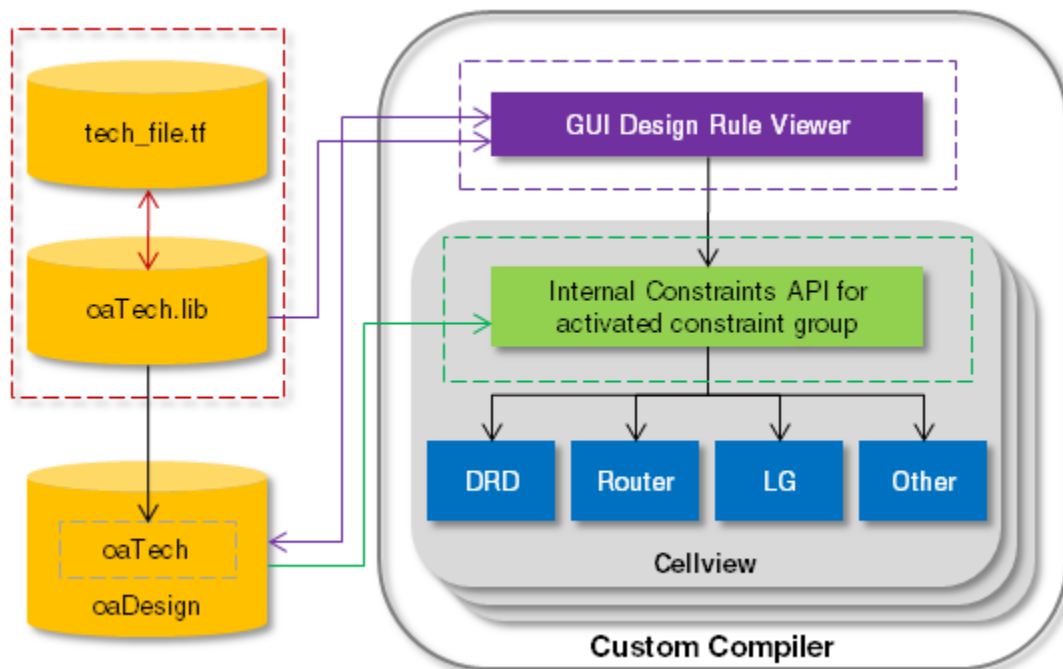
This topic describes process constraints and covers the following:

- [How the Custom Compiler Components Interact with Constraints](#)
- [Interactions with the Technology File](#)
- [Changing Rules for an Existing Library](#)
- [Understanding and Interpreting 1-D, 2-D, and 3-D Table Values](#)
- [Range Data Syntax](#)
- [cutClass and cutClassList](#)
- [Boolean Values](#)
- [Common Options](#)

- [Constraint Types](#)
- [Voltage Dependent Design Rules](#)
- [Forbidden End-to-End Rules](#)

How the Custom Compiler Components Interact with Constraints

The following figure shows how the Custom Compiler environment interacts with design rule constraints.



The technology file for a design is part of the OpenAccess technology data for the library (`oaTech.lib`). The interactive Constraint Manager tool reads in data from the `oaTech.lib` and passes this information to an internal API for handling constraints. The internal API also reads in constraint data from the design database. The Custom Compiler tool uses this as you perform design rule checks from within the Custom Compiler interface.

A heuristic search is performed on the constraint groups to determine the effective constraint for a specific constraint specification.

Following is the list of supported OA container objects in order of precedence from highest to lowest:

- `oaTech`
- `oaDesign`

- oaGroup
- oaNet
- oaRoute
- oaFigGroup
- oaBoundary

When you export technology data, the Custom Compiler tool exports OpenAccess design constraints into the ASCII technology file.

When you import technology data, the Custom Compiler tool parses and validates the design rules and converts those rules into constraints in the OpenAccess database. If there are any errors during the import, the Custom Compiler tool reports them in the Console window.

Interactions with the Technology File

By default, the Custom Compiler Connectivity Engine uses the `leCEConstraintGroups` section in the technology file when propagating signals throughout a layout design. The constraint group must contain the [validRoutingLayers](#) and [validRoutingVias](#) constraints.

If you want the Connectivity Engine to use a different constraint group during signal propagation, set the `leCEConstraintGroups` Tcl preference to specify the constraint group. In addition, you can set the `leNonPropagatingLayerMaterials` Tcl preference to specify the layer material type, which controls signal propagation.

For example:

```
constraintGroups(  
    (foundry nil  
      ...  
    )  
    (leCEConstraintGroup  
      (validRoutingLayers( v9 v8 v7 v6 v5 v4 v3 v2 v1  
                           m10 m9 m8 m7 m6 m5 m4 m3 m2  
                           m1 cont poly))  
    )  
)
```

Changing Rules for an Existing Library

If you change the technology data for a library so that the design rules differ from the original technology data, this change does not affect placed PCells immediately. Design rules are checked only during creation or modification of objects. Any violations inside the PCell are not noted until you make an edit to the PCell.

Understanding and Interpreting 1-D, 2-D, and 3-D Table Values

This topic describes the following:

- [1-D and 2-D Table Values](#)
- [3-D Table Values](#)

1-D and 2-D Table Values

When defining a constraint using 1-D or 2-D table values, you must specify a default value or the default value is set to 0. When specifying a 2-D table value, you must provide the list of values in a tabular format, otherwise, a warning message including the information of missing entry will be reported. The missing entry will be set to default value for all types of constraints excluding `oaIntRangeArray2DTblValueType` (`allowedSpacingRange`, `allowedParallelSpanSpacing`, `allowedCutSpacing`, `allowedClearanceRange`).

A 1-D or 2-D table can be specified with interpolate and extrapolate types such that you get the values with a specific key. The following table lists the interpolate and extrapolate types:

Interpolation Types	Extrapolation Types
SnapDown	SnapDown
SnapUp	SnapUp
Linear	Linear
InclusiveSnapDown	-
InclusiveSnapUp	-

Note:

If the interpolate and extrapolate types are not defined for a table in the technology file, then the technology importer automatically sets the default types when creating a 1-D and 2-D table values.

Syntax:

```
ruleName <List of Strings|LPPs>Layers
    [<float|int|
        dualValue|rangeArrayValue>defaultValue]
    <1DTable>TableValue
    [-iType <string>interValue]
    [-eLowerType <string>exterValue]
    [-eUpperType <string>exterValue]
```

```
[-isHard true|false]
[-comment [<string rule_id>] <string rule_desc>]
```

Syntax:

```
ruleName <List of Strings|LPPs>Layers
    [<float|int|
        dualValue|rangeArrayValue>defaultValue]
    <2DTable>TableValue
[-iType <string>interValue]
[-eLowerType <string>exterValue]
[-eUpperType <string>exterValue]
[-iColType <string>interValue]
[-iRowType <string>interValue]
[-eRowLowerType <string>exterValue]
[-eRowUpperType <string>exterValue]
[-eColLowerType <string>exterValue]
[-eColUpperType <string>exterValue]
[-isHard true|false]
[-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
Layers	Layers can be defined as a list of strings or a LPPs.
TableValue	• 1DTable syntax: ((key value)*) Where key is the width and value the value of the rule • 2DTable syntax: (((key1 key2) value)*) Where key1 is the width, key2 is the parallel run length, and value is the value of the rule
-iType	Specifies the interpolation type. The valid values are: • SnapDown (Default) • SnapUp • Linear • InclusiveSnapDown • InclusiveSnapUp
-eLowerType	Specifies the lower extrapolation type. The valid values are: • SnapDown • SnapUp (Default) • Linear
-eUpperType	Specifies the upper extrapolation type. The valid values are: • SnapDown (Default) • SnapUp • Linear

Argument	Description
-iColType	Specifies the column interpolation type. The valid values are: • SnapDown (Default) • SnapUp • Linear • InclusiveSnapDown • InclusiveSnapUp
-iRowType	Specifies the row interpolation type. The valid values are: • SnapDown (Default) • SnapUp • Linear • InclusiveSnapDown • InclusiveSnapUp
-eRowLowerType	Specifies the row lower extrapolation type. The valid values are: • SnapDown • SnapUp (Default) • Linear
-eRowUpperType	Specifies the row upper extrapolation type. The valid values are: • SnapDown (Default) • SnapUp • Linear
-eColLowerType	Specifies the column lower extrapolation type. The valid values are: • SnapDown • SnapUp (Default) • Linear
-eColUpperType	Specifies the column upper extrapolation type. The valid values are: • SnapDown (Default) • SnapUp • Linear

This is a 1-D table example with default value 0.1:

Rule	Description	Value (in microns)
M1.SP.1	M1 Spacing	0.1
M1.SP.2	M1 Spacing with M1 width > 0.08	0.11
M1.SP.3	M1 Spacing with M1 width > 0.09	0.12

```
minSpacing M1 0.1 ((0.08 0.11) (0.09 0.12))
```

The following 2-D table example contains the default value 0.1:

Rule	Description	Value (in microns)
M1.SP.1	M1 Spacing	0.1
M1.SP.4	M1 Spacing with M1 width > 0.08 and common run > 0.145	0.13
M1.SP.5	M1 Spacing with M1 width > 0.09 and common run > 0.160	0.14
M1.SP.6	M1 Spacing with M1 width > 0.095 and common run > 0.175	0.15

Width / Common Run	0.145	0.160	0.175
0.08	0.13	0.13	0.13
0.09	0.13	0.14	0.14
0.095	0.13	0.14	0.15

```
minSpacing M1 0.1 ((0.08 0.145) 0.13)
                  ((0.08 0.160) 0.13)
                  ((0.08 0.175) 0.13)
                  ((0.09 0.145) 0.14)
                  ((0.09 0.160) 0.14)
                  ((0.09 0.175) 0.13)
                  ((0.095 0.145) 0.15)
                  ((0.095 0.160) 0.15)
                  ((0.095 0.175) 0.15))
```

3-D Table Values

You can use 2-D tables to represent 3-D tables by setting variant values with the same key values. The syntax format of a 3-D table is the same as a 2-D table syntax format, which is described in [1-D and 2-D Table Values](#).

For 2-D spacing tables, where the `widthLengthTable` parameter is set to `twoWidthParallelRunLengthTable`, a specialized extraction process is used that matches what is defined in the *LEF Reference Manual 5.7*. The 2-D lookup table requirements are as follows:

- The lookup table must have the same number of rows and columns.
- In the lookup table, the rows contain width thresholds. Within each row, the values must be in increasing order.
- In the lookup table, the columns contain length thresholds. Within each column, the values must be in increasing order.

A length threshold might be less than 0, and if so, indicates that the column is not dependent on the parallel run length.

- A width thresholds index table and a length thresholds index table are used to determine the lookup table indexes.

To determine the minimum spacing between these two shapes using a lookup table:

1. Using the width thresholds index table, determine the index of `width1` and `width2` by finding the last index which is less than the given width. These indexes will be called `widthIndex1` and `widthIndex2`.

For example, two shapes have widths 0.1 and 0.25. Using the width thresholds index table, you can determine that the `width1` index is 2 and the `width2` index is 4.

Index	1	2	3	4	5	6
width threshold	0.0	0.08	0.15	0.2	0.5	1.5

2. Using the length thresholds index table, determine the index of `length` by finding the last index which is less than the given length. This index will be called `lengthIndex`.

For example, two shapes have a parallel run length of 0.3. Using the length thresholds index table, you can determine that the `length` index is 3, because the length of 0.3 is not greater than the 0.3 threshold for index 4.

Index	1	2	3	4	5	6
length threshold	-1	0.0	0.0	0.3	0.7	1.5

3. Look up the minimum spacing value in the lookup table as follows:
 - a. Determine the smaller index value between `widthIndex1` and `lengthIndex`. This will be the row number you use in the lookup table.
 - b. Use the smaller index value between `widthIndex2` and `lengthIndex`. This will be the column number you use in the lookup table.

For example, in steps 2 and 3, you determined that:

- `widthIndex1 = 2`
- `widthIndex2 = 4`
- `length = 3`

The row number to use is 2 (the smaller of 2 and 3), and the column number to use is the 3 (the smaller of 4 and 3). So, in the lookup table, look for the value in row 2, column 3. The minimum spacing value is 0.2.

Note that the second (or last) length threshold of zero is used. This is effectively padding so that the width threshold at index 3 (0.15) also applies when parallel run length is between 0.0 and 0.3.

Width	Length					
	-1	0.0	0.0	0.3	0.7	1.5
0	0.15	0.15	0.18	0.25	0.4	0.6
0.08	0.15	0.15	0.2	0.3	0.43	0.6
0.15	0.18	0.2	0.22	0.32	0.48	0.6
0.2	0.25	0.3	0.32	0.44	0.5	0.6
0.7	0.4	0.43	0.48	0.5	0.56	0.6
1.5	0.6	0.6	0.6	0.6	0.6	0.6

Range Data Syntax

Many constraints allow you to specify ranges of values (`rangeValue` or `frangValue`), such as a range of widths. The range of `rangeValue` thresholds are in database units, but the range of `frangValue` keeps original float value. When specifying ranges, use the following syntax:

```
(([lowerRangeType] lowerWidth [upperRangeType] upperWidth)
```

Where:

Argument	Description
<code>lowerRangeType</code>	<code>></code> <code>>=</code> Default: <code>></code>
<code>lowerWidth</code>	float MINUS_INFINITY
<code>upperRangeType</code>	<code><</code> <code><=</code> Default: <code><</code>
<code>upperWidth</code>	float INFINITY

To specify a lower or upper bound use a single value with an operator. For example, a width of (≥ 0.06) means the width cannot exceed 0.06.

Note:

The operator `==` can be used for `rangeValue`. For example, a range of ($=0.2$) means ($\geq 0.2 \leq 0.2$).

The following symbols can be used when specifying ranges of values:

[	means	$\leq$	(less than or equal to)
]	means	$\geq$	(greater than or equal to)
(	means	$<$	(less than)
)	means	$>$	(greater than)

The following is a `minEndOfLineSpacing` constraint example. The range of widths is between 0.1 and 0.2, inclusive. The range of other-end widths is between 0.5 and 0.7 inclusive.

```
minEndOfLineSpacing ml 0.06
    -widthRange ( $\geq 0.1 \leq 0.2$ )
    -otherEndWidthRange ( $\geq 0.5 \leq 0.7$ )
```

cutClass and cutClassList

Some constraints allow you to specify a number of cuts using one parameter: `-cutClassList`. You can provide either a list of paired float values or a list of cutClass names. You cannot combine paired values and names in one list.

For some constraints, when you add `-cutClass`, you can also specify a list of paired float values or a list of cutClass names, similar to `-cutClassList`. (The constraint description explains whether you can list a number of cuts using one constraint parameter, or you have to use multiple `-cutClass` parameters.)

During the technology import process, the list of cut classes will be converted to individual constraints in a constraint group. During the technology export process, the individual cut class constraints that have the same settings (except cut class values) will be grouped into a list.

Syntax:

```
-cutClass <cutClassArray | cutClassNameList>
-cutClassList <cutClassArray | cutClassNameList>
```

Argument Value	Description
cutClassArray	List of paired float values in microns: (((<float> <float>)<float> <float>)...). Each pair specifies the width and height of the cut. For example: ((0.01 0.02) (0.03 0.05)).
cutClassNameList	List of cut class names: (<string>). When cut class names are listed in the technology file, the technology import and export processes perform the following conversions: Import: The technology import process looks for the first <code>cutClass</code> constraint specified in the same hierarchical constraint group and converts it to the corresponding cut class size. If no <code>cutClass</code> constraint exists, an error message is displayed. Export: The technology export process looks for the first <code>cutClass</code> constraint specified in the same hierarchical constraint group to exists and converts the cut class size to cut class name. If no <code>cutClass</code> constraint exists, the <code>cutClass</code> parameter is exported with cut class size in dual value.

Example:

```
-cutClass ((0.06 0.06) (0.06 0.08))
-cutClass (MyCutClass1 MyCutClass2 MyCutClass3)
-cutClassList ((0.06 0.06) (0.06 0.08))
-cutClassList (MyCutClass1 MyCutClass2 MyCutClass3)
```

Boolean Values

The following table lists the equivalent Boolean values that you can specify when defining constraints:

Boolean Value	Equivalent Values
true	t, 1
false	f, 0, nil

Common Options

You can specify the following options with all constraints, regardless of [Constraint Types](#):

Option	Values	Description
<code>-isHard</code>	<code>true false</code>	If set to <code>true</code> (the default), the design rule is a hard rule. If set to <code>false</code> , the design rule is a recommended rule for DFM.
<code>[-comment]</code>	<code>[<string rule_id>] <string rule_desc>]</code>	A comment giving an ID number for this rule, and a string describing the rule. Only the description string is required. The ID is optional within the optional comment.

The `-comment` option is used by the Custom Compiler environment to report design rule violations while creating a design.

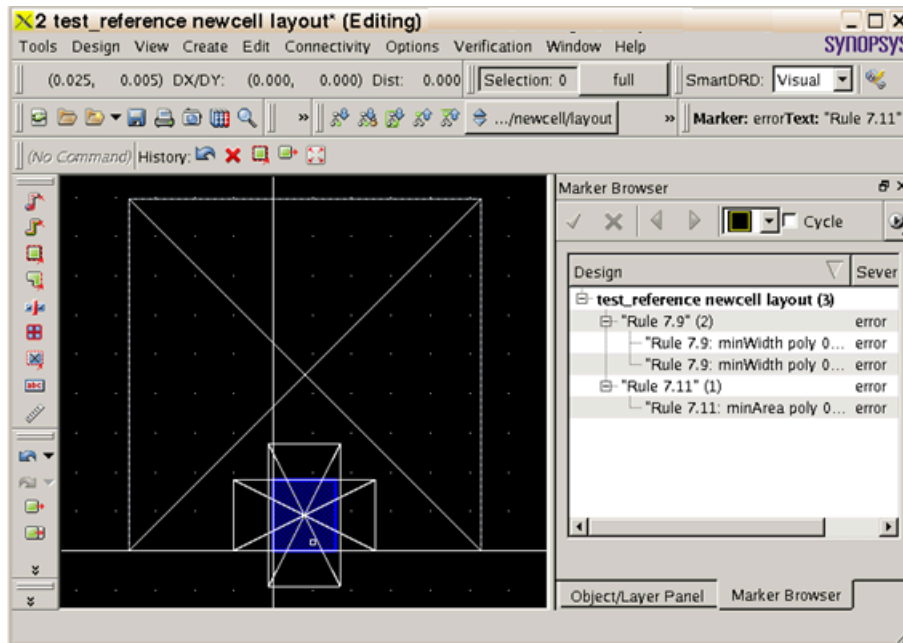
Syntax:

```
ruleName...[-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
<code>[rule_id]</code>	The Custom Compiler DRD engine uses the string specified here as the top-level node in the marker browser tree. The <code>rule_id</code> is displayed in the right top corner of the Custom Compiler Layout Editor window when the marker is the active object as shown in the below example.
<code>[rule_desc]</code>	The Custom Compiler DRD engine uses the string specified here as the detailed description of the error/warning under the top-level node in the error marker tree.

For example,

```
(minWidth poly 0.09 -comment "Rule 7.9" "Rule 7.9: minWidth poly 0.09um")
(minArea poly 0.05 -comment "Rule 7.11" "Rule 7.11: minArea poly 0.05um^2")
```



Constraint Types

The Custom Compiler environment supports the following types of OpenAccess and Custom Compiler constraints:

- [drcRule Definitions](#)
- [Simple Constraints](#)
- [Layer Constraints](#)
- [Layer Pair Constraints](#)
- [Layer Array Constraints](#)
- [DRD Assist Constraints](#)

Caution:

All constraints must be enclosed within parentheses “()” in the technology file.

Voltage Dependent Design Rules

Some design rules apply only to shapes associated with high voltage nets. Usually, the higher the voltage, the more restrictive are the corresponding spacing, clearance or extension rules.

If the `-voltageRange` parameter is specified for a spacing or clearance constraint, the rule only applies if the difference between voltages of the two shapes under consideration lies in this range. The voltage of a shape is determined by the voltage assigned to the net the shape belongs to.

Applying voltage dependent rules requires the nets in the design to be annotated with minimum and maximum voltage values. If one shape belongs to a net (net1) with annotated voltage between `Vmin1` and `Vmax1`, and another shape belongs to another net (net2) with annotated voltage between `Vmin2` and `Vmax2`, the difference between voltages of these two shapes is calculated as $\text{deltaV} = \max(|V_{\text{max1}} - V_{\text{min2}}|, |V_{\text{max2}} - V_{\text{min1}}|)$.

The voltage dependent constraint only applies when net1 and net2 are different nets, and `deltaV` is in the range specified by the `-voltageRange` parameter. If the `-voltageRange` parameter is specified for a constraint applicable only to the shapes from the same net (for example, minimum metal extension over via cut rule), the applicability of the constraint depends on voltage annotation (`Vmin` and `Vmax`) for this single net only. $\max(|V_{\text{max}}|, |V_{\text{min}}|)$ has to lie in the range specified by the `-voltageRange` parameter in order for the rule to apply.

If `minVoltageClearance` constraint is defined between two layers which are not electrically connected when overlapped, the overlap of two shapes from different nets which satisfy the voltage requirement is also considered a violation of the voltage clearance rule.

Forbidden End-to-End Rules

When Auto Via observes that via extensions violate end-to-end spacing rules, it extends the via extensions and then places the vias. Forbidden end-to-end rules allow you to set notifications and enable Auto Via to create markers.

The following preferences are used to indicate notifications for forbidden end-to-end rules checking:

- `leAutoViaNotification`: Shows notifications while checking the rules. The default is `false`.
- `leAutoViaCreateMarkers`: Creates markers for notifications. The default is `false`.

The **Forbidden End-to-End Rules** option is available only when you select the Environment Aware and **Same Wire Layer Spacing** options (both options are `true`). Note that the **Forbidden End-to-End Rules** and **Notification (Forbidden End-to-End Rules)** options are hidden if `allowedEndToEndSpacing` rules are not defined in the technology file.

Mode: ☒ Auto ☐ Manual

Depth: ☒ 0x ☐ View

Nets:

☐ Stacked Bottom: Top:

Options Rules

☒ Environment Aware

☒ Same Cut Layer Spacing ☒ Maximum Vias in Stack

☒ Same Wire Layer Spacing ☒ Voltage Dependent Rules

☒ Forbidden End-to-End Rules

☒ Different Cut Layer Spacing

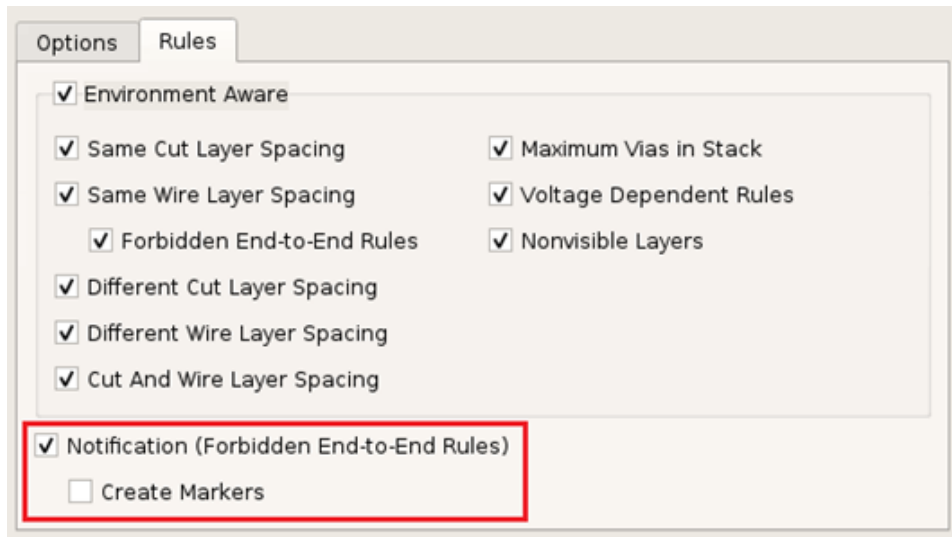
☒ Different Wire Layer Spacing

☒ Cut And Wire Layer Spacing

☒	Layer	Via Definition	Cut Class	Spacing
☒	DIFF	auto	auto	auto
☒	POLY	auto	auto	auto
☒	MODIFF2	auto	auto	auto
☒	M1	auto	auto	auto
☒	M2	auto	auto	auto

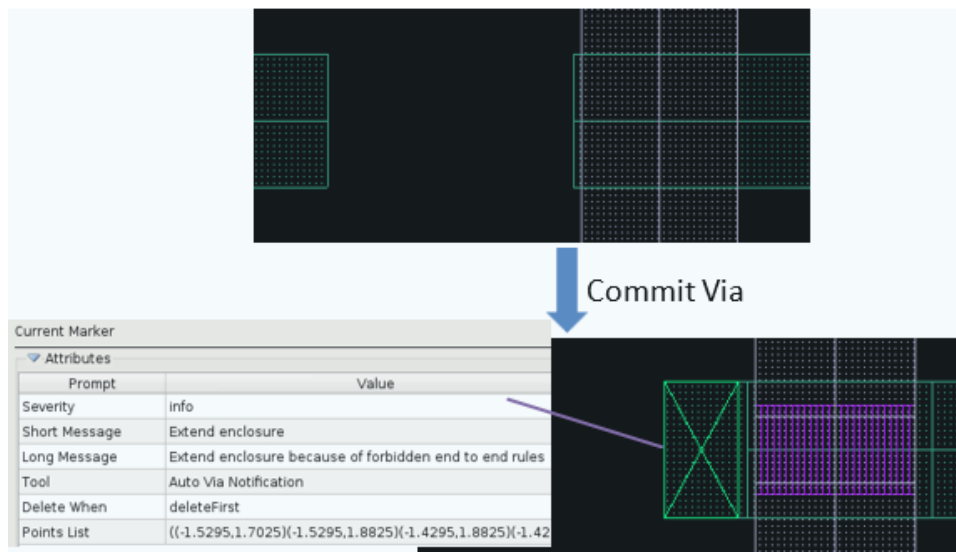
Help Hide Defaults

When you select the **Notification (Forbidden End-to-End Rules)** option, Auto Via shows message on the console while checking forbidden end-to-end rules. The notification indicates that the enclosures of some vias are extended because of forbidden end-to-end rules. Note that only allowedEndToEndSpacing rule is supported.



When you select the **Create Markers** option, Auto Via creates markers and sets the severity of the markers as *Information* on the extended enclosures after commit vias, as the enclosures of the vias are extended due to allowedEndToEndSpacing. The attributes of markers are set as follows after commit vias:

- Short Message: Extend enclosure only.
- Long Message: Extend enclosure because of forbidden end-to-end rules.



5

drcRule Definition

Describes the drcRule constraint and provides the syntax with examples.

The `drcRule` definition is stored in the technology file separately and referenced by name. The `drcRule` is a constraint of type `oaLayerArrayConstraint`. The constraint definition has the number of layers set to 0 so that any number of layers can be used. The parameter name “values” is an array of the `$value<int>` of the `drcRuleDef` that is being referenced.

Constraint Category	Design Rule Checking Definition
Constraint Type	Simple
OA Rule	Custom Compiler Custom Constraint

Syntax:

```
drcRule    <List of strings|LPPs>layers
           ruleDefName
           -values valueArray
           [-isHard true|false]
           [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layers	Specify the list of layers.
ruleDefName	Name of the DRC rule definition.
-valueArray	Array of values to be applied to the operator expressions defined in the rule definition. The <code>valueArray</code> is represented as: ([floatValue]* [intValue]* [string*])

For example:

```
constraintGroups(
  foundry(
    (drcRule (M1 V1) mycheck -values (0.1 0.12) -comment "rule 12" "This is new")
    (drcRule (M2 V2) mycheck -values (0.5 0.15))
    (drcRule M6 otherCheck -values 3.5)
```



```
(drcRule (M1 M2 V2) nextCheck -values (2 0.2 "text string"))  
)
```

6

Simple Constraints

Describes simple constraints and provides the syntax with examples.

A simple constraint does not require a layer definition. It sets rules such as the placement grid, the routing grid, valid routing vias, and so forth.

Simple constraints can take a single value or a preferred value. The general syntax is:

```
ruleName  value
          [-isHard true|false]
          [-comment [<string rule_id>] <string rule_desc>]
```

The following table lists the simples constraints you can define in the technology. Click a constraint name to view the details. For general information about technology constraints, see [Constraints Overview](#).

Constraint / ruleName	Constraint Category	Constraint Type
cumulativeMetalAntenna	Antenna	Complex
cumulativeViaAntenna	Antenna	Complex
default135RouteGridOffset	Routing	Simple
default135RouteGridPitch	Routing	Simple
default45RouteGridOffset	Routing	Simple
default45RouteGridPitch	Routing	Simple
defaultHorizontalRouteGridOffset	Routing	Simple
defaultHorizontalRouteGridPitch	Routing	Simple
defaultVerticalRouteGridOffset	Routing	Simple
defaultVerticalRouteGridPitch	Routing	Simple
excludeRoutingVias	Routing	Complex
forbiddenOffTrackSpacing	Spacing	Complex
horizontalPlacementGridOffset	Placement and Alignment	Simple

Constraint / ruleName	Constraint Category	Constraint Type
horizontalPlacementGridPitch	Placement and Alignment	Simple
leConnectingLayers	Routing	Simple
maxFloatingArea	Antenna	Complex
minUncutEndToEndSpacing	Spacing	Simple
shapeAngle	Miscellaneous	Complex
taperHalo	Routing	Simple
validBridgeVias	Via Construction	Complex
validRoutingLayers	Routing	Simple
validRoutingVias	Routing	Simple
verticalPlacementGridOffset	Placement and Alignment	Simple
verticalPlacementGridPitch	Placement and Alignment	Simple
viaStackLimit	Via Construction	Complex

cumulativeMetalAntenna

The cumulativeMetalAntenna constraint applies to area values accumulated through all the poly and metal layers attached to a gate.

The constraint specifies the maximum antenna ratios for one or more antenna models (oxide thicknesses).

This constraint holds the gateAreaRatio — a value that specifies the maximum allowed ratio of this cumulative sum to the gate area and how that maximum ratio changes with the diodeAreaRatio, if a diode is present on the wire.

Constraint Category	Antenna
Constraint Type	Complex
OA Rule	oaCumMetalAntenna

Syntax:

```
cumulativeMetalAntenna
    <string> defaultModel (
        -diodeRatio <1DTblValue>diodeRatio
```

```
-gateRatio <float>gateRatio
[-isSide true|false])

[<string> secondModel(
  -diodeRatio <1DTblValue>diodeRatio
  -gateRatio <float>gateRatio
  [-isSide true|false]])

[<string> thirdModel(
  -diodeRatio <1DTblValue>diodeRatio
  -gateRatio <float>gateRatio
  [-isSide true|false]])

[<string> fourthModel(
  -diodeRatio <1DTblValue>diodeRatio
  -gateRatio <float>gateRatio
  [-isSide true|false]])
[-isHard true|false]
[-comment [<string rule_id>] <string rule_desc>]
```

Where diodeRatio is a 1DTblValue:

```
(<float>default_value ((<float>diodeArea
                        <float>diodeAreaRatio)*))
```

Argument	Description
-diodeRatio	1DLookupTbl where: • key = diode area. • value = the maximum allowed antenna area if connected to the diffusion diode.
-gateRatio	A ratio to the gate area of the maximum allowed, accumulated area (that is not connected to the diffusion diode) of all the poly and metal Layers attached to a gate.
-isSide	If area is true then sidewall else top surface. Default is “false”.

This example defines a cumulativeMetalAntenna constraint with the default model:

```
cumulativeMetalAntenna defaultModel(
  -diodeRatio(50((0.1 100) (0.4 200)))
  -gateRatio 10
  -isSide true)
```

For more information on defining range tables, see [Understanding and Interpreting 1-D, 2-D, and 3-D Table Values](#) section.

cumulativeViaAntenna

The cumulativeViaAntenna constraint applies to area values accumulated through all the cut and via layers attached to a gate.

The constraint specifies the maximum antenna ratios for one or more antenna models (oxide thicknesses).

This constraint holds the gateAreaRatio, a value that specifies the maximum allowed ratio of this cumulative sum to the gate area and how that maximum ratio changes with the diodeAreaRatio, if a diode is present on the wire.

Constraint Category	Antenna
Constraint Type	Complex
OA Rule	oaCumViaAntenna

Syntax:

```
cumulativeViaAntenna
  <string|LPP>layer
  <string> defaultModel(
    -diodeRatio <1DTblValue>diodeRatio
    -gateRatio <float>gateRatio
    [-isSide true|false])

  [<string> secondModel(
    -diodeRatio <1DTblValue>diodeRatio
    -gateRatio <float>gateRatio
    [-isSide true|false])]

  [<string> thirdModel(
    -diodeRatio <1DTblValue>diodeRatio
    -gateRatio <float>gateRatio
    [-isSide true|false])]

  [<string> fourthModel(
    -diodeRatio <1DTblValue>diodeRatio
    -gateRatio <float>gateRatio
    [-isSide true|false])]

  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Where diodeRatio is a 1DTblValue:

```
(<float>default_value ((<float>diodeArea
  <float>diodeAreaRatio*))
```

Argument	Description
-diodeRatio	1DLookupTbl where: • key = diode area. • value = the maximum allowed antenna area if connected to the diffusion diode.
-gateRatio	A ratio to the gate area of the maximum allowed, accumulated area (that is not connected to the diffusion diode) of all the via and cut Layers attached to a gate.
-isSide	If area is true then sidewall else top surface. Default is "false".

This example defines a cumulativeViaAntenna constraint with the default, second, and third model:

```
cumulativeViaAntenna defaultModel(  
    -diodeRatio(40((0.2 200)(0.3 400)))  
    -gateRatio 20  
    -isSide true)  
secondModel(  
    -diodeRatio(100((0.2 500)(0.3 1000)))  
    -gateRatio 100)  
thirdModel(  
    -diodeRatio(50((1.0 500)(2.0 1000)))  
    -gateRatio 100  
    -isSide true)
```

For more information on defining range tables, see [Understanding and Interpreting 1-D, 2-D, and 3-D Table Values](#) section.

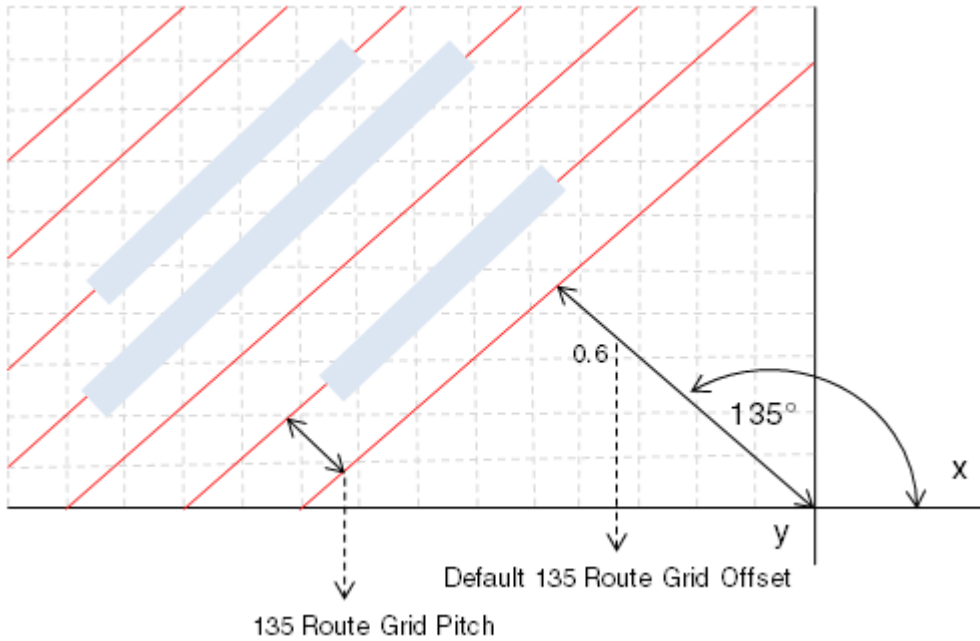
default135RouteGridOffset

This simple constraint specifies the default 135 degree route grid offset that is applicable for all layers.

Either the [default135RouteGridPitch](#) or the [135RouteGridPitch](#) determines the 135 routing grid pitch.

Note:

If the 135RouteGridOffset is set for any layer, that value has precedence.



Constraint Category	Routing
Constraint Type	Simple
OA Rule	oacDefault135RouteGridOffset

Syntax:

```
default135RouteGridOffset <float>offsetValue
    [-isHard true|false]
    [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
offsetValue	Specifies the default 135 degree route grid offset value.

For example:

```
default135RouteGridOffset 0.6
```

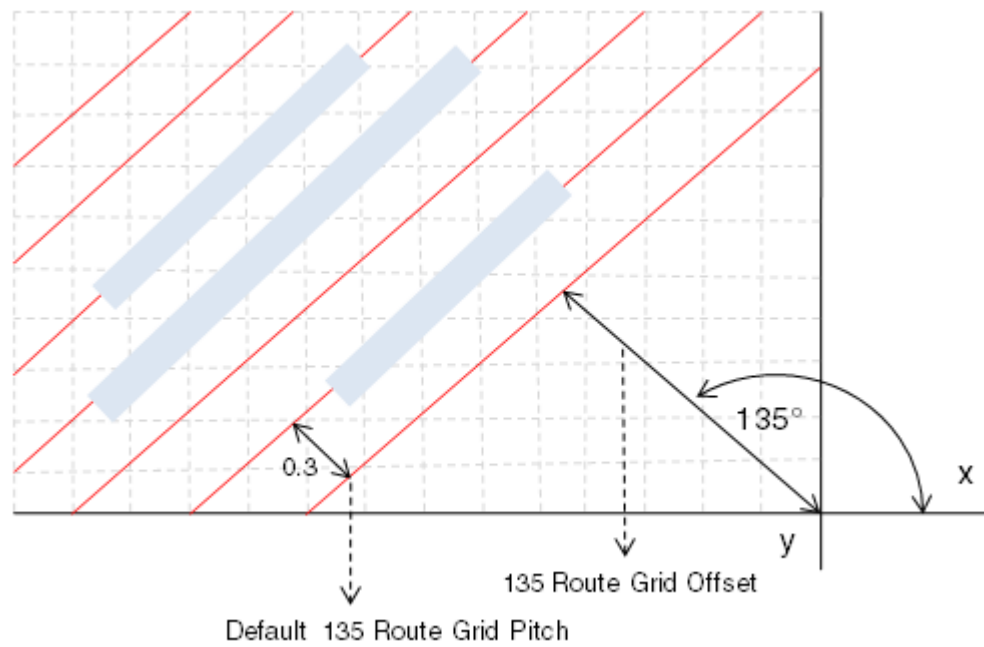
default135RouteGridPitch

This simple constraint specifies the default left diagonal route grid pitch for all layers.

Either the `default135RouteGridOffset` or `135RouteGridOffset` determines the 135 route grid offset.

Note:

If the `135RouteGridPitch` constraint is specified for any layer, that value has precedence.



Constraint Category	Routing
Constraint Type	Simple
OA Rule	oacDefault135RouteGridPitch

Syntax:

```
default135RouteGridPitch <float>pitchValue
    [-isHard true|false]
    [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
pitchValue	Specifies the default 135 degree route grid pitch value.

For example:

```
default135RouteGridPitch 0.3
```

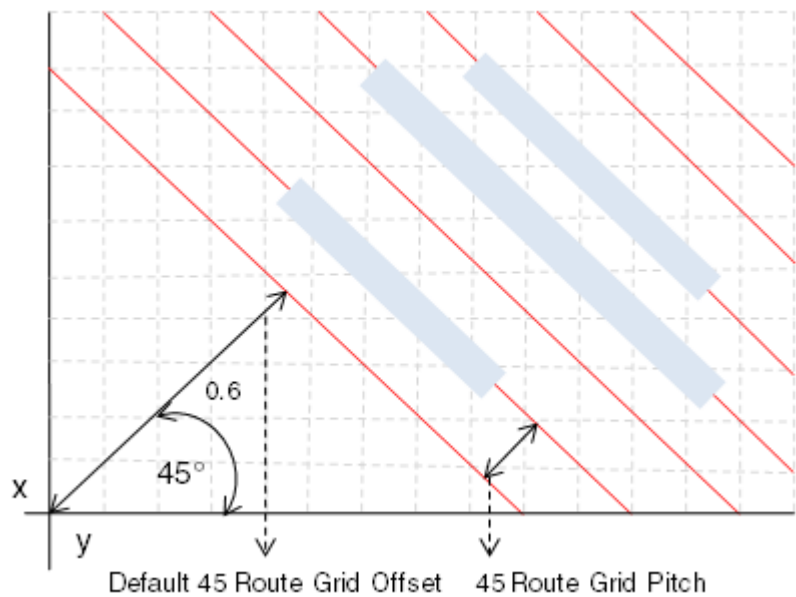
default45RouteGridOffset

This simple constraint specifies the default 45 route grid offset for all layers.

Either the [default45RouteGridPitch](#) or the [45RouteGridPitch](#) constraint determines the 45 route grid pitch.

Note:

If the 45RouteGridOffset is set for any layer, that value has precedence.



Constraint Category	Routing
Constraint Type	Simple
OA Rule	oacDefault45RouteGridOffset

Syntax:

```
default45RouteGridOffset <float>offsetValue  
  
[-isHard true|false]  
[-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
offsetValue	Specifies the default 45 degree route grid offset value.

For example:

```
default45RouteGridOffset 0.6
```

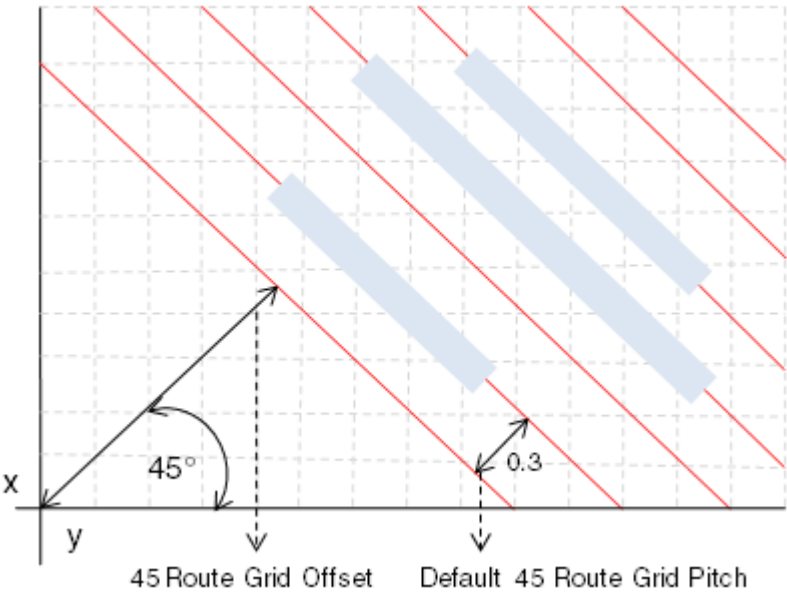
default45RouteGridPitch

This simple constraint specifies the default right diagonal route grid pitch for all layers.

Either the [default45RouteGridOffset](#) or [45RouteGridOffset](#) determines the 45 routing grid offset.

Note:

If the 45RouteGridPitch constraint is set for any layer, that value has precedence.



Constraint Category	Routing
Constraint Type	Simple
OA Rule	oacDefault45RouteGridPitch

Syntax:

```
default45RouteGridPitch <float>pitchValue

    [-isHard true|false]
    [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
pitchValue	Specifies the default 45 degree route grid pitch value.

For example:

```
default45RouteGridPitch 0.3
```

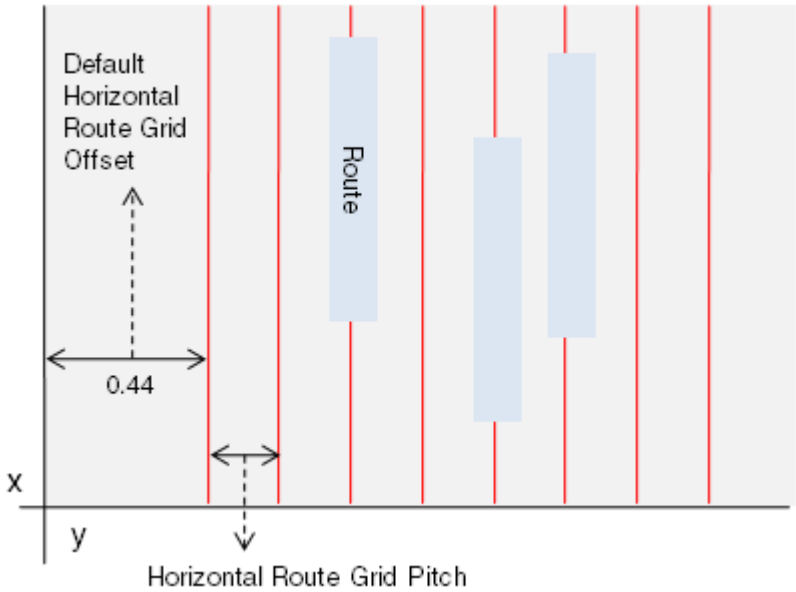
defaultHorizontalRouteGridOffset

This simple constraint specifies the default horizontal route grid offset for all layers.

Either the [defaultHorizontalRouteGridPitch](#) or the [horizontalRouteGridPitch](#) determines the horizontal routing grid pitch.

Note:

If the [horizontalRouteGridOffset](#) is set for any layer, that value has precedence.



Constraint Category	Routing
---------------------	---------

Constraint Type	Simple
OA Rule	oacDefaultHorizontalRouteGridOffset

Syntax:

```
defaultHorizontalRouteGridOffset <float>offsetValue  
  
    [-isHard true|false]  
    [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
offsetValue	Specifies the default horizontal route grid offset value.

For example:

```
defaultHorizontalRouteGridOffset 0.44
```

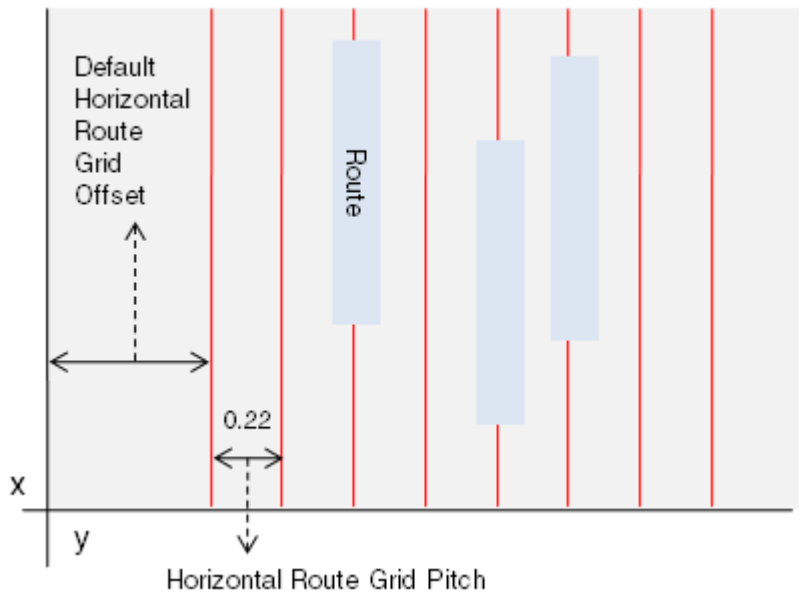
defaultHorizontalRouteGridPitch

This simple constraint specifies the default horizontal route grid pitch for all layers.

Either the [defaultHorizontalRouteGridOffset](#) or [horizontalRouteGridOffset](#) determines the route grid offset.

Note:

If the [horizontalRouteGridPitch](#) constraint is set for any layer, that value has precedence.



Constraint Category	Routing
Constraint Type	Simple
OA Rule	oacDefaultHorizontalRouteGridPitch

Syntax:

```
defaultHorizontalRouteGridPitch <float>pitchValue  
  
    [-isHard true|false]  
    [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
pitchValue	Specifies the default horizontal route grid pitch value.

For example:

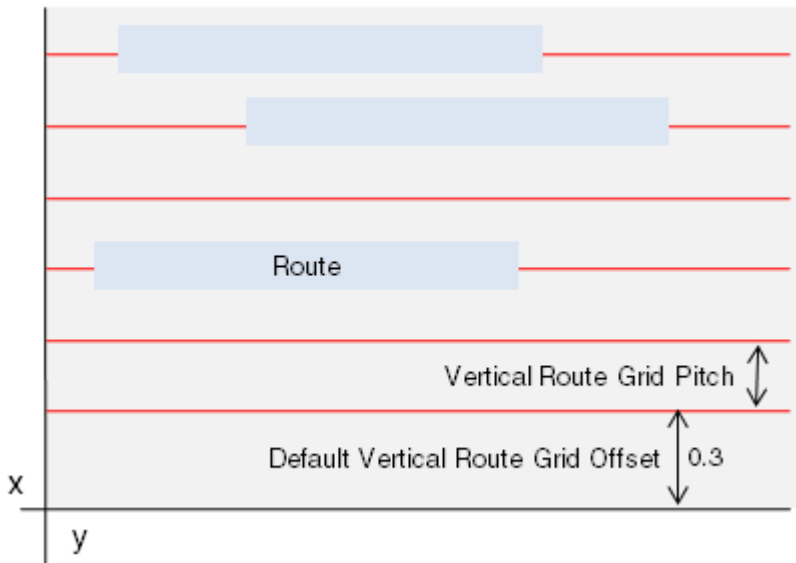
```
defaultHorizontalRouteGridPitch 0.22
```

defaultVerticalRouteGridOffset

This simple constraint specifies the default vertical route grid offset for all layers.

Note:

If the [verticalRouteGridOffset](#) is specified for any layer, that value has precedence.



Constraint Category	Routing
Constraint Type	Simple
OA Rule	oacDefaultVerticalRouteGridOffset

Syntax:

```
defaultVerticalRouteGridOffset <float>offsetValue
    [-isHard true|false]
    [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
offsetValue	Specifies the default vertical route grid offset value.

For example:

```
defaultVerticalRouteGridOffset 0.3
```

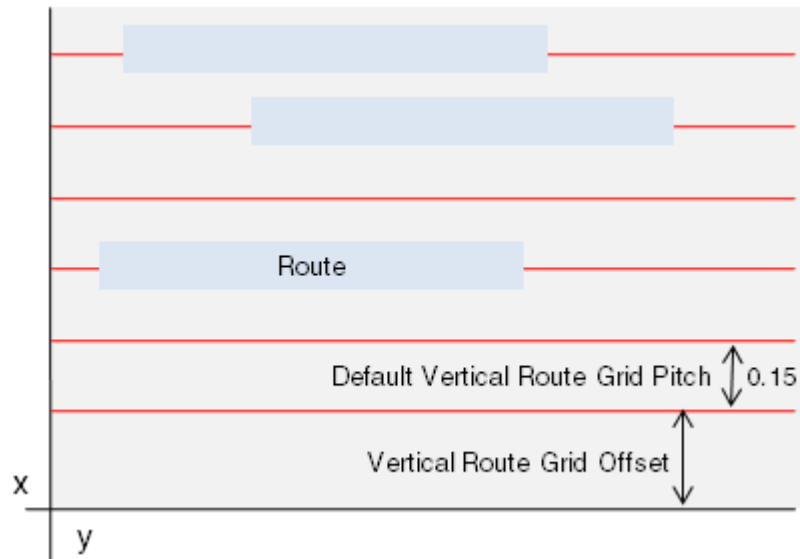
defaultVerticalRouteGridPitch

This simple constraint specifies the default vertical route grid pitch for all layers.

Either the `defaultVerticalRouteGridOffset` or `verticalRouteGridOffset` determines the route grid offset.

Note:

If the `verticalRouteGridPitch` constraint is specified for any layer, that value has precedence.



Constraint Category	Routing
Constraint Type	Simple
OA Rule	oacDefaultVerticalRouteGridPitch

Syntax:

```
defaultVerticalRouteGridPitch <float>pitchValue
    [-isHard true|false]
    [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
pitchValue	Specifies the default vertical route grid pitch value.

For example:

```
defaultVerticalRouteGridPitch 0.15
```

excludeRoutingVias

This constraint specifies whether a specific contact code is not allowed for signal routing.

Constraint Category	Routing
Constraint Type	Complex
OA Rule	Custom Compiler Custom Layer Constraint

Syntax:

```
excludeRoutingVias <List of strings>viaNames
    [-signalTypes (signal|power|ground|
                  clock|tieoff|tieHi|
                  tieLo|analog|scan|reset) ]

    [-isHard true|false]
    [-comment [<string rule_id>] <string rule_desc>]
```

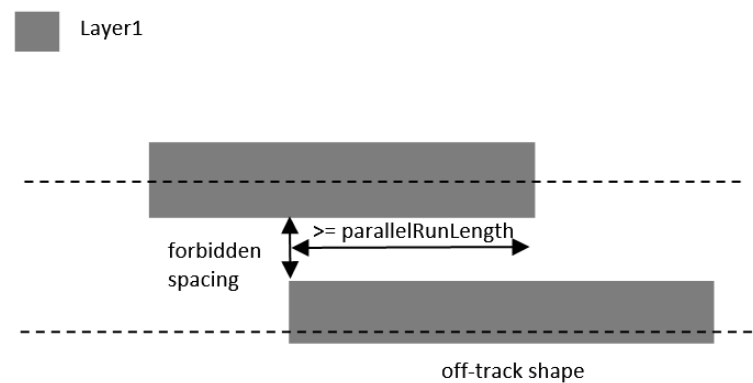
Argument	Description
viaNames	List of viaDefs or viaVariants that are not allowed.
-signalTypes	List of signalTypes that apply to this constraint. If not given, all signalTypes apply.

For example:

```
excludeRoutingVias (v12 v23 v34) -signalTypes (power ground tieoff reset)
```

forbiddenOffTrackSpacing

Some processes do not allow an off-track metal shape to be located at a certain distance from another metal shape depending on the common parallel run length. There can be several constraints present for the same layer with different forbidden spacing and parallel run length values.



Constraint Category	Spacing
Constraint Type	Complex
OA Rule	Custom Compiler Custom Layer Constraint

Syntax:

```
forbiddenOffTrackSpacing
  <string|LPP>layer
  <float>spacing
  [-parallelRunLength floatValue]

  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

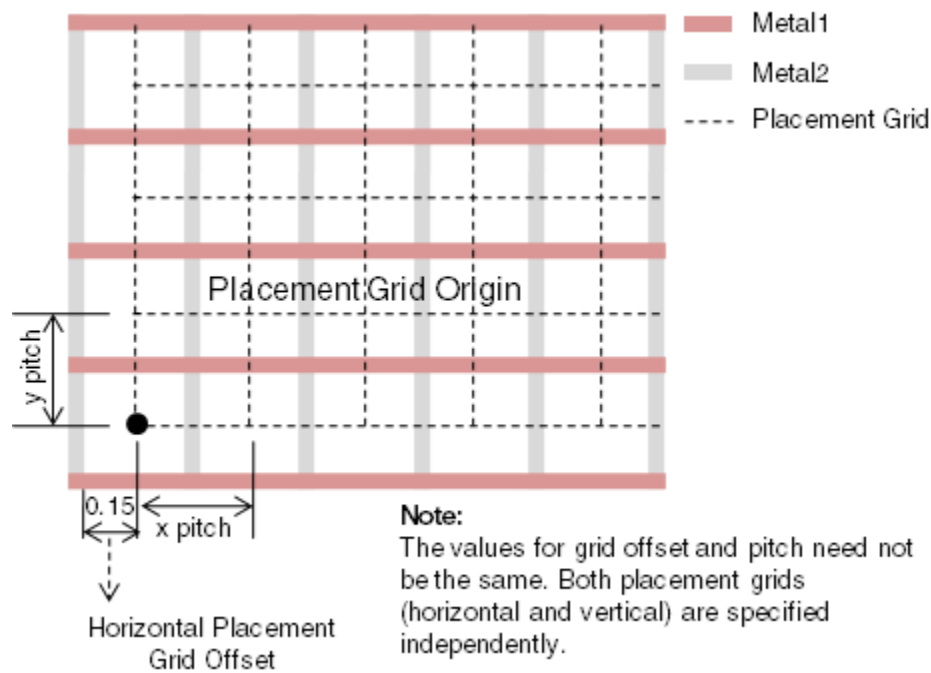
Argument	Description
layer	Layer can be defined as a string or as LPP.
spacing	The forbidden spacing.
-parallelRunLength	List of viaDefs or viaVariants that are not allowed.

For example:

```
forbiddenOffTrackSpacing m1 0.06 -parallelRunLength 0.05
```

horizontalPlacementGridOffset

This simple constraint specifies a grid offset relative to the design origin, in the horizontal placement grid while placing instances.



Constraint Category	Placement and Alignment
Constraint Type	Simple
OA Rule	oacHorizontalPlacementGridOffset

Syntax:

```
horizontalPlacementGridOffset <float>offsetValue
    [-isHard true|false]
    [-comment [<string rule_id>] <string rule_desc>]
```

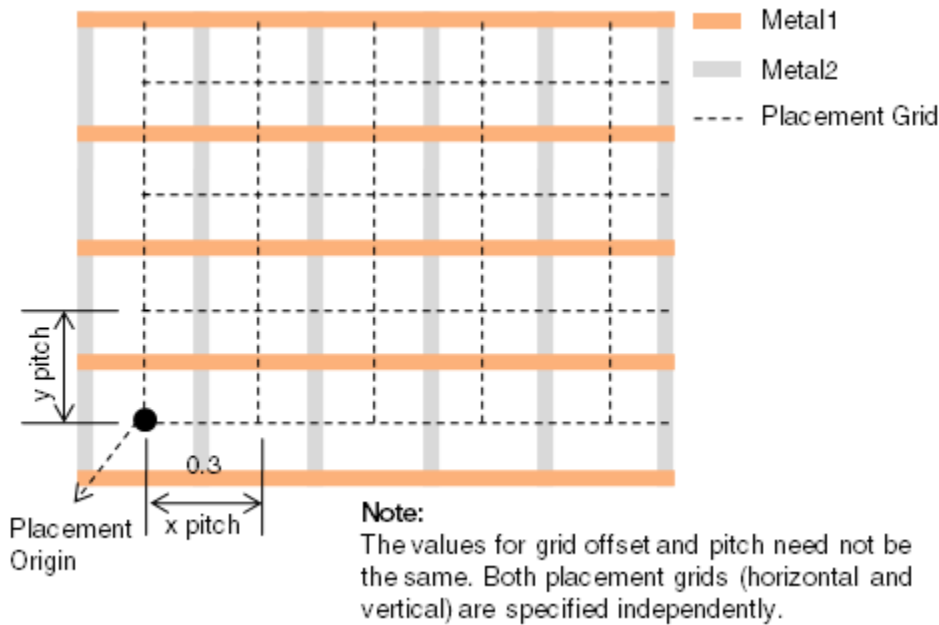
Argument	Description
offsetValue	Specifies the horizontal placement grid offset value.

For example:

```
horizontalPlacementGridOffset 0.15
```

horizontalPlacementGridPitch

This simple constraint specifies an explicit placement grid pitch for placing instances. The grid is interpreted relative to the origin of the design.



Constraint Category	Placement and Alignment
Constraint Type	Simple
OA Rule	oacHorizontalPlacementGridPitch

Syntax:

```
horizontalPlacementGridPitch <float>pitchValue

    [-isHard true|false]
    [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
pitchValue	Specifies the horizontal placement grid pitch value.

For example:

```
horizontalPlacementGridPitch 0.3
```

leConnectingLayers

The leConnectingLayers simple constraint specifies:

- Two layers that are electrically connected. The two layers must be defined by the [validRoutingLayers](#) constraint.
- (Optional) The cut layer that connects the two specified layers.

Constraint Category	Routing
Constraint Type	Simple
OA Rule	Not an OA defined rule. This is a Custom Compiler constraint used to connect layers while routing.

Syntax:

```
leConnectingLayers
<List of strings>layers
[-byLayer <string>layerName]

[-isHard true|false]
[-comment [<string rule_id>] <string rule_desc>]
```

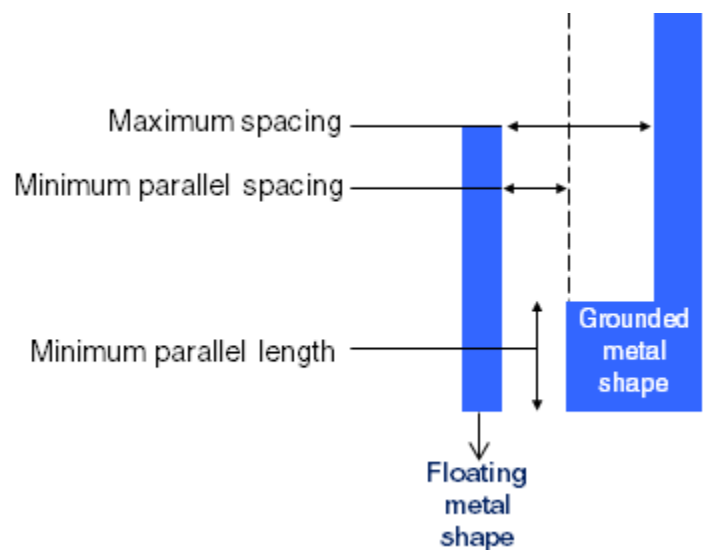
Argument	Description
layers	Specifies the list of layers that are to be considered for connecting while routing. If you specify -byLayer, you can only specify two layers.
-byLayer	Specifies the cut layer that connects the two specified layers.

For example:

```
leConnectingLayers (m1 m2) -byLayer v1
```

maxFloatingArea

This constraint defines the maximum area rules for floating metal shapes that are not connected to a diffusion or polysilicon-gate



Constraint Category	Antenna
Constraint Type	Complex
OA Rule	oacMaxFloatingArea

Syntax:

```
maxFloatingArea <float>maxArea
    -floatingMetal single|connected|allConnected
    -routingLayers <list of strings>(layerName*)
    -floatingAreaSpacing <float2DTable>

    [-isHard true|false]
    [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
maxArea	The maximum area.
-floatingMetal	This parameter stores the inclusive list of routing layers between range of routing layers to which the constraint applies. This parameter can take one of the following values: single, connected, and allConnected.
-routingLayers	This parameter stores the inclusive range of oaLayerValue values that correspond to the layers listed for minRoutingLayer to maxRoutinglayer.

Argument	Description
-floatingAreaSpacing	This parameter stores pairs of minimum spacings and minimum parallel spacings for a given layer. The index of the table corresponds to the index of the layer in the oaLayerArrayValue in routinglayers. The floatingAreaSpacing 2D table is represented as: (layerName (minSpacing minParSpacing minParallelLength))

This example defines an antenna constraint with the default model:

```
maxFloatingArea 0.08
-floatingMetal single
-routingLayers (m1 m2)
-floatingAreaSpacing ((m1 (0.10 0.08 0.12))
                      (m2 (0.12 0.10 0.14)))
```

For more information on defining range tables, see [Understanding and Interpreting 1-D, 2-D, and 3-D Table Values](#) section.

minUncutEndToEndSpacing

This constraint defines the maximum area rules for floating metal shapes that are not connected to a diffusion or polysilicon-gate

Constraint Category	Spacing
Constraint Type	Simple
OA Rule	Not an OA defined rule. This is a Custom Compiler specific constraint used to connect layers while routing.

Syntax:

```
minUncutEndToEndSpacing <strings>layer
                        <float|1DTable>spacing

                        [-isHard true|false]
                        [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Specifies the layer for which the constraint applies.
spacing	float: Distance between the shape ends. 1DTable: One dimensional table indexed by width with values for minimum end to end spacing in database unit in which no cut shape is required.

For example:

```
minUncutEndToEndSpacing m1 0.07
minUncutEndToEndSpacing m1 0.08 ((0.06 0.10) (0.08 0.12))
```

shapeAngle

This simple constraint specifies the default geometry orientations that are allowed by the technology.

Note:

The default orientation is designated by a single enumerated type and applies to all layers in the technology.

Constraint Category	Miscellaneous
Constraint Type	Complex
OA Rule	oacShapeAngle

Syntax:

```
shapeAngle <string all|orthogonalOnly|diagonalOnly>value
[-isHard true|false]
[-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
value	• all: All angles • orthogonalOnly: Orthogonal only • diagonalAndOrthogonal: 45° or 135° and orthogonal

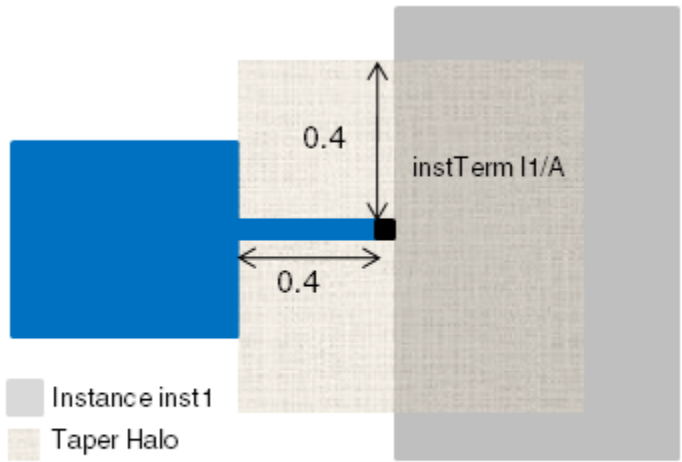
For example:

```
shapeAngle all
```

taperHalo

This simple constraint specifies the offset around pin figures within which tapering must be performed according to the spacing and width constraints specified in the taper constraint group. A single value designates the offset from the pin figure on all sides.

The taperHalo value is denoted in microns.



taperHalo indicates the way in which wide wires taper down to minimum width/spaced pins and vice versa.

Constraint Category	Routing
Constraint Type	Simple
OA Rule	oacTaperHalo

Syntax:

```
taperHalo <float>taperValue
    [-isHard true|false]
    [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
taperValue	Specifies the offset around pin figures within which tapering must be performed according to the spacing and width constraints specified in the taper constraint group.

For example:

```
taperHalo 0.4
```

validBridgeVias

This simple constraint is used for valid bridge vias.

Constraint Category	Via Construction
---------------------	------------------

Constraint Type	Complex
OA Rule	oaViaTopologyArrayValue

Syntax:

```
validBridgeVias
    <List of strings>viaNames

    [-isHard true|false]
    [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
viaNames	List of viaDefs or viaVariants that are not allowed.

For example:

```
validBridgeVias (v12 v23 v34)
```

validRoutingLayers

This simple constraint specifies the list of layers that are to be considered as available while routing. If this constraint is not defined, the Custom Compiler environment tries to create a new constraint that contains a list of valid layers.

The following steps show how the Custom Compiler environment builds the valid routing layers constraint:

1. The Custom Compiler tool looks for a `validRoutingLayers` constraint in the active constraint group.
2. The precedence order is based on the technology database. If not found, the Custom Compiler environment looks for an existing `validRoutingLayers` constraint according to a precedence order based on the technology database. If not found, then the Custom Compiler environment creates a new constraint based on the available layers information.

3. The Custom Compiler environment also looks for a `validRoutingVias` constraint. If found, the Custom Compiler environment does either of the following:
 - Collects all `layer1` and `layer2` definitions listed in `validRoutingVias` into a `layerList`.
 - If no `validRoutingVias` is found, if the technology data contains via definitions, the Custom Compiler environment collects all `layer1` and `layer2` definitions from all via definitions into a `layerList`, and collects all layers with a material value of poly, metal, or contactless metal into a `layerList`.
4. If the technology data includes a default constraint group, the Custom Compiler environment creates the `validRoutingLayers` constraint in that group.

If the technology data does not include a default constraint group, the Custom Compiler environment creates the `validRoutingLayers` constraint in the foundry constraint group.
5. The Custom Compiler environment adds all layers from the previously created `layerList` to the newly created `validRoutingLayers` constraint.

Constraint Category	Routing
Constraint Type	Simple
OA Rule	oacValidRoutingLayers

Note:

Every `validRoutingLayer` must be the name of a single layer and not that of a layer purpose pair.

Syntax:

```
validRoutingLayers <List of strings>layers
    [-isHard true|false]
    [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layers	Specifies the list of layers that are to be considered as available during routing.

For example:

```
validRoutingLayers (v8 v7 v6 v5 v4 v3 v2 v1 m9 m8 m7 m6 m5 m4 m3 m2
    m1 cont poly diff)
```

validRoutingVias

This simple constraint specifies the vias that can be used in routing.

The constraint can include either or both of the following:

- Actual specifications of particular vias
- List of the via definitions

The constraint can also specify valid or default power routing vias.

Constraint Category	Routing
Constraint Type	Simple
OA Rule	oacValidRoutingVias

Syntax:

```
validRoutingVias <viaArrayValue|via2DTblValue> viaDefs
                -viaLayerDirection (2DTblValue)

                [-isHard true|false]
                [-comment [<string rule_id>] <string rule_desc>]
```

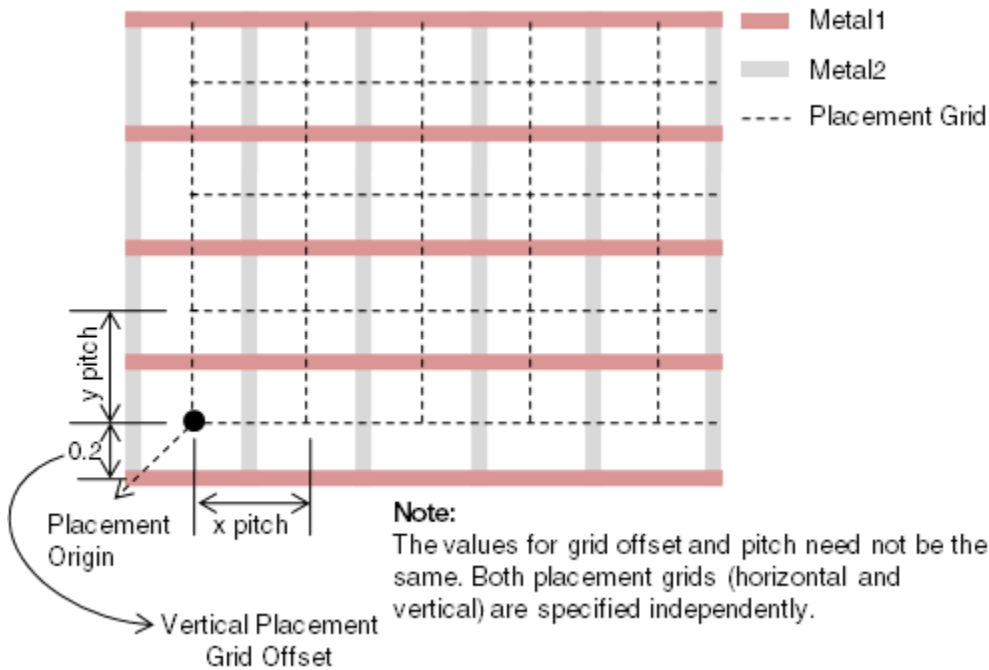
Argument	Description
viaDefs	viaArrayValue: Specifies the list of vias that can be used in routing. The list can consist of via definition names and via variant definition names. via2DTblValue: The table of viaDef names keyed off of width for both rows and columns. ((<float>width of lower metal <float>width of upper metal) viaArrayValue)
-viaLayerDirection	2DTblValue: The routing direction of the via layer. Each direction in the 2D table corresponds to the via layer specified by viaDefs. Valid direction values are any, horizontal, and vertical.

For example:

```
validRoutingVias (v12 v23 v34)
validRoutingVias (v12 v23 v34)
                  (((0.06 0.08) (v12s v23s v34s))
                   ((0.06 0.10) (v12s v23s v34d))
                   ((0.08 0.08) (v12s v23d v34d))
                   ((0.08 0.10) (v12d v23d v34d)))
validRoutingVias (M1 PO M2 M1)
                  -viaLayerDirection ((horizontal horizontal) (horizontal
vertical))
```

verticalPlacementGridOffset

This simple constraint specifies the grid offset relative to the design origin, in the vertical placement grid while placing instances.



Constraint Category	Placement and Alignment
Constraint Type	Simple
OA Rule	oacVerticalPlacementGridOffset

Syntax:

```
vertical Placement Grid Offset <float>offsetValue  
  
    [-isHard true|false]  
    [-comment [<string rule_id>] <string rule_desc>]
```

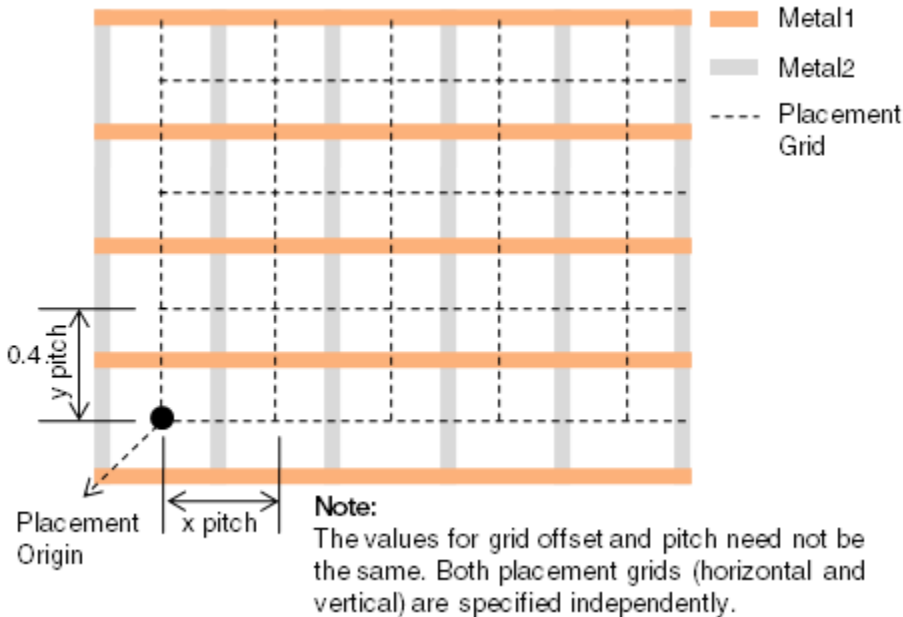
Argument	Description
offsetValue	Specifies the vertical placement grid offset value.

For example:

```
verticalPlacementGridOffset 0.2
```

verticalPlacementGridPitch

This simple constraint specifies an explicit vertical placement grid pitch for placing instances. The grid is specified by the placement grid offset from the design's origin.



Constraint Category	Placement and Alignment
Constraint Type	Simple
OA Rule	oacVerticalPlacementGridPitch

Syntax:

```
verticalPlacementGridPitch <float>pitchValue  
  
    [-isHard true|false]  
    [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
pitchValue	Specifies the vertical placement grid pitch value.

For example:

```
verticalPlacementGridPitch          0.4
```

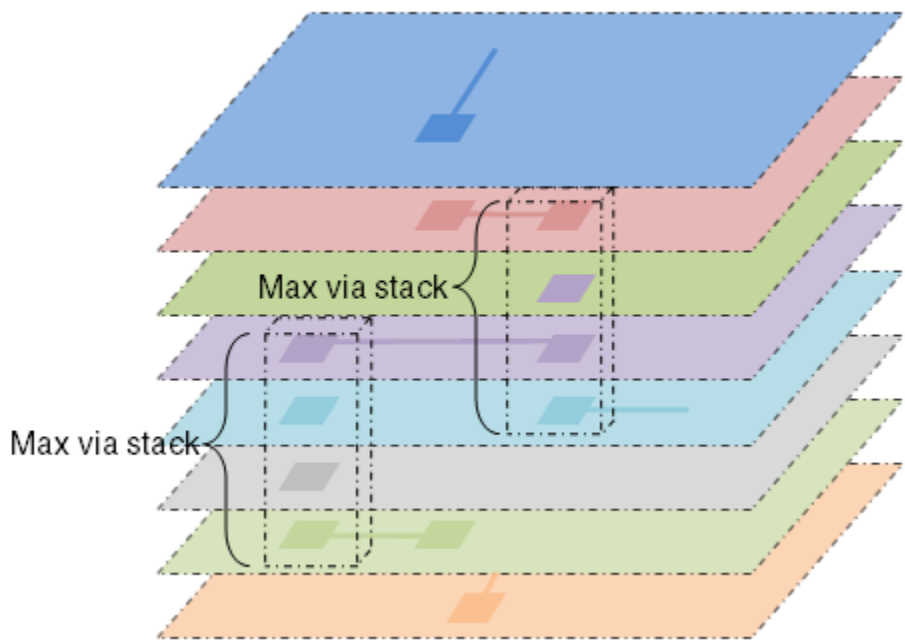
viaStackLimit

This simple constraint specifies the maximum number of continuous, stacked vias that are allowed.

Note:

A via is considered to be in a stack with another via if the cut of one via overlaps any part of the cut of the other via.

Two additional parameters (specifying the bottom and top layers of the range) can be used to set the range of layers for which the constraint applies. This constraint applies to all layers, unless otherwise specified.



Constraint Category	Via Construction
Constraint Type	Complex
OA Rule	oacViaStackLimit

Syntax:

```
viaStackLimit <int>  
  [-lowerLayer <string|LPP>layer]  
  [-upperLayer <string|LPP>layer]  
  [-noSingleCutVia true|false]  
  [-minNumCuts intValue]
```

```
[-maxNumCuts intValue]
[-cutDistance float]
[-exceptEnclosure float]
[-exceptPGNet true|false]

[-isHard true|false]
[-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
int	Specifies the number of single-cut vias.
-lowerLayer	Specifies the lower layer.
-upperLayer	Specifies the upper layer.
-noSingleCutVia	Specifies if the stacked vias are single-cut or multiple-cut. Default is "false". This parameter cannot be used together with -minNumCuts and -maxNumCuts parameters.
-minNumCuts	Specifies the minimum number of via cuts needed for the via to be considered as part of the stack. This parameter cannot be used together with the oaNoSingleCutVia parameter.
-maxNumCuts	Specifies the maximum number of via cuts needed for the via to be considered as part of the stack. This parameter cannot be used together with the oaNoSingleCutVia parameter.
-cutDistance	By default, a via is considered to be in a stack with another via if the cut of one via overlaps with the cut of the other via. If this parameter is given, via cuts that do not overlap are also considered for stacking if the distance between the cuts is less than this value in database units.
-exceptEnclosure	If given, the via cuts that have metal enclosure (in all directions for both lower and upper metal layers) greater than or equal to this value in database units are excluded from the stacking consideration.
-exceptPGNet	If true, the rule does not apply to power and ground nets. By default, the constraint applies to all nets. Default: false

Argument	Description
	Array of values representing a list of (cut layer1, cut layer2, cut class list for cut layer1, cut class list for cut layer2) tuples. For every list item, the cut layer1 and cut layer2 are two adjacent cut layers. If this parameter is given, the via stacking limit applies only to those stacked vias that have at least one pair of stacked cuts from the same list item – one layer1 cut from cut class list for cut layer1, another layer2 cut from cut class list for cut layer2. If the lowerLayer or upperLayer parameters are also given, cut layer1 and cut layer2 cannot be below lowerLayer and cannot be above upperLayer. The first element of the value array is oalnt value representing the size (N) of the cutClassPairList followed by N sequences of array elements where each sequence represents one list item and has variable size depending on the number of cut classes in the corresponding cut class lists. For each list item, the sequence consists of following elements: oaLayer value for the cut layer1, oaLayer value for the cut layer2, oalnt value for the size of the cut class list for cut layer1, a sequence of oaDualInt values each defining width and height of a cut class in database units for cut layer1, oalnt value for the size of the cut class list for cut layer2, a sequence of oaDualInt values each defining width and height of a cut class in database units for cut layer2.

Note:

If -lowerLayer and -upperLayer are...	then...
specified	The constraint only applies when stacking vias in the range between the two specified layers.
not specified	The constraint applies to all layers.

For example:

```
viaStackLimit 2
viaStackLimit 2 -lowerLayer m1 -upperLayer m2 -noSingleCutVia true
viaStackLimit 3 -minNumCuts 2 -cutDistance 1.5 -exceptEnclosure 1.2
                  -exceptPGNet true
viaStackLimit 3 -maxNumCuts 2
viaStackLimit 3 -minNumCuts 2 -maxNumCuts 5
viaStackLimit 3 -minNumCuts 2 -maxNumCuts 2
viaStackLimit 2 -isHard false
viaStackLimit 2 -lowerLayer m1 -upperLayer m2 -noSingleCutVia true

-cutClassPairList ((
    -cutLayer1 m1
    -cutLayer2 m2
    -cutClassList1 ((0.03 0.04))
    -cutClassList2 ((0.02 0.03))
))
(
    -cutLayer1 m1
    -cutLayer2 m2
    -cutClassList1 ((0.03 0.04))
)
```



```
-cutClassList2 ((0.02 0.03))  
))
```

7

Layer Constraints

Describes layer constraints and provides the syntax with examples.

Layer constraints are constraints that are defined for one specific layer.

Simple layer constraints take a single value or a preferred value. The general syntax is:

```
ruleName <string|LPP>layer
        <float|bool>value
        [-isHard true|false]
        [-comment [<string rule_id>] <string rule_desc>]
```

Complex layer constraint can take a range of values as with multiple parameters as inputs on which the constraint depends.

The following table lists the layer constraints you can define in the technology. Click a constraint name to view the details. For general information about technology constraints, see [Constraints Overview](#).

Constraint / ruleName	Constraint Category	Constraint Type
135RouteGridOffset	Routing	Simple
135RouteGridPitch	Routing	Simple
45RouteGridOffset	Routing	Simple
45RouteGridPitch	Routing	Simple
alignedWireSpacing	Spacing	Complex
alignToOtherLayerTracks	Spacing	Complex
allowedCutClass	Via Construction	Complex
allowedLengthRange	Length and Width	Complex
allowedCutSpacing	Spacing	Complex
allowedEndToEndSpacing	Spacing	Complex
allowedEndToSideSpacing	Spacing	Complex

Constraint / ruleName	Constraint Category	Constraint Type
allowedNeighborViaGrid	Spacing	Complex
allowedParallelSpanSpacing	Spacing	Complex
allowedSpacingRange	Spacing	Complex
allowedSpanLengthRanges	Spacing	Complex
allowedViaGroupSpacing	Spacing	Complex
allowedWidthRange	Length and Width Process Rules	Complex
allowedWireSpacing	Spacing	Complex
antenna	Antenna	Complex
areaCapacitance	Spacing	Complex
commonNeighborViaCount	Via Construction	Complex
concaveCornerKeepout	Spacing	Complex
cornerViaClusterSpacing	Spacing	Complex
criticalEndKeepout	Spacing	Complex
cumulativeLayerAntenna	Antenna	Complex
cutClass	Via Construction	Complex
cutMetalAvoidanceKeepout	Spacing	Complex
defWidth	Length and Width Process Rules	Complex
diagonalShapesAllowed	Miscellaneous	Complex
dualEndOfLineKeepout	Spacing	Complex
edgeCapacitance	Spacing	Complex
enclosureMergeUnderErasing	Miscellaneous	Simple
endOfLineKeepout	Spacing	Complex
endOfLineNeighborKeepout	Spacing	Complex
endOfLineProjectionSpacing	Spacing	Complex
erasingLayer	Miscellaneous	Complex
erasingShapeNeededThreshold	Spacing	Complex
errorLayer	Miscellaneous	Simple
excludeSignalTypeRouting	Routing	Complex

Constraint / ruleName	Constraint Category	Constraint Type
extendedNotchSpacing	Spacing	Complex
forbiddenPitch	Spacing	Complex
forbiddenSpacing	Spacing	Complex
forbiddenTwoSidedViaSpacing	Spacing	Complex
forbiddenWidthTransition	Spacing	Complex
gateClearance	Clearance	Complex
gateOrientation	Miscellaneous	Complex
generalCutClassSpacing	Spacing	Complex
generalSpanSpacing	Spacing	Complex
generalViaClusterSpacing	Spacing	Complex
horizontalRouteGridOffset	Routing	Simple
horizontalRouteGridPitch	Routing	Simple
keepAlignedShapeAndBoundary	Placement and Alignment	Simple
layerShapeAngle	Miscellaneous	Complex
maxACCurrentDensity	Density	Complex
maxClusterDiagonalNumCuts	NumCut	Complex
maxDCCurrentDensity	Density	Complex
maxDensity	Density	Complex
maxDiagonalEdgeLength	Length and Width Process Rules	Simple
maxDiffDensity	Density	Complex
maxInlineEndOfLineProjections	Spacing	Complex
maxInlineNumCuts	NumCut	Complex
maxLength	Length and Width Process Rules	Simple
maxNeighborCuts	Via Construction	Complex
maxNumWiresEndOfLineKeepout	Spacing	Complex
maxRoutingDistance	Routing	Simple
maxTapSpacing	Spacing	Simple
maxUnalignedViaNeighbors	NumCut	Complex

Constraint / ruleName	Constraint Category	Constraint Type
maxViaClusterNumCuts	NumCut	Complex
maxWidth	Length and Width Process Rules	Simple
maxWireWidthCount	NumCut	Complex
mergeSpaceAllowed	Spacing	Simple
minAdjacentEndOfLineProjectionSpacing	Spacing	Complex
minAdjacentViaSpacing	Via Construction	Complex
minArea	Area	Simple
minAreaEdgeLength	Length and Width Process Rules	Complex
minBoundaryExtension	Extension	Complex
minBoundaryExteriorHalo	Spacing	Complex
minBoundaryInteriorHalo	Spacing	Simple
minCenterToCenterSpacing	Spacing	Simple
minClusterDiagonalSpacing	Spacing	Complex
minConcaveEdgeLength	Length and Width Process Rules	Complex
minConstrainedViaSpacing	Spacing	Complex
minConvexEdgeLength	Length and Width Process Rules	Complex
minCornerToCornerDistance	Spacing	Simple
minCutClassSpacing	Via Construction	Complex
minDensity	Density	Complex
minDiagonalEdgeLength	Length and Width Process Rules	Complex
minDiagonalSpacing	Spacing	Complex
minDiagonalWidth	Length and Width Process Rules	Simple
minDiffPotentialSpacing	Spacing	Simple
minEdgeAdjacentDistance	Length and Width Process Rules	Complex
minEdgeAdjacentLength	Length and Width Process Rules	Complex
minEdgeByType	Length and Width Process Rules	Complex
minEdgeLength	Length and Width Process Rules	Complex
minEdgeMaxCount	Length and Width Process Rules	Complex

Constraint / ruleName	Constraint Category	Constraint Type
minEnclosedArea	Area	Complex
minEnclosedSpacing	Spacing	Complex
minEndOfLineAdjacentToStep	Length and Width	Complex
minEndOfLineDirectionSpacing	Spacing	Complex
minEndOfLineExtensionSpacing	Spacing	Complex
minEndOfLineJogDistance	Spacing	Complex
minEndOfLineNotchSpacing	Spacing	Complex
minEndOfLineOrthoSpacing	Spacing	Complex
minEndOfLinePerpSpacing	Spacing	Complex
minEndOfLineSpacing	Spacing	Complex
minEndOfLineToConcaveCornerSpacing	Spacing	Complex
minEndOfLineViaNeighborSpacing	Spacing	Complex
minEndOfNotchSpacing	Spacing	Complex
minFillPatternSpacing	Spacing	Simple
minFillToFillSpacing	Spacing	Simple
minGeneralViaSpacing	Via Construction	Complex
minInnerConcaveCornerDistance	Spacing	Complex
minLargeViaArrayCutSpacing	Via Construction	Complex
minLargeViaArraySpacing	Via Construction	Complex
minLargeViaArrayWidth	Via Construction	Simple
minLength	Length and Width Process Rules	Simple
minLengthAlignedProximity	Length and Width Process Rules	Complex
minMaskArea	Area	Complex
minNeighborRegionParallelSpacing	Spacing	Complex
minNeighborViaSpacing	Via Construction	Complex
minNotchSpacing	Spacing	Complex
minNotchSpanSpacing	Spacing	Complex
minNumCut	NumCut	Complex

Constraint / ruleName	Constraint Category	Constraint Type
minOppositeSpanSpacing	Spacing	Complex
minOrthogonalViaSpacing	Via Construction	Complex
minParallelSpanSpacing	Spacing	Complex
minParallelViaSpacing	Via Construction	Simple
minParallelWithinViaSpacing	Via Construction	Simple
minPerimeter	Length and Width Process Rules	Simple
minProtrudedProximitySpacing	Spacing	Complex
minProtrusionLength	Length and Width Process Rules	Complex
minProtrusionNumCut	NumCut	Complex
minProtrusionSpacing	Spacing	Complex
minProtrusionWidth	Length and Width Process Rules	Complex
minProximitySpacing	Spacing	Complex
minRectArea	Area	Simple
minRectAreaProximity	Area	Complex
minRedundantViaSetback	Via Construction	Complex
minSameMetalSharedEdgeViaSpacing	Via Construction	Complex
minSameNetSpacing	Spacing	Complex
minSeparatedViaSpacing	Via Construction	Complex
minSideSpacing	Spacing	Complex
minSize	Length and Width Process Rules	Complex
minSpacing	Spacing	Complex
minViaRegionClusterSpacing	Spacing	Complex
minViaSpacing	Via Construction	Complex
minVoltageSpacing	Spacing	Complex
minWidth	Length and Width Process Rules	Complex
minWireExtension	Extension	Complex
multiEndOfLineEdgeKeepout	Spacing	Complex
multiPatternCutSpacing	Spacing	Complex

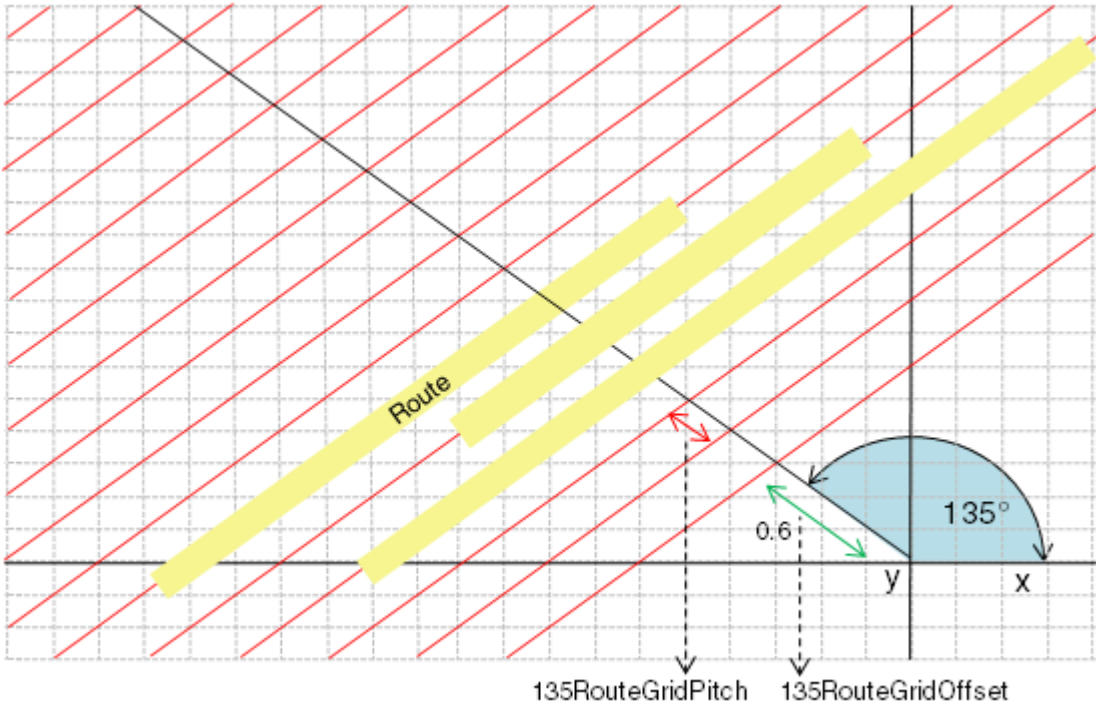
Constraint / ruleName	Constraint Category	Constraint Type
multiPatternSpacing	Spacing	Complex
nonIsolatedViaCornerSpacing	Spacing	Complex
nonSelfAlignedViaSpacing	Spacing	Complex
onlyLargestEnclosedCutClassAllowed	Spacing	Complex
onlyRectanglesAllowed	Spacing	Complex
onTrackRoutingOnly	Routing	Simple
oppositeEndOfLineSpacing	Spacing	Complex
parallelEndOfLineKeepout	Spacing	Complex
preferredRoutingDirection	Routing	Complex
preferredRoutingOnly	Routing	Simple
rectangularLargeViaArraysAllowed	Via Construction	Simple
rectangularShapeDirection	Miscellaneous	Complex
selfAlignedViaClusterSpacing	Via Construction	Complex
selfAlignedViaDiagMinSpacing	Via Construction	Complex
selfAlignedViaMaxAlignedCuts	Via Construction	Complex
selfAlignedViaSpacing	Via Construction	Simple
selfAlignedViaZigzagMinSpacing	Via Construction	Complex
sheetResistance	Spacing	Complex
stoppingLayer	Miscellaneous	Complex
threeWireForbiddenSpacing	Spacing	Complex
twoSidedParallelMinSpacing	Spacing	Complex
unalignedEndToEndKeepout	Routing	Simple
verticalRouteGridOffset	Routing	Simple
verticalRouteGridPitch	Routing	Simple

135RouteGridOffset

This layer constraint specifies the route grid points on the left diagonal axis. The route grid points are completely defined using the 135RouteGridOffset constraint along with the [135RouteGridPitch](#).

Note:

The left diagonal is found by rotating the horizontal axis 135 degrees counterclockwise.



Constraint Category	Routing
Constraint Type	Simple
OA Rule	oac135RouteGridOffset

Syntax:

```
135RouteGridOffset
  <string|LPP>layer
  <float>offsetValue

  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or as LPP.
offsetValue	Specifies the 135 degree route grid offset value.

For example:

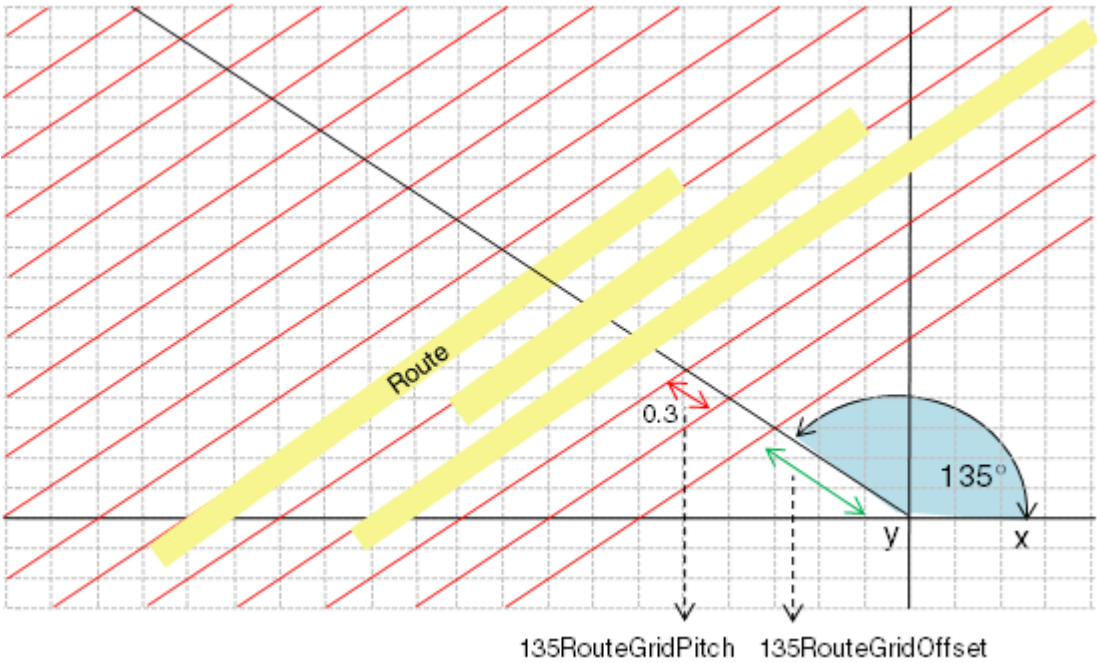
```
135RouteGridOffset    poly  0.6
```

135RouteGridPitch

This layer constraint specifies the route grid points on the left diagonal axis. The route grid points are completely defined using the 135RouteGridPitch and [135RouteGridOffset](#).

Note:

The left diagonal axis is found by rotating the horizontal axis 135 degrees counterclockwise.



Constraint Category	Routing
Constraint Type	Simple
OA Rule	oac135RouteGridPitch

Syntax:

```
135RouteGridPitch
    <string|LPP>layer
    <float>pitchValue

    [-isHard true|false]
    [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or as LPP.
pitchValue	Specifies the 135 degree route grid pitch value.

For example:

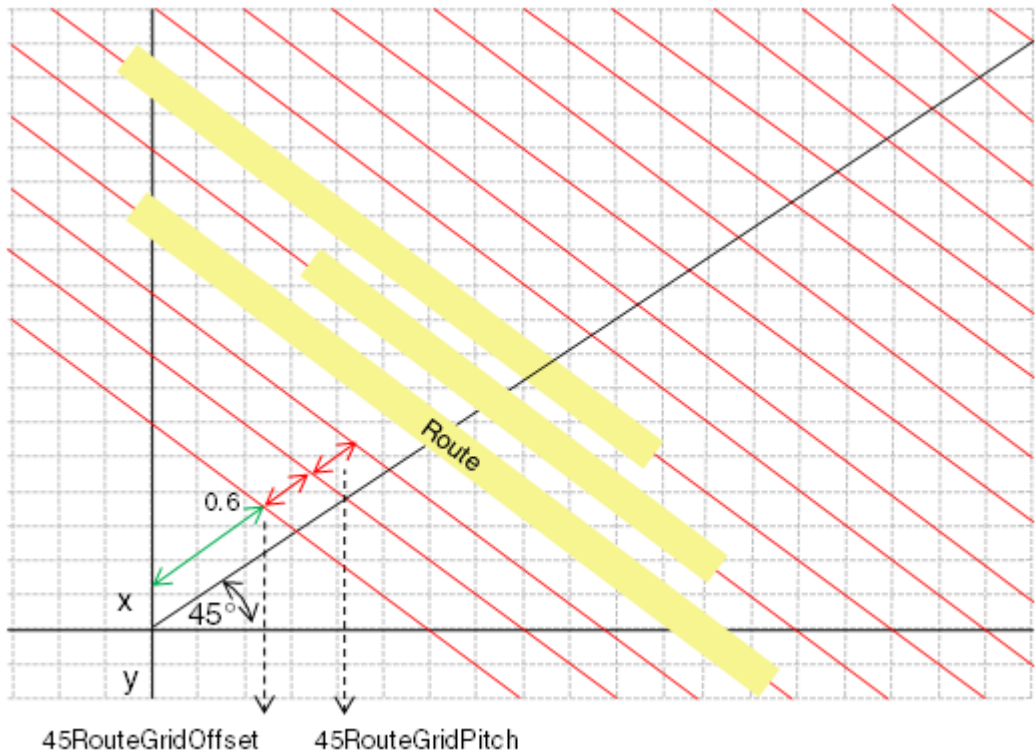
```
135RouteGridPitch    poly    0.3
```

45RouteGridOffset

This layer constraint specifies the route grid points on the right diagonal axis. The route grid points are completely defined using the 45RouteGridOffset and [45RouteGridPitch](#).

Note:

The right diagonal axis is found by rotating the horizontal axis 45 degrees counterclockwise.



Constraint Category	Routing
Constraint Type	Simple
OA Rule	oac45RouteGridOffset

Syntax:

```
45RouteGridOffset
  <string|LPP>layer
  <float>offsetValue

  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or as LPP.
offsetValue	Specifies the 45 degree route grid offset value.

For example:

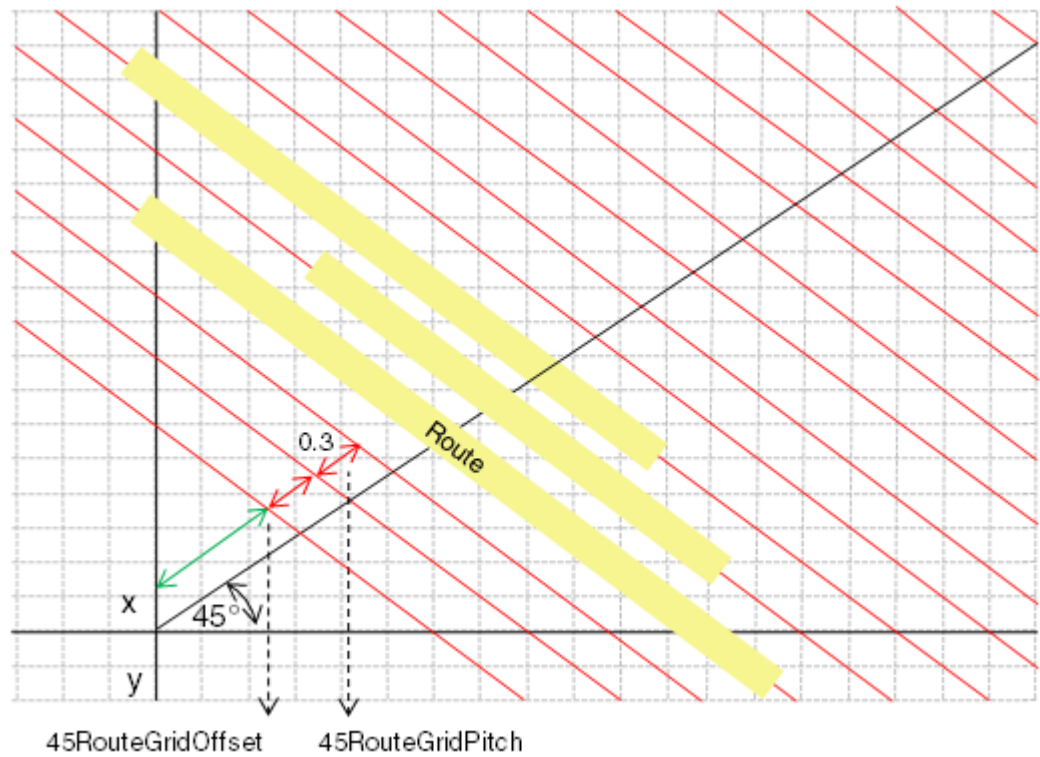
```
45RouteGridOffset    poly    0.6
```

45RouteGridPitch

This layer constraint specifies the route grid points on the right diagonal axis. The route grid points are completely defined using the 45RouteGridPitch and [45RouteGridOffset](#).

Note:

The right diagonal axis is found by rotating the horizontal axis 45 degrees counterclockwise.



Constraint Category	Routing
Constraint Type	Simple
OA Rule	oac45RouteGridPitch

Syntax:

```
45RouteGridPitch
    <string|LPP>layer
    <float>pitchValue

    [-isHard true|false]
    [-comment [<string rule_id>] <string rule_desc>]
```

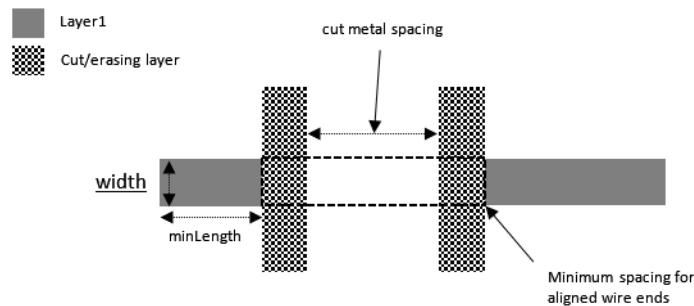
Argument	Description
layer	Layer can be defined as a string or as LPP.
pitchValue	Specifies the 45 degree route grid pitch value.

For example:

```
45RouteGridPitch      poly  0.3
```

alignedWireSpacing

The constraint specifies the spacing of aligned wire ends. The advanced processes that support metal cutting can have spacing requirements on the cutting shapes. In order to allow tools to focus on metal layer construction that will be DRC correct when doing the final cut metal shape insertion, a rule is defined which forces a minimum spacing between aligned wires.



Syntax:

```
alignedWireSpacing
  <string|LPP>layer
  <float>spacingValue
  -minLength float
  -width float
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

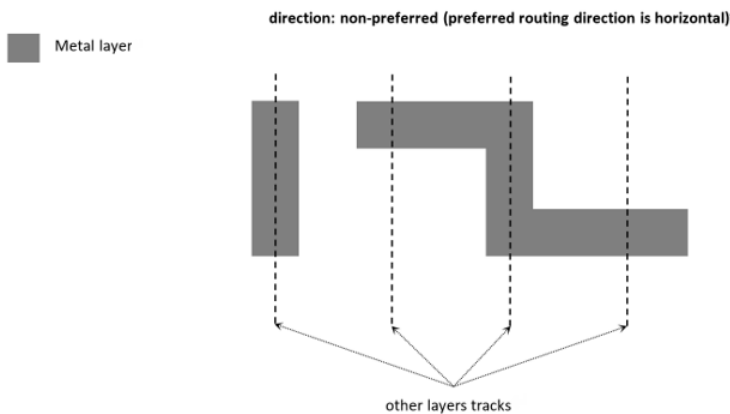
Argument	Description
layer	Name of the layer or LPP.
-minLength	One of the aligned wires must be less than or equal to this length for the constraint to apply. This is required.
-width	Wires must have width less than or equal to this value for the constraint to apply. This is required.

```
alignedWireSpacing    m1 0.02    -minLength 0.75 -width 0.05
```

alignToOtherLayerTracks

Some processes may have additional requirements for lower metal layer shapes oriented in non-preferred direction to be aligned with tracks of other layers. For example, poly layer. To model this kind of rules, this constraint is introduced with the constraint value listing the layers with tracks to be aligned to. For flexibility, the direction (preferred or non-preferred) of the shapes to be aligned is provided as a parameter of the constraint.

The rule applies only if there are no tracks defined for the layer under consideration in the specified direction but there are tracks defined for other layers (listed in the constraint value) and oriented in this direction. In this case, the constraint requires the centerlines of the metal shapes oriented in the given direction to be aligned with any other layer track centerline oriented in this direction. There are no requirements related to the metal shape width with respect to the track width.



Syntax:

```
alignToOtherLayerTracks
    <string|LPP>layer
    <List of strings>layers
    -direction preferred|nonpreferred

    [-isHard true|false]
    [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Name of the layer or LPP.
layers	Specifies the list of layers that are defining the other layers with tracks.

Argument	Description
-direction	The rule only applies to the metal shapes oriented in this direction. Valid values: 1 (preferred direction) or 2 (non-preferred direction). This is required.

Note:

The tool does not check these layerNames are metal shapes.

For example:

```
alignToOtherLayerTracks    PO (poly cpoly dpoly) -direction preferred
```

allowedCutClass

The `allowedCutClass` constraint controls the cut classes that are permitted for specific upper and lower metal layer widths. The constraint applies to cut layers only.

Syntax:

```
allowedCutClass
    <string|LPP>layer
    <2DTable>cutLayerWidths
    -cutClass <float><float>|<string>cutClassName
    [-upperLayer <string>]
    [-lowerLayer <string>]
    [-voltageRange frangeValue]

    [-isHard true|false]
    [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Name of the layer or LPP.

Argument	Description
cutLayerWidths	2D table that specifies the following: - Width of the lower layer, width of the upper layer - Indicator of whether the cut class is permitted (true if permitted, false if not permitted) 2-D table: <pre>defaultValue (((lowerWidth<float> upperWidth<float>) allowedValue<boolean> [-allowedLowerUpperDir (direction*)]))*) [2DTableOptions]</pre> -allowedLowerUpperDir: Lets you specify the permitted directions on the lower and upper via layers: lowerLayerDirection and upperLayerDirection. Note: - defaultValue: booleanValue (booleanValue [allowedDir]) - direction*: VV (vertical, vertical) VH (vertical, horizontal) HH (horizontal, horizontal) HV (horizontal, vertical)
-cutClass	The cut dimensions or the cutClass name. See also cutClass and cutClassList. When you import a technology file, the import process converts cutClass names to cut dimensions based on the first matched cutClass constraint in the same hierarchical constraint group. The export process converts the cutClass dimensions to the first matched cutClass constraint name in the same hierarchical constraint group.
-upperLayer	Name of the layer immediately above the cut layer. Default: The metal layer immediately above the cut layer in the layer stack
-lowerLayer	Name of the layer immediately below the cut layer. Default: The metal layer immediately below the cut layer in the layer stack
-voltageRange	If given, the rule only applies if the difference between voltages of the two shapes under consideration lies in this range in volts. To specify the ranges, see Range Data Syntax. See Voltage Dependent Design Rules.

Example 1:

```
allowedCutClass V1((0.03 0.05) true)
                  (0.03 0.08) false)
                  (0.12 0.05) false)
                  (0.12 0.08) true))
-cutClass 0.04 0.06
-upperLayer M2
-lowerLayer M1
```

Example 2:

```
allowedCutClass V1 true (((0.03 0.05) true)
                          (0.03 0.08) false)
                          (0.12 0.05) false)
                          (0.12 0.08) true))
-cutClass 0.04 0.06
```

```
-upperLayer M2
-lowerLayer M1
```

Example 3:

```
allowedCutClass V1 (true -allowedLowerUpperDir (VV HH))
  (((0 0) true -allowedLowerUpperDir (VH HH HV))
   ((0 0.16) true)
   ((0 0.165) false)
   ((0.13 0) false)
   ((0.13 0.16) true -allowedLowerUpperDir (VV))
   ((0.13 0.165) false)
   ((0.135 0) false)
   ((0.135 0.16) false)
   ((0.135 0.165) false))
-cutClass Vial_LRG
-upperLayer M2
-lowerLayer M1
```

allowedCutSpacing

Some processes have advanced spacing requirements between cuts.

Constraint Category	Spacing
Constraint Type	Complex
OA Rule	Custom Compiler Custom Constraint

Syntax:

```
allowedCutSpacing
  <string|LPP>layer
  <rangeValue | rangeValue2DTable> spacingRange
  -cutSpacingType edgeX | edgeY | corner | center | edge
  -cutClassList cutClassArray | cutClassNameList
  [-color colorName | any | same | diff | cycle]
  [-connectivityType
    anyConnectivityType |
    sameNetConnectivityType |
    contiguousShapesConnectivityType |
    directShapesConnectivityType |
    sameViaConnectivityType]
  [-exceptConnectivityType
    anyConnectivityType |
    sameNetConectivityType |
    contiguousShapesConnectivityType |
    directShapesConnectivityType |
    sameViaConnectivityType]
  [-xyParallelRunLength dualValue |
    [-xParallelRunLengthRange rangeValue]
    [-yParallelRunLengthRange rangeValue]]
```

```
[-viaType [selfAlignToSelfAlign|SA2SA] |
          [selfAlignToNonSelfAlign|SA2NSA] |
          [nonSelfAlignToNonSelfAlign|NSA2NSA]]
[-selfAlignEdgeType [selfAlignToSelfAlign|SA2SA] |
                    [selfAlignToNonSelfAlign|SA2NSA] |
                    [nonSelfAlignToNonSelfAlign|NSA2NSA]]
[-otherLayer layerName]
[-otherLayerWidth rangeData]
[-exactAligned <int>]

[-isHard true|false]
[-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Name of the layer or LPP.
spacingRange	2DTable: Specify when the spacing is dependent on the width of one of the two shapes: <pre>default<float>((width1<float> width2<float>) spacing<float>))) [2DTableOptions]</pre> rangeValue2DTable: Specify when the spacing is dependent of the width of both the shapes. <pre>default_range_value<float>((width1<float> width2<float>) rangeValue*) [2DTableOptions]</pre> For information about rangeValue, see Range Data Syntax.
-cutSpacingType	Specifies the type of spacing with the following: • edgeX: Specifies horizontal spacing between edges. • edgeY: Specifies vertical spacing between edges. • corner: Specifies corner to corner spacing. • center: Specifies center to center spacing. • edge: Specifies spacing between edges.
-cutClassList	An Array of values, each defining width and height of cut class for the specific index. Number of elements defines the size of the table for the constraint value. The cutClassList is represented as: <pre>cutClassArray<float1> <float2> cutClassName<string></pre> Note: If cutClass parameter is specified with cutClassName in the ASCII file, the technology importer looks for the first cutClass constraint specified in the same hierarchical constraint group to convert to cut class size. If no cutClass constraint exists, an error message is displayed. While exporting, the technology exporter looks for the first cutClass constraint specified in the same hierarchical constraint group to exists and converts the cut class size to cut class name. If no cutClass constraint exists, the cutClass parameter is exported with cut class size in dual value.

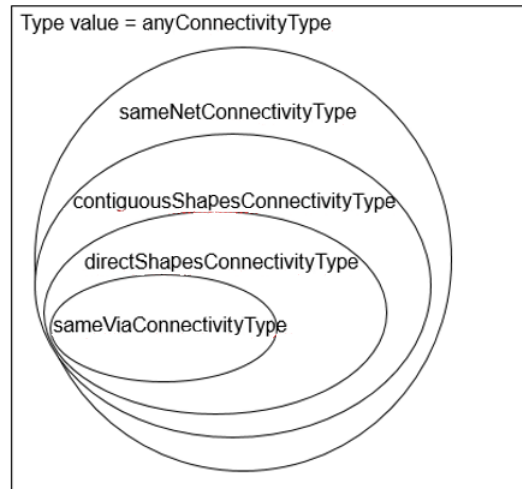
Argument	Description
-color	Defines the specific color of the cut shapes for which this constraint applies if the value is positive. A value of zero means any color. A value of -1 means same color and a value of -2 means diff color. A value of -3 means cycle color which is for the rules that need to avoid odd loops. The default value is diff. If the cut layer is uncolorable, this parameter is not considered.
-connectivityType	Indicates whether the constraint applies only if the cuts are on the same net, same metal, same wire segment, same via, or any.
-exceptConnectivityType	Specifies the type of connectivity between the two cuts that will cause the constraint to not apply.
-xyParallelRunLength	Minimum X and Y values for parallel run lengths. Each layer can have multiple constraints, each with different parallel run length requirements. . For -cutSpacingType of corner, the parallel overlap must be less than the X and Y parameter value for the constraint to apply. This parameter is ignored for -cutSpacingType of center.
-xParallelRunLengthRange	Defines the x parallel run length range within which the constraint to apply. The parameter is mutually exclusive with xyParallelRunLength.
-yParallelRunLengthRange	Defines the y parallel run length range within which the constraint to apply. The parameter is mutually exclusive with xyParallelRunLength.
-viaType	Enumeration indicating the type of vias this constraint applies to. The parameter specifies via types for the two cuts to be either self-aligned or non self-aligned. For example, if the value is selfAlignToNonSelfAlign, the constraint applies only when the first cut is self-aligned and the second cut is non self-aligned. This parameter supports the following three enumeration values: - selfAlignToSelfAlign SA2SA: Self-aligned via to self-aligned via. - selfAlignToNonSelfAlign SA2NSA: Self-aligned via to non self-aligned via. - nonSelfAlignToNonSelfAlign NSA2NSA: Non self-aligned via to non self-aligned via.
-selfAlignEdgeType	Enumeration indicating the type of edges this constraint applies to. An edge can either be self-aligned edge, which only exists for a self-aligned via, or non self-aligned edge, which could come from either a self-aligned or non self-aligned via. This parameter should be defined together with viaType. This parameter supports the following three enumeration values: - selfAlignToSelfAlign SA2SA: Self-aligned via to self-aligned edge. - selfAlignToNonSelfAlign SA2NSA: Self-aligned via to non self-aligned edge. - nonSelfAlignToNonSelfAlign NSA2NSA: Non self-aligned via to non self-aligned edge.
-otherLayer	The cut shapes must interact with material on the given layer, otherwise the constraint doesn't apply. The two cut shapes between which spacing is measured should interact with the same metal shape, so the constraint only applies when connectivityType = directShapesConnectivityType.

Argument	Description
-otherLayerWidth	If given along with otherLayer, the interacting material under the cut shapes must have a width in this range, otherwise the constraint doesn't apply. To specify the ranges, see Range Data Syntax .
-exactAligned	If given and the value is 1, the constraint applies only when two facing edges have the exact same size and also fully aligned. This constraint is ignored if the value is 0.

For more information on defining 1-D/2-D tables, see [Understanding and Interpreting 1-D, 2-D, and 3-D Table Values](#) section.

Note:

1. If `cutClass` parameter is specified with `cutClassName` in ASCII file, technology importer looks-up the first `cutClass` constraint specified in the same hierarchical constraint group to convert to cut class size. If a `cutClass` constraint does not exist then an error message is displayed.
2. `colorName` should be specified in controls colors section and the color mask value cannot bigger than the `numberOfColorMasks` value specified in the `layerRules` functions.
3. Report errors and stop importing when following scenarios occurs:
 - -same and -color parameters are defined together.
 - -selfAlignEdgeType parameter is specified and -viaType parameter is not specified.
 - When -exceptConnectivityType has a larger scope than -connectivityType, -connectivityType parameters conflict with -exceptConnectivityType parameters.



For example, when `-connectivityType = any` and `-exceptConnectivityType = sameVia`, no error is reported.

When `-connectivityType = sameVia` and `-exceptConnectivityType = sameNet`, errors are reported.

When `-connectivityType = sameNet` and `-exceptConnectivityType = sameNet`, errors are reported.

4. The warning message is displayed in the following situations:

- The constraint value table size is larger than $(\text{cutClassList size})^2$. For example, `cutClassList` has 1 element but table contains 2×2 elements.

Example 54

```
allowedCutSpacing v1 (((0 0) 0.234))
-cutClassList ((0.667 0.667))
-cutSpacingType edge
-color red
-sameSegExclusion
((( 0 0 ) ((0.3 0.5) (0.7 0.8 ))))
-xyParallelRunLength 0.03 0.05
```

Example 55

```
allowedCutSpacing v3 (((0 0) ((>=0.75) (<=0.9)))
((0 1) ((>=0.25) (<=0.3)))
((1 0) ((>=0.45)))
((1 1) ((>=0.95))))
-cutClassList ((0.04 0.06) (0.04 0.08))
-xyParallelRunLength -0.0105 0.00
-cutSpacingType edgeX
```

Example 56

```
allowedCutSpacing v4 (((0 0) 0.234))
-cutClassList ((0.667 0.667))
-cutSpacingType edge
-color red -exactAligned 1
-sameSegExclusion
(((0 0) ((0.3 0.5) (0.7 0.8))))
-xParallelRunLengthRange (>=0.1 <=0.2)
-yParallelRunLengthRange (0.7 <=0.9)
```

allowedLengthRange

Specifies the set of allowed length ranges for shapes on a layer. If the table value is used, the only allowed indices are the widths specified in a separate [allowedWidthRange](#) constraint.

Syntax:

```
allowedLengthRange
  <string|LPP>layer
  <rangeValue|rangeValue1DTable>spacingRange
  [-spacingDirection any|horizontal|vertical]
  [-color colorName|any]

  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Name of the layer or LPP.
spacingRange	rangeValue: Specify when the spacing is independent of the width of the shape. To specify the ranges, see Range Data Syntax . rangeValue1DTable: Specify when the spacing is dependent of the width of one of the two shapes. The value range 1D table is represented as: default_range_value (width rangeValue)
-spacingDirection	This parameter defines the spacing direction that represents the direction in which the constraint applies. The default value is any.
-color	This parameter represents the color mask.
Caution: You must specify the <i>colorName</i> in the Controls Section > colors group and the color mask value cannot be greater than the <i>numberOfColorMasks</i> value specified in the Layer Rules Section > functions group.	

rangeArrayValue example:

```
allowedLengthRange m1 ((>=0.1 <=0.2) (0.5 0.6) (0.7 <=0.9))
```

rangeArray1DTable example:

```
allowedLengthRange m1 ((0.08 0.1) (0.1 0.2))  
  ((0.1 ((0.1 0.2) (>=0.5 <=0.6)))  
    (0.5 ((>=0.7 0.8) (0.9 <=1.2))))  
-spacingDirection vertical -color red
```

For more information on defining range tables, see [Understanding and Interpreting 1-D, 2-D, and 3-D Table Values](#) section.

allowedEndToEndSpacing

This layer constraint specifies the allowable spacing ranges between end-to-end wires.

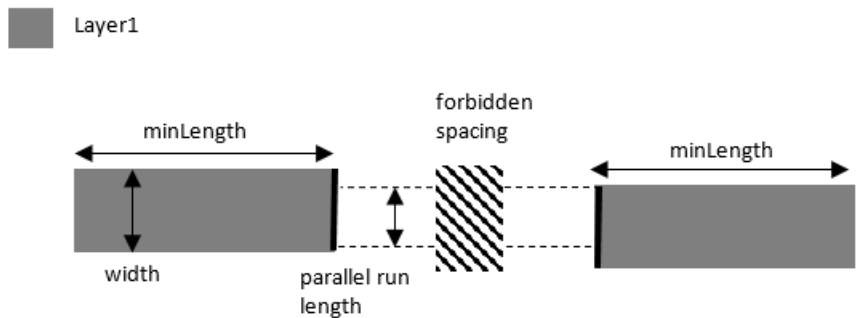
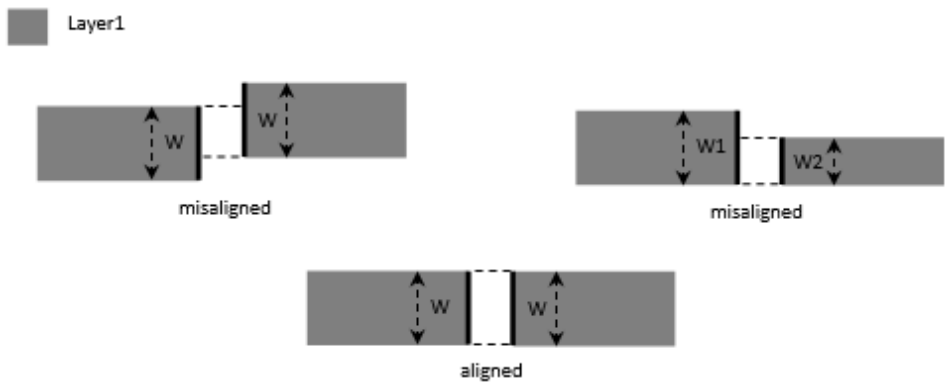


Figure 1 alignmentType Parameter



Constraint Category	Spacing
Constraint Type	Complex
OA Rule	Custom Compiler Custom Layer Constraint

Syntax:

```
allowedEndToEndSpacing
    <string|LPP>layer
    <rangeValue>spacingRange
    -parallelRunLength float
    -width float|-widthRange rangeData
    [-otherWidthRange rangeData]
    [-minLength float]
    [-color colorName|any|same|diff]
    [-spacingDirection any|horizontal|vertical]
    [-alignmentType any|aligned|misaligned]
    [-voltageRange frangeValue]

    [-isHard true|false]
    [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Name of the layer or LPP.
spacingRange	An array of ranges that specify the allowable area between the two end-to-end wires. To specify the ranges, see Range Data Syntax .
-parallelRunLength	Minimum parallel run length required in database units (dbu). The parallel run length between two end-of-line edges must be greater than or equal to this value for the constraints to apply.
-width	Maximum width of the two end-to-end wires.
-widthRange	The wire width must lie in the given range for the constraint to apply. This parameter or the <code>-width</code> parameter must be given, but not both.
-otherWidthRange	If specified, the constraint applies only when the second wire width lies in this range. The values are in database units. By default, the second wire can have any width.
-minLength	Minimum length of the two end-to-end wires.
-color	Color of the shape that cannot be within the forbidden spacing. Valid values are colorName, 0 (any), 1 (same), and 2 (diff). The colorName must be defined in the Controls Section . Default: 0
-spacingDirection	Defines the spacing direction in which this constraint applies: only horizontal, only vertical or both. Default: any
-alignmentType	Optional parameter defining restrictions on the line ends alignment. If this value is 1, the rule only applies to aligned line ends. If this value is 2, the rule only applies to misaligned line ends. Default: any. A pair of line ends facing each other is considered aligned if both wires have the same width, and length of projection of the line ends to each other is equal to the wire width. Otherwise, the line ends are misaligned. See Figure 1 .

Argument	Description
-voltageRange	If given, the rule only applies if the difference between voltages of the two shapes under consideration lies in this range in volts. To specify the ranges, see Range Data Syntax . See Voltage Dependent Design Rules .

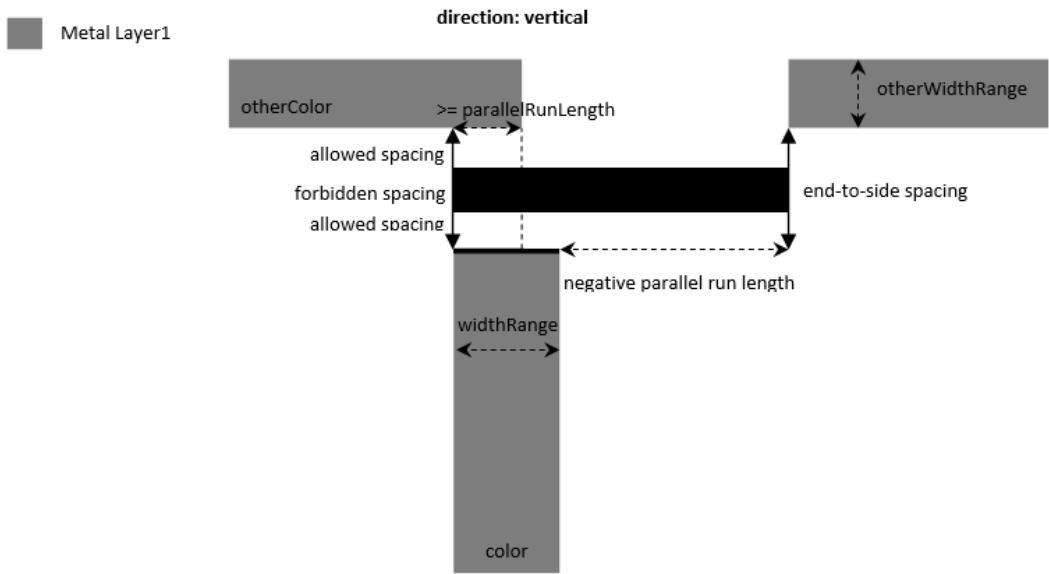
For example:

```
allowedEndToEndSpacing m1 ( (>=0.1 <=0.2) (0.5 0.6) (0.7 <=0.9))
    -parallelRunLength 0.55
    -width 0.35
    -minLength 0.17
    -color red
    -otherWidthRange (>=0.3 <=0.5) -alignmentType aligned

allowedEndToEndSpacing m1 ( (>=0.1 <=0.2) (0.5 0.6) (0.7 <=0.9))
    -parallelRunLength 0.55
    -spacingDirection vertical
    -widthRange (>=0.3 <=0.5)
    -minLength 0.17
    -color red
    -voltageRange (1.5 2)
```

allowedEndToSideSpacing

Some processes do not allow certain spacing ranges between end of line edges of one wire and a side of another wire depending on width and orientation of the wires. Optionally, the end of line edges can be extended before measuring the spacing which is modeled by negative parallel run length parameter. The spacing is measured as Manhattan distance.



Constraint Category	Spacing
Constraint Type	Complex
OA Rule	Custom Compiler Custom Layer Constraint

Syntax:

```
allowedEndToSideSpacing
    <string|LPP>layer
    <rangeValue>spacingRange
    [-widthRange rangeData]
    [-otherWidthRange rangeData]
    [-parallelRunLength float]
    [-spacingDirection any|horizontal|vertical]
    [-color colorName|any|same|diff]
    [-otherColor colorName]

    [-isHard true|false]
    [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Name of the layer or LPP.
spacingRange	An array of ranges, where each range value defines a range of allowed spacing from the end of line edge to the other wire side. To specify the ranges, see Range Data Syntax .
-widthRange	Optional parameter to define the explicit width range of the end-of-line wire in which this constraint applies.
-otherWidthRange	If specified, the spacing is checked only to the side of wires which have width in this range.
-parallelRunLength	If given, the constraint only applies when the parallel run between end of line of the two metal shapes is greater than or equal to this value in database units.
-spacingDirection	Defines the spacing direction in which this constraint applies: only horizontal, only vertical or both. Default: any
-color	If a positive value is given, defines the specific mask color of the end-of-line wire to which the constraint applies. Valid values are colorName, 0 (any), -1 (same), and -2 (diff). Default: any -1 (same): the rule only applies if the mask color of the other shape is the same as the mask color of the end-of-line wire. -2 (diff): the rule only applies if the mask color of the other shape is different from the mask color of the end-of-line wire. 0 (any): any color for both shapes.

Argument	Description
-otherColor	Optional parameter defining the specific mask color of the other shape. Must be a positive number. This can be specified only if the color parameter is also a positive number, otherwise, this parameter is ignored. If the color parameter is a positive number, and this parameter is omitted, its value is assumed to be equal to the value of the color parameter.

Note:

- Report errors and stop importing when following scenarios occurs:
 - If the -otherColor parameter is any (0), -1 (same), or -2 (diff).
 - The -otherColor parameter is specified and if the -color parameter is not a positive number.
 - The -otherColor parameter is specified if the -color parameter is not specified.

For example:

```
allowedEndToSideSpacing M1 ( (0.1 0.2) (0.2 0.3) )
    -widthRange (>=0.3 <=0.5)
    -otherWidthRange (>=0.4 <=0.6)
    -parallelRunLength 0.2
    -spacingDirection horizontal
    -color red
    -otherColor green
```

allowedNeighborViaGrid

The allowedNeighborViaGrid constraint facilitates the placement of adjacent vias on the same mask. The constraint can be helpful in advanced-node designs where rules have complex limitations.

Constraint Category	Spacing
Constraint Type	Complex
OA Rule	Custom Compiler Custom Layer Constraint

Syntax:

```
allowedNeighborViaGrid
    <string|LPP>layer
    <2DTable>validity
```

```
[-isHard true|false]
[-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer on which the cuts are placed. <code>layer</code> can be a string (layer name) or LPP.
validity	2D table that specifies the validity of the adjacent vias. In each table cell, valid values are: - Integer value - a or A (available) - o or O (occupied) - n or N (not available) Valid integer values that you can specify are: 0: Available 1: Occupied 2: Not available

For more information on defining 2-D tables, see [Understanding and Interpreting 1-D, 2-D, and 3-D Table Values](#).

Example 1:

```
allowedNeighborViaGrid v1
((0 1 1 2))
(1 2 0 0)
(2 0 1 1))
```

Example 1 would result in the following 2D table in the database:

	Column Index 0	Column Index 1	Column Index 2	Column Index 3
Row Index 0	0	1	1	2
Row Index 1	1	2	0	0
Row Index 2	2	0	1	1

Example 2:

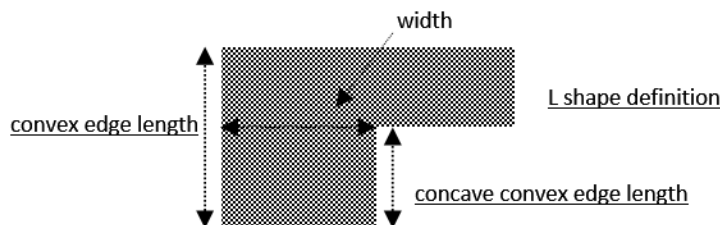
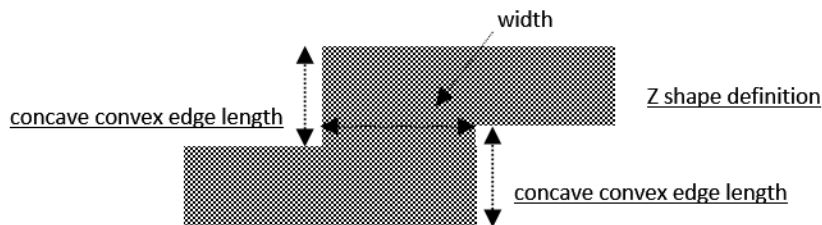
```
allowedNeighborViaGrid v1
((a o n))
(o n n)
(a a n))
```

Example 2 would result in the following 2D table in the database:

	Column Index 0	Column Index 1	Column Index 2
Row Index 0	a	o	n
Row Index 1	o	n	n
Row Index 2	a	a	n

allowedParallelSpanSpacing

Some processes have additional spacing requirements for parallel edges. Instead of basing the spacing on the wider shape and the parallel run length, the spacing is dependent on the span of the shape.



Syntax:

```
allowedParallelSpanSpacing
  <string|LPP>layer
  <rangeValue|rangeValue2DTable>spacingRange
  [-spacingDirection any|horizontal|vertical]
  [-exceptEOLMaxWidth float]
  [-exceptMinSpan float]
  [-exceptEOLMaxLength float]
  [-colorType any|same|diff]
  [-color colorName|any|same|diff]
```

```
[-excludedSpans rangeValue]
[-excludedWidthPRL rangeValue]
[-nonCornerNegativePRL true|false]
[-excludedZFlagPRL rangeValue1DTable]
[-excludedZFlagPrefDir rangeValue1DTable]
[-includeLWire true|false]
[-includeTWire true|false]
[-facingEdgeOnly booleanValue1DTable]

[-isHard true|false]
[-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Name of the layer or LPP.
spacingRange	rangeValue: Specify when the spacing is independent of the width of the shape. To specify the ranges, see Range Data Syntax. rangeValue2DTable: Specify when the spacing is dependent of the width of both the shapes. <pre>default_range_value<float>((width1<float> width2<float>) rangeValue*) [2DTableOptions]</pre>
-spacingDirection	This parameter defines the spacing direction that represents the direction in which the constraint applies. The default value is any.
-exceptEOLMaxWidth	If given, an EOL edge does not apply if the edge length is less than or equal to this value.
-exceptMinSpan	If given, defines the threshold span from an EOL edge for this constraint to apply. Spans from an EOL edge that are greater than or equal to this value makes this constraint not apply.
-exceptEOLMaxLength	If given, both edges adjacent to EOL edge must be less than this value, else the constraint does not apply.
-colorType	This is an optional parameter that dictates when the constraint applies depending on the color of the two shapes. A value of <i>any</i> means any color for either shape, <i>same</i> means that the shapes must be on the same color, and <i>diff</i> means the shapes must be on different colors. Default is <i>any</i> .
-color	This is an optional parameter that supercedes the colorType parameter. A value of zero means any color. A value of -1 means same color and a value of -2 means diff color. Default is <i>any</i>. The colorName should be specified in the controls colors section and the color mask value cannot bigger than the numberOfColorMasks value specified in the layerRules functions.
-excludedSpans	Spacings to exclude. Each element in the array should consist of the following: • width1 range • width2 range • length range • spacing range To specify the ranges, see Range Data Syntax.

Argument	Description
-excludedWidthPRL	Additional exclusion conditions. Each element in the array should consist of the following: • width1 range • width2 range • spacing range To specify the ranges, see Range Data Syntax .
-nonCornerNegativePRL	Performs non-corner checking of negative parallel run lengths when set to <code>true</code> . Default: <code>false</code>
-excludedZFlagPRL	Exclusions conditions for the Z shapes. The exclusion conditions are contained in a 1D table: (width concaveRange convexRange) where: • width: Width of the Z shape. • concaveRange: Range of the concave edge length. • convexRange: Range of the convex edge length. To specify the ranges, see Range Data Syntax .
-excludedZFlagPrefDir	Exclusions conditions for Z shapes that have the preferred direction. The exclusion conditions are contained in a 1D table: (width concaveRange convexRange) where: • width: Width of the Z shape. • concaveRange: Range of the concave edge length. • convexRange: Range of the convex edge length. To specify the ranges, see Range Data Syntax .
-includeLWire	If you specified <code>excludedZFlagPRL</code> and <code>excludedZFlagPrefDir</code> , then includes L-shaped wires if set to <code>true</code> . Default: <code>false</code>
-includeTWire	If you specified <code>excludedZFlagPRL</code> and <code>excludedZFlagPrefDir</code> , then includes T-shaped wires if set to <code>true</code> . Default: <code>false</code>
-facingEdgeOnly	Optional parameter which is an array of <code>oaBooleanValue</code> objects. The index into the array corresponds to the row index in the constraint value table. Default is <code>false</code> for each element. Note that we use 1D table as its syntax and the values are stored into the array correspondingly based on table's index (i.e. span).

For example:

```
allowedParallelSpanSpacing m1 ((0.08 0.1) (0.1 0.2))
                               (((0.1 0.2) ((0.3 0.5) (>=0.6 <=0.8))))
                               ((0.1 0.4) ((0.3 0.5) (>=0.6 <=0.8)))
                               ((0.3 0.2) ((0.3 0.5) (>=0.6 <=0.8)))
                               ((0.3 0.4) ((>=0.7 0.8) (>=0.9 <=1.2))))
-excludedSpans (
  ((>0.2 <0.5) (>=0.4 <=0.6) (>=0.3 <=0.9) (>=0.5 <=0.6))
  ((>1.02 <1.5) (>=1.4 <=1.6) (>=1.3 <=1.9) (>=1.5 <=1.6))
)
-excludedWidthPRL (
```



```

        ((>0.3 <0.7) (>=0.45 <=0.68) (>=0.34 <=0.59))
        ((>1.22 <1.58) (>=1.4 <=1.6) (>=1.3 <=1.9))
    )
    -nonCornerNegativePRL true
    -excludedZFlagPRL ((0.2 ((0.1 0.2) (>=0.5 <=0.6)))
                      (0.4 ((>=0.7 0.8) (0.9 <=1.2))) )
    -includeLWire true
    -color same
    -facingEdgeOnly ((0.1 true)
                    (0.3 false))

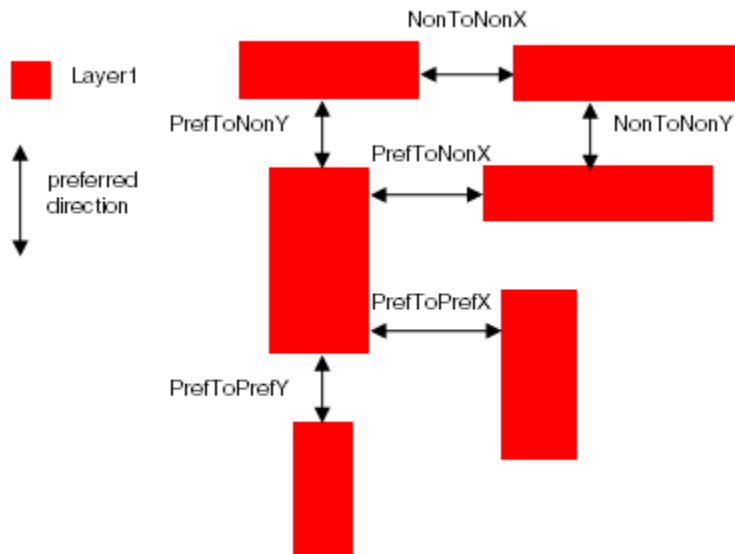
```

For more information on defining range tables, see [Understanding and Interpreting 1-D, 2-D, and 3-D Table Values](#) section.

allowedSpacingRange

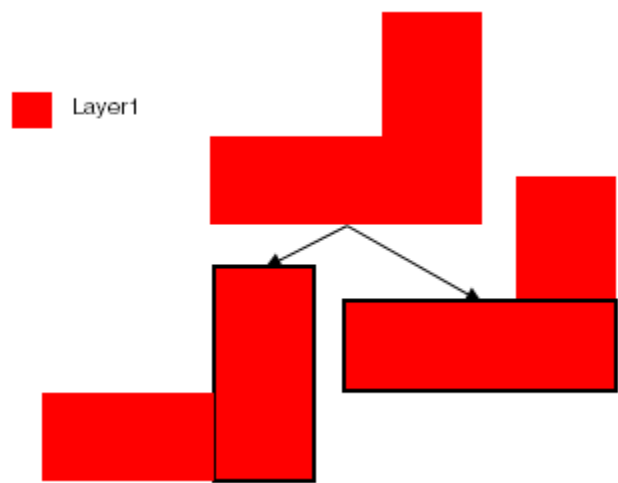
This layer constraint specifies a range of allowed spacings between shapes on a layer.

Some processes have layers with direction dependent spacing requirements. The direction of the two shapes and the perpendicular direction between the edges of the two shapes dictates the spacing. There could also be ranges of spacings that must not be used, also called exclusion ranges.



For non-rectangular shapes, maximal rectangles must be used when determining shape direction and each maximal rectangle must have the correct spacing with neighbor shapes.

Figure 2 L Shape Decomposed into 2 Maximal Rectangles



Constraint Category	Spacing
Constraint Type	Complex
OA Rule	oacAllowedSpacingRange

Syntax:

```
allowedSpacingRange
  <string|LPP>layer
  <rangeValue|rangeValue1DTable|
    rangeValue2DTable>spacingRange
  [-width <float>]
  [-parallelRunLengthTable true|false]
  [-spacingDirection any|horizontal|vertical]
  [-spacingType any|preferred|nonpreferred|
    preferredToNonpreferred|
    nonpreferredToPreferred]
  [-color colorName|any|same|diff]
  [jointSpacingRanges rangeValue|
    rangeValue1DTable|
    rangeValue2DTable]
  [jointLengthRanges rangeValue1DTable|
    rangeValue2DTable]

  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Name of the layer or LPP

Argument	Description
spacingRange	rangeValue: Specify when the spacing is independent of the width of the shape. To specify the ranges, see Range Data Syntax. rangeValue1DTable: Specify when the spacing is dependent of the width of one of the two shapes. The value range 1D table is represented as: <pre>default_range_value (width rangeValue)</pre> rangeValue2DTable: Specify when the spacing is dependent of the width of both the shapes. The value range 2D table is represented as: <pre>default_range_value ((width1 width2) rangeValue)</pre>
-width	Specifies if the constraint applies when both widths of adjacent shapes are less than the given width value.
-parallelRunLengthTable	If this parameter is true, then the index into the 1D table is the parallel run length between two shapes, rather than the width.
-spacingDirection	This parameter defines the spacing direction that represents the direction in which the constraint applies. The default value is any.
-spacingType	This parameter defines whether the spacing table is dependent on the preferred direction. The default value is any 0. See Applying Constraint at Wire Joints .
-color	If given, the rule only applies if both shapes have this mask color. A value of zero means any color. A value of -1 means same color. A value of -2 means diff color. Default: any.
-jointSpacingRanges	If given, this parameter is used to specify different spacing requirements at wire joints. See Applying Constraint at Wire Joints . The meaning of the table keys is the same as in the constraint table value.
-jointLengthRanges	If given, the spacing requirements specified by the <code>-jointSpacingRanges</code> parameter applies only at joints with joint length in the given ranges. See Applying Constraint at Wire Joints. This parameter can be specified only if the constraint value is a 1D or 2D table. The meaning of the table keys is the same as in the constraint table value. This parameter can be given only when the <code>-jointSpacingRanges</code> parameter is also given.

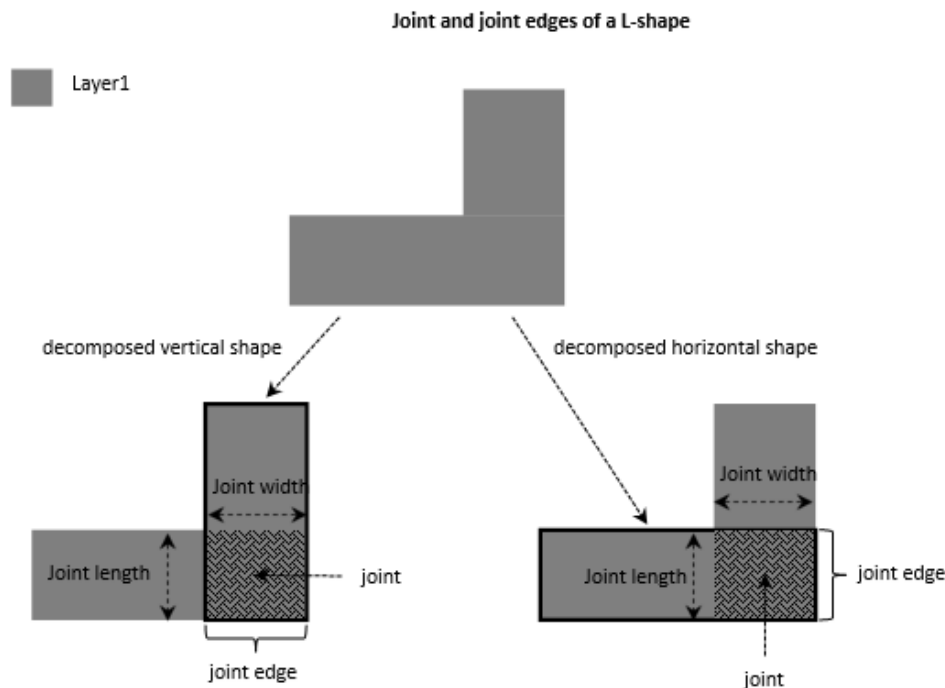
For more information on defining 1-D/2-D tables, see [Understanding and Interpreting 1-D, 2-D, and 3-D Table Values](#) section.

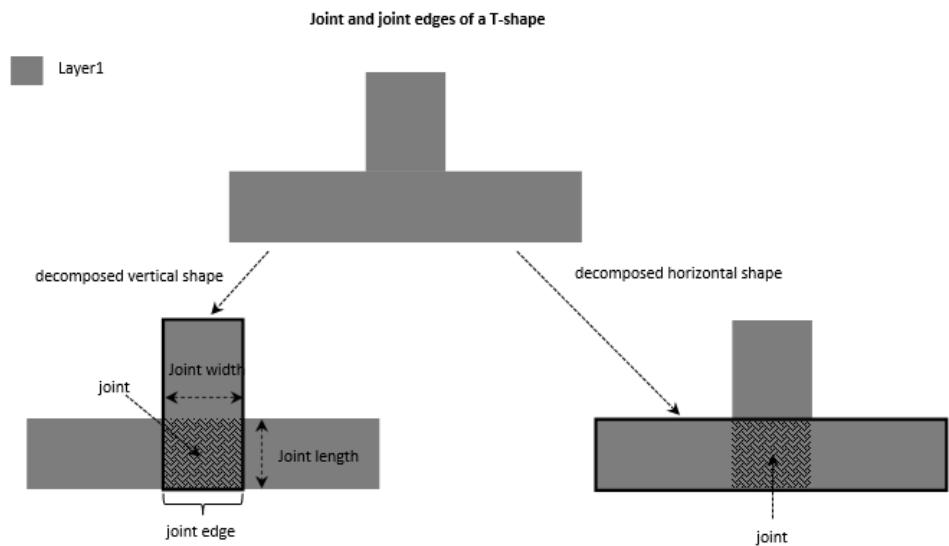
Applying Constraint at Wire Joints

By default, the spacing requirements specified by the constraint value are applied to all edges of the shapes on the metal layer. A wire joint is an overlap between horizontal and vertical wire segments forming a L-shape or a T-shape.

To apply the constraint at a wire join, the L-shape or T-shape is first decomposed into the horizontal and vertical wire segments as [Figure 2](#), then the spacing requirements are applied to the relevant edges of the decomposed shapes.

If a non-default value is specified for the `-spacingType` parameter, only one of the decomposed shapes (horizontal or vertical) is under consideration depending on the preferred routing direction of the metal layer. A joint edge is a sub-edge of the original shape at a joint which corresponds to the end-of-line edge of the orthogonal decomposed shape. If the `-jointSpacingRanges` parameter is specified, the spacing requirements for the joint edges are different and can be more restrictive or less restrictive than the constraint value.



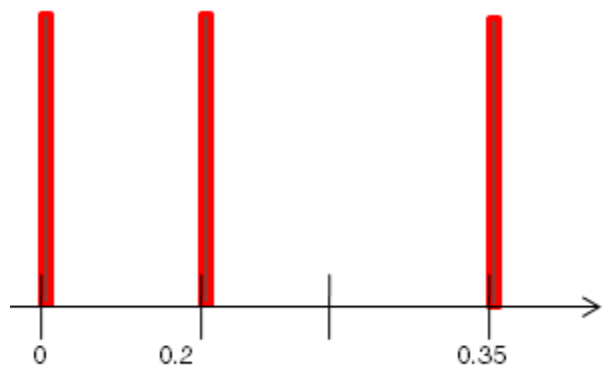


Note:

- `colorName` should be specified in controls colors section and the color mask value cannot be bigger than the `numberOfColorMasks` value specified in the `layerRules` functions.

In this example, the GATE - GATE spacing needs to be exactly 0.2 or 0.35 when the gate length ≤ 0.04 .

Rule	Description	Value (in microns)
PO.SP.2	GATE-GATE spacing when gatelength ≤ 0.04	=0.2 or =0.35

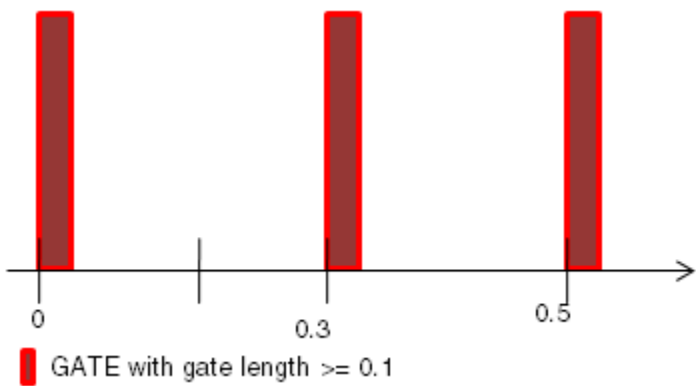


 GATE with Gate length ≤ 0.04

```
allowedSpacingRange GATE ([0.2 0.2][0.35 0.35] ) -width 0.04
```

In this example, the GATE -GATE spacing needs to be exactly 0.2 or > 0.5 when the gate length is >= 0.1.

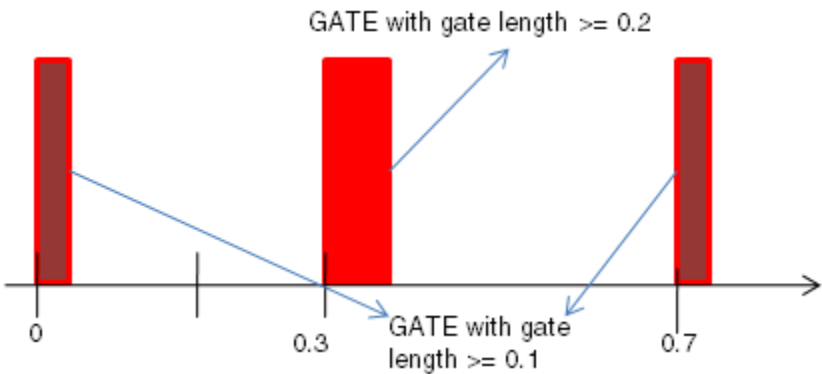
Rule	Description	Value (in microns)
PO.SP.2	GATE-GATE spacing when gatelength >=0.1	=0.2 or should be > 0.5



```
allowedSpacingRange GATE ([0.2 0.2]) ((0.1 ([0.2 0.2] (0.5 INFINITY))))
```

In this example, the GATE -GATE spacing needs to be exactly 0.3 or >0.6 when one of the gate length >=0.1 and the other gate length >=0.2.

Rule	Description	Value (in microns)
PO.SP.3	GATE-GATE spacing when one of the gate length >=0.1 and the other gate length >= 0.2	=0.3 or should be > 0.7



```
allowedSpacingRange GATE ([0.3 0.3]) (((0.1 0.2) ([0.3 0.3] (0.7 INFINITY))))
```

allowedSpanLengthRanges

The allowedSpanLengthRange constraint specifies the permitted span lengths for a given layer.

Constraint Category	Length and Width Process Rules
Constraint Type	Complex
OA Rule	oacAllowedSpanLengthRanges

Syntax:

```
allowedSpanLengthRanges
    <string|LPP>layer
    <rangeValue>value
    [-spacingDirection any|horizontal|vertical]
    [-orthogonalLength <float>]
    [-otherSpanLength <float>]

    [-isHard true|false]
    [-comment [<string rule_id>] <string rule_desc>]
```

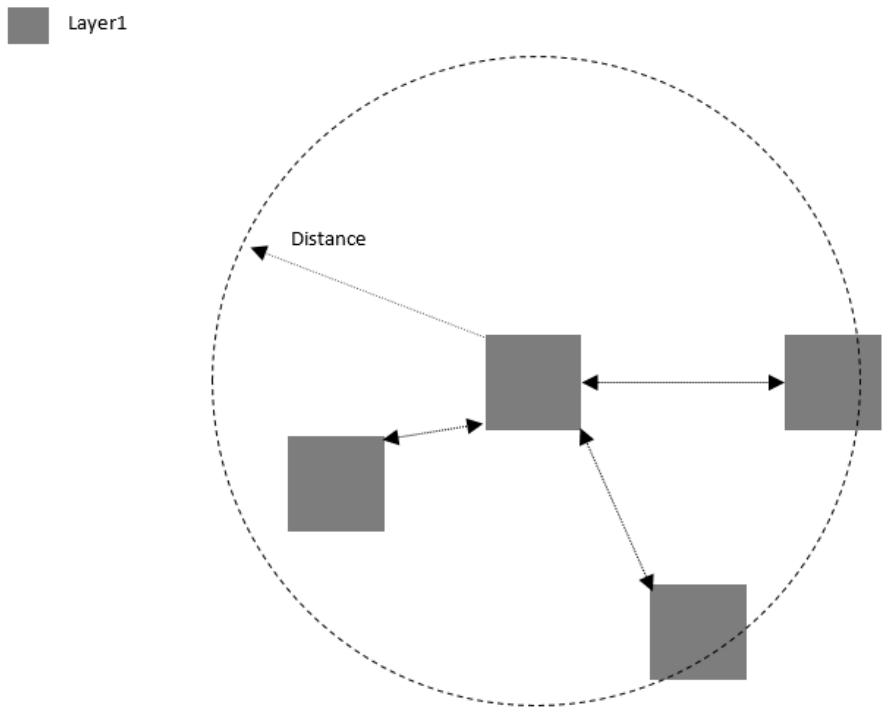
Argument	Description
layer	Layer can be defined as a string or as LPP.
value	List of valid ranges for the layer length. To specify the ranges, see Range Data Syntax .
-spacingDirection	Direction to which the constraint applies. The available values are any, horizontal, and vertical. Default is any.
-orthogonalLength	Distance between the two inside-facing corners. The constraint applies when this distance is greater than or equal to the specified value.
-otherSpanLength	Length of the layer. Define this argument only if the layer The constraint applies when the length is greater than or equal to the specified value. Define this argument only for layers that have a rectangularShapeDirection constraint.

For example:

```
allowedSpanLengthRanges m1 ( (>=0.1 <=0.2) (0.5 0.6) (0.7 <=0.9))
    -spacingDirection vertical
    -orthogonalLength 0.2
    -otherSpacingLength 0.4
```

allowedViaGroupSpacing

This layer constraint specifies the spacing requirements of a group of vias that are within a specific distance of each other.



Constraint Category	Spacing
Constraint Type	Complex
OA Rule	Custom Compiler custom layer constraint

Syntax:

```
allowedViaGroupSpacing
  <string|LPP>layer
  <rangeValue>spacingRange
  -distance float
  -cutClass <cutClassArray | cutClassNameList>

  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```


Argument	Description
layer	Name of the layer or LPP.
spacingRange	An array of ranges that specify the distances between the vias. To specify the ranges, see Range Data Syntax .
-distance	Distance within which a via must be to be part of the via group.
-cutClass	List of paired float values or a list of cutClass names. See cutClass and cutClassList . The constraint applies only in cases where the cuts comply with these specifications.

For example:

```
allowedViaGroupSpacing m1 ((>=0.1 <=0.2) (0.5 0.6) (0.7 <=0.9))
    -distance 0.55
    -cutClass ((0.06 0.06) (0.06 0.08))
```

allowedWidthRange

The allowedWidthRange constraint restricts a particular set of width values as legal for a given layer instead of defining a minimum and maximum width.

Constraint Category	Length and Width Process Rules
Constraint Type	Complex
OA Rule	oacAllowedWidthRange

Syntax:

```
allowedWidthRange
    <string|LPP>layer
    <rangeValue>value
    [-length float]
    [-spacingDirection any|horizontal|vertical]
    [-color colorName|any]
    [-orthogonal true|false]

    [-isHard true|false]
    [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or as LPP.
value	Define the list of valid ranges for the width of shapes on the layer. To specify the ranges, see Range Data Syntax .

Argument	Description
-length	This constraint does not apply to wires shorter than this length.
-spacingDirection	This parameter is used to model the direction to which the rule applies. The available values are any, horizontal, and vertical. Default is "any".
-color	The name of the color. Default is <code>any</code> . This constraint applies only to shapes on the given color mask. Caution: You must specify the <code>colorName</code> in the Controls Section > colors group, and the color mask value cannot be greater than the <code>numberOfColorMasks</code> value specified in the Layer Rules Section > functions group.
-orthogonal	This optional constraint parameter indicates that the orthogonal distance between two internal corners of a rectilinear shape must be greater than or equal to the smallest width in opposite direction width range constraint. If the parameter is not present, it should be interpreted as false. The default is <i>false</i> .
-lowerLayer	If given, the shape must interact with the material on the given layer, otherwise the constraint does not apply.
-lowerLayerWidth	If given along with <code>lowerLayer</code> , the interacting material under the shape must have a width in this range, otherwise the constraint does not apply. To specify the ranges, see Range Data Syntax .

Note:

The `colorName` must be defined in the [Controls Section](#), and the color mask value cannot be greater than the `numberOfColorMasks` value defined in the [functions](#) section.

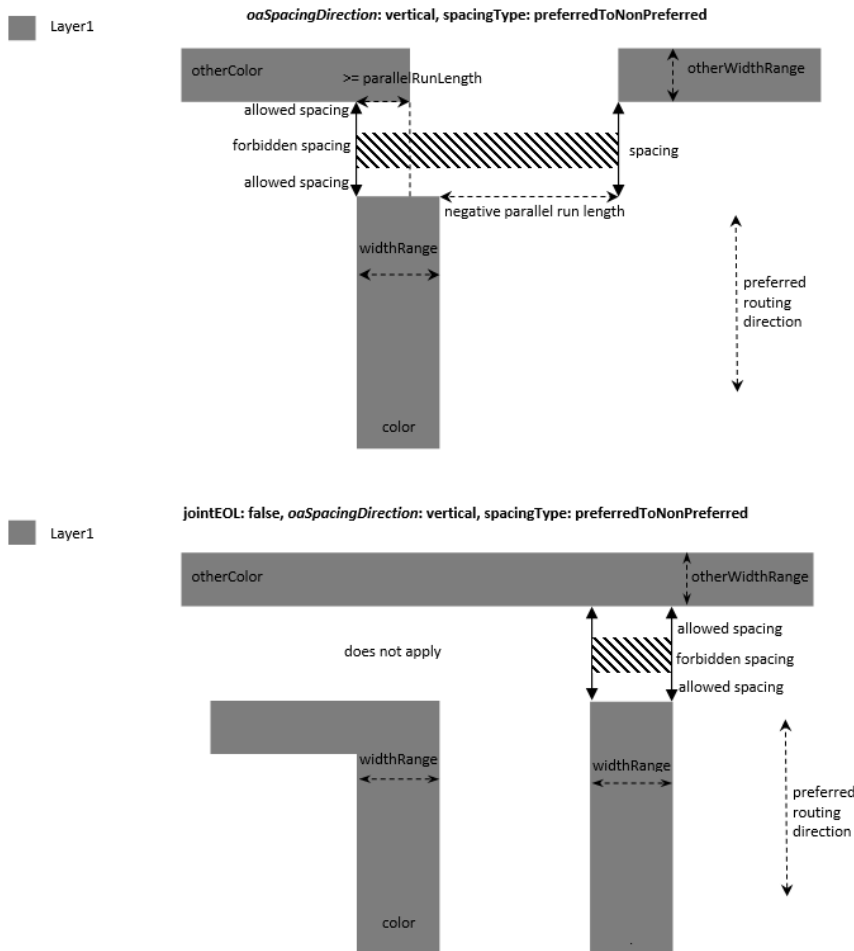
The warning message is displayed when `-lowerLayerWidth` is specified and `-lowerLayer` is not specified.

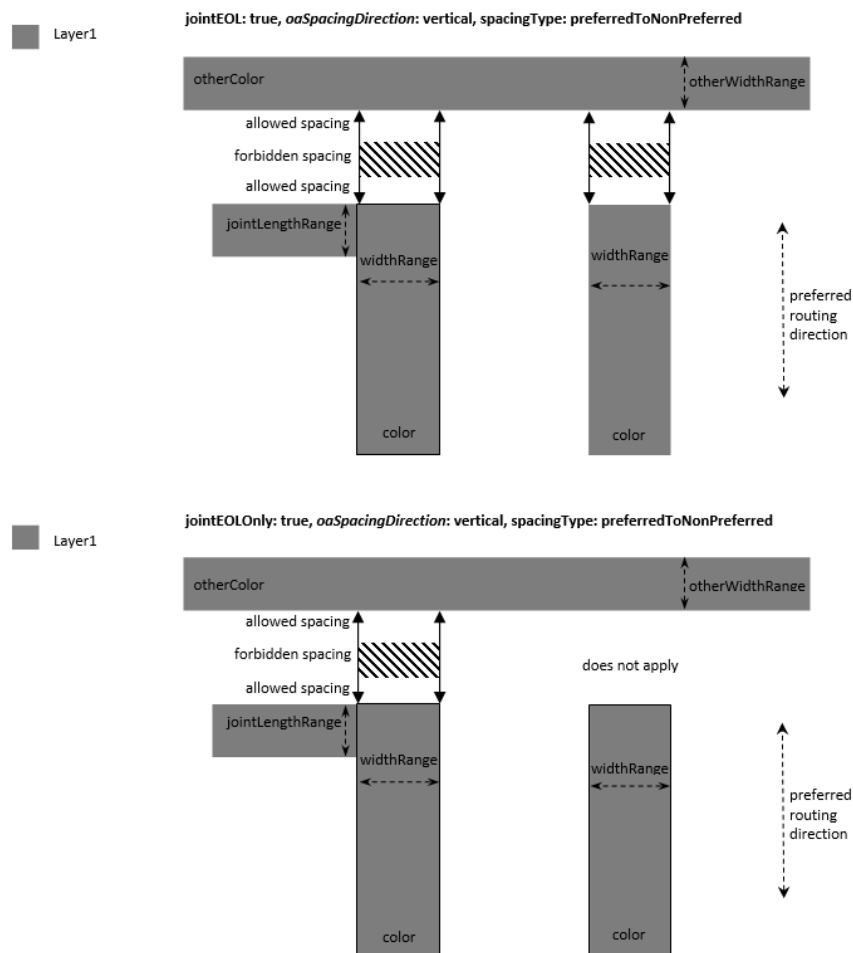
For example:

```
allowedWidthRange m1 ( (>=0.1 <=0.2) (0.5 0.6) (0.7 <=0.9))
```

allowedWireSpacing

A new `allowedWireSpacing` constraint is introduced with non-table value and parameters to specify applicable wire widths, colors, directions, and joint lengths. It is expected to have several constraints for the same metal layer with different values for different parameters. This constraint can be used to model spacing requirements for different geometrical configurations of wires (end-to-end, side-to-side, end-to-side) by specifying the `oaSpacingDirection` and `-spacingType` parameters. In case of negative parallel run length between the wires, the spacing is measured as Manhattan distance.





Constraint Category	Spacing
Constraint Type	Complex
OA Rule	Custom Layer Constraint

Syntax:

```
allowedWireSpacing
  <string|LPP>layer
  <rangeValue>value
  [-widthRanges rangeData]
  [-otherWidthRanges rangeData]
  [-parallelRunLength floatValue]
  [-spacingDirection any|horizontal|vertical]
  [-spacingType any|preferred|nonpreferred|
    preferredToNonpreferred|nonpreferredToPreferred]
  [-color colorName|any|same|diff]
```

```
[ -otherColor colorName]
[ -jointEOL true|false]
[ -jointEOLOnly true|false]
[ -jointLengthRange rangeData]
[ -voltageRange frangeData]
[ -eolMinLength floatValue]
[ -eolTwoSides true | false]

[ -isHard true|false]
[ -comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or as LPP.
value	Array of range values, where each range value defines a range of allowed spacing between two wires.
-widthRanges	Optional parameter specifying an array of one or more integer ranges in database units. If given, the constraint applies only if the width of the first wire lies in one of these ranges.
-otherWidthRanges	Optional parameter specifying an array of one or more integer ranges in database units. If given, the constraint applies only if the width of the second wire lies in one of these ranges.
-parallelRunLength	The constraint only applies when the parallel run between the two wires is greater than or equal to this value in database units. Default: 0.
-spacingDirection	Defines the spacing direction in which this constraint applies: only horizontal (1), only vertical (2) or both horizontal and vertical directions (0). Default: any.
-spacingType	Defines the orientation of the wires (with respect to the preferred routing direction of the metal layer) the rule applies to. Valid values: • any • preferred • nonpreferred • preferredToNonpreferred • nonpreferredToPreferred Default: any.
-color	If a positive value is given, defines the specific mask color of the first wire to which the constraint applies. • A value of -1 (same) means the rule only applies if the mask color of the second wire is the same as the mask color of the first wire. • A value of -2 (diff) means the rule only applies if the mask color of the first wire is different from the mask color of the second wire. • A value of zero means any color for both wires. Default: any.

Argument	Description
-otherColor	Optional parameter defining the specific mask color of the second wire. Must be a positive number. This can be specified only if the color parameter is also a positive number, otherwise, this parameter is ignored. If the color parameter is a positive number, and this parameter is omitted, its value is assumed to be equal to the value of the color parameter.
-jointEOL	When true, if the rule applies to line ends, it also applies to merged line ends at wire joints. When false, the rule does not apply to merged line ends at wire joints (see figures above). This parameter cannot be true when the -jointEOLOnly parameter is also true. Default: false.
-jointEOLOnly	If true, the rule only applies to merged line ends at wire joints (see figures above). This parameter cannot be true when the -jointEOL parameter is also true. Default: false.
-jointLengthRange	This parameter can be given only if the -jointEOL parameter is true, or if the -jointEOLOnly parameter is true. Defines the length of merged line ends at wire joints to which the rule applies (see figures above).
-voltageRange	If given, the rule only applies if the difference between voltages of the two shapes under consideration lies in this range in volts.
-eolMinLength	If given, the rule only applies to end-of-line edges (edges between two convex corners) of the first wire with length of the adjacent edge that is greater than or equal to this value in database units.
-eolTwoSides	This parameter is used only in conjunction with the eolMinLength parameter. If true, both edges adjacent to the first wire end-of-line edge must have length greater than or equal to the eolMinLength parameter value for the rule to apply. If false, the constraint applies when at least one edge adjacent to the first wire end-of-line edge has length greater than or equal to the eolMinLength parameter value. If the eolMinLength parameter is not given, this parameter has no effect.

Note:

- Report errors and stop importing when following scenarios occurs:
 1. If the -otherColor parameter is not a positive number.
 2. The -otherColor parameter is specified and if the -color parameter is not a positive number.
 3. The -otherColor parameter is specified if the -color parameter is not specified.
 4. The -jointEOL parameter is specified as true and the -jointEOLOnly parameter is specified as true.

5. The `-jointLengthRange` parameter is specified if the `-jointEOL` parameter or `-jointEOLOnly` parameter is not true.
6. The `-eolTwoSides` parameter is specified if the `-eolMinLength` parameter is not specified.

For example:

```
allowedWireSpacing m1 ( (0.08 0.1) (0.1 0.2) )
                      -widthRanges ((>=0.3 <=0.5) (0.5 0.6) (0.7 <=0.9))
                      -otherWidthRanges ((>=0.3 <=0.4) (0.4 0.5))
                      -parallelRunLength 0.3
                      -spacingDirection vertical -color red -otherColor blue
                      -jointEOL true

allowedWireSpacing m1 ( (>=0.2) )
                      -spacingType nonpreferredToPreferred
                      -widthRanges ((==0.076))
                      -parallelRunLength -0.0755
                      -spacingDirection vertical
                      -eolMinLength 0.0905
                      -eolTwoSides true
```

antenna

The antenna layer constraint specifies the maximum antenna ratios for one or more antenna models (oxide thicknesses) for a particular layer. This constraint holds a value that specifies the maximum allowed ratio of the layer's antenna area to the gate area (`gateAreaRatio`), and specifies how that maximum ratio changes if a diode is present on the wire (`diodeAreaRatio`).

Constraint Category	Antenna
Constraint Type	Complex
OA Rule	oaAntenna

Syntax:

```
antenna <string|LPP>layer
      <string> defaultModel(
        -diodeRatio <1DTblValue>diodeRatio
        -gateRatio <float>gateRatio
        [-isSide true|false])

      [<string> secondModel(
        -diodeRatio <1DTblValue>diodeRatio
        -gateRatio <float>gateRatio
        [-isSide true|false])]
```

```
[<string> thirdModel(  
  -diodeRatio <1DTblValue>diodeRatio  
  -gateRatio <float>gateRatio  
  [-isSide true|false]])  
  
[<string> fourthModel(  
  -diodeRatio <1DTblValue>diodeRatio  
  -gateRatio <float>gateRatio  
  [-isSide true|false]])  
  
  -antennaAreaFactor float  
  -antennaDiffPlusFactor float  
  -antennaDiffMinusFactor float  
  -antennaDiffAreaReduceFactor float1DTblValue  
  -antennaCumRoutingPlusCut true|false  
  
[-isHard true|false]  
[-comment [<string rule_id>] <string rule_desc>]
```

Where diodeRatio is a 1DTblValue:

```
(<float>default_value ((<float>diodeArea  
  <float>diodeAreaRatio)*))
```

Argument	Description
layer	Layer can be defined as a string or as LPP.
-diodeRatio	1DLookupTbl where: • key = diode area • value = the maximum allowed antenna area if connected to the diffusion diode.
-gateRatio	A ratio to the gate area of the maximum allowed, accumulated area (that is not connected to the diffusion diode) of the specific layer attached to a gate.
-isSide	If area is true then sidewall else top surface. Default is "false".
-antennaAreaFactor	The metal factor used to calculate antenna area: • The metal sideFactor, if isSide() • The metal areaFactor, if !isSide() Value range is from 0.0 to 1.0. Default is 1.0
-antennaDiffPlusFactor	The factor indicating how the diffusion area connected to the Net is added to the gate area in antenna/gate ratio calculations. Default is 0.
-antennaDiffMinusFactor	The factor (default==0) indicating how the diffusion area connected to the Net is subtracted from the antenna area in calculating antenna/gate ratio.

Argument	Description
-antennaDiffAreaReduceFactor	A piecewise-linear model that applies to the metal and cut area sum used in the Antenna, cumulativeMetalAntenna , and cumulativeViaAntenna constraints. Table values (default==1) are usually between 0 and 1, and if nonzero, the subtraction of the diffusion area attached to the Net, multiplied by the antennaDiffMinusFactor, must be performed after the metal and cut areas are multiplied by the diffAreaReduceFactor.
-antennaCumRoutingPlusCut	• Default is false. • true: Calculate the CAR for a given Layer by adding the CAR of the Layer immediately below it (metal for cut Layers, cut for metal Layers). • false: Calculate the CAR for a given Layer by adding the CAR of next Layer of similar material below it (metal for metal Layers, cut for cut Layers).

This example defines a antenna constraint with the default model:

```
antenna V1 defaultModel(
    -diodeRatio(5((0.1 10) (0.4 20)))
    -gateRatio 10
    -isSide true)
-antennaAreaFactor 0
-antennaCumRoutingPlusCut true
-antennaDiffAreaReduceFactor(0.2((0.1 0.3)
                                (0.5 0.5)))
```

For more information on defining range tables, see [Understanding and Interpreting 1-D, 2-D, and 3-D Table Values](#) section.

areaCapacitance

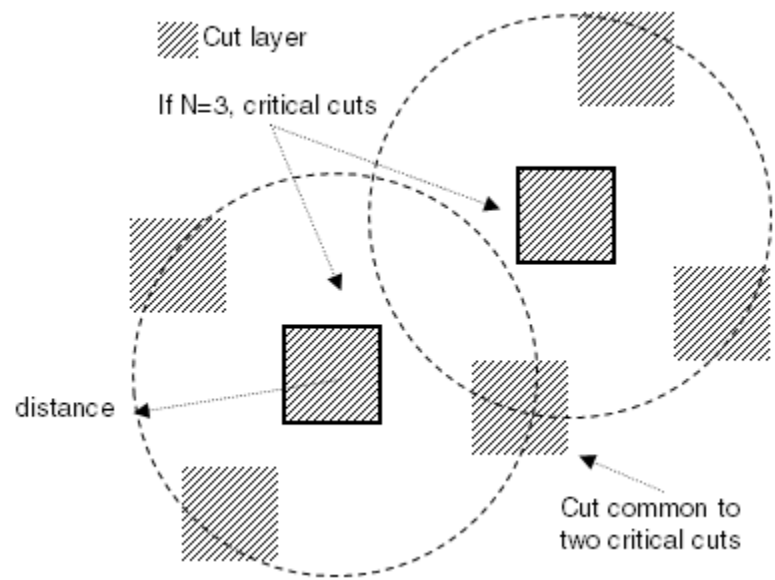
The areaCapacitance layer constraint specifies the capacitance for each square unit, in picofarads per square micron. You use the measurement to determine the wire-to-ground capacitance.

To set the design option to control whether the tool calculates and displays static or dynamic measurement results as your cursor moves on the canvas, see Layout Cursor Tab.

commonNeighborViaCount

Advanced processes have spacing requirements of cuts that are simultaneously close to two different groups of cuts that have been classified as critical cuts. A critical cut is one which has N number of cuts on the same layer and potentially same mask which are closer than a given distance measured center to center. The cuts that are within this

distance are called critical neighbor cuts and processes require that all critical neighbor cuts be near at most one critical cut.



Constraint Category	Via Construction
Constraint Type	Complex
OA Rule	Custom Compiler Custom Layer Constraint

Syntax:

```
commonNeighborViaCount
  <string|LPP>layer
  <int>numOfCuts
  -distance float
  -count int
  [-color any|same|diff]
  [-cutClass <cutClassArray | cutClassNameList>]

  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or as LPP.
numOfCuts	The number of critical cuts that are allowed to be within the given distance.
-distance	Distance threshold for classify neighbors. Neighbor cuts of the same cut class, It must be less than this value measured center to center.

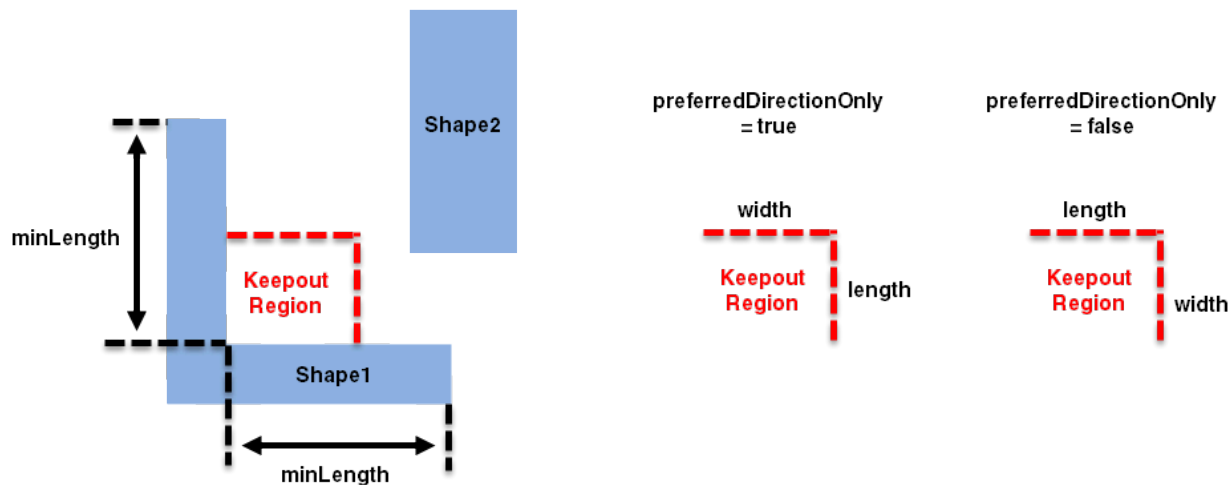
Argument	Description
-count	The number of neighbor cuts within distance required for a cut to be considered critical.
-color	Defines the mask color for all cuts to which this constraint applies to. A value of <i>any</i> means any color for either shape, <i>same</i> means that the shapes must be on the same color, and <i>diff</i> means the shapes must be on different colors. Default is <i>any</i> . Caution: You must specify the <i>colorName</i> in the Controls Section > colors group, and the color mask value cannot be greater than the <i>numberOfColorMasks</i> value specified in the Layer Rules Section > functions group.
-cutClass	List of paired float values or a list of cutClass names. See cutClass and cutClassList . The constraint applies only in cases where the cuts comply with these specifications.

For example:

```
commonNeighborViaCount v1 5 -distance 0.55
                             -count 3
                             -color green
                             -cutClass 0.4 0.6
```

concaveCornerKeepout

The `concaveCornerKeepout` constraint prevents neighboring shapes from being placed within the defined keepout region of concave shapes on the same layer. The keepout region is adjacent to the inside corner of concave shapes. Both routes and shields are aware of the constraint.



Constraint Category	Spacing
Constraint Type	complex
OA Rule	Layer

Syntax:

```
concaveCornerKeepout
    <string|LPP>layer
    (<float>width <float>length)
    [-minLength <float>length]
    [-preferredDirectionOnly]

    [-isHard true|false]
    [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	The layer on which the concave shape and neighboring shapes to which the constraint applies. The layer can be defined as a string or a layer-purpose pair.
width	The width of the keepout region.
length	The length of the keepout region.
-minLength	The constraint applies when the length of the outer sides of the concave shape is greater than or equal to the specified <code>minLength</code> value.
-preferredDirectionOnly	The preferred direction of the keepout region: • true: The width is horizontal and the length is vertical. • false: The width is vertical and the length is horizontal. Default: false

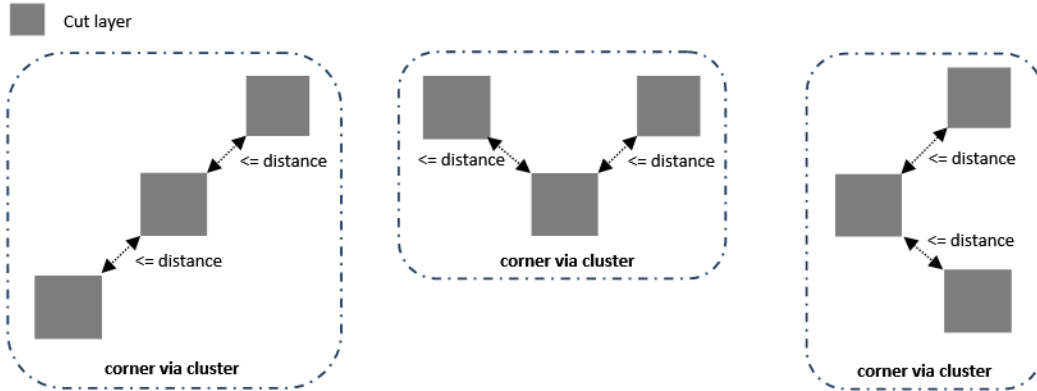
In the following example, the constraint specifies that if the outer sides of the concave shape on the m1 layer is greater than 5, the keepout region would apply. The keepout region is 0.02 wide x 0.06 long.

```
concaveCornerKeepout m1 (0.02 0.06) -minLength 0.05 -preferredDirectionOnly true
```

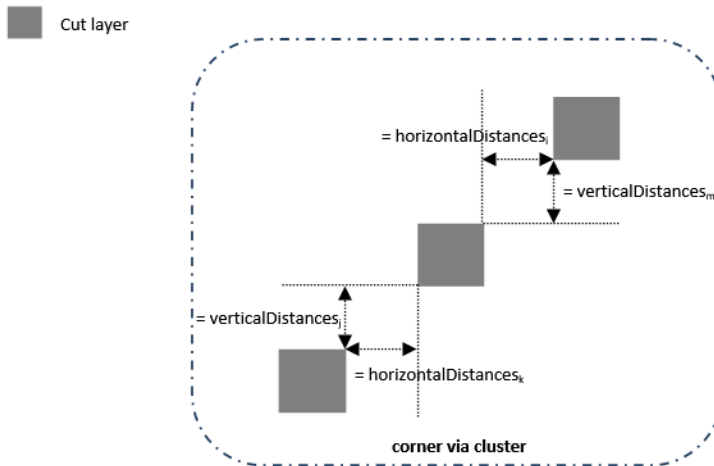
cornerViaClusterSpacing

Advanced processes may require different cut spacing between vias forming a corner via cluster and other via cuts. A corner via cluster is a group of vias each of which has an adjacent via in the group with negative parallel run and satisfying particular requirements on the diagonal distance to the adjacent cut. The diagonal distance between the adjacent cuts can be measured as one corner-to-corner value, or as two separate values along

X-axis and Y-axis. If the distance parameter is given, two adjacent via cuts belong to the same corner via cluster if corner-to-corner spacing between them is less than or equal to the distance. Examples below show how the distance parameter is used.

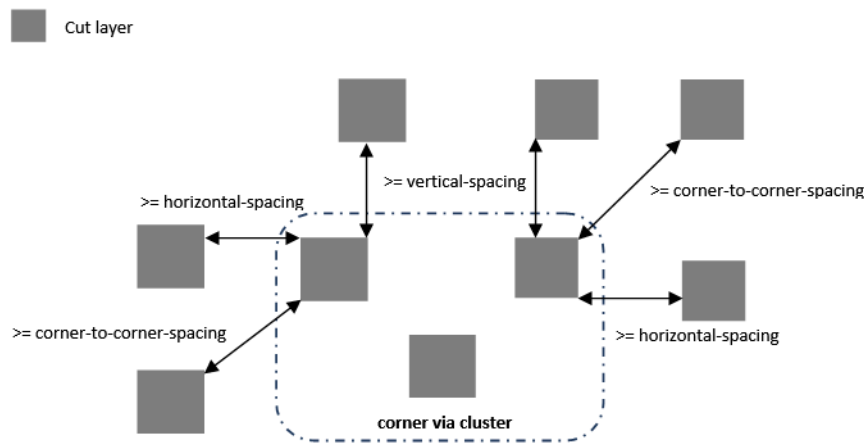


If the `horizontalDistances` and `verticalDistances` parameters are given, two adjacent via cuts belong to the same corner via cluster if the horizontal distance between them is one of the distances specified in the `horizontalDistances` parameter and if the vertical distance between them is one of the distances specified in the `verticalDistances` parameter. An example below shows how the `horizontalDistances` and `verticalDistances` parameters are used.

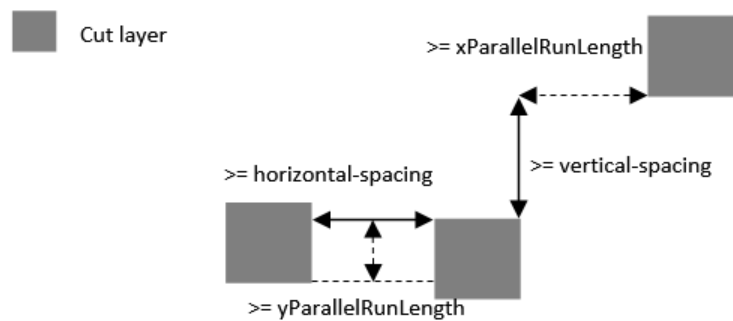


The rule requires a minimum spacing from a cut in a corner via cluster to another cut. The minimum spacing is provided as three separate values: corner-to-corner-spacing, horizontal-spacing, and vertical-spacing. If another cut has negative parallel run with a cut in a corner via cluster, the diagonal spacing between the cuts - measured corner-to-corner - must be greater than or equal to the corner-to-corner-spacing value. If projections of the cuts to Y-axis overlap each other, the horizontal spacing between the cuts must be greater than or equal to the horizontal-spacing value. If projections of the cuts to X-axis overlap

each other, the horizontal spacing between the cuts must be greater than or equal to the vertical-spacing value.



By default, horizontal-spacing and vertical-spacing are required for cuts with any non-negative parallel run length. Optionally, a particular value can be specified for the parallel run length along X or Y axis, including a negative value.



If both this rule and Non-Isolated Via Corner Spacing rule are defined, the corner via cluster definition becomes a waiving condition for the Non-Isolated Via Corner Spacing rule.

Constraint Category	Spacing
Constraint Type	Complex
OA Rule	Custom Compiler Custom Layer Constraint

Syntax:

```
cornerViaClusterSpacing
  <string|LPP>layer
  <float>(cornerSpacing horizontalSpacing verticalSpacing)values
  -numCuts intValue
```

```
-distance floatValue|-horizontalDistances (spacingValue ...)
                        -verticalDistances (spacingValue ...)
                        |-distanceWithinRange rangeData
[-maxNumCuts intValue]
[-color colorName | any | same | diff]
[-cutClass dualValue|cutClassName]
[-xParallelRunLength floatValue]
[-yParallelRunLength floatValue]
[-xyMaxAdditionalDistances dualValue]
[-mergeCutDistanceRange rangeData]

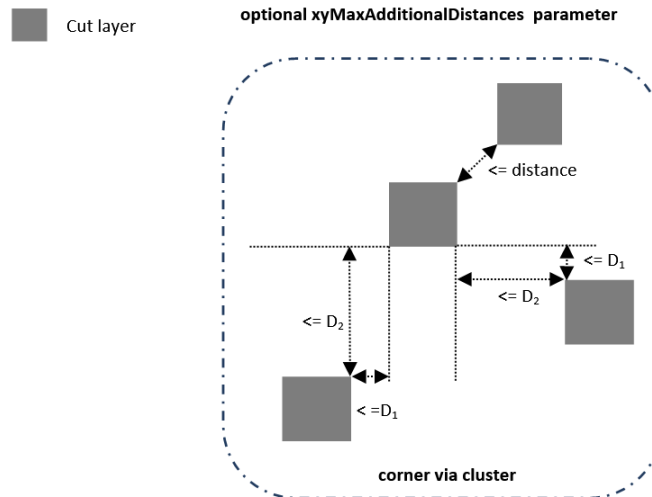
[-isHard true|false]
[-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	The layer can be defined as a string or a layer-purpose pair.
values	Array containing three oalntValue objects. The spacing values in database units, in this order, are corner-to-corner-spacing, horizontal-spacing, and vertical-spacing.
-numCuts	The minimum number of cuts in a corner via cluster required for the rule to apply.
-distance	The distance threshold in database units to determine if adjacent vias belong to a corner via cluster. This parameter or the horizontalDistances/verticalDistances parameters must be given, but not both.
-horizontalDistances	Array of oalntValue objects defining a set of discrete values in database units. Distance between adjacent cuts in a corner via cluster measured along X-axis can only be one of these values. This parameter or the distance parameter must be given, but not both.
-verticalDistances	Array of oalntValue objects defining a set of discrete values in database units. Distance between adjacent cuts in a corner via cluster measured along Y-axis can only be one of these values. This parameter or the distance parameter must be given, but not both.
-distanceWithinRange	This can be used instead of the distance parameter to specify range of distances in database units to determine if adjacent vias belong to a corner via cluster.
-maxNumCuts	If given, it is a violation to have more than this number of cuts in a corner via cluster.
-color	The constraint only applies when all cut shapes under consideration have this mask color. A value of zero means any color. A value of -1 means same color. A value of -2 means diff color. Default: 0. Note: When color parameter is -2, the spacing is measured between different color cuts from two different via clusters. However, only via cuts of the same color form a via cluster. For example, the color of all via cuts inside each cluster is the same.

Argument	Description
-cutClass	If given, the constraint only applies to cut shapes from this cut class.
-xParallelRunLength	The <code>vertical-spacing</code> applies to cuts with common parallel run length along X axis greater than or equal to this value. Default: 0.
-yParallelRunLength	The <code>horizontal-spacing</code> applies to cuts with common parallel run length along Y axis greater than or equal to this value. Default: 0.
-xyMaxAdditionalDistances	A pair of values (D1, D2). This parameter can be used together with the <code>distance</code> or <code>distanceWithinRange</code> parameters to specify additional condition for forming a corner via cluster. If horizontal distance between two via cuts is less than or equal to one of the values (D1 or D2) and vertical distance between the cuts is less than or equal the other value, then the cuts are also considered to be in the same corner via cluster.

Note:

The values D1 and D2 are interchangeable: either x-distance should be less than or equal to D1 and y-distance is less than or equal to D2, or x-distance should be less than or equal to D2 and y-distance is less than or equal to D1.



-mergeCutDistanceRange	If given, two fully aligned via cuts with distance between them in this range are considered a single “merged” cut with bounding box equal to the combined bounding boxes of the two original cuts, and all rule measurements are performed against the merged cut.
------------------------	---

Note:

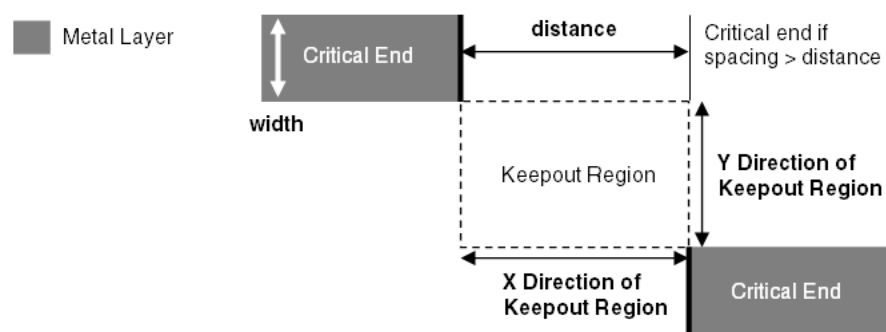
- The warning message is displayed in the following situation:
 - None of `-distance` | `{-horizontalDistances or -verticalDistances}` | `-distanceWithinRange` is defined.
 - More than one of `-distance` | `{-horizontalDistances or -verticalDistances}` | `-distanceWithinRange` are defined.
 - Only one of `{-horizontalDistances or -verticalDistances}` is defined.
 - `-maxNumCuts` is smaller than `-numCuts`.
- The `colorName` must be defined in the [Controls Section](#), and the color mask value cannot be greater than the `numberOfColorMasks` value defined in the [functions](#) section.
- If `cutClass` parameter is specified with `cutClassName` in ASCII file, technology importer looks-up the first `cutClass` constraint specified in the same hierarchical constraint group to convert to cut class size. If a `cutClass` constraint does not exist then an error message is displayed.

For example:

```
cornerViaClusterSpacing V1 (0.08 0.07 0.06)
    -numCuts 3
    -horizontalDistances (0.01 0.02)
    -verticalDistances (0.01 0.03 0.05)
    -mergeCutDistanceRange (0.02 0.03)
```

criticalEndKeepout

A critical end is a line end that does not have other shapes with parallel run lengths greater than 0 within a certain distance of it. When two facing critical ends are farther than a certain distance, the `criticalEndKeepout` constraint defines a rectangular keepout region between them. The corners of the keepout region touch the adjacent end-of-line corners. No metal shapes are permitted within the keepout region.



Constraint Category	Spacing
Constraint Type	complex
OA Rule	Custom Compiler Custom Layer Constraint

Syntax:

```
criticalEndKeepout <string|LPP>layer
    (<rangeValue>regionXRange
     <rangeValue>regionYRange)
    -width float
    -distance float

    [-isHard true|false]
    [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer on which the critical ends are placed. <code>layer</code> can be a string (layer name) or LPPr.
regionXRange	X-direction length of the keepout region, in microns. To specify the range, see Range Data Syntax .
regionYRange	Y-direction length of the keepout region, in microns. To specify the range, see Range Data Syntax .
-width	Maximum width of the critical end wires, in microns. The width must be less than or equal to the specified value.
-distance	Minimum distance between the two facing line ends, in microns. The distance between the two facing line ends must be greater than the specified value to be considered critical.

Example:

```
criticalEndKeepout m1 ( (>=0.1 <=0.2) (>=0.5 <=0.6) )
    -width 0.25 -distance 0.76
```

cumulativeLayerAntenna

The `cumulativeLayerAntenna` constraint overrides the global `cumulativeMetalAntenna` and `cumulativeViaAntenna` constraints to allow modeling of per layer cumulative antenna ratio.

Constraint Category	Antenna
Constraint Type	Complex
OA Rule	Custom Compiler Custom Layer Constraint

Syntax:

```
cumulativeLayerAntenna
  <string|LPP>layer
  <string> defaultModel(
    -diodeRatio <1DTblValue>diodeRatio
    -gateRatio <float>gateRatio
    [-isSide true|false])

  [<string> secondModel(
    -diodeRatio <1DTblValue>diodeRatio
    -gateRatio <float>gateRatio
    [-isSide true|false])]

  [<string> thirdModel(
    -diodeRatio <1DTblValue>diodeRatio
    -gateRatio <float>gateRatio
    [-isSide true|false])]

  [<string> fourthModel(
    -diodeRatio <1DTblValue>diodeRatio
    -gateRatio <float>gateRatio
    [-isSide true|false])]

  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Where `diodeRatio` is a `1DTblValue`:

```
(<float>default_value ((<float>diodeArea
  <float>diodeAreaRatio)*))
```

Argument	Description
-diodeRatio	1DLookupTbl where: - key = diode area - value = the maximum allowed antenna area if connected to the diffusion diode.

Argument	Description
-gateRatio	A ratio to the gate area of the maximum allowed, accumulated area (that is not connected to the diffusion diode) of all the poly and metal Layers attached to a gate.
-isSide	If area is true then sidewall else top surface. Default is "false".

This example defines a cumulativeLayerAntenna constraint with the default model:

```
cumulativeLayerAntenna v1 defaultModel(  
    -diodeRatio diodeRatio (5 ((0.1 10) (0.4 20)))  
    -gateRatio 10  
    -isSide true)
```

For more information on defining range tables, see [Understanding and Interpreting 1-D, 2-D, and 3-D Table Values](#) section.

cutClass

The constraint defines a class of cut shapes for a via with a particular dual cut value. See also [cutClass](#) and [cutClassList](#).

Constraint Category	Via Construction
Constraint Type	Complex
OA Rule	oacCutClass

Syntax:

```
cutClass <string|LPP>layer  
    <float>viaWidthMin <float>viaLengthMin  
    -className string  
    -numCuts int  
    -viaWidthMin true|false]          [-viaLengthMin true|false]  
  
    [-isHard true|false]  
    [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or as LPP. The layer must be a cut layer.
-className	Name of the cutClass or the list of cut classes.
-numCuts	Indicates how many equivalent cuts should be used in resistance calculations.

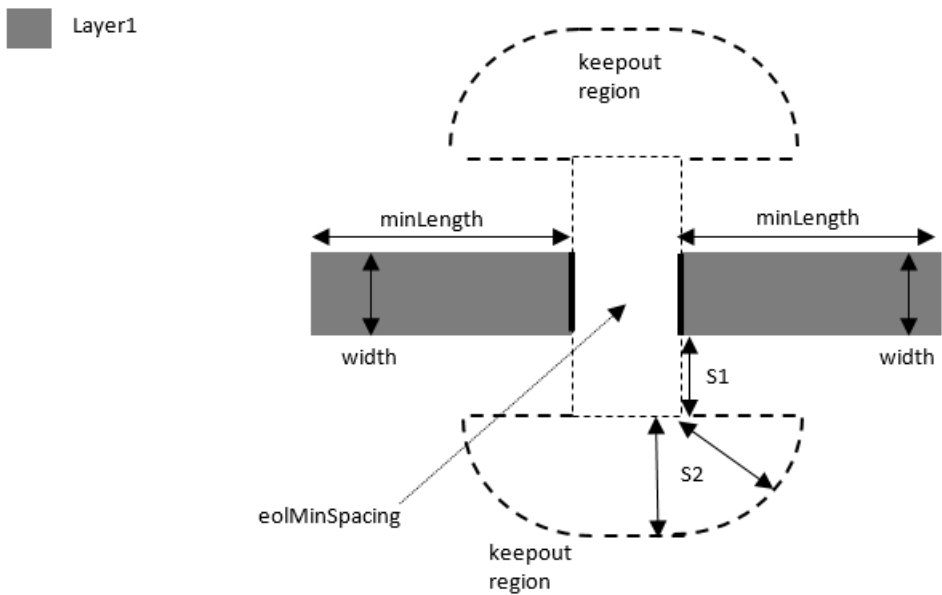
Argument	Description
-viaWidthMin	Indicates whether the width is fixed or a minimum value.
-viaLengthMin	Indicates whether the length is fixed or a minimum value.

For example:

```
cutClass v1 0.04 0.06 -className viaPw
                    -viaWidthMin true
                    -numCuts 2
```

cutMetalAvoidanceKeepout

This layer constraint supports cut metal avoidance in advanced node designs.



Constraint Category	Spacing
Constraint Type	complex
OA Rule	Custom Compiler custom layer constraint

Syntax:

```
cutMetalAvoidanceKeepout
  <string|LPP>layer
  (<float>S1 <float>S2)
  -width float
```

```
-eolMinSpacing float
[-minLength float]
[-color colorName]any|same|diff]

[-isHard true|false]
[-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer of the concave shape and neighboring shapes. The layer can be defined as a string or a layer-purpose pair.
S1 S2	Distances that determine the keepout regions.
width	Maximum width of the wires.
eolMinSpacing	Maximum distance between the two end-to-end wires. The constraint applies only when the distance between the two end-to-end wires is less than eolMinSpacing.
-minLength	Minimum length of the wires.
-color	Color of the wires. Valid values are colorName, 0 (any), 1 (same), and 2 (diff). colorName must be defined in the Controls Section . Default: 0

Example:

```
cutMetalAvoidanceKeepout m1 0.35 0.78 true
                        -width 0.35
                        -eolMinSpacing 0.25
                        -minLength 0.17
                        -color red
```

defWidth

This constraint is to define the default width of the specified layer.

Constraint Category	Length and Width Process Rules
Constraint Type	Complex
OA Rule	Custom Compiler Custom Layer Constraint

Syntax:

```
defWidth <string|LPP>layer
        <float>width
        [-spacingDirection horizontal|vertical|any]
```

```
[-isHard true|false]
[-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or a LPP.
width	This parameter specifies the default width.
-spacingDirection	This parameter specifies the direction for the default width. Default is “any”.

For example:

```
defWidth m1 0.10 -spacingDirection horizontal
```

diagonalShapesAllowed

Specifies whether diagonal shapes are permitted on a specified layer.

Syntax:

```
diagonalShapesAllowed
    <string|LPP>layer
    <bool>allowedValue

    [-isHard true|false]
    [-comment [<string rule_id>] <string rule_desc>]
```

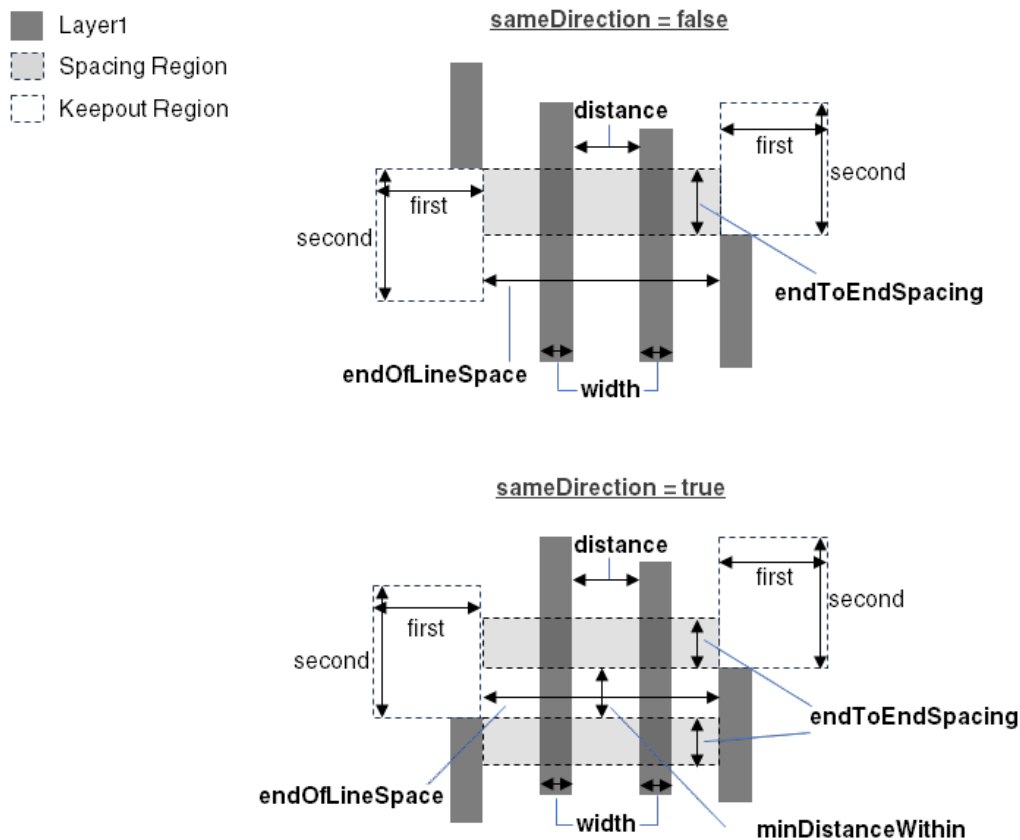
Argument	Description
layer	Name of the layer or LPP.
allowedValue	Specifies whether diagonal shapes are permitted on the specified layer.

In the following example, diagonal shapes are permitted on the m1 layer.

```
diagonalShapesAllowed m1 true
```

dualEndOfLineKeepout

The `dualEndOfLineKeepout` constraint requires certain spacing and keepout regions between neighboring wire ends (end of lines) that have a certain number of wires between them. The following diagram shows the constraint requirements.



Constraint Category	Spacing
Constraint Type	Complex
OA Rule	Custom Compiler Custom Layer Constraint

Syntax:

```
dualEndOfLineKeepout
  <string|LPP>layer
  <firstFloat secondFloat>keepoutDimensions
  -width float
  [-sameDirection true | false]
  -distance float
  [-count int
  -endOfLineSpace float
  -endToEndSpacing float
  [-minDistanceWithin float]

  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```


Argument	Description
layer	Layer can be defined as a string or a LPP.
keepoutDimensions	Dimensions that define the keepout regions: first second.
-width	Width of the in-between wires.
-sameDirection	Indicator of whether the outer wire ends are in the same direction.
-distance	Distance of the in-between wires.
-count	Number of wires between the wire ends that must interact.
-endOfLineSpace	Distance between the inner edges of the wire ends.
-endToEndSpacing	Length of the spacing regions.
-minDistanceWithin	Distance between the spacing regions. Applicable only if -sameDirection is true.

For example:

```
dualEndOfLineKeepout M2 0.18 0.27
    -width 0.056
    -sameDirection false
    -distance 0.056
    -count 3
    -endOfLineSpace 0.28
    -endToEndSpacing 0.26
    -minDistanceWithin 0.1
```

edgeCapacitance

The `edgeCapacitance` layer constraint specifies the peripheral capacitance, a floating-point value in picofarads per micron. You use edge capacitance to estimate the capacitance before and after routing.

The post-route capacitance calculation is segment-based:

$$C_{\text{segment}} = (\text{layer capacitance/sq} \times \text{width}_{\text{segment}} \times \text{length}_{\text{segment}}) + (C_{\text{peripheral}} \times 2 (\text{width}_{\text{segment}} + \text{length}_{\text{segment}}))$$

To set the design option to control whether the tool calculates and displays static or dynamic measurement results as your cursor moves on the canvas, see [Layout Cursor Tab](#).

enclosureMergeUnderErasing

In lower nodes, although most erasable layers have different enclosure requirements for shapes that are cut by a shape on a cutting/erasing layer, some layers require enclosures to be measured in a way that erasing layer shapes are ignored. For such layers, no matter shapes are erased or not, a merged geometry should be used for all the physically touching pieces.

Constraint Category	Miscellaneous
Constraint Type	Simple
OA Rule	Custom Compiler custom layer constraint

Syntax:

```
enclosureMergeUnderErasing
    <string|LPP>layer
    <bool>value

    [-isHard true|false]
    [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or as LPP.
value	If true, merge geometry of all the physically touching shapes when measuring enclosure .

For example:

```
enclosureMergeUnderErasing PO true
```

endOfLineKeepout

In some advanced processes, spacing from the end of a line to other shapes on the same layer is specified via a simple keep-out region around the end of the line. It can be a violation or not a violation:

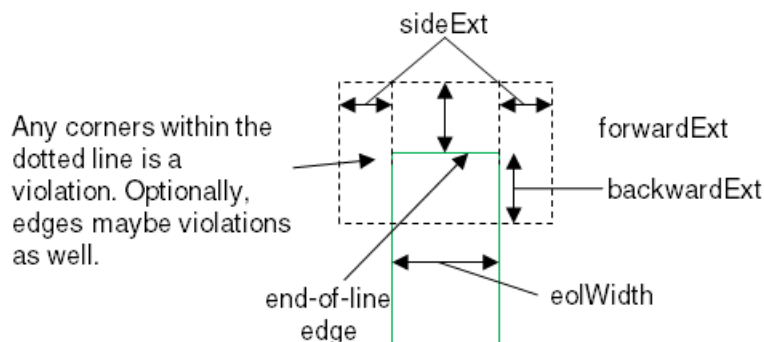
- In some processes, it is only a violation if a corner of another shape intersects the keep-out area, and in others, neither a corner nor an edge might intersect the keep-out area.
- In some processes, it is not a violation if the corner or edge which intersects the keep-out area is part of the same metal shape. The rule applies to wires with end of lines less than a given `eolWidth`.

Although there are some similarities with the [minEndOfLineSpacing](#) constraint, this should be treated as a separate constraint as application of the constraint does not depend on the presence of parallel lines within a specific distance. In addition, the keep-out area extends both `forwardExt` and `backwardExt` from the end of the line, as well as to the `sideExt`.

The `extValue` array contains the following values:

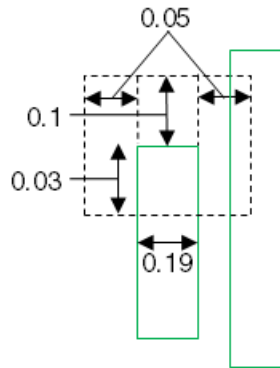
- `backwardExt`: defines the part of the keep-out area that extends back from the end of line edge
- `sideExt`: defines the lateral dimensions of the keep-out area which extend from the sides of the end of line edge
- `forwardExt`: specifies the keep-out region that is forward from the end of line edge.

Figure 3 Definition of `endOfLineKeepout`

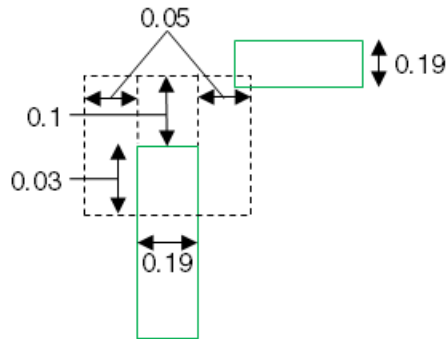


backwardExt 0.03
sideExt 0.5
forwardExt 0.1
width 0.2
cornerOnly false (exceptConnectivityType not defined)

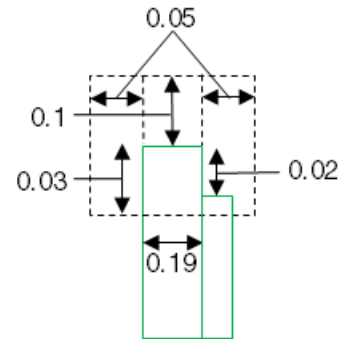
□ Layer
□ Area to check



a) Violation. Side neighbor intrudes into keep-out region. This would not be a violation if `cornerOnly` is true because only an edge can intersect the keep-out area.



b) Violation. Any neighbor in the dotted region is a violation.

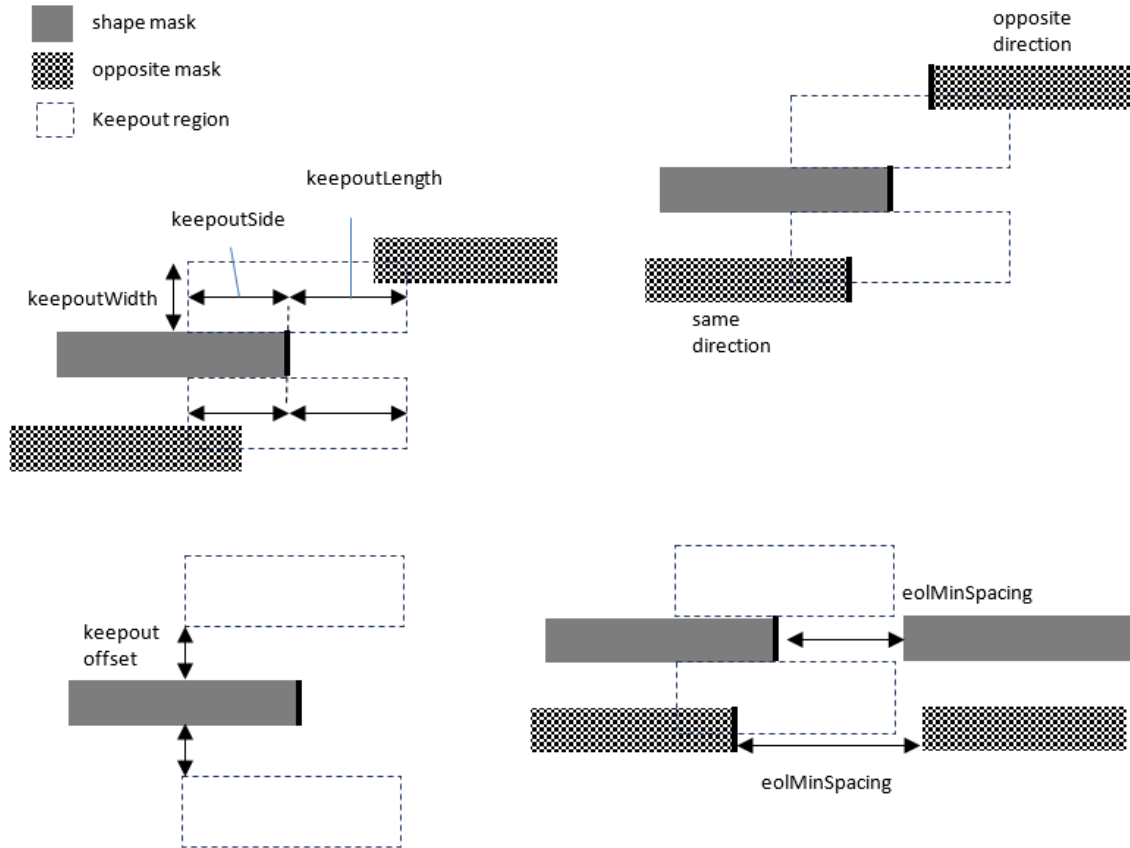


c) Violation. Any neighbor in the dotted region, including same metal is a violation. This would not be a violation if `exceptConnectivityType` is `contiguousShapesConnectivityType`.

Advanced node designs require strict spacing between end-of-wire edges on the same mask and on different masks that are on the same layer. The spacing requirement depends on whether the end-to-end wires are in the same or opposite direction. A keepout regions are established as shown in the following diagram.

Note:

`endOfLineKeepout` replaces the following 10 nm rules: Unaligned End of Line Keepout and Unaligned Parallel End of Line Keepout.



Some processes allow exceptions that when satisfied means that the constraint does not apply.

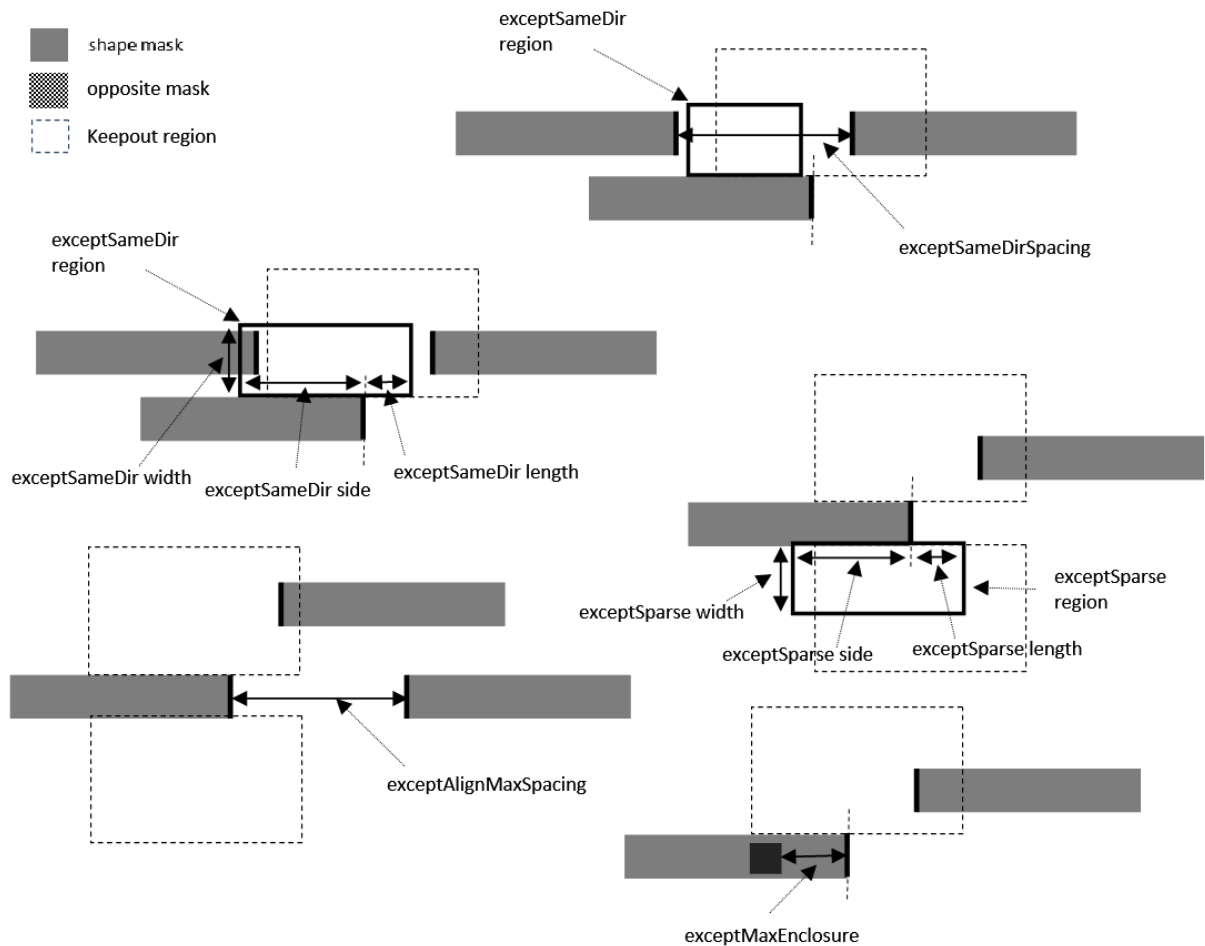


Figure 4 jointEOL parameter (horizontal direction)

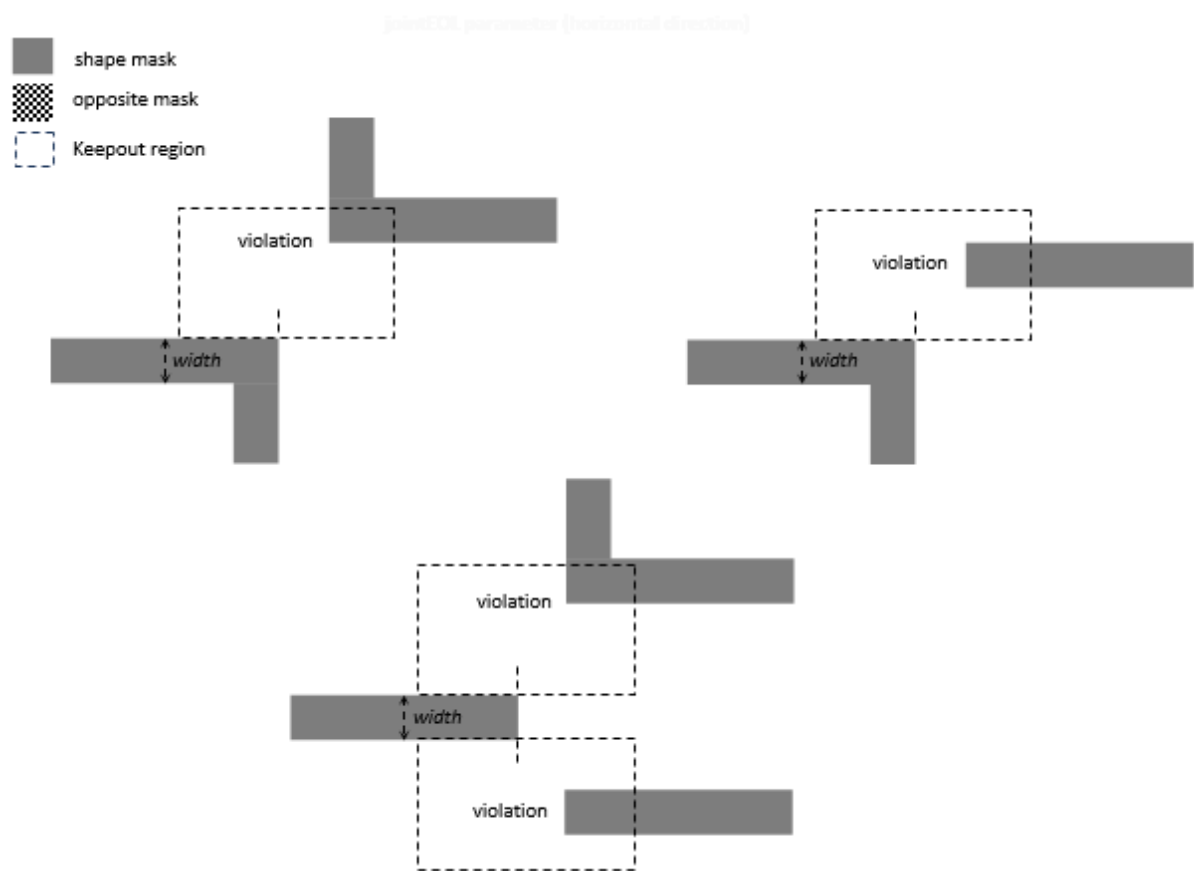


Figure 5 exceptAligned parameter set to 0 (none)

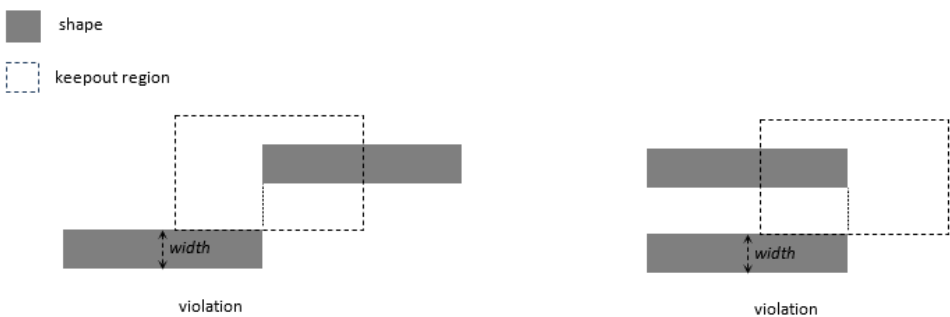


Figure 6 *exceptAligned* parameter set to 1 (all)

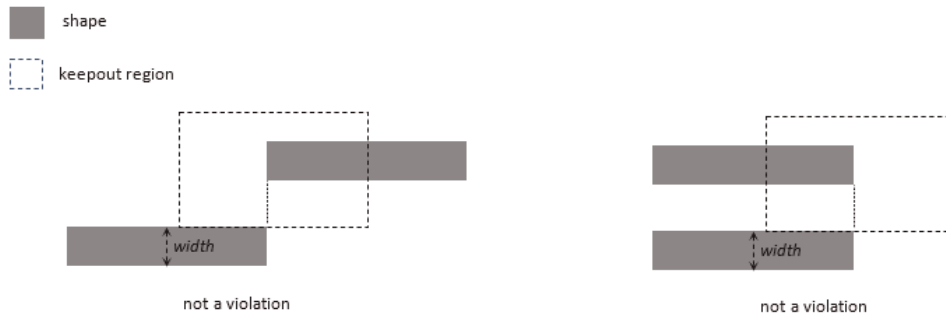


Figure 7 *exceptAligned* parameter set to 2 (*checkSeparated*)

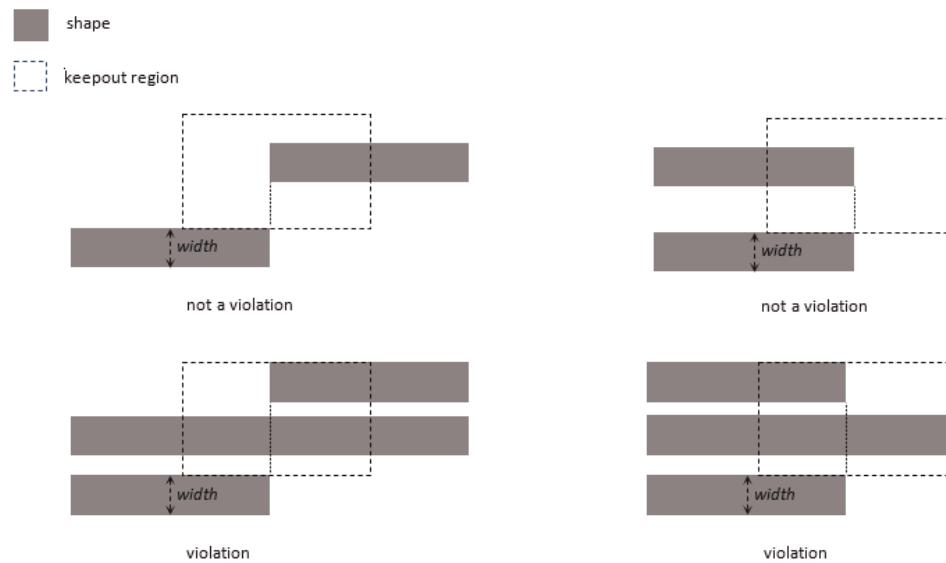
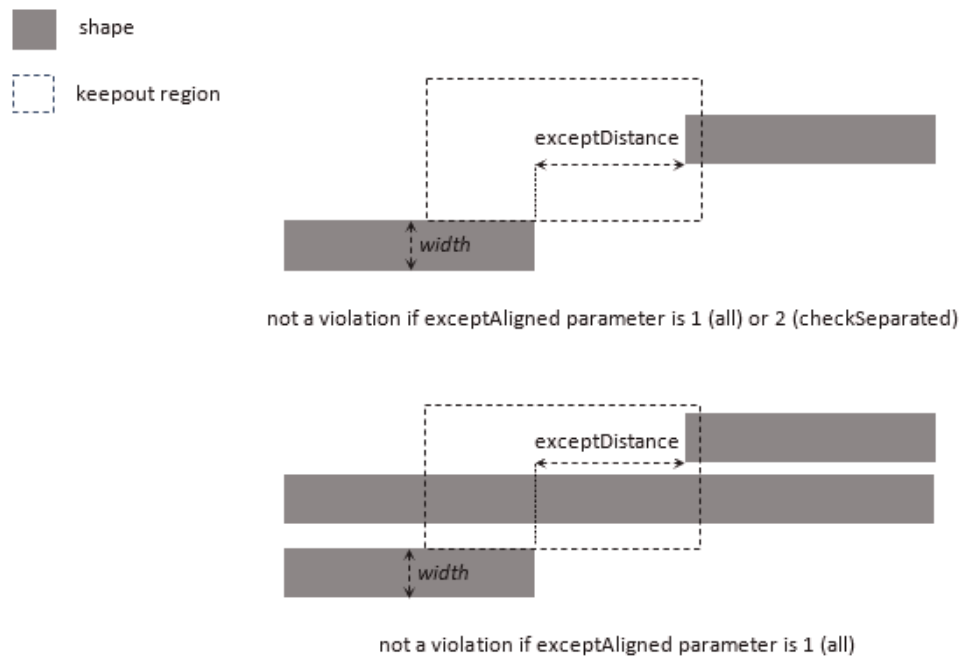


Figure 8 Usage of exceptDistance parameter



Constraint Category	Spacing
Constraint Type	Complex
OA Rule	Custom Compiler Custom Layer Constraint

Syntax:

```
endOfLineKeepout
  <string|LPP>layer
  <float>(backwardExt sideExt forwardExt)extValue
  -width float|-widthRange rangeData
  [-cornerOnly true|false]
  [-exceptConnectivityType anyConnectivityType|
    sameNetConnectivityType|
    contiguousShapesConnectivityType|
    sameViaConnectivityType|
    directShapesConnectivityType]
  [-minLength float]
  [-sameDirection true|false]
  [-color colorName|any|same|diff]
  [-otherColor colorName]
  [-exceptDistance float]
  [-exceptAligned int]
  [-keepoutOffset float]
  [-secondKeepout (float1 float2 float3 float4)]
```

```
[-secondOffset float]
[-eolMinSpacing float]
[-cutToNatural true|false]
[-naturalToNatural true|false]
[-eolThreshold float]
[-exceptSameDir (float1 float2 float3)]
[-exceptSparse (float1 float2 float3)]
[-exceptSameDirSpacing float]
[-exceptMaxEnclosure float]
[-exceptAlignedMaxSpacing float]
[-spacingDirection any|horizontal|vertical]
[-jointEOL true|false]
[-otherWidthRange rangeData]
[-isHard true|false]
[-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or a LPP.
extValue	Array of values defining the <code>backwardExt</code> , <code>sideExt</code> and <code>forwardExt</code> .
-width	This constraint applies for all lines with end of line width < Width (eolWidth in diagrams). This parameter or the <code>-widthRange</code> parameter must be given, but not both.
-widthRange	The wire width must lie in the given range for the constraint to apply. This parameter or the <code>-width</code> parameter must be given, but not both.
-cornerOnly	This is an optional parameter. When not present, or false, it is considered a violation if either an edge or a corner of another shape intersect the keep-out region. If this parameter is true, only a corner is considered a violation.
-exceptConnectivityType	This is an optional parameter. When this parameter is set to <code>contiguousShapesConnectivityType</code>, this constraint does not apply to edges and corners of the same metal shape. If this parameter is not specified, the constraint applies to all shapes. This parameter supports the following three enum values: • anyConnectivityType: Use this option to designate that the constraint applies to any type of connectivity (default). • sameNetConnectivityType: The parallel overlap rule does not apply to same-net cuts. • contiguousShapesConnectivityType: The parallel overlap rule does not apply to cuts that share the same metal shapes either above or below the cuts. • sameViaConnectivityType: The parallel overlap rule does not apply to cuts that share the same metal shapes above and below metal layers to cover the overlap area between the cuts. • directShapesConnectivityType: Use this option to designate that the constraint only applies when the two cuts are part of the same metal shape on the upper or lower metal and the metal shape covers the region between the two cuts.
-minLength	Minimum wire length, in microns, measured from the end of the wire.

Argument	Description
-sameDirection	Wires whose ends face each other must be in the same direction when this argument is set to <code>true</code> . Default: <code>true</code>
-color	Color of the shape that cannot have a corner in its keepout region. Specify a color name defined in the Controls Section. A value of -1 (same) means the rule only applies if the mask color of the other shape in the keepout region is the same as the mask color of the shape generating the keepout. A value of -2 (diff) means the rule only applies if the mask color of the other shape in the keepout region is different from the mask color of the shape generating the keepout. A value of zero means any color for both shapes. Default: any
-otherColor	This is an optional parameter defining the specific mask color of the other shape in the keepout region. Must be a positive number. Can be specified only if the color parameter is also a positive number, otherwise, this parameter is ignored. If the color parameter is a positive number, and this parameter is omitted, its value is assumed to be equal to the value of the color parameter.
-exceptDistance	If two end of line-edges of opposite direction wires are separated using the <code>-exceptDistance</code> option, then the wires are considered to be aligned.
-exceptAligned	Specifies alignment of end of line-edge of two wires for which the <code>endOfLineKeepout</code> constraint is applied on. Valid values are: 0 (none): Applies the constraint irrespective of wire alignment. 1 (all): Does not apply the constraint if the wires are aligned. 2 (checkSeparated): Does not apply the constraint if the wires are aligned; however, the constraint is applied if the aligned wires are separated by one or more wires on the same layer.
-keepoutOffset	Distance between the keepout region and the wire.
-secondKeepout	Second keepout region, defined by the <code>keepoutSide</code> , <code>keepoutWidth</code> , <code>keepoutLength</code> , and <code>keepout offset</code> (optional). All values are in microns.
-secondOffset	Distance between the second keepout region and the wire.
-cutToNatural	The type of wire end for which the constraint applies. Valid values are: <code>true</code>: One of EOL edge must be natural and the other must be cut. This parameter and the <code>-naturalToNatural</code> parameter cannot both be <code>true</code>. <code>false</code> and the <code>-naturalToNatural</code> parameter is <code>false</code>: Only cut EOL edges apply. Default: <code>false</code>

Argument	Description
-naturalToNatural	Optional parameter that defines the type of EOL edge pairs that apply to the constraint. Valid values are: • true: Both EOL edges must be natural. This parameter and the <code>-cutToNatural</code> parameter cannot both be true. • false and the <code>-cutToNatural</code> parameter is false: Only cut EOL edges apply. Default: false
-eolMinSpacing	Minimum distance between the two end-to-end wires where one wire end was the result of a cut action and the other wire end was originally (or naturally) the end of a wire.
-eolThreshold	This is an optional parameter that defines the spacing threshold required for a facing EOL edge in order for the constraint to apply. If a facing edge with PRL>0 is less than or equal to this distance away from the EOL, then the constraint does not apply. This parameter should not be used with <code>eolMinSpacing</code> .
-exceptSameDir	This is an optional parameter defined by the <code>exceptSameDirSide</code> , <code>exceptSameDirWidth</code> , and <code>exceptSameDirLength</code> . That is a value array in the same format as the constraint value that defines a region which if contains a same direction EOL edge, then an opposite direction EOL edge in keepout region is waived and the constraint does not apply.
-exceptSparse	This is an optional parameter defined by the <code>exceptSparseSide</code> , <code>exceptSparseWidth</code> , and <code>exceptSparseLength</code> . That is a value array in the same format as the constraint value that defines a region which if empty allows an opposite direction EOL edge in the opposite side keepout region to be waived and the constraint does not apply.
-exceptSameDirSpacing	This is an optional parameter defining the spacing threshold ($\leq$) between two facing EOL edge, one of which is in the keepout region. If the second facing edge is not in <code>exceptSameDir</code> region and spacing to facing EOL edge is less than or equal to this value, the constraint does not apply.
-exceptMaxEnclosure	If the EOL edge contains a cut above or below metal layer which has a distance to the edge less than or equal to this parameter value, then opposite direction EOL edges in the keepout region is allowed and this constraint does not apply.
-exceptAlignedMaxSpacing	This is an optional parameter defining the spacing threshold ($\leq$) of an exactly aligned facing EOL edge. If the aligned facing EOL is less than or equal to this value, the constraint does not apply.
-spacingDirection	Defines the spacing direction in which this constraint applies: only horizontal, only vertical or both. Default: any
-jointEOL	If true, in addition to line ends, the constraint also applies to convex corner of L-shapes. See Figure 4 . Default: false
-otherWidthRange	If specified, the constraint applies only to those shapes in the keepout region which have width in this range.

For examples:

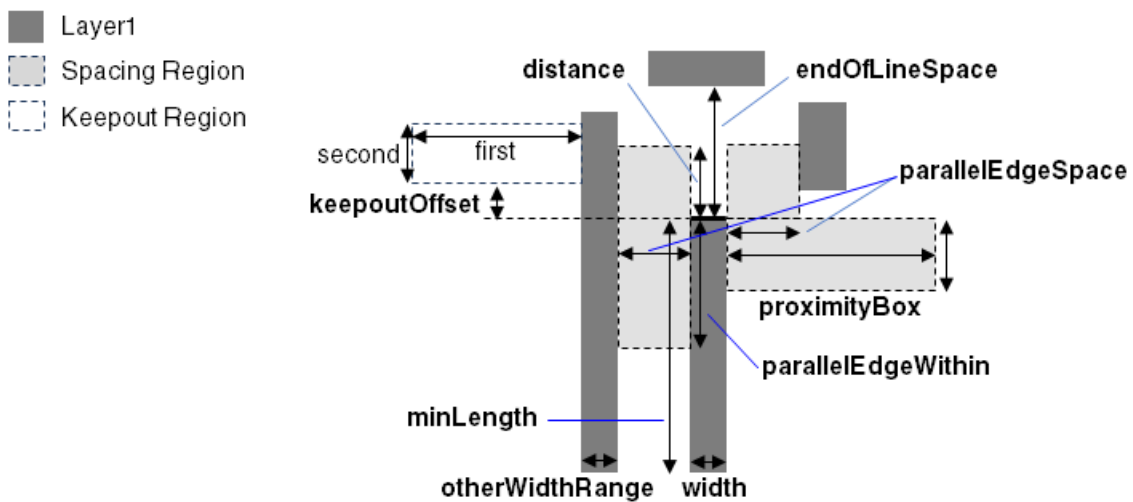
```
endOfLineKeepout M1 (0.04 0.06 0.08) -width 0.15 -cornerOnly true
    -minLength 0.5 -sameDirection false -color red -exceptDistance 0.25
    -exceptAligned 2 -keepoutOffset 0.75
    -secondKeepout (0.45 0.25 0.77 0.55) -eolMinSpacing 0.45
    -cutTuNatural true

endOfLineKeepout M1 ( 0.04 0.06 0.08) -widthRange (>=0.3 <=0.8) -cornerOnly
true
    -minLength 0.5 -sameDirection false -eolThreshold 0.1
    -spacingDirection vertical -exceptSameDir (0.06 0.03 0.05)
    -exceptSparse (0.07 0.02 0.05) -exceptSameDirSpacing 0.1
    -exceptMaxEnclosure 0.02
    -otherWidthRange (>=0.2 <=0.5)
```

endOfLineNeighborKeepout

The `endOfLineNeighborKeepout` constraint requires certain spacing between neighboring wire ends (end of lines) to prevent interactions. The constraint requires the following conditions:

- When a shape has a certain width (`width`), a keepout region and three spacing regions surrounding the shape are defined based on `distance`, `parallelEdgeSpace`, `parallelEdgeWithin`.
- The neighboring shape next to the largest spacing region must have a width within `otherWidthRange`.
- The keepout region dimensions are defined by `first` and `second`.
- No shape can be within the `endOfLineSpace`.
- No shape can be within the `proximityBox`.



Constraint Category	Spacing
Constraint Type	Complex
OA Rule	Custom Compiler Custom Layer Constraint

Syntax:

```
endOfLineNeighborKeepout
    <string|LPP>layer
    <firstFloat secondFloat>keepoutDimensions
    -width float
    -otherWidthRange rangeData
    -distance float
    -endOfLineSpace float
    -parallelEdgeSpace float
    -parallelEdgeWithin float
    -keepoutOffset float
    -proximityBox float float
    [-minLength float]

    [-isHard true|false]
    [-comment [<string rule_id>] <string rule_desc>]
```

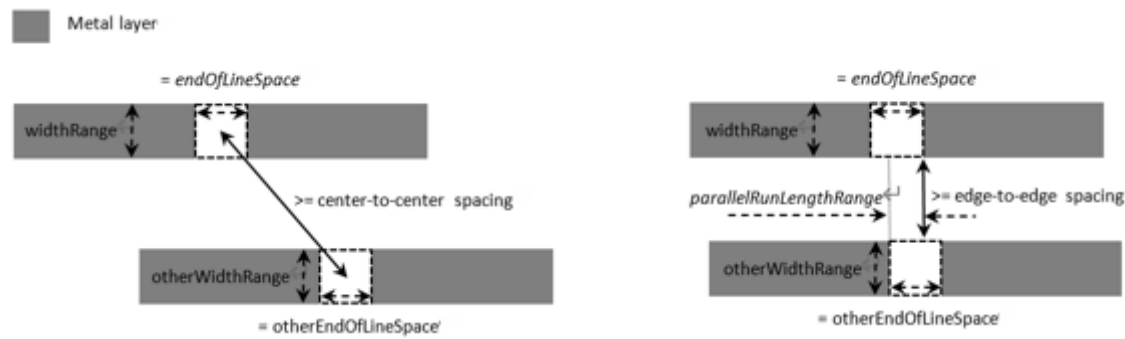
Argument	Description
layer	Layer can be defined as a string or a LPP.
keepoutDimensions	Dimensions that define the keepout region: first second.
-width	Width of the wire end.
-otherWidthRange	Width of the neighboring wire that is next to the largest spacing region. The width must be within the specified range. See Range Data Syntax .
-distance	Distance between the wire end and the top edge of the largest spacing region.
-endOfLineSpace	Minimum distance between the wire end and a neighboring shape whose longer edge runs perpendicular to the wire end.
-parallelEdgeSpace	Width of the spacing regions.
-parallelEdgeWithin	Length of the largest spacing region.
-keepOutOffset	Distance between the wire end and the keepout region.
-proximityBox	Dimensions of the proximity box: first (horizontal), second (vertical).
-minLength	Minimum length of the wire. Default: 0

For example:

```
endOfLineNeighborKeepout M2 0.062 0.077
    -width 0.056
    -otherWidthRange (>=0.01 <=0.062)
    -distance 0.09
    -endOfLineSpace 0.34
    -parallelEdgeSpace 0.056
    -parallelEdgeWithin 0.133
    -keepoutOffset 0.035
    -proximityBox 0.26 0.084
    -minLength 0.2
```

endOfLineProjectionSpacing

Some processes may have additional requirements for spacing of cut-metal layer shapes between line ends of the corresponding metal layer shapes. This rule is formulated in terms of metal layer shape projections to cover different methodologies for creating cut-metal shapes: drawn in the layout at the same time as the metal shapes, or created as post-processing step after the metal shapes are created. The constraint applies when fully aligned metal line ends are facing each other at a fixed distance (`endOfLineSpace`). Typically, it is the width of the cut-metal layer. The rule requires a minimum spacing between two projections of metal line ends. The spacing can be measured center-to-center or edge-to-edge.



Constraint Category	Spacing
Constraint Type	Complex
OA Rule	Custom Compiler Custom Layer Constraint

Syntax:

```
endOfLineProjectionSpacing
    <string|LPP>layer
    <float>spacingValue
    -endOfLineSpace floatValue
    [-otherEndOfLineSpace floatValue ]
```

```
[ -widthRange rangeData]
[ -otherWidthRange rangeData]
[ -color colorName|any|same|diff]
[ -centerToCenter true | false]
[ -parallelRunLengthRange rangeData]

[ -isHard true|false]
[ -comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or a LPP.
spacingValue	The minimum spacing in database units required between the end-of-line projections.
-endOfLineSpace	Exact distance in database units between the aligned metal line ends required for the constraint to apply.
-otherEndOfLineSpace	If given, the constraint only applies when the distance between one pair of aligned metal line ends is exactly <code>-endOfLineSpace</code> , and the distance between another pair of aligned metal line ends is exactly this value in database units. By default, <code>-endOfLineSpace</code> applies to both pairs of aligned metal line ends.
-widthRange	If given, the constraint only applies when the width of metal lines lies in this range in database units. See Range Data Syntax .
-otherWidthRange	If given, the constraint only applies when the width of one pair of aligned metal lines lies in <code>-widthRange</code> , and the width of another pair of aligned metal lines lies in this range in database units. By default, <code>-widthRange</code> applies to both pairs of aligned metal lines. Default: none.
-color	If given, the constraint only applies when all metal shapes under consideration have this mask color. A value of zero means any color. A value of -1 means same color (For example, the pair of facing metal shapes forming one of the end-of-line projection is the same color as the other pair). A value of -2 means diff color (For example, the color of the pair of facing metal shapes forming one of the end-of-line projection is different from the color of the other pair). Default: any.
-centerToCenter	Optional parameter indicating if the spacing is measured center-to-center (true) or edge-to-edge (false). Default: false.
-parallelRunLengthRange	If given, the constraint only applies when parallel run length between the end-of-line projections lies in this range in database units. By default, the parallel run length must be non-negative. See Range Data Syntax . Default: (≥ 0).

Note:

You must specify the *colorName* in the [Controls Section](#) > [colors](#) group and the color mask value cannot be greater than the `numberOfColorMasks` value specified in the [Layer Rules Section](#) > [functions](#) group.

For example:

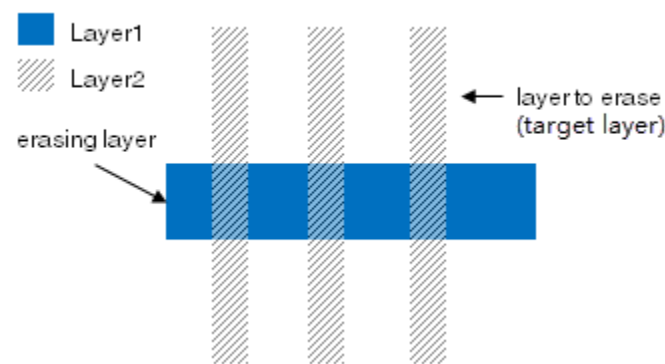
```
endOfLineProjectionSpacing m1 0.04 -endOfLineSpace 0.06
-widthRange (>=0.05 <=0.06)
-otherWidthRange (>=0.04 <=0.06)
-color same
```

erasingLayer

In advanced technology nodes, shapes need to be closer together than permitted by defined constraints. To achieve the desired spacing without violating constraints, you can add a layer (erasing layer) and use it to create cuts (or the desired spacing) in shapes.

The `erasingLayer` constraint defines the following:

- The layer that erases shapes.
- The targets (layers) of the erasing operation. These are the layers that will be erased by the erasing layer.
- The colors that will be excluded from the erasing operations.



The constraint also uses the `-color` argument as follows to determine the behavior during the erasing operation:

Layers	Colorable	Erasing Behavior
erasing layer	No	The erasing layer erases only target layers that have the specified <code>-color</code> . For example, if you specify <code>-color red</code> , the erasing layer will erase only red shapes.
target layer	Yes	

Layers	Colorable	Erasing Behavior
erasing layer target layer	Yes Yes	The erasing layer erases only the target layers that have the identical color. Do not specify -color in this case. Doing so will result in errors. For example, if the erasing layer is red, then it will erase only red shapes. If the erasing layer has no color, then it will erase only shapes that have no color.
erasing layer target layer	Yes No	The -color argument applies to the erasing layer. If you do not specify -color, the erasing layer can be any color. For example, if you specify -color red, then only red erasing layers will perform erasing operations on non-colorable shapes.

In all three cases, the `-exceptColor` argument can be applied to exclude layers (shapes) of specified colors.

Constraint Category	Miscellaneous
Constraint Type	Complex
OA Rule	-

Syntax:

```
erasingLayer
    <string|LPP>layer
    <List of strings>layers
    [-color colorName]
    [-exceptColors (<colorName1 colorName2 ...>)]
    [-maxWidth floatValue]
    [-isHard true|false]
    [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	The erasing layer. The layer can be defined as a string or as LPP.
layers	Specifies the list of layers that are to be erased for erasing layer.
-color	The color of the erasing layer or the target layer that will be erased. See the constraint description.

Caution:

You must specify the *colorName* in the [Controls Section > colors](#) group, and the color mask value cannot be greater than the `numberOfColorMasks` value specified in the [Layer Rules Section > functions](#) group.

Argument	Description
-exceptColors	The color of the layers that will not be erased. This argument is only applicable if the erasing layer is multi-colored, and you do not specify -color or if -color is 0 or any.
-maxWidth	Defines the maximum width of the shape to be erased for the constraint to apply. For shapes whose widths are larger than this max width, the constraint doesn't apply.

For example:

```
erasingLayer PO (poly cpoly dpoly) -color red -isHard false
erasingLayer PO (poly cpoly dpoly ) -exceptColors (green yellow blue)
```

erasingShapeNeededThreshold

Advanced process nodes support layer cutting on metal and poly layers to permit smaller separation of end-to-end wires. The use of an erasing shape is only needed if the end-to-end spacing is less than a given threshold.

Constraint Category	Spacing
Constraint Type	Complex
OA Rule	Custom Compiler custom layer constraint

Syntax:

```
erasingShapeNeededThreshold
    <string|LPP>layer
    <float>distance

    [-isHard true|false]
    [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or as LPP.
distance	Distance between the end-to-end wires at which an erasing shape is not required.

Example:

```
erasingShapeNeededThreshold m1 0.35
```

errorLayer

This layer constraint specifies whether geometries can exist on a specific derived layer.

Note:

Geometries should not be created on an error layer. Applications should detect the existence of an error layer and use an oaMarker to indicate the error.

Constraint Category	Miscellaneous
Constraint Type	Simple
OA Rule	oacErrorLayer

Syntax:

```
errorLayer
    <string|LPP>layer
    <bool>errorLayerValue

    [-isHard true|false]
    [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or as LPP.
errorLayerValue	Specifies whether geometries can exist on the specified derived layer.

For example:

```
errorLayer gate f
```

excludeSignalTypeRouting

This layer constraint allows specific masks on layers to be excluded for routing purposes.

Constraint Category	Routing
Constraint Type	Complex
OA Rule	Custom Compiler Custom Constraint

Syntax:

```
excludeSignalTypeRouting
    <string|LPP>layer
    <string> colorName
    [-signalTypes (signal|power|ground|clock|
                  tieoff|tieHi|tieLo|analog|
                  scan|reset) ]

    [-isHard true|false]
    [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or as LPP.
colorName	Defines the color mask that must not be routed for the given list of signal types. A value of <code>any</code> means any color for either shape, <code>same</code> means that the shapes must be on the same color, and <code>diff</code> means the shapes must be on different colors. Default is <code>any</code> . Caution: You must specify the <code>colorName</code> in the Controls Section > colors group, and the color mask value cannot be greater than the <code>numberOfColorMasks</code> value specified in the Layer Rules Section > functions group.
-signalTypes	List of signalTypes that apply to this constraint. If not given, all signalTypes apply.

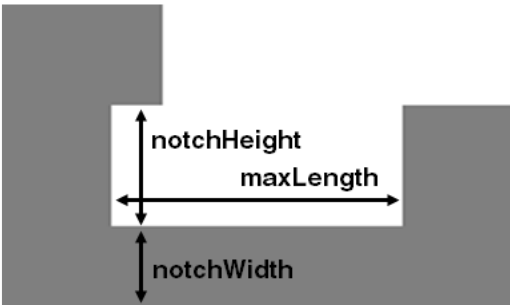
For example:

```
excludeSignalTypeRouting M1 green -signalTypes (power ground tieoff reset)
```

extendedNotchSpacing

The `extendedNotchSpacing` constraint requires greater notch height when three consecutive corners are formed.

■ Layer1



Constraint Category	Spacing
Constraint Type	Complex
OA Rule	Custom Compiler Custom Layer Constraint

Syntax:

```
extendedNotchSpacing
    <string|LPP>layer
    <float>notchHeight
    -notchWidth <float>
    -maxLength <float>

    [-isHard true|false]
    [-comment [<string rule_id>] <string rule_desc>]
```

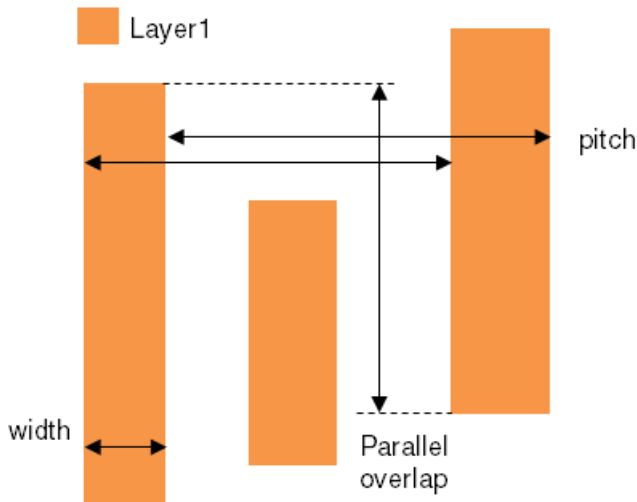
Argument	Description
layer	Layer on which the shape is placed. Layer can be defined as a string or as LPP.
notchHeight	Minimum height of the notch.
-notchWidth	Maximum width of the notch base. The constraint applies only if the width is less than or equal to the specified value.
-maxLength	Maximum length of the interior base of the notch. The constraint applies only if the length is less than or equal to the specified value.

For example:

```
extendedNotchSpacing m2 0.17
    -notchWidth 0.061
    -maxLength 0.076
```

forbiddenPitch

Some processes do not allow a range of pitches between two shapes on a layer when the shapes are separated by a single third shape and the two shapes enough parallel overlap. The rule is also dependent on the narrower of the two shapes being less than or equal to a threshold.



Constraint Category	Spacing
Constraint Type	Complex
OA Rule	Custom Compiler Custom Layer Constraint

Syntax:

```
forbiddenPitch
    <string|LPP>layer
    <rangeValue>value
    [-width float]
    [-parallelRunLength float]

    [-isHard true|false]
    [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or as LPP.
value	Specifies the pitch distances which are not allowed between like edges of two shapes on the given layer separated by a third rectangular shape. To specify the ranges, see Range Data Syntax .

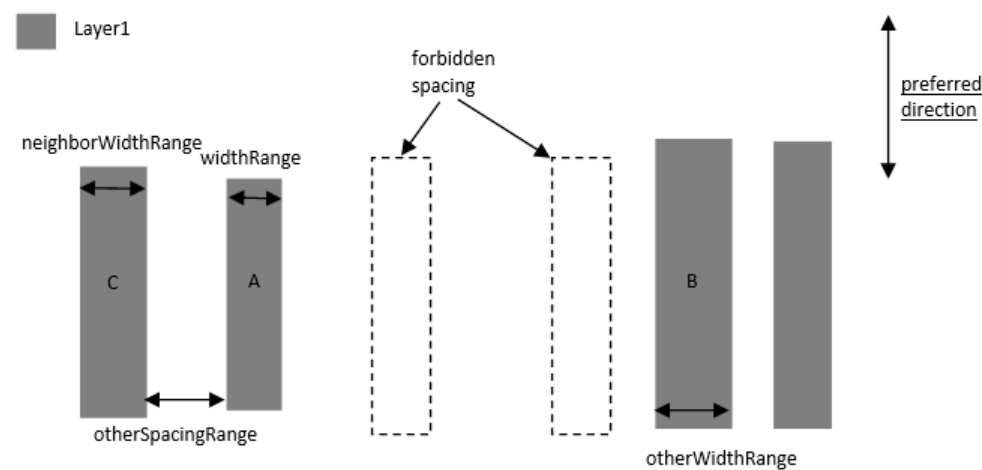
Argument	Description
-width	Specifies the width of the shape. The default is 0.
-parallelRunLength	Specifies the parallel run length threshold in microns.

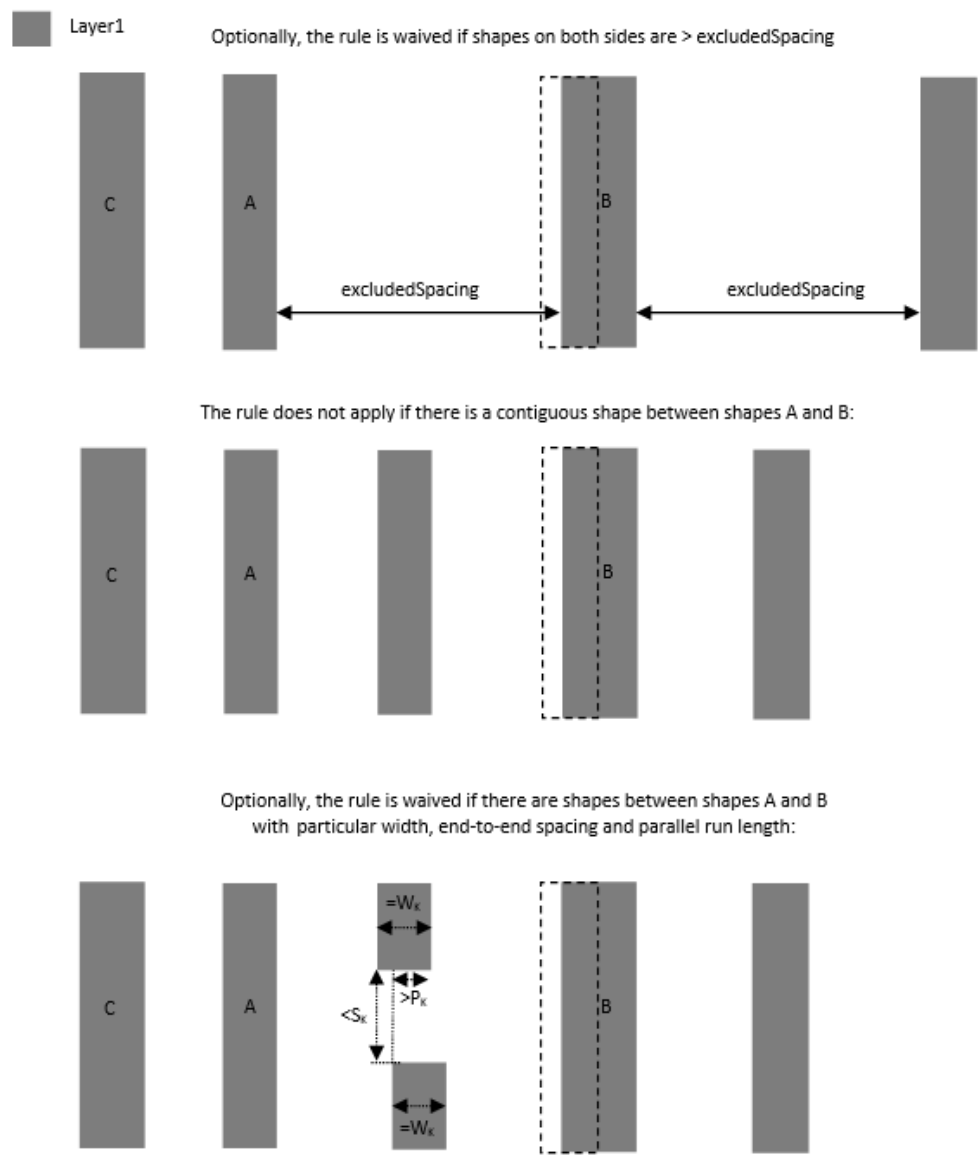
For example:

```
forbiddenPitch M1 (>=0.4 <=0.5) -width 0.15 -parallelRunLength 0.3
```

forbiddenSpacing

Some processes do not permit a shape to be a certain distance from another shape under certain conditions. The following diagram shows the conditions of the `forbiddenSpacing` constraint. Two shapes (A and B) of specific widths are a distance apart. Both shapes are in the preferred routing direction. A shape (A) of a specific width in the preferred routing direction has another shape (C) a certain distance away (also in the preferred routing direction). The constraint dictates that additional shapes are forbidden between shapes A and B within the specified regions.





Constraint Category	Spacing
Constraint Type	Complex
OA Rule	Custom Compiler Custom Layer Constraint

Syntax:

```
forbiddenSpacing
    <string|LPP>layer
    <rangeValue>distanceRange
```

```
-widthRange rangeValue
-otherWidthRange rangeValue
-otherSpacingRange rangeValue
[-neighborWidthRange rangeValue]
[-excludedSpacing rangeValue]
[-excludedEndToEndSpacings valueArray]

[-isHard true|false]
[-comment [<string rule_id>] <string rule_desc>]
```

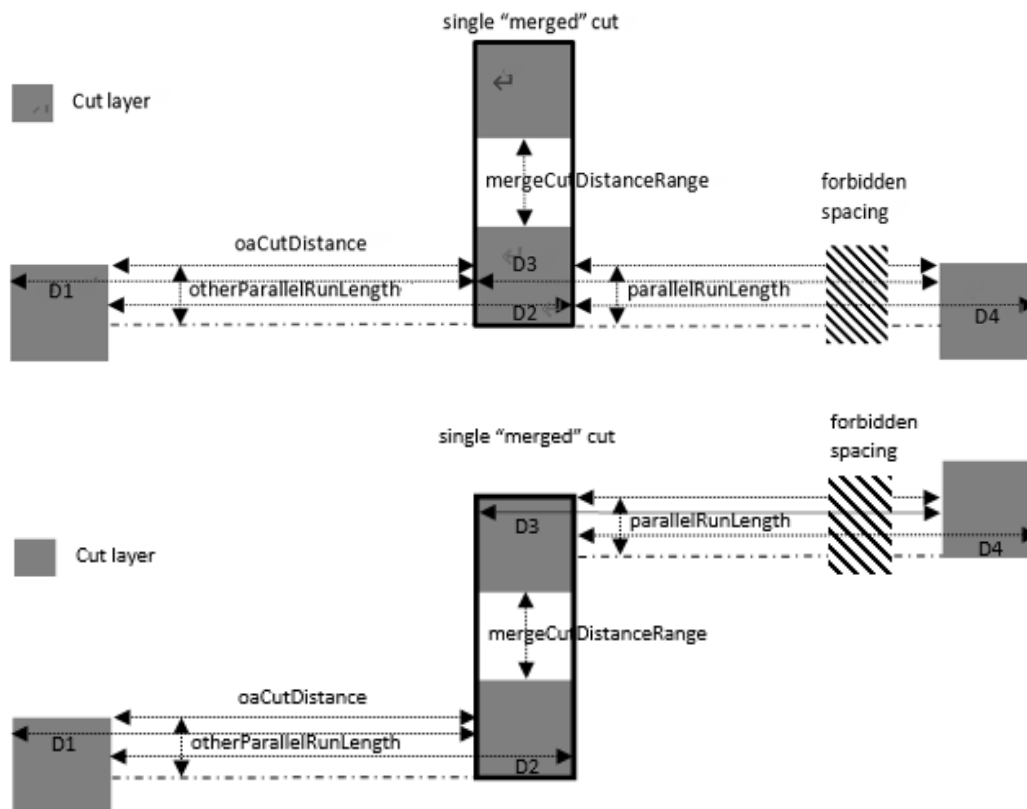
Argument	Description
layer	Layer can be defined as a string or as LPP.
distanceRange	Distances from the edge of shape A. Other shapes are not permitted within this range. To specify the ranges, see Range Data Syntax .
-widthRange	Range of widths of shape A. See Range Data Syntax .
-otherWidthRange	Range of widths of shape B. See Range Data Syntax .
-otherSpacingRange	Range of distances between shapes A and C. See Range Data Syntax .
-neighborWidthRange	Optional parameter defining width of the neighboring shape C in database units. See Range Data Syntax .
-excludedSpacing	Area to exclude from the constraint.
-excludedEndToEndSpacings	Optional parameter that defines additional exception conditions. The array holds multiples of four value objects: three float values and one range array value object. The <code>valueArray</code> is represented as: (float float float rangeArrayValue)* Each group of the four values represents width, end-to-end spacing, parallel run length of the shapes between shapes A and B, and spacing ranges between shapes A and B which allow the rule to be waived. All spacing ranges in <code>rangeArrayValue</code> must be sub-ranges of the forbidden spacing ranges specified in the constraint value.

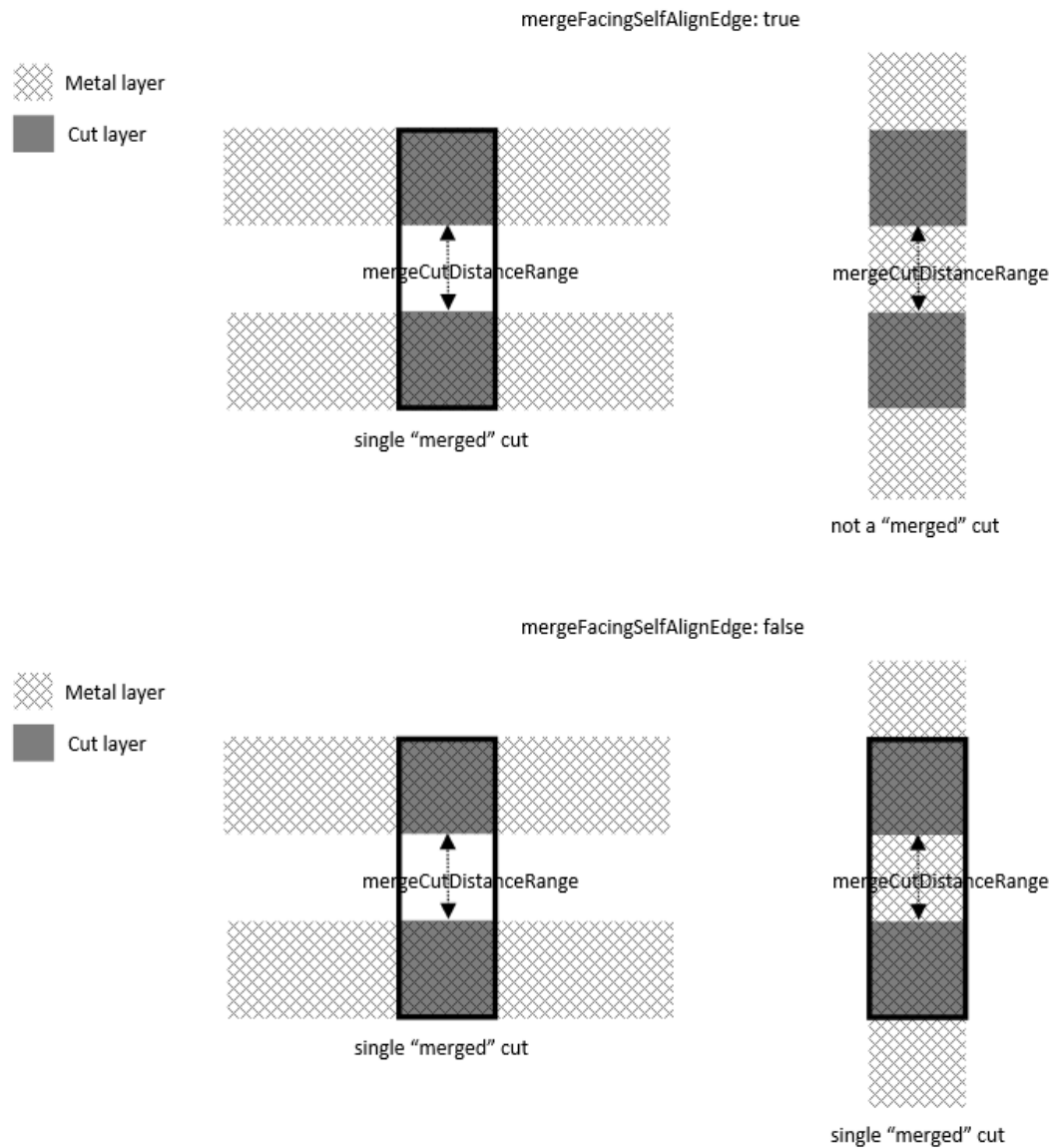
For example:

```
forbiddenSpacing m1 ((>=0.1 <=0.2) (0.5 0.6) (0.7 <=0.9))
                  -widthRange (>=0.3 <=0.5)
                  -otherWidthRange (>=0.1 <=0.4)
                  -excludedSpacing floatValue
                  -neighborWidthRange (>=0.3 <=0.5)
                  -excludedEndToEndSpacings ((0.1 0.2 0.5 (>=0.1 <=0.2)
                                                (0.2 0.3 0.5 (>=0.2 <=0.3)
                                                )
                  )
```

forbiddenTwoSidedViaSpacing

Spacing between via cut shapes which project to each other can have restrictions depending on the presence of another cut shape on the other side of the cut shape under consideration. It can also depend on the mask color assigned to the cut shapes.





Constraint Category	Spacing
Constraint Type	Complex
OA Rule	Custom Compiler Custom Layer Constraint

Syntax:

```
forbiddenTwoSidedViaSpacing
  <string|LPP>layer
  <rangeValue>value
```

```

[-cutDistance floatValue]
[-cutPitchRange rangeData]
[-color colorName | any | same | diff]
[-otherParallelRunLength floatValue]
[-parallelRunLength floatValue]
[-spacingDirection any | horizontal | vertical ]
[-cutClass dualValue | cutClassName]
[-cutClassList cutClassArray|cutClassNameList]
[-leftcutClassList cutClassArray|cutClassNameList]
[-rightcutClassList cutClassArray|cutClassNameList]
[-mergedViaLocations center | centerOnly | sides
                        | sidesOnly | leftSide | leftSideOnly
                        | rightSide| rightSideOnly]
[-mergeCutDistanceRange rangeData]
[-mergeFacingSelfAlignEdge true|false]
[-bridgeViaLocations center | centerOnly | sides | sidesOnly
                        | leftSide | leftSideOnly| rightSide|
rightSideOnly]

[-isHard true|false]
[-comment [<string rule_id>] <string rule_desc>]

```

Argument	Description
layer	Layer can be defined as a string or as LPP.
value	Specifies the range of forbidden spacing values between cut shapes. To specify the ranges, see Range Data Syntax .
-cutDistance	Distance from the cut shape in the middle to the cut shape on the other side must be less than this value for the constraint to apply. This is required. This parameter or the <code>cutPitchRange</code> parameter must be given, but not both.
-cutPitchRange	Pitch between the cut shape in the middle and the cut shape on the other side must be in this range for the constraint to apply. To measure the pitch, it is necessary to measure two distances (D1 and D2) between edges of the cuts oriented in the same direction. If either of the distances D1 or D2 is in the given range, the constraint applies. This parameter or the <code>cutDistance</code> parameter must be given, but not both. Also, if this parameter is specified, the constraint value represents the range of forbidden pitch values (D3 and D4) between the cut shapes.
-color	This is an optional parameter defining the specific color of the cut shapes for which this constraint applies if the value is positive. A value of zero means any color. A value of -1 means same color and a value of -2 means diff color. Default is any. The <code>colorName</code> should be specified in the controls colors section and the color mask value cannot bigger than the <code>numberOfColorMasks</code> value specified in the <code>layerRules</code> functions.
-otherParallelRunLength	This is an optional parameter requiring minimum parallel overlap between the cut shape in the middle and the cut shape on the other side. The parallel overlap must be greater than or equal to this value for the constraint to apply.

Argument	Description
-parallelRunLength	This is an optional parameter requiring minimum parallel overlap between the cut shapes. The parallel overlap must be greater than or equal to this value for the constraint to apply.
-spacingDirection	This is an optional parameter defining the direction in which the constraint applies. 0: Any direction. 1: Horizontal direction. 2: Vertical direction.
-cutClass	If given, the constraint only applies if the via in the middle is from this cut class. If the middle via is a merged via, both cuts forming the merged via must be from this cut class. Note: The via to the left and the via to the right can be from any cut class. The <code>-cutClass</code> parameter and <code>-cutClassList</code> parameter are mutually exclusive.
-cutClassList	Array of <code>oaDualInt</code> values, each defining width and height of a cut class in database units. If given, the constraint only applies if the cut class of the via in the middle is from this list. If the middle via is a merged via, both cuts forming the merged via must be from this cut class list. Note: The via to the left and the via to the right can be from any cut class. The <code>-cutClass</code> parameter and <code>-cutClassList</code> parameter are mutually exclusive.
-leftCutClassList	Array of <code>oaDualInt</code> values, each defining width and height of a cut class in database units. If given, the constraint only applies if the cut class of the via to the left is from this list. If the left via is a merged via, both cuts forming the merged via must be from this cut class list.
-rightCutClassList	Array of <code>oaDualInt</code> values, each defining width and height of a cut class in database units. If given, the constraint only applies if the cut class of the via to the right is from this list. If the right via is a merged via, both cuts forming the merged via must be from this cut class list.
-mergeCutDistanceRange	If given, two fully aligned via cuts with distance between them in this range are considered a single “merged” cut with bounding box equal to the combined bounding boxes of the two original cuts, and all rule measurements are performed against the merged cut. The orientation of the merged cut is orthogonal to the forbidden spacing measuring direction.
-mergeFacingSelfAlignEdge	Optional parameter defining how “merged” cuts are formed. By default, to form a “merged” cut, the facing fully aligned edges of the original cuts must be self-aligned edges of self-aligned vias. If false, self-alignment is not required to form a “merged” cut. Default: true This parameter can be specified only if the <code>-mergeCutDistanceRange</code> parameter is also specified.

Argument	Description
-mergedViaLocations	Provides additional conditions for applying the rule depending on the presence and location of merged vias among the three vias under consideration. For enumeration names and values: - center (1): The center via must be a merged via. No restrictions on the left and right vias. - centerOnly (2): The center via must be a merged via. The left and right vias must be single cut vias. - sides (3): Both left and right vias must be merged vias. No restrictions on the center via. - sidesOnly (4): Both left and right vias must be merged vias. The center via must be a single cut via. This parameter can be specified only if the <code>-mergeCutDistanceRange</code> parameter is also given.
-bridgeViaLocations	Provides additional conditions for applying the rule depending on the presence and location of bridge vias among the three vias under consideration.

Note:

- The `colorName` must be defined in the [Controls Section](#), and the color mask value cannot be greater than the `numberOfColorMasks` value defined in the [functions](#) section.
- If `-cutClass` parameter is specified with `cutClassName` in ASCII file, technology importer looks-up the first `cutClass` constraint specified in the same hierarchical constraint group to convert to cut class size. If a `cutClass` constraint does not exist then an error message is displayed. When exporting, technology exporter looks-up the first `cutClass` constraint specified in the same hierarchical constraint group to convert cut class size to cut class name. If a `cutClass` constraint does not exist, `-cutClass` parameter with cut class size in dual value is exported.
- Either `cutDistance` or `cutPitchRange` should be defined, but not both.
- If the `-mergeCutDistanceRange` parameter is specified and the `-cutClass` parameter or `-cutClassList` or `leftCutClassList` or `rightCutClassList` parameter is also given, the cut class restrictions apply to the size of original cuts, not to the merged cut size.
- The `leftCutClassList` and `rightCutClassList` parameters are used to control cut classes for neighboring vias separately on each side. If the spacing direction is horizontal, `leftCutClassList` is for the via on the left, and `rightCutClassList` is for the via on the right. If spacing direction is vertical, `leftCutClassList` is for the via on the bottom, and `rightCutClassList` is for the via on the top.

- The `mergedViaLocation` parameter is defined if `mergeCutDistanceRange` is not defined.
- The following error messages are displayed in the following situations:
 - The `-mergeFacingSelfAlignEdge` parameter is defined if `-mergeCutDistanceRange` is not defined.
 - `-cutClass` parameter be defined and `-cutClassList` parameter be defined.
 - `-mergedViaLocations` parameter be defined if `-mergeCutDistanceRange` is not defined.

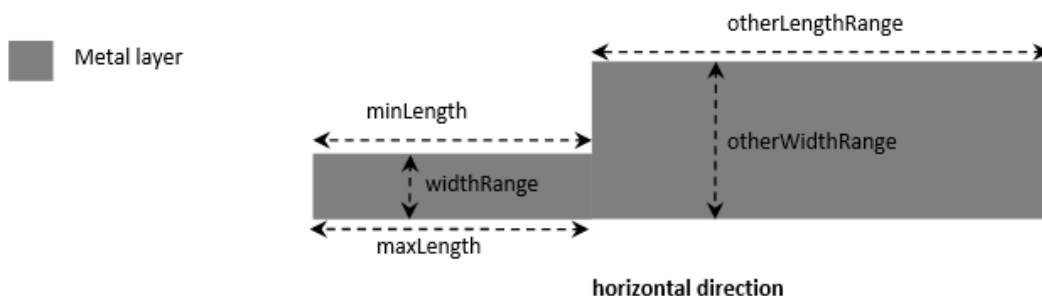
For example:

```
forbiddenTwoSidedViaSpacing V1 (>=0.08 0.09)
                             -cutDistance 0.07
                             -color same
                             -cutClass 0.06 0.08 // or -cutClass via12
                             -spacingDirection horizontal
                             -mergedViaLocations center
                             -mergeCutDistanceRange (0.07 0.08)

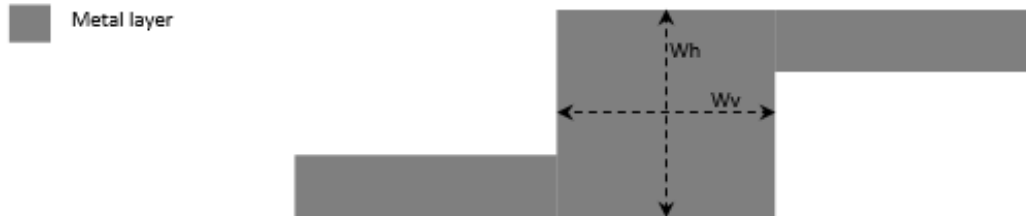
forbiddenTwoSidedViaSpacing V2 (>=0.08 0.09)
                             -cutPitchRange (0.06 0.07)
                             -color same
                             -cutClass 0.06 0.08
                             -leftCutClassList ((0.06 0.07))
                             -rightCutClassList ((0.05 0.06) (0.06 0.07))
                             -spacingDirection horizontal
                             -mergedViaLocations center
                             -mergeCutDistanceRange (0.07 0.08)
                             -bridgeViaLocations center
```

forbiddenWidthTransition

Some processes restrict what wire widths can be used during wire width transition. A width transition is when a wire does not change its direction but changes its width. Optionally, the applicability of the rule may depend on the length of the wire segments, wire direction (preferred or non-preferred) and color.



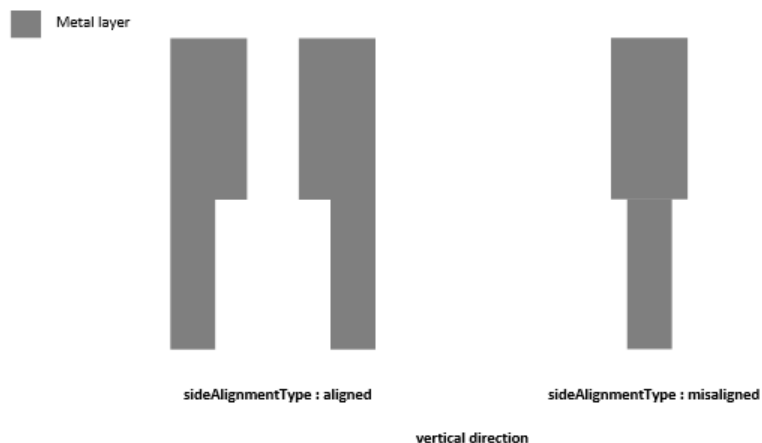
In some situations, it is not obvious if there is a wire width transition or a jog. A jog is when wire changes its direction.



To distinguish between a width transition and a jog, it is necessary to determine the directions of the adjacent wire segment. There are several considerations to take into account when deciding if a wire segment is horizontal or vertical:

- First of all, horizontal and vertical sizes of a wire segment should be checked. If the horizontal size (W_v) is an allowed width for a vertical wire, and the vertical size (W_h) is not an allowed width for a horizontal wire, then the direction of the wire segment is vertical.
- If the vertical size (W_h) is an allowed width for a horizontal wire, and the horizontal size (W_v) is not an allowed width for a vertical wire, then the direction of the wire segment is horizontal.
- If both W_h and W_v are allowed widths in the corresponding directions, then the aspect ratio of the wire segment sizes should be checked. If W_v is greater than W_h , then the direction of the wire segment is horizontal.
- If W_h is greater than W_v , then the direction of the wire segment is vertical. If W_h is equal to W_v , then the wire segment orientation is the same as preferred routing direction for the metal layer.

Figure 9 *sideAlignmentType*



Constraint Category	Spacing
Constraint Type	Complex
OA Rule	Custom Compiler Custom Layer Constraint

Syntax:

```

forbiddenWidthTransition
    <string|LPP>layer
    <bool>allowedValue
    -widthRange rangeData
    -otherWidthRange rangeData
    [-minLength float]
    [-maxLength float]
    [-otherLengthRange rangeData]
    [-color colorName|any]
    [-direction any|preferred|nonpreferred]
    [-sideAlignmentType any|aligned|misaligned]

    [-isHard true|false]
    [-comment [<string rule_id>] <string rule_desc>]

```

Argument	Description
layer	Layer can be defined as a string or as LPP.
allowedValue	If true, forbids the wire width transition defined by the constraint parameters.
-widthRange	Transition from wire width in this range in database units to wire width in the range specified by the <code>-otherWidthRange</code> parameter is forbidden. Note: <code>-widthRange</code> and <code>-otherWidthRange</code> are interchangeable, it is a pair of width ranges which cannot be put together when <code>-direction</code> of the wire does not change.
-otherWidthRange	Transition from wire width in this range in database units to wire width in the range specified by the <code>-widthRange</code> parameter is forbidden.
-minLength	If specified, the rule applies only if the length of the wire segment with width in <code>-widthRange</code> is greater than or equal to this value in database units.
-maxLength	If specified, the rule applies only if the length of the wire segment with width in <code>-widthRange</code> is less than or equal to this value in database units.
-otherLengthRange	If specified, the rule applies only if the length of the wire segment with width in <code>-otherWidthRange</code> lies in this range in database units.
-color	If given, the constraint only applies when the wire has this mask color (positive value). A value of zero means any color. Default is any.

Argument	Description
-direction	The direction of wire segments the rule applies to. Default is any. 0: Any direction. 1: Preferred direction. 2: Non-preferred direction.
-sideAlignmentType	Optional parameter defining restrictions on the alignment of the wire segments sides. Default is any. - If this value is 1 (aligned), the rule only applies if one side of the wire segments is aligned. - If this value is 2 (misaligned), the rule only applies if both sides of the wire segments are not aligned. See Figure 9.

For example:

```
forbiddenWidthTransition M1 true -widthRange (>=0.3 <=0.5)
                                -otherWidthRange (>=0.4 <=0.6)
                                -minLength 0.2 -minLength 0.3
                                -otherLengthRange (>=0.3 <=0.6)
                                -color red -direction preferred
                                -sideAlignmentType aligned
```

gateClearance

This constraint specifies spacing ranges for the clearance between a gate and neighboring poly shapes.

Constraint Category	Clearance
Constraint Type	Complex
OA Rule	oacGateClearance

Syntax:

```
gateClearance
    <string|LPP>layer
    <rangeValue>value
    -minLengthRatio float
    [-maxChannelLength

    [-isHard true|false]
    [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or as LPP.

Argument	Description
value	Spacing ranges for the clearance between a gate and neighboring poly shapes. To specify the ranges, see Range Data Syntax .
-minLengthRatio	The <code>minLengthRatio</code> (oaFloatValue) parameter is used to indicate the allowed ratio. The rule does not apply for ratios less than those indicated by the parameter.
-maxChannelLength	This rule might apply only to gates up to a certain gate shape width (channel length). The <code>maxChannelLength</code> (oaInt) parameter is used to indicate the maximum channel length.

For example:

```
gateClearance poly ((>=0.1 <=0.15) (0.2 0.25))
-minLengthRatio 0.8
```

gateOrientation

This layer constraint specifies the orientation required for building gates. The acceptable options are horizontal, vertical, or any.

Gates can be built using any orientation if either of the following conditions are true:

- The orientation is specified with this constraint is *any*.
- If this constraint is not specified.

An optional parameter limits application of the constraint to only those gates that have a width equal to or less than a specified value.

Constraint Category	Miscellaneous
Constraint Type	Complex
OA Rule	oacGateOrientation

Syntax:

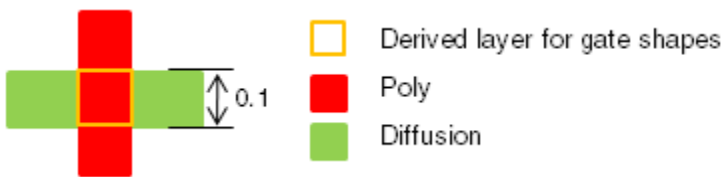
```
gateOrientation
    <string|LPP>layer
    <direction any|horizontal|vertical>
    [-width <float>]

    [-isHard true|false]
    [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Name of the layer or LPP.
direction	• any: Specifies the direction of the gate. Can be either horizontal or vertical. This is the default gate orientation. • horizontal: Specifies the horizontal direction of the gate. • vertical: Specifies the vertical direction of the gate.
-width	Specifies the maximum width of the gate.

In this example, the orientation of the gates for channel length ≤ 0.1 is specified.

Rule	Description	Value (in microns)
PO.O.1	Orientation of the gates need to be vertical for gates with channel length $\leq 0.1\mu\text{m}$	N/A



gateOrientation GATE vertical -width 0.1

generalCutClassSpacing

Both `minCutClassSpacing` and `allowedCutSpacing` are commonly used to model cut spacing rules. There are lots of common functionalities between the two constraints and also some differences. It is not clear under what situations which one should be used, thus causing confusions to PDK developers. They are also both table-based constraints which makes them hard to use because a lot of times having a full table filled with meaningful values is not an easy task. Considering all of these, this new constraint is used to replace the usage of `minCutClassSpacing` and `allowedCutSpacing` in most of the cases. Moving forward new requirements on cut spacing constraints are also supported by this constraint as much as possible.

edgeX, edgeY, and edge are Manhattan spacing while corner and center are Euclidean spacing.

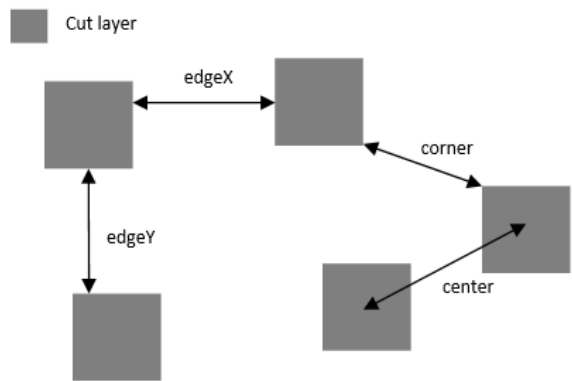
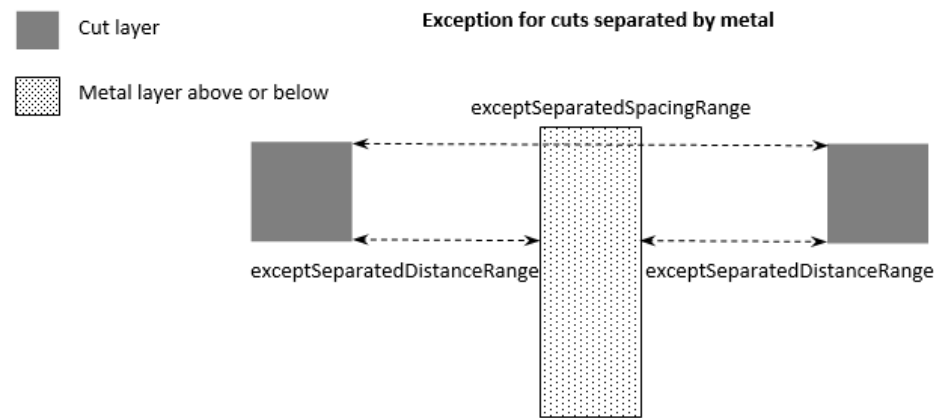


Figure 10 Exception Rule



Constraint Category	Spacing
Constraint Type	Complex
OA Rule	Custom Compiler Custom Layer Constraint

Syntax:

```
generalCutClassSpacing
    <string|LPP>layer
    <intValue|rangeValue|rangeArrayValue>spacingValue
    -cutSpacingType edgeX|edgeY|corner|center|edge
    [-cutClassList cutClassArray|
        cutClassNameList]
    [-cutClassList2 cutClassArray|
        cutClassNameList]
    [-color colorName|any|same|diff|cycle]
    [-connectivityType
```

```
anyConnectivityType|
sameNetConnectivityType|
contiguousShapesConnectivityType|
directShapesConnectivityType|
sameViaConnectivityType]
[-exceptConnectivityType
anyConnectivityType|
sameNetConectivityType|
contiguousShapesConnectivityType|
sameViaConnectivityType|
directShapesConnectivityType]
[-xParallelRunLengthRange rangeValue]
[-yParallelRunLengthRange rangeValue]]
[-selfAlignViaType any|selfAlign|SA|nonSelfAlign|NSA]
[-selfAlignViaType2 any|selfAlign|SA|nonSelfAlign|NSA]
[-selfAlignEdgeType any|selfAlign|SA|nonSelfAlign|NSA]
[-selfAlignEdgeType2 any|selfAlign|SA|nonSelfAlign|NSA]
[-otherLayer layerName]
[-otherLayerWidth rangeData]
[-otherLayerInteractType any|sameShape|diffShapes]
[-otherLayerDirectionType any|same|diff|horizontal|vertical]
[-exactAligned intValue]
[-exceptExactAligned true|false]
[-cutExtension float1 float2]
[-exceptSeparatedByLayer layerName]
[-exceptSeparatedDistanceRange rangeData]
[-exceptSeparatedSpacingRange rangeData]

[-isHard true|false]
[-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Name of the layer or LPP.
spacingValue	intValue : Minimum spacing value between specific cut classes in database units. rangeValue : Allowed spacing range between specific cut classes in database units. rangeArrayValue : Allowed spacings as an array of ranges between specific cut classes in database units.
-cutSpacingType	Specifies the type of spacing with the following: • edgeX: Specifies horizontal spacing between edges. • edgeY: Specifies vertical spacing between edges. • corner: Specifies corner to corner spacing. • center: Specifies center to center spacing. • edge: Specifies spacing between edges.
-cutClassList	An Array of values, each defining width and height of cut class for the specific index.

Argument	Description
-cutClassList2	An Array of values, each defining width and height of cut class for the specific index. This parameter should not be defined if <code>cutClassList</code> is not defined. When both <code>cutClassList</code> and <code>cutClassList2</code> are defined, one cut should come from <code>cutClassList</code> and the other cut should from <code>cutClassList2</code> otherwise the constraint does not apply. Note: If none of <code>cutClassList</code> or <code>cutClassList2</code> are defined, constraint applies to all the cut classes. If only <code>cutClassList</code> is defined, the constraint applies to all the cut class combinations defined by the list. For example, if <code>cutClassList</code> has three cut classes: A, B, and C, the constraint applies when two cut's cut classes belong to either of these combinations: A-A, A-B, A-C, B-B, B-C, and C-C. If both <code>cutClassList</code> and <code>cutClassList2</code> are defined, the constraint applies when one cut's cut class is defined in one list and the other cut's cut class is defined in the other list. For example, if <code>-cutClassList</code> is (A B) and <code>-cutClassList2</code> is (C D), the constraint applies when two cuts' cut classes belong to either of these combinations: A-C, A-D, B-C, and B-D.
-color	Defines the specific color of the cut shapes for which this constraint applies if the value is positive. A value of zero means any color. A value of -1 means same color and a value of -2 means diff color. A value of -3 means cycle color which is for the rules that need to avoid odd loops. The default value is diff. If the cut layer is uncolorable, this parameter is not considered.
-connectivityType	Indicates whether the constraint applies only if the cuts are on the same net, same metal, same wire segment, same via, or any.
-exceptConnectivityType	Specifies the type of connectivity between the two cuts that cause the constraint to not apply.
-xParallelRunLengthRange	Defines the x parallel run length range within which the constraint to apply.
-yParallelRunLengthRange	Defines the y parallel run length range within which the constraint to apply.
-selfAlignViaType	Enumeration indicating the type of <code>-selfAlignVia</code> this constraint applies to. If only <code>-selfAlignViaType</code> is defined, it applies to both cuts. This parameter supports the following three enumeration values: • <code>any</code>: Self-aligned via or non self-aligned via. • <code>selfAlign</code>: Self-aligned via only. • <code>nonSelfAlign</code>: Non self-aligned via only.
-selfAlignViaType2	See <code>-selfAlignViaType</code>. This parameter should not be defined if <code>-selfAlignEdgeType</code> is not defined. When both <code>-selfAlignEdgeType</code> and <code>-selfAlignEdgeType2</code> are defined, <code>-selfAlignEdgeType</code> applies to one cut and <code>-selfAlignEdgeType2</code> applies to the other cut.

Argument	Description
-selfAlignEdgeType	Enumeration indicating the type of edges this constraint applies to. An edge can either be self-aligned edge, which only exists for a self-aligned via, or non self-aligned edge, which could come from either a self-aligned or non self-aligned via. If only <code>-selfAlignEdgeType</code> is defined, it applies to both cuts. This parameter supports the following three enumeration values: • <code>any</code>: Self-aligned via or non self-aligned edge. • <code>selfAlign SA</code>: Self-aligned edge only. • <code>nonSelfAlign NSA</code>: Non self-aligned edge only.
-selfAlignEdgeType2	See <code>-selfAlignEdgeType</code>. This parameter should not be defined if <code>-selfAlignEdgeType</code> is not defined. When both <code>-selfAlignEdgeType</code> and <code>-selfAlignEdgeType2</code> are defined, <code>-selfAlignEdgeType</code> applies to one cut and <code>-selfAlignEdgeType2</code> applies to the other cut.
-otherLayer	The cut shapes must interact with material on the given layer, otherwise the constraint doesn't apply.
-otherLayerWidth	If given along with <code>-otherLayer</code>, the interacting material under the cut shapes must have a width in this range, otherwise the constraint doesn't apply. To specify the ranges, see Range Data Syntax.
-otherLayerInteractType	This parameter should be defined together with <code>-otherLayer</code>. It defines the interaction type of the shapes on other layer with the two cuts. This parameter supports the following three enumeration values: • <code>any</code>: Two cuts could interact with either the same shape, or different shapes on <code>-otherLayer</code>. • <code>sameShape</code>: Two cuts should interact with the same shape on <code>-otherLayer</code>. • <code>diffShapes</code>: Two cuts should interact with different shapes on <code>-otherLayer</code>.
-otherLayerDirectionType	This parameter should be defined together with <code>-otherLayer</code>. It specifies the direction configuration of the interacting shapes on other layer. This parameter supports the following five enumeration values: • <code>any</code>: The shapes on <code>-otherLayer</code> two cuts interacting with can be in any directions. • <code>same</code>: The shapes on <code>-otherLayer</code> two cuts interacting with should be in the same direction. • <code>diff</code>: The shapes on <code>-otherLayer</code> two cuts interacting with should be in the different directions. • <code>horizontal</code>: The shapes on <code>-otherLayer</code> two cuts interacting with should be horizontal. • <code>vertical</code>: The shapes on <code>-otherLayer</code> two cuts interacting with should be vertical.
-exactAligned	If given and the value is 1, the constraint applies only when two facing edges have the exact same size and also fully aligned. This constraint is ignored if the value is 0. This parameter cannot be 1 when the <code>-exceptExactAligned</code> parameter is true.

Argument	Description
-exceptExactAligned	If true, the constraint applies only between two edges which face each other (the parallel run length is greater than or equal to zero) but are not fully aligned (the length of the edges is different or the parallel run length is less than the edge length). Default: false This parameter cannot be true when <code>-exactAligned</code> parameter is 1.
-cutExtension	Optionally specifies an extension distance applied to either one of the cut before measuring cut spacing. The first value applies to horizontal direction and the second value applies to vertical direction. This parameter doesn't impact the checking of other parameters. When checking other parameters, the original cut dimension is still used.
-exceptSeparatedByLayer	Optional parameter to specify the metal layer below or the metal layer above the cut layer. If given, the rule does not apply if there is a metal shape on this layer between two via cuts under consideration. See Figure 10 .
-exceptSeparatedDistanceRange	Optional parameter to specify an additional condition to apply the rule exception in case when the cuts are separated by a metal layer shape. If given, the rule does not apply only when the distance between the cuts and the metal shape in between is in this range in database units. See Figure 10 . This can be specified only if the <code>-exceptSeparatedByLayer</code> parameter is also specified.
-exceptSeparatedSpacingRange	Optional parameter to specify an additional condition to apply the rule exception in case when the cuts are separated by a metal layer shape. If given, the rule does not apply only when the spacing between the cuts is in this range in database units. See Figure 10 . This can be specified only if the <code>-exceptSeparatedByLayer</code> parameter is also specified.

Note:

- If only `-selfAlignViaType` is defined, both cuts should satisfy this parameter so that the constraint applies. If both `-selfAlignViaType` and `-selfAlignViaType2` are defined, one cut should satisfy `-selfAlignViaType` and the other cut should satisfy `-selfAlignViaType2`. The rules for `-selfAlignEdgeType` and `-selfAlignEdgeType2` are similar.

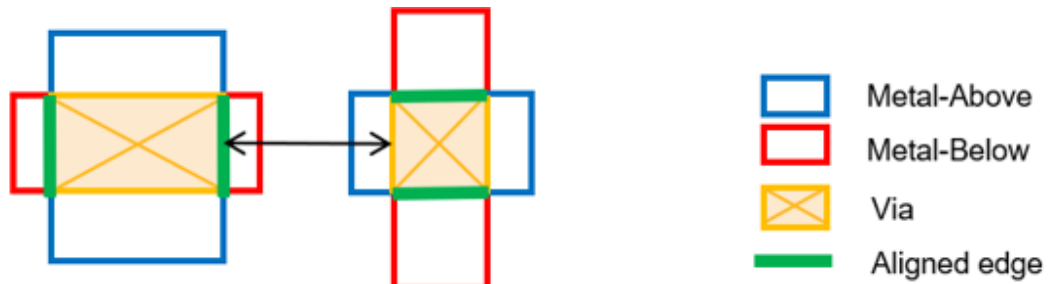
When multiple parameters with postfix 2 are defined together (for example, `-cutClassList2`, `-selfAlignViaType2`, and `-selfAlignEdgeType2`), these parameters should apply to the same cut, while those corresponding parameters without postfix 2 should apply to the same cut.

Two cuts of different cut classes are both self-aligned (edges aligned with metal shapes on above layer). To model the spacing between self-aligned

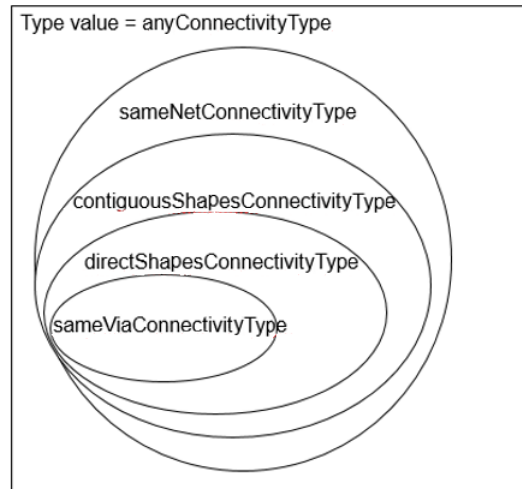
edge from one cut to the non self-aligned edge from the other cut, the constraint can be defined:

```
generalCutClassSpacing ...
    -cutClassList (A) -cutClassList2 (B)
    -selfAlignViaType selfAligned -selfAlignEdgeType
    selfAligned -selfAlignEdgeType2 nonSelfAligned
```

-selfAlignEdgeType applies to the cut of cutClass A, while
-selfAlignedEdgeType2 applies to the cut of cutClass B. Since only
-selfAlignViaType is defined, it applies to both cuts.



- Report errors and stop importing when following scenarios occurs:
 - -cutClassList2 parameter is defined if -cutClassList is not defined.
 - -xParallelRunLengthRange parameter is defined and -yParallelRunLengthRange parameter is defined.
 - -selfAlignViaType2 parameter is defined and -selfAlignViaType parameter is not defined.
 - -selfAlignEdgeType2 parameter is defined and -selfAlignEdgeType parameter is not defined.
 - -otherLayerInteractType parameter is defined if -otherLayer parameter is not defined.
 - -otherLayerDirectionType parameter is defined if -otherLayer parameter is not defined.
 - When -exceptConnectivityType has a larger scope than -connectivityType, -connectivityType parameters conflict with -exceptConnectivityType parameters.



- The `-exceptSeparatedDistanceRange` parameter is defined if `-exceptSeparatedByLayer` is not defined.
- The `-exceptSeparatedSpacingRange` parameter is defined if `-exceptSeparatedByLayer` is not defined.

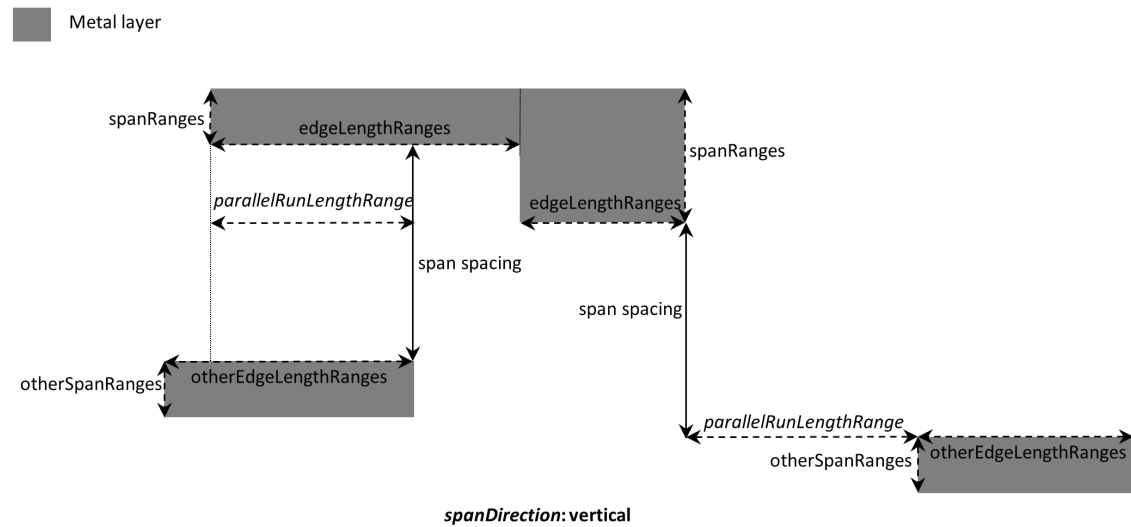
For example:

```
generalCutClassSpacing V1 0.04 -cutSpacingType edgeX
    -cutClassList ((0.06 0.08))
    -cutClassList2 ((0.08 0.08)) -color same
    -exceptSeparatedByLayer M1
    -exceptSeparatedDistanceRange (0.05 0.08)
    -exceptSeparatedSpacingRange (0.1 0.16)
```

generalSpanSpacing

Both `minParallelSpanSpacing` and `allowedParallelSpanSpacing` constraints are commonly used to model spacing rules between parallel edges of shapes depending on the shapes spans. However, they are both table-based constraints which makes them hard to use because having a full table filled with meaningful values is not an easy task. The `generalSpanSpacing` constraint has simpler syntax and is intended to replace the `minParallelSpanSpacing` and `allowedParallelSpanSpacing` constraints in most cases.

This constraint specifies the spacing that is allowed between a pair of parallel edges depending on the shapes spans measured in the direction orthogonal to the orientation of the edges. The spacing is measured in the same direction as the spans. The allowed spacing also depends on the length of the edges and the parallel run length between them.



Constraint Category	Spacing
Constraint Type	Complex
OA Rule	Custom Layer Constraint

Syntax:

```
generalSpanSpacing layerName| LPP value [-isHard true|false]
[-comment ["rule_ref_no"] "string"]
value ::= FloatValue | rangeValue | rangeValueArray

[-spanDirection horizontal | vertical]
[-spanRanges rangeValueArray]
[-otherSpanRanges rangeValueArray] //default is >=0
[-edgeLengthRanges rangeValueArray] //default is >=0
[-otherEdgeLengthRanges rangeValueArray] //default is >=0
[-color colorName | any | same | diff ] // default value is any;
[-parallelRunLengthRange rangeData ] //default is >=0
```

Argument	Description
spanDirection	Defines the direction in which this constraint applies. Both the spacing between parallel edges and the shape spans are measured in this direction. Valid values are 1 (horizontal) and 2 (vertical).
spanRanges	The constraint applies only to the edges of the first shape where the shape span lies in one of the given ranges.
otherSpanRanges	The constraint applies only to the edges of the second shape where the shape span lies in one of the given ranges.

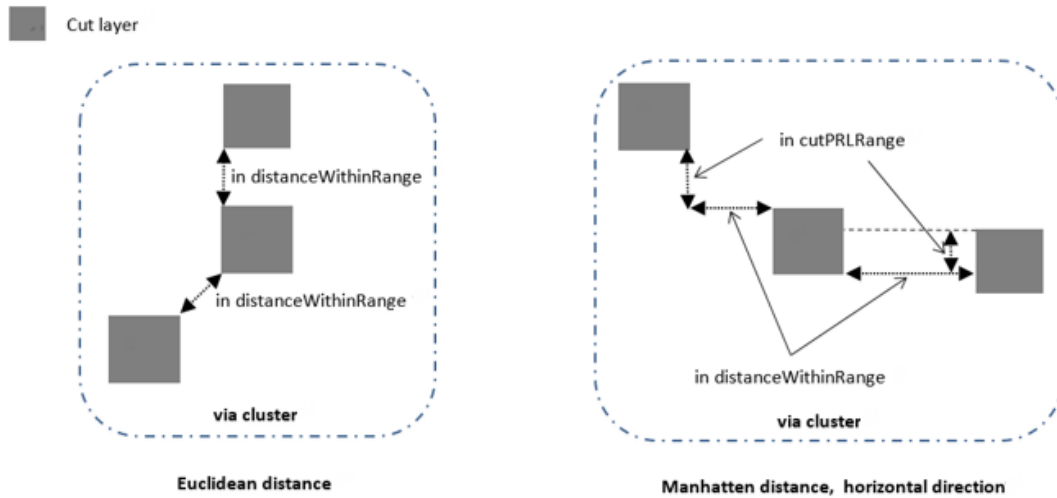
Argument	Description
edgeLengthRanges	The constraint applies only to the first shape edges which length lies in one of the given ranges.
otherEdgeLengthRanges	The constraint applies only to the second shape edges which length lies in one of the given ranges.
color	If a positive value is given, the rule applies only if both shapes are of this mask color. A value of -1 (same) means the rule applies only if the mask colors of both shapes are the same. A value of -2 (diff) means the rule applies only if the mask colors of the two shapes are different from each other. A value of zero means any color for both shapes.
parallelRunLengthRange	The constraint applies only if parallel run length between the two parallel edges lies in this range in database units. Must have a finite lower bound.

For example:

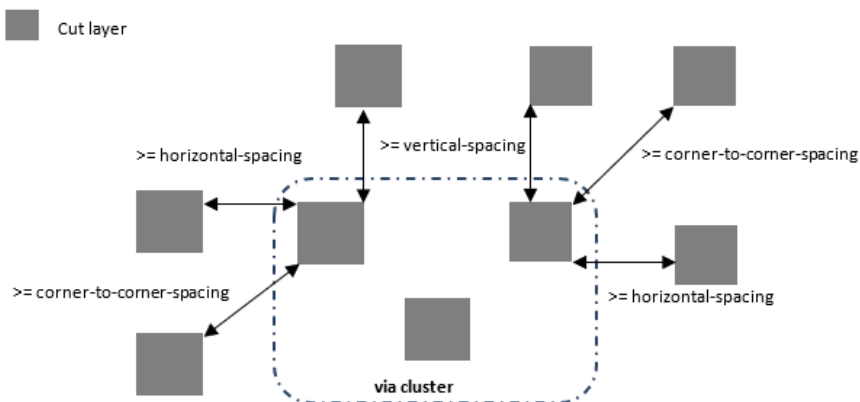
```
generalSpanSpacing M1 2 -spanRanges ((0.3 0.4) (0.5 0.6))
                                -spanDirection horizontal
                                -otherSpanRanges ((0.25 0.35) (0.45
0.55))
                                -edgeLengthRanges ((0.35 0.45) (0.45
0.55))
                                -otherEdgeLengthRanges ((0.5 0.55) (0.65
0.75))
                                -color red
                                -parallelRunLengthRange (0.5 0.6)
```

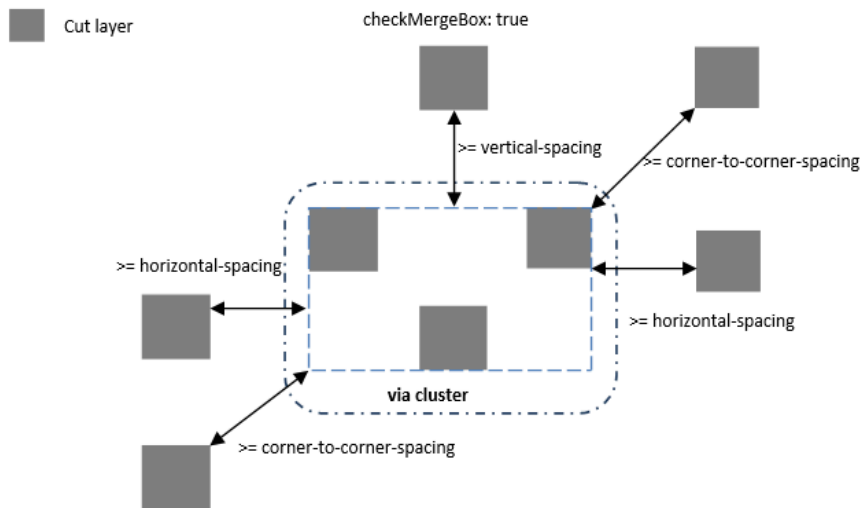
generalViaClusterSpacing

Advanced processes may require different cut spacing between vias forming a via cluster and other via cuts, as well as different cut spacing between via cuts from two different via clusters. A via cluster is a group of vias each of which has an adjacent via in the group within a given distance range. The distance between the adjacent cuts can be measured as Euclidean or as Manhattan distance, in horizontal, vertical, or both directions (depending on the parameters). Examples below show how `distanceWithinRange`, `distanceMeasureType` and `oaSpacingDirection` parameters are used.

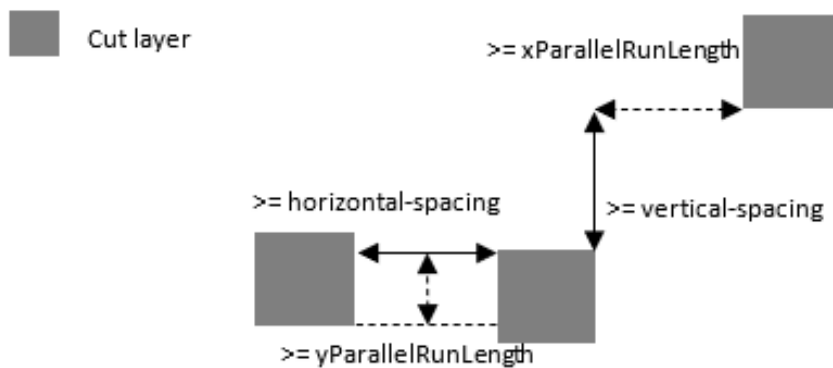


The rule requires a minimum spacing from a cut in a via cluster to another cut which does not belong to any via cluster. The minimum spacing is provided as three separate values: corner-to-corner-spacing, horizontal-spacing, and vertical-spacing. If another cut has negative parallel run with a cut in a corner via cluster, the diagonal spacing between the cuts - measured corner-to-corner - must be greater than or equal to the corner-to-corner-spacing value. If projections of the cuts to Y-axis overlap each other, the horizontal spacing between the cuts must be greater than or equal to the horizontal-spacing value. If projections of the cuts to X-axis overlap each other, the horizontal spacing between the cuts must be greater than or equal to the vertical-spacing value.

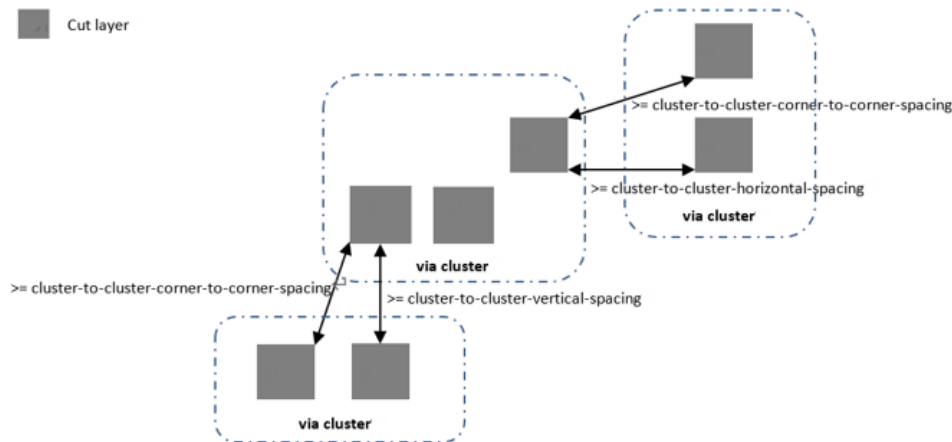




By default, horizontal-spacing and vertical-spacing are required for cuts with any non-negative parallel run length. Optionally, a particular value can be specified for the parallel run length along X or Y-axis, including a negative value.



In a similar manner, the rule requires a minimum spacing from a cut in one via cluster to a cut in another via cluster. The minimum spacing is provided as three separate values: cluster-to-cluster-corner-to-corner-spacing, cluster-to-cluster-horizontal-spacing, and cluster-to-cluster-vertical-spacing.



Constraint Category	Spacing
Constraint Type	Complex
OA Rule	Custom Layer Constraint

Syntax:

```
generalViaClusterSpacing
  <string|LPP>layer
  <float>(cornerSpacing horizontalSpacing verticalSpacing
clusterSpacing clusterHorizontalSpacing clusterVerticalSpacing)values
  -minNumCuts intValue
  -distanceWithinRange rangeData
  [-cutPRLRange rangeData]
  [-distanceMeasureType euclidean|manhattan]
  [-spacingDirection any|horizontal|vertical]
  [-checkMergeBox true|false]
  [-maxNumCuts intValue]
  [-cutClass dualValue|cutClassName]
  [-cutClassList cutClassArray|cutClassNameList]
  [-otherCutClassList cutClassArray|cutClassNameList]
  [-xClusterToNonClusterPRL intValue]
  [-yClusterToNonClusterPRL intValue]
  [-xClusterToClusterPRL intValue]
  [-yClusterToClusterPRL intValue]
  [-color colorName|any|same]

  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	The layer can be defined as a string or a layer-purpose pair.

Argument	Description
values	Array containing six <code>oaIntValue</code> objects. The spacing values in database units, in this order, are corner-to-corner-spacing, horizontal-spacing, vertical-spacing, cluster-to-cluster-corner-to-corner-spacing, cluster-to-cluster-horizontal-spacing, and cluster-to-cluster-vertical-spacing.
-minNumCuts	The minimum number of cuts in a via cluster required for the rule to apply.
-distanceWithinRange	Distance range in database units to determine if adjacent cuts belong to a via cluster.
-cutPRLRange	If given, only pairs of cuts with common parallel run length in this range are considered when forming a via cluster.
-distanceMeasureType	This is an optional parameter defining how the distance between cuts is measured in order to form a via cluster. The value represents an <code>oaDistanceMeasureEnum</code> value and can be either euclidean or manhattan. If manhattan is specified, the <code>cutPRLRange</code> parameter must also be specified. Default: euclidean.
-SpacingDirection	This is an optional parameter defining how the distance between cuts is measured in order to form a via cluster. By default, it is measured in any direction but can be limited to either horizontal or vertical direction.
-checkMergeBox	Optional parameter defining how the distance from a via cluster is measured. By default, the spacing requirements apply to each cut in the via cluster individually. If true, the spacing requirements apply to the merged bounding box of all cuts in the cluster. Default: false
-maxNumCuts	If given, it is a violation to have more than this number of cuts in a corner via cluster.
-cutClassList	An Array of values, each defining width and height of a cut class. If given, a via cluster is formed only by cuts from these cut classes.
-otherCutClassList	Array of <code>oaDualInt</code> values, each defining width and height of a cut class. If given, the requirements for spacing from a via cluster to a non-cluster cut are only enforced if the non-cluster cut class is in this list.
-xClusterToNonClusterPRL	The <code>vertical-spacing</code> applies to cuts with common parallel run length along X axis greater than or equal to this value. Default: 0.
-yClusterToNonClusterPRL	The <code>horizontal-spacing</code> applies to cuts with common parallel run length along Y axis greater than or equal to this value. Default: 0.
-xClusterToClusterPRL	The <code>cluster-to-cluster-vertical-spacing</code> applies to cuts with common parallel run length along X axis greater than or equal to this value. Default: 0.
-yClusterToClusterPRL	The <code>cluster-to-cluster-horizontal-spacing</code> applies to cuts with common parallel run length along Y axis greater than or equal to this value. Default: 0.

Argument	Description
-color	The constraint only applies when all cut shapes under consideration have this mask color (positive value). • A value of zero means any color. • A value of -1 means same color. Default: any.

Note:

- If `cutClass` parameter is specified with `cutClassName` in ASCII file, technology importer looks-up the first `cutClass` constraint specified in the same hierarchical constraint group to convert to cut class size. If a `cutClass` constraint does not exist, then an error message is displayed.

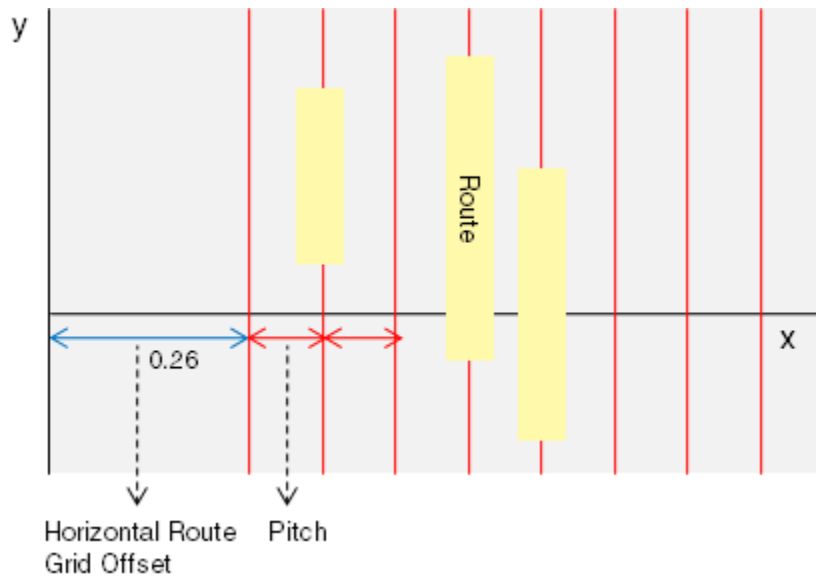
In exporting, technology importer looks-up the first `cutClass` constraint specified in the same hierarchical constraint group to convert cut class size to cut class name. If a `cutClass` constraint does not exist, export the `cutClass` parameter with cut class size in dual value.

For example:

```
generalViaClusterSpacing V1 (0.06 0.06 0.06 0.12 0.1 0.12)
    -minNumCuts 2
    -distanceWithinRange (>=0.05 <0.15)
    -spacingDirection horizontal
```

horizontalRouteGridOffset

This layer constraint specifies the route grid points on the X axis. The route grid points are completely define using the `horizontalRouteGridOffset` and [horizontalRouteGridPitch](#).



Constraint Category	Routing
Constraint Type	Simple
OA Rule	oacHorizontalRouteGridOffset

Syntax:

```
horizontalRouteGridOffset
  <string|LPP>layer
  <float>offsetValue

  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

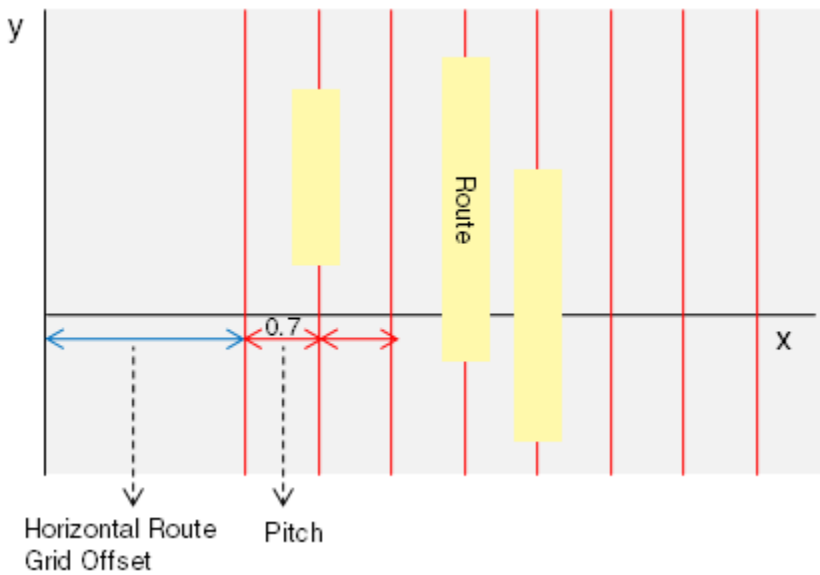
Argument	Description
layer	Layer can be defined as a string or as LPP.
offsetValue	Specifies the route grid offset points on the X axis.

For example:

```
horizontalRouteGridOffset      M2      0.26
```

horizontalRouteGridPitch

This layer constraint specifies the route grid points on the X axis. The route grid points are completely defined using horizontalRouteGridPitch and [horizontalRouteGridOffset](#).



Constraint Category	Routing
Constraint Type	Simple
OA Rule	oacHorizontalRouteGridPitch

Syntax:

```
horizontalRouteGridPitch
    <string|LPP>layer
    <float>pitchValue

    [-isHard true|false]
    [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or as LPP.
pitchValue	Specifies the route grid pitch points on the X axis.

For example:

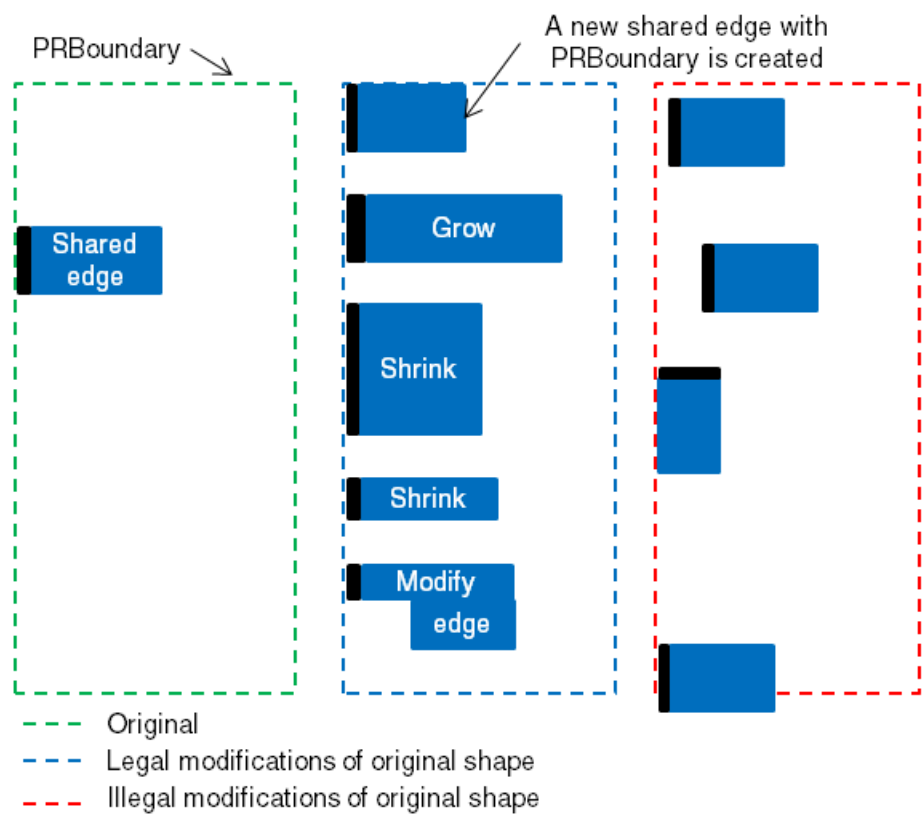
```
horizontalRouteGridPitch      M1  0.7
```

keepAlignedShapeAndBoundary

This layer constraint specifies that a layer that has a common edge with the oaDesign PRBoundary must continue to maintain the common edge. For the constraint to take effect, all shapes on the specified layer must also be entirely contained within the PRBoundary.

The constraint is also subject to the following considerations:

- If an application moves a shape away from the PRBoundary, other applications cannot enforce the constraint with respect to that shape.
- If an application moves a shape on a layer that has this constraint such that the shape now has a common edge with the PRBoundary, the constraint now takes effect on the shape.
- Moving the PRBoundary away from any edges shared with shapes on a specified layer also results in a violation.



Constraint Category

Placement and Alignment

Constraint Type

Simple

OA Rule oacKeepAlignedShapeAndBoundary

Syntax:

```
keepAlignedShapeAndBoundary
  <string|LPP>layer
  <bool>shapeBoundaryValue

  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or as LPP.
shapeBoundaryValue	Specifies the shape boundary value.

For example:

```
keepAlignedShapeAndBoundary PO true
```

layerShapeAngle

This layer constraint specifies the allowed geometry orientations for a layer.

Note:

The default orientation is designated by a single enumerated type and applies to all layers in the technology.

Constraint Category	Miscellaneous
Constraint Type	Complex
OA Rule	oacLayerShapeAngle

Syntax:

```
layerShapeAngle
  <string|LPP>layer
  <value all|orthogonalOnly|diagonalAndOrthogonal>

  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or a LPP.
value	• any: All angles. • orthogonalOnly: Orthogonal only. • diagonalAndOrthogonal: 45° or 135° and orthogonal.

In this example, the allowed angles for POLY shapes is mentioned:

Rule	Description	Value (in microns)
PO.O.2	Only Orthogonal POLY shapes are allowed	N/A

```
layerShapeAngle POLY orthogonalOnly
```

maxACCurrentDensity

Constraint Category	Density
Constraint Type	Complex
OA Rule	oacMaxACCurrentDensity

Syntax:

```
maxACCurrentDensity
    <string|LPP>layer
    <float|float1DTable|float2DTable>areaValue
    [-currentDensityType peak|RMS|average]
    [-hoursFactor hour1DTable]
    [-tempFactor temp1DTable]

    [-isHard true|false]
    [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or a LPP.
-currentDensityType	Optional parameter. Values must be one of: • peak • RMS (Default) • average

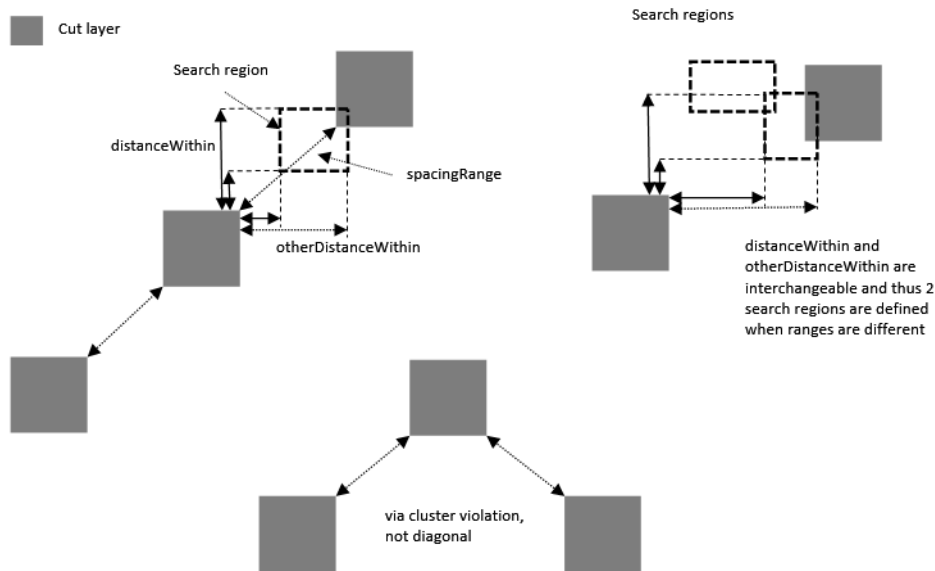
Argument	Description
-hoursFactor	Optional parameter. If present, the current density derived from the constraint's value can be multiplied by table entry for hours of operation to determine the time-dependent current density value. The interpolation and extrapolation of the table should be linear. The key of the table is time in hours and the table entry is a unit-less multiplicative factor.
-tempFactor	Optional parameter. If present, the current density derived from the constraint's value can be multiplied by the table entry for temperature to determine the temperature-dependent current density value. The multiplier should be derived from a linear interpolation (or extrapolation) of the table. The key of the table is temperature in degrees Celsius and the table entry is a unit-less multiplicative factor.

For example,

```
maxACCurrentDensity M1 9.3 -currentDensityType RMS
maxACCurrentDensity M1 8.653
((1 0.2) 9.36775)
((1 0.4) 19.5501)
((2 0.2) 39.9148)
((2 0.4) 80.6441)
-currentDensityType peak
-hoursFactor(0.4 ((1 0.6) (2 0.8)))
-tempFactor(0.2 ((15 0.5) (25 0.7)))
```

maxClusterDiagonalNumCuts

This layer constraint specifies the number of cuts allowed in a diagonal via cluster. An adjacent cut that interacts with a search region and in a given spacing range is considered part of a via group/cluster. The search region is defined by two ranges for X and Y dimension relative to a cut corner. A via that satisfies this criteria with a via already a member of via cluster will be added to the same via cluster. A via cluster must only be diagonally oriented and must have at most a certain number of vias in a diagonal.



Syntax:

```
maxClusterDiagonalNumCuts
  <string|LPP>layer
  <int>numCuts
  -distanceWithinRange rangeData
  [-otherDistanceWithinRange rangeData]
  -spacingRange rangeData
  [-cutClass <<float> <float> | cutClassName>]
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or a LPP.
numCuts	Specifies number of cuts allowed in a diagonal via cluster.
-distanceWithinRange	Specifies the range defining one dimension of the search region. This is required.
-otherDistanceWithinRange	Specifies the range defining the other dimension of the search region. If not given, the search region is defined by distanceWithinRange and is square. This is optional. To specify the ranges, see Range Data Syntax .
-spacingRange	The range in which the diagonal cut shapes must be within in order to apply as a cluster. This is required.
-cutClass	If specified, the specific cut class is required to be spaced away from the via cluster.

```
maxClusterDiagonalNumCuts v1 3 -distanceWithinRange (>=0.09 <=0.15)
                             -spacingRange (>=0.02 <=0.05) -cutClass 0.2 0.4
```

maxDCCurrentDensity

Constraint Category	Density
Constraint Type	Complex
OA Rule	oacMaxDCCurrentDensity

Note:

If you specify hour/temp as a float value, a warning message is displayed and the values are automatically rounded to an integer value.

Syntax:

```
maxDCCurrentDensity
    <string|LPP>layer
    <float|float1DTable>areaValue
    [-hoursFactor hour1DTable]
    [-tempFactor temp1DTable]

    [-isHard true|false]
    [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or a LPP.
-hoursFactor	Optional parameter. If present, the current density derived from the constraint's value can be multiplied by table entry for hours of operation to determine the time-dependent current density value. The interpolation and extrapolation of the table should be linear. The key of the table is time in hours and the table entry is a unit-less multiplicative factor.
-tempFactor	Optional parameter. If present, the current density derived from the constraint's value can be multiplied by the table entry for temperature to determine the temperature-dependent current density value. The multiplier should be derived from a linear interpolation (or extrapolation) of the table. The key of the table is temperature in degrees Celsius and the table entry is a unit-less multiplicative factor.

For example,

```
maxDCCurrentDensity M1 9.3
maxDCCurrentDensity M1 8.653
                    ((0.2 9.36775) (0.4 19.5501))
```

```
-hoursFactor(0.4 ((1 0.6) (2 0.8)))  
-tempFactor( 0.2 ((15 0.5) (25 0.7)))
```

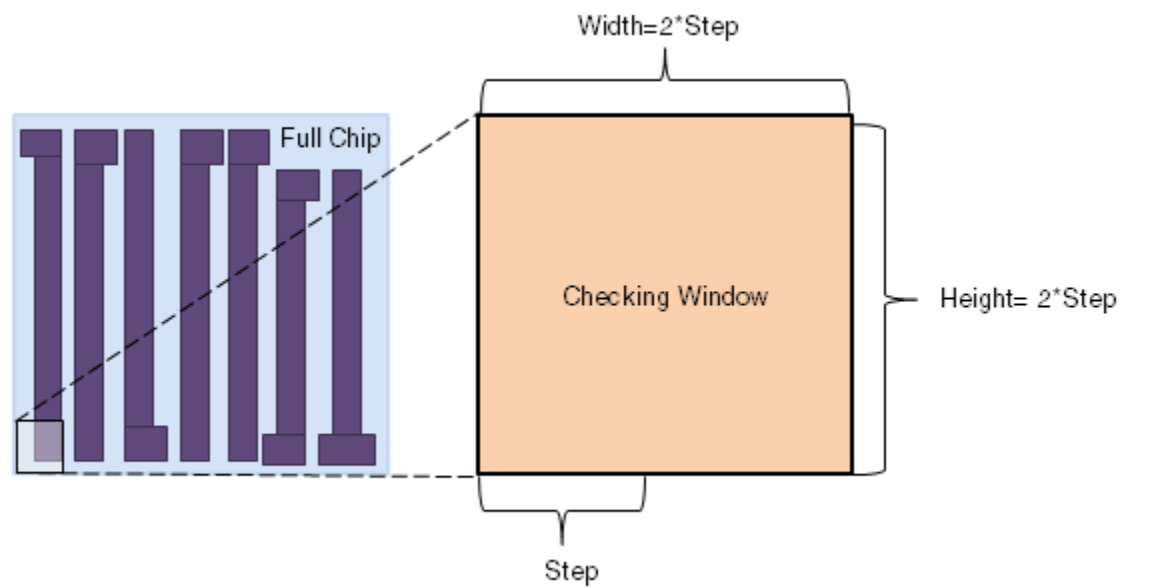
maxDensity

This layer constraint specifies the maximum percentage of an area of the design that must be filled with shapes on a given layer.

Density constraints are generally represented in two ways:

- As the maximum density for an area across the entire design.
- As a window-based maximum density. Here the area in consideration is a window that can be moved across the design.

You can use the minDensity layer constraint is used in conjunction with the maxDensity constraint to ensure optimum metal density for a design.



Constraint Category	Density
Constraint Type	Complex
OA Rule	oacMaxDensity

Syntax:

```
maxDensity  
  <string|LPP>layer  
  <float|float1DTable>areaValue  
  [-windowStepSize 1DTableParamValue]
```

```
[-isHard true|false]
[-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or a LPP.
areaValue	float: Maximum density required for the layer over the entire design float1DTable: Minimum density required for each window area of the design tiled into windows with a height and width each equal to twice the step size. defaultValue ((float float)*) [1DTableOptions]
-windowStepSize	This parameter is only applicable when the constraint value is a table. The value of this parameter is a table where the key is the step size in microns and the value is the window size in microns. (default value ((float float)*))

In this example, the maximum density of the POLY across the chip is mentioned:

Rule	Description	Value (in microns)
PO.DEN.1	Density of POLY across the chip	<=45%

```
maxDensity POLY 0.45
```

In this example, the maximum local density of the POLY is mentioned:

Rule	Description	Value (in microns)
PO.DEN.2	Density of POLY in 30um X 30um window	<=20%

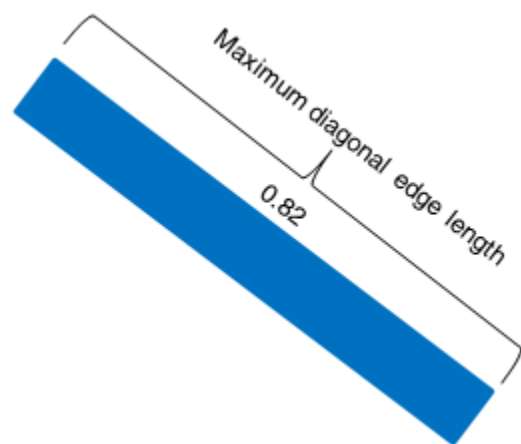
```
maxDensity POLY 0.2 ((15 0.1))
```

Additional Examples,

```
maxDensity m2 50 ((0.05 60) (0.06 80))
maxDensity m1 20 ((0.05 20) (0.06 30))
-windowStepSize (0.10 ((0.05 0.12) (0.06 0.18)))
```

maxDiagonalEdgeLength

This layer constraint specifies the actual distance permitted along the diagonal edge of a geometric object. The value is specified in microns.



Constraint Category	Length and Width Process Rules
Constraint Type	Simple
OA Rule	oacMaxDiagonalEdgeLength

Syntax:

```
maxDiagonalEdgeLength
    <string|LPP>layer
    <float>edgeLengthValue
    [-isHard true|false]
    [-comment [<string rule_id>] <string rule_desc>]
```

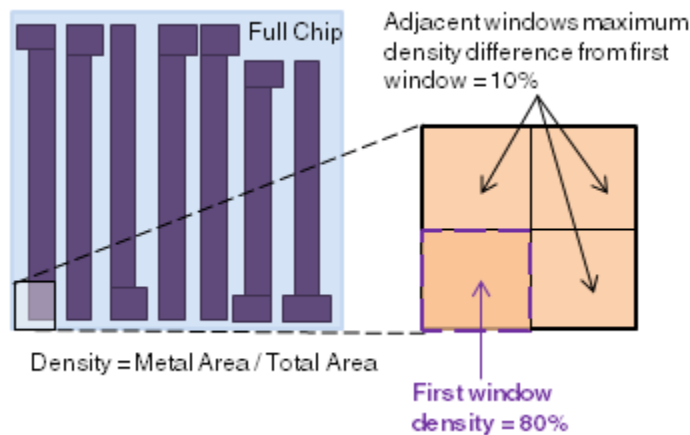
Argument	Description
layer	Layer can be defined as a string or as LPP.
edgeLengthValue	Specifies the actual distance permitted along the diagonal edge of a geometric object.

For example:

```
maxDiagonalEdgeLength M2 0.82
```

maxDiffDensity

The metal density for adjacent, non-overlapping density checking windows should be within a certain percentage range for some processes. The maximum density difference between different windows is represented as a float between 0 and 1.



Constraint Category	Density
Constraint Type	Complex
OA Rule	oacMaxDiffDensity

Syntax:

```
maxDiffDensity
  <string|LPP>layer
  <float1DTable>maxDiffValue
  -windowStepSize float1DParameterTable
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

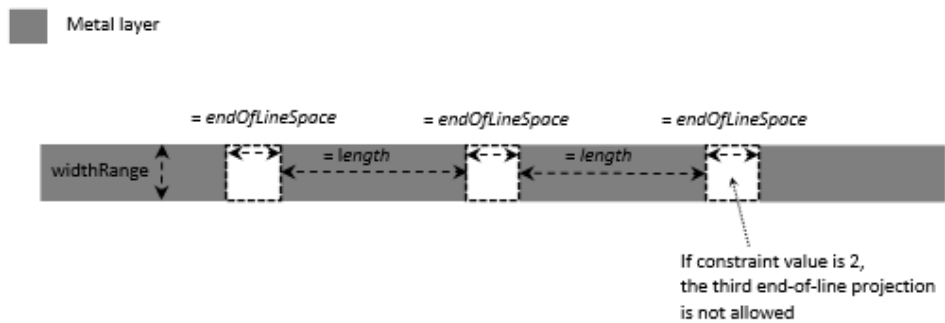
Argument	Description
layer	Layer can be defined as a string or as LPP.
maxDiffValue	Maximum density difference between adjacent windows. The maxDiffValue is represented as: <float>defaultDensity ((<float>density <float>stepsize)*)
-windowStepSize	The key is the different window size in microns and the value is the window size in microns. The windowStepSize is represented as: <float>defaultDensity ((<float>density <float>stepsize)*)

For example:

```
maxDiffDensity m1 20((0.05 20) (0.06 30))
               -windowStepSize (0.10((0.05 0.12) (0.06 0.18)))
```

maxInlineEndOfLineProjections

Some processes may have additional limitation on number of consecutive metal shapes satisfying minimum length requirements allowed only in presence of cut-metal.



Constraint Category	Spacing
Constraint Type	Complex
OA Rule	Custom Compiler Custom Layer Constraint

Syntax:

```
maxInlineEndOfLineProjections
  <string|LPP>layer
  <int>numProjections
  -length float
  -endOfLineSpace float
  [-widthRange rangeData]
  [-color colorName|any|same|diff]
  [-spacingDirection horizontal|vertical|any]

  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or a LPP.
numProjections	The maximum number of end-of-line projections allowed in a row. The constraint value should be an integer number (number of projections).
-length	Exact length in database units of the aligned metal lines required for the constraint to apply.
-endOfLineSpace	Exact distance in database units between the aligned metal line ends required for the constraint to apply.

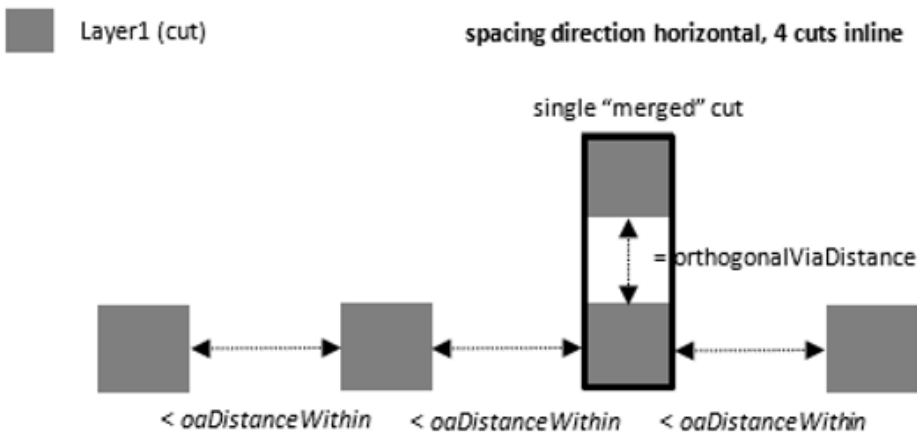
Argument	Description
-widthRange	If given, the constraint only applies when the width of metal lines lies in this range in database units. Default: none
-color	This is an optional parameter. The constraint only applies when all metal shapes under consideration have this mask color if the value is positive. A value of zero means any color. A value of -1 means same color and a value of -2 means diff color. Default is any. The <code>colorName</code> should be specified in the controls colors section and the color mask value cannot bigger than the <code>numberOfColorMasks</code> value specified in the <code>layerRules</code> functions.
-spacingDirection	This is an optional parameter specifying the orientation of metal lines to which the rule applies: 0: Any direction. 1: Horizontal direction. 2: Vertical direction. Default: 0

Example:

```
maxInlineEndOfLineProjections m1 0.04 -length 0.75 -endOfLineSpace 0.06
    -widthRange (>=0.05 <=0.06) -color same
```

maxInlineNumCuts

Advanced processes have limitations on how many cuts can be collinear (in line). The cuts must be within a specified distance for the constraint to apply. To be considered inline, the cuts need to be within a certain distance from each other and each cut must have a non-negative common parallel run with its immediate neighbor.



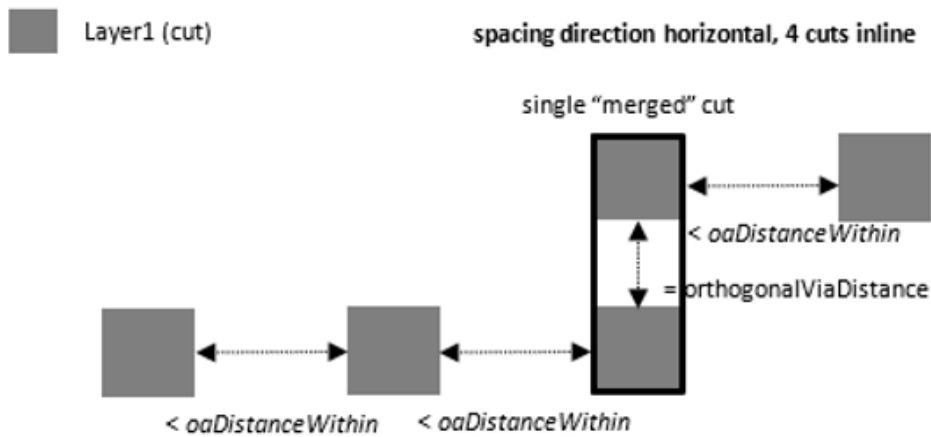
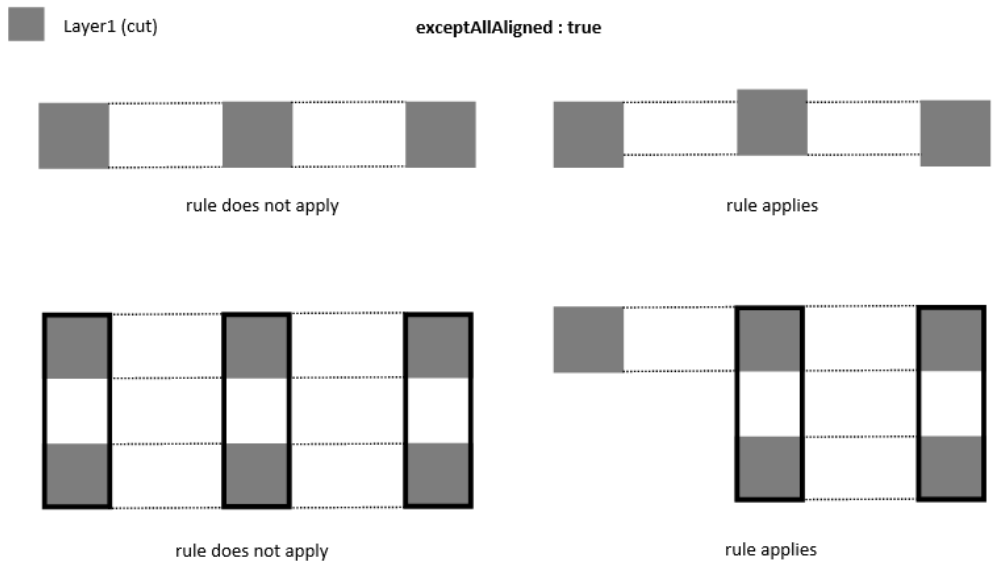


Figure 11 *-exceptAllAligned*



Constraint Category	NumCut
Constraint Type	Complex
OA Rule	Custom Compiler Custom Layer Constraint

Syntax:

```
maxInlineNumCuts
  <string|LPP>layer
  <int>numCuts
  -distanceWithin float|-distanceWithinRange rangeData
  [-spacingDirection horizontal|vertical|any]
```

```
[ -orthogonalViaDistance floatValue |
-orthogonalViaDistanceRange rangeData]
[-color string|any|same|diff]
[-cutClassList cutClassArray|cutClassNameList]
[-exceptAllAligned true|false]

[-isHard true|false]
[-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer on which the cuts are placed. <code>layer</code> can be a string (layer name) or a LPP.
numCuts	Maximum number of via cuts that can be collinear (in line).
-distanceWithin	Maximum distance between the via cuts. The cuts must have parallel overlap greater than 0 and less than the specified <code>-distanceWithin</code> to be considered collinear (in line). Either <code>-distanceWithin</code> or <code>-distanceWithinRange</code> must be specified.
-distanceWithinRange	If given, the cuts must have parallel overlap greater than 0 and be in the range of the specified <code>-distanceWithinRange</code> to be considered collinear (in line). Either <code>-distanceWithin</code> or <code>-distanceWithinRange</code> must be specified.
-spacingDirection	Specifies the direction of the wire: 0: Any direction. 1: Horizontal direction. 2: Vertical direction. Default: 0
-orthogonalViaDistance	If given, the rule only applies if there is a neighboring merged via with this exact distance in the orthogonal direction to one of the inline cuts.
-color	This is an optional parameter defining the specific color of the inline cut shapes for which this constraint applies if the value is positive. A value of zero means any color. A value of -1 means same color and a value of -2 means diff color. Default is any. The <code>colorName</code> should be specified in the controls colors section and the color mask value cannot bigger than the <code>numberOfColorMasks</code> value specified in the <code>layerRules</code> functions.
-orthogonalViaDistanceRange	Defines a range for a merged via in orthogonal direction. This parameter and <code>orthogonalViaDistance</code> are mutually exclusive.
-cutClassList	An Array of <code>oaDualInt</code> values, each defining width and height of a cut class. If given, the rule only applies when all via cuts in the 'row' are from this list. The <code>cutClassList</code> is represented as: <code>cutClassArray<float1> <float2> cutClassName<string></code>
-exceptAllAligned	If true, the rule does not apply if all cuts in the inline array are perfectly aligned (see Figure 11).

Example:

```
maxInlineNumCuts v1 3 -distanceWithin 0.55 -spacingDirection vertical

maxInlineNumCuts Vx 1 -distanceWithinRange (>=0.2 <0.5)
                    -spacingDirection horizontal
                    -orthogonalViaDistanceRange (>=0.3 <0.5)

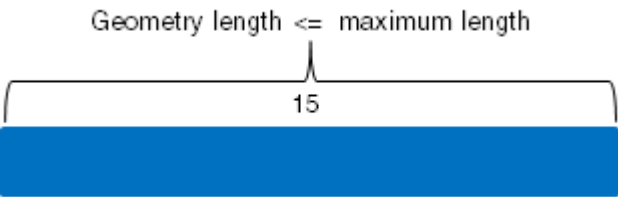
maxInlineNumCuts Vx 2 -distanceWithinRange (>=0.2 <0.5)
                    -spacingDirection horizontal
                    -exceptAllAligned true
```

maxLength

This layer constraint specifies the maximum length for any geometry on the specified layer. The value is specified in microns.

Note:

The maxLength would be a check on the larger dimension either horizontal or vertical.



Constraint Category	Length and Width Process Rules
Constraint Type	Simple
OA Rule	oacMaxLength

Syntax:

```
maxLength <string|LPP>layer
         <float>lengthValue
         [-isHard true|false]
         [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or as LPP.
lengthValue	Specifies the maximum length for any geometry on the specified layer.

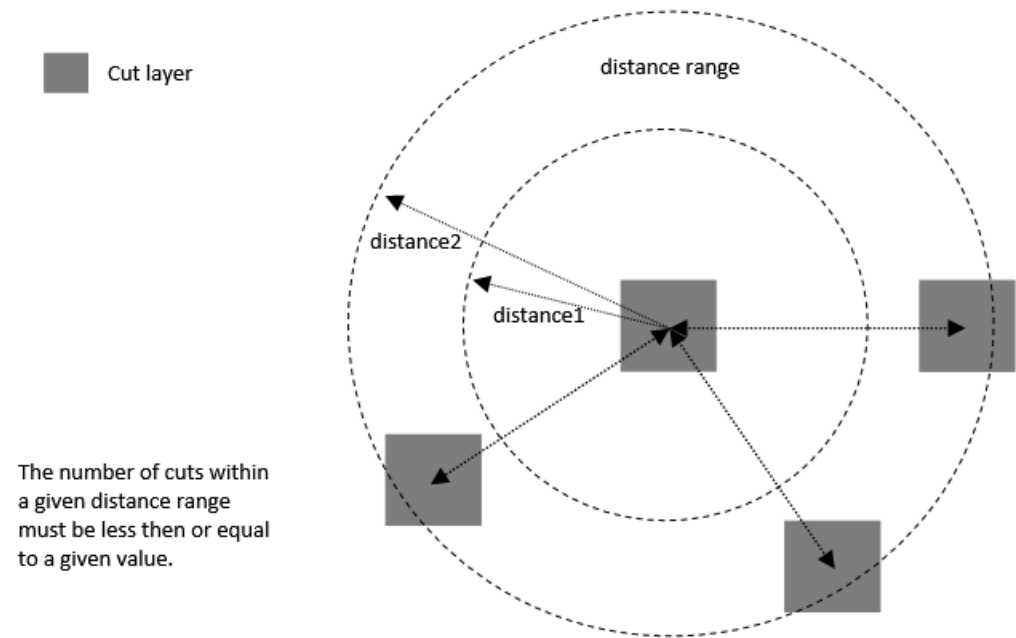
For example:

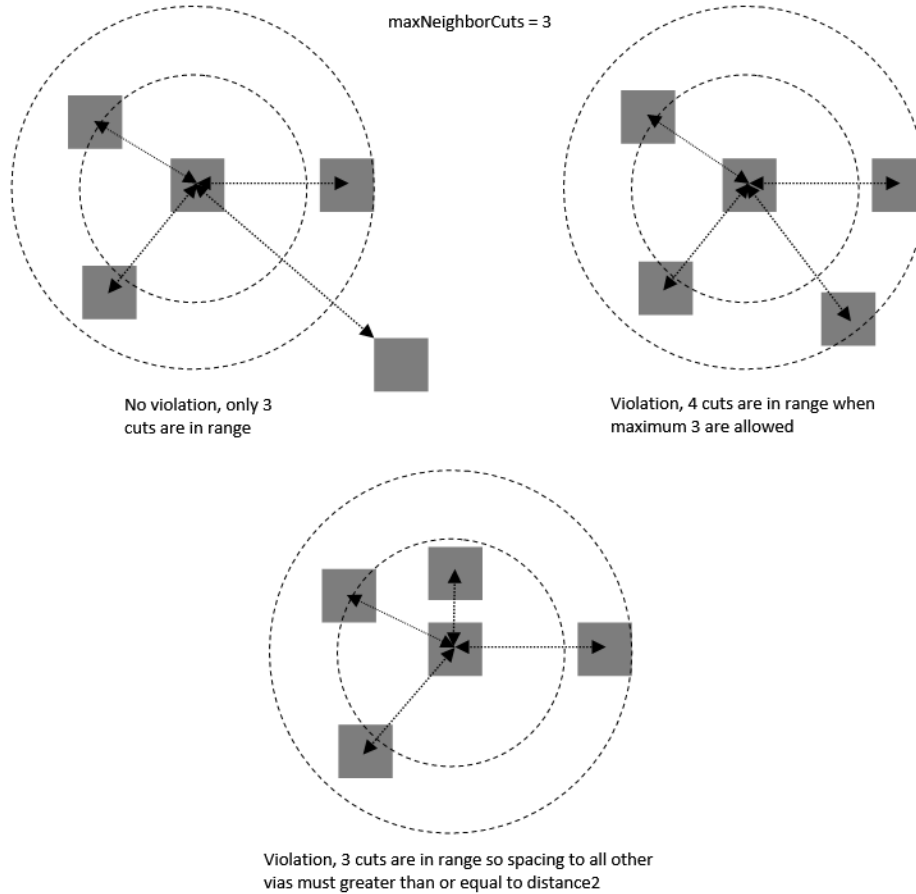
Rule	Description	Value (in microns)
M1.L.1	Maximum length of M1	15
M1.L.1R	Recommended maximum length of M1	0.1

```
maxLength M1 15
maxLength M1 0.1 -isHard false
```

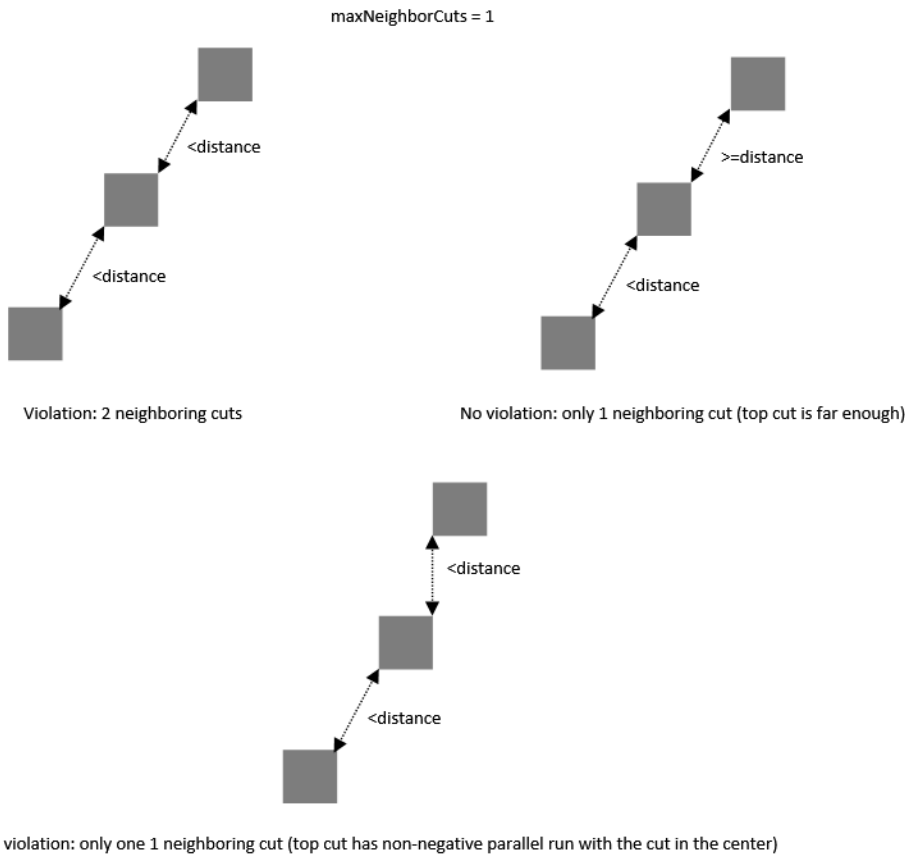
maxNeighborCuts

Advanced processes have limits on number of neighboring vias. If a via has N neighboring vias, then the N+1 via must be located further away. When the distance between vias is measured in order to determine if a via is considered a neighboring via, different cut spacing types can be used. The examples below show center-to-center spacing type with range parameter ($\geq \text{distance1} < \text{distance2}$).

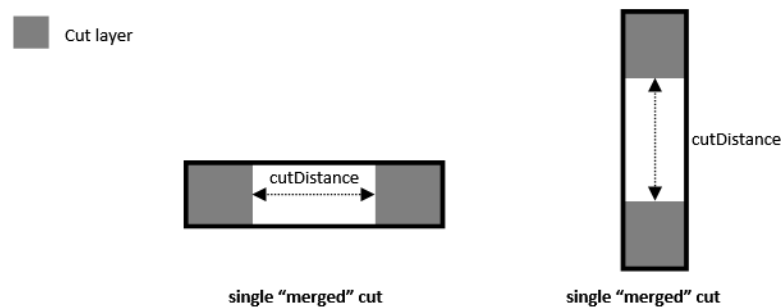




The examples below show corner-to-corner spacing type with range parameter ($< \text{distance}$). When this spacing type is used, only cuts which have negative parallel run in both horizontal and vertical directions are considered to be neighbors. Cuts with non-negative parallel run are not considered neighbors regardless of distance between them.



Optionally, for the purpose of this rule, any two fully aligned via cut shapes with a particular exact distance between them (`oaCutDistance`) can be considered a single merged cut with bounding box equal to the combined bounding boxes of the two original cuts, and all measurements are performed against the merged cut.



Constraint Category	Via Construction
Constraint Type	Complex
OA Rule	Custom Compiler Custom Constraint

Syntax:

```
maxNeighborCuts
    <string|LPP>layer
    <int>numOfCuts
    -range rangeValue
    [-color colorName|any]
    [-cutClass <cutClassArray|cutClassNameList>]
    [-cutClassList cutClassArray|cutClassNameList]
    [-exceptCutClassList <cutClassArray|cutClassNameList>]
    [-cutSpacingType corner|center|cornerAndEdge]
    [-cutDistance floatValue]
    [-bridgeViaTypes bridge|merged | none]
    [-otherBridgeViaTypes bridge|merged | none]
    [-isHard true|false]
    [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or as LPP.
numOfCuts	Maximum allowed number of neighboring cuts.
-range	Required parameter defining the distance range in database units for determining if a cut is a neighboring cut. This range must have a finite upper bound.
-color	Defines the specific color mask for all cuts this constraint applies to. A value of <i>any</i> means any color for either shape, <i>same</i> means that the shapes must be on the same color, and <i>diff</i> means the shapes must be on different colors. Default is <i>any</i>. Caution: You must specify the <i>colorName</i> in the Controls Section > colors group and the color mask value cannot be larger than the <i>numberOfColorMasks</i> value specified in the Layer Rules Section > functions group.
-cutClass	If given, the constraint only applies if the main cut is of this cut class. The neighboring cuts can be of any cut class. This parameter cannot be given if the <i>cutClassList</i> or <i>exceptCutClassList</i> parameters are given.
-cutClassList	An array of <i>oaDualInt</i> values, each defining width and height of a cut class. If given, the constraint only applies if the main cut belongs to one of this cut classes. The neighboring cuts can be of any cut class. This parameter cannot be given if the <i>cutClass</i> or <i>exceptCutClassList</i> parameters are also given.
-cutSpacingType	This is an optional parameter defining how the distance to a neighboring cut is measured. Allowed values are 2 (corner to corner) and 3 (center to center). <i>center</i> is set as default.

Argument	Description
-cutDistance	This is an optional parameter defining the exact distance between two fully aligned via cuts at which these two cuts are considered a single merged cut with bounding box equal to the combined bounding boxes of the two original cuts.
-bridgeViaTypes	This is an optional parameter defining an array of <code>oaInt</code> values. Each value sets a via type and defines valid enumeration names and values. If given, the rule applies only when the type of the main via is listed in the parameter value.
-otherBridgeViaTypes	This is an optional parameter defining an array of <code>oaInt</code> values. Each value sets a via type, and defines valid enumeration names and values. If given, a neighboring via is counted against the maximum allowed number only if its type is listed in the parameter value.
-exceptCutClassList	This is an optional parameter defining list of cut sizes (<code>oaDualIntValue</code>) which are excluded from this rule. If all cuts belong to the same cut class from the list, and there are no merged cuts among them, the rule does not apply. This parameter cannot be given if the <code>cutClass</code> parameter is given.

The following table describes enumerations for the `-cutSpacingType` argument:

Description	Name	Value
Corner to corner spacing	corner	2
Center to center spacing	corner	3
Corner to corner spacing when parallel run length is negative, or if there is spacing between edges	cornerAndEdge	5

The following table describes enumerations for the `- bridgeViaTypes` and the `otherBridgeViaTypes` argument:

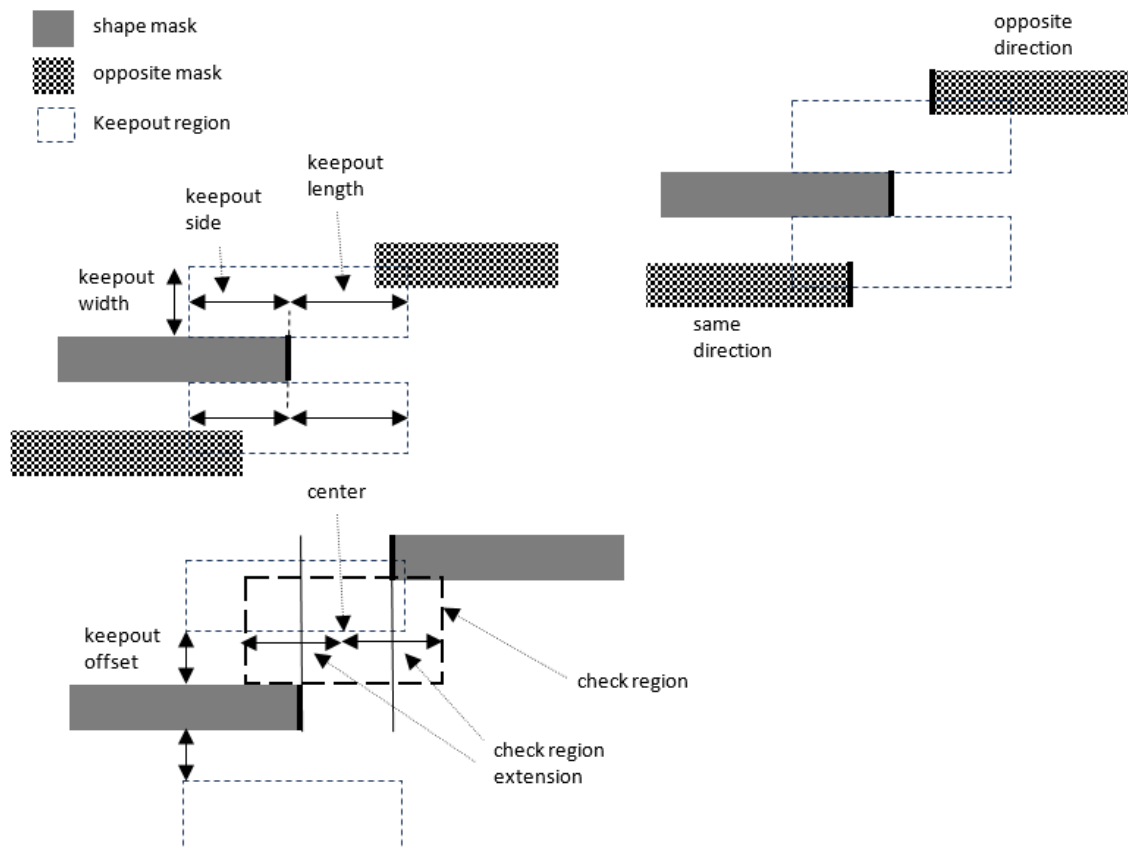
Description	Name	Value
A single cut via interacting with only one wire on top and bottom metal layers.	none	0
A bridge via consisting of a single cut connecting two parallel metal wires	bridge	1
A merged via consisting of two separate cuts located at the specified distance from each other such that the merged cut acts as a bridge via connecting the two parallel metal wires.	merged	2

For example:

- maxNeighborCuts v1 3 -range (≥ 0.09 ≤ 0.15)
-color green
-cutClass ((0.4 0.6) (0.6 0.8))
- maxNeighborCuts VIA2 2 -range (< 0.040) -cutDistance 0.013
-cutSpacingType
cornerAndEdge -bridgeViaTypes (none)
-otherBridgeViaTypes (bridge merged))

maxNumWiresEndOfLineKeepout

Advanced process nodes limit the number of wires in certain regions between two convex corners of end-of-line edges. The the two wires can be in the same or opposite direction. The center point of the two corners in the direction of the wires define where the check region will be. Wires routed through the check region will be counted and the number must not exceed the specified value.



Constraint Category	Spacing
Constraint Type	Complex
OA Rule	Custom Compiler custom layer constraint

Syntax:

```
maxNeighborCuts
    <string|LPP>layer
    <float>keepoutSide
    <float>keepoutWidth
    <float>keepoutLength
    <float>checkRegionExtension
    -width float
    -count int
    [-cornerOnly true|false]
    [-exceptDistance float]
    [-exceptAligned int]
    [-keepOutOffset float]
    [-sameDirection true|false]
    [-isHard true|false]
    [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or as LPP.
keepoutSide keepoutWidth keepoutLength checkRegionExtension	Measurements that determine the keepout regions.
-width	Maximum wire width. The constraint applies only if the wire width is less than the <code>width</code> value.
-count	Maximum number of wires permitted in the check region.
-cornerOnly	Whether corners only (true), or corners and edges (false) are not permitted in the checkout region. Default: false
-exceptDistance	Distance between two end-to-end wires at which the constraint does not apply.
-exceptAligned	Condition under which the constraint does not apply: 1: The two end-to-end wires are collinear. 2: The aligned end-to-end wires have one or more shapes between them on the same layer. Default: 1
-keepoutOffset	Distance at which the keepout region starts. Default: 0 (the keepout region abuts the side of the wire).

Argument	Description
-sameDirection	Requires that the end-to-end wires are in the same direction (true) or in opposing directions (false). Default: true

Example:

```
maxNumWiresEndOfLineKeepout m1 (0.35 0.78 0.25 0.47)
    -width 0.35 -count 3 -cornerOnly true -exceptDistance 0.8
    -exceptAligned 2 -keepoutOffset 0.3 -sameDirection false
```

maxRoutingDistance

This layer constraint specifies the maximum allowed routing distance for a layer. This constraint is usually used for poly routing and defined for each net and each layer. The value is specified in microns.



Constraint Category	Routing
Constraint Type	Simple
OA Rule	oacMaxRoutingDistance

Syntax:

```
maxRoutingDistance
    <string|LPP>layer
    <float>distanceValue
    [-isHard true|false]
    [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or as LPP.
distanceValue	Specifies the maximum allowed routing distance for a layer.

In this example, the maximum routing distance for poly is mentioned:

```
maxRoutingDistance POLY 2
```

maxTapSpacing

This layer constraint specifies the maximum center-to-center spacing between cut shapes that are used as Well ties to Nwell or Pwell within a given well.

This constraint applies both to cut and local interconnect layers.

Constraint Category	Spacing
Constraint Type	Simple
OA Rule	oacMaxTapSpacing

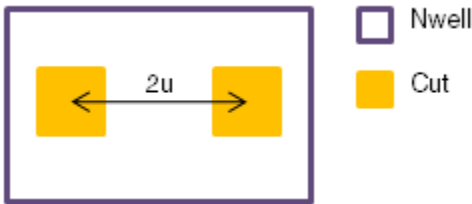
Syntax:

```
maxTapSpacing
  <string|LPP>layer
  <float>spacingValue
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or as LPP.
spacingValue	Specifies the maximum center-to-center spacing between the cut shapes.

In this example, the maximum Tap spacing is mentioned:

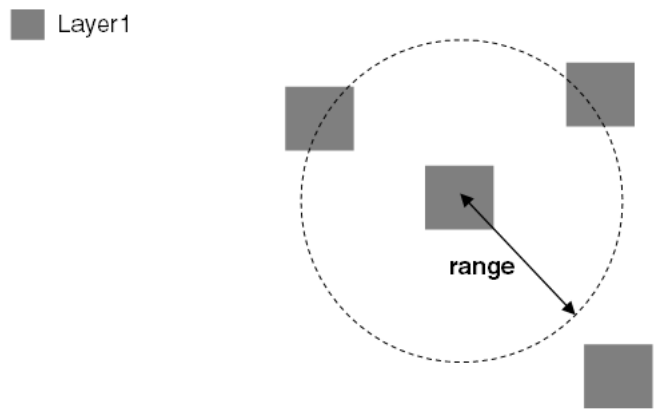
Rule	Description	Value (in microns)
wellTie.SP.1	max well Tie Spacing	2



```
maxTapSpacing wellTie 2
```

maxUnalignedViaNeighbors

The `maxUnalignedViaNeighbors` constraint limits the number of unaligned vias within a specified `range` of a self-aligned via.



Constraint Category	NumCut
Constraint Type	Complex
OA Rule	Custom Compiler Custom Layer Constraint

Syntax:

```
maxUnalignedViaNeighbors
  <string|LPP>layer
  <int>numCuts
  [-range rangeValue]
  [-cutClass1 float1 float2|<string>cutClassName]
  [-cutClass2 float1 float2|<string>cutClassName]
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or a LPP.
numCuts	Maximum number of unaligned vias permitted within <code>range</code> .
-range	Center-to-center distance between the self-aligned via and neighboring unaligned vias. You can specify a range of distances. See Range Data Syntax .
-cutClass1	cutClass name or cut dimensions (width height) of the self-aligned via. The constraint applies only to the specified cut class.

Argument	Description
-cutClass2	cutClass name or cut dimensions (width height) of the neighboring unaligned vias. The constraint applies only to the specified cut class.

For example:

```
maxUnalignedViaNeighbors v1 4
                        -range (>=0.02 <=0.05)
                        -cutClass 0.02 0.04
                        -cutClass2 0.06 0.08
```

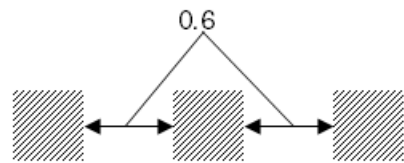
maxViaClusterNumCuts

In some advanced processes, the spacing between via cuts can be smaller if they are built in a particular configuration. They can be built in a horizontal line, exactly aligned, up to a limit of `perpendicularNumCuts`, separated by at least `minWithin`.

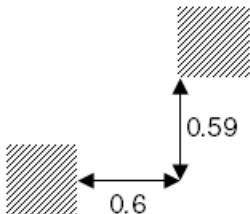
Up to `diagonalNumCuts` vias can be placed in a straight diagonal line, with `minWithin` distance between them in the X and Y direction. Vias placed this close together are defined to be in the same via cluster.

maxViaClusterNumCuts 3
cutClass Vx
diagonalNumCut 3
minDistanceWithin 0.06
maxDistanceWithin 0.7

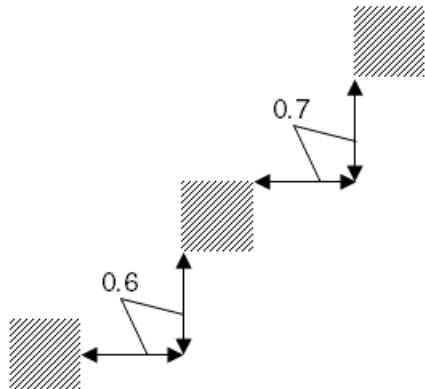
 Cut



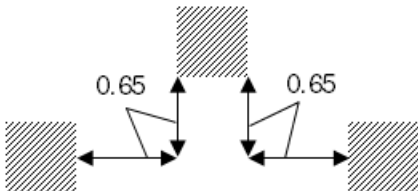
Qualified, perfectly aligned horizontal and within 0.6 apart



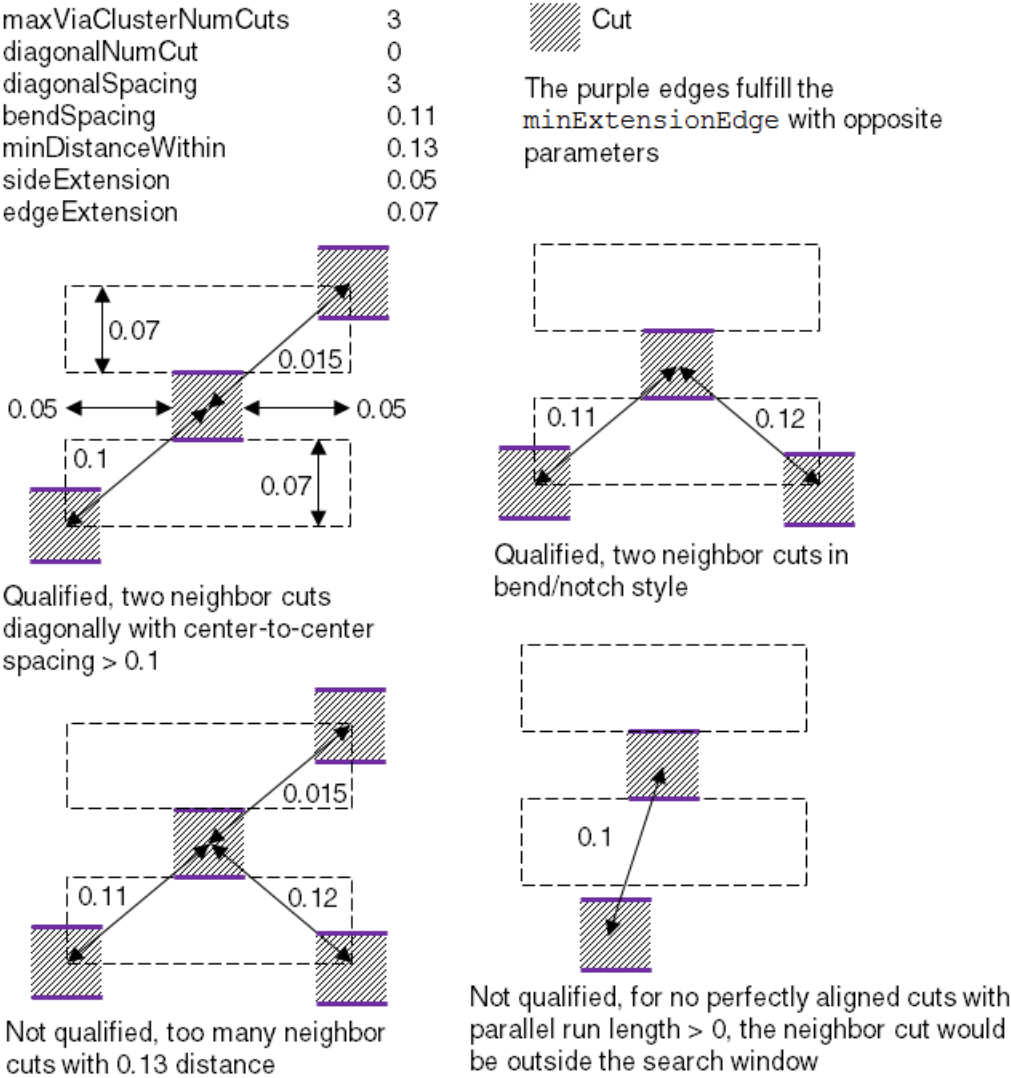
Not qualified, 0.6 distance requirement must be applied to both X and Y direction in diagonal align case



Qualified, both 0.6 and 0.7 within requirements are met



Not qualified, the cluster cannot bend to form notch



```
<int>numCuts
-minDistanceWithin float
-maxDistanceWithin float
[-cutClass <cutClassArray | cutClassNameList>]
[-diagonalNumCut int]
[-diagonalSpacing float]
[-bendSpacing float]
[-diagonalDistanceWithin float]
[-sideExtension float]
[-edgeExtension float]
[-distanceMeasureType euclidean|manhattan]
[-isHard true|false]
[-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or a LPP.
numCuts	The maximum number of via cuts of a cluster perfectly aligned in a direction perpendicular to the cut edge that fulfills the overhang defined in the minExtensionEdge constraint with the opposites parameter.
-minDistanceWithin	The minimum distance between the via cuts in the square-corner style (max X and Y) of a via in the cluster, measured in both X and Y direction. If it is part of a cluster, it is subject to the spacing specified in this constraint.
-maxDistanceWithin	The maximum distance between the via cuts in both the X and Y directions in the square-corner style. Spacing between cuts not in the same via cluster is subject to regular cut spacing rules. This is a mandatory parameter.
-cutClass	List of paired float values or a list of cutClass names. See cutClass and cutClassList . The constraint applies only in cases where the cuts comply with these specifications.
-diagonalNumCut	The maximum number of diagonal cuts that can be part of the cluster. Spacing between diagonal cuts must be measured in both the X and Y direction. Diagonal cuts in a cluster must run in straight diagonal lines.
-diagonalSpacing	The center-to-center diagonal distance between adjacent cuts. When <code>diagonalNumCut</code> is 0, there can be 2 cuts within <code>diagonalDistanceWithin</code> that are $\geq$ <code>diagonalSpacing</code> spacing. Both <code>diagonalDistanceWithin</code> and <code>diagonalSpacing</code> are measured center-to-center.
-bendSpacing	(Required only if <code>-diagonalSpacing</code> is specified) At most 2 non-perfectly aligned neighbor cuts can be found within <code>diagonalDistanceWithin</code> with spacing $\geq$ <code>bendSpacing</code> , both measurements are center to center.
-diagonalDistanceWithin	The center-to-center distance between cuts when the cuts are aligned diagonally. This argument is required only if <code>-diagonalSpacing</code> is specified.

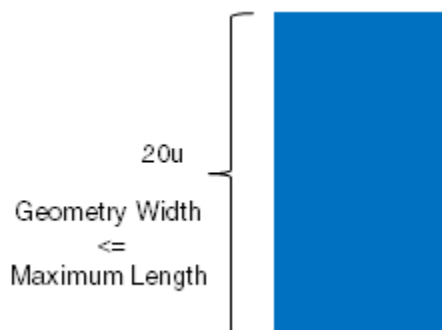
Argument	Description
-sideExtension	(Required only if <code>-diagonalSpacing</code> is specified) The horizontal distance extended from the left and right edges of the cut to form the horizontal dimension of the cut neighbor search window. One diagonal cut neighbor without bend or notch, or two cut neighbors in bend or notch positions must be found within the search window to be considered part of the via cluster. The side extensions must meet the overhang defined in the minExtensionEdge constraint, and <code>-edgeExtension</code> must be perpendicular to those edges.
-edgeExtension	(Required only if <code>diagonalSpacing</code> is specified) The vertical distance extended from the top and bottom edges of the cut to form the vertical dimension of the cut neighbor search window.
-distanceMeasureType	Type of distance measurement: Euclidean, or Manhattan. Default: Manhattan

For example:

```
maxViaClusterNumCuts M1 3 -cutClass ((0.6 0.8) (0.6 0.8))
                        -diagonalNumCut 3
                        -minDistanceWithin 0.06
                        -maxDistanceWithin 0.7
maxViaClusterNumCuts M1 0 -cutClass 0.6 0.8
                        -diagonalNumCut 3
                        -maxDistanceWithin 0.7
                        -distanceMeasureType euclidean
```

maxWidth

This layer constraint specifies the maximum width for a piece of geometry in microns.



Constraint Category	Length and Width Process Rules
Constraint Type	Simple
OA Rule	oacMaxWidth

Syntax:

```
maxWidth
  <string|LPP>layer
  <float>widthValue
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or as LPP.
widthValue	Specifies the maximum width for a piece of geometry.

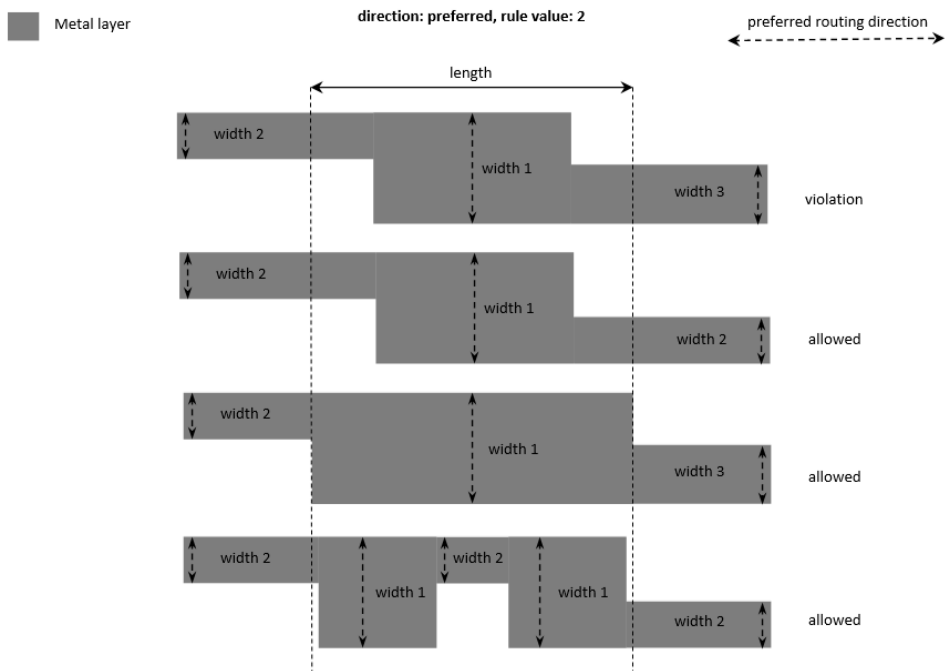
For example:

```
maxWidth m1 20
```

maxWireWidthCount

Some processes restrict the number of different widths that can be used for a wire oriented in preferred or non-preferred direction within a run of specified length.

Figure 12 maxWireWidthCount



Constraint Category	NumCut
Constraint Type	Complex

Syntax:

```
maxWireWidthCount
  <string|LPP>layer
  <int>numOfWidths
  -length float
  [-direction any|preferred|nonpreferred]
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or as LPP.
numOfWidths	The maximum allowed number of different widths.
-length	Specifies the length along the wire within which the number of different widths needs to be checked. (see Figure 12.)
-direction	Specifies the direction of a wire to apply the rule: • Only preferred (1) • Only non-preferred (2) • Both preferred and non-preferred directions (0) Default: any Length is measured in this direction. Width is measured in the orthogonal direction.

For example:

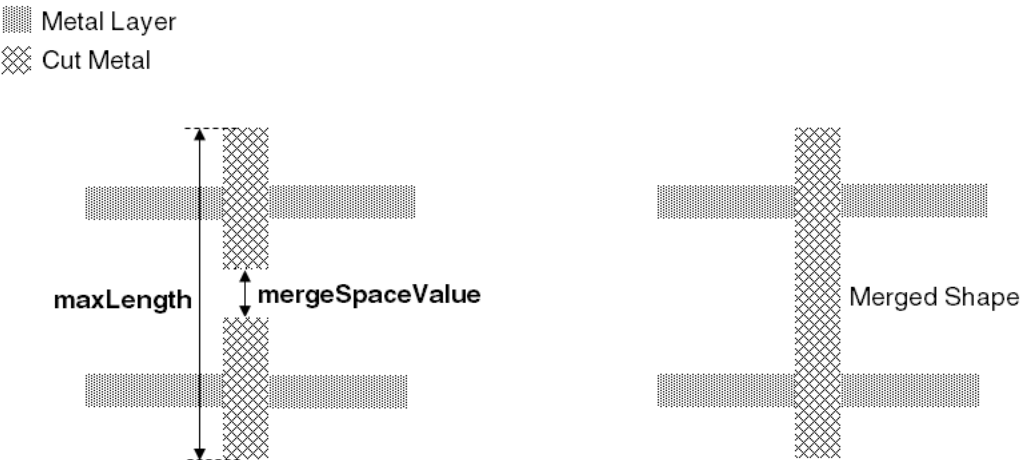
```
maxWireWidthCount M1 2 -length 0.3 -direction preferred
```

mergeSpaceAllowed

The `mergeSpaceAllowed` layer constraint specifies the distance below which shapes on the same layer (that do not meet the required minimum spacing) can be merged to form a single shape.

Note:

The threshold is usually the minimum spacing between the layers.



Constraint Category	Spacing
Constraint Type	Complex
OA Rule	oacMergeSpaceAllowed

Syntax:

```
mergeSpaceAllowed
  <string|LPP>layer
  <float>mergeSpacingValue
  [-maxLength <float>]
  [-color <int>]
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

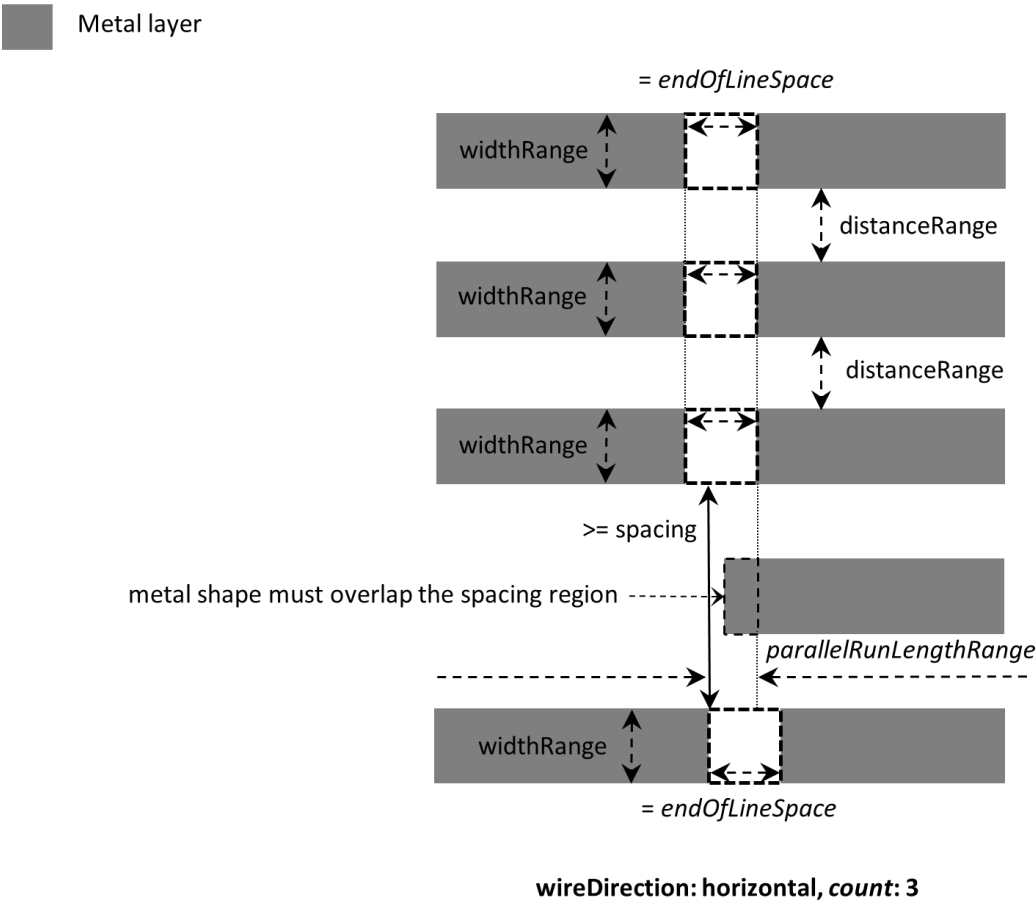
Argument	Description
layer	Layer can be defined as a string or as LPP.
mergeSpacingValue	The threshold below which geometries on the same layer can be merged forming one shape.
-maxLength	The maximum length of the merged shape.
-color	The color of the merged shape. To specify Any color, set -color to 0.

Example:

```
mergeSpaceAllowed M1 0.05 -maxLength 0.15 -color red
```

minAdjacentEndOfLineProjectionSpacing

This constraint models spacing of adjacent end of line projections when there are several of them aligned and located on the adjacent tracks and there is a metal shape between them and an another end-of-line projection.



Constraint Category	Spacing
Constraint Type	Complex
OA Rule	Custom Layer Constraint

Syntax:

```
minAdjacentEndOfLineProjectionSpacing layerName |
LPP value [-isHard true|false] [-comment ["rule_ref_no"] "string"]

[-endOfLineSpace floatValue]
```

```
[-wireDirection horizontal | vertical]  
[-count intValue]  
[-distanceRange rangeData]  
[-widthRange rangeData]  
[-parallelRunLengthRange rangeData]
```

Argument	Description
endOfLineSpace	Exact distance in database units between the aligned metal line ends required for the constraint to apply.
wireDirection	Optional parameter specifying the orientation of metal lines to which the rule applies. Valid values are 1 (horizontal) and 2 (vertical).
count	Minimum number of adjacent aligned end-of-line projections needed for the rule to apply. Must be positive number.
distanceRange	Aligned end-of-line projections are counter against the count parameter only if the distance between them (i.e., the distance between the corresponding metal lines) lies in this range in database units.
widthRange	If given, the constraint only applies when the width of metal lines lies in this range in database units.
parallelRunLengthRange	If given, the constraint only applies when parallel run length between the end-of-line projections lies in this range in database units. Must have a finite lower bound.

For example:

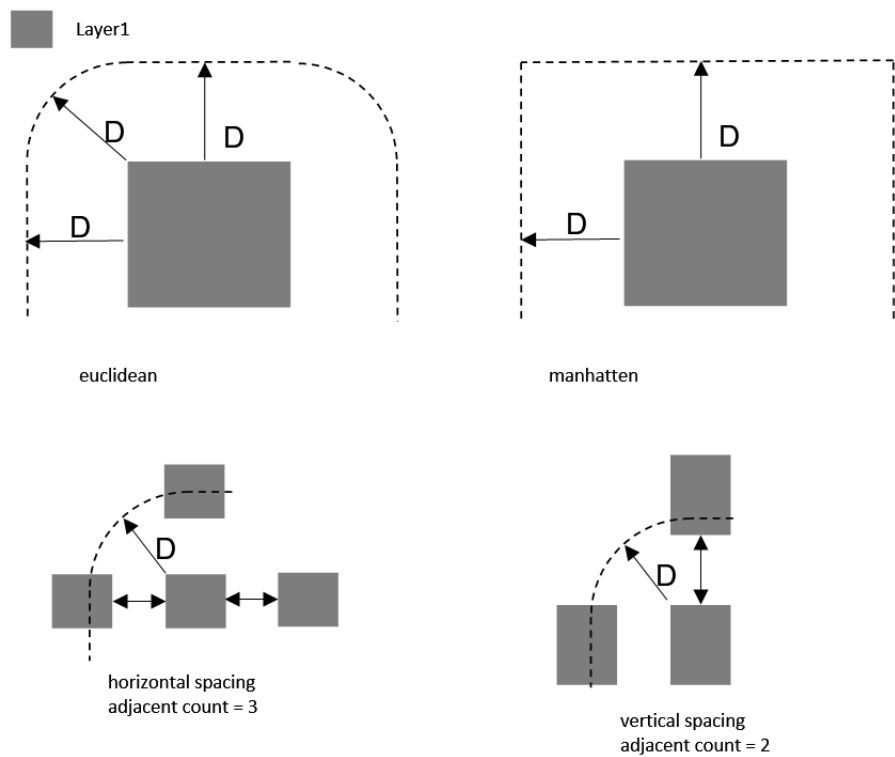
```
minAdjacentEndOfLineProjectionSpacing M1 2  
    -endOfLineSpace 0.3  
    -wireDirection horizontal  
    -count 3  
    -distanceRange (0.35 0.45)  
    -widthRange (0.25 0.3)  
    -parallelRunLengthRange (0.5 0.6)
```

minAdjacentViaSpacing

This layer constraint specifies the minimum required distance between via cuts that are adjacent. Two via cuts are considered adjacent if they lie within the specified distance from one another.

Three optional parameters determine how via spacing is measured:

- center-to-center or edge-to-edge
- whether the constraint applies to only certain connectivity types
- whether the constraint applies to vias on power and ground nets



Constraint Category	Via Construction
Constraint Type	Complex
OA Rule	oacMinAdjacentViaSpacing

Syntax:

```
minAdjacentViaSpacing
  <string|LPP>layer
  <float>spacingValue
  -numCuts int
  [-exactAligned <int>]
  -distance <float>
  [-centerToCenter true|false]
  [-connectivityType
    anyConnectivityType|
    sameNetConnectivityType|
    contiguousShapesConnectivityType|
    directShapesConnectivityType|
    sameViaConnectivityType]
  [-exceptSamePGNet true|false]
    [-exactAligned intValue]
  [-sideParallelOverlap true|false]
  [-distanceMeasureType euclidian|manhattan]
```

```
[ -cutClass <cutClassArray | cutClassNameList>]
[ -noParallelRunLength true|false]
[ -twoCuts Int]
[ -includeSecondary true|false]
[ -isHard true|false]
[ -comment [<string rule_id>] <string rule_desc>]
[ -spacingDirection any | horizontal | vertical]
[ -colorType any | same | diff]
```

Argument	Description
layer	Layer can be defined as a string or a LPP.
spacingValue	Specifies the minimum distance required between adjacent vias.
-numCuts	Specifies the minimum number of cuts required before the constraint is applied.
-exactAligned	Stores the number of cuts that must be perfectly aligned. Default is 0.
-distance	Specifies the distance of vias before the constraint is applied.
-centerToCenter	Specifies if cut spacing should be measured from the center of one cut to the center of the other cut. If not specified, the cut spacing metric is from the edge of one cut to the edge of the other cut.
-connectivityType	• anyConnectivityType: Use this option to designate that the constraint applies to any type of connectivity (default). • sameNetConnectivityType: Use this option to designate that the constraint only applies to cuts on the same net. • contiguousShapesConnectivityType: Use this option to designate that the constraint only applies to cuts on the same metal. • directShapesConnectivityType: Use this option to designate that the constraint only applies when the two cuts are part of the same metal shape on the upper or lower metal and the metal shape covers the region between the two cuts. • sameViaConnectivityType: This is similar to directShapesConnectivityType except that both lower and upper metal shapes must cover the cuts and the region between the cuts.
-exceptSamePGNet	If set to "true", the constraint does not apply between cuts that are on the same power or ground net.
-exactAligned	If given, specifies a separate minimum number of neighboring cuts needed for the rule to apply when all neighboring cuts are fully aligned with the main cut (horizontally or vertically). This value must be greater than the value of numCuts parameter. If not all of the neighboring cuts are fully aligned, the value of numCuts parameter is used to determine whether the constraint applies.
-sideParallelOverlap	Is set to "true", if the adjacent via spacing applies and there is non-zero parallel edge overlap between two rectangular vias.
-distanceMeasureType	Optional parameter defining how the distance is to be measured. The value represents an oaDistanceMeasureEnum value or either euclidean or manhattan value.

Argument	Description
-cutClass	List of paired float values or a list of cutClass names. See cutClass and cutClassList . The constraint applies only in cases where the cuts comply with these specifications.
-noParallelRunLength	If present, the rule only applies among cuts with no common parallel run length. If not present, it should be considered false. The default value is <i>false</i> .
-twoCuts	The via spacing value applies between two cuts if each of them has at least <code>twoCuts</code> neighbor cuts at less than distance.
-includeSecondary	Includes secondary cuts within the distance of primary cuts when determining the total number of adjacent cuts. Default: false
-spacingDirection	Defines how to apply the constraint value spacing. By default it applies to both direction but it can be limited to either horizontal or vertical direction. If horizontal or vertical spacing direction is given, then spacing applies only to edges with parallel overlap greater than or equal to zero. Default: any
-colorType	This is an optional parameter that dictates when the constraint applies depending on the color of the two cut shapes. A value of zero means any color for either cut, 1 means that the cuts must be on the same color, and 2 means the cuts must be on different colors. Default: any

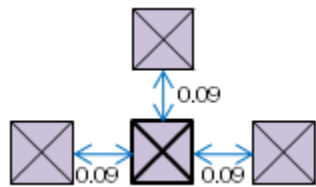
Note:

If `cutClass` parameter is specified with `cutClassName` in ASCII file, technology importer looks-up the first `cutClass` constraint specified in the same hierarchical constraint group to convert to cut class size. If a `cutClass` constraint does not exist then an error message is displayed.

The warning message is displayed when `-noParallelRunLength` and `-sideParallelOverlap` are set to true at the same time.

In this example, the via spacing depends on the number of neighboring vias and their distance:

Rule	Description	Value (in microns)
VIA1.S.4	VIA1 - VIA1 spacing when there are 3 adjacent Vias, and the distance between them is less than 0.085	≥ 0.09



```
minAdjacentViaSpacing VIA1 0.09 -distance 0.085 -numCuts 3
-centerToCenter true -exceptSamePGNet true -exactAligned 2
```

minArea

This layer constraint specifies the minimum metal area required for non-rectangular polygon shapes in microns squared. Minimum area rules most often apply to via contacts, especially for stacked vias.

Note:

To specify the minimum area allowed for rectangular shapes on a layer, use the [minRectArea](#) constraint.



Non-rectangular area must be >= minArea (0.05 um^2)

Constraint Category	Area
Constraint Type	Simple
OA Rule	oacMinArea

Syntax:

```
minArea layerName| LPP value params [-isHard true|false]
      [-comment ["rule id"] "string"]
      value::=floatValue
      params::= [-viaLayers (layer*)]
      layer::=layerName
```

Argument	Description
layer	Layer can be defined as a string or as LPP.

Argument	Description
areaValue	Specifies the minimum metal area required for non-rectangular polygon shapes.
viaLayers	Array of <code>oaLayerNum</code> values, each defining a cut layer immediately above or below the metal layer. If given, the rule only applies to the metal layer shape if there is a cut shape on any of these cut layers connecting the metal layer shape to the metal layer above or below.

For example:

Rule	Description	Value (in μm^2)
DIFF.A.1	Minimum Area of DIFF	0.05

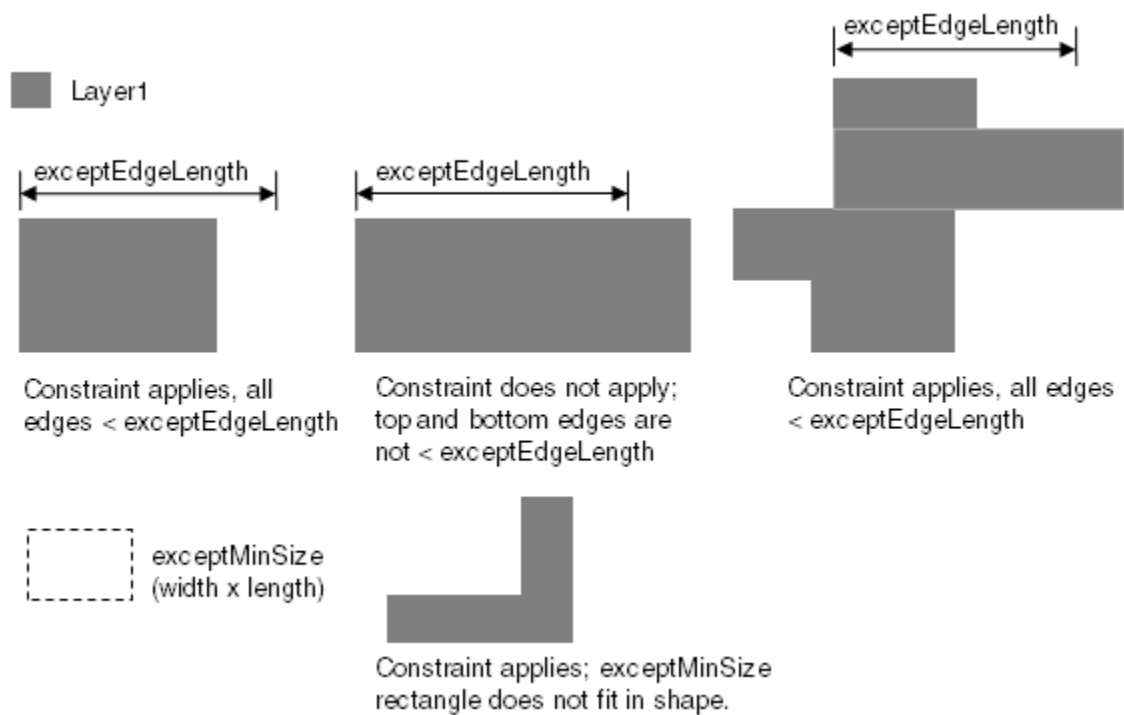
For example:

```
minArea DIFF 0.05
```

```
minArea m2 0.10 -viaLayers (v1 v2)
```

minAreaEdgeLength

The `minAreaEdgeLength` constraint specifies the minimum area required for non-rectangular polygon shapes in microns squared. This minimum area is dependent of all the edge length of the polygon. It applies only when the lengths of all vertices are smaller than the specified threshold value.



Constraint Category	Length and Width Process Rules
Constraint Type	Complex
OA Rule	Not Available in OA

Syntax:

```
minAreaEdgeLength
<string|LPP>layer
<float>areaValue
[-exceptEdgeLength float]
[-exceptMinEdgeLength float]
[-exceptMinSize sizeValue|sizeValeList]
[-exceptMinSizeArray arrayValue]
[-exceptAdjacentEdge float]
[-width float]
[-isHard true|false]
[-comment [<string rule_id>] <string rule_desc>]
[-color colorName]
```

Argument	Description
layer	Layer can be defined as a string or a LPP.

Argument	Description
-exceptEdgeLength	Length threshold in microns that all edges must be less than in order for the constraint to apply. Default is infinity.
-exceptMinEdgeLength	Optional parameter that holds the minimum edge length threshold in microns. If given, the constraint only applies if at least one edge has a length greater than or equal to this value.
-exceptMinSize	Width and length dimensions, in microns, of a rectangle used to determine whether the constraint applies. If the area of the shape is smaller than the area of this <code>MinSize</code> rectangle, the constraint applies. If the area of the shape is larger than the <code>MinSize</code> rectangle, the constraint does not apply.
-exceptMinSizeArray	This optional parameter holds an array of <code>dualFloat</code> objects with width and length values. The area constraint applies if a rectangle with any of these width/length pairs cannot fit inside the shape. This parameter is optional and cannot be used together with <code>exceptMinSize</code> .
-exceptAdjacentEdge	Optional parameter that holds the minimum edge length for an edge adjacent to an edge that is greater than <code>exceptMinEdgeLength</code> . If the adjacent edge length is greater than or equal to this value, then the constraint does not apply.
-width	Euclidean width that shape must be less than for the constraint to apply.
-color	Optional parameter that represents the mask or color. Caution: You must specify the <code>colorName</code> in the Controls Section > colors group and the color mask value cannot be greater than the <code>numberOfColorMasks</code> value specified in the Layer Rules Section > functions group.

For example,

```
minAreaEdgeLength m1 0.06
                  -exceptEdgeLength 0.40
                  -exceptMinSize(0.06 0.08)

minAreaEdgeLength m1 0.06
                  -exceptEdgeLength 0.40
                  -exceptMinSize((0.06 0.08) (0.06 0.06 ))
                  -color red
```

minBoundaryExtension

This layer constraint specifies the minimum distance a shape must extend past a `PRBoundary` in microns.

This constraint is ideal to specify the necessary extension of a well past a boundary.

Constraint Category	Extension
---------------------	-----------

Constraint Type	Complex
OA Rule	oacMinBoundaryExtension

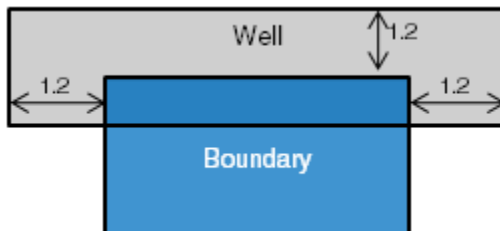
Syntax:

```
minBoundaryExtension
  <string|LPP>layer
  <float>extensionValue
  [-coincidentAllowed true|false]
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or a LPP.
extensionValue	Specifies the minimum that the shape must extend past the boundary.
-coincidentAllowed	If specified as “true”, the shapes can either meet the minimum extension or their edges can be coincident else the edges must meet the minimum extension value specified and cannot be coincident.

In this example, the NWELL must extend the boundary by 1.2:

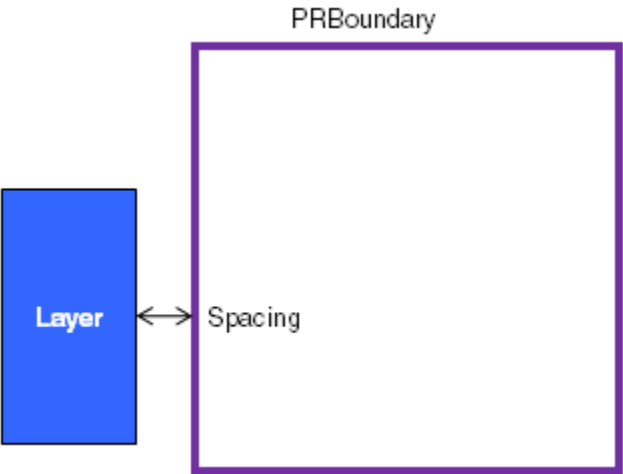
Rule	Description	Value (in microns)
NWELL.S.4	Boundary extension	>= 1.2



```
minBoundaryExtension NWELL 1.2 -coincidentAllowed false
```

minBoundaryExteriorHalo

The minBoundaryExteriorHalo defines the minimum spacing between shapes on the specified layer in the current level of hierarchy and the PRBoundaries of all instances at the current level of hierarchy.



Constraint Category	Spacing
Constraint Type	Complex
OA Rule	oacMinBoundaryExteriorHalo

Syntax:

```
minBoundaryExteriorHalo
<string|LPP>layer
<float>haloValue
[-coincidentAllowed true|false]
[-isHard true|false]
[-comment [<string rule_id>] <string rule_desc>]
```

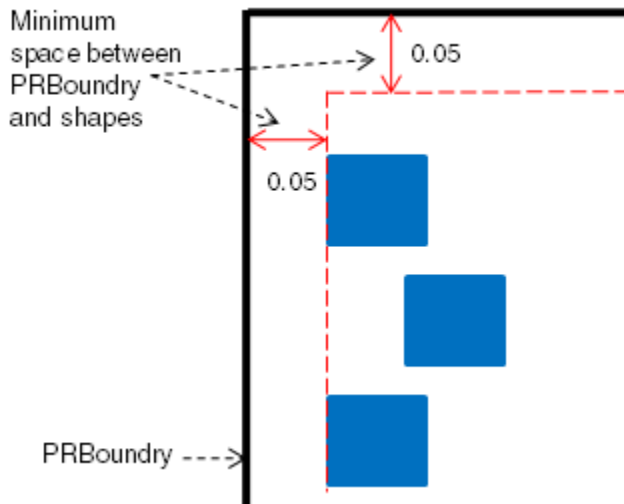
Argument	Description
layer	Layer can be defined as a string or as LPP.
haloValue	Specifies the minimum spacing required between the shape and the boundary in microns.
-coincidentAllowed	When the coincidentAllowed parameter is specified and is true, then shapes can either meet the minimum distance specified or their edges can be coincident with the boundary. If the parameter is false or is not specified, then the edges must meet the minimum distance specified and cannot be coincident.

For example:

```
minBoundaryExteriorHalo (m1 drawing) 0.10
```

minBoundaryInteriorHalo

This layer constraint specifies the minimum distance required between a shape on a specified layer and the enclosing PRBoundary.



Note:

The `minBoundaryInteriorHalo` value is usually half of the minimum spacing, so that, when you abut two cells, you need to have `minSpacing` between the shapes.

Constraint Category	Spacing
Constraint Type	Simple
OA Rule	<code>oacMinBoundaryInteriorHalo</code>

Syntax:

```
minBoundaryInteriorHalo
  <string|LPP>layer
  <float>haloValue
  [-coincidentAllowed true|false]
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or as LPP.
haloValue	Specifies the minimum distance required between a shape.

Argument	Description
-coincidentAllowed	When this parameter is specified and is set to true, the shapes can either meet the minimum distance specified or their edges can be coincident with the boundary else the edges meet the minimum distance specified and cannot be coincident.

In this example, the minimum distance that the M1 shapes need to be placed at from the PR Boundary is mentioned:

```
minBoundaryInteriorHalo M1 0.05
```

minCenterToCenterSpacing

This layer constraint specifies the minimum distance required between the centers of two geometries on a specified cut layer.

In particular, this constraint is used to specify the spacing between adjacent vias.

Constraint Category	Spacing
Constraint Type	Simple
OA Rule	oacMinCenterToCenterSpacing

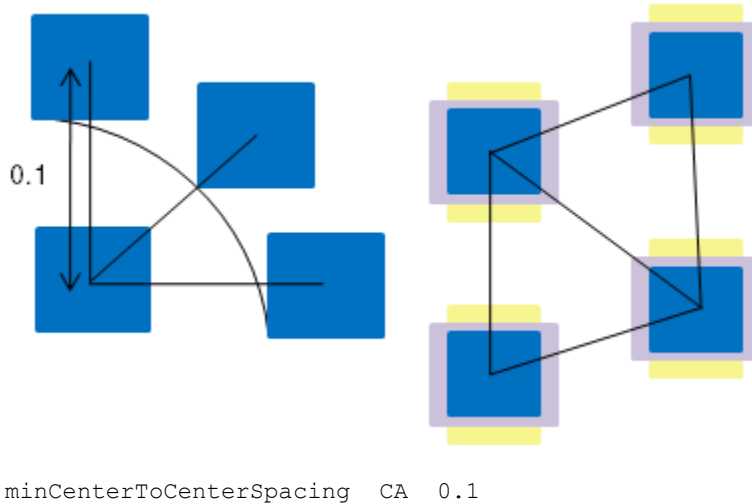
Syntax:

```
minCenterToCenterSpacing
  <string|LPP>layer
  <float>spacingValue
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or as LPP.
spacingValue	Specifies the minimum distance required between the centers of two geometries on a specified cut layer.

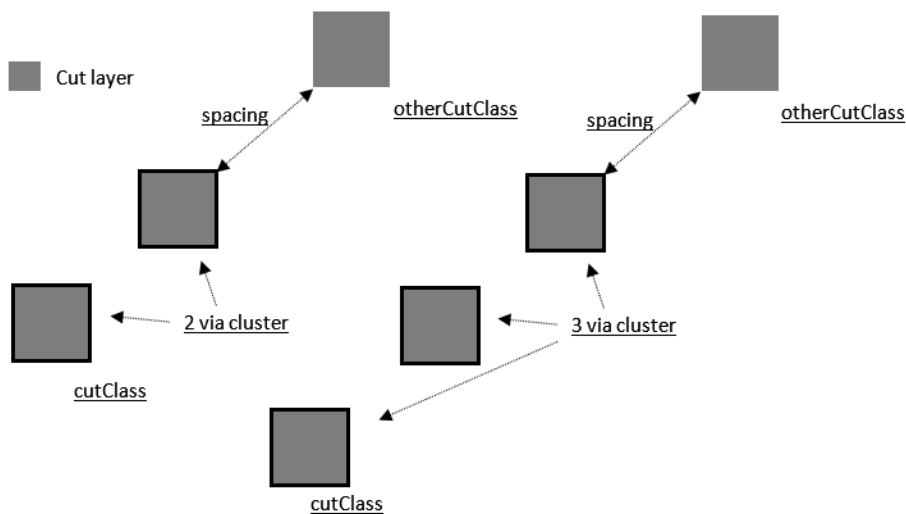
In this example, the minimum center to center spacing for contact is specified:

Rule	Description	Value (in microns)
CA.SP.1	minimum spacing between Contacts (center-center)	0.1



minClusterDiagonalSpacing

This layer constraint specifies the minimum spacing between neighboring vias in a via cluster. Some advanced processes require a minimum spacing between a two- or three-via cluster and a cut shape of a specific size.



Syntax:

```
minClusterDiagonalSpacing
    <string|LPP>layer
    <floatArray>spacingValues
    -cutClass <<float> <float> | cutClassName>
    [-otherCutClass <<float> <float> | cutClassName>]
```

```
[ -isHard true|false ]  
[ -comment [<string rule_id>] <string rule_desc> ]
```

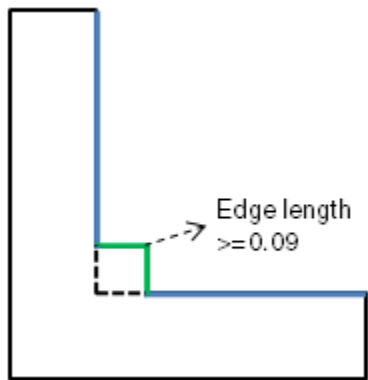
Argument	Description
layer	Name of the layer or LPP.
spacingValues	Array of floating values representing the required minimum spacing. The elements of the array apply to the K'th+2 number of vias in the via cluster. For example, the zero'th element in the array represents the spacing between a two-cut via cluster and another cut shape on the same layer. The first element is the array is for a three-cut via cluster, and so on. The via clusters are formed using <code>maxClusterDiagonalNumCuts</code> constraint. Syntax: (<float1> <float2> ... <floatN>)
-cutClass	Specifies the cut class used to form the via cluster. This is required.
-otherCutClass	If specified, the specific cut class is required to be spaced away from the via cluster.

```
minClusterDiagonalSpacing v1 (0.02 0.04) -cutClass 0.2 0.6  
-otherCutClass 0.04 0.06
```

minConcaveEdgeLength

This layer constraint specifies the minimum edge length required for an edge in a concave corner of a shape.

If both edges of the step are less than the minimum edge length, the step pattern should be filled in.



Note:

To find out whether a shape with a step pattern is suitable for Optical Pattern Correction (OPC), you can use the [minConcaveEdgeLength](#) constraint in conjunction with the [minConvexEdgeLength](#) and [minEdgeLength](#) constraints.

Constraint Category	Length and Width Process Rules
Constraint Type	Complex
OA Rule	oacMinConcaveEdgeLength

Syntax:

```
minConcaveEdgeLength
  <string|LPP>
  <float>lengthValue
  [-lengthSum float>
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or a LPP.
extensionValue	Specifies the minimum concave edge length.
-lengthSum	When the lengthSum parameter is specified, the constraint does not apply if the sum of the consecutive edges shorter than the minimum edge length value is less than value of the lengthSum parameter.

In this example, the minimum concave corner edge length of M1 is mentioned:

Rule	Description	Value (in microns)
M1.EL.1	Minimum edge length	0.09

```
minConcaveEdgeLength M1 0.09
```

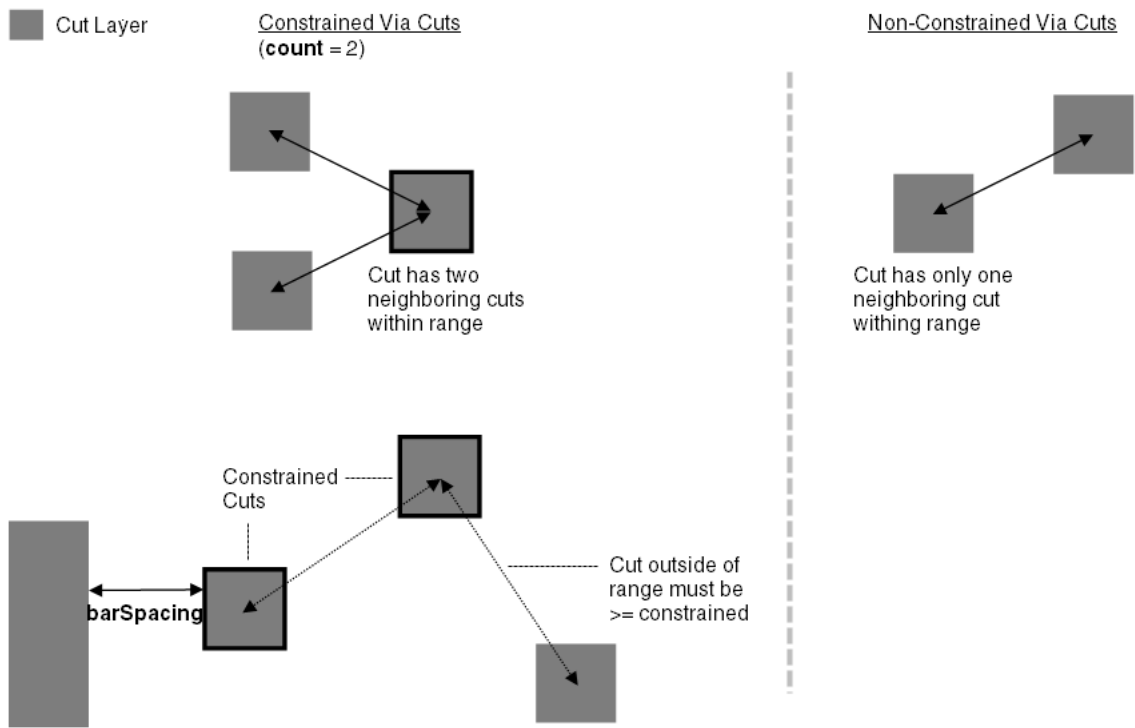
In this example, the minimum concave corner edge length is checked only when the sum of the short edges is $\geq 0.1\mu\text{m}$:

Rule	Description	Value (in microns)
M1.EL.1.1	Minimum edge length rule needs to be 0.09 when the sum of consecutive short edges is ≥ 0.1	N/A

```
minConcaveEdgeLength M1 0.09 -lengthSum 0.1
```

minConstrainedViaSpacing

This constraint specifies the minimum spacing required between a via cut and a constrained via cut. Advanced processes having more specialized enclosed via rules in that a via cut must be spaced a certain distance from a constrained via cut. A constrained via cut is one which is a certain range away from another via cut on the same layer and mask.



Constraint Category	Spacing
Constraint Type	Complex
OA Rule	Custom Compiler Custom Constraint

Syntax:

```
minConstrainedViaSpacing
  <string|LPP>layer
  <float>spacingValue
  -range rangeValue
  -count int
  [-cutClass <<float> <float> | cutClassNameList>]
  [-barSpacing float]
```

```
[-isHard true|false]
[-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or as LPP.
spacingValue	Specify the minimum spacing required between a via cut and a constrained via cut.
-range	Defines the distance range in which constrained via cuts are identified. To specify the ranges, see Range Data Syntax .
-cutClass	List of paired float values or a list of cutClass names. See cutClass and cutClassList . The constraint applies only in cases where the cuts comply with these specifications.
-count	Number of neighboring cuts within the specified <code>range</code> .
-barSpacing	(Optional) Edge-to-edge spacing between the constrained cut and nearby via bar.

Example:

```
minConstrainedViaSpacing v1 0.55 -range (>=0.09 <=0.15)
                                -cutClass 0.06 0.08
                                -count 3
                                -barSpacing 0.75
```

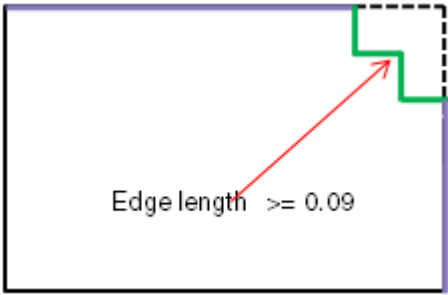
minConvexEdgeLength

This layer constraint specifies the minimum edge length for an edge in a convex corner of a shape.

If both edges of the step are less than the minimum edge length, the step pattern should be filled in.

Note:

To find out whether a shape with a step pattern is suitable for Optical Pattern Correction (OPC) you can use the [minConcaveEdgeLength](#) constraints in conjunction with the [minConcaveEdgeLength](#) and [minEdgeLength](#) constraints.



Constraint Category	Length and Width Process Rules
Constraint Type	Complex
OA Rule	oacMinConvexEdgeLength

Syntax:

```
minConvexEdgeLength
  <string|LPP>layer
  <float>lengthValue
  [-lengthSum float>
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or a LPP.
lengthValue	Specifies the minimum convex edge length.
-lengthSum	When the lengthSum parameter is specified, the constraint does not apply if the sum of the consecutive edges shorter than the minimum edge length value is less than value of the lengthSum parameter.

In this example, the minimum convex corner edge length of M1 is mentioned:

Rule	Description	Value (in microns)
M1.EL.1	Minimum edge length	0.09

```
minConvexEdgeLength M1 0.09
```

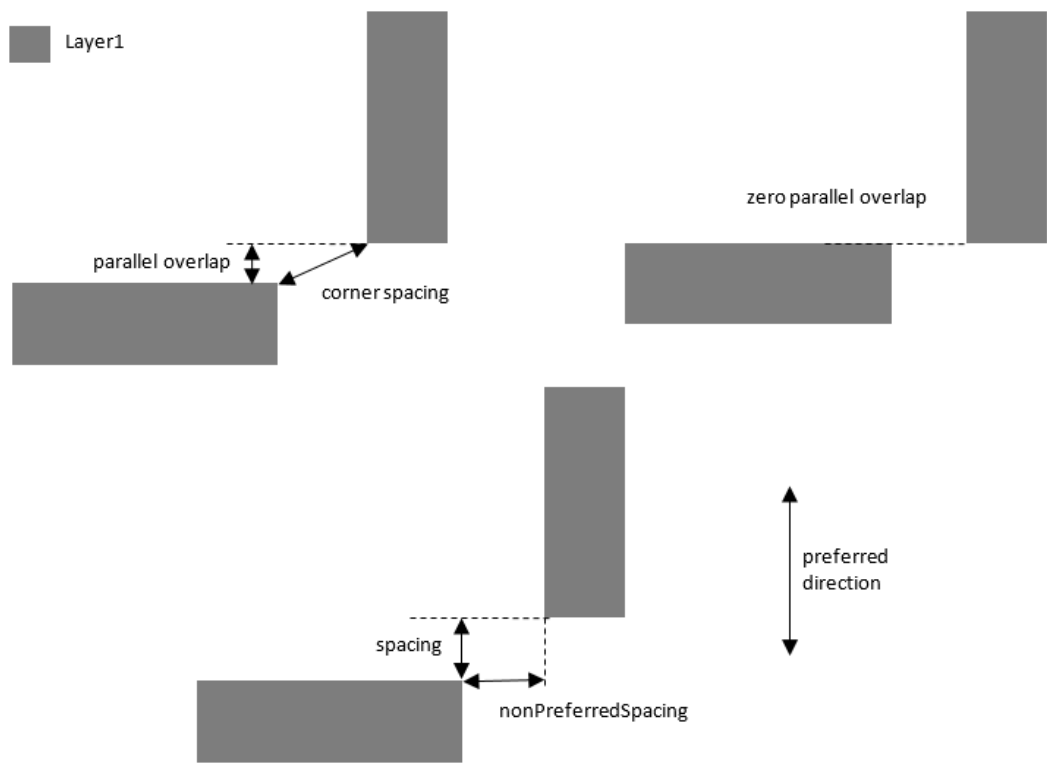
In this example, the minimum convex corner edge length is checked only when the sum of the short edges is $\geq 0.1\mu\text{m}$:

Rule	Description	Value (in microns)
M1.EL.1.1	Minimum edge length rule needs to be 0.09 when the sum of consecutive short edges is ≥ 0.1	N/A

```
minConvexEdgeLength M1 0.09 -lengthSum 0.1
```

minCornerToCornerDistance

This layer constraint specifies the minimum spacing between shape corners.



Constraint Category	Spacing
Constraint Type	Simple
OA Rule	Custom Compiler custom layer constraint

Syntax:

```
minCornerToCornerDistance
  <string|LPP>layer
  <float>minSpacing
  [-nonPreferredSpacing float]
  [-spacingType any|preferred|nonpreferred|
    preferredToNonPreferred|
    nonpreferredToPreferred]
  [-voltageRange frangeValue]
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or as LPP.
minSpacing	Specifies the minimum spacing between corners.
-nonPreferredSpacing	Manhattan distance between the adjacent corners of two shapes, one in the preferred direction and another in the non-preferred direction.
-spacingType	This parameter defines how the rule applies to metal shapes depending on their orientation (preferred or non-preferred direction). For example, if <code>spacingType</code> is 1 (preferred), then the rule only applies if both wires are oriented in preferred direction. The default value is any 0.
-voltageRange	If given, the rule only applies if the difference between voltages of the two shapes under consideration lies in this range in volts. To specify the ranges, see Range Data Syntax . See Voltage Dependent Design Rules .

For example:

```
minCornerToCornerDistance m1 0.05 -nonPreferredSpacing 0.07
                             -voltageRange (1.5 2)
```

minCutClassSpacing

The `minCutClassSpacing` constraint specifies the spacing between cuts of different classes on the same layer. Each cut class combination is duplicated for both rows and columns to allow for short edge and long edge spacing values to be held in the same table. The short edge spacing value is listed first and the long edge spacing value is listed second in each pair.

Constraint Category	Via Construction
Constraint Type	Complex
OA Rule	oacMinCutClassSpacing

Syntax:

```
minCutClassSpacing
  <string|LPP>layer
  <cutClass2DTableSpacing>spacingValue
  [-connectivityType anyConnectivityType|
    sameNetConnectivityType|
    contiguousShapesConnectivityType|
    directShapesConnectivityType|
    sameViaConnectivityType]
  [-parallelRunLength float]
  [-exactAlignedSpacing <float>spacingValue (
    <float>widthSpacing1
    <float>widthSpacing2) ]
  [-color colorName|any|same|diff]
  [-otherLayer layerName]
  [-otherLayerWidth rangeData]
  [-exceptSameNetConnectivity true|false]
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or as LPP.
spacingValue	A specialized two dimensional table that is 2Nx2N in size where N is the number of unique cut classes. The table holds both short and long edge spacing values. To know more about how spacing value is represented, see minCutClassSpacing spacingValue Representation.
-connectivityType	• anyConnectivityType: Use this option to designate that the constraint applies to any type of connectivity (default). • sameNetConnectivityType: Use this option to designate that the constraint only applies to cuts on the same net. • contiguousShapesConnectivityType: Use this option to designate that the constraint only applies to cuts on the same metal. • directShapesConnectivityType: Use this option to designate that the constraint only applies when the two cuts are part of the same metal shape on the upper or lower metal and the metal shape covers the region between the two cuts. • sameViaConnectivityType: This is similar to directShapesConnectivityType except that both lower and upper metal shapes must cover the cuts and the region between the cuts.

Argument	Description
-parallelRunLength	Specifies whether the cut edges in database units (dbu) must be parallel: -1: Cut edges must not be parallel. 0: Cuts must have ≥ 0 parallel run length. 1: Cuts must have > 0 parallel run length. - All other values: Minimum parallel run length between two end-of-line edges must be greater than or equal to the value for the constraint to apply. If the value is $> +1$ dbu, then the value is the threshold ($>$) parallel run required for the constraint to apply. The default is 0. 1 dbu = 1 user unit/DBUperUU (Database Units per User Unit). A user unit is usually 1um. For example, if dbuPerUU is 2000, then 1 dbu is calculated as $1\text{um}/2000 = 0.0005$.
-exactAlignedSpacing	If this parameter is present, it specifies an oaValueArray value of oaDualInt values. The first value in each oaDualIntValue is the width of a square cut class. The second value is the spacing (or clearance) in DBU required between exactly aligned horizontal or vertical cuts of the cut class.
-color	Color of the cut shape for which the constraint applies. Valid values are 0 (any), 1 (same), and 2 (different). Default: 0
-otherLayer	The cut shapes must interact with material on the given layer, otherwise the constraint doesn't apply. The two cut shapes between which spacing is measured should interact with the same metal shape, so connectivityType should be sameVia or directShape.
-otherLayerWidth	If given along with otherLayer, the interacting material under the cut shapes must have a width in this range, otherwise the constraint doesn't apply. To specify the ranges, see Range Data Syntax.
-exceptSameNetConnectivity	If the value is true, and the connectivityType is any, then the constraint applies only when the two cut shapes are on different nets. Default: false.

minCutClassSpacing spacingValue Representation

The spacingValue is represented as:

```
(<float>cutClassKey <float>cutClassData)
```

Where cutClassKey equals:

```
(shortEdge
  (cutEdge
    (squareCutEdge
      (<float>cutWidth <string>edgeFlag(-shortEdge|-longEdge))) |
    rectangleCutEdge
      ((<float>cutWidth <float>cutLength)
        <string>edgeFlag(-shortEdge|-longEdge)))
  longEdge
    (cutEdge
      (squareCutEdge
        (<float>cutWidth <string>edgeFlag(-shortEdge|-longEdge))) |
```

```
rectangleCutEdge
((<float>cutWidth <float>cutLength)
 <string>edgeFlag(-shortEdge|-longEdge))
)
```

And cutClassData equals:

```
<float>spacing|
spacingWithExtension(<float>spacing <float>extension)|
spacingWithMeasureType(<float>spacing <string>measureType(
    -edgeToEdge|
    -centerToCenter|
    -centerAndEdge|
    -centerAndEdgeNoPRL))|
spacingWithMeasureTypeAndExtension(<float>spacing <float>measureTypeAndExtension)
```

The `measureType` parameter should be used as specified by the enum entry for each cutclass combination:

- `centerAndEdge`: The maximum of all applicable values should be measured `center-to-center`. The minimum of all applicable values should be measured `edge-to-edge`.
- `centerAndEdgeNoPRL`: The spacing between two cuts should be the greater of the following two values: the maximum of all applicable values, measured `center-to-center`; and the minimum of all applicable values, measured `edge-to-edge`.

Note:

The `colorName` must be defined in the [Controls Section](#), and the color mask value cannot be greater than the `numberOfColorMasks` value defined in the [functions](#) section.

The warning message is displayed in the following situations:

- Per cut class pair, different cut class combination has different checking. Setting means spacing, measure type and extension.

	cutClass2 shortEdge	cutClass2 longEdge
cutClass1 shortEdge	Setting A	Setting B
cutClass1 longEdge	Setting C	Setting D

- Square cut class to square cut class: All settings (A, B, C, D) should be the same. Otherwise, the warning message is displayed.
- Square cut class to rectangular cut class: Setting of rectangular cut class to shortEdge or longEdge of square cut class should be the same. If cutClass1 is square and cutClass2 is rectangular, then setting A should

equal to setting C and setting B should equal to setting D. Otherwise, the warning message is displayed.

- Rectangular cut class to rectangular cut class (same size): Setting B should equal to setting C. Otherwise, the warning message is displayed.
- The setting of cutClassA → cutClassB is not equal to cutClassB → cutClassA.

Note:

Since `cutClassCenterToCenter` and `cutClassMeasureType` parameters are represented by `oaSpacingMeasureType` enumeration, and `cutClassMeasureType` is the superset:

- When importing, always create both `cutClassMeasureType` and `cutClassCenterToCenter` parameters for backward compatibility. But only `edgeToEdge` and `centerToCenter` values go into the `cutClassCenterToCenter` table.
- When exporting, use the `cutClassMeasureType` table if it exists. If it does not exist, then use `cutClassCenterToCenter` table if it exists.

To define spacing center based between square cuts short edge:

```
(minCutClassSpacing V0
(
((0.032 -shortEdge) (0.032 -shortEdge)) (0.107 -centerToCenter)
...
)
)
```

To define spacing edge based between rectangle cuts short edge:

```
(minCutClassSpacing V0
(
...
(((0.032 0.064) -shortEdge) ((0.032 0.064) -shortEdge)) 0.112
...
)
)
```

To define spacing edge based with extension value between square cut short edge and rectangle cut short edge:

```
(minCutClassSpacing V0
(
...
(((0.032 -shortEdge) ((0.032 0.064) -shortEdge)) (0.088 0.06)
...
)
)
```

To define spacing center based with extension value between rectangle cut long edge and square cut short edge:

```
(minCutClassSpacing V0
(
...
(((0.032 0.064) -longEdge) (0.032 -shortEdge))
(0.068 -centerToCenter 0.06)
...
)
)
```

To define spacing with extension and `measureType=centerAndEdgeNoPRL` value between rectangle cut long edge and square cut short edge:

```
(minCutClassSpacing V0
(
...
(((0.032 0.064) -longEdge)
(0.032 -shortEdge))
(0.068 -centerAndEdgeNoPRL 0.06)
...
)
)
```

To define `minCutClassSpacing` for cutClass (0.032 0.032) and (0.032 0.064):

```
(minCutClassSpacing V0
(
((0.032 -shortEdge) (0.032 -shortEdge)) (0.107 -centerToCenter)
((0.032 -shortEdge) (0.032 -longEdge)) (0.107 -centerToCenter)
((0.032 -shortEdge) ((0.032 0.064) -shortEdge)) 0.088
((0.032 -shortEdge) ((0.032 0.064) -longEdge)) 0.068
((0.032 -longEdge) (0.032 -shortEdge)) (0.107 -centerToCenter)
((0.032 -longEdge) (0.032 -longEdge)) (0.107 -centerToCenter)
((0.032 -longEdge) ((0.032 0.064) -shortEdge)) 0.088
((0.032 -longEdge) ((0.032 0.064) -longEdge)) 0.068
(((0.032 0.064) -shortEdge) (0.032 -shortEdge)) 0.088
(((0.032 0.064) -shortEdge) (0.032 -longEdge)) 0.088
(((0.032 0.064) -shortEdge) ((0.032 0.064) -shortEdge)) 0.112
(((0.032 0.064) -shortEdge) ((0.032 0.064) -longEdge)) 0.088
(((0.032 0.064) -longEdge) (0.032 -shortEdge)) 0.068
(((0.032 0.064) -longEdge) (0.032 -longEdge)) 0.068
(((0.032 0.064) -longEdge) ((0.032 0.064) -shortEdge)) 0.088
(((0.032 0.064) -longEdge) ((0.032 0.064) -longEdge)) 0.088
)
)
```

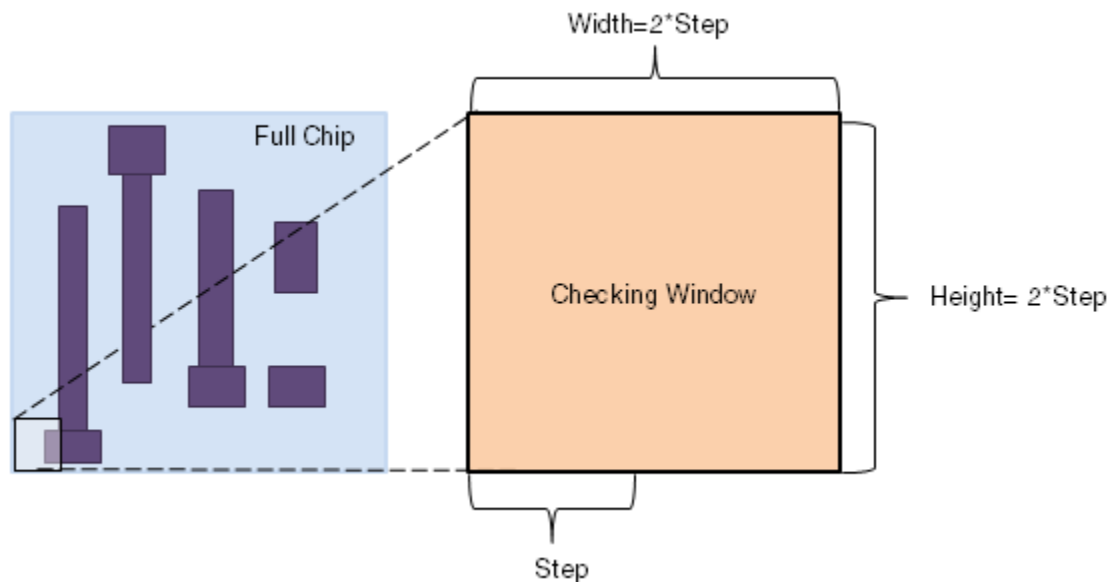
minDensity

This layer constraint specifies the minimum percentage of an area of the design that must be filled with shapes on a particular layer.

Density constraints are generally represented in two ways:

- As the minimum density for an area across the entire design.
- As a window-based minimum density. Here the area of concern is a window that is moved across the design.

You can use the [maxDensity](#) layer constraint in conjunction with the [minDensity](#) constraint to ensure optimum metal density for a design.



Constraint Category	Density
Constraint Type	Complex
OA Rule	oacMinDensity

Syntax:

```
minDensity
  <string|LPP>layer
  <float|float1DTable>areaValue
  [-windowStepSize 1DTableParamV]
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or a LPP.

Argument	Description
areaValue	• float: Minimum density required for the layer over the entire design. • float1DTable: Minimum density required for each window area of the design tiled into windows with a height and width each equal to twice the step size.
-windowStepSize	This parameter is only applicable when the constraint value is a table. The value of this parameter is a table where the key is the step size in microns and the value is the window size in microns.

In this example, the minimum density of the POLY across the chip is mentioned:

Rule	Description	Value (in microns)
PO.DEN.1	Density of POLY across the chip	>=20%

```
minDensity POLY 0.2
```

In this example, the minimum local density of the POLY is mentioned:

Rule	Description	Value (in microns)
PO.DEN.2	Density of POLY in 30um X 30um window	>=10%

```
minDensity POLY 0.2 ((15 0.1))
```

minDiagonalEdgeLength

This layer constraint specifies the minimum edge length required for a diagonal shape. The distance is specified as the Euclidean distance in microns.

Constraint Category	Length and Width Process Rules
Constraint Type	Complex
OA Rule	oacMinDiagonalEdgeLength

Syntax:

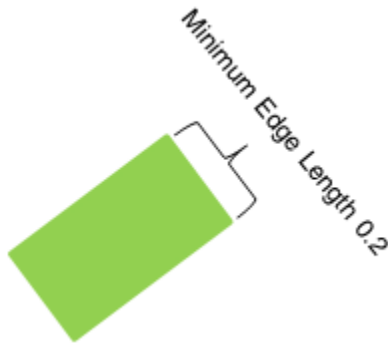
```
minDiagonalEdgeLength
    <string|LPP>layer
    <float>lengthValue
```

```
[-isHard true|false]
[-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or a LPP.
lengthValue	Specifies the minimum edge length required for a diagonal shape.

In this example, the minimum diagonal edge length of DIFF is mentioned:

Rule	Description	Value (in microns)
DIFF.L.1	minimum Diagonal Edge Length for DIFF	0.2



```
minDiagonalEdgeLength DIFF 0.2
```

minDiagonalSpacing

This layer constraint specifies the minimum diagonal distance required between any two geometries. The distance is specified as the Euclidean distance in microns.

Constraint Category	Length and Width Process Rules
Constraint Type	Complex
OA Rule	oacMinDiagonalSpacing

Syntax:

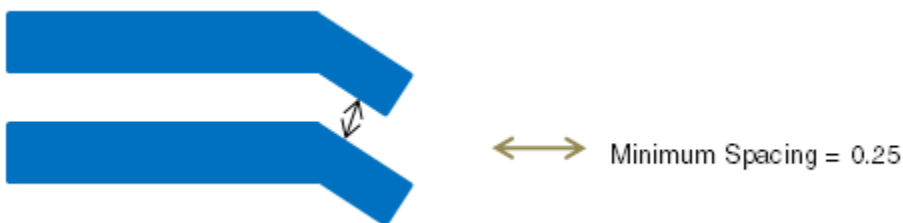
```
minDiagonalSpacing
  <string|LPP>layer
  <float|float1DTable|float2DTable>minSpacing
```

```
[-isHard true|false]
[-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or a LPP.
minSpacing	float: Minimum diagonal spacing required between the two shapes. float1DTable: Specifies the width of the wider of the two shapes and the minimum diagonal spacing required between the two shapes. float2DTable: Specifies the width of the wider of the two shapes, parallel run length between the two shapes, and the minimum diagonal spacing required between the two shapes.

In this example, the minimum diagonal spacing for `DIFF` shapes is mentioned:

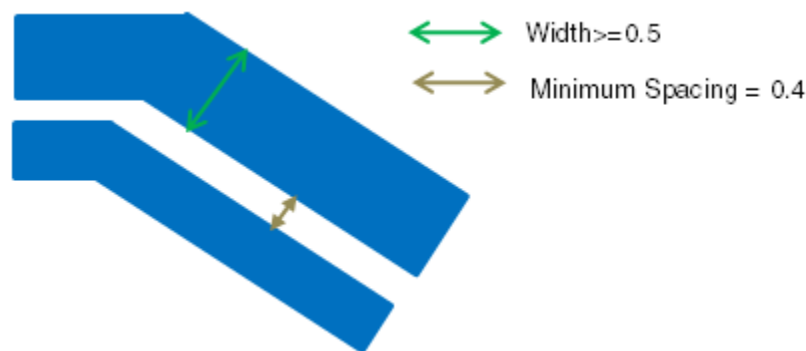
Rule	Description	Value (in microns)
DIFF.SP.1	minimum Diagonal Spacing for DIFF	0.25



```
minDiagonalSpacing DIFF 0.25
```

In this example, the minimum width dependent diagonal spacing for `DIFF` shapes is mentioned:

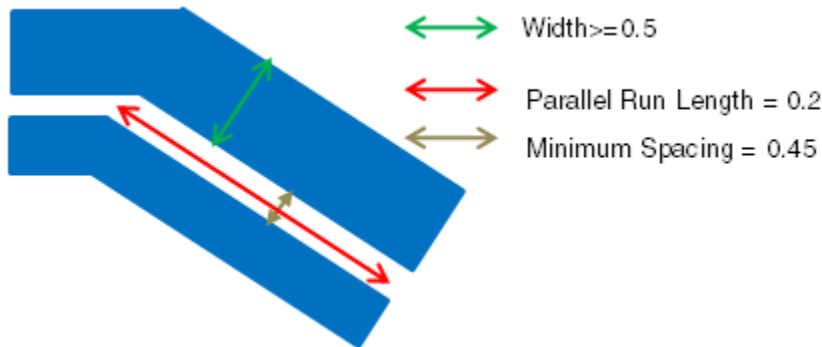
Rule	Description	Value (in microns)
DIFF.SP.1.1	minimum Diagonal Spacing for DIFF whose width ≥ 0.5	0.4



```
minDiagonalSpacing DIFF 0.25 ((0.5 0.4))
```

In this example, the minimum width and common run dependent diagonal spacing for DIFF shapes is mentioned:

Rule	Description	Value (in microns)
DIFF.SP.1.2	minimum Diagonal Spacing for DIFF whose width ≥ 0.5 and for a CR 0.45 of 0.2	



```
minDiagonalSpacing DIFF 0.25 (((0.5 0.2) 0.45))
```

minDiagonalWidth

This layer constraint specifies the minimum diagonal width for any geometry on the specified layer. This value is specified in microns.

Constraint Category	Length and Width Process Rules
Constraint Type	Simple
OA Rule	oacMinDiagonalWidth

Syntax:

```
minDiagonalWidth
  <string|LPP>layer
  <float>widthValue
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or as LPP.
widthValue	Specifies the minimum diagonal width for any geometry on the specified layer.

In this example, the minimum diagonal width of diffusion is specified:

Rule	Description	Value (in microns)
DIFF.DW.1	minimum Diagonal Width for DIFF	0.2

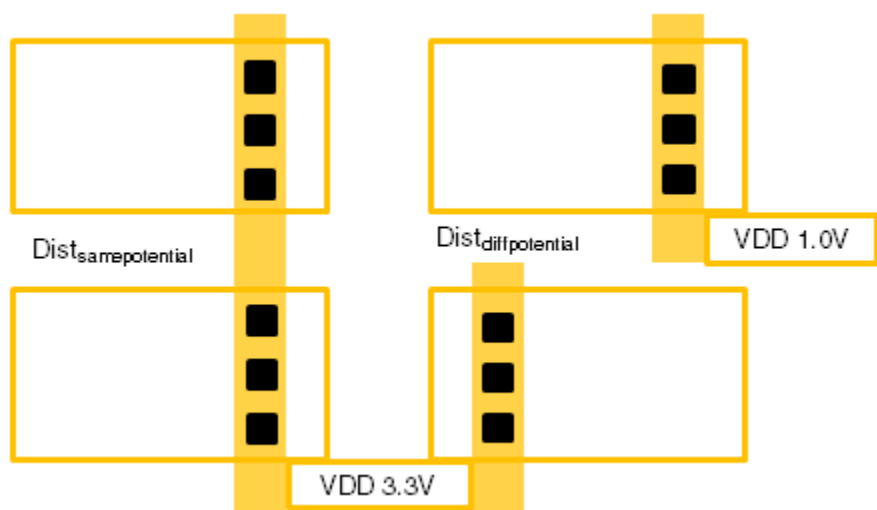


```
minDiagonalWidth DIFF 0.2
```

minDiffPotentialSpacing

This layer constraint specifies the minimum distance required between any two objects that are on the same layer at different voltage levels.

This constraint typically applies to wells, transistors, and devices that are at different voltage levels.



Constraint Category	Spacing
Constraint Type	Simple
OA Rule	oacMinDiffPotentialgonalWidth

Syntax:

```
minDiffPotentialSpacing
  <string|LPP>layer
  <float>spacingValue
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or as LPP.
spacingValue	Specifies the minimum distance required between any two objects that are on the same layer at different voltage levels.

In this example, the spacing between N-well layers with different potentials is mentioned:

Rule	Description	Value (in microns)
NWELL.SP.2	N-Well spacing to another N-Well at different potential	1.5

```
minDiffPotentialSpacing  NWELL  1.5
```

minEdgeAdjacentDistance

This layer constraint specifies the minimum required distance between two sets of connected minimum length edges when measured along the edges.

The constraint also allows you to specify the following conditions that cause this constraint to come into effect:

- The maximum edge length for the edge to be considered as the short edge.
- The minimum number of minimum length edges that must be connected for this constraint to apply.

Constraint Category	Length and Width Process Rules
Constraint Type	Complex
OA Rule	oacMinEdgeAdjacentDistance

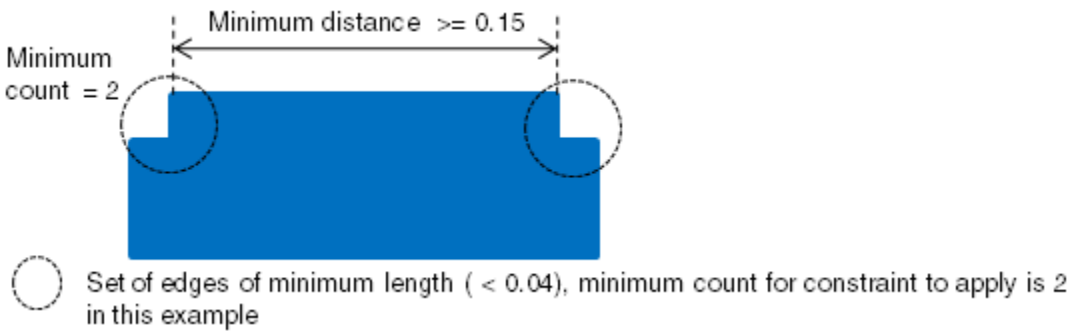
Syntax:

```
minEdgeAdjacentDistance
    <string|LPP>layer
    <float>minDistance
    -length float
    [-count int]
    [-exceptSameCorner true|false]
    [-isHard true|false]
    [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or a LPP.
minDistance	Specifies the minimum distance along the edge between one set of connected minimum length edges and the other set of connected minimum length edges.
-length	Specifies the length under which the constraint does not apply.
-count	Specifies the number of edges over which the constraint does not apply.
-exceptSameCorner	Specifies that the constraint does not apply for an edge that has the same type (concave or convex) of 90-degree corner at each end.

In this example, the minimum distance between two sets of connected minimum length edges (which have 2 short edges having a length of < 0.04) is specified:

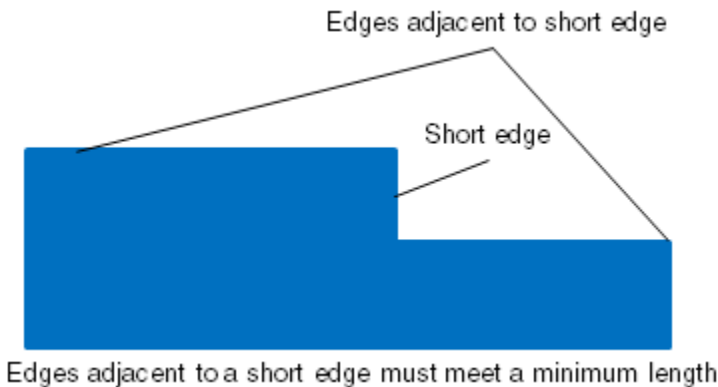
Rule	Description	Value (in microns)
M1.SE.3	Minimum distance between two connected short edge pairs, where each pair has at least 2 short edges with length < 0.04	0.15



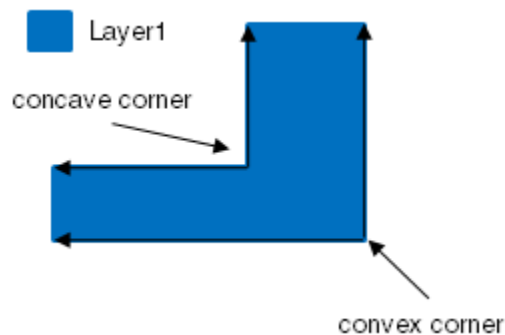
minEdgeAdjacentDistance M1 0.15 -length 0.04 -count 2

minEdgeAdjacentLength

This layer constraint specifies the minimum length of edges on a polygon when the length of an adjacent short edge is less than a specified value.



Some processes require minimum edge lengths for edges adjacent to one or more consecutive short edges. Specializations of this rule require two edges to form a concave or convex corner.



Constraint Category	Length and Width Process Rules
Constraint Type	Complex
OA Rule	oacMinEdgeAdjacentLength

Syntax:

```
minEdgeAdjacentLength
  <string|LPP>layer
  <float|dualFloat>lengthValue
  -maxLength <float>
  [-count int]
  [-convexCorner true|false]
  [-concaveCorner true|false]
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or a LPP.
lengthValue	float: Specifies the minimum length of both adjacent edges. dualFloat: Specifies the minimum length of two adjacent edges separately specified in each value of oaDualInt.
-maxLength	Specifies the maximum length of short edge. The rule only applies when the length of the short edge is less than this value in database units.
-count	Specifies the maximum number of connected short edges that trigger this constraint.
-convexCorner	Only valid if the constraint value type is oaIntVaue. It indicates that if a convex corner is between two concave corners, and if one of the length of the edges to form the convex corner is less than the constraint value, then the other length must be greater than or equal to <code>maxLength</code> . Default value is <i>false</i> .

Argument	Description
-concaveCorner	Indicates that if a concave corner is between two convex corners, and if one of the length of the edges to form the concave corner is less than the constraint value, then the other length must be greater than or equal to <code>maxLength</code> . This parameter and <code>convexCorner</code> cannot both be true. Default value is <i>false</i> .

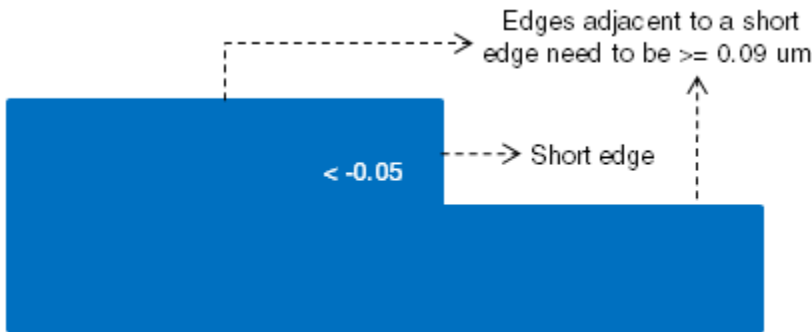
Note:

The error message is displayed in the following situations:

- `-convexCorner` and `-concaveCorner` are set to true at the same time.
- `-convexCorner` or `-concaveCorner` is specified and the constraint value is dual value.

In this example, the minimum edge length of the adjacent side to a short edge of length < 0.05 is mentioned:

Rule	Description	Value (in microns)
M1.SE.1	Minimum length of an edge adjacent to an edge of length <0.05	0.09

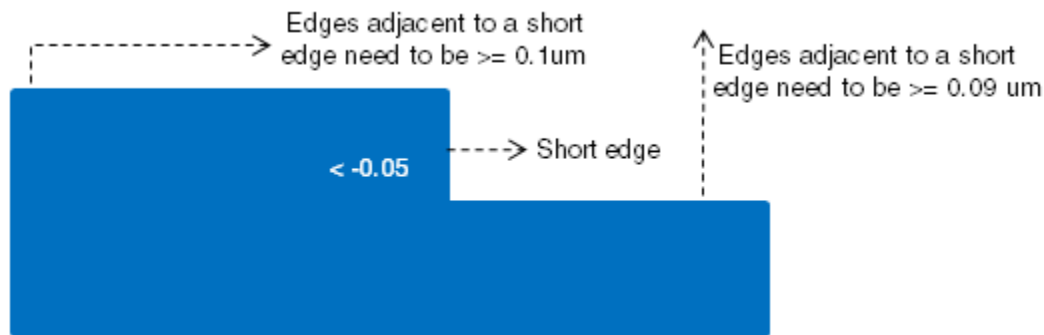


Edges adjacent to a short edge must meet a minimum length

```
minEdgeAdjacentLength M1 0.09 -maxLength 0.05
```

In this example, the minimum edge length of the two adjacent sides to a short edge of length < 0.5 is mentioned:

Rule	Description	Value (in microns)
M1.SE.2	Minimum lengths of the edges adjacent to an edge of length <0.05	0.09, 0.1

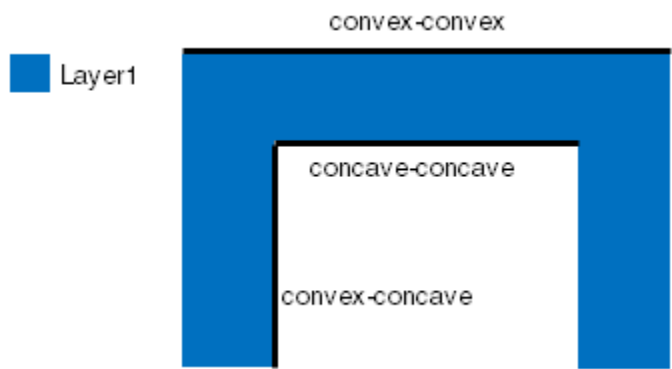


Edges adjacent to a short edge must meet a minimum length

```
minEdgeAdjacentLength M1 0.09 0.1 -maxLength 0.05
```

minEdgeByType

This layer constraint specifies the edge type associated with the two end points.



Some processes have edge length requirements based upon the type of corners on either side of the edge. The corner types are concave or convex.

Therefore, there are three types of edge for the different combinations:

- convex-convex
- convex-concave
- concave-concave

Constraint Category	Length and Width Process Rules
Constraint Type	Complex
OA Rule	Custom Compiler Custom Layer Constraint

Syntax:

```
minEdgeByType
  <string|LPP>layer
  <float>LengthValue
  [-edgeType convexConvex|
    convexConcave|
    concaveConcave]
  [-edgeDirection any|horizontal|vertical]
  [-forbiddenRanges rangeValue]
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or a LPP.
lengthValue	Specifies the minimum edge length.
-edgeType	The edge type associated with the two end points. The types are: - convexConcave (Default) - convexConvex - concaveConcave
-edgeDirection	Optional parameter specifying the direction of edges the rule applies to. Valid values: 0 (any), 1 (horizontal), and 2 (vertical). Default: any
-forbiddenRanges	If given, imposes additional restriction on the edge length: it cannot lie in any of these ranges in database units.

For example:

```
minEdgeByType m1 0.10
                        -edgeType concaveConcave
                        -edgeDirection vertical
                        -forbiddenRanges ((0.02 0.03) (0.04 0.05))
```

minEdgeLength

Some processes may have different requirements for minimum edge length of metal shapes depending on the width of the wire along the edge. Also, these requirements may be different for edges oriented in preferred and non-preferred routing directions.

Note:

The minEdgeLength constraints should be used in conjunction with [minConcaveEdgeLength](#) and [minConvexEdgeLength](#) constraints to determine whether a shape with a step pattern is suitable for Optical Pattern Correction (OPC).



Constraint Category	Length and Width Process Rules
Constraint Type	Complex
OA Rule	oacMinEdgeLength

Syntax:

```
minEdgeLength
  <string|LPP>layer
  <float>LengthValue
  [-lengthSum float]
  [-widthRange rangeValue]
  [-direction any|preferred|nonpreferred]

  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or a LPP.
lengthValue	Specifies the minimum edge length.
-lengthSum	When the lengthSum parameter is specified, the constraint does not apply if the sum of the consecutive edges shorter than the minimum edge length value is less than value of the lengthSum parameter.
-widthRange	If given, the rule only applies to edges where the width of the wire at some point along the edge is greater than or equal to this value in database units. The width is measured in the direction perpendicular to the direction of the edge.
-direction	The direction of edges the rule applies to. Valid values are 0 (any), 1 (preferred direction), 2 (non-preferred direction). Default: any

In this example, the minimum edge length of M1 is mentioned:

Rule	Description	Value (in microns)
M1.EL.1	Minimum edge length	0.09

```
minEdgeLength M1 0.09
```

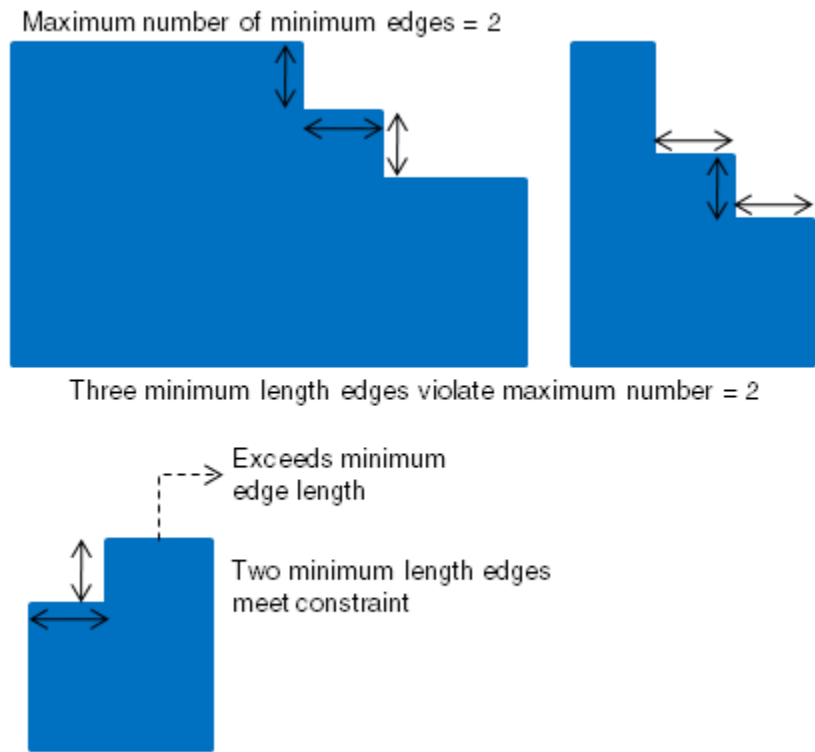
In this example, the minimum edge length of M1 is checked only when the sum of the consecutive short edges is $\geq 0.1\mu\text{m}$:

Rule	Description	Value (in microns)
M1.EL.1.1	Minimum edge length rule needs to be 0.09 when the sum of consecutive short edges is ≥ 0.1	N/A

```
minEdgeLength m1 0.04 -lengthSum 0.06 -widthRange (>=0.05 <0.10)
```

minEdgeMaxCount

This layer constraint specifies a maximum number of consecutive small edges allowed in a step that is equal to or smaller than the minEdgeLength.



Constraint Category	Length and Width Process Rules
Constraint Type	Complex
OA Rule	oacMinEdgeMaxCount

Syntax:

```
minEdgeMaxCount
  <string|LPP>layer
  <int>maxNumberOfEdges
  <-maxLength float>
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or a LPP.
maxNumberOfEdges	Specifies the maximum number of consecutive small edges.
-maxLength	Specifies the maximum length of the edge in a set of consecutive small edges. The maximum length must be < the specified <code>maxLength</code> value.

In this example, the maximum number of consecutive small edges with edge length <= 0.05 is specified:

Rule	Description	Value (in microns)
M1.SE.Count	Maximum number of consecutive small edges having an edge of length <0.05	2

```
minEdgeMaxCount M1 2 -maxLength 0.05
```

minEnclosedArea

This layer constraint specifies the minimum area for an empty area that is enclosed by metal (when the metal forms a donut hole).

Note:

You can use this rule along with [minEnclosedSpacing](#), because although a hole might meet the minimum area requirement, it might not meet the minimum space requirement.

Constraint Category	Area
Constraint Type	Complex
OA Rule	oacMinEnclosedArea

Syntax:

```
minEnclosedArea
    <string|LPP>layer
    <float|float1DTable>areaValue
    [-isHard true|false]
    [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or a LPP.
areaValue	float: specifies the minimum enclosed area in μm^2. float1DTable: - Is a list of two values. The first value specifies the width of the shape and the second value is the enclosed area of the hole in the shape.

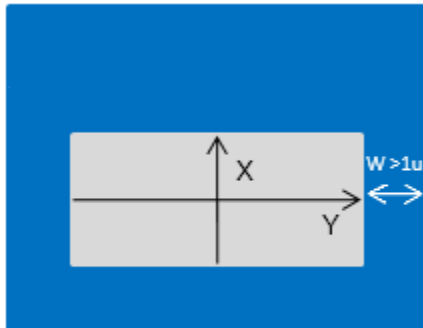
Rule	Description	Value (in μm^2)
M1.EA.1	Minimum M1 enclosed Area	0.2



$X * Y \geq \text{Minimum enclosed area (0.1}\mu\text{m}^2)$

```
minEnclosedArea M1 0.2
```

Rule	Description	Value (in μm^2)
M1.EA.1.1	Minimum M1 enclosed Area for M1 whose width $>1\mu$	0.4



$X * Y \geq \text{Minimum enclosed area } (0.4\mu\text{m}^2)$

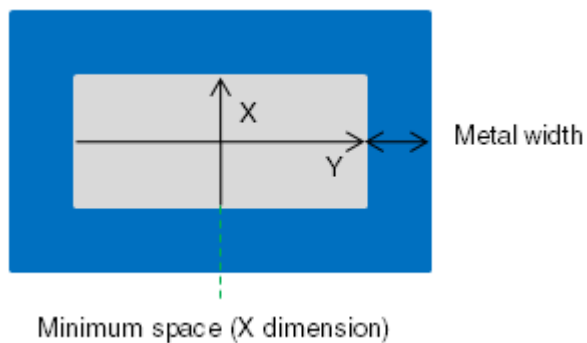
```
minEnclosedArea M1 0.2 ((1 0.4))
```

minEnclosedSpacing

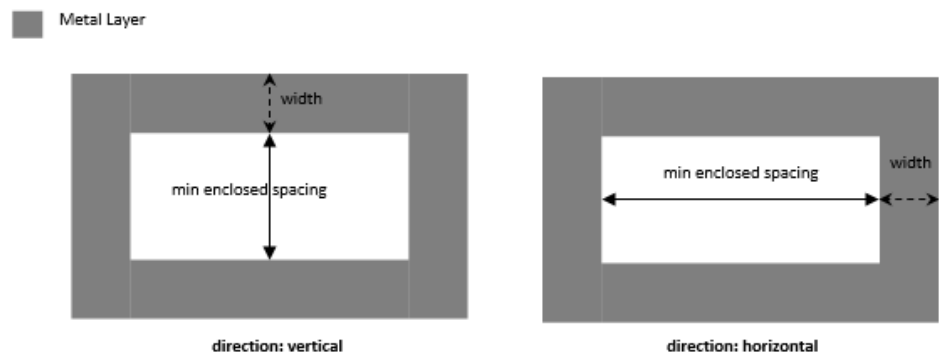
This layer constraint specifies the minimum X and Y dimensions of a hole in a geometry. Both the X and Y dimensions must exceed the minimum spacing value established by this constraint.

Note:

This constraint should be used in conjunction with the [minEnclosedArea](#) constraint. It is possible for geometries to meet the minimum area requirement, but not meet the minimum enclosed space.



Some processes may have separate requirements for horizontal and vertical dimensions of holes in polygons on some layers. This rule is modeled by the built-in `minEnclosedSpacing` constraint with new parameter `-spacingDirection`.



Constraint Category	Spacing
Constraint Type	Complex
OA Rule	oacMinEnclosedSpacing

Syntax:

```
minEnclosedSpacing
  <string|LPP>layer
  <float|float1DTable>minSpacing
  [-spacingDirection any|horizontal|vertical]
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or a LPP.
minSpacing	• float: Specifies the minimum space required. • float1DTable: Specifies the width of the enclosing shape, and the minimum space required for a hole in both X and Y direction surrounded by shape (of specified width)
-spacingDirection	Optional parameter specifying the direction in which the rule applies. Valid values: 0 (any), 1 (horizontal), 2 (vertical). Default: any

In this example, the minimum Enclosed Spacing for M1 is mentioned:

Rule	Description	Value (in microns)
M1.ESP.1	minimum Enclosed Spacing for M1	0.12

```
minEnclosedSpacing M1 0.12
```

In this example, the minimum enclosed spacing based on the width of the enclosing Metal donut is mentioned:

Rule	Description	Value (in microns)
M1.ESP.1.1	minimum Enclosed Spacing for M1 when width of Metal donut is ≥ 0.2	0.15

```
minEnclosedSpacing M1 0.12 ((0.2 0.15))
```

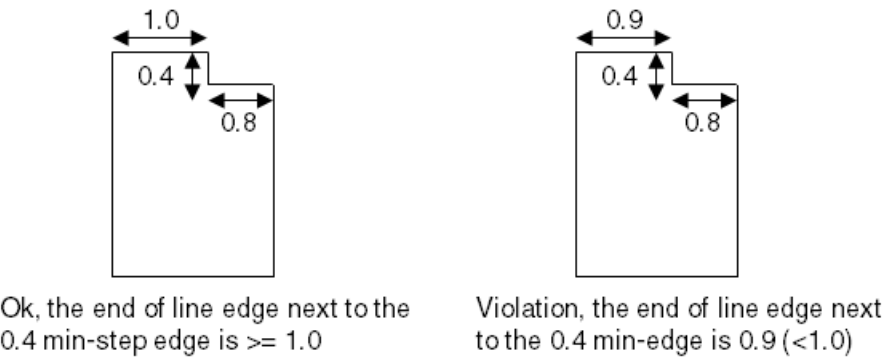
In this example, the minEnclosedSpacing constraint specified on the poly with simple value.

```
minEnclosedSpacing poly 0.10 -spacingDirection horizontal
```

minEndOfLineAdjacentToStep

A minimum step rule optionally requires a certain minimum length for end-of-line edges that are adjacent to a minimum edge. If a minimum step edge has an adjacent edge that is also an end-of-line edge, that end-of-line edge must not be shorter than a certain value.

minEndOfLineAdjacentToStep, value = 1.0, maxLength = 0.5



Constraint Category	Length and Width Process
Constraint Type	Complex
OA Rule	oacMinEndOfLineAdjacentToStep

Syntax:

```
minEndOfLineAdjacentToStep
  <string|LPP>layer
  <float>spacingValue
```

```
-maxLength float
[-noBetweenEol true|false]
[-isHard true|false]
[-comment [<string rule_id>] <string rule_desc>]
```

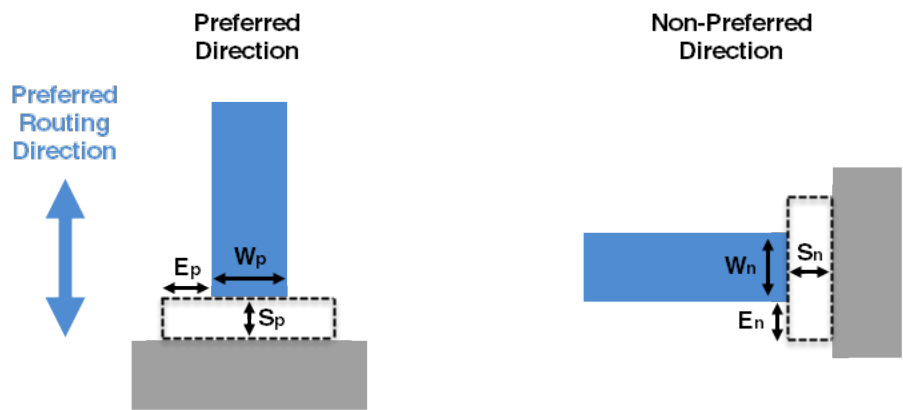
Argument	Description
layer	Layer can be defined as a string or a LPP.
spacingValue	This value represents the required minimum length of an end-line edge adjacent to an edge less than maxLength.
-maxLength	This constraint applies if the end-of-line edge has an adjacent edge < maxLength.
-noBetweenEol	If specified and set to true, any end-of-line with length less than eolWidth (the value of the constraint) cannot have adjacent steps of length less than the maxLength parameter.

For example:

```
minEndOfLineAdjacentToStep M1 0.4 -maxLength 0.5 -noBetweenEol true
```

minEndOfLineDirectionSpacing

The `minEndOfLineDirectionSpacing` constraint describes the minimum spacing (S) needed between the end of a wire and a nearby shape. The requirement can differ, depending on whether the wire and the edge at the end of the wire is in the preferred (p) or non-preferred (n) routing direction. The requirement also considers the width at the end of the wire (W) and the spacing extension (E), as the following diagram shows.



Constraint Category	Spacing
Constraint Type	Complex

OA Rule Custom Compiler Custom Layer Constraint

Syntax:

```
minEndOfLineDirectionSpacing
    <string|LPP>layer
    <float>spacingValue
    -extension float
    -width float
    [-nonPrefDir true|false]
    [-minLength float]
    [-maxLength float]
    [-color any|same|diff]
    [-isHard true|false]
    [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or a LPP.
spacingValue	Spacing required at the end of the wire that is in either the preferred or nonpreferred direction.
-extension	The lateral extension of the spacing (keepout region) between the wire and the neighboring object.
-width	The width at the wire end. For the constraint to apply, the width must be less than or equal to the specified value. In addition, the edge of the wire end must be 90° from the wire's running length . 
-nonPrefDir	Specifies whether the spacing is for wires that are in the preferred direction or non-preferred direction.
-minLength	The minimum length of the wire for this constraint to apply. If the length measured from the end of the wire is less than this value, the constraint does not apply. Default: 0.0
-maxLength	The maximum length of the wire for this constraint to apply. If the length measured from the end of the wire is greater than this value, the constraint does not apply. Default: infinity
-color	Color of the end-to-end shape for which the constraint applies. Valid values are 0 (any), 1 (same), and 2 (different). Default: 0

For example:

```
minEndOfLineDirectionSpacing m1 0.15 -extension 0.12
                                -width 0.16
                                -nonPrefDir true
                                -maxLength 0.9
```

minEndOfLineExtensionSpacing

This constraint specifies the minimum spacing between the end of line edge and neighboring wire edges for a given end of line width where the end of line should be extended by the specified extension. If the length of the sides adjacent to the end of line edge is less than an optional minimum length, then the constraint does not apply. The minimum length requirement optionally depends on one or both of the sides adjacent to the end of line edge.

Constraint Category	Spacing
Constraint Type	Complex
OA Rule	oacMinEndOfLineExtensionSpacing

Syntax:

```
minEndOfLineExtensionSpacing
    <string|LPP>layer
    <float>spacingValue
    -extensionSpacing dual1DTblValue
    [-length float]
    [-twoSides true|false]
    [-isHard true|false]
    [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or a LPP.
spacingValue	The spacing value in microns.
-extensionSpacing	Two 1D tables with the extension that is applied before checking the spacing, indexed by width. The first table applies to end-to-side cases, and the second table applies to end-to-end. The extensionSpacing is represented as: ((1DTblData1*) (1DTblData2*)) Where: 1DTblData1= (<float>width1 <float>endToSide) 1DTblData2= (<float>width2 <float>endToEnd)
-length	The constraint does not apply if the end-of-line length is less than this value along both sides.

Argument	Description
-twoSides	The constraint does not apply if the end-of-line length is less than the value of the length parameter along any one side.

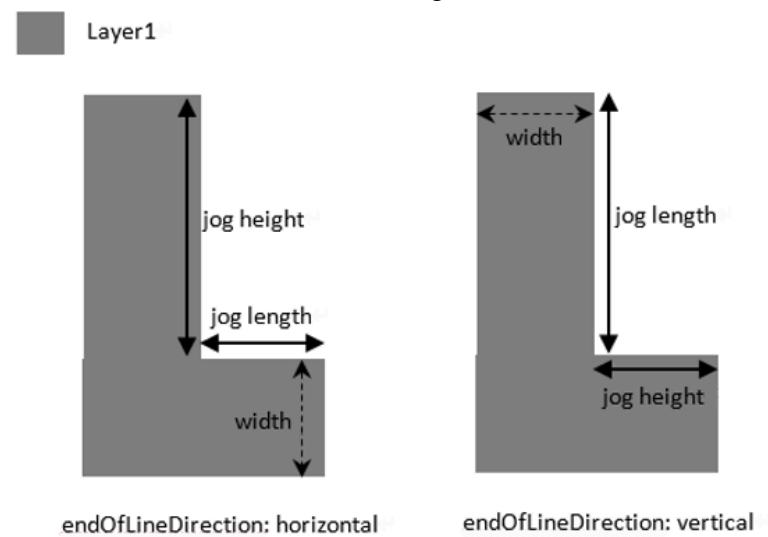
For example:

```
minEndOfLineExtensionSpacing ml 0.06 -extensionSpacing (
    ((0.1 0.04) (0.12 0.06))
    ((0.1 0.06) (0.12 0.08)))
```

minEndOfLineJogDistance

The `minEndOfLineJogDistance` defines the jog that is permitted when the width at the end of the wire is within a certain value. A jog at wire end is a convex/convex/concave/convex sequence of vertices. The two connected edges adjacent to the end of the line, edge with two convex corners, form a jog. The jog length must be greater than or equal to a given length threshold or the jog height must be greater than or equal to a height threshold.

Figure 13 Minimum End Of Line Jog Distance Rule



Constraint Category	Spacing
Constraint Type	Complex
OA Rule	Layer

Syntax:

```
minEndOfLineJogDistance
  <string|LPP>layer
  <float>length
  <float>height
  -width float|-widthRange rangeData
  [-bothEdges true|false]
  [-endOfLineDirection any|horizontal|vertical]
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

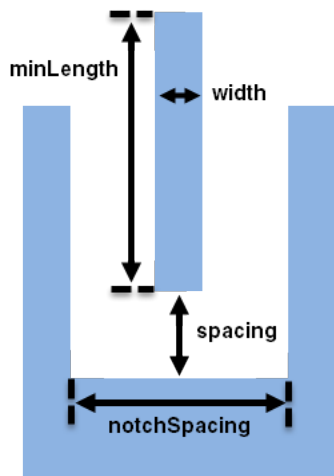
Argument	Description
layer	The layer on which the shape is placed. The layer can be defined as a string or a LPP.
length	The minimum jog length.
height	The minimum jog height.
-width	The constraint is applied when the wire width is less than the specified width value. This parameter or the -widthRange parameter must be given, but not both.
-widthRange	If given, the constraint only applies if the length of the end of line edge lies in the given range in database units. This parameter or the -width parameter must be given, but not both.
-bothEdges	The jog height and jog length must be greater than or equal to their constraint values when set to true. Default: false
-endOfLineDirection	If specified, the rule applies only to end of line wire segments oriented in this direction. Valid values are 0 (any), 1 (horizontal), 2 (vertical). Default: any

In the following example, the constraint requires that m1 wires with widths less than 0.08 are permitted to have jogs that are at least 0.06 in length and 0.04 in height.

```
minEndOfLineJogDistance m1 0.06 0.04
                        -width 0.08
```

minEndOfLineNotchSpacing

The minEndOfLineNotchSpacing constraint specifies the minimum spacing required between the two inside edges of a notch structure when the end of a shape is within the notch area on the same layer. Both routes and shields are aware of the constraint.



Constraint Category	Spacing
Constraint Type	Complex
OA Rule	Layer

Syntax:

```
minEndOfLineNotchSpacing
  <string|LPP>layer
  -width <float>
  -minLength <float>
  -maxNotchSpacing <float>
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

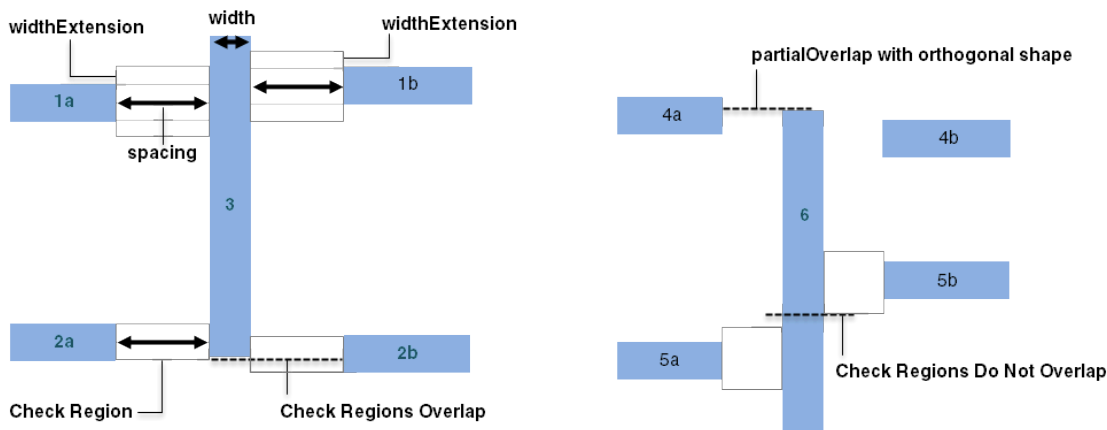
Argument	Description
layer	The layer on which the shapes are placed. The layer can be defined as a string or a layer-purpose pair.
-width	The maximum width of the shape that is within the notch. The constraint applies if the width of shape is <= the specified width value.
-minLength	The minimum length of the shape that is within the notch. The constraint applies if the length of the shape is >= the specified minLength value.
-maxNotchSpacing	The maximum distance between the two inside edges of the notch. The constraint applies if the distance is <= the specified maxNotchSpacing value.

In the following example, the constraint requires that for m1 shapes that are within notch structures, the shape width must be less than or equal to 0.02, the shape length must be 0.05 or longer, and the notch distance cannot exceed 0.04.

```
minEndOfLineNotchSpacing m1 0.02
                           -width 0.02
                           -minLength 0.05
                           -maxNotchSpacing 0.04
```

minEndOfLineOrthoSpacing

The `minEndOfLineOrthoSpacing` constraint specifies the spacing requirements when the ends of shapes that face each other (1a and 1b) are separated by an orthogonal shape (3) on the same layer. The distance (spacing) between each of the facing shapes (1a and 1b) and the corresponding orthogonal shape (3) form the check region. The pair of check regions must overlap for the constraint to apply. (In the following example, the constraint does not apply to shapes 5a, 5b, and 6 because the check regions do not overlap.)



Constraint Category	Spacing
Constraint Type	Complex
OA Rule	Layer

Syntax:

```
minEndOfLineOrthoSpacing
  <string|LPP>layer
  <float>spacing
  -width <float>
  -maxSpacing <float>
  [-partialOverlap true|false]
  [-widthExtension float]
```

```
[-isHard true|false]
[-comment [<string rule_id>] <string rule_desc>]
```

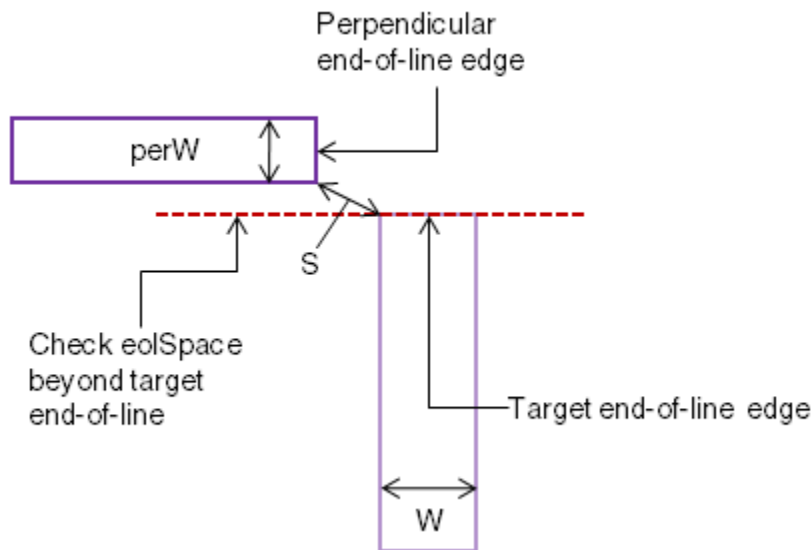
Argument	Description
layer	The layer on which the shapes are placed. The layer can be defined as a string or a layer-purpose pair.
-width	The maximum width of the shapes. The constraint applies if the width of the shape ends is < the specified <code>width</code> value.
-maxSpacing	The maximum distance between the end of the facing shapes and the orthogonal shape. The constraint applies if the distance between one of the shape ends is < the specified <code>maxSpacing</code> value.
-partialOverlap	(Optional) Applies the constraint even if one of the shapes partially overlaps with the orthogonal wire. The default is <code>false</code> (applies the constraint only if all wires fully overlap with the orthogonal wire). In the example shown, if <code>partialOverlap</code> is <code>true</code> , then the constraint applies to shapes 2a, 2b, and 3. If <code>partialOverlap</code> is <code>false</code> , then the constraint does not apply to shapes 2a, 2b, and 3.
-widthExtension	(Optional) Extends the width of the check region by the specified amount. The default is 0. In the example shown, the width of the check regions for shapes 1a and 1b is extended.

In the following example, the constraint requires that the ends of m1 shapes have a width narrower than 0.08. The maximum distance between the ends of the shapes and the orthogonal shape must be less than 0.12. The constraint applies even if at least one shape end overlaps with the orthogonal shape. The width of check regions are increased by 0.02.

```
minEndOfLineOrthoSpacing m1 0.06
    -width 0.08
    -maxSpacing 0.12
    -partialOverlap true
    -widthExtension 0.02
```

minEndOfLinePerpSpacing

Some processes require a particular end of line spacing for lines with a width less than a given width beyond the end-of-line to a perpendicular edge with width less than a minimum width. Typically, spacing is slightly larger than the minimum allowed regular spacing or end of line spacing on the layer. Although, only one such end of line spacing rule is expected for a given layer, there can be an end of line spacing rule to a perpendicular edge rule coupled with multiple regular spacing or end of line spacing rules for a single layer.



Constraint Category	Spacing
Constraint Type	Complex
OA Rule	oacMinEndOfLinePerpSpacing

Syntax:

```
minEndOfLinePerpSpacing
  <string|LPP>layer
  <float>spacingValue
  -width float
  -endOfLinePerpWidth float
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or a LPP.
spacingValue	Specifies the end of line spacing for lines with a width less than a given width.
-width	Minimum width in microns.
-endOfLinePerpWidth	Minimum perpendicular width of the end-of-line edge.

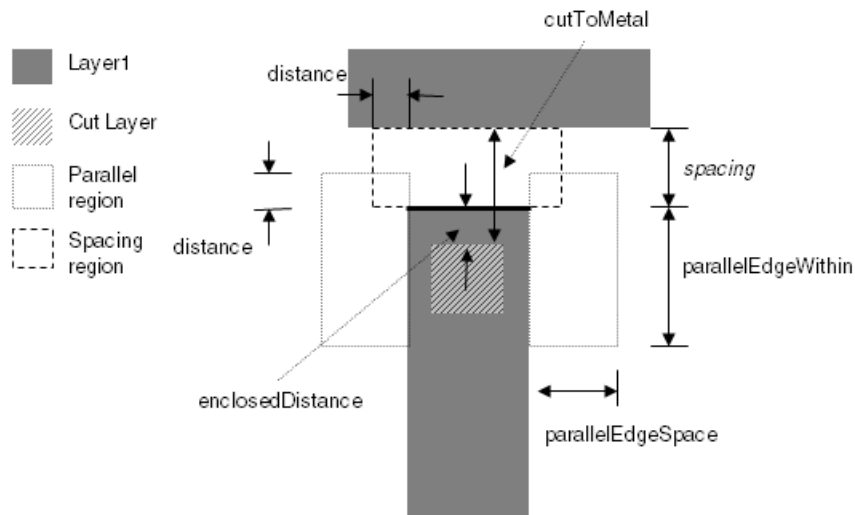
For example:

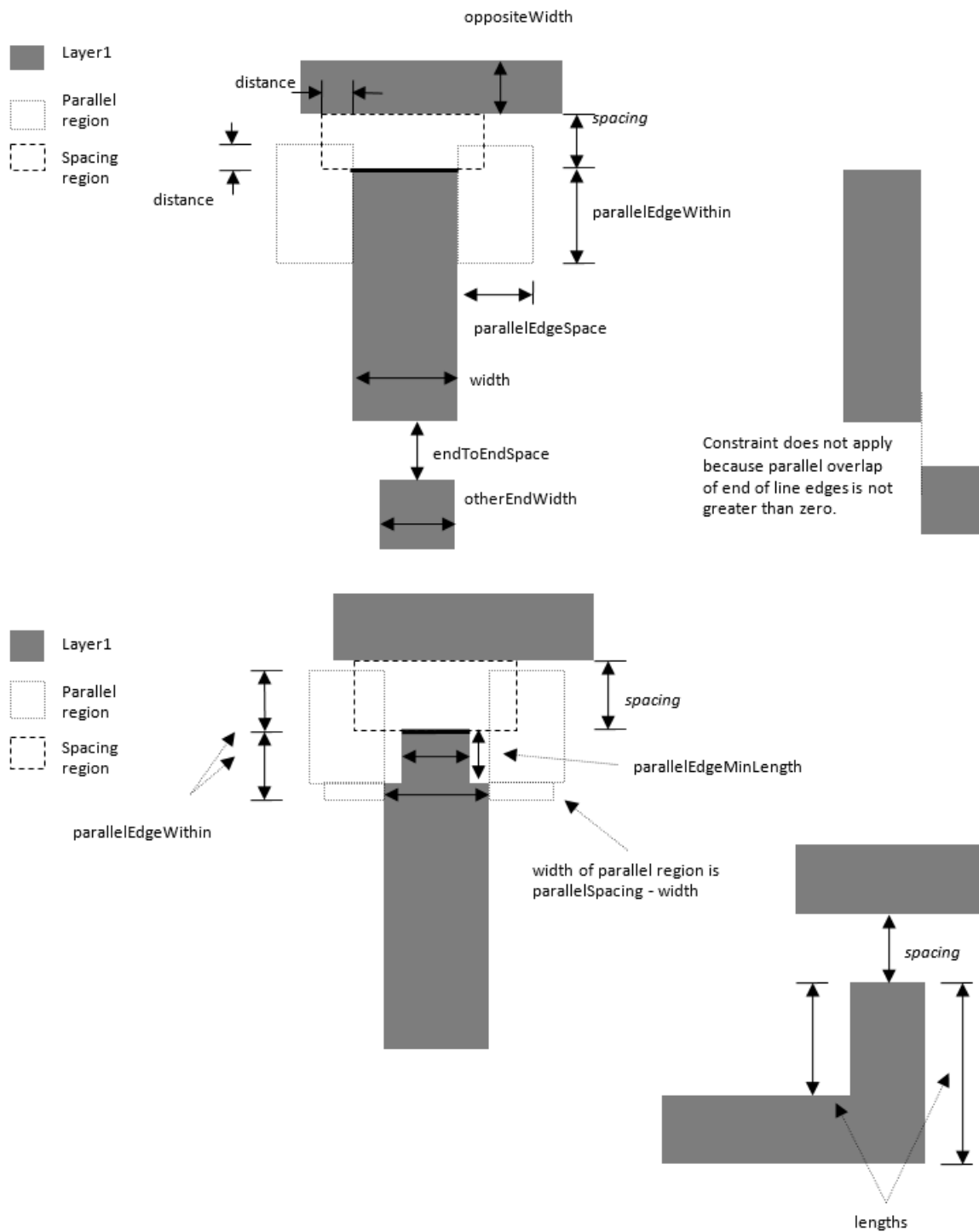
```
minEndOfLinePerpSpacing m1 0.10
    -width 0.08
    -endOfLinePerpWidth 0.10
```

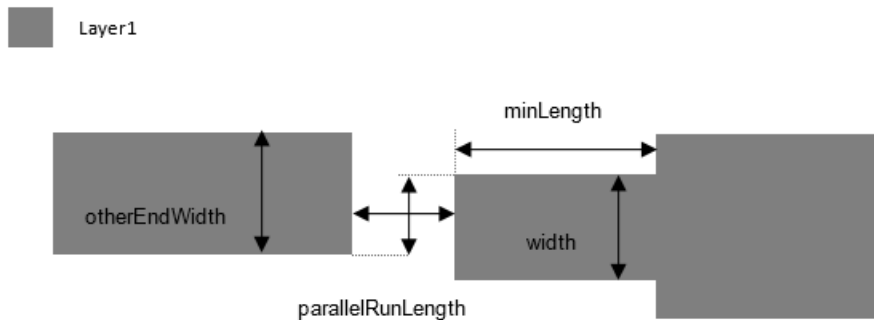
minEndOfLineSpacing

This layer constraint specifies the minimum spacing between the end of a wire and the closest geometry. The value is specified as a float.

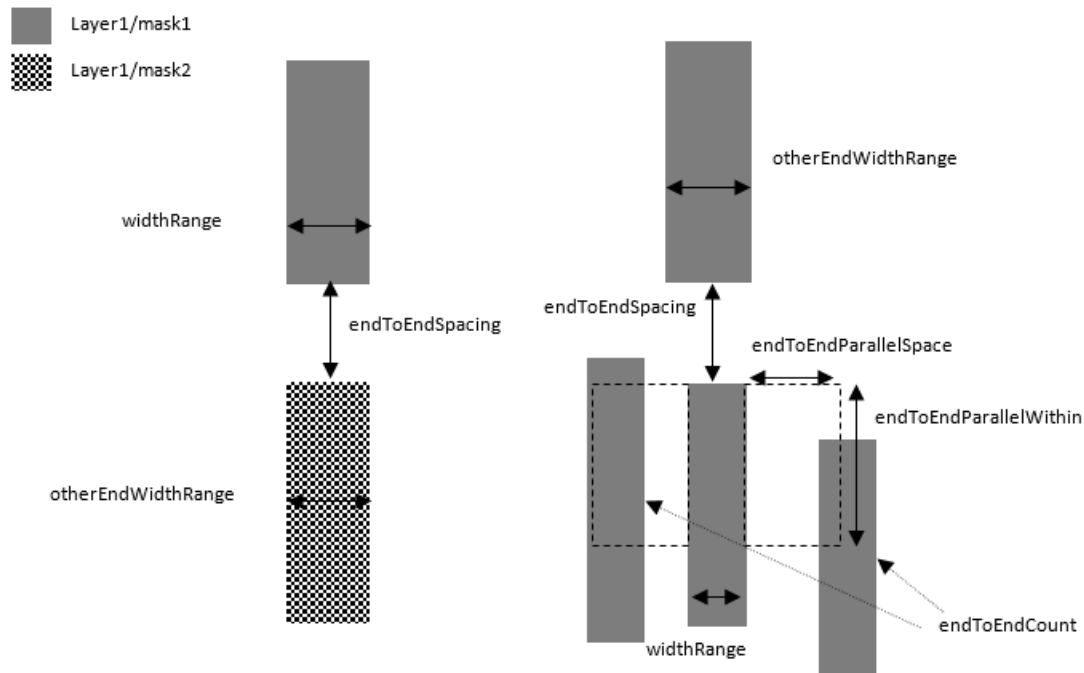
You can use various optional arguments to determine when the constraint applies. The following diagrams demonstrate the use of the optional arguments.



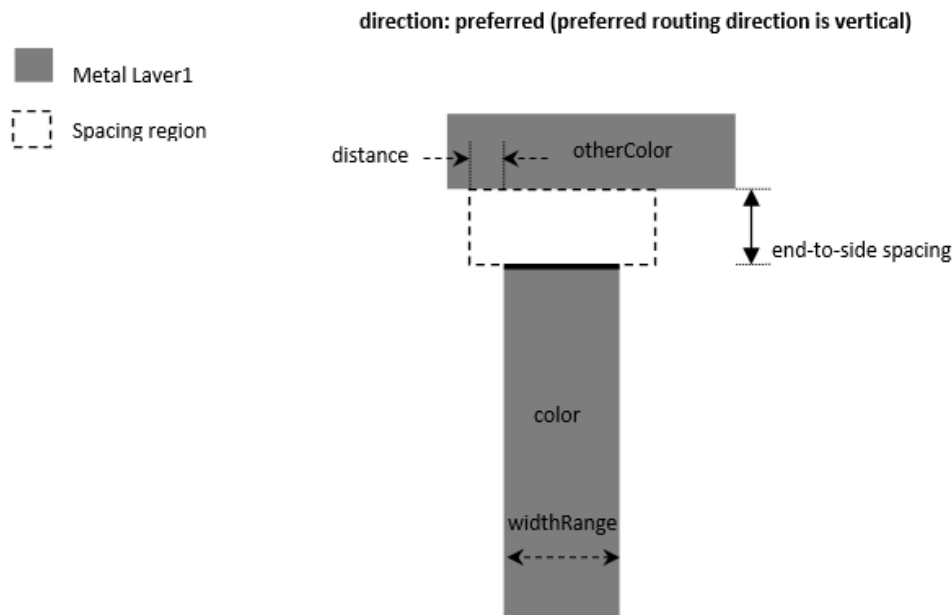




When two wire ends face each other, the following arguments can be optionally used to determine whether the constraint applies:



Some processes may have additional requirements for spacing between an end of line of one wire and a side of another wire depending on orientation of the wires as well as on particular mask colors of the wires. This rule is modeled by the built-in `minEndOfLineSpacing` constraint with the custom arguments: `-endToSideOnly`, `-color`, `-otherColor`, `-widthRange`, `-distance`, and `-direction`.



Constraint Category	Spacing
Constraint Type	Complex
OA Rule	oacMinEndOfLineSpacing

Syntax:

```
minEndOfLineSpacing
  <string|LPP>layer
  <float>spacingValue
  [-width float]
  [-widthRange rangeValue]
  [-distance float]
  [-parallelEdgeSpace float]
  [-parallelEdgeWithin float]
  [-count int]
  [-enclosedDistance float]
  [-cutToMetalSpacing float]
  [-enclosedCut all|below|above]
  [-allCuts false|true]
  [-minOppositeWidth float]
  [-endToEndSpacing float]
  [-endToEndParallelSpace float]
  [-endToEndParallelWithin float]
  [-endToEndCount int]
  [-otherEndWidth float]
  [-otherEndWidthRange range]
  [-minLength float]
```

```
[ -maxLength float]
[ -twoSides true|false]
[ -equalRectWidth true|false]
[ -subtractEndOfLineWidth true|false]
[ -parallelEdgeMinLength float]
[ -parallelRunLength float]
[ -onlyConcaveCorner true|false]
[ -cutEolSpace float]
[ -cutLayerEol cutEolAboveBelow|
               cutEolAbove|
               cutEolBelow]
[ -enclosureEndWidth float]
[ -enclosureEndWithin float]
[ -wrongDirWithin float]
[ -oneCutSpace float]
[ -twoCutSpace floatValue]
[ -rightDirExtension float]
[ -wrongDirExtension float]
[ -connectivityType anyConnectivityType|
                    sameNetConectivityType|
                    contiguousShapesConnectivityType|
                    directShapesConnectivityType|
                    sameViaConnectivityType]
[ -nonEolCornerOnly true|false]
[ -cutClass <cutClassArray | cutClassNameList>]
[ -colorType any|same|diff]
[ -direction any|preferred|nonpreferred]
[ -endToSideOnly true|false]
[ -color colorName|any|same|diff]
[ -otherColor colorName]

[ -isHard true|false]
[ -comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or a LPP.
spacingValue	Minimum spacing required between the end of a line and its neighboring shapes.
-width	Width of a wire end.
-widthRange	Range of widths for which this constraint applies. If you specify a range, -width is ignored. To specify the ranges, see Range Data Syntax .
-distance	Distance of line-end within distance from line-end edge in the perpendicular direction of line-end.
-parallelEdgeSpace	Spacing between two parallel lines.
-parallelEdgeWithin	Distance of two line-end on parallel lines.

Argument	Description
-count	Number of parallel edges within the region. You can specify a value in the range 0 to 2.
-enclosedDistance	If given, cut edge nearest to end of line must have a distance to the end of line that is less than this value for the constraint to apply.
-cutToMetalSpacing	If given, cut edge nearest to end of line must have a distance to edge opposite to end of line that is less than this value for the constraint to apply.
-enclosedCut	Enumerated type value which defines whether this constraint applies to cuts above, below, or all the metal layer. The default value is all.
-allCuts	Indicates whether all cuts must satisfy the rule. If false then only one cut must satisfy the rule. The default value is false.
-minOppositeWidth	Width of shape opposite to end of line edge must be greater than this value for the constraint to apply.
-endToEndSpacing	Distance between the two end-to-end wires, in microns.
-endToEndParallelSpace	Distance between the parallel wires, as shown in the diagram.
-endToEndParallelWithin	Length within which parallel wires must be within for the constraint to apply.
-endToEndCount	Maximum number of shapes that are adjacent to the end-to-end wires. If the number of adjacent shapes is greater than this value, the constraint value will be used instead of <code>-endToEndSpacing</code> .
-otherEndWidth	Width of the facing wire end. The width must be less than this value for the constraint to apply.
-otherEndWidthRange	Range of widths for which this constraint applies. If you specify a range, <code>-width</code> is ignored. To specify the ranges, see Range Data Syntax .
-minLength	Minimum length of the wire in microns for this constraint to apply. If edge lengths of both sides of the end of line wire is less than this value the constraint does not apply.
-maxLength	Maximum length of the wire in microns for this constraint to apply. If edge lengths of both sides of the end of line wire are greater than this value the constraint does not apply.
-twoSides	If this value is true, both edge lengths must be greater than or equal to the minimum length parameter for the constraint to apply. Default value is false.
-equalRectWidth	If true indicates that the constraint only applies if the end of line edge length is less than or equal to the segment length. Default value is false.

Argument	Description
-subtractEndOfLineWidth	If given, the dimension perpendicular to the metal edge of the parallel region is the parallel spacing value minus the width of the metal shape. Default value is false.
-parallelEdgeMinLength	Minimum end of line length for parallel edge check.
-parallelRunLength	Minimum parallel run length required in microns. The parallel run length between the two end of line edges must be greater than or equal to this value for the constraint to apply.
-onlyConcaveCorner	If true, the short adjacent edge adjacent to the end of line edge must originate from a concave corner and be less than maxLength parameter to apply. The parameter is ignored if maxLength parameter is not specified. Default value is false.
-cutEolSpace	Spacing required if the metal end-of-line edge under consideration intersects a cut shape on the cut layer either above or below the metal layer. This is spacing to another non-end-of-line edge. Default is 0.
-cutLayerEol	Specifies that the cutEolSpace is only applied if the metal end-of-line touches a cut shape in the cut layer above, below, or both. The default is both, which should be assumed if the constraint parameter is not present.
-enclosureEndWidth	Specify when a cutEolSpace parameter is specified. The EOL spacing rule applies to an EOL edge touching a via cut if the edge length is less than the enclosureEndWidth. The default is 1.
-enclosureEndWithin	Lateral verification distance for ends of lines touching via cuts applied in end-of-line to non-end-of-line spacing. See wrongDirExtension below for lateral verification distance used in end-to-end spacing checks. If enclosureEndWithin is specified, enclosureEndWidth must also be specified.
-wrongDirWithin	Lateral checking distance used for wires running in the non-preferred direction is different from that used with preferred direction wires. If this parameter is specified, the existing oaDistanceConstraintParamType parameter only specifies the preferred direction lateral verification distance.
-oneCutSpace	Spacing required between an end-of-line which touches a single cut, and one that does not. The endToEndSpacing parameter must be specified if oneCutSpace is specified.
-twoCutSpace	Spacing required between two ends of lines, both of which touch a via cut. The endToEndSpacing parameter must be specified if twoCutSpace is specified.
-rightDirExtension	Lateral spacing distance used to check end-of-line spacing between two ends of lines routed in the preferred direction.
-wrongDirExtension	Lateral spacing distance used to check end-of-line spacing between two ends of lines routed in the non-preferred direction.

Argument	Description
-connectivityType	When set to <code>contiguousShapesConnectivityType</code>, the rule only applies when the neighbors to the side(s) and beyond the EOL edge are connected as same-metal. • anyConnectivityType: Use this option to designate that the constraint applies to any type of connectivity (default). • sameNetConnectivityType: Use this option to designate that the constraint only applies to cuts on the same net. • contiguousShapesConnectivityType: Use this option to designate that the constraint only applies to cuts on the same metal. • directShapesConnectivityType: Use this option to designate that the constraint only applies when the two cuts are part of the same metal shape on the upper or lower metal and the metal shape covers the region between the two cuts. • sameViaConnectivityType: This is similar to <code>directShapesConnectivityType</code> except that both lower and upper metal shapes must cover the cuts and the region between the cuts.
-nonEolCornerOnly	Parallel edge neighbor wire must contain a corner, which does not belong to another EOL edge. The default is <i>false</i> .
-cutClass	List of paired float values or a list of cutClass names. See cutClass and cutClassList . The constraint applies only in cases where the cuts comply with these specifications.
-colorType	Color of the end-to-end wires for which the constraint applies. Valid values are 0 (any), 1 (same), and 2 (different). Default: 0
-direction	Direction of the end-to-end wires for which the constraint applies. Valid values are 0 (any), 1 (preferred direction), or 2 (non-preferred). Default: 0
-endToSideOnly	If true, the constraint only applies between the end-of-line and the other wire side. Default: false
-color	If a positive value is given, defines the specific mask color of the end-of-line wire to which the constraint applies. A value of -1 (same) means the rule only applies if the mask color of the other shape is the same as the mask color of the end-of-line wire. A value of -2 (diff) means the rule only applies if the mask color of the other shape is different from the mask color of the end-of-line wire. A value of zero means any color for both shapes. This parameter cannot be given if the <code>-colorType</code> parameter is also specified. Default: any

Argument	Description
-otherColor	This is an optional parameter defining the specific mask color of the other shape in the keepout region. Must be a positive number. Can be specified only if the color parameter is also a positive number, otherwise, this parameter is ignored. If the color parameter is a positive number, and this parameter is omitted, its value is assumed to be equal to the value of the color parameter.

Note:

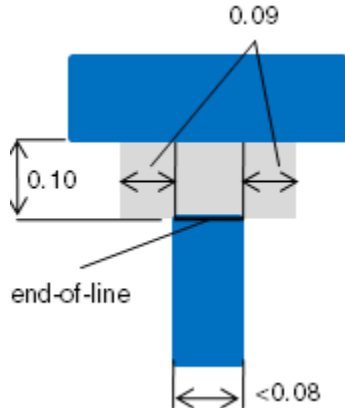
If `cutClass` parameter is specified with `cutClassName` in ASCII file, technology importer looks-up the first `cutClass` constraint specified in the same hierarchical constraint group to convert to cut class size. If a `cutClass` constraint does not exist then an error message is displayed.

The warning message is displayed in the following situations:

- `-widthRange` and `-width` are both specified.
- `-otherEndWidthRange` and `-otherEndWidth` are both specified.
- `-twoSides` is set to true and `-minLength` is not specified.
- `-onlyConcaveCorner` is set to true and `-maxLength` is not specified.
- `-minLength` and `-maxLength` are specified and `-maxLength` is smaller than or equal to `-minLength`.
- If the `-otherColor` parameter is a positive number.
- `-otherColor` is specified and if the `-color` parameter is not a positive number.
- `-color` and `-colorType` are both specified.
- `-otherColor` is specified if the `-color` parameter is not specified.

In the following example, the end of line spacing must be 0.100 if the wire width is less than 0.08:

Rule	Description	Value (in microns)
M1.S.7	EOL spacing when width<0.08 and spacing in perpendicular direction is >=0.09	>= 0.1



```
minEndOfLineSpacing METAL1 0.10 -width 0.08 -distance 0.09
```

In the following example, the `minEndOfLineSpacing` constraint is applied to the poly and drawing layers when the distance between the facing shapes is 0.06.

```
minEndOfLineSpacing (poly drawing) 0.06
```

In the following example, the `minEndOfLineSpacing` constraint is applied to the METAL1 layer when the distance between the facing shapes is 0.10, the wire width is 0.08, and the distance is 0.09.

```
minEndOfLineSpacing METAL1 0.10 -width 0.08 -distance 0.09
```

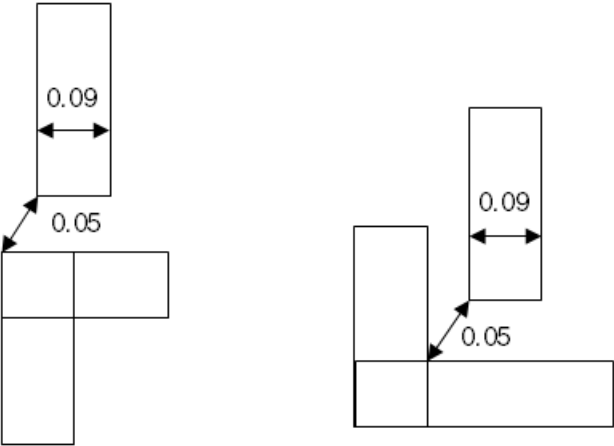
In the following example, the `minEndOfLineSpacing` constraint is applied to the m1 layer when the distance between end-to-end wires is 0.06, one of the wires has a width between 0.1 and 0.2 (inclusive), the wire width is between 0.5 and 0.7 (inclusive), the `endToEndParallelSpace` is 0.10, the `endToEndParallelWithin` is 0.08, there are three adjacent shapes, and the two end-to-end wires have the same color.

```
minEndOfLineSpacing m1 0.06 -widthRange (>=0.1 <=0.2) -distance 0.10
                        -otherEndWidthRange (>=0.5 <=0.7) -endToEndParallelSpace
0.10 -endToEndParallelWithin 0.08 -endToEndCount 3 -colorType same
```

minEndOfLineToConcaveCornerSpacing

Some processes require specific spacing between the end of any line with width less than or equal to `eolWidth` and a concave corner.

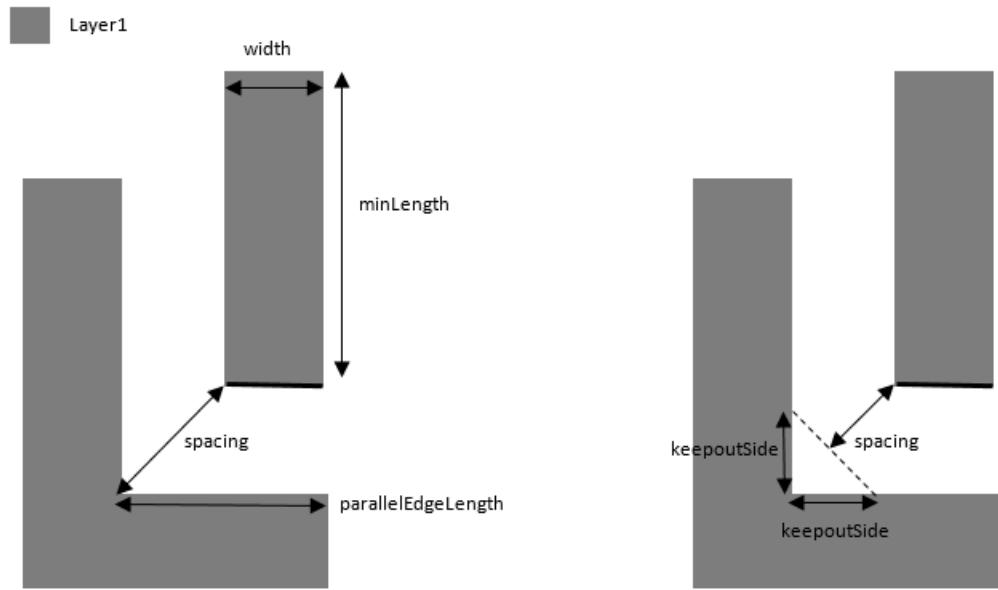
minEndOfLineToConcaveCornerSpacing
with value = 0.052 and width = 0.1



Ok, the spacing applies to concave corner only

Violation, the spacing to the concave corner is 0.05 (≤ 0.052)

As the following diagram shows, the spacing can be measured using one of two methods: convex corner to concave corner, or convex corner to the hypotenuse of the triangle at the concave corner.



Constraint Category	Spacing
Constraint Type	Complex

OA Rule **oacMinEndOfLineToConcaveCornerSpacing**

Syntax:

```
minEndOfLineToConcaveCornerSpacing
    <string|LPP>layer
    <float>spacingValue
    -width float
    [-minLength float]
    [-parallelEdgeLength float]
    [-keepoutSide float]
    [-isHard true|false]
    [-comment [<string rule_id>] <string rule_desc>]
```

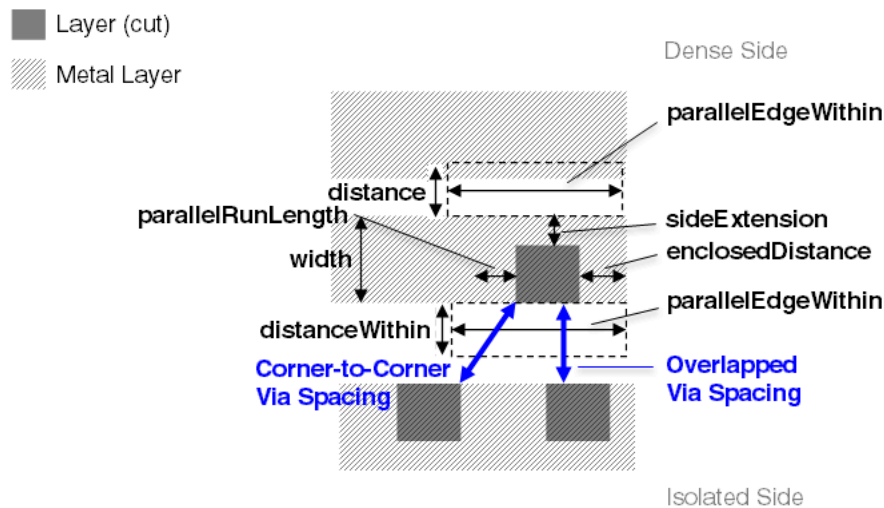
Argument	Description
layer	Layer can be defined as a string or a LPP.
spacingValue	Specifies the required spacing between an end of line and a convex corner.
-width	End of line width (eolWidth). The rule does not apply if the shape is wider than the width.
-minLength	Specifies the spacing rule from a qualifying end of line to a neighboring concave corner. This constraint applies only when the end of line length is $\geq$ minLength along both sides.
-parallelEdgeLength	Minimum length of the edge that is parallel to the wire end.
-keepoutSide	Length of the sides of the triangular keepout region.

For example:

```
minEndOfLineToConcaveCornerSpacing M1 0.052 -width 0.1 -minLength 0.15
    -parallelEdgeLength 0.55 -keepoutSide 0.75
```

minEndOfLineViaNeighborSpacing

The minEndOfLineViaNeighborSpacing constraint requires a minimum distance between the via at the end of a wire and neighboring vias. Depending on the positions, the neighboring vias have either orthogonal (overlapped) or corner-to-corner spacing requirements.



Constraint Category	Spacing
Constraint Type	Complex
OA Rule	Custom Compiler Custom Layer Constraint

Syntax:

```
minEndOfLineViaNeighborSpacing
  <string|LPP>layer
  <dualValue | arrayValue>spacing
  -width float
  -parallelRunLength float
  -parallelEdgeWithin float
  -distance float
  -distanceWithin float
  -enclosedDistance float
  -sideExtension float
  [-cutClass <cutClassArray | cutClassNameList>]
  [-aboveOnly true | false]
  [-cutClassList1 cutClassArray |
    ClassNameList]
  [-cutClassList2 cutClassArray |
    ClassNameList]
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Cut layer. <code>layer</code> can be defined as a string or a LPP.

Argument	Description
spacing	Minimum distance between the via at the end of the wire and the neighboring vias. Specify two values in the order shown: overlapped via spacing and corner-to-corner spacing. See parallelRunLength .
-width	Width threshold, in microns, of the end of the wire that has a via. The wire width must be less than the specified value.
-parallelRunLength	Length, in microns, that is parallel to the wire and measures from the inner side of the cut shape. If the parallel run length is greater than or equal to the specified value, corner-to-corner via spacing is required. Otherwise, overlapped via spacing is required.
-parallelEdgeWithin	Length, in microns, of the region that is parallel to the wire.
-distance	Width of the region that is parallel to the wire, on the dense side.
-distanceWithin	Width of the region that is parallel to the wire, on the isolated side.
-enclosedDistance	Extension threshold of the end of the wire beyond the via. The wire extension must be less than the specified value.
-sideExtension	Extension of the width of the wire beyond the via.
-cutClass	List of paired float values or a list of cutClass names. See cutClass and cutClassList . The constraint applies only in cases where the cuts comply with these specifications.
-aboveOnly	Consider only the shapes on the metal layer above the cut layer (true), or consider the metal shapes on layers above and below the cut layer (false). Default: true
-cutClassList1 -cutClassList2	List of cut sizes that define the rows (cutClass1) and columns (cutClass2) in the 2DTable when <code>spacing</code> is an array. See Understanding and Interpreting 1-D, 2-D, and 3-D Table Values .

Example when `spacing` consists of dual values with `cutClass`:

```
minEndOfLineViaNeighborSpacing v1 0.25 0.35
    -width 0.2
    -cutClass ((0.02 0.04) (0.6 0.8))
    -parallelRunLength 0.05
    -aboveOnly false
    -parallelEdgeWithin 0.25
    -distance 0.28
    -distanceWithin 0.23
    -enclosedDistance 0.27
    -sideExtension 0.45
```

Example when `spacing` is an array with `cutClassList1` and `cutClassList2`:

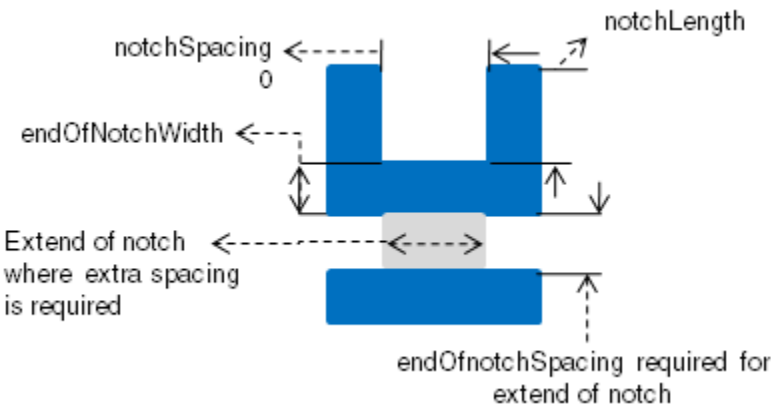
```
minEndOfLineViaNeighborSpacing v1
    ( -overlap (
        ( ( (0.02 0.04) (0.03 0.05) ) 0.1)
```

```
( ( (0.02 0.04) (0.07 0.09) ) 0.2)
( ( (0.06 0.08) (0.03 0.05) ) 0.3)
( ( (0.06 0.08) (0.07 0.09) ) 0.4) )
-corner (
  ( ( (0.02 0.04) (0.03 0.05) ) 0.5)
  ( ( (0.02 0.04) (0.07 0.09) ) 0.6)
  ( ( (0.06 0.08) (0.03 0.05) ) 0.7)
  ( ( (0.06 0.08) (0.07 0.09) ) 0.8) ) )
-width 0.2
-cutClassList1 ((0.02 0.04) (0.06 0.08))
-cutClassList2 ((0.03 0.05) (0.07 0.09))
-parallRunLength 0.05
-aboveOnly false
-parallelEdgeWithin 0.25
-distance 0.28
-distanceWithin 0.23
-enclosedDistance 0.27
-sideExtension 0.45
```

minEndOfNotchSpacing

This layer constraint specifies the minimum spacing between the notch and shapes overlapping within the extent of the notch.

The minimum spacing is required between the notch and shapes overlapping within the extent of the notch if the notch width is less than endOfNotchWidth, the notch spacing is less than or equal to minNotchSpacing, and the notch length is greater than or equal to minNotchLength.



Constraint Category	Spacing
Constraint Type	Complex
OA Rule	oacMinEndOfNotchSpacing

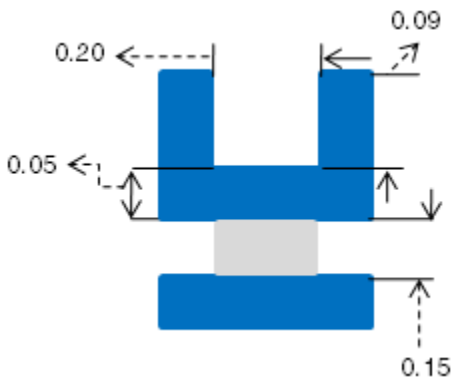
Syntax:

```
minEndOfNotchSpacing
  <string|LPP>layer
  <float>minSpacing
  -notchWidth <float>
  -notchLength <float>
  -notchSpacing <float>
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or a LPP.
minSpacing	Specifies the minimum spacing between the notch and shapes overlapping within the extent of the notch.
-notchWidth	Specifies the width of the notch.
-notchLength	Specifies the length of the notch.
-notchSpacing	Specifies the spacing of the notch.

In this example, the end of notch spacing must be 0.15 if a notch length is 0.09 or longer:

Rule	Description	Value (in microns)
M1.S.4	End of notch spacing when length >= 0.09, width <= 0.05, and the notch spacing is <= 0.2.	>= 0.150



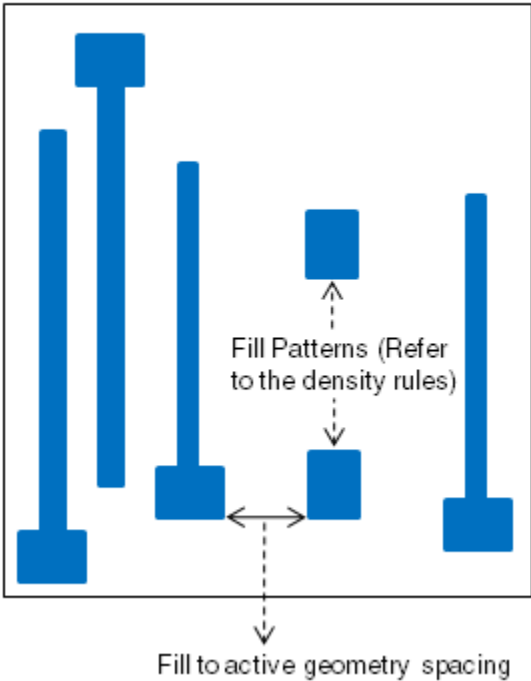
```
minEndOfNotchSpacing M1 0.15 -notchLength 0.09 -notchWidth 0.05 -notchSpacing
0.20
```

minFillPatternSpacing

This layer constraint specifies the minimum distance required between metal filler geometries and active geometries.

This constraint is not a hard constraint since it could conflict with the [minDensity](#) constraint.

Additional geometry is added to the areas to increase the density.



Constraint Category	Spacing
Constraint Type	Simple
OA Rule	oacMinFillPatternSpacing

Syntax:

```
minFillPatternSpacing
  <string|LPP>layer
  <float>spacingValue
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

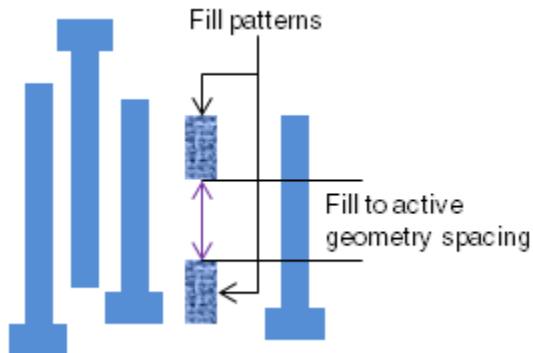
Argument	Description
layer	Layer can be defined as a string or as LPP.
spacingValue	Specifies the minimum distance required between metal filler geometries and active geometries.

For example:

```
minFillPatternSpacing M8 2
```

minFillToFillSpacing

This constraint defines the minimum spacing between fill pattern shapes on the same layer.



Constraint Category	Spacing
Constraint Type	Simple
OA Rule	oacMinFillToFillSpacing

Syntax:

```
minFillToFillSpacing
    <string|LPP>layer
    <float>minSpacing
    [-isHard true|false]
    [-comment [<string rule_id>] <string rule_desc>]
```

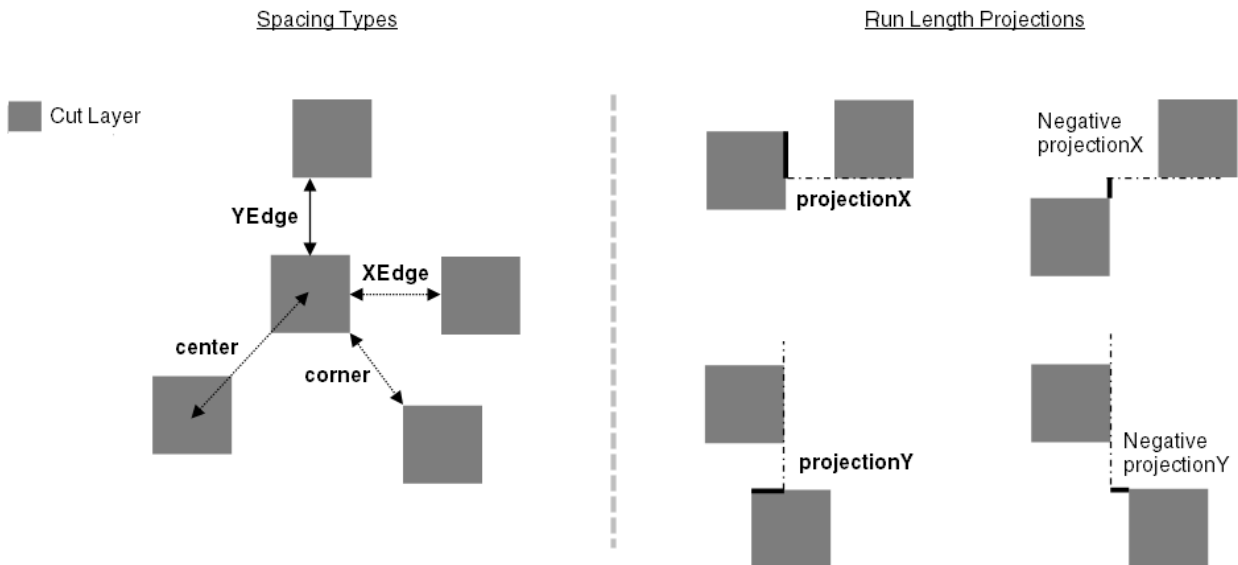
Argument	Description
layer	Layer can be defined as a string or as LPP.
minSpacing	The minimum spacing value between fill shapes.

For example:

```
minFillToFillSpacing m1 0.2
```

minGeneralViaSpacing

The `minGeneralViaSpacing` constraint can be used to address various spacing requirements between cuts of differing sizes. Spacing requirements can depend on the cut size, cut orientation, spacing direction, parallel run length (or projection), and connectivity (same or different net).



The `minGeneralViaSpacing` constraint value is a 2D table that specifies four types of spacing in its quadrants: **XEdge**, **YEdge**, **centerline** (center to center), and **corner** (corner to corner). The following table shows how each spacing type is stored in the spacing table based on the cut class index in the `cutClassList` parameter value.

	1	...	N	1	...	N
1	centerline(1,1)	centerline(1,K)	centerline(1,N)	XEdge(1,1)	XEdge(1,K)	XEdge(1,N)
...	centerline(J,1)	centerline(J,K)	centerline(J,N)	XEdge(J,1)	XEdge(J,K)	XEdge(J,N)
N	centerline(N,1)	centerline(N,K)	centerline(N,N)	XEdge(N,1)	XEdge(N,K)	XEdge(N,N)
1	YEdge(1,1)	YEdge(1,K)	YEdge(1,N)	corner(1,1)	corner(1,K)	corner(1,N)
...	YEdge(J,1)	YEdge(J,K)	YEdge(J,N)	corner(J,1)	corner(J,K)	corner(J,N)
N	YEdge(N,1)	YEdge(N,K)	YEdge(N,N)	corner(N,1)	corner(N,K)	corner(N,N)

Constraint Category	Via Construction
Constraint Type	Complex
OA Rule	Custom Compiler Custom Layer Constraint

Syntax:

```
minGeneralViaSpacing
    <string|LPP>layer
    <2DTable>spacingValue
    -cutClassList cutClassArray|cutClassNameList
    [-connectivityType
        anyConnectivityType|
        sameNetConnectivityType|
        contiguousShapesConnectivityType|
        directShapesConnectivityType|
        sameViaConnectivityType]
    [-projectionX 2DTable]
    [-projectionY 2DTable]
    [-noZeroProjection true|false]
    [-includeOrthoCheck 2DTable]
    [-isHard true|false]
    [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer on which the cuts are places. <code>layer</code> can be a string (layer name) or LPP.
spacingValue	Two-dimensional table of spacing values, where the rows and columns are cut class indexes. The table consists of quadrants (four NxN arrays), where N is the number of elements in the <code>cutClassList</code> value array. Each quadrant (array) in the table contains the spacing values for the following spacing types: • centerline: Uses via centerlines to determine the required distance between vias. • XEdge: Determines the distance between vias along the X direction. • YEdge: Determines the distance between vias along the Y direction. • corner: Determines the corner-to-corner distance between vias.

Argument	Description
-cutClassList	An ordered array of cut dimension pairs. The array is used to determine the key to index into the constraint 2-D table value. The <code>cutClassList</code> is represented as: <pre>cutClassArray<float1> <float2> cutClassName<string></pre> Note: If <code>cutClass</code> parameter is specified with <code>cutClassName</code> in the ASCII file, the technology importer looks for the first <code>cutClass</code> constraint specified in the same hierarchical constraint group to convert to cut class size. If no <code>cutClass</code> constraint exists, an error message is displayed. While exporting, the technology exporter looks for the first <code>cutClass</code> constraint specified in the same hierarchical constraint group to exists and converts the cut class size to cut class name. If no <code>cutClass</code> constraint exists, the <code>cutClass</code> parameter is exported with cut class size in dual value.
-connectivityType	Indicator that the constraint applies when the cuts have one of the following connectivity types: • anyConnectivityType: (Default) The cuts can have any type of connectivity. • sameNetConnectivityType: The cuts must be on the same net. • contiguousShapesConnectivityType: The cuts must be on the same metal. • directShapesConnectivityType: The two cuts are part of the same metal shape on the upper or lower metal and the metal shape covers the region between the two cuts. • sameViaConnectivityType: Similar to <code>directShapesConnectivityType</code>, except both the lower and upper metal shapes must cover the cuts and the region between the cuts.
-projectonX	Parallel run length requirements for each cut combination. If the value is not specified, a projection ≥ 0 is required for YEdge spacing. If <code>noZeroProjection</code> is true, then only projection > 0 is allowed.
-projectonY	Projection in the Y direction for parallel run lengths along the X direction for each cut combination. If the value is not specified, a projection ≥ 0 is required for XEdge spacing. If <code>noZeroProjection</code> is true, then no projection > 0 is allowed.
-noZeroProjection	Indicator of whether an exact 0 projection is included or ignored. Default: false
-includeOrthoCheck	Indicator of whether XEdge and YEdge Manhattan spacing checks are performed, regardless of whether the cut meets the projection requirement. When false, only corner spacing is checked when the required projection is not met. If the table is not given, then only corner checks are performed.

Example:

```
minGeneralViaSpacing v1
( ((0 0) 0.10 -spacingType centerline
  ((0 1) 0.12 -spacingType centerline)
```

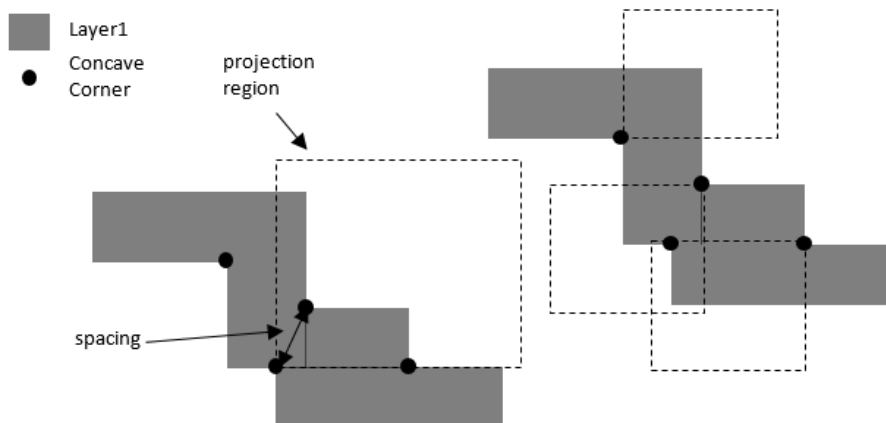
```

((1 0) 0.12 -spacingType centerline)
((1 1) 0.15 -spacingType centerline)
((0 0) 0.02 -spacingType XEdge)
((0 1) 0.05 -spacingType XEdge)
((1 0) 0.05 -spacingType XEdge)
((1 1) 0.08 -spacingType XEdge)
((0 0) 0.02 -spacingType YEdge)
((0 1) 0.05 -spacingType YEdge)
((1 0) 0.05 -spacingType YEdge)
((1 1) 0.08 -spacingType YEdge)
((0 0) 0.04 -spacingType corner)
((0 1) 0.05 -spacingType corner)
((1 0) 0.05 -spacingType corner)
((1 1) 0.06 -spacingType corner) )
-cutClassList ( (0.06 0.06) (0.06 0.08) )
-connectivityType anyConnectivityType
-projectionX ( ((0 0) -0.01)
               ((0 1) -0.02)
               ((1 0) -0.02)
               ((1 1) -0.03) )
-projectionY ( ((0 0) -0.01)
               ((0 1) -0.02)
               ((1 0) -0.02)
               ((1 1) -0.03) )
-noZeroProjection false
-includeOrthoCheck ( ((0 0) false)
                     ((0 1) false)
                     ((1 0) false)
                     ((1 1) false) )

```

minInnerConcaveCornerDistance

This constraint specifies the minimum spacing between two concave corners within the same shape on the same layer. The concave corners must be within a projection region. The projection region is defined by a rectangle whose corner begins at one of the concave corners, as shown in the following diagram.



Constraint Category

Spacing

Constraint Type	Complex
OA Rule	Custom Compiler custom layer constraint

Syntax:

```
minInnerConcaveCornerDistance
    <string|LPP>layer
    <float>spacing
    [-nonPreferredWidth <float>]
    [-distanceMeasureType euclidean|manhattan]
    [-isHard true|false]
    [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or as LPP.
spacing	Distance between two concave corners within the same shape and are within the projection region.
-nonPreferredWidth	Minimum width, in microns, in the non-preferred direction. If specified, the constraint value is the minimum width in preferred direction and the measurement is Mahattan.
-distanceMeasureType	How the distance is measured. Default: euclidean

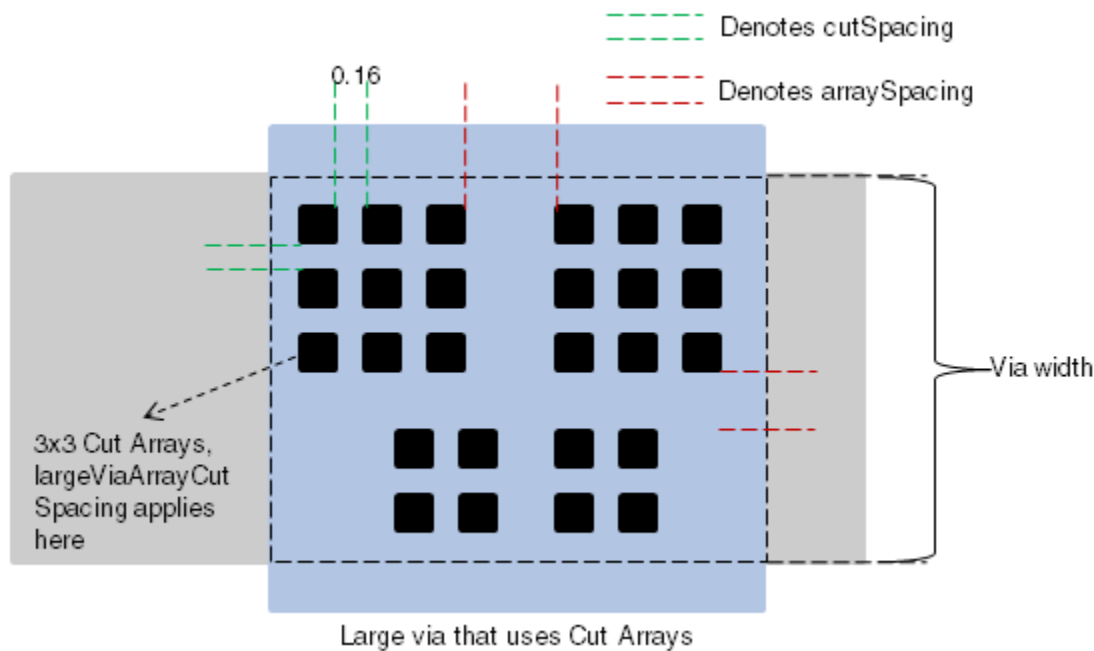
For example:

```
minInnerConcaveCornerDistance m1 0.05
```

minLargeViaArrayCutSpacing

This layer constraint specifies the minimum spacing between via cuts in a large cut array, which is independent of the spacing between cuts that are not in a large array.

This constraint is associated with a cut layer, and applies within via arrays that have more than a specified number of cuts.



Constraint Category	Via Construction
Constraint Type	Complex
OA Rule	oacMinLargeViaArrayCutSpacing

Syntax:

```
minLargeViaArrayCutSpacing
  <string|LPP>layer
  <float>spacingValue
  <-minNumCuts int>
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

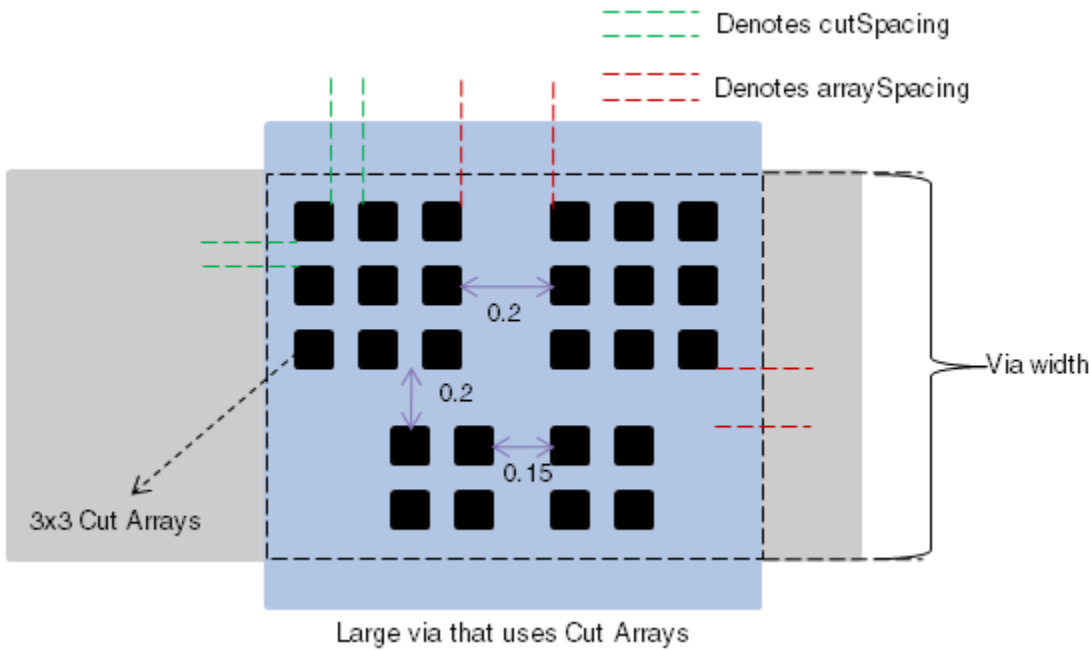
Argument	Description
layer	Layer can be defined as a string or a LPP.
spacingValue	Specifies the minimum cut spacing between via cut.
-minNumCuts	Specifies the number of cuts in the largest dimension of two adjacent cut arrays.

In this example, the minimum cut spacing required in the Via arrays is mentioned:

```
minLargeViaArrayCutSpacing VIA1 0.16 -minNumCuts 3
```

minLargeViaArraySpacing

This constraint defines the number of cuts (numCuts) allowed in a cut array, as well as the minimum spacing required between cut arrays (arraySpacing). The constraint only applies to the spacing between cut-arrays. It does not apply to the spacing between the cuts within an array (cutSpacing).



Constraint Category	Via Construction
Constraint Type	Complex
OA Rule	oacMinLargeViaArraySpacing

Syntax:

```
minLargeViaArraySpacing
  <string|LPP>layer
  <float1DTable>spacingValue
  [-cutClass <cutClassArray | cutClassNameList>]
  [-parallelOverlap true|false]
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or a LPP.

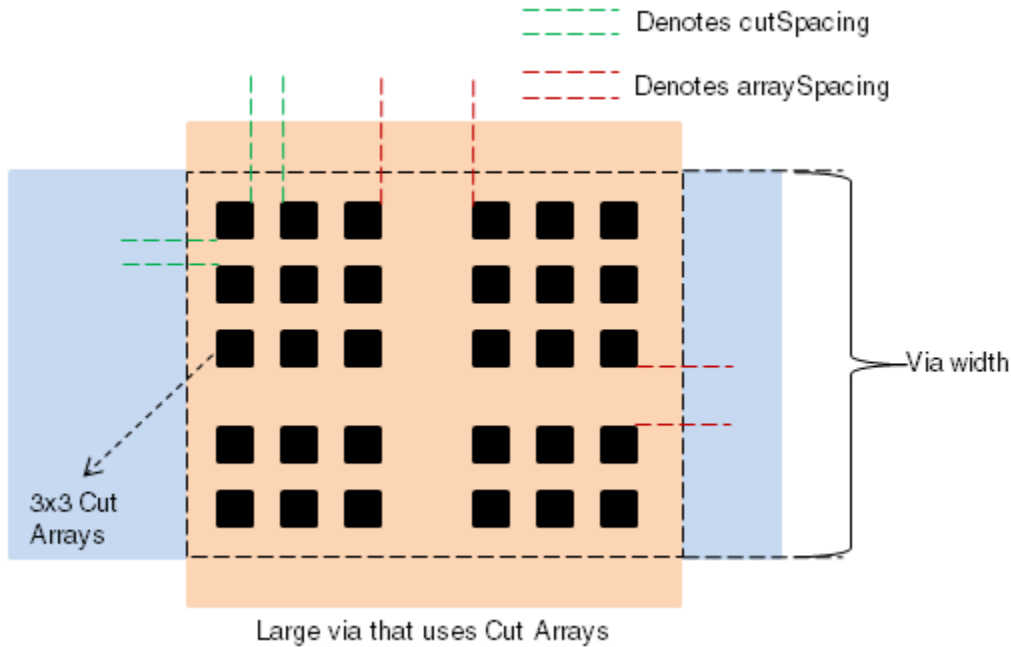
Argument	Description
spacingValue	The spacingValue is represented as: default_value ((numCut minViaArraySpacing)*) • Row key: The number of cuts in the largest dimension of two adjacent cut arrays • Value: The minimum spacing between cut-arrays.
-cutClass	List of paired float values or a list of cutClass names. See cutClass and cutClassList . The constraint applies only in cases where the cuts comply with these specifications.
-parallelOverlap	If set to true, this parameter specifies that the rule applies only if the cut arrays have non-zero parallel overlap. Default is <i>false</i> .

For example:

```
minLargeViaArraySpacing v1 0.08 ((2 0.15) (3 0.2))
```

minLargeViaArrayWidth

This layer constraint specifies the conditions under which via arrays creation must use the spacing. When a via is wider than or equal to the via width specified by this constraint (based on the width of the intersection of the two metal wires that are connected by the cut layer) this constraint takes effect.



Constraint Category	Via Construction
---------------------	------------------

Constraint Type	Simple
OA Rule	oacMinLargeViaArrayWidth

Syntax:

```
minLargeViaArrayWidth
  <string|LPP>layer
  <float>spacingValue
  [-cutClass <cutClassArray | cutClassNameList>]
  [-parallelOverlap true|false]
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or as LPP.
spacingValue	Specifies the conditions under which via arrays creation must use the spacing.
-cutClass	List of paired float values or a list of cutClass names. See cutClass and cutClassList . The constraint applies only in cases where the cuts comply with these specifications.
-parallelOverlap	If set to true, this parameter specifies that the rule applies only if the cut arrays have non-zero parallel overlap. Default is false.

In this example, the minimum width for a Via array to be considered a large Via is defined:

```
minLargeViaArrayWidth VIA1 0.08 -cutClass 0.06 0.08
                                -parallelOverlap true
```

minLength

This layer constraint specifies the minimum orthogonal length for any geometry on the specified layer. Some processes may have different requirements for minimum length of metal layer shapes depending on the presence or absence of a via shape on adjacent cut layer. This is a built-in constraint definition that has additional parameters: `viaLayers`, `exceptViaLayers` and `maxWidth`.

Constraint Category	Length and Width Process Rules
Constraint Type	Simple
OA Rule	oacMinLength

Syntax:

```
minLength <string|LPP>layer
         <float>lengthValue
         [-exceptViaLayers (layerName*)]
         [-viaLayers (layerName*)]
         [-maxWidth floatValue]

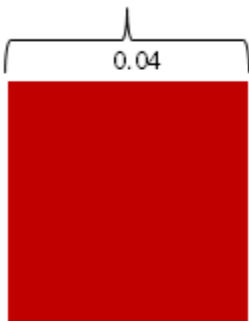
         [-isHard true|false]
         [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or as LPP.
lengthValue	Specifies the minimum orthogonal length for any geometry on the specified layer.
-exceptViaLayers	An array of oaLayer values, each defining a cut layer immediately above or below the metal layer. If given, the rule does not apply to the metal layer shape if there is a cut shape on any of these cut layers connecting the metal layer shape to the metal layer above or below.
-viaLayers	An array of oaLayerNum values, each defining a cut layer immediately above or below the metal layer. If given, the rule only applies to the metal layer shape if there is a cut shape on any of these cut layers connecting the metal layer shape to the metal layer above or below.
-maxWidth	If given, the rule does not apply to metal shapes with width greater than this value.

In this example, the minimum gate length is mentioned:

Rule	Description	Value (in microns)
PO.LE.1	minimum Gate Length	0.04

Geometry length >= minimum length



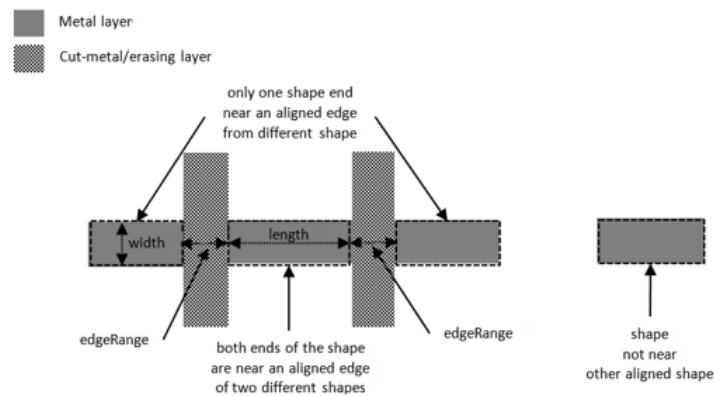
```
minLength GATE 0.04
```

Examples:

```
minLength m2 0.06 -exceptViaLayers (v1 v2) -maxWidth 0.88  
minLength m1 0.08 -viaLayers (v1 v2) -maxWidth 0.8
```

minLengthAlignedProximity

Some processes have different minimum length requirements for (metal) shapes that are cut by a shape on an erasing (cut-metal) layer. The cut-metal shapes may be already drawn in the layout or may be inserted into layout later. In the latter case, the presence of a cut shape can be deduced from the existing metal shapes located in proximity to each other. If line-ends of two metal shapes are fully aligned and distance between them is in a given range, then a cut shape can be inserted between them.



Constraint Category	Length and Width Process Rules
Constraint Type	Complex
OA Rule	oacMinLength

Syntax:

```
minLengthAlignedProximity <string|LPP>layer  
    <float>lengthValue  
    -edgeRange rangeData  
    [-widthRange rangeData]  
    [-maxWidth floatValue]  
    [-count <int>]  
    [-color colorName|any|same|diff]  
  
    [-isHard true|false]  
    [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or as LPP.
lengthValue	Specifies the minimum length in database units required for a shape on the metal layer when its line-ends touch one, two or none cut-metal shapes.
-edgeRange	Range that defines the distance required for the second shape edge to be considered in proximity.
-widthRange	If given, this constraint only applies to shapes with width in this range.
-count	The rule only applies to a shape which has this exact number of line-ends touching cut-metal shapes. Valid values are 0, 1 or 2. Default: 2.
-color	If given, the constraint only applies when all metal shapes under consideration have this mask color. A value of zero means any color. A value of -1 means same color (For example, all count+1 metal shapes have the same color). A value of -2 means diff color (For example, there are two metal shapes with different colors). Default: any.

Example:

```
minLengthAlignedProximity m2 0.06 -edgeRange (>=0.05 <0.10)
                                -widthRange (>=0.05 <0.10) -count 2
```

minMaskArea

This layer constraint specifies the minimum area required for different color masks on the same layer. The area is expressed in microns².

Constraint Category	Area
Constraint Type	Complex
OA Rule	Custom Compiler custom layer constraint

Syntax:

```
minMaskArea
    <string|LPP>layer
    <((string1 float) (string2 float)
    (string3 float))> colorName area
    [-isHard true|false]
    [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer of the mask. Layer can be defined as a string or as LPP.

Argument	Description
colorName area	Mask color and the area, in the following format: (colorName area). The color name must be defined in the Controls Section .

In the following example, the area of the red, blue, and green masks are defined for layer m1:

```
minMaskArea m1 ((red 0.25) (blue 0.64) (green 0.09))
```

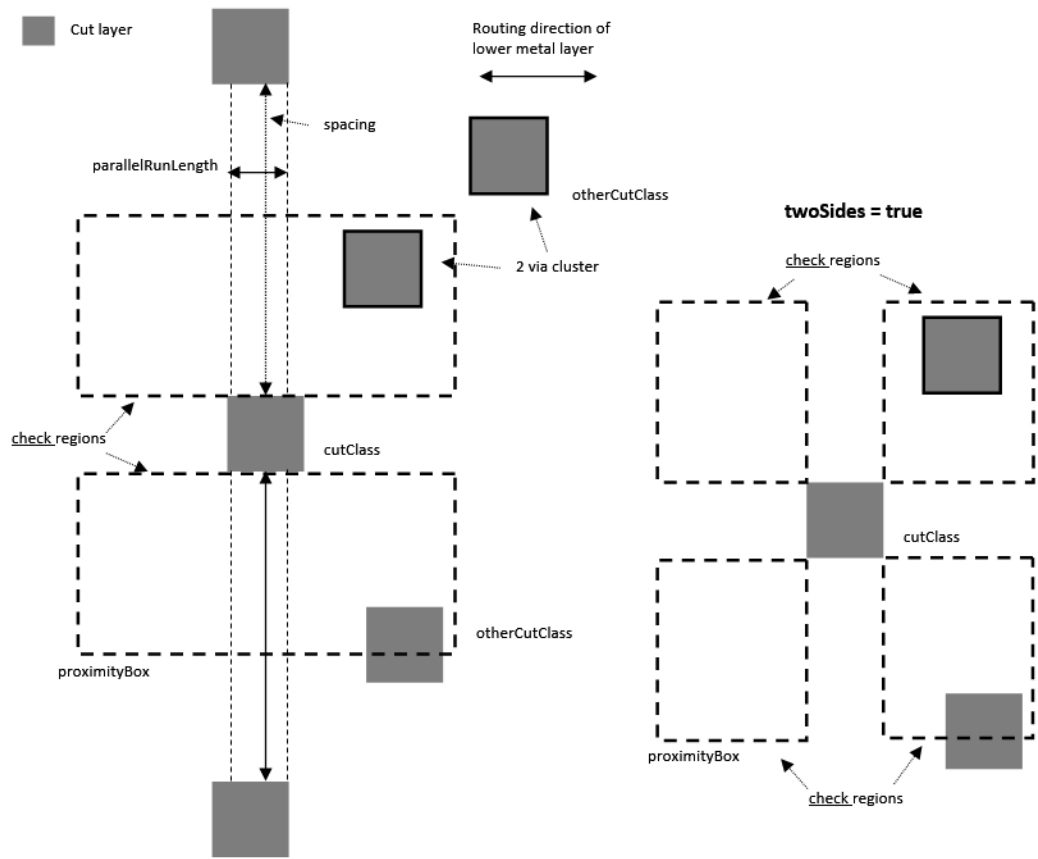
In the following example, the area of the red and blue masks are defined for layer m2:

```
minMaskArea m2 ((red 0.5) (blue 0.9))
```

minNeighborRegionParallelSpacing

The layer constraint specifies the minimum parallel spacing with adjacent vias.

Some advanced processes require a minimum spacing to a parallel overlap cut shape when there is a separate via or via cluster that interacts with a check region. The parallel spacing required is only for the side in which a cut or via cluster interacts with the check region.



Syntax:

```
minNeighborRegionParallelSpacing
  <string|LPP>layer
  <float>spacingValue
  -cutClass <<float> <float> | cutClassName>
  [-otherCutClass <<float> <float> | cutClassName>]
  -proximityBox cutClassArray
  -distanceWithinRange rangeData
  [-otherDistanceWithinRange rangeData]
  -spacingRange rangeData
  [-count intValue]
  [-twoSides true | false]
  [-parallelRunLength floatValue]
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

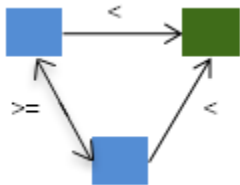
Argument	Description
layer	Layer can be defined as a string or a LPP.

Argument	Description
spacingValue	Specifies orthogonal spacing required between a cut shape and an arbitrary cut shape with a certain parallel run length when a specific cut shape is found in a check region.
-cutClass	Specifies the specific cut class used to form the via cluster. This is required.
-otherCutClass	If specified, the specific cut class is required to be spaced away from the via cluster.
-proximityBox	Definition of the check region dimensions (width and height). This is required.
-parallelRunLength	Specifies the parallel run length threshold of the orthogonal cut for the constraint to apply. The cut shape must have parallel overlap greater than this value.
-distanceWithinRange	Specifies the range defining one dimension of the search region. This is required. To specify the ranges, see Range Data Syntax .
-otherDistanceWithinRange	Specifies the range defining the other dimension of the search region. If not specified, the search region is defined by distanceWithinRange and is square. This is optional.
-spacingRange	Specifies the range in which the diagonal cut shapes must be within in order to apply as a cluster. This is required.
-count	Specifies the minimum number of cut shapes required to interact with check region for the constraint to apply. The default value is 1.
-twoSides	If true, then there are two (split) check regions per side. When split regions are specified, interaction with the check region can include abutment but not point touch. The default value is false.

```
minNeighborRegionParallelSpacing v1 0.02 -cutClass 0.2 0.6
    -otherCutClass 0.04 0.06 -proximityBox 0.06 0.12
    -parallelRunLength 0.05 -distanceWithinRange (>=0.12 <=0.25)
    -spacingRange (>=0.22 <=0.75)
```

minNeighborViaSpacing

The `minNeighborViaSpacing` constraint specifies the minimum spacing between two vias when both vias have a common neighbor. If both vias are less than a specified distance (`< distanceWithin`) from the common neighbor, the two vias have to be at least a certain distance (`>= viaSpacingValue`) apart.



Constraint Category	Via Construction
Constraint Type	Complex
OA Rule	oacMinNeighborViaSpacing

Syntax:

```
minNeighborViaSpacing
  <string|LPP>layer
  <float>viaSpacingValue
  <-distanceWithin float>
  [-centerToCenter true|false]
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer, defined as a string or as LPP
viaSpacingValue	The minimum distance between the two vias.
-distanceWithin	The constraint applies only if the common neighbor is less than (<) distanceWithin from both vias.
-centerToCenter	If this parameter is set to true, distances are measured from the via center. Otherwise, distances are measured from the via edge. Default: false

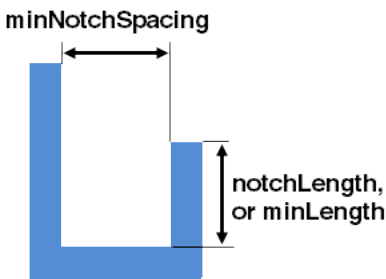
In the following example, when the common neighbor is less than 0.06 from each of the two vias, the two vias must be at least 0.08 apart, as measured from the center of the vias.

Rule	Description	Value (in microns)
Rule: V1.S.4	Spacing to neighboring via when distance < 0.06	0.08

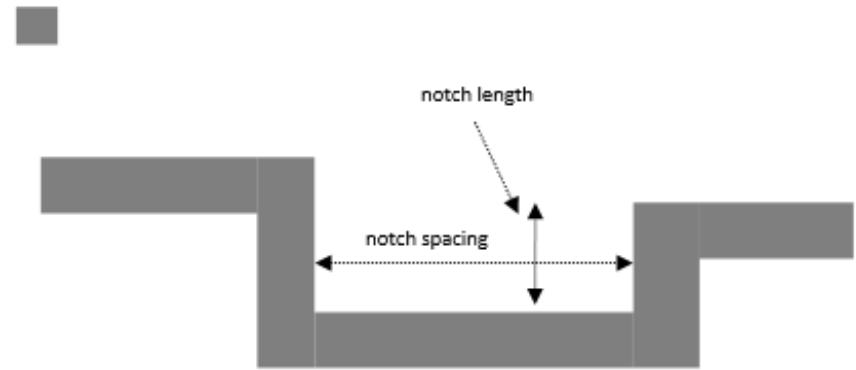
```
minNeighborViaSpacing v1 0.08 -distanceWithin 0.06 -centerToCenter true
```

minNotchSpacing

This layer constraint specifies the minimum spacing required in a notch.



Some processes have restrictions on the type of notch that can exist. Typically notches must be deeper than a specific threshold. However, newer processes have the opposite criteria in which the notch depth must be less than a given distance for the notch structure to be correct.



Constraint Category	Spacing
Constraint Type	Complex
OA Rule	oacMinNotchSpacing

Syntax:

```
minNotchSpacing
  <string|LPP>layer
  <float>spacingValue
  [-minLength float]
  [-notchLength float]
  [-notchWidth float|-notchWidthRange rangeValue]
  [-spacingDirection horizontal|vertical|any]
  [-color colorName]
```



```
[-isHard true|false]
[-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer name or layer-purpose pair.
spacingValue	Minimum spacing in the notch.
-notchLength	Maximum length of the notch for the constraint to apply. If the length is longer than the -notchLength value, the constraint does not apply. If you specify -notchLength, do not specify -minLength. Only one is required.
-minLength	Minimum length of the notch for the constraint to apply. If the length is less than the -minLength value, the constraint does not apply. If you specify -minLength, do not specify -notchLength. If both are specified, -minLength is ignored. Note: -minLength is a Custom Compiler custom argument.
-notchWidth	Optional parameter that specifies the minimum width of the shape at the notch in which the rule does not apply. The width of the shape at the notch must be less than this value for the rule to apply. This parameter cannot be given together with the -notchWidthRange parameter.
-notchWidthRange	If given, the width of the shape at the notch must lie in this range for the rule to apply. The range thresholds are in database units. This parameter cannot be given together with the -oaNotchWidth parameter.
-spacingDirection	Defines the spacing direction in which this constraint applies. 1: Only horizontal 2: Only vertical 0: Any direction. Default: any
-color	If a positive value is given, the rule only applies to shapes of this mask color. A value of zero means any color. Default: any

```
minNotchSpacing M1 0.20 -notchLength 0.09
```

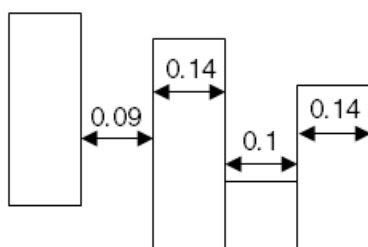
In the examples, specifies on LPP m1/drawing with parameter value.

```
minNotchSpacing (m1 drawing) 0.06 -notchLength 0.12
minNotchSpacing (m1 drawing) 0.06 -minLength 0.14
    -notchWidthRange (0.1 <=0.5)
    -spacingDirection horizontal
    -color green
```

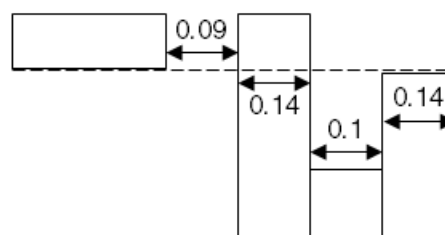
minNotchSpanSpacing

In some advanced processes, the spacing between legs of a notch can depend on the width of the legs, or their span.

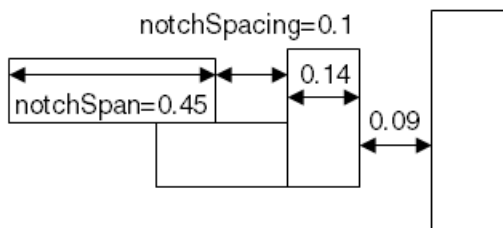
The constraint specifies a spacing between neighbor wires and a notch where the span of the legs on either sides is less than `notchSpan`, and the legs are less than `notchSpacing` apart. In such cases, the portion of the notch length edge having common parallel run length greater than 0 on the opposite side must have a spacing greater than or equal to the constraint's value to any neighbor wire with common parallel run length greater than 0. In addition, even if the neighbor wires are farther away that the specified spacing, if there are two consecutive U shaped notches where the span is less than `notchSpan` and the legs are less than `notchSpacing` apart, it is considered a violation. However, if the notch length is exactly `exceptNotchLength`, the rule does not apply.



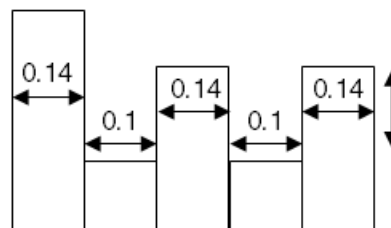
Violation, the notch span of 0.14 (< 0.15) and notchSpacing of 0.1 (< 0.12) require spacing ≥ 0.1 to neighbor wire



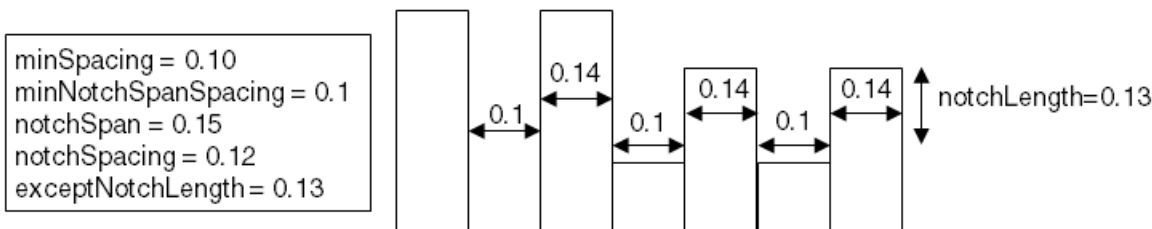
Ok, the neighbor wire does not common parallel run length to the common portion of the notch length edge



Ok, the left wire has span of 0.45 (≥ 0.15), the rule does not apply



Violation, multiple consecutive notches is bad independent of existence of neighbor wires



Ok, because the length of the notch exactly matches `exceptNotchLength` and hence the constraint does not apply.

Constraint Category	Num Cut
Constraint Type	Complex
OA Rule	oacMinNotchSpanSpacing

Syntax:

```
minNotchSpanSpacing
  <string|LPP>layer
  <float>spacingValue
  -notchSpan <float>
  -notchSpacing <float>
  [-exceptNotchLength float]
  [-isHard true|false]
  [comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or a LPP.
spacingValue	The value of the spacing between a notch leg and a neighboring wire.
-notchSpan	Specifies the notch span, which determines if the constraint applies.
-notchSpacing	Specifies the spacing between the legs of the notch.
-exceptNotchLength	Specifies the exact notch length at which this constraint does not apply (as long as neighbor wires are far enough away). This parameter applies to the smallest notch.

For example:

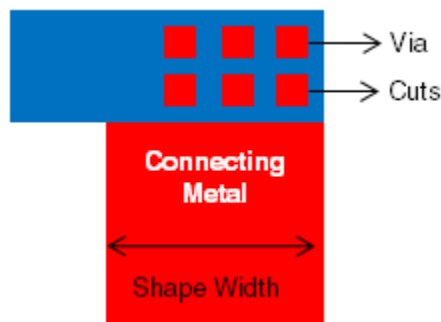
```
minNotchSpanSpacing M1 0.1 -notchSpan 0.15
  -notchSpacing 0.12
  -exceptNotchLength 0.13
```

minNumCut

This layer constraint specifies the minimum number of cuts required in a via object or via instance when connecting a wire shape to a pin or two wide wire shapes or when connecting a wide shape to a pin.

The distance at which additional cut shapes cannot be counted for meeting this constraint can be defined using an optional parameter.

The minNumCut constraint is specified for cut layers only.



Constraint Category	Num Cut
Constraint Type	Complex
OA Rule	oacMinNumCut

Syntax:

```
minNumCut <string|LPP>layer
          <float1DTable|float2DTable>value
          [-distanceWithin <float>]
          [-neighborLayerMetal upperLower|
                                upper|
                                lower]
          [-cutClass cutClassArray | cutClassNameList]
          [-isHard true|false]
          [comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or a LPP.
value	float1DTable: • Key: Width of either metal shape connecting to a via. • Value: Minimum number of cuts required for the Via connecting to the wide Shape or Pin required if the width $\geq$ key. float2DTable: 2D lookup table with row headers for lower metal width, column headers for upper metal width and table entries corresponding to the minimum number of cuts required for the given widths.
width	Specifies the width of the connecting metal and the number of cuts required
-distanceWithin	Specifies the distance at which cuts cannot be counted, if vias are not within this distance.
-neighborLayerMetal	Specifies the cut layers to which this constraint applies. If this parameter is not specified, the constraint applies to both upper and lower metal layers. Default value is upperLower.

Argument	Description
-cutClass	List of paired float values or a list of cutClass names. See cutClass and cutClassList . The constraint applies only in cases where the cuts comply with these specifications.

Rule	Description	Value
V1.M.1	Minimum number of Vias at spacing <= 0.3um used to connect M1 and M2 when either M1 or M2 width >=0.8 um	2

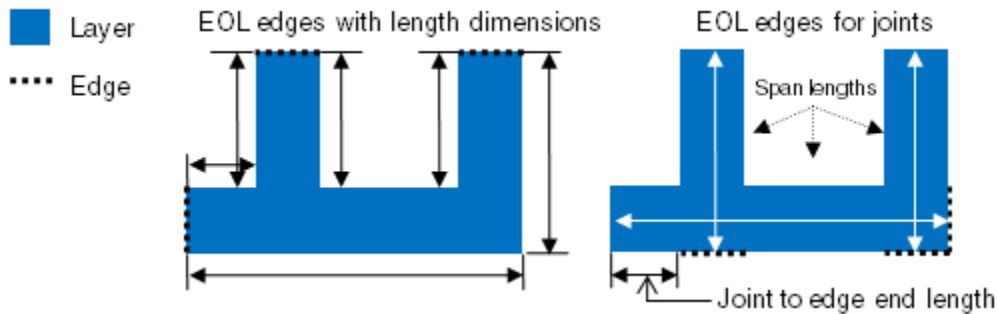
```
minNumCut m1 ((0.2 1) (0.5 3)) -distanceWithin 0.12
minNumCut v1
    ((0.12 0.14)2)
    ((0.12 0.18)3))
```

minOppositeSpanSpacing

This layer constraint specifies spacing rules for edges that are on opposite sides of a common shape on the same layer. The geometric configuration of the neighbor shapes dictates the amount of spacing required and the spacing requirement can be different for the two neighbor shapes. The neighbor edges must be either from an end of line or a joint and are not necessarily complete sides from the neighbor shapes. A joint edge is defined as a segment of a side of a neighbor shape which has a span greater than a given value, a length greater than a given value, and which is not an end of line edge.

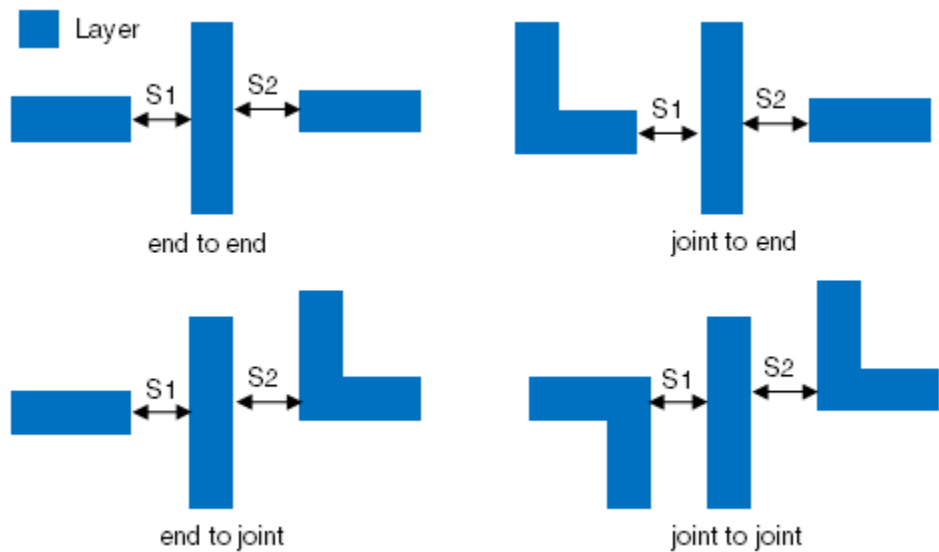
Figure 14 shows end of line edges on the left and joint edges on the right. End of line edges must meet a minimum length requirement for one or both of the adjacent edges while the joint edges are dependent on the span length which is the distance from the end of line edge to the joint edge. Also important for joint edges is that the distance from the joint edge to the nearest end point of the shape edge containing the joint edge is less than a certain value. For both types of edges, the length of the edge must be less than an optional threshold for the constraint to apply.

Figure 14 minOppositeSpanSpacing: End of line edges and Joint edges



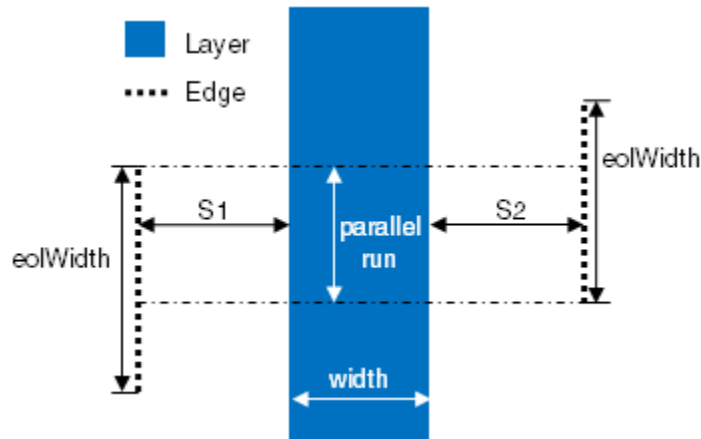
The opposite span spacing rule applies to the four cases illustrated in [Figure 15](#). Each scenario can have a different pair of spacing values for S1 and S2.

Figure 15 minOppositeSpanSpacing: Scenarios



The projected parallel common run between the two neighbor shape edges is also a condition of this rule and at a minimum must be greater than zero. The parallel run length might also have an optional upper bound. [Figure 16](#) shows a close-up view of the spacing rule based on the determined end of line or joint edges.

Figure 16 minOppositeSpanSpacing: Close-up view of spacing rule



Constraint Category	Spacing
Constraint Type	Complex

OA Rule

oacMinOppositeSpanSpacing

Syntax:

```
minOppositeSpanSpacing
    <string|LPP>layer
    <dualValueArray>spacing
    -width float
    -endOfLineWidth float
    -spanLength float
    [-length float]
    [-jointWidth float]
    [-jointToEdgeEndLength float]
    [-exceptEdgeLengthPRL (float1 float2)]
    [-sideLength float]
    [-parallelRunLength float]
    [-isHard true|false]
    [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or a LPP.
spacing	An array to hold four dual float values. Each dual float value specifies S1 and S2 spacing value for four situations illustrated in the description section. - The first dual value is for endToEnd space type. - The second dual value is for endToJoint. - The third dual value is for jointToEnd. - The fourth dual value is for jointToJoint.
-width	The constraint does not apply when the width of the center shape is greater than this parameter value.
-endOfLineWidth	End of line edges must be less than this value for the constraint to apply.
-spanLength	The value which an edge must be greater than to be considered a joint edge.
-length	If exists defines the minimum length for an edge to be considered an end of line edge.
-jointWidth	If exists defines the threshold length for identifying joint edges. Edges greater than or equal to this value are not considered joint edges.
-jointToEdgeEndLength	Defines the threshold distance from a joint edge to the outer vertex of shape edge. If the joint to edge distance greater than or equal to this value than that edge is not a joint edge.
-exceptEdgeLengthPRL	The first array index is a float value to hold the length that both opposite edges must be less than for the constraint to apply. The second array index is a float value which holds the minimum parallel run length for the constraint to apply.

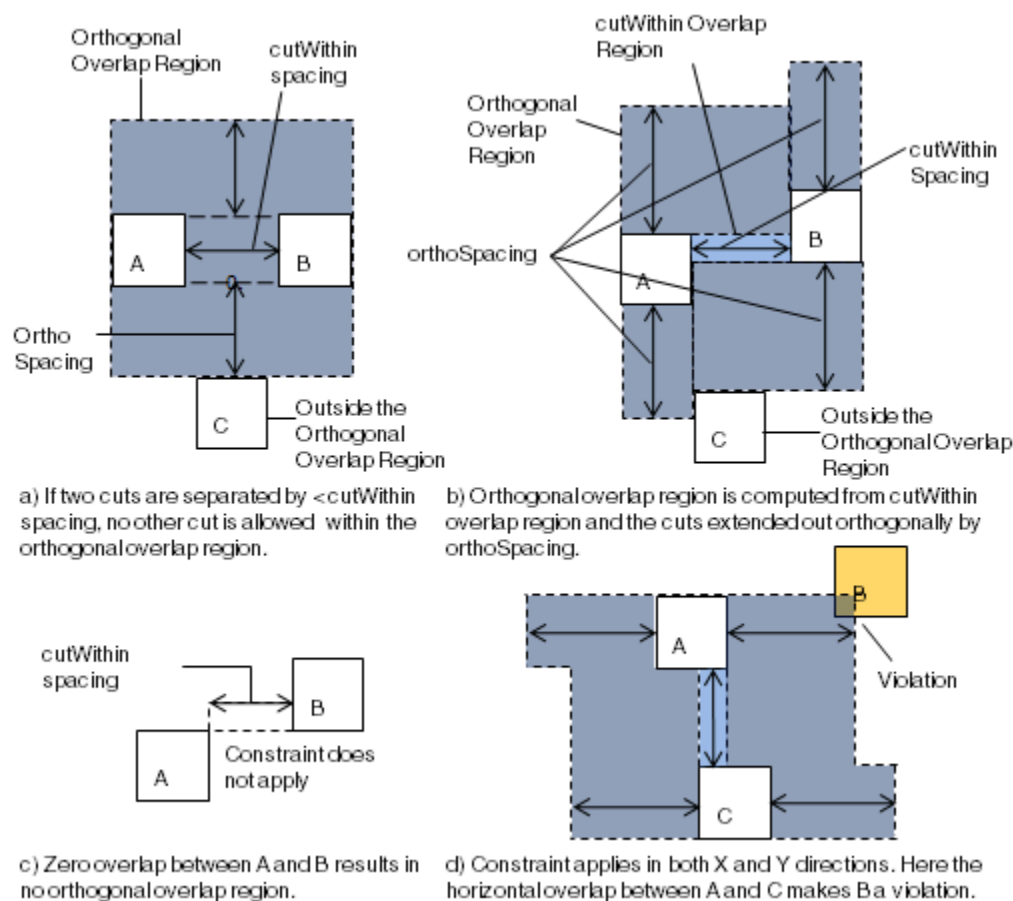
Argument	Description
-sideLength	If specified, this constraint specifies the required length of a neighbor wire with end-of-line less than or equal to <code>endOfLineWidth</code> . If the neighbor line has the required width and length, the <code>sideToEnd</code> and <code>sideToJoint</code> spacings are required.
-parallelRunLength	The constraint applies if the projected run length of the target wire's two neighbor wires must be less than <code>parallelRunLength</code> . If the <code>parallelRunLength</code> is negative, the constraint would apply if one extended the end, joint or side edges by <code>abs(parallelRunLength)</code> before checking for parallel run length.

For example:

```
minOppositeSpanSpacing m1 ((0.10 0.12)
                           (0.08 0.08)
                           (0.08 0.12)
                           (0.08 0.10))
                           -width 0.08
                           -endOfLineWidth 0.10
                           -spanLength 0.15
                           -exceptEdgeLengthPRL (0.12 0.10)
```

minOrthogonalViaSpacing

This layer constraint specifies a table that specifies the required orthogonal spacing between overlapping via cut pairs and other vias. The cut pair can be overlapping either vertically or horizontally.



Constraint Category	Via Construction
Constraint Type	Complex
OA Rule	oacMinOrthogonalViaSpacing

Syntax:

```
minOrthogonalViaSpacing
  <string|LPP>layer
  <float1DTable>cutWithinSpacing
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or a LPP.

Argument	Description
cutWithinSpacing	Specifies the cut within spacing between the cut pair and the orthogonal spacing between overlapping via cut pair and other vias.

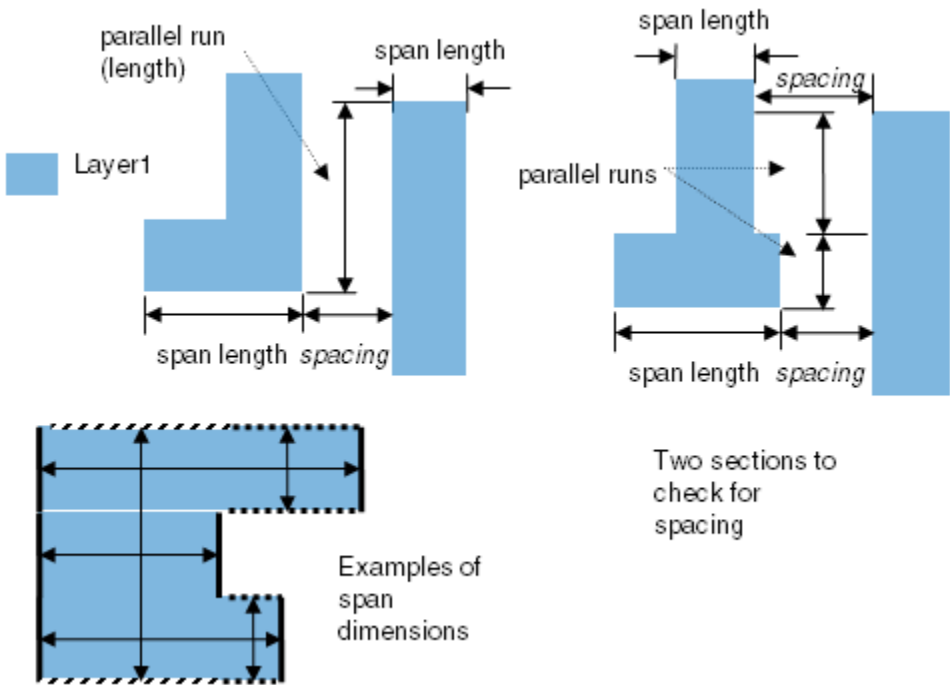
In this example, the minimum required orthogonal spacing between overlapping `VIA1` cut pairs (when they are within a certain spacing) and other `VIA1s` is mentioned:

Rule	Description	Value (in microns)
VIA1.SP.3	overlapping VIA1 cut pairs within 0.1um space to other VIA1	0.12

```
minOrthogonalViaSpacing VIA1 0.09 ((0.1 0.12))
```

minParallelSpanSpacing

This constraint defines the minimum spacing between two shapes on the same layer, where the maximum span length of both shapes is greater than the defined span length and the parallel run length of each span section is greater than the defined parallel run length between shapes.



Constraint Category	Spacing
---------------------	---------

Constraint Type	Complex
OA Rule	oacMinParallelSpanSpacing

Syntax:

```
minParallelSpanSpacing
  <string|LPP>layer
  <float2DTable>spacingValue
  [-length float]
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or as LPP.
spacingValue	The minimum spacing along the common parallel run edges. • Row key: Span length of one shape. • Column key: Span length of another shape.
-length	It specifies the run length threshold. Run lengths between two shapes must be greater than this value in order for the constraint to apply.

For example:

```
minParallelSpanSpacing m1 0.05 ( ((0.08 0.12) 0.06)
                                   ((0.08 0.14) 0.06)
                                   ((0.08 0.16) 0.06)
                                   ((0.10 0.12) 0.06)
                                   ((0.10 0.14) 0.08)
                                   ((0.10 0.16) 0.08)
                                   ((0.12 0.12) 0.08)
                                   ((0.12 0.14) 0.08)
                                   ((0.12 0.16) 0.10))
  -length 0.3
```

minParallelViaSpacing

This layer constraint specifies the minimum spacing for vias that have overlapping parallel edges. This constraint applies to cut layers only when vias do not share the same metal above or below. The constraint uses a float to specify the required spacing.

Constraint Category	Via Construction
Constraint Type	Simple
OA Rule	oacMinParallelViaSpacing

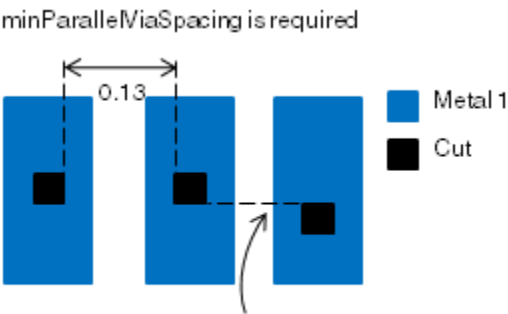
Syntax:

```
minParallelViaSpacing
  <string|LPP>layer
  <float>spacingValue
  [-exceptConnectivityType
    anyConnectivityType |
    sameNetConnectivityType |
    contiguousShapesConnectivityType |
    sameViaConnectivityType |
    directShapesConnectivityType]
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or a LPP.
spacingValue	Specifies the minimum spacing for vias that have parallel edges with an overlap greater than 0.
-exceptConnectivityType	This parameter supports the following three enum values: • anyConnectivityType: Use this option to designate that the constraint applies to any type of connectivity (default). • sameNetConnectivityType: The parallel overlap rule does not apply to same-net cuts. • contiguousShapesConnectivityType: The parallel overlap rule does not apply to cuts that share the same metal shapes either above or below the cuts. • sameViaConnectivityType: The parallel overlap rule does not apply to cuts that share the same metal shapes above and below metal layers to cover the overlap area between the cuts. • directShapesConnectivityType: Use this option to designate that the constraint only applies when the two cuts are part of the same metal shape on the upper or lower metal and the metal shape covers the region between the two cuts.

In this example, the minimum VIA1 spacing is mentioned for VIA1s on different net and having common run length > 0:

```
minParallelViaSpacing VIA1 0.13 -exceptConnectivityType
contiguousShapesConnectivityType
```

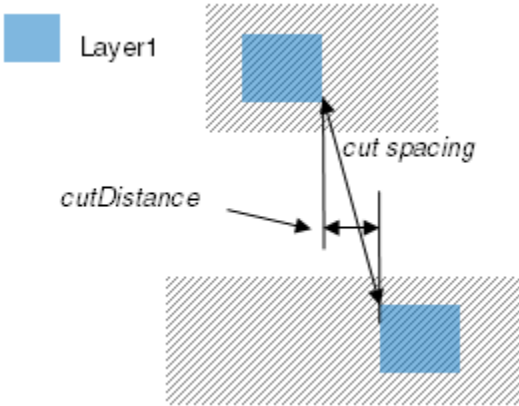


Cuts require greater than zero parallel overlap for constraint to apply

Rule	Description	Value (in microns)
VIA1.SP.4	VIA1 - VIA1 spacing for VIA1s on different net and when parallel run length > 0	0.13

minParallelWithinViaSpacing

Some flaps might have a special rule for spacing (cutSpacing) between cuts where the edge of a neighbor cut is in cutDistance, beyond the edge of another cut as shown in the example provided below. The rule might not apply if the cuts are on the same net.



Constraint Category	Via Construction
Constraint Type	Complex
OA Rule	oacMinParallelWithinViaSpacing

Syntax:

```
minParallelWithinViaSpacing
  <string|LPP>layer
  <float>spacingValue
  -cutDistance <float>
  [-exceptConnectivityType
    anyConnectivityType|
    sameNetConnectivityType|
    contiguousShapesConnectivityType|
    sameViaConnectivityType|
    directShapesConnectivityType]
  [-centerToCenter true|false]
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

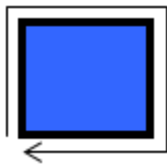
Argument	Description
layer	Layer can be defined as a string or a LPP.
spacingValue	The minimum spacing between via cut corner to corner.
-cutDistance	Orthogonal separation width threshold. Separation width must be less than this value for the constraint to be applied.
-exceptConnectivityType	The constraint does not apply if the cuts have this connectivity type. The only allowed value is sameNet: • anyConnectivityType: Use this option to designate that the constraint applies to any type of connectivity (default). • sameNetConnectivityType: The parallel overlap rule does not apply to same-net cuts. • contiguousShapesConnectivityType: The parallel overlap rule does not apply to cuts that share the same metal shapes either above or below the cuts. • sameViaConnectivityType: The parallel overlap rule does not apply to cuts that share the same metal shapes above and below metal layers to cover the overlap area between the cuts. • directShapesConnectivityType: Use this option to designate that the constraint only applies when the two cuts are part of the same metal shape on the upper or lower metal and the metal shape covers the region between the two cuts.
-oaCenterToCenter	This parameter is a boolean that designates whether cut spacing is measured from the center of one cut to the center of the other cut. If not specified, cut clearance is measured from edge to edge.

For example:

```
minParallelWithinViaSpacing v1 0.18 -cutDistance 0.10
```

minPerimeter

The constraint defines the minimum perimeter of a shape on a particular layer.



Constraint Category	Length and Width Process Rules
Constraint Type	Simple
OA Rule	oacMinPerimeter

Syntax:

```
minPerimeter
  <string|LPP>layer
  <float>minPerimeter
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or as LPP.
minPerimeter	Specifies the minimum perimeter of a shape.

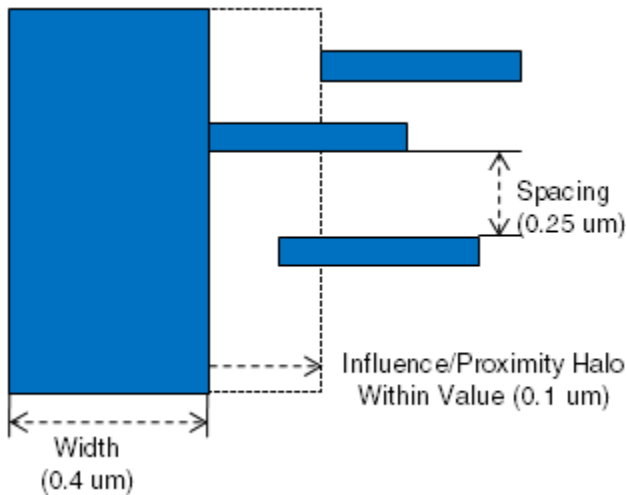
For example:

```
minPerimeter m1 0.35
```

minProtrudedProximitySpacing

This layer constraint specifies the minimum distance necessary between a protrusion and another geometry on another net on the same layer.

The large shape to which it connects influences the spacing required by the protrusion. In effect, the smaller shape picks up the spacing required by the wider shape.



Constraint Category	Spacing
Constraint Type	Complex
OA Rule	oacMinProtrudedProximitySpacing

Syntax:

```
minProtrudedProximitySpacing
  <string|LPP>layer
  <float|float1DTable|float2DTable>minSpacing
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or a LPP.
minSpacing	• float: Specifies the minimum spacing required between the protrusion and the small shape. • float1DTable: Specifies the width of the large influencing shape, and the minimum spacing required between the protrusion and the small shape. • float2DTable: Specifies the width of the large influencing shape, the distance between the small shape and the large influencing shape, and the minimum spacing required between the protrusion and the small shape.

This example specifies the minimum protruded proximity spacing between the protrusion and the other shape:

Rule	Description	Value (in microns)
M1.PR.SP.1	Minimum M1 Protrusion - M1 Spacing	0.1

```
minProtrusionProximitySpacing M1 0.1
```

This example specifies the minimum spacing between the protrusion and the other M1 shapes based on the width of the large influencing shape:

Rule	Description	Value (in microns)
M1.PR.SP.1.1	Minimum M1 Protrusion - M1 Spacing when the width of the larger influencing shape ≥ 0.3	0.15
M1.PR.SP.1.2	Minimum M1 Protrusion - M1 Spacing when the width of the larger influencing shape ≥ 0.5	0.2

```
minProtrudedProximitySpacing M1 0.1 ((0.3 0.15) (0.5 0.2))
```

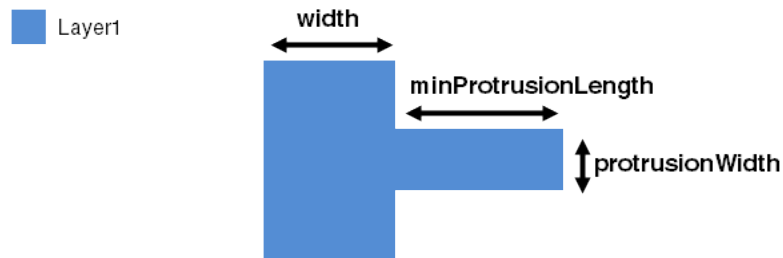
This example specifies the minimum spacing between the M1 protrusion and other M1 shapes based on the width of the large influencing shape and the distance of the small shapes from the larger shape:

Rule	Description	Value (in microns)
M1.PR.SP.1.3	Minimum M1 Protrusion - M1 Spacing when the width of the larger influencing shape ≥ 0.4 and the distance of the closest smaller M1 shape is < 0.1	0.25

```
minProtrudedProximitySpacing M1 0.09 (((0.4 0.1) 0.25))
```

minProtrusionLength

This layer constraint specifies that protrusions from a wide shape must have specific dimensions.



Constraint Category	Length and Width Process Rules
Constraint Type	Complex
OA Rule	Custom Compiler Custom Layer Constraint

Syntax:

```
minProtrusionWidth
  <string|LPP>layer
  <float>width
  -protrusionWidth rangeData
  [-excludeWithVia true|false]
  [-protrusionFillWidth float]
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer on which the shape is placed. <code>layer</code> can be a string (layer name) or an LPP.
width	Minimum width of the wide shape.
-protrusionWidth	Width of the protrusion. You can specify a range of widths, in microns. See Range Data Syntax .
-excludeWithVia	(Optional) Indicator of whether the constraint applies: • true: Do not apply the constraint if the protrusion contains a connected via with the proper extensions. • false: Apply the constraint even if the protrusion contains a connected via with the proper extensions. Default: false
-protrusionFillWidth	(Optional) Width of a fill rectangle, where the length of the rectangle is defined by the constraint value. The fill must fit in the protrusion leg, or it is a violation.

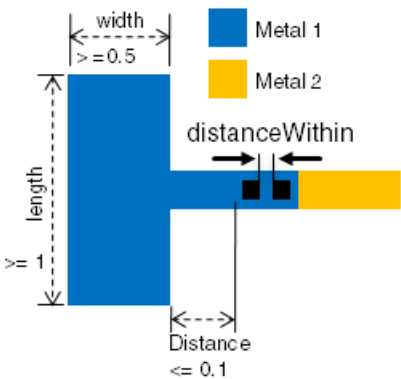
Example: Minimum M1 protrusion width is 0.5 μm when M1 is connected to a wire of width $\geq 2 \mu\text{m}$ and the length of the protrusion $< 1 \mu\text{m}$.

```
minProtrusionLength M1 0.5 -width 0.07 -protrusionWidthRange (>=0.2
<=0.5) -excludeWidthVia true -protrusionFillWidth 0.05
```

minProtrusionNumCut

This layer constraint specifies the number of via cuts required while connecting a thin wire to a wide wire or pin.

The constraint is applicable when the minimum wide wire width and length specified are exceeded. These values are specified through optional parameters. An optional third parameter specifies the maximum distance from the wide metal to the first via cut.



Constraint Category	NumCut
Constraint Type	Complex
OA Rule	oacMinProtrusionNumCut

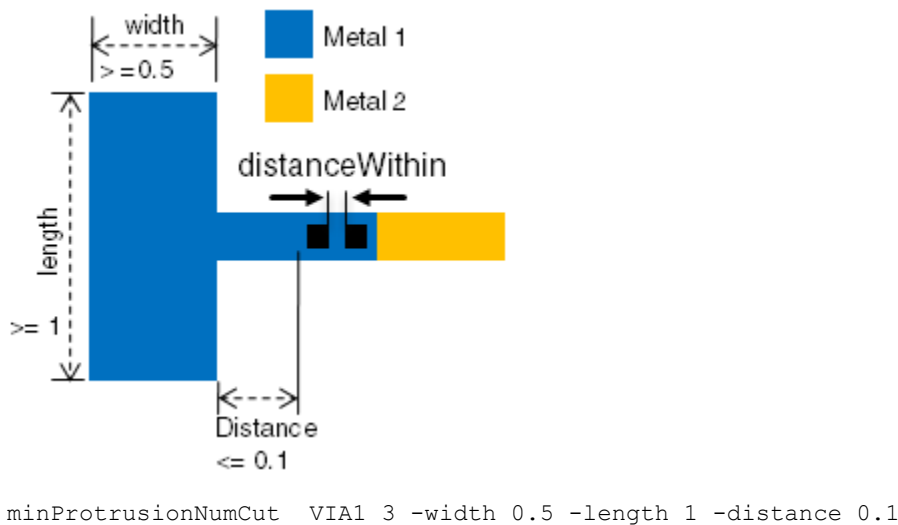
Syntax:

```
minProtrusionNumCut
  <string|LPP>layer
  <int>numOfCuts
  -width <float>
  -length <float>
  [-distance <float>]
  [-distanceWithin <float>]
  [-area float]      [-neighborLayerMetal upperLower|upper|lower]
  [-validVias viaDefArrayValue]
  [-cutClass <cutClassArray | cutClassNameList>]
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or a LPP.
numOfCuts	Specifies the minimum number of cuts required for a via placed on a thin wire.
-width	Specifies the width of the wide wire connected to the protrusion.
-length	Specifies the length of the wide wire connected to the protrusion.
-distance	Specifies the distance from the outside edge of the wide wire to the outside edge of the first via cut on the protrusion.
-distanceWithin	Distance between the cuts. If the cuts are at this distance or farther apart, they will not be counted.
-area	The number of cuts can also depend on the area of the wires. If width < width of default routing wire and is specified along with area, the minimum cut requirement on the routing vias varies as per the area of the routing wire on the layer. If only area and width are specified, this rule checks only the thin wire connected to a wide-wire whereas the wide-wire is not checked.
-neighborLayerMetal	Specifies the cut layers to which this constraint applies. If this parameter is not specified, the constraint applies to both upper and lower metal layers. Default is <i>upperLower</i> .
-validVias	Specifies the list of valid viaDefs to use in the protrusion.
-cutClass	List of paired float values or a list of cutClass names. See cutClass and cutClassList . The constraint applies only in cases where the cuts comply with these specifications.

In this example, the minimum number of cuts of `VIA1` required to connect `M1` and `M2` when one of the metals had `width >= 0.5` and `length >= 1` is mentioned, the distance of first cut of `VIA1` from the wide metal is also mentioned:

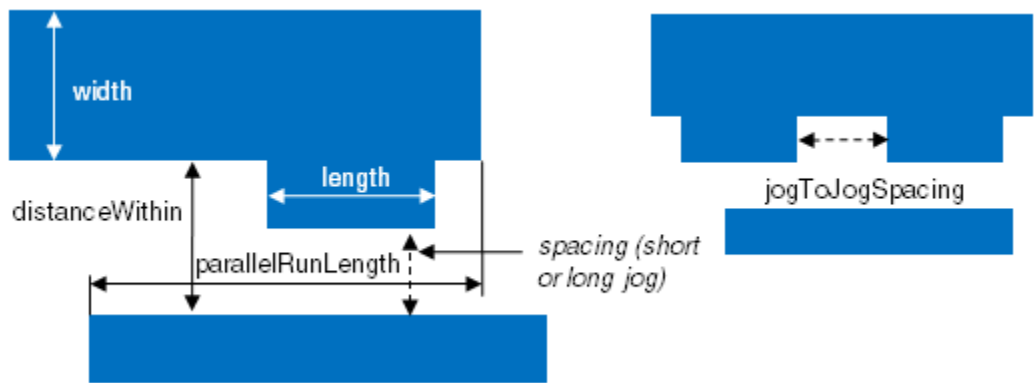
Rule	Description	Value (in microns)
VIA1.Pcut	There should be at least 3 VIA1s connecting M1 and M2 when width of either of metals is <code>>=0.5</code> and <code>length >=1</code> , the distance of the first VIA1 from the wide metal needs to be within 0.1	N/A



minProtrusionSpacing

This constraint is for wide wires to be spaced at a minimum distance (`jogToJogSpacing`) when a parallel shape on the same layer is within a given distance (`distanceWithin`) and have a minimum amount of common parallel run (`parallelRunLength`). The specific spacing from the protrusion to the parallel shape is within the distance and has a minimum common parallel run. The spacing is dependent on the width of the protrusion (`length`).

The `length` parameter defines the threshold of determining whether a protrusion is considered a short or long protrusion. If the protrusion width is less than or equal to the `length` parameter value, then the protrusion is considered a short protrusion and must honor the short jog spacing value, else it is a long protrusion and must honor the long jog spacing value.



Constraint Category	Spacing
Constraint Type	Complex

OA Rule oacMinProtrusionSpacing

Syntax:

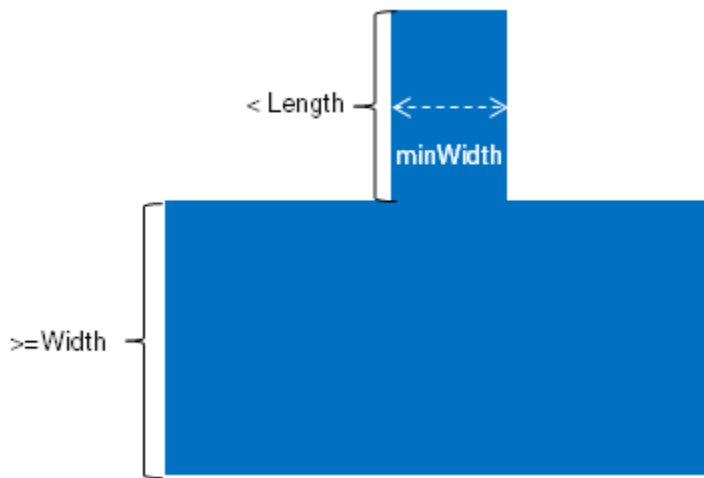
```
minProtrusionSpacing
    <string|LPP>layer
    <dualValue float>spacingValue
    -width <float>
    -parallelRunLength <float>
    -distanceWithin <float>
    -length <float>
    [-isHard true|false]
    [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or a LPP.
spacingValue	The value is an array of two values. The first array element represents the short jog spacing and the second represents the long jog spacing. The second array element represents the jog to jog spacing.
-width	This parameter specifies the width in microns in which wide shapes must be greater than in order for the constraint to apply.
-parallelRunLength	This parameter specifies the parallel run length threshold in microns. The constraint applies only when the parallel run is greater than this value.
-distanceWithin	This parameter specifies the distance threshold in microns for the parallel shape. The shape must be within this distance for the constraint to apply.
-length	This parameter specifies the jog width threshold in microns that defines short and long protrusion. A short protrusion is less than or equal to this value.

```
minProtrusionSpacing m1 ((0.08 0.06)0.10)
    -width 0.12
    -parallelRunLength 0.24
    -distanceWithin 0.12
    -length 0.12
```

minProtrusionWidth

The minimum width(minWidth) of a protrusion is required if the protruded shape is shorter than the specified length and if it connects to a wire that has a width greater than or equal to a specified width.



Constraint Category	Length and Width Process Rules
Constraint Type	Complex
OA Rule	oacMinProtrusionWidth

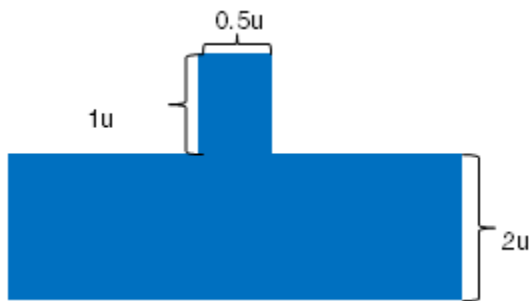
Syntax:

```
minProtrusionWidth
  <string|LPP>layer
  <float>minWidth
  <-width float>
  <-length float>
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or a LPP.
minWidth	Specifies the minimum width of protrusion shape.
-width	Specifies the width of the wide wire connected to the protrusion. The constraint applies when the width of the wide wire is >= the specified width value and the length requirement is met.
-length	Specifies the length of the wide wire connected to the protrusion. The constraint applies when the length of the protruded stub is < the specified length value and the width requirement is met.

This example specifies the minimum width of the protrusion of `M1`, if the protruded metal is shorter than `1um` and when connecting to a wire with a width $\geq 2um$:

Rule	Description	Value (in microns)
M1.PR.W	Minimum M1 protrusion width when M1 is connected to a wire of 0.5 width $\geq 2um$ and the length of the protrusion $< 1um$.	

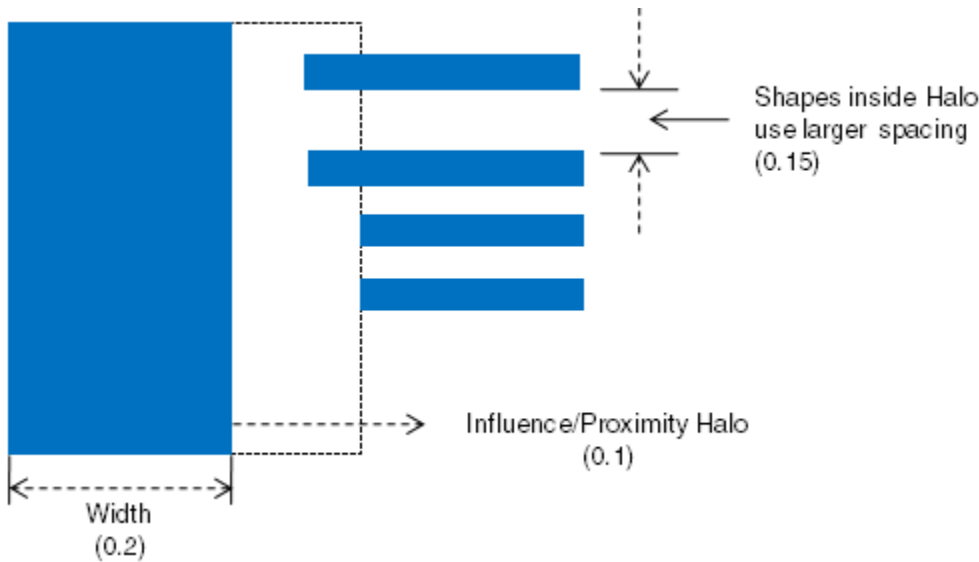


```
minProtrusionWidth M1 0.5 -width 2 -length 1 -distance 0.1
```

minProximitySpacing

This layer constraint specifies the minimum distance between two shapes when they are near a very large shape.

The smaller shapes effectively pick up the spacing required by the wider neighboring shape.



Constraint Category	Spacing
Constraint Type	Complex
OA Rule	oacMinProximitySpacing

Syntax:

```
minProximitySpacing
    <string|LPP>layer
    <float|float1DTable|float2DTable>spacingValue
    [-isHard true|false]
    [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or a LPP.
spacingValue	float: Specifies the minimum spacing required between the two shape. float1DTable: Specifies the width of the large influencing shape and the minimum spacing required between the two smaller shapes. float2DTable: Specifies the width of the large influencing shape, the distance between the closest of the two smaller shapes and the large influencing shape, and the minimum spacing required between the two smaller shapes.

This example specifies the minimum spacing between the two shapes:

Rule	Description	Value (in microns)
M1.PROX.SP.1	Minimum M1 - M1 Spacing	0.09

```
minProximitySpacing M1 0.09
```

This example specifies the minimum spacing between the two shapes based on the width of the large influencing shape:

Rule	Description	Value (in microns)
M1.PROX.SP.1.1	Minimum M1 - M1 Spacing when the width of the larger influencing shape >= 0.2	0.12
M1.PROX.SP.1.2	Minimum M1 - M1 Spacing when the width of the larger influencing shape >= 0.5	0.14

```
minProximitySpacing M1 0.09 ((0.2 0.12) (0.5 0.14))
```

This example specifies the minimum spacing between the two shapes based on the width of the large influencing shape and the distance of the two smaller shapes from the larger shape:

Rule	Description	Value (in microns)
M1.PROX.SP.1.3	Minimum M1 - M1 Spacing when the width of the larger influencing shape >= 0.2 and the distance of the closest smaller M1 shape is < 0.1	0.15

```
minProximitySpacing M1 0.09 (((0.2 0.1) 0.15))
```

minRectArea

This layer constraint specifies the minimum area for rectangular shapes on a layer. For non-rectangular, polygonal shapes, use the [minArea](#) constraint.



Constraint Category	Area
Constraint Type	Simple
OA Rule	oacMinRectArea

Syntax:

```
minRectArea
  <string|LPP>layer
  <float>areaValue
  [-width float]
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
  [-widthRange rangeData]
```

Argument	Description
layer	Layer can be defined as a string or a LPP.
areaValue	Specifies the minimum rectangular area for the shape in μm^2 .
-width	If given, the rule applies only to rectangles with width greater than or equal to this parameter value.
-widthRange	Defines the width requirements for the short edge of the rectangle. The width must lie in the given range. Cannot be used if <code>width</code> is given. If both are presented, the <code>widthRange</code> parameter will be used. To specify the ranges, see Range Data Syntax .

Rule	Description	Value (in μm^2)
DIFF.A.2	Minimum Rectangular Area of DIFF	0.04

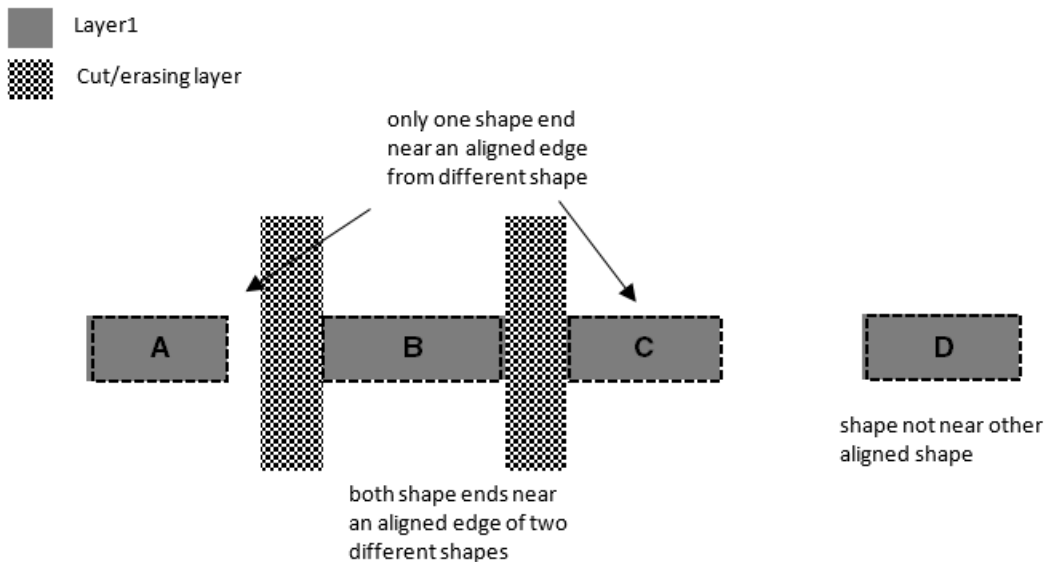
Examples:

```
minRectArea DIFF 0.04
```

```
minRectArea m1 0.10 -widthRange (>=0.1 <=0.2)
```

minRectAreaProximity

This layer constraint specifies the minimum area and proximity requirements for shapes involved in erase-layer operations.



Constraint Category	Area
Constraint Type	Complex
OA Rule	Custom Compiler custom layer constraint

Syntax:

```
minRectAreaPromixity
    <string|LPP>layer
    <float>areaValue1 areaValue2 areaValue3
    -edgeRange <rangeData>
    [-width float]
    [-color colorName|any]
    [-isHard true|false]
    [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer of the shape. Layer can be defined as a string or a LPP.
areaValue1	Minimum area of shape A.
areaValue2	Minimum area of shape C.
areaValue3	Minimum area of shape B.
-edgeRange	Range of distances between the eraser and erasee shapes. See Range Data Syntax .
-width	Minimum width of the erasee shape on layer 1. The constraint applies when the erasee shape is greater than the <code>width</code> value.
-color	Defines the specific color (positive value) of the shapes for the rule to apply. A value of zero means any color. Default is any.

Example:

```
minRectAreaProximity m1 (0.25 0.49 0.64)
    -edgeRange (>=0.1 <=0.2)
    -width 0.55
    -color red
```

minRedundantViaSetback

This layer constraint specifies the metal enclosure required on a metal layer when redundant vias are aligned on the same edge of that metal layer.

This constraint is associated with layer 1 being the cut layer and layer 2 being the metal layer.

Constraint Category	Via Construction
Constraint Type	Complex
OA Rule	oacMinRedundantViaSetback

Syntax:

```
minRedundantViaSetback
  <string|LPP>layer
  <float1DTable>minEnclosure
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Name of the Layer or LPP.
minEnclosure	Specifies the minimum enclosure width required on the layer when redundant vias are aligned on the same edge of that metal layer.

In this example, the minimum enclosure of the metal layer over the redundant via is specified:

Rule	Description	Value (in microns)
VIA1.EN.2	Minimum M1 enclosure for redundant Via when M1 width >= 0.6u	0.1

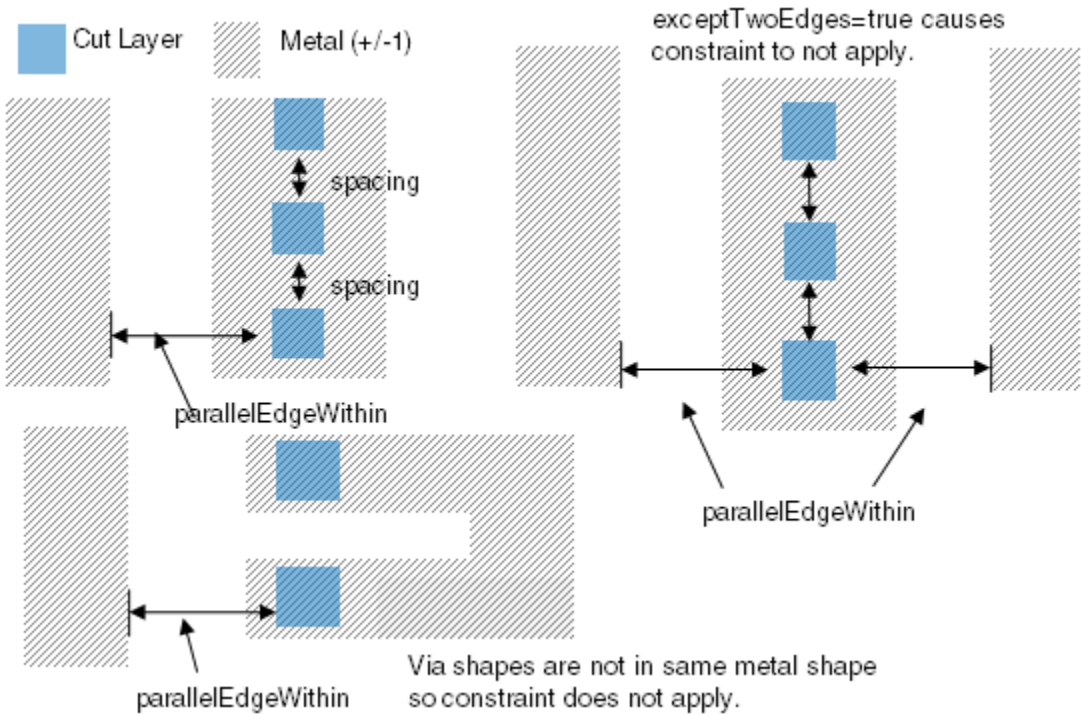


```
minRedundantViaSetback v1 m1 0.05 ((0.6 0.1))
```

minSameMetalSharedEdgeViaSpacing

This constraint is when some new processes require a different cutSpacing between 2 via cuts that have common parallel run length is greater than 0. They must also have a

neighbor wire within parWithin distance from them on the same edge. The rule should be defined on a cut layer. It might also depend on a cut class.



Constraint Category	Via Construction
Constraint Type	Complex
OA Rule	oacMinSameMetalSharedEdgeViaSpacing

Syntax:

```
minSameMetalSharedEdgeViaSpacing
  <string|LPP>layer
  <float>spacingValue
  <-parallelEdgeWithin float>
  [-exceptTwoEdges true|false]
  [-exceptSameViaCount int]
  [-aboveOnly true|false]
  [-cutClass <cutClassArray | cutClassNameList>]
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or a LPP.

Argument	Description
spacingValue	Minimum spacing between shapes on layer1 in same metal shape.
-parallelEdgeWithin	The distance which a one neighbor shape on a metal layer above or below cut layer must be within.
-exceptTwoEdges	If true, the constraint does not apply if two neighboring metal shapes are within the parallelEgeWithin distance. Default is "false".
-exceptSameViaCount	If exists, value defines the minimum number of cuts in metal shape to have this constraint not apply. Default is "Infinity".
-aboveOnly	If true, the constraint only applies if the common metal shape is the layer above the cut layer. Default is "false".
-cutClass	List of paired float values or a list of cutClass names. See cutClass and cutClassList . The constraint applies only in cases where the cuts comply with these specifications.

For example:

```
minSameMetalSharedEdgeViaSpacing v1 0.06
                                  -parallelEdgeWithin 0.12
                                  -exceptSameViaCount 2
```

minSameNetSpacing

This layer constraint specifies the minimum distance required between two geometries that are electrically equivalent.

This constraint should be used in conjunction with [minSpacing](#) (different net spacing) and should only be used when the same net spacing is smaller than the different net spacing.

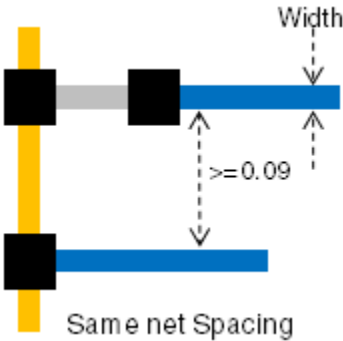
Constraint Category	Spacing
Constraint Type	Complex
OA Rule	oacMinSameNetSpacing

Syntax:

```
minSameNetSpacing
  <string|LPP>layer
  <float|float1DTable>spacingValue
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

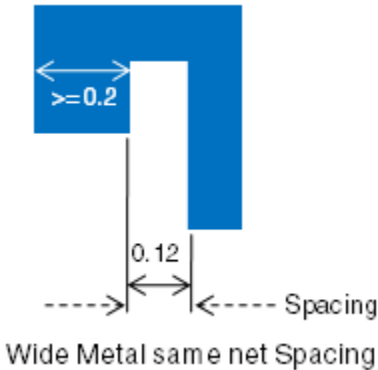
Argument	Description
layer	Layer can be defined as a string or as LPP.
SpacingValue	• float: Specifies the minimum spacing required between the two shapes. • float1DTable: Is a list of two values. The first value specifies the width of the wider of the two shapes and the second value specifies the minimum spacing required between the two shapes.

Rule	Description	Value (in microns)
M1.SP.1	Minimum M1-M1 spacing for M1 on same net	0.09



minSameNetSpacing M1 0.09

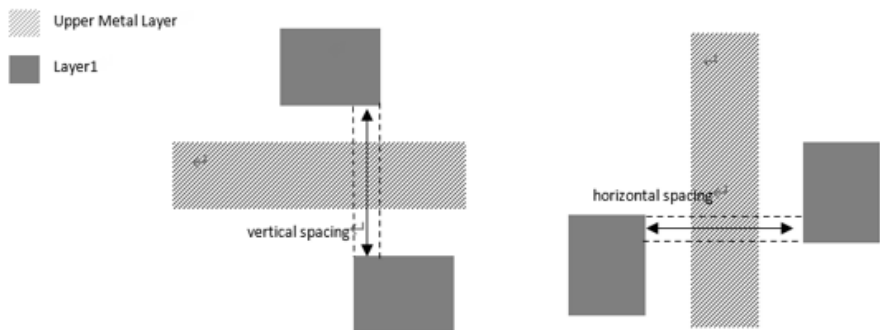
Rule	Description	Value (in microns)
M1.SP.1.1	Minimum M1-M1 spacing for M1 on same net and M1 width $\geq 0.2u$	0.12



minSameNetSpacing M1 0.09 ((0.2 0.12))

minSeparatedViaSpacing

Two specified types of cut shapes which have greater than zero parallel overlap and are separated by an unconnected upper metal shape between them can have a minimum spacing requirement measured edge to edge. The spacing requirement is exempted if the two cut shapes are both spaced with a certain value from the middle upper metal shape.



Constraint Category	Via Construction
Constraint Type	Complex
OA Rule	Custom Compiler Custom Layer Constraint

Syntax:

```
minSeparatedViaSpacing
  <string|LPP>layer
  <float>spacingValue
  -cutClass <cutClassArray | cutClassNameList>
  -spacingDirection horizontal | vertical
  [-otherCutClass cutClassArray | cutClassNameList]
  [-excludeCutToMetalSpacing float1 float2]
  [-otherWidthRange rangeData]

  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Cut layer. layer can be defined as a string or a LPP.
spacing	Minimum distance in database units between specific cut classes with parallel overlap and separated by an unconnected upper metal shape between them.
-cutClass	List of paired float values or a list of cutClass names. See cutClass and cutClassNameList . The constraint applies only in cases where the cuts comply with these specifications.

Argument	Description
-spacingDirection	Defines how the constraint value spacing is applied. Only horizontal and vertical are allowed.
-otherCutClass	Optional parameter if given defines the other cut class which is needed so that the spacing constraint applies.
-excludeCutToMetalSpacing	Optional parameter defining excluded spacings between two cut shapes and the upper layer metal. If both spacings match the parameter values, the constraint doesn't apply.
-otherWidthRange	Optional parameter defining the width range of the upper layer metal that makes spacing constraint to apply.

Note:

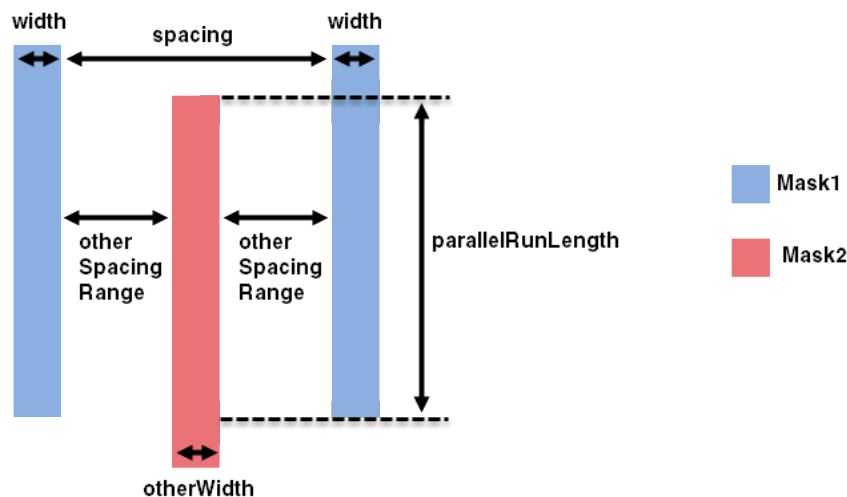
If `cutClass` parameter is specified with `cutClassName` in ASCII file, technology importer looks-up the first `cutClass` constraint specified in the same hierarchical constraint group to convert to cut class size. If a `cutClass` constraint does not exist then an error message is displayed. While exporting, the tech exporter looks-up the first `cutClass` constraint specified in the same hierarchical constraint group to convert the cut class size to cut class name. If no `cutClass` constraint exists; the `cutClass` parameter is exported with cut class size in dual value.

For example:

```
minSeparatedViaSpacing v1 0.08 -cutClass 0.06 0.08
                        -spacingDirection horizontal
                        -excludeCutToMetalSpacing 0.02 0.03
```

minSideSpacing

The `minSideSpacing` constraint specifies spacing requirements for shapes on different masks that are placed next to each other on the same layer. The shapes on one mask straddle the shape on another mask. In the example shown, the two blue shapes straddle the red shape.



Constraint Category	Spacing
Constraint Type	Complex
OA Rule	Layer

Syntax:

```
minSideSpacing
  <string|LPP>layer
  -width <float>
  -otherWidthRange <float>
  -otherSpacingRange <rangeValue>
  -parallelRunLength <float>
  -otherColor <integer>
```

Argument	Description
layer	The layer on which the shapes are placed. The layer can be defined as a string or a layer-purpose pair.
-width	The minimum width of the shapes that are straddling the third shape. The width of the straddling shapes must be $\geq$ the specified <code>width</code> value.
-otherWidthRange	The width of the middle shape on the other mask. The width must be within the specified range.
-otherSpacingRange	The distance between the shapes. The constraint applies if the distance is within the specified range.
-parallelRunLength	The length within which all three shapes overlap. The constraint applies if the length is $\geq$ the specified <code>parallelRunLength</code> value.

Argument	Description
-otherColor	Masks the color of the middle shape. By default, the constraint only applies if the mask color of the middle shape is different from the mask color of the outer shapes. Outer shapes should always be the same mask color. A value of -1 (same) means the mask color of the middle shape must be the same as the mask color of the outer shapes. A value of zero means any color for the middle shape. If a positive value is given, defines the specific mask color of the middle shape.

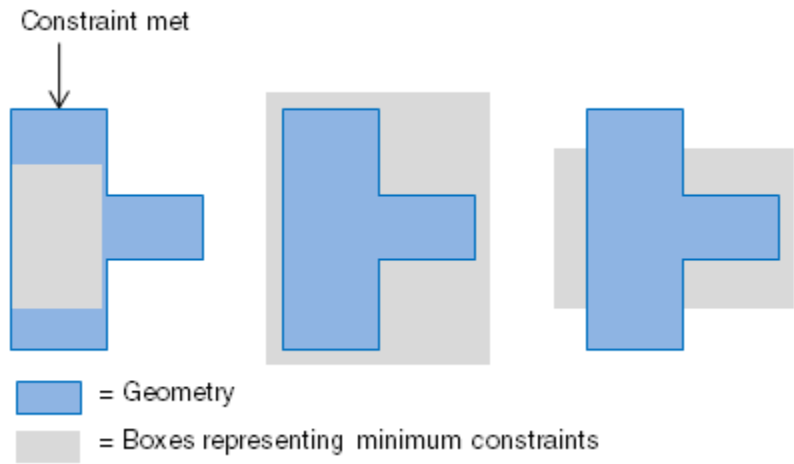
In the following example, the constraint applies if the width of the straddling shapes on the m1 layer is 0.08 or wider, the width of the middle shape must be between 0.2 and 0.5 (inclusive), and the distance between the shapes must be between 0.4 and 0.9 (inclusive).

```
minSideSpacing m1 0.06
    -width 0.08
    -otherWidthRange (>=0.2 <=0.5)
    -otherSpacingRange (>=0.4 <=0.9)
    -otherColor red
```

minSize

This layer constraint specifies the minimum length and width for any shape. minSize constraints use boxes to represent the minimum size.

To determine if a piece of geometry meets this constraint, try to fit each box from the list inside your shape. The constraint is met if any of the boxes fit inside your shape.



Constraint Category	Length and Width Process Rules
Constraint Type	Complex
OA Rule	oacMinSize

Syntax:

```
minSize <string|LPP>layer
      <float>box
      [-rectOnly true|false]
      [-isHard true|false]
      [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or as LPP.
box	Specifies the list of minimally sized boxes with which to test your shape.
-rectOnly	The constraint only applies to rectangular shapes. The default is <i>false</i> .

For example:

```
minSize poly (0.12 0.14 0.5 0.6) -isHard false
```

minSpacing

This layer constraint specifies the minimum orthogonal spacing required between two geometries. The distance is specified in microns. The spacing can be either a Euclidian or Manhattan measure.

minSpacing can be specified in the following ways:

- As the minimum spacing between any two adjacent geometries
- Based on the vertical or horizontal spacing direction for adjacent geometries
- Based on the width of the wider of the two geometries plus the parallel run length
- Based on the width of both geometries plus the parallel run length

Optional parameters determine whether the constraint applies dependent on the type of connectivity and dependent on whether the shapes belong to power or ground nets.

Constraint Category	Spacing
Constraint Type	Complex
OA Rule	oacMinSpacing

Syntax:

```
minSpacing
  <string|LPP>layer
```

```
<float2DTable>spacingValue
[-distanceMeasureType euclidian|manhattan]
[-widthLengthTableType
    widthWidthTableType|
    widthParallelRunLengthTableType
    twoWidthParallelRunLengthTableType]
[-spacingDirection
    horizontal|vertical|any]
[-connectivityType
    anyConnectivityType|
    sameNetConnectivityType|
    contiguousShapesConnectivityType|
    directShapesConnectivityType|
    sameViaConnectivityType]
[-PGnet true|false]
[-inLayerDirection <string>layerName]
[-oaArea float>]
[-endOfLineSpacing float]
[-parallelRunLengthRange rangeValue]
[-widthRange rangeValue]
[-otherWidthRange rangeValue]
[-color colorName|any|same|diff]
[-isHard true|false]
[-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or as LPP.
spacingValue	float: Specifies the minimum spacing required between the two shapes. float1DTable: A float1DTable is a list of two float values. The first float value specifies the width of the wider of the two shapes. The second float value specifies the minimum spacing required between the two shapes. float2DTable: The value of which depends on the <code>widthLengthTableType</code> argument. For <code>widthWidthTableType</code>: widthWidthTableType: The rows and the columns represent the widths of the two shapes being analyzed, the float value represents the minimum spacing between the shapes widthParallelRunLengthTableType: The row ("width") represents the width of the wider of the two shapes, the column ("length") represents the parallel run length between the two shapes, and the float value represents the minimum spacing required between the two shapes twoWidthParallelRunLengthTableType: The rows represent the widths of the shapes and the columns represent the parallel run lengths between the two shapes, the float value represents the minimum spacing required between the two shapes
-distanceMeasureType	Specifies if the spacing values are in Euclidian or Manhattan value. Default value is <i>euclidean</i> .

Argument	Description
-widthLengthTableType	widthWidthTableType: Use this option to designate that the constraint applies to the widths of the two shapes widthParallelRunLengthTableType: Use this option to designate that the constraint applies to the wider of two adjacent shapes and their parallel run length (default) twoWidthParallelRunLengthTableType: Use this option to designate that the constraint applies to the width of both shapes and the parallel run length
-spacingDirection	any: Specifies the spacing direction that the constraint applies. Can be either horizontal or vertical (default) horizontal: Specifies the horizontal spacing direction vertical: Specifies the vertical spacing direction
-connectivityType	• anyConnectivityType: Use this option to designate that the constraint applies to any type of connectivity (default). • sameNetConnectivityType: Use this option to designate that the constraint only applies to cuts on the same net. • contiguousShapesConnectivityType: Use this option to designate that the constraint only applies to cuts on the same metal. • directShapesConnectivityType: Use this option to designate that the constraint only applies when the two cuts are part of the same metal shape on the upper or lower metal and the metal shape covers the region between the two cuts. • sameViaConnectivityType: This is similar to directShapesConnectivityType except that both lower and upper metal shapes must cover the cuts and the region between the cuts.
-PGnet	If set to "true", the constraint only applies to power and ground nets, else by default the constraint applies to all nets (default).
-inLayerDirection	If specified, the constraint is limited to edge spacing for the layer in the direction of the shape on this layer which overlaps the edges on the constraint layer.
-oaArea	Indicates that the spacing only applies between any shape with area less than the area specified by oaAreaConstraintParamType and all other shapes on that layer. If this parameter is not used, the spacing is not dependent on shape area. No other constraint parameters are permitted in combination with this constraint parameter.
-endOfLineSpacing	An edge must be greater than or equal to endOfLineWidth to be considered a side edge. This parameter is optional, but if used, it must appear in conjunction with the spacingDirection parameter. The default is 0.
-parallelRunLengthRange	If given, parallel run length between the shapes must lie in this range in database units for the constraint to apply. If the oaSpacingDirection parameter is 1 (horizontal) or 2 (vertical), the default of the parallelRunLengthRange parameter is (>=0).

Argument	Description
-widthRange	Constraint applies only if at least one shape width is in the range. To specify the ranges, see Range Data Syntax .
-otherWidthRange	Constraint applies only if one shape width is in the range of widthRange and the other is in the range of otherWidthRange. To specify the ranges, see Range Data Syntax .
-color	If given, the rule only applies if both shapes have this mask color. A value of zero means any color. A value of -1 means same color and a value of -2 means diff color.

Note:

For more information on defining range tables, see [Understanding and Interpreting 1-D, 2-D, and 3-D Table Values](#) section.

The warning message is displayed in the following situations:

- -spacingDirection is set to horizontal or vertical and -distanceMeasureType is set to euclidean.
- -widthLengthTableType is set to twoWidthParallelRunLengthTableType (3-D table) and the row/column of the constraint value table is in decreasing order.

In this example, the minSpacing constraint is specified on M1 layer with the simple value and defined with parameters:

Rule	Description	Value (in microns)
M1.SP.2	M1 - M1 Spacing	0.1



```
minSpacing M1 0.1 -distanceMeasureType manhattan
```

In this example, the minSpacing constraint is specified on M1 layer with the width depending value (1DTable):

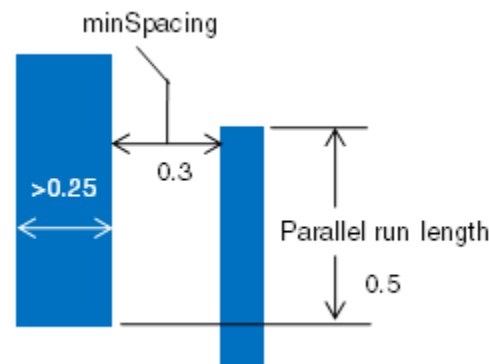
Rule	Description	Value (in microns)
M1.SP.2.1	M1 - M1 Spacing for M1 Width > 0.25u	0.2



```
minSpacing M1 0.1 ((0.25 0.2))
```

In this example, the `minSpacing` constraint is specified on LPP `m1/drawing` with the width length depending value (2DTable):

Rule	Description	Value (in microns)
M1.SP.2R	M1-M1 Recommended Spacing	0.3

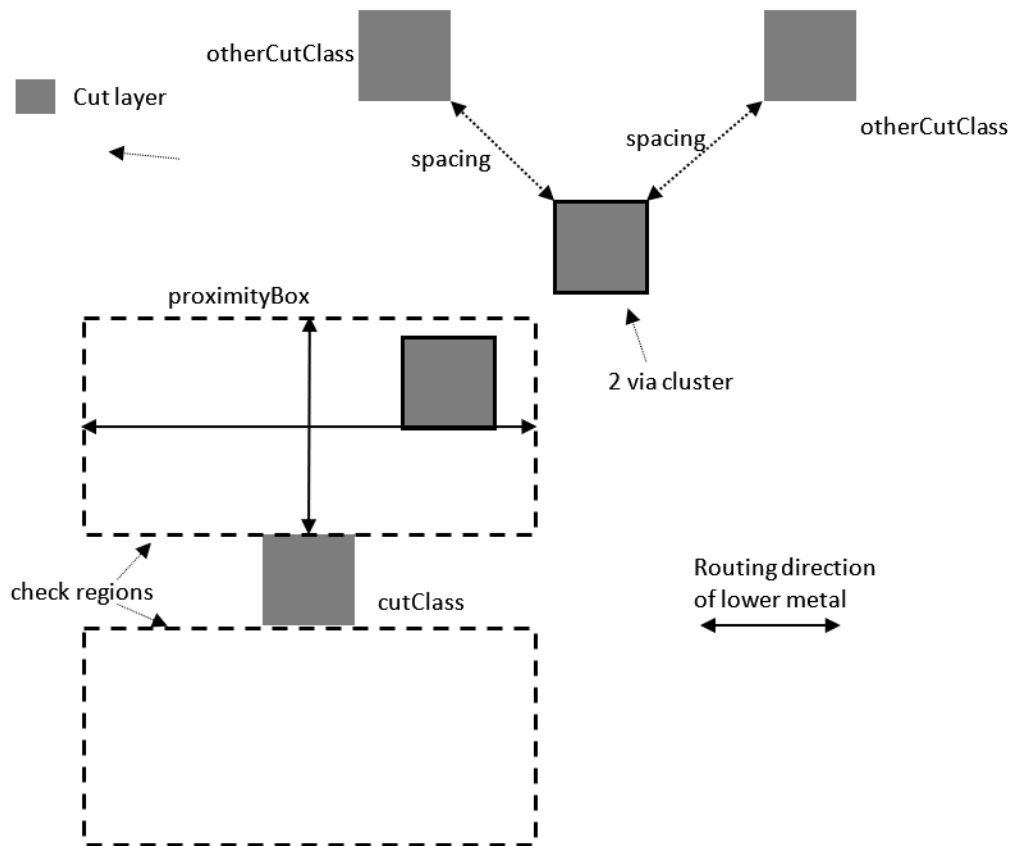


```
minSpacing (M1 drawing) 0.1 (((0.25 0.5) 0.3))
```

minViaRegionClusterSpacing

This layer constraint specifies the minimum spacing to via cluster in the region.

Advanced processes require a specific spacing from a 2 via cluster when one of the cut shapes in that cluster interacts with a check region. The check region is defined as a rectangle which abuts a cut shape of a certain size. Two check regions are involved which are parallel to the routing direction of the lower metal layer.



Syntax:

```
minViaRegionClusterSpacing
  <string|LPP>layer
  <float>spacingValue
  -cutClass <cutClass | cutClassName>
  [-otherCutClass <cutClass | cutClassName>]
  -proximityBox cutClassArray
  -clusterCutClass <cutClass | cutClassName>
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or a LPP.
spacingValue	float: Specifies the minimum via cut space on the same net or on the same metal.
-cutClass	Specifies the specific cut class for the main cut shape which defines the location of the check region. This is required.

Argument	Description
-otherCutClass	If specified, the specific cut class is required to be spaced away from the via cluster.
-proximityBox	Definition of the check region dimensions (width and height). This is required.
-clusterCutClass	Specifies to define the specific cut class of the via cluster.

```
minViaRegionClusterSpacing vl 0.02 -cutClass 0.2 0.6 -otherCutClass 0.04 0.06
-proximityBox 0.06 0.12 -clusterCutClass 0.04 0.08
```

minViaSpacing

This layer constraint specifies the via cut spacing for cuts on the same net or for cuts on the same metal.

You can specify whether to measure cut spacing from edge-to-edge or from center-to-center using an optional parameter. The constraint can apply to shapes having any connectivity, to shapes having only same-net connectivity, or only to cut shapes on the same metal based on an optional parameter.

You can also control whether cuts must have an area larger than a specified value before the spacing applies through another optional parameter.

Constraint Category	Via Construction
Constraint Type	Complex
OA Rule	oacMinViaSpacing

Syntax:

```
minViaSpacing
  <string|LPP>layer
  <float|float1DTable|float2DTable>spacingValue
  [-centerToCenter true|false]
  [-connectivityType
    anyConnectivityType|
    sameNetConnectivityType|
    contiguousShapesConnectivityType|
    directShapesConnectivityType|
    sameViaConnectivityType]
  [-area <float>]
  [-parallelRunLength float]
  [-exactParallelRunLength float]
  [-distanceMeasureType euclidian|manhattan]
```

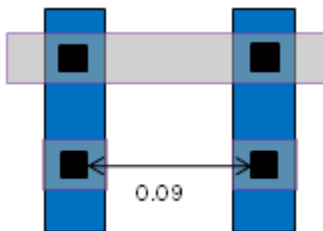
```
[-exactSpacing true|false]
[-spacingDirection any|horizontal|vertical]
[-cutClass <cutClassArray | cutClassNameList>]
[-otherLayer layerName]
[-otherLayerWidth rangeData]
[-isHard true|false]
[-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or a LPP.
spacingValue	float: Specifies the minimum via cut space on the same net or on the same metal. float1DTable: Specifies the width of the larger shape and the minimum via cut spacing required between the two shapes. float2DTable: Specifies the width of one of the shapes and the minimum via cut spacing required between the two shapes.
-centerToCenter	Specifies if cut spacing should be measured from the center of one cut to the center of the other cut. If not specified, the cut spacing is applied from the edge of one cut to the edge of the other cut. Default is "false".
-connectivityType	anyConnectivityType: Use this option to designate that the constraint applies to any type of connectivity (default). sameNetConnectivityType: Use this option to designate that the constraint only applies to cuts on the same net. contiguousShapesConnectivityType: Use this option to designate that the constraint only applies to cuts on the same metal. directShapesConnectivityType: Use this option to designate that the constraint only applies when the two cuts are part of the same metal shape on the upper or lower metal and the metal shape covers the region between the two cuts. sameViaConnectivityType: This is similar to directShapesConnectivityType except that both lower and upper metal shapes must cover the cuts and the region between the cuts.
-area	Specifies the cut area, which when exceeded, requires the specified clearance.
-parallelRunLength	Specifies whether the cut edges in database units (dbu) must be parallel: -1: Cut edges must not be parallel. 0: Cuts must have ≥ 0 parallel run length. 1: Cuts must have > 0 parallel run length. - All other values: Minimum parallel run length between two end-of-line edges must be greater than or equal to the value for the constraint to apply. If the value is $> +1$ dbu, then the value is the threshold ($>$) parallel run required for the constraint to apply. The default is 0. 1 dbu = 1 user unit/DBUpperUU (Database Units per User Unit). A user unit is usually 1um. For example, if dbuPerUU is 2000, then 1 dbu is calculated as $1\text{um}/2000 = 0.0005$.
-exactParallelRunLength	The constraint only applies if the parallel run is exactly this value. This parameter is mutually exclusive with parallelRunLength parameter.

Argument	Description
-distanceMeasureType	This parameter should be set to oacManhattanDistanceMeasureType for a max XY rule. Default is <i>Euclidian</i> .
-exactSpacing	When this parameter value is true, then the spacing has to be exactly the value of the constraint. This parameter would only be used together with <i>exactParallelRunLength</i> .
-spacingDirection	Direction for which this constraint applies. Default is any.
-cutClass	List of paired float values or a list of cutClass names. See cutClass and cutClassList . The constraint applies only in cases where the cuts comply with these specifications.
-otherLayer	The cut shapes must interact with material on the given layer, otherwise the constraint doesn't apply. The two cut shapes between which spacing is measured should interact with the same metal shape, so <i>connectivityType</i> should be <i>sameVia</i> or <i>directShape</i> .
-otherLayerWidth	If given along with otherLayer, the interacting material under the cut shapes must have a width in this range, otherwise the constraint doesn't apply. To specify the ranges, see Range Data Syntax .

In this example, the spacing between two VIA1 on the same net is mentioned:

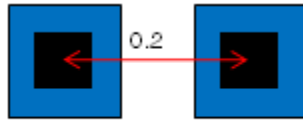
Rule	Description	Value (in microns)
VIA1.SP.1	VIA1 - VIA1 spacing when VIA1's are on the same net	0.09



```
minViaSpacing VIA1 0.09 -connectivityType sameNetConnectivityType
```

In this example, the center-center spacing between two VIA1s is mentioned:

Rule	Description	Value (in microns)
VIA1.SP.2	VIA1 - VIA1 center to center spacing	0.2



CenterToCenter=true

```
minViaSpacing VIA1 0.2 -centerToCenter true
```

minViaSpacing constraint specified with width depending value:

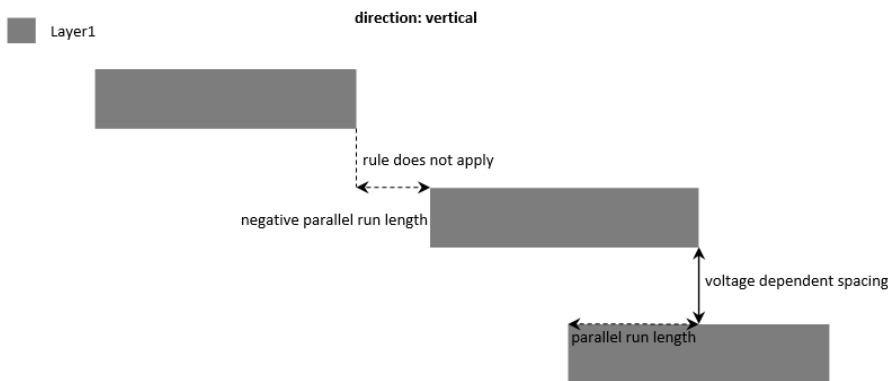
```
minViaSpacing v1 0.06 ((0.12 0.08) (0.15 0.10))
```

minViaSpacing constraint specified with two width depending value:

```
minViaSpacing v1 0.06 ( ((0.12 0.14) 0.08)
                        ((0.12 0.18) 0.08)
                        ((0.15 0.14) 0.08)
                        ((0.15 0.18) 0.10) )
-area 0.6
```

minVoltageSpacing

Some processes require different minimum spacing values in horizontal and vertical directions depending on the voltage difference between shapes on the same layer. This rule is modeled by built-in OA constraint `minVoltageSpacing` with new parameter `-spacingDirection`. By default, spacing is measured as Euclidian, and corner-to-corner spacing is enforced. However, if only horizontal or vertical direction is specified, the spacing is enforced only when the shapes are facing each other, that is, the parallel run length (in the orthogonal direction) between the shapes is greater than or equal to zero.



The constraint is used when you check for voltage-dependent spacing violations as you edit the design layout. The Custom Router also uses the constraint to determine violations.

Constraint Category	Spacing
Constraint Type	Complex
OA Rule	oacMinVoltageSpacing

Syntax:

```
minVoltageSpacing
  <string|LPP>layer
  <float1DTable>spacingValue
  [-spacingDirection any|horizontal|vertical]
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or a LPP.

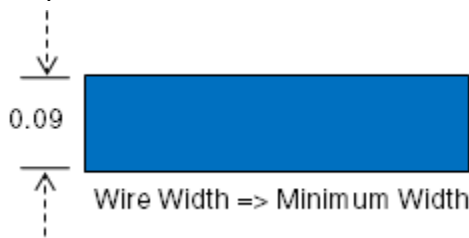
Argument	Description
spacingValue	The constraint value consists of an array of float pairs, where voltage is represented as a float and the spacing value is represented as a float in microns. The voltage index represents the maximum voltage difference between the two shapes. The spacingValue is represented as: (<float>voltageValue <float>spacingValue) The voltage value has a >= semantic, that specifies the required spacing which applies to all voltage differences greater than or equal to it.
-spacingDirection	Defines the spacing direction in which this constraint applies: only horizontal (1), only vertical (2) or any (0). Default: any

For example:

```
minVoltageSpacing m1 ((1.5 0.1)(1.8 0.2)) -spacingDirection any
// If voltage >=1.5 then spacing must be >= 0.1, voltage >=1.8 then spacing must
be >= 0.2.
```

minWidth

This layer constraint specifies the minimum orthogonal width for any geometry. The value is specified in microns.



Constraint Category	Length and Width Process Rules
Constraint Type	Complex
OA Rule	oacMinWidth

Syntax:

```
minWidth
    <string|LPP>layer
    <float>widthValue
    [-spacingDirection horizontal|vertical|any]
    [-exceptPointTouch true|false]          [-distanceMeasureType
euclidian|manhattan]          [-exceptLineTouch rangeValue]
    [-isHard true|false]
    [-comment [<string rule_id>] <string rule_desc>]
```


Argument	Description
layer	Layer can be defined as a string or a LPP.
widthValue	Specifies the minimum width for the shape.
-spacingDirection	horizontal: Specifies the horizontal spacing direction. vertical: Specifies the vertical spacing direction. any: Specifies the spacing direction that the constraint applies. Can be either horizontal or vertical (default).
-exceptPointTouch	If set to true, this Boolean parameter is specifies that a shape can point touch a second shape on the same layer at a corner. Default is <i>false</i> .
-distanceMeasureType	This parameter determines whether how spacing values are measured. Default value is <i>euclidian</i> .
-exceptLineTouch	This parameter is used to specify the length range within which the shapes are allowed to share the common line touch. To specify the ranges, see Range Data Syntax .

Rule	Description	Value (in microns)
M1.W.EX.1	Minimum M1width	0.09
M1.W.1R	Recommended M1 width	0.1

```
minWidth M1 0.09
minWidth M1 0.1 -isHard false
```

minWireExtension

This layer constraint specifies the minimum distance a wire is required to extend past a via cut. The distance is specified in microns.

Wire extensions are measured from the center of the via cut to the inside edge of the wire shape.



The constraint definition applies to both cut layers, the one below and the one above.

Constraint Category	Extension
Constraint Type	Complex

OA Rule oacMinWireExtension

Syntax:

```
minWireExtension
    <string|LPP>layer
    <float>extensionValue
    [-coincidentAllowed true|false]
    [-distance float]
    [-isHard true|false]
    [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or a LPP.
extensionValue	Specifies the minimum distance that the inside edge of a wire shape contacting a via must extend past the center of that via shape.
-coincidentAllowed	When this parameter is specified and is set to true, the shapes can either meet the minimum extension or their edges can be coincident else the edges meet the minimum extension specified and cannot be coincident.
-distance	if the value is greater than 0, the constraint is applied only to the extensions on a layer on which neighboring shapes are located within the specified distance.

In this example, the extension of M1 wire from the center of the Contact is specified:

Rule	Description	Value (in microns)
M1.W.EX.1	M1 wire extension from center of contact	0.08

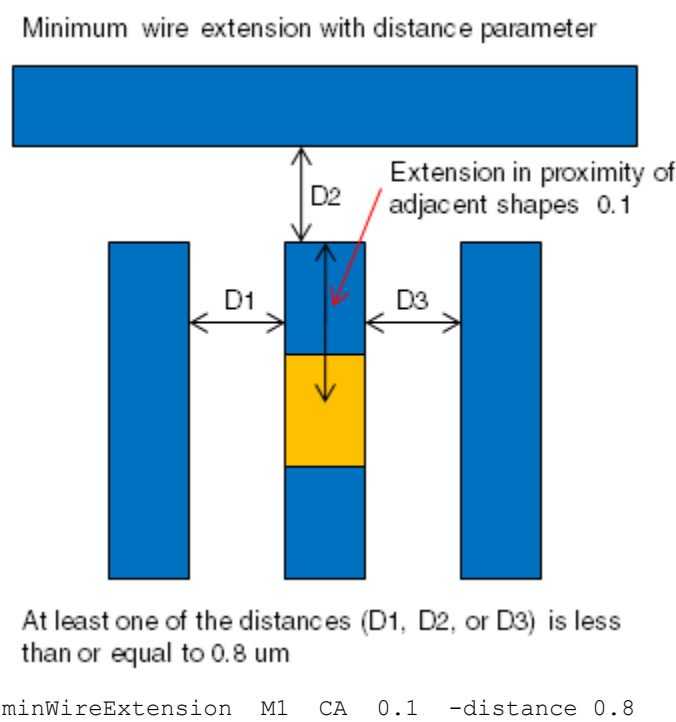
Minimum wire extension 0.08



```
minWireExtension M1 CA 0.08
```

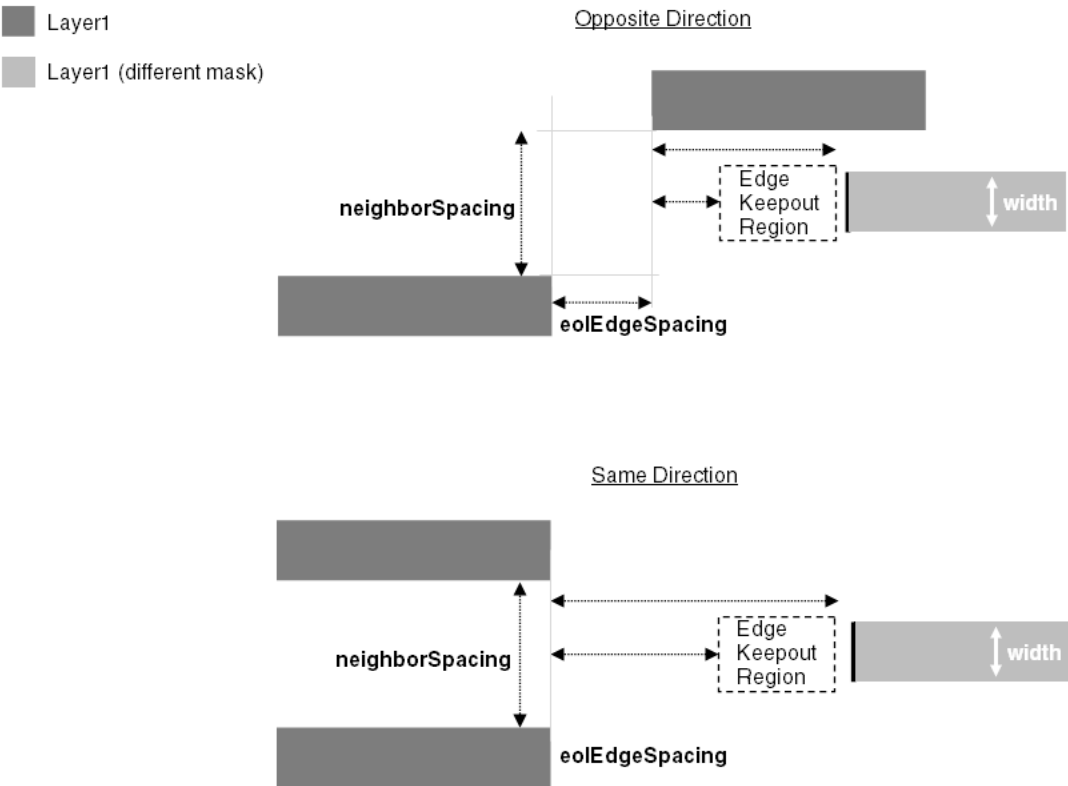
In this example, the extension of M1 wire from the center of the Contact is specified when there is at least 1 neighboring M1 shape at a distance less than 0.8um:

Rule	Description	Value (in microns)
M1.W.EX.2	M1 wire extension from center of contact when there is at least 1 M1 shape at a distance < 0.8um from this	0.1



multiEndOfLineEdgeKeepout

Advanced processes have limitations for line ends that are within proximity to other line ends.



Constraint Category	Spacing
Constraint Type	Complex
OA Rule	Custom Compiler Custom Layer Constraint

Syntax:

```
multiEndOfLineEdgeKeepout
  <string|LPP>layer
  <rangeData>edgeKeepoutSpacing
  -width float
  -neighborSpacing rangeData
  -eolEdgeSpacing rangeData
  [-sameDirection true|false]
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer on which the line end is placed. <code>layer</code> can be a string (layer name) or a LPP.

Argument	Description
edgeKeepoutSpacing	Keepout spacing for the end-of-line edge. The spacing must be within the specified range. See Range Data Syntax .
-width	Maximum width, in microns, of the end-of-line wire. The width must be less than or equal to the specified value.
-neighborSpacing	Maximum side-to-side distance, in microns, between the two adjacent wires that are on the same mask. The distance must be within the specified range. See Range Data Syntax .
-eolEdgeSpacing	Maximum end-to-end distance, in microns, between the two wires that are on the same mask. The distance must be within the specified range. See Range Data Syntax .
-sameDirection	(Optional) Wires must be in the same direction (true) or opposite direction (false) for the constraint to apply. Default: true

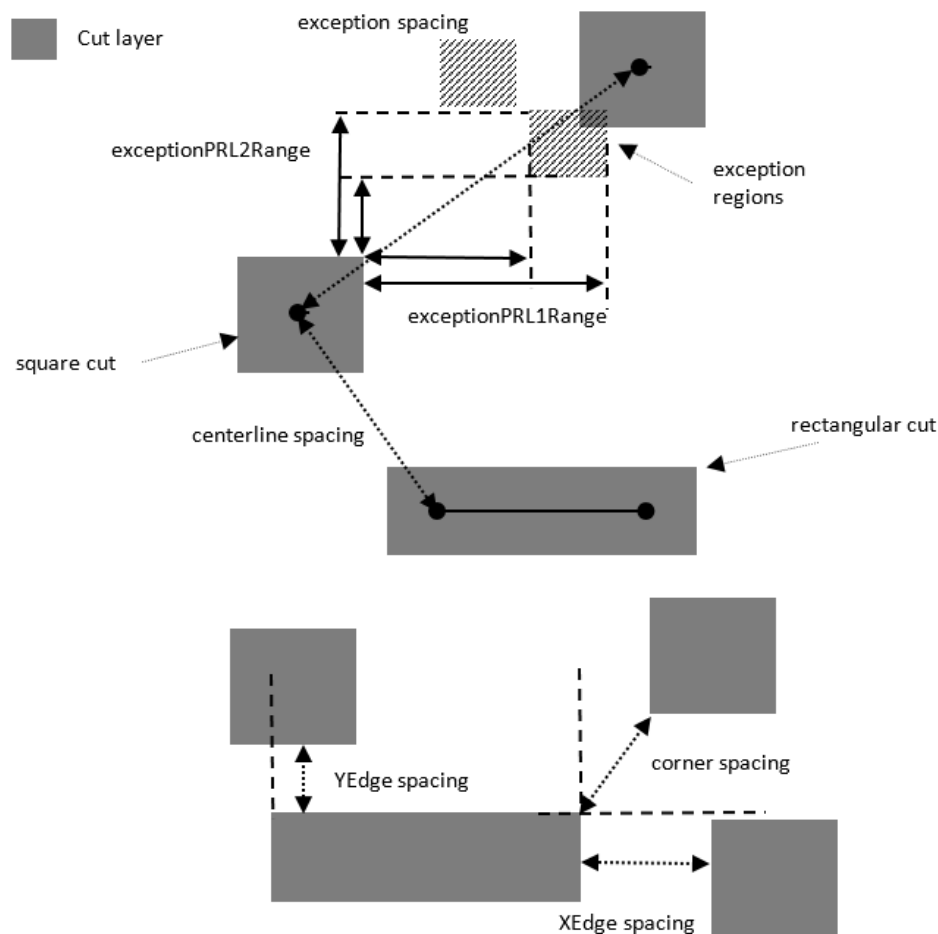
Exmample:

```
multiEndOfLineEdgeKeepout M1 (>=0.09 <=0.15)
    -width 0.55
    -neighborSpacing (>=0.05 <=0.25)
    -eolEdgeSpacing (>=0.12 <=0.23)
    -sameDirection false
```

multiPatternCutSpacing

This layer constraint specifies multiple pattern cut violations. Advanced processes now support multiple masks for cut layers. The determination of whether two cuts are too close to exist on the same mask is done by using the centerline of the cut shape. The centerline is defined by a line or point that is half width perpendicular distance from each edge of the cut. For square cuts, the centerline is a dot. Centerline spacing between two cuts must be greater than a specific value in order for those two cut to be allowed on the same mask. The centerline spacing is dependent on the cut sizes.

In addition to centerline spacing, the constraint also supports corner-to-corner spacing, as the following diagram shows.



Constraint Category	Spacing
Constraint Type	Complex
OA Rule	Custom Compiler Custom Layer Constraint

Syntax:

```
multiPatternCutSpacing
  <string|LPP>layer
  <2DTable>spacingValue
  -cutClassList cutClassArray|cutClassNameList
  -spacingType centerline|XEdge|YEdge|corner
  [-exceptionPRL1Range (rangeData)]
  [-exceptionPRL2Range (rangeData)]
  [-exceptionSpacing (float)]
  [-maxExceptions (float)]
```

```
[-isHard true|false]
[-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or a LPP.
spacingValue	Two dimensional table of spacing values where the rows and columns represent cut class size indexes and the table values represent the centerline to centerline spacing between two specific cut classes. The table size of NxN where N is the number of elements in the <code>cutClassList</code> value array. 2-D table: <pre>default<float>((cutClassIndex1<int> cutClassIndex2<int>) spacing<float>*)) [2DTableOptions]</pre>
-cutClassList	An ordered array of cut dimension pairs. The array is used to determine the key to index into the constraint 2-D table value. This is required value. The <code>cutClassList</code> is represented as: <pre>cutClassArray<float1> <float2> cutClassName<string></pre> Note: If <code>cutClass</code> parameter is specified with <code>cutClassName</code> in the ASCII file, the technology importer looks for the first <code>cutClass</code> constraint specified in the same hierarchical constraint group to convert to cut class size. If no <code>cutClass</code> constraint exists, an error message is displayed. While exporting, the technology exporter looks for the first <code>cutClass</code> constraint specified in the same hierarchical constraint group to exists and converts the cut class size to cut class name. If no <code>cutClass</code> constraint exists, the <code>cutClass</code> parameter is exported with cut class size in dual value.
-spacingType	Type of via spacing constraint. Valid values are centerline, XEdge, YEdge, and corner. - centerline: Uses via centerlines to determine the required distance between vias. - XEdge: Determines the distance between vias along the X direction. - YEdge: Determines the distance between vias along the Y direction. - corner: Determines the corner-to-corner distance between vias.
-exceptionPRL1Range	Range of X distances between the corner of a cut and the exception region.: (X1 range) (X2 range). X1 and X2 are the distances between the corner of the cut and the start and end points of the exception region along the X direction, respectively. The constraint does not apply when the X distances are within these ranges. To specify the ranges, see Range Data Syntax.
-exceptionPRL2Range	Range of Y distances between the corner of a cut and the exception region.: (Y1 range) (Y2 range). Y1 and Y2 are the distances between the corner of the cut and the start and end points of the exception region along the Y direction, respectively. The constraint does not apply when the Y distances are within these ranges. To specify the ranges, see Range Data Syntax.

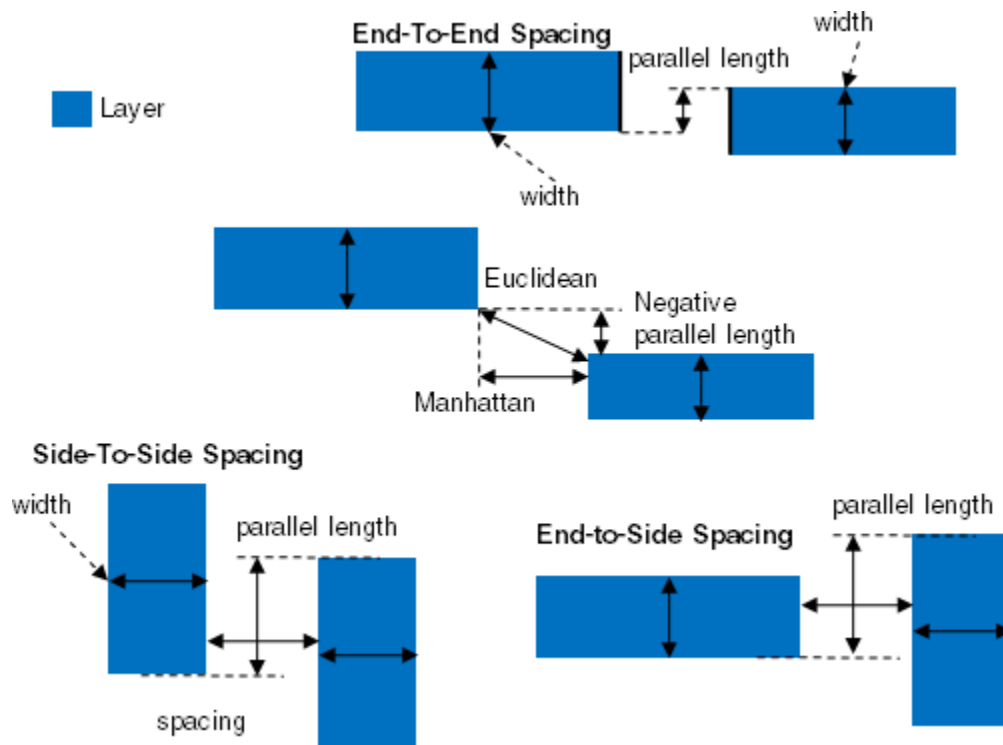
Argument	Description
-exceptionSpacing	Centerline-to-centerline distance between two cuts at which the constraint does not apply.
-maxExceptions	Maximum number of exception spacings that is permitted per cut.

For example,

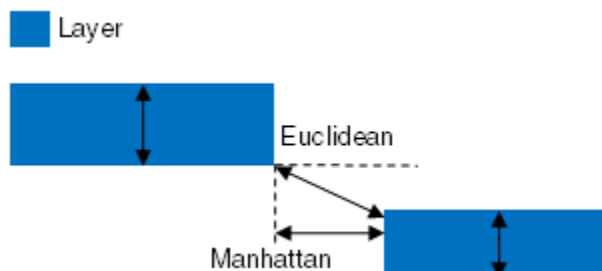
```
multiPatternntCutSpacing v1
  ((0 0) 0.02) -spacingType centerline
  -exceptionPRL1Range ((>=0.11 <0.12) (>=0.05 <=0.07))
  -exceptionPRL2Range ((>0.13 <0.15) (0.06 0.08))
  -exceptionSpacing 0.15
  -maxExceptions 0.79)
  ((0 0) 0.03 -spacingType XEdge
  -exceptionPRL1Range ((>=0.11 <=0.15))
  -exceptionPRL2Range ((>=0.13 <=0.15))
  -exceptionSpacing 0.15
  -maxExceptions 0.39)
  ((0 0) 0.01 -spacingType YEdge
  -exceptionPRL1Range ((>0.25 <0.28))
  -exceptionPRL2Range ((>=0.12 <0.15) (>0.47 <0.59))
  -exceptionSpacing 0.76
  -maxExceptions 0.95)
  ((0 0) 0.13 -spacingType corner
  -exceptionPRL1Range ((>=0.45 <0.67) (>=0.55 <0.58))
  -exceptionPRL2Range ((>0.15 <0.19))
  -exceptionSpacing 0.33
  -maxExceptions 0.77)
  -cutClassList ((0.06 0.06) (0.06 0.08))
```

multiPatternSpacing

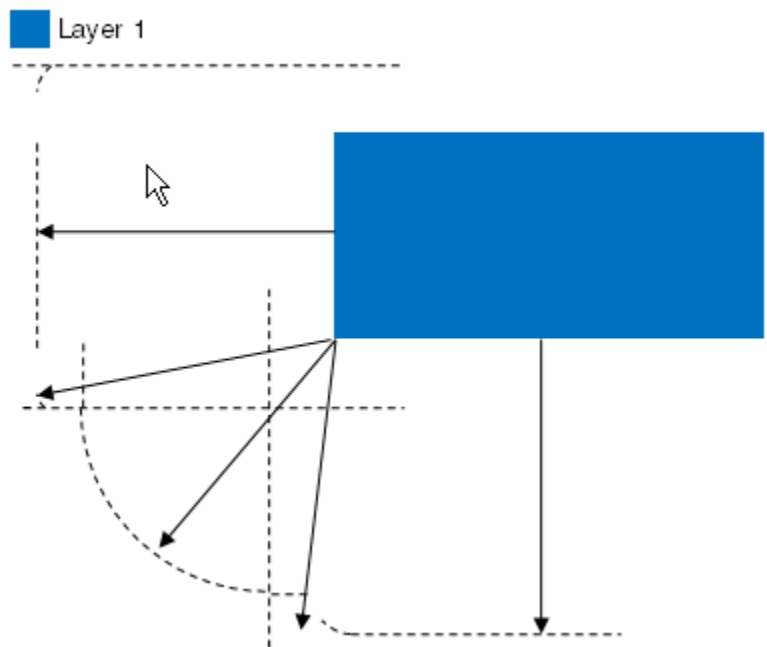
This layer constraint specifies multiple patterning violations. Each violation is considered an edge in a graph with the offending shapes as nodes. If the graph contains odd cycles, then it is impossible to produce the two masks from the layer and what is called a multi-patterning violation exists.



Corners beyond the negative parallel run length for edge spacing must be separated by a minimum value measure either Manhattan or Euclidean.



The following illustrations shows how corner spacing is to be applied. Note that the corner spacing begins beyond any negative parallel run length from end to end, end to side, or side to side spacing rules.



This constraint contains an array of four `oaValues` defining the spacing requirements for end-to-end, end-to-side, side-to-side, and corner spacing respectively. Each value can be a simple `oaIntValue` specifying the spacing in microns. The second element (end-to-side spacing) can also be a 1-D spacing table in microns that are dependent on the width, in microns, of the side shape. The third element (side-to-side spacing) can also be a 2-D spacing table with rows representing width thresholds and columns representing parallel run length thresholds. The larger width between the two shapes must be used. An optional fifth element of type integer can exist to define the joint-to-joint spacing between two joints with parallel overlap less than or equal to zero. A joint is a side with length less than or equal to `jointWidth`, span greater than or equal to `spanLength`, EOL width less than or equal to `jointMaxEOL`, and edges adjacent to end greater than or equal to `minLength`.

Constraint Category	Spacing
Constraint Type	Complex
OA Rule	Custom Compiler Custom Layer Constraint

Syntax:

```
multiPatternSpacing
  <string|LPP>layer
  <float>endToEndSpacing
  <float>|<1DTable>endToSideSpacing
  <float>|<1DTable>|<2DTable>sideToSideSpacing
  <float>cornerSpacing
  [<float>jointToJointspacing])
```

```

[-width float]
[-minLength float]
[-endToEndPRL float]
[-endToSidePRL float]
[-sideToSidePRL float]      [-distanceMeasureType
euclidean|manhattan]
[-jointWidth float]
[-spanLength float]
[-jointMaxEOL float]
[-jointEOLEnLength float]
[-sideFatToThin array]
[-sideFatToFat array]
[-colorWidths array]
[-colorMinLengths array]
[-colorEndToEndPRLs array]
[-colorEndToSidePRLs array]
[-isHard true|false]
[-comment [<string rule_id>] <string rule_desc>]

```

Note:

The `-sideFatToThin` and `-sideFatToFat` arrays require four elements: `width1`, `width2`, `runLength1`, and `spacing`.

Argument	Description
layer	Layer can be defined as a string or a LPP.
-width	Specifies the width threshold in microns for determining whether an edge is an end (< width) or a side (>= width).
endToEndSpacing	End-to-end spacing value.
endToSideSpacing	The endToSideSpacing value is represented as: default<float>((width<float> spacing<float>)*) [1DTableValues] // width dependent 1D table.
sideToSideSpacing	The sideToSideSpacing value is represented as a float value or 1-D table: default<float>((width<float> spacing<float>)*) [1DTableOptions] // width dependent 1D table. 2-D table: default<float>((width<float> length<float> spacing<float>)*)) [2DTableOptions] // width dependent 1D table. //rows for width thresholds, column for parallel run length thresholds
cornerSpacing	Corner spacing value.
jointToJointsacing	Joint-to-joint spacing value.
-minLength	Optional value in microns defining the minimum edge length (>=minLength) in which endToEndSpacing applies.

Argument	Description
-endToEndPRL	Optional value in microns to specify the amount of parallel run length required before the endToEndSpacing applies. Note, the value can be negative which indicates an 'extension' from the corner in which to check.
-endToSidePRL	Optional value in microns to specify the amount of parallel run length required before the endToSideSpacing applies. Note, the value can be negative which indicates an 'extension' from the corner in which to check.
-sideToSidePRL	Optional value in microns to specify the amount of parallel run length required before the sideToSideSpacing applies. Note, the value can be negative which indicates an 'extension' from the corner in which to check.
-distanceMeasureType	Defines how the spacing is to be measured, either euclidean or manhattan. Default is manhattan.
-jointWidth	Defines the maximum length for a joint. Only applicable if a fifth spacing element is in the constraint value array, defines the maximum length for a joint.
-spanLength	Defines the minimum span length. Only applicable if a fifth spacing element is in the constraint value array, defines minimum span ($\geq$) required for the joint spacing to apply.
-jointMaxEOL	Defines the maximum EOL edge length. Only applicable if a fifth spacing element is in the constraint value array, defines the maximum EOL edge length for the joint spacing to apply.
-jointEOLLength	Defines the minimum length for edges adjacent to EOL edge. Only applicable if a fifth spacing element is in the constraint value array, defines the minimum length ($\geq$) required for edges adjacent to EOL edge.
-sideFatToThin	Defines additional side spacing requirements. One width greater than or equal to <code>width1</code> , second shape width less than or equal to <code>width2</code> , with parallel run greater than or equal to <code>runLength</code> .
-sideFatToFat	Defines additional side spacing requirements. One width greater than or equal to <code>width1</code> , second shape width greater than or equal to <code>width2</code> , with parallel run greater than or equal to <code>runLength</code> .
-colorWidths	Widths of colored masks. The first element in the array must be color 1, the second element must be color 2, and so on. The number of elements in the array must match the number of color names defined in the Controls Section . If you specify this argument, <code>-width</code> is ignored.
-colorMinLengths	Minimum length ($\geq$ minLength) of the colored mask in which endToEndSpacing applies. Specify (colorName minWidth) for each color defined in the Controls Section . If you specify this argument, <code>-minLength</code> is ignored.
-colorEndToEndPRLs	The parallel run lengths of colored masks that are required before endToSideSpacing applies. Specify (colorName endToEndPRL) for each color defined in the Controls Section . If you specify this argument, <code>-endToEndPRL</code> is ignored.

Argument	Description
-colorEndToSidePRLs	The parallel run lengths of colored masks that are required before the endToSideSpacing applies. Specify (colorName endToSidePRL) for each color defined in the Controls Section . If you specify this argument, -endToSidePRL is ignored.

For example,

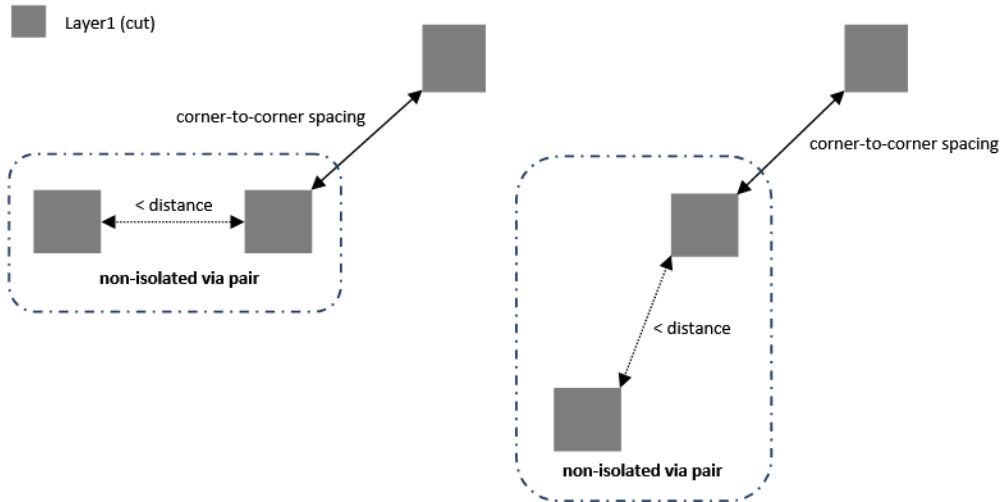
```
multiPatternSpacing m1 (0.06 0.06 0.08 0.08)
                    -width 0.06
                    -minLength 0.10
                    -endToEndPRL 0.04
                    -endToSidePRL 0.04
                    -sideToSidePRL 0.08
                    -distanceMeasureType manhattan
                    -sideFatToThin (0.03 0.05 0.07 0.09)

multiPatternSpacing m1 ((0.04 0.05) (0.03 0.07) (0.002 0.005))
                    -colorWidths ((red 0.02) (blue 0.04))
                    -colorMinLengths ((red 0.03) (blue 0.05))
                    -colorEndToEndPRL ((red 0.01) (blue 0.06))
                    -colorEndToSidePRL ((red 0.034) (blue 0.046))

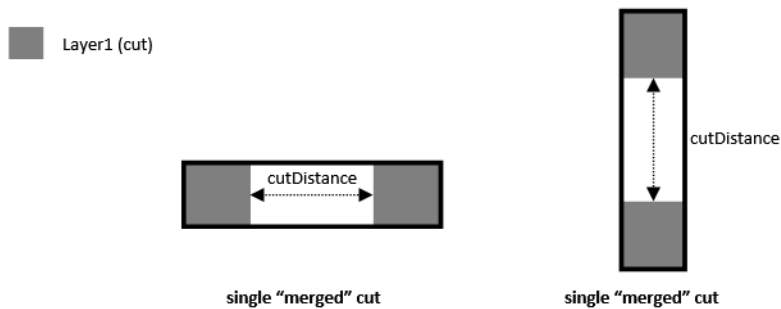
multiPatternSpacing m1 (0.06
                    (0.02 (0.12 0.08) (0.15 0.10))
                    (0.06 ((0.12 0.14) 0.08)
                    ((0.12 0.18) 0.08)
                    ((0.15 0.14) 0.08)
                    ((0.15 0.18) 0.10) )
                    0.08 0.08)
                    -width 0.06
                    -minLength 0.10
                    -endToEndPRL 0.04
                    -endToSidePRL 0.04
                    -sideToSidePRL 0.08
                    -distanceMeasureType manhattan
                    -jointWidth 0.04
                    -spanLength 0.06
                    -jointMaxEOL 0.04
                    -jointEOLLength 0.04
```

nonIsolatedViaCornerSpacing

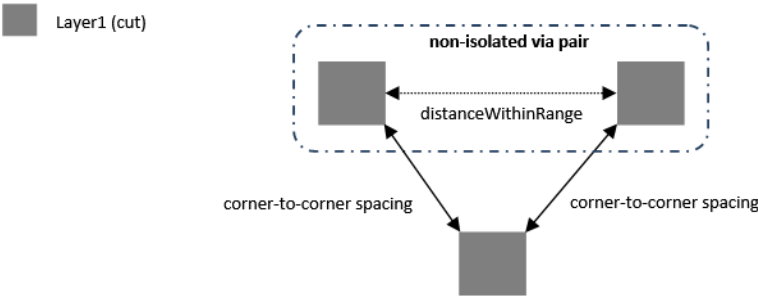
Spacing between via cut shapes may be different for isolated and non-isolated vias. An isolated via is a via that is spaced away from all other vias in all directions. For example, the distance to any other via is greater than or equal to a particular threshold (distance). A non-isolated via pair is a pair of vias with distance between them less than the threshold. This rule defines minimum corner-to-corner spacing allowed between a non-isolated via and another vias.



For the purpose of this rule, any two fully aligned via cut shapes with a particular exact distance between them ($oaCutDistance$) are considered a single merged cut with bounding box equal to the combined bounding boxes of the two original cuts, and all measurements are performed against the merged cut.



Optionally, this rule can be applied only to non-isolated via pairs oriented in specific direction and other vias facing the non-isolated via pair. If the $oaSpacingDirection$ parameter is horizontal or vertical, the rule only applies to non-isolated via pairs oriented in this direction and facing each other, and to other via which face this non-isolated via pair. For example, the direction is horizontal, vias in the non-isolated pair face each other (their projections to Y-axis overlap each other) and the other via face the non-isolated via pair (their projections to X-axis overlap each other).



Constraint Category	Spacing
Constraint Type	Complex
OA Rule	Custom Compiler Custom Layer Constraint

Syntax:

```
nonIsolatedViaCornerSpacing
  <string|LPP>layer
  <float>spacingValue
  -distance floatValue|-distanceWithinRange rangeData
  [-cutDistance floatValue]
  [-spacingDirection any|horizontal|vertical]
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	layer can be a string (layer name) or LPP.
spacingValue	Minimum corner-to-corner spacing from the non-isolated via to another via in database units. The corner-to-corner spacing applies only if the vias do not face each other. For example, both horizontal and vertical parallel run lengths are negative.
-distance	Distance threshold in database units to determine if a via is isolated or non-isolated. Either -distance or -distanceWithinRange must be specified.
-distanceWithinRange	Via pair is considered non-isolated if the distance between the vias is in this range. Either -distance or -distanceWithinRange must be specified.
-cutDistance	This is an optional parameter defining the exact distance between two fully aligned via cuts at which these two cuts are considered a single merged cut with bounding box equal to the combined bounding boxes of the two original cuts.

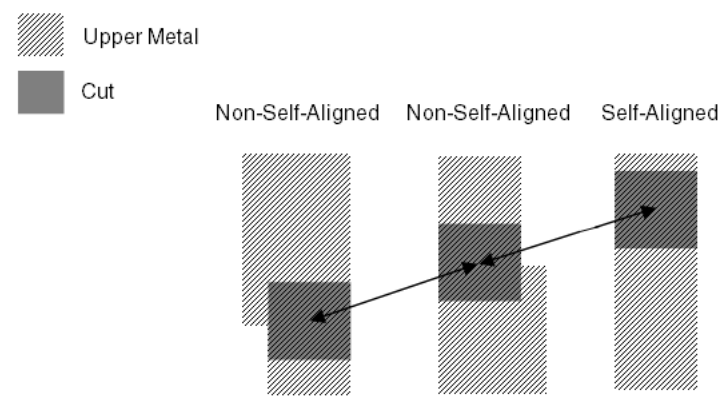
Argument	Description
-spacingDirection	This is an optional parameter defining the direction of non-isolated via pairs to which the constraint applies. 0: Any direction. 1: Horizontal direction. 2: Vertical direction.

Example:

```
nonIsolatedViaCornerSpacing v1 0.09
                             -distance 0.06 -cutDistance 0.07
```

nonSelfAlignedViaSpacing

Some processes have different spacing requirements for cut shapes in vias that are not self-aligned. The `nonSelfAlignedViaSpacing` constraint requires a minimum center-to-center spacing between a non-self-aligned via cut and another cut on the same layer. The neighboring via can be either a self-aligned via or a non-self-aligned via.



Constraint Category	Spacing
Constraint Type	Complex
OA Rule	Custom Compiler Custom Layer Constraint

Syntax:

```
nonSelfAlignedViaSpacing
  <string|LPP>layer
  <float>spacingValue
  [-cutClass1 float1 float2|<string>cutClassName]
  [-cutClass2 float1 float2|<string>cutClassName]
```



```
[-isHard true|false]
[-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer on which the via is placed. <code>layer</code> can be a string (layer name) or LPP.
-spacingValue	Minimum center-to-center distance, in microns, between the cuts.
-cutClass1	(Optional) Cut class of the non-self-aligned via cut shape.
-cutClass2	(Optional) Cut class of the neighboring via cut shape.

Example:

```
nonSelfAlignedViaSpacing v1 0.75
                        -cutClass 0.04 0.06 -cutClass2 0.12 0.16
```

onlyLargestEnclosedCutClassAllowed

Some processes only allow the largest cut class that can fit into the overlapping region between metal layer shapes to be used for some certain layers.

Constraint Category	Spacing
Constraint Type	Complex
OA Rule	Custom Compiler Custom Layer Constraint

Syntax:

```
onlyLargestEnclosedCutClassAllowed
<string|LPP>layer
<bool>allowedValue
[-isHard true|false]
[-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or a LPP.
allowedValue	If true, only the largest cut class that can fit into the overlap region is allowed.

Example:

```
onlyLargestEnclosedCutClassAllowed v12 true
```

onlyRectanglesAllowed

Lower process nodes have rigid requirements on shapes on some layers to guarantee proper manufacturing. One of the rule is that only rectangular shapes are allowed for a given layer. This requirement can be applied to all shapes on the layer, or to a subset of wires depending on their mask color, orientation, and width.



Constraint Category	Spacing
Constraint Type	Complex
OA Rule	Custom Compiler Custom Layer Constraint

Syntax:

```
onlyRectanglesAllowed
  <string|LPP>layer
  <bool>allowedValue
  [-widthRanges rangeData]
  [-spacingDirection any|horizontal|vertical]
  [-color colorName|any]

  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or a LPP.
allowedValue	If true, shapes on the layer must be rectangle. If false, all shapes are allowed.

Argument	Description
-widthRanges	Optional parameter specifying an array of one or more integer ranges in database units. If given, the constraint applies only to wires that have a segment with width that belongs to one of these ranges. Default: any.
-spacingDirection	Optional parameter specifying the direction in which the width of a wire segment is measured. Valid Values are 0 (any), 1 (horizontal), 2 (vertical). Non-default values can be specified only if the -widthRanges parameter is also specified. Default: any. Note: • If the direction is horizontal or vertical, -widthRanges parameter must also be specified, otherwise, it is an error. • For vertical wires, the width is measured in horizontal direction, and for horizontal wires, the width is measured in vertical direction.
-color	If given, this constraint only applies to shapes on the given color mask (positive value).

For example,

```
onlyRectanglesAllowed m1 true -widthRanges ((>=0.1 <=0.2) (0.5 0.6) (0.7 <=0.9))
```

onTrackRoutingOnly

This layer constraint specifies that only on track routing is allowed for a given layer.

Constraint Category	Routing
Constraint Type	Simple
OA Rule	Custom Compiler Custom Layer Constraint

Syntax:

```
onTrackRoutingOnly
  <string|LPP>layer
  <bool>allowedValue
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

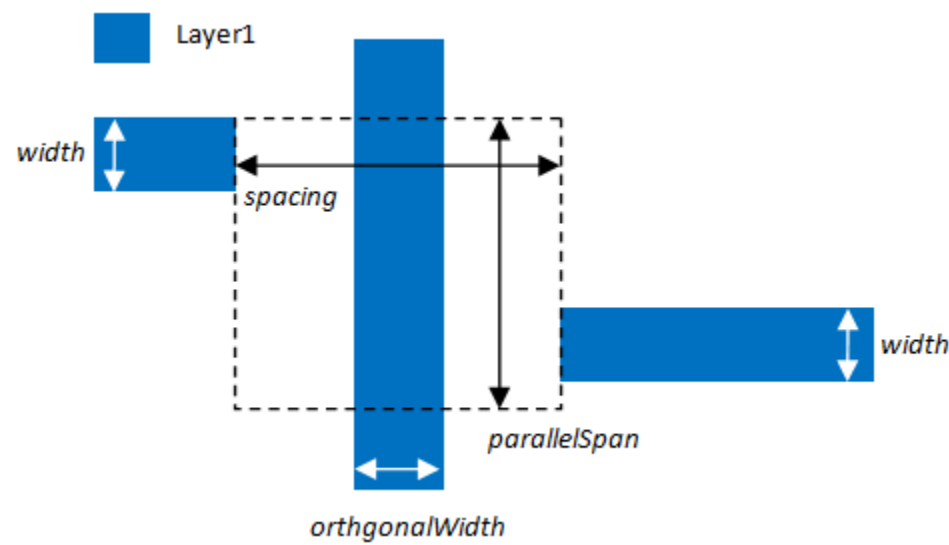
Argument	Description
layer	Layer can be defined as a string or a LPP.
-allowedValue	If true, on tack routing on the layer is allowed.

For example,

```
onTrackRoutingOnly m1 true
```

oppositeEndOfLineSpacing

This layer constraint specifies the spacing between two wires separated by a third orthogonal wire to have a minimum orthogonal spacing when the parallel span is less than a certain distance.



Constraint Category	Spacing
Constraint Type	Complex
OA Rule	oacOppositeEndOfLineSpacing

Syntax:

```
oppositeEndOfLineSpacing
  <string|LPP>layer
  <float>spacingValue
  -parallelSpan <float>
  -width <float>
  -orthogonalWidth <float>
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or a LPP.

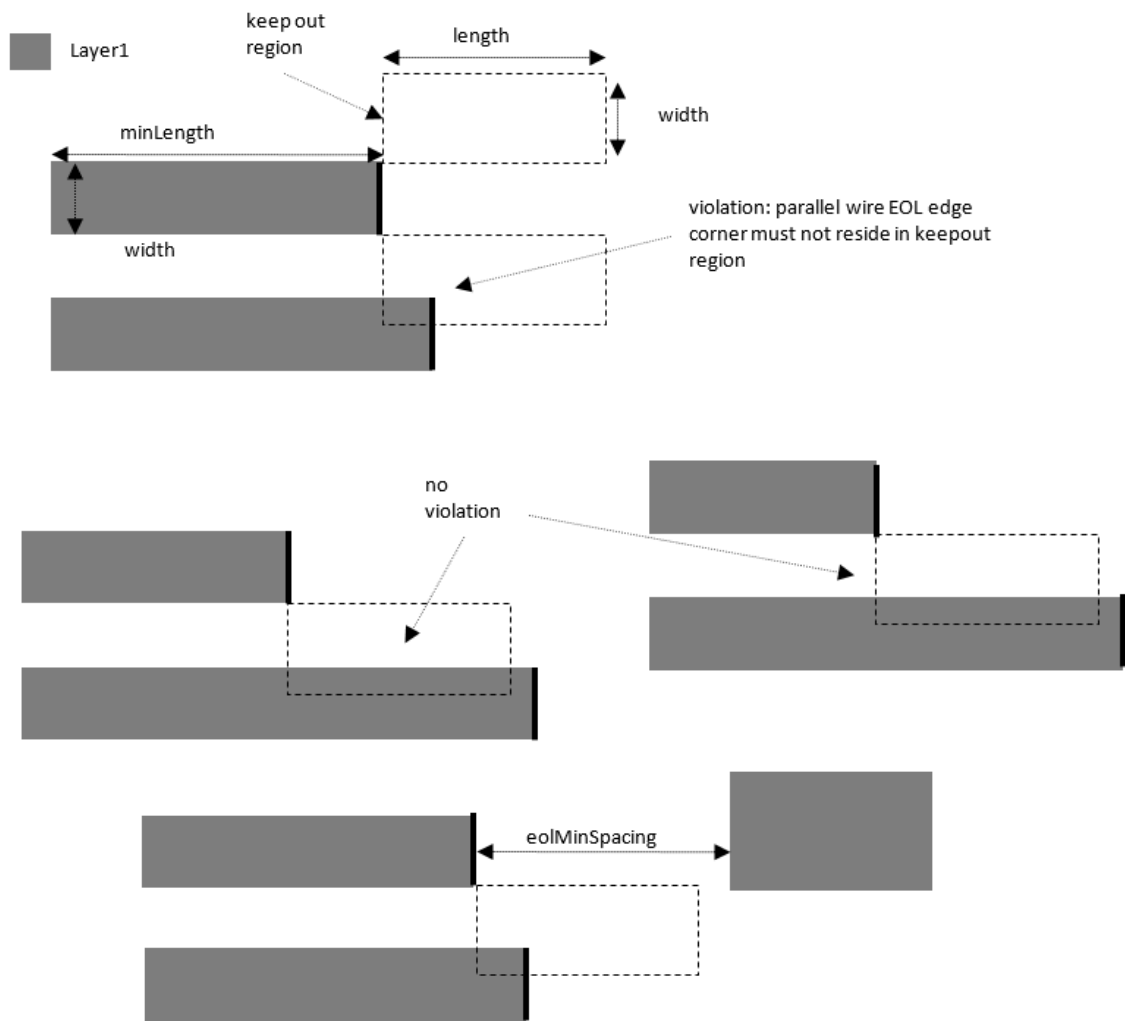
Argument	Description
spacingValue	Specifies the spacing required between end of line edges on opposite sides of an orthogonal wire.
-parallelSpan	The maximum parallel span, measured from the outer most edges of the two shapes. Value is in microns and must be greater than zero.
-width	The width of both approaching wires must be less than or equal to this value for the constraint to apply. The value is in microns and must be non-negative.
-orthogonalWidth	The width of the orthogonal wire must be less than this value for the constraint to apply. The value is in microns and must be non-negative.

For example:

```
oppositeEndOfLineSpacing m1 0.40
                        -parallelSpan 0.8
                        -width 0.10
                        -orthogonalWidth 0.10
```

parallelEndOfLineKeepout

This layer constraint specifies the keepout width and length. Advanced processes have restrictions on end of line edges from parallel wires that are in close proximity. The end of line edge corners from shapes parallel to the given shape must not reside in a keepout region which is defined by a width and length away from the given shape's end of line edge corner.



Constraint Category	Spacing
Constraint Type	Complex
OA Rule	Custom Compiler Custom Constraint

Syntax:

```
parallelEndOfLineKeepout
  <string|LPP>layer
  <float>width <float2>length
  -width float
  [-minLength float]
  [-color any|same|diff]
  [-eolMinSpacing float]
```

```
[-isHard true|false]
[-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or a LPP.
width length	Specify the values representing the keepout width and length respectively.
-width	The constraint applies only when the width of the EOL edge is less than or equal to this value.
-minLength	The length of wire must be greater than or equal too this value for the constraint to apply. The default value is 0.
-color	Defines the specific color mask this constraint applies to. A value of <i>any</i> means any color for either shape, <i>same</i> means that the shapes must be on the same color, and <i>diff</i> means the shapes must be on different colors. Default is <i>any</i> . Caution: You must specify the <i>colorName</i> in the Controls Section > colors group and the color mask value cannot be greater than the <i>numberOfColorMasks</i> value specified in the Layer Rules Section > functions group.
-eolMinSpacing	Minimum distance between the end of wire and the adjacent shape on the same layer.

For example:

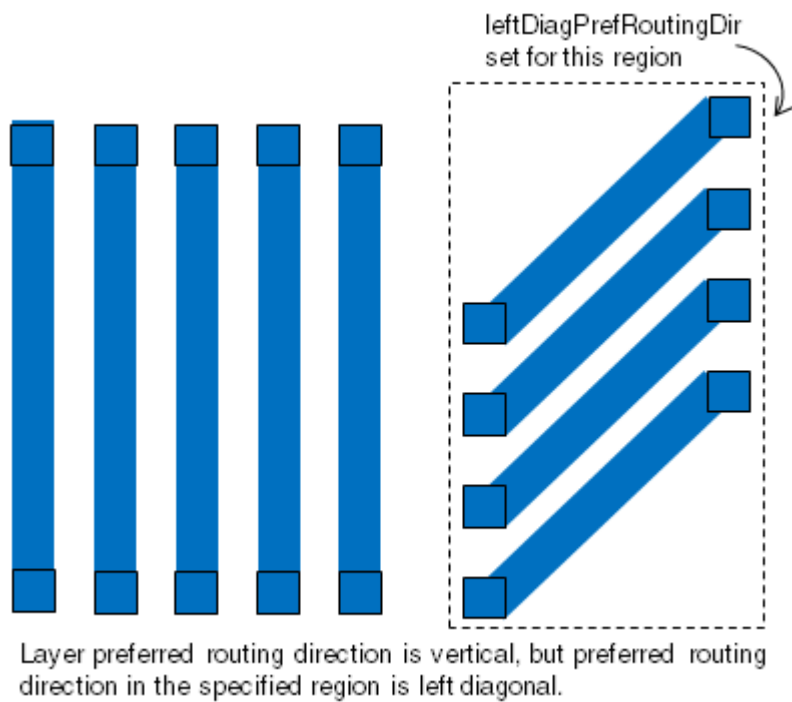
```
parallelEndOfLineKeepout m1 0.25 0.75 -width 0.25
                                         -minLength 0.45
                                         -color green
                                         -eolMinSpacing 0.5
```

preferredRoutingDirection

This layer constraint specifies the preferred routing direction. The routing directions can be one of the one of the orthogonal or one of the diagonal routing directions.

Note:

If this constraint is set to one of the diagonal routing directions, the [preferredRoutingOnly](#) constraint must be set to true.



Constraint Category	Routing
Constraint Type	Complex
OA Rule	oacPreferredRoutingDirection

Syntax:

```
preferredRoutingDirection
  <string|LPP>layer
  <value notApplicable|none|horizontal|vertical|
    leftDiag|rightDiag>directionValue
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or a LPP.
directionValue	• notApplicable: Specifies there is no preferred routing direction • none: Specifies routing is allowed in all directions • horizontal: Specifies horizontal routing is allowed • vertical: Specifies vertical routing is allowed • leftDiag: Specifies left diagonal routing is allowed • rightDiag: Specifies right diagonal routing is allowed

For example:

```
preferredRoutingDirection m1 vertical
```

preferredRoutingOnly

This layer constraint specifies that only preferred routing is allowed for a given layer.

Constraint Category	Routing
Constraint Type	Simple
OA Rule	Custom Compiler Custom Layer Constraint

Syntax:

```
preferredRoutingOnly
  <string|LPP>layer
  <bool>allowedValue
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

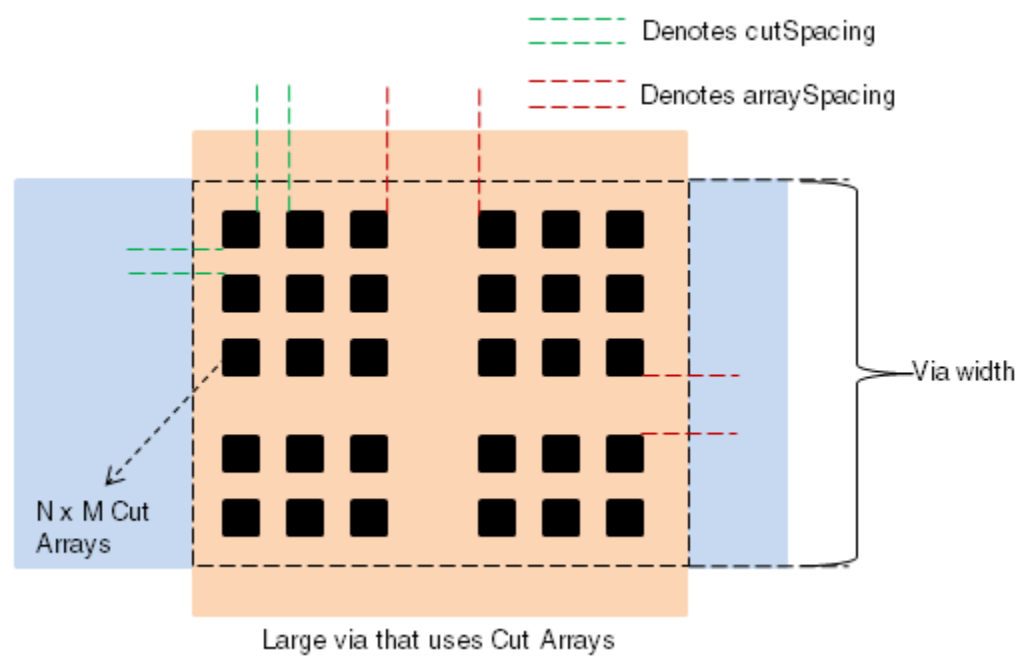
Argument	Description
layer	Layer can be defined as a string or a LPP.
allowedValue	If true, preferred routing on the layer is allowed.

For example,

```
preferredRoutingOnly m8 true
```

rectangularLargeViaArraysAllowed

This layer constraint specifies whether a rectangular array of via cuts is allowed.



Constraint Category	Via Construction
Constraint Type	Simple
OA Rule	oacRectangularLargeViaArraysAllowed

Syntax:

```
rectangularLargeViaArraysAllowed
  <string|LPP>layer
  <bool>allowedValue
  [-cutClass <cutClassArray | cutClassNameList>]
  [-parallelOverlap true | false]
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or a LPP.
allowedValue	Specifies whether a rectangular array of via cuts is allowed.
-cutClass	List of paired float values or a list of cutClass names. See cutClass and cutClassList . The constraint applies only in cases where the cuts comply with these specifications.
-parallelOverlap	Specifies that the rule applies only if the cut arrays have non-zero parallel overlap.

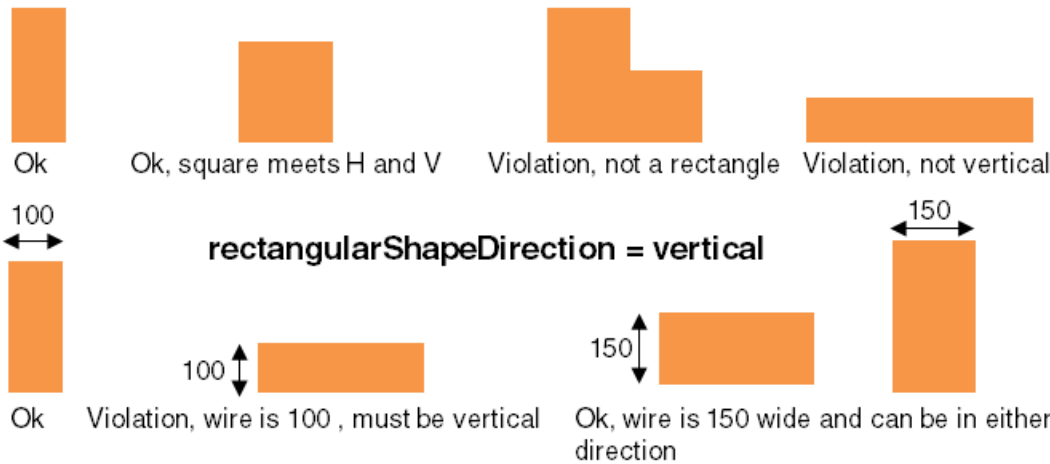
This example mentions that the rectangular via arrays is not allowed for VIA1 cuts:

```
rectangularLargeViaArraysAllowed VIA1 false
                                -cutClass 0.06 0.08
                                -parallelOverlap true
```

rectangularShapeDirection

Shapes on this layer must be rectangular. Optionally, the shapes might also be restricted to a certain direction, depending on their width. A square shape would meet both vertical and horizontal.

This constraint is used with [allowedWidthRange](#) to specify a set of discrete widths for the layer. The corresponding [allowedLengthRange](#) constraint should also be there to specify a minimum length greater than or equal to the minimum width to avoid ambiguity when determining the direction.



Constraint Category	Miscellaneous
Constraint Type	Complex
OA Rule	oacRectangularShapeDirection

Syntax:

```
rectangularShapeDirection
  <string|LPP>layer
  <string|float|Dtable>directionValue
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or a LPP.
directionValue	• string: Specifies the direction type. The allowed values are any, horizontal and vertical. • float1DTable: Specifies the Layer1 shape width and value refers to a value from the direction type. The allowed values are any, horizontal and vertical. The float1DTable is represented as: defaultDirection (<float>width <string>directionType any horizontal vertical)

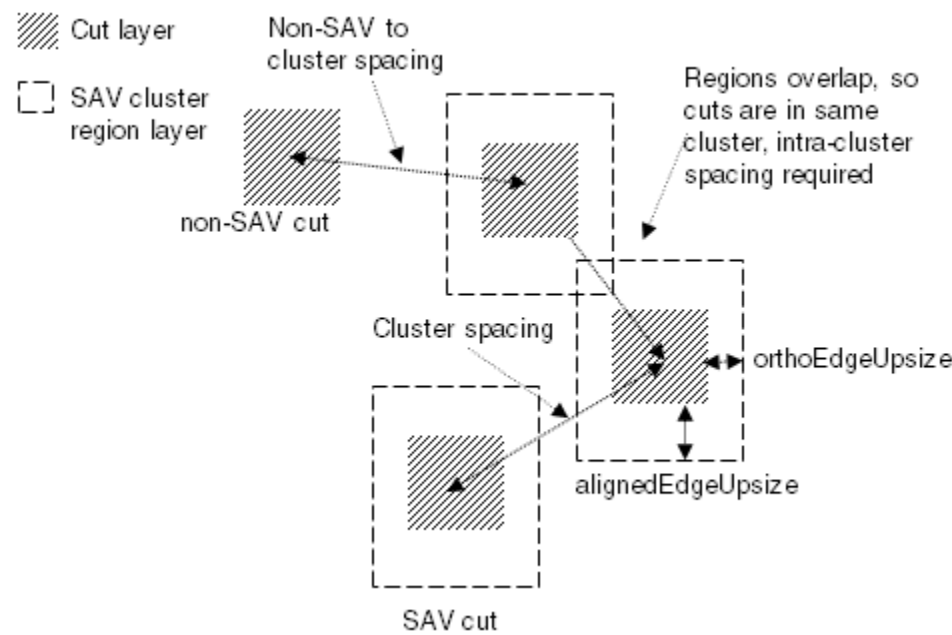
For example:

```
rectangularShapeDirection M1 vertical

rectangularShapeDirection M1 ((0.10 any) (0.12 vertical) (0.25 horizontal))
```

selfAlignedViaClusterSpacing

This constraint specifies the minimum spacing required between a via cut and a constrained via cut. Advanced processes have special spacing requirements for self aligned via (SAV) cuts that are in close proximity. The proximity is defined by growing the cut shape edges outward to define a region. If regions from different SAV cut overlap, they are merged into the same via cluster set. There are different spacing requirements between separate via clusters, from a non-SAV via cut to a via cluster, and between cuts in the same via cluster



Constraint Category	Via Construction
Constraint Type	Complex
OA Rule	Custom Compiler Custom Layer Constraint

Syntax:

```
selfAlignedViaClusterSpacing
    <string|LPP>layer
    <2DTable>spacingValue
    -cutClassList cutClassArray|cutClassNameList
    -clusterSpacingType cluster|nonCluster|intraCluster
    [-count int]
    [-color any|same|diff]
    [-isHard true|false]
    [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	layer can be defined as a string or LPP.
spacingValue	Two dimensional table of spacing values where the rows and columns represent cut class size indexes and the table values represent the centerline to centerline spacing between two specific cut classes. The table size of NxN where N is the number of elements in the cutClassList value array. 2-D table: default<float>(((cutClassIndex1<int> cutClassIndex2<int>) spacing<float> [-cutClusterCount int])* [2DTableOptions]
-cutClassList	An ordered array of cut dimension pairs. The array is used to determine the key to index into the constraint 2-D table value. The cutClassList is represented as: cutClassArray<float1> <float2> cutClassName<string> Note: If cutClass parameter is specified with cutClassName in the ASCII file, the technology importer looks for the first cutClass constraint specified in the same hierarchical constraint group to convert to cut class size. If no cutClass constraint exists, an error message is displayed. While exporting, the technology exporter looks for the first cutClass constraint specified in the same hierarchical constraint group to exists and converts the cut class size to cut class name. If no cutClass constraint exists, the cutClass parameter is exported with cut class size in dual value.
-clusterSpacingType	Type of SAV cut spacing that applies to this constraint. The enumerated values are: cluster=0, nonCluster= 1, and intraCluster= 2.
-count	(Optional) Number of overlapping search regions required to be considered a cluster. The default is 1.

Argument	Description
-color	(Optional) Defines the specific color of the metal shape for which this constraint applies. A value of <code>any</code> means any color for either shape, <code>same</code> means that the shapes must be on the same color, and <code>diff</code> means the shapes must be on different colors. Default is <code>any</code>. Caution: You must specify the <code>colorName</code> in the Controls Section > colors group, and the color mask value cannot be greater than the <code>numberOfColorMasks</code> value specified in the Layer Rules Section > functions group.

Note:

- The `colorName` must be defined in the [Controls Section](#), and the color mask value cannot be greater than the `numberOfColorMasks` value defined in the [functions](#) section.
- If `cutClass` parameter is specified with `cutClassName` in ASCII file, technology importer looks-up the first `cutClass` constraint specified in the same hierarchical constraint group to convert to cut class size. If a `cutClass` constraint does not exist then an error message is displayed.
- The warning message is displayed if the constraint value table size is larger than $(\text{cutClassList size})^2$. For example, `cutClassList` has 1 element but table contains 2×2 elements.

Example 1:

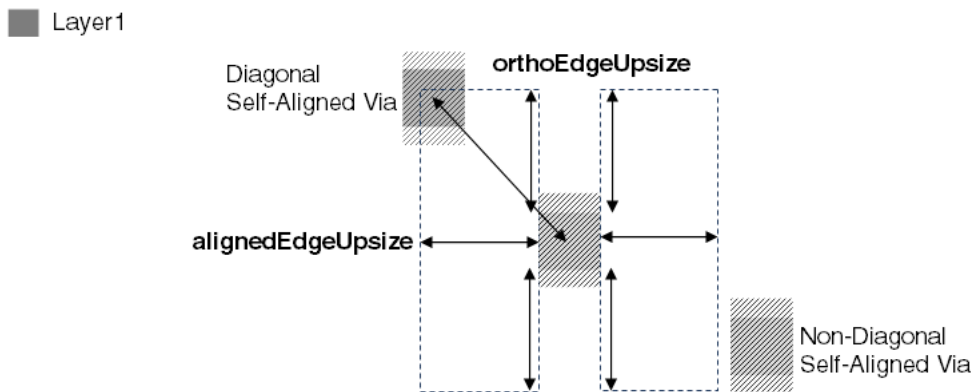
```
selfAlignedViaClusterSpacing v1 ((0 0) 0.02)
                                ((0 1) 0.05)
                                ((1 0) 0.05)
                                ((1 1) 0.08))
                                -cutClassList ((0.06 0.06) (0.06 0.08))
                                -clusterSpacingType nonCluster
                                -count 2
                                -color green
```

Example 2:

```
selfAlignedViaClusterSpacing v1 (((0 0) 0.02 -cutClusterCount 2)
                                ((0 1) 0.05 -cutClusterCount 4)
                                ((1 0) 0.05 -cutClusterCount 2)
                                ((1 1) 0.08 -cutClusterCount 3))
                                -cutClassList ((0.06 0.06) (0.06 0.08))
                                -clusterSpacingType nonCluster
                                -color green
```

selfAlignedViaDiagMinSpacing

The `selfAlignedViaDiagMinSpacing` requires a minimum amount of spacing between self-aligned vias that are positioned diagonally.



Constraint Category	Via Construction
Constraint Type	Complex
OA Rule	Custom Compiler Custom Layer Constraint

Syntax:

```
selfAlignedViaDiagMinSpacing
  <string|LPP>layer
  <float>spacing
  [-alignedEdgeUpsize float]
  [-orthoEdgeUpsize float]
  [-cutClass <cutClassArray | cutClassNameList>]
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	layer can be defined as a string or LPP.
spacing	Center-to-center distance, in DBU, between two diagonally positioned vias.
-alignedEdgeUpsize	Distance from the aligned edge and opposite edge of the via, used to define the cluster search region.
-orthoEdgeUpsize	Distance from the non-aligned edges of the via to define the cluster search region.

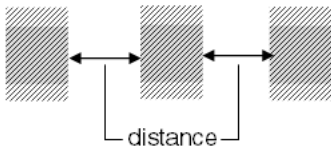
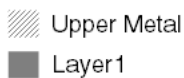
Argument	Description
-cutClass	List of paired float values or a list of cutClass names. See cutClass and cutClassList . The constraint applies only in cases where the cuts comply with these specifications.

For example:

```
selfAlignedViaDiagMinSpacing    v1 0.05
                                -alignedEdgeUpsize 0.04
                                -orthoEdgeUpsize 0.03
                                -cutClass ((0.02 0.04) (0.6 0.8))
```

selfAlignedViaMaxAlignedCuts

The `selfAlignedViaMaxAlignedCuts` limits the number of self-aligned vias that can be placed within a certain distance and aligned with each other.



Constraint Category	Via Construction
Constraint Type	Complex
OA Rule	Custom Compiler Custom Layer Constraint

Syntax:

```
selfAlignedViaMaxAlignedCuts
  <string|LPP>layer
  <int>numOfVias
  -distance float
  [-cutClass <cutClassArray | cutClassNameList>]
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	layer can be defined as a string or LPP.

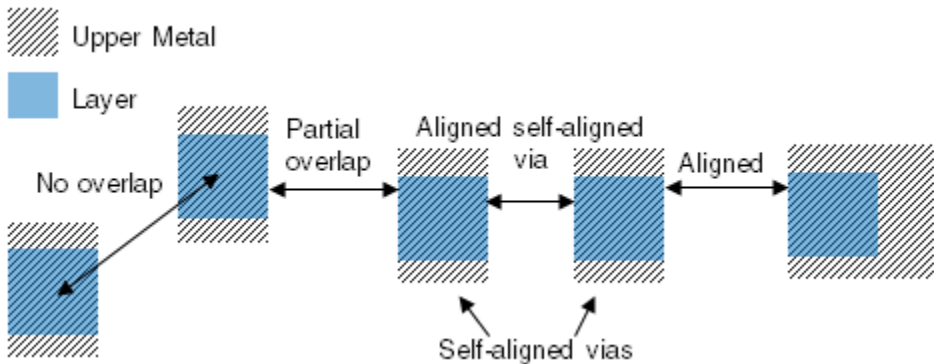
Argument	Description
numOfVias	Maximum number of self-aligned vias permitted to be aligned and within the specified distance of each other.
-distance	Maximum distance between two adjacently aligned vias.
-cutClass	List of paired float values or a list of cutClass names. See cutClass and cutClassList . The constraint applies only in cases where the cuts comply with these specifications.

For example:

```
selfAlignedViaMaxAlignedCuts    vl 4
                                -distance 0.05
                                -cutClass ((0.02 0.04) (0.6 0.8))
```

selfAlignedViaSpacing

The selfAlignedViaSpacing constraint contains four spacing values. The spacing values in microns, in order, are no overlap, partial overlap, aligned self-aligned via, and aligned. The no overlap spacing is measured center to center with the other spacings measure edge to edge.



Some processes have different spacing requirements for cut shapes depending on whether the edges overlap and whether overlapping cut edges are from a self aligned via.

Constraint Category	Via Construction
Constraint Type	Simple
OA Rule	Custom Compiler Custom Layer Constraint

Syntax:

```
selfAlignedViaSpacing
  <string|LPP>layer
  (<float>noOverlapSpacing <float>partialOverlapSpacing
  <float>savSpacing <float>alignedSpacing)
  [-cutClassList float1 float2|<string>cutClassName]
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	layer can be defined as a string or LPP.
noOverlapSpacing	Spacing, in microns, between two vias with no parallel-run length. The no overlap spacing is measured center to center with the other spacings measure edge to edge.
partialOverlapSpacing	Spacing, in microns, between two vias with some partial parallel-run length.
savSpacing	Spacing, in microns, between two self-aligned vias.
alignedSpacing	Spacing, in microns, between a self-aligned via and a non-self-aligned via.
-cutClassList	An ordered array of cut dimension pairs. Note: If <code>cutClass</code> parameter is specified with <code>cutClassName</code> in the ASCII file, the technology importer looks for the first <code>cutClass</code> constraint specified in the same hierarchical constraint group to convert to cut class size. If no <code>cutClass</code> constraint exists, an error message is displayed. While exporting, the technology exporter looks for the first <code>cutClass</code> constraint specified in the same hierarchical constraint group to exists and converts the cut class size to cut class name. If no <code>cutClass</code> constraint exists, the <code>cutClass</code> parameter is exported with cut class size in dual value.

For example:

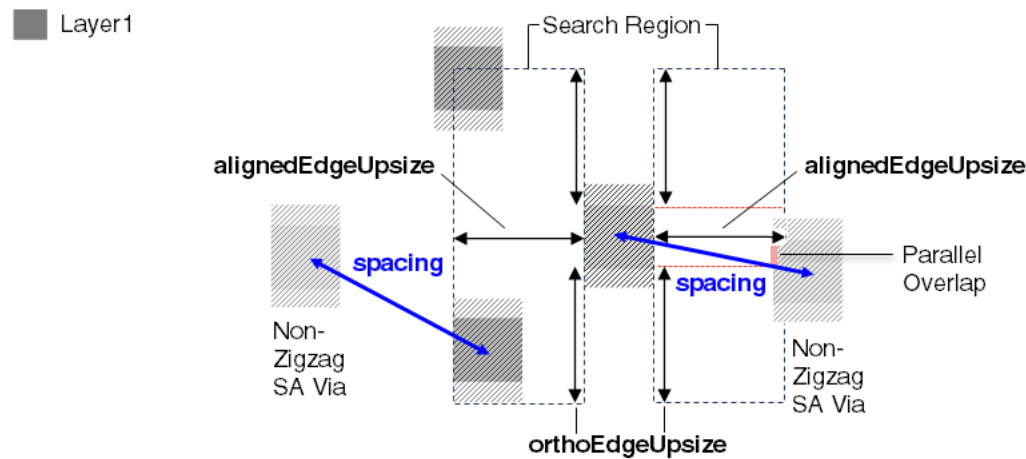
```
selfAlignedViaSpacing    v1    (0.10 0.08 0.06 0.05)
                           -cutClassList (0.06 0.08)
```

selfAlignedViaZigzagMinSpacing

The `selfAlignedViaZigzagMinSpacing` constraint requires a minimum distance (`spacing`) between self-aligned vias that are positioned in a zigzag pattern and self-aligned vias that are not part of the pattern.

A set of self-aligned vias are in a zigzag pattern when both of the following conditions are met:

- One of the vias has search regions with the following dimensions:
alignedEdgeUpside X orthoEdgeUpside.
- The neighboring vias are partially or fully within one of the search regions, but they have no parallel overlap with the main via.



Constraint Category	Via Construction
Constraint Type	Complex
OA Rule	Custom Compiler Custom Layer Constraint

Syntax:

```
selfAlignedViaZigzagMinSpacing
  <string|LPP>layer
  <float>spacing
  [-alignedEdgeUpside float]
  [-orthoEdgeUpside float]
  [-cutClass <cutClassArray | cutClassNameList>]
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	layer can be defined as a string or LPP.
spacing	Center-to-center distance, in DBU, between a self-aligned via in a zigzag pattern and a nearby self-aligned via not in the pattern.

Argument	Description
-alignedEdgeUpsize	Distance from the aligned edge and opposite edge of the via, used to define the cluster search region.
-orthoEdgeUpsize	Distance from the non-aligned edges of the via to define the cluster search region.
-cutClass	List of paired float values or a list of cutClass names. See cutClass and cutClassList . The constraint applies only in cases where the cuts comply with these specifications.

For example:

```
selfAlignedViaMaxAlignedCuts    v1 0.05
                                -alignedEdgeUpsize 0.04
                                -orthoEdgeUpsize 0.03
                                -cutClass 0.02 0.04
```

sheetResistance

The `sheetResistance` constraint is used to calculate wire and via resistance during in-design resistance reporting. The constraint specifies the following:

- Sheet resistance (`resistanceValue`)
- Temperature-derating coefficients (`tempDerating1` and `tempDerating2`)

You can specify all three parameters with or without wire-width dependent tables.

If you supply only one temperature-derating parameter (`tempDerating1`), the final resistance is calculated as follows:

$$T_{difference} = T_{operating} - T_{nominal}$$

$$R_{final} = (1 + tempDerating_{width} \times T_{difference}) \times R_{measured}$$

If you supply both temperature-derating parameters (`tempDerating1` and `tempDerating2`), the final resistance is calculated using the following quadratic equation:

$$R_{final} = (1 + tempDerating1_{width} \times T_{difference} + tempDerating2_{width} \times T_{difference}^2) \times R_{measured}$$

Constraint Category	Spacing
Constraint Type	Complex
OA Rule	Custom Compiler custom layer constraint

Syntax:

```
sheetResistance
    <string|LPP>layer
    <float|1DTableValue>resistanceValue
    [-corner min|typical|max]
    [-nominalTemp float]
    [-tempDerating1 temp1DTable]
    [-tempDerating2 temp1DTable]
    [-isHard true|false]
    [-comment [<string rule_id>] <string rule_desc>]
```

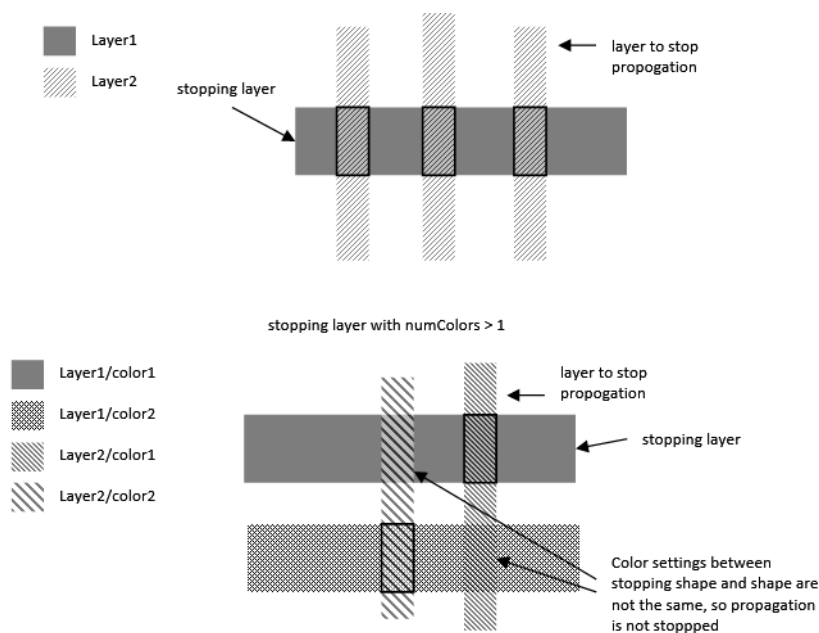
Argument	Description
layer	Layer (string or LPP).
resistanceValue	float: Non-width-dependent sheet resistance value, in Ohms per square. float1DTable: A list of two float values that specify width-dependent sheet resistance. The first float value is the wire width, in microns. The second float value is the sheet resistance, in Ohms per square. Example: ((wireWidth1 sheetResistance1) (wireWidth2 sheetResistance2) ...)
-corner	The simulation corner (or simulation condition) this constraint applies to: min, typical, or max. Default: max
-nominalTemp	Nominal temperature in degrees Celsius. Default: 25
-tempDerating1	float: Non-width-dependent CRT1 coefficient. float1DTable: A list of two float values that specify width-dependent CRT1 coefficients. The first float value is the wire width, in microns. The second float value is the CRT1 coefficient. Example: ((wireWidth1 CRT1_1) (wireWidth2 CRT1_2) ...)
-tempDerating2	float: Non-width-dependent CRT2 coefficient. float1DTable: A list of two float values that specify width-dependent CRT2 coefficients. The first float value is the wire width, in microns. The second float value is the CRT2 coefficient. Example: ((wireWidth1 CRT2_1) (wireWidth2 CRT2_2) ...)

For example:

```
sheetResistance M1 0.15
sheetResistance M1 0.12 ((0.10 0.14) (0.12 0.16))
                -corner max
                -nominalTemp 12
                -tempDerating1(0.4 ((0.10 0.6) (0.12 0.8)))
                -tempDerating2(0.2 ((0.08 0.5) (0.10 0.7)))
```

stoppingLayer

The constraint is used to control connectivity propagation. Propagation goes through the shape on the constraint's layer (stopping layer) until it is overlapped by a layer in the layer array constraint value.



Syntax:

```
stoppingLayer
  <string|LPP>layer
  <List of strings>layers
  [-color colorName]
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer (string or LPP).
layers	Specifies the list of layers that are to be stopped propagation for stopping layer.
color	If this parameter is specified, this constraint only applies to shapes on the given color mask on the associated layers if they are colorable. If this parameter is specified and the associated layers are not colorable, this value applies to the stopping layer shapes if colorable. If this parameter is not specified, or 0, and the constraints layer has more than one color, the shapes on this layer will only stop propagation on shapes on the associated layers with the same color.

Note:

The error message is displayed in the following situations:

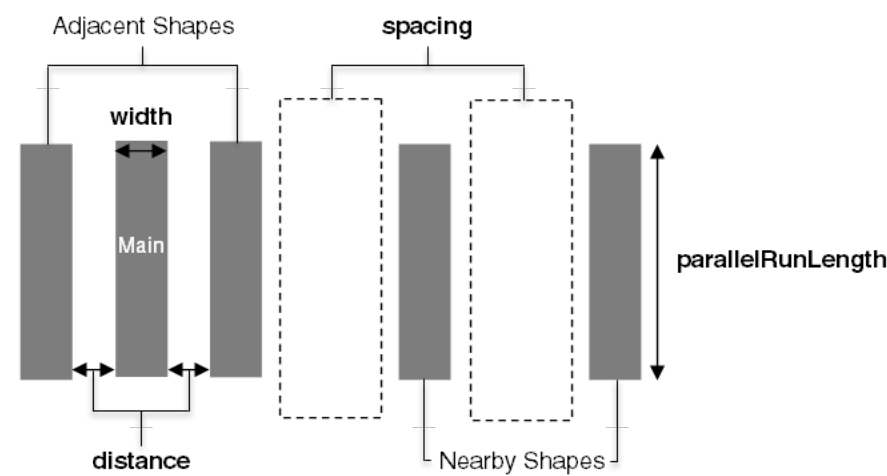
- erasing layer is non-colorable, erased layer is non-colorable, and -color is specified.
- erasing layer and erased layer are colorable, and -color is specified.
- erasing layer and erased layer are colorable, but they have different `numberOfColorMasks` values defined in the [functions](#) section.

For example:

```
stoppingLayer      m1 (m2 m3)
stoppingLayer      m1 (m2 m3) -color red
```

threeWireForbiddenSpacing

The `threeWireForbiddenSpacing` constraint requires spacing clearance (forbidden spacing) surrounding a metal line (main line) when the main line is in a three-line formation, as shown. The constraint only applies when the main line’s width, distance between the two adjacent lines, and parallel run length of the main line and other nearby parallel lines meet the specified requirements.



Constraint Category	Spacing
Constraint Type	Complex
OA Rule	Custom Compiler Custom Layer Constraint

Syntax:

```
threeWireForbiddenSpacing
  <string|LPP>layer
  <rangeValue>spacingRange
  -width float
  -distance float
  -parallelRunLength float
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

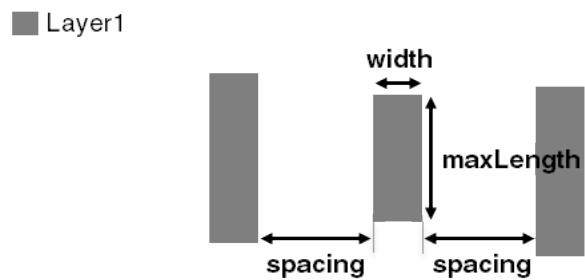
Argument	Description
layer	Layer on which the shapes are placed. <code>layer</code> can be a string (layer name) or LPP.
spacingRange	An array of prohibited spacing ranges. To specify the ranges, see Range Data Syntax .
-width	Width of the main line, in microns. The width must be less than or equal to the specified value.
-distance	Distance, in microns, between the main line and the two adjacent lines.
-parallelRunLength	Length, in microns, of the neighboring shapes that are parallel to the main line. The run length must be greater than or equal to the specified value.

Example:

```
threeWireForbiddenSpacing m1 ( (>=0.1 <=0.2) (0.5 0.6) (0.7 <=0.9))
                               -width 0.35 -distance 0.55 -parallelRunLength 0.23
```

twoSidedParallelMinSpacing

The `twoSidedParallelMinSpacing` constraint requires that short, narrow shapes are a certain distance from adjacent shapes.



Constraint Category	Spacing
Constraint Type	Complex

OA Rule

Custom Compiler Custom Layer Constraint

Syntax:

```
twoSidedParallelMinSpacing
    <string|LPP>layer
    <float>spacing
    -width <float>
    -maxLength <float>
    [-isHard true|false]
    [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer on which the wire shapes are placed. <code>layer</code> can be a string (layer name) or LPP.
spacing	Minimum edge-to-edge distance between the wire and the adjacent shapes.
-width	Width of the wire shape. The constraint applies only if the width is less than the specified value.
-maxLength	Maximum length of the wire. The constraint only applies if the length is less than or equal to the specified value.

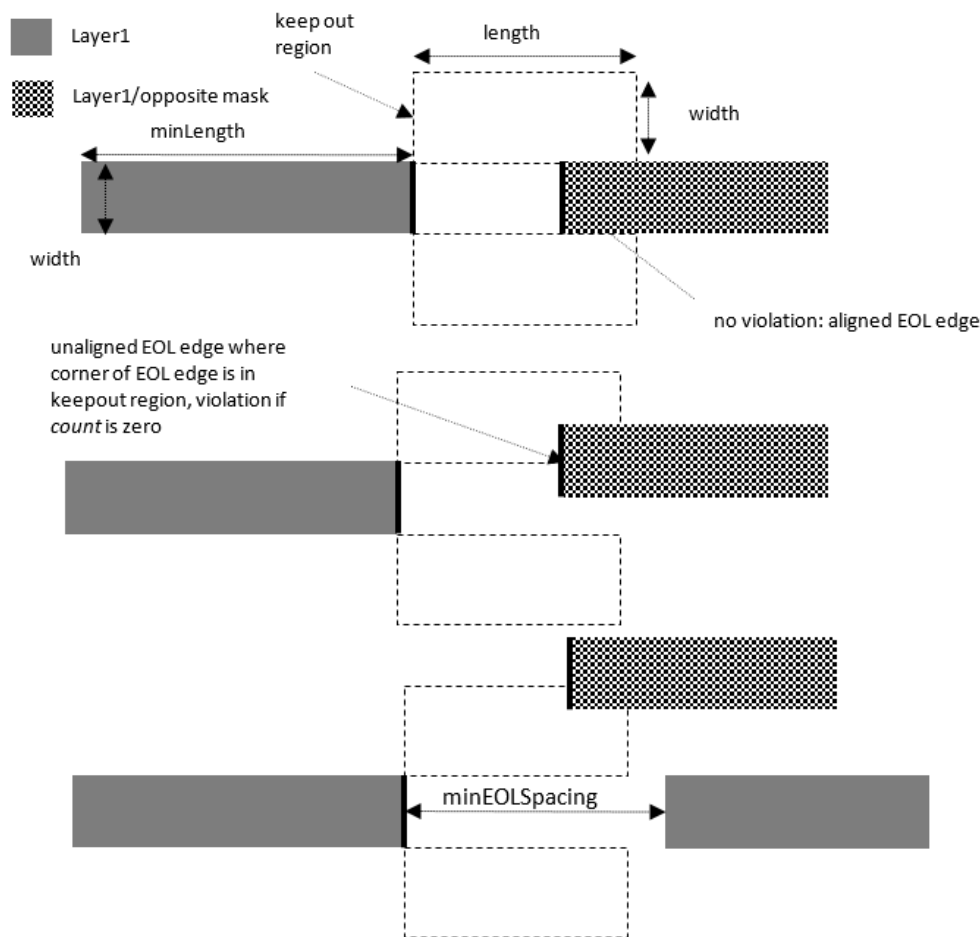
Example:

```
twoSidedParallelMinSpacing m2 0.23
    -width 0.22 -maxLength 0.77
```

unalignedEndToEndKeepout

Advanced processes have restrictions on end of line edges that are in close proximity. They are required to be aligned or separated if unaligned. The separation is controlled by a keepout region which is defined by a width and length away from an end of line edge

corner. An optional parameter `maxCount` defines the allowed number of keepout regions that can contain an EOL edge corner before a violation occurs:



Constraint Category	Routing
Constraint Type	Complex
OA Rule	Custom Compiler Custom Constraint

Syntax:

```
unalignedEndToEndKeepout
  <string|LPP>layer
  <float>width <float2>length
  -width float
  [-isThreshold true|false]
  [-minLength float]
  [-color any|same|diff]
```

```
[ -count int]
[ -oelMinSpacing float]
[ -maxWidth float]
[ -excludeZFlag rangeValue1 rangeValue2 rangeValue3 bool]
[ -isHard true|false]
[ -comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or a LPP.
width length	Specify the values representing the keepout width and length respectively.
-width	The width of the EOL edge must be greater than this value if <code>isThreshold</code> is <i>false</i> for the constraint to apply. When <code>isThreshold</code> is <i>true</i> , the width must be greater than this width for the constraint to apply.
-isThreshold	Specifies how the width parameter is used during constraint application. The default value is <i>false</i> .
-minLength	The length of wire must be greater than or equal to this value for the constraint to apply. The default value is 0.
-color	Defines the specific color of the EOL shape for which this constraint applies. A value of <i>any</i> means any color for either shape, <i>same</i> means that the shapes must be on the same color, and <i>diff</i> means the shapes must be on different colors. Default is <i>any</i> . Caution: You must specify the <i>colorName</i> in the Controls Section > colors group, and the color mask value cannot be greater than the <code>numberOfColorMasks</code> value specified in the Layer Rules Section > functions group.
-count	Maximum number of EOL keepout regions that can have a facing EOL corner. The default value is 0.
-oelMinSpacing	Minimum distance between end-to-end wires.
-maxWidth	Maximum width at the end of the wire.
-excludedZFlag	Z-shape conditions under which the constraint does not apply: - width: Range of widths. - concave length: Range of lengths of the concave edge. - height: Range of heights. - LShape: Indicator that L shapes are included. To specify the ranges, see Range Data Syntax .

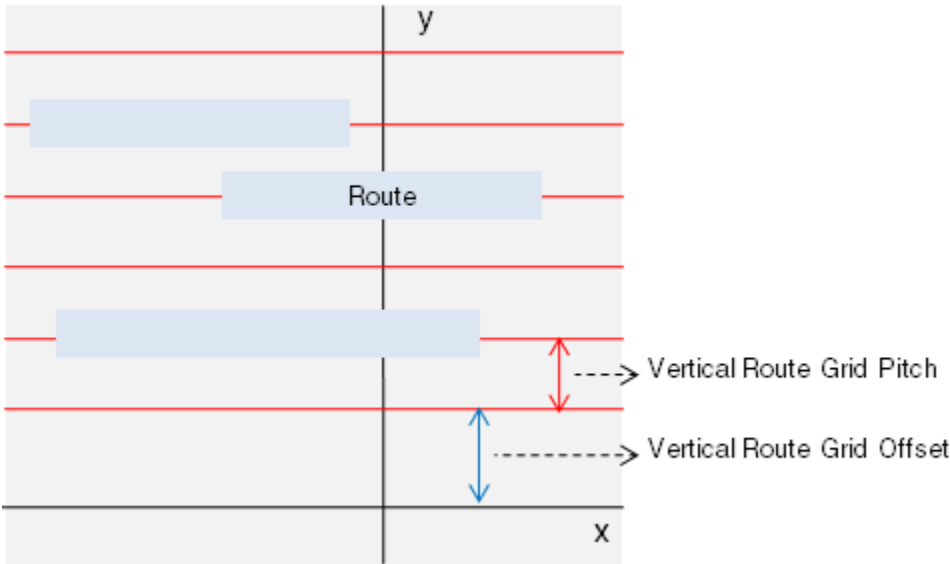
For example:

```
unalignedEndToEndKeepout m1 0.15 0.55
    -width 0.25
    -isThreshold true
    -minLength 0.05
    -color diff
    -count 3
    -oelMinSpacing 0.5
```

```
-maxWidth 0.35
-excludedZFlag ((>=0.25 <=0.75) (>0.034 <0.075) (>=0.55
<0.85) true)
```

verticalRouteGridOffset

This layer constraint specifies the route grid points on the Y axis. The route grid points are completely defined using the verticalRouteGridOffset and verticalRouteGridPitch.



Constraint Category	Routing
Constraint Type	Simple
OA Rule	oacVerticalRouteGridOffset

Syntax:

```
verticalRouteGridOffset
  <string|LPP>layer
  <float>offsetValue
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

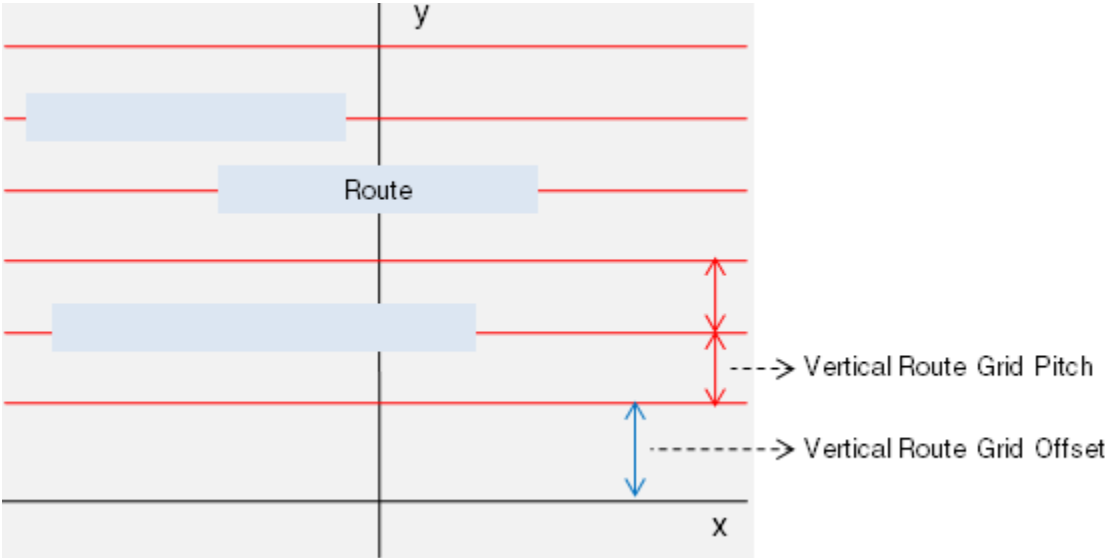
Argument	Description
layer	Layer can be defined as a string or a LPP.
offsetValue	Specifies the route grid offset points on the Y axis.

For example:

```
verticalRouteGridOffset M1 0.1
```

verticalRouteGridPitch

This layer constraint specifies the route grid points on the Y axis. The route grid points are completely defined using the verticalRouteGridPitch and verticalRouteGridOffset



Constraint Category	Routing
Constraint Type	Simple
OA Rule	oacVerticalRouteGridPitch

Syntax:

```
verticalRouteGridPitch
  <string|LPP>layer
  <float>pitchValue
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Layer can be defined as a string or a LPP.
pitchValue	Specifies the route grid pitch points on the Y axis.

For example:

```
verticalRouteGridPitch M2 0.3
```

8

Layer Pair Constraints

Describes layer pair constraints and provides the syntax with examples.

About Layer Pair Constraints

Layer pair constraints are defined on two specific layers and take a single value. The single value can either be a minimum DRC value or a preferred value.

General Syntax

```
ruleName <string|LPP>layer1
         <string|LPP>layer2
         <float|int|bool>value
         [-isHard true|false]
         [-comment [<string rule_id>] <string rule_desc>]
```

Layer Symmetry

Some layer pair constraints are asymmetrical. In these cases, you cannot reverse the order of the layers in the constraint specifications.

List of Layer Pair Constraints

The following table lists the layer pair constraints you can define in the technology. Click a constraint name to view the details. For general information about technology constraints, see [Constraints Overview](#).

ruleName	Constraint Category	Constraint Type
allowedClearanceRange	Clearance	Complex
allowedEnclosedViaProximitySpacing	Spacing	Complex
allowedViaClusters	Via Construction	Complex
concaveCornerToCutSpacing	Spacing	Complex
convexEndOfLineExtension	Spacing	Complex
denseEndOfLineExtensionSpacing	Spacing	Complex
forbiddenTwoSidedViaClearance	Clearance	Complex

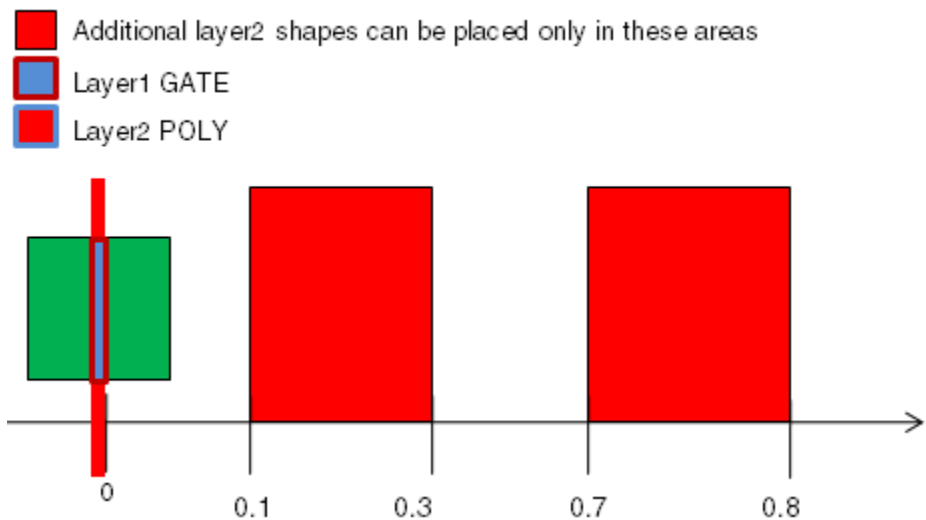
ruleName	Constraint Category	Constraint Type
generalCutClassClearance	Clearance	Complex
illegalHGatePattern	Length and Width Process Rules	Complex
keepAlignedShapes	Placement and Alignment	Complex
maxClearance	Clearance	Simple
maxExtension	Extension	Complex
maxInteractCutKeepout	Spacing	Complex
maxTouchingDirectionLength	Length and Width Process Rules	Complex
metalJointViaExtension	Extension	Complex
minCenterlineExtension	Extension	Complex
minClearance	Clearance	Complex
minConcaveConvexEdgeExtension	Spacing	Complex
minConcaveCornerExtension	Extension	Complex
minConcaveCornerOverlap	Overlap	Simple
minConstrainedViaExtension	Via Construction	Complex
minCrossMetalViaGroupExtension	Extension	Complex
minCutClassClearance	Via Construction	Complex
minCutMetalOverlap	Overlap	Complex
minCutRoutingClearance	Clearance	Complex
minCutToMetalCornerLength	Spacing	Complex
minCutToMetalSpacing	Spacing	Complex
minDualEndOfLineExtension	Extension	Complex
minDualExtension	Extension	Complex
minEndOfLineEdgeExtension	Extension	Complex
minEndOfLineExtension	Extension	Complex
minExtension	Extension	Complex
minExtensionAlignedProximity	Extension	Complex
minExtensionEdge	Extension	Complex
minExtensionToCorner	Extension	Complex

ruleName	Constraint Category	Constraint Type
minGateExtension	Extension	Complex
minOuterVertexSpacing	Spacing	Complex
minOverlap	Overlap	Simple
minSameNetClearance	Clearance	Simple
minSideExtension	Extension	Complex
minSparseDenseExtension	Extension	Complex
minSparseDenseViaSpacing	Via Construction	Complex
minTotalEnclosure	Spacing	Complex
minUpperMetalExtension	Spacing	Complex
minViaClearance	Clearance	Complex
minViaExtension	Via Construction	Complex
minViaJointExtension	Extension	Complex
minVoltageClearance	Clearance	Complex
nonSelfAlignedViaClearance	Spacing	Complex
nonSelfAlignedViaEdgeToMetalSpacing	Via Construction	Complex
selfAlignedViaEdgeClearance	Via Construction	Complex
selfAlignedViaExtension	Via Construction	Complex
selfAlignedViaMetalClearance	Via Construction	Complex
vertexInsideNotAllowed	Via Construction	Complex
viaStackingAllowed	Via Construction	Simple
viaSurroundingMetalKeepout	Spacing	Complex

allowedClearanceRange

This layer pair constraint specifies the range of clearances that are allowed between shapes on different layers.

Both bounds of the range (upper and lower) can be given a single value. This means that the clearance for this range must be equal to the specified value.



Constraint Category	Clearance
Constraint Type	Complex
OA Rule	oacAllowedClearanceRange
Layer Symmetry	Asymmetric

Syntax:

```
allowedClearanceRange
  <string|LPP>gateLayer
  <string|LPP>polyLayer
  <rangeValue|rangeValue1DTable|
    rangeValue2DTable>clearanceValue
  [-width float]
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
gateLayer	gateLayer can be defined as a string or LPP.
polyLayer	polyLayer can be defined as a string or LPP.

Argument	Description
clearanceValue	rangeValue: Specifies the allowed spacing range values for the spacing between layer1 and layer2. To specify the ranges, see Range Data Syntax. rangeValue1DTable: Specifies the allowed spacing range values based on the width of one of the two shapes. The value range 1D table is represented as: <pre>default_range_value (width rangeValue)</pre> rangeValue2DTable: Specifies the allowed spacing range values based on the width of both the shapes. The value range 2D table is represented as: <pre>default_range_value ((width1 width2) rangeValue)</pre>
-width	Specifies if the constraint applies when both widths of layer1 and layer2 are less than the given width value.

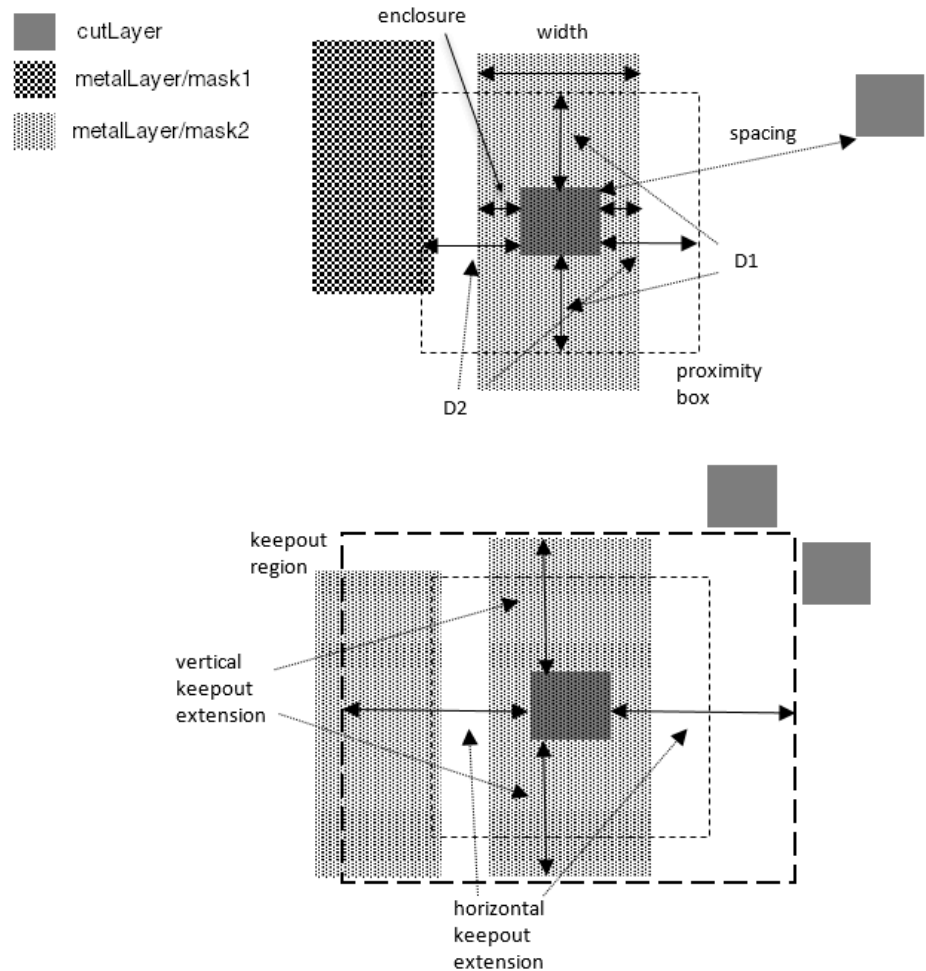
Example:

Rule	Description	Value (in microns)
PO.CL.1	GATE to POLY spacing when width of both GATE and POLY <= 0.1u	>= 0.1 and <=0.3 OR >=0.7 and <=0.8
PO.CL.1.1	GATE to POLY spacing when GATE length >= 0.5um	>=0.4 and <= 0.6
PO.CL.1.2	GATE to POLY spacing when GATE length >= 0.4um and POLY width >=0.6 um	>=0.9 and <=1.1

```
allowedClearanceRange GATE POLY ((0.1 0.3) (0.7 0.8)) -width 0.1
allowedClearanceRange GATE POLY ((0.1 0.3)) ((0.5 ((0.4 0.6))))
allowedClearanceRange GATE POLY ((0.1 0.3)) (((0.4 0.6) ((0.9 1.1))))
```

allowedEnclosedViaProximitySpacing

This layer pair constraint specifies the spacing required between cuts that are enclosed by shapes and other shapes on the same layer within a certain distance from the enclosure.



Constraint Category	Spacing
Constraint Type	Complex
OA Rule	Custom Compiler custom layer pair constraint
Layer Symmetry	Asymmetric

Syntax:

```
allowedEnclosedViaProximitySpacing
<string|LPP>cutLayer
<string|LPP>metalLayer
<rangeData|dualFloat>spacingValue
[-cutClass <cutClassArray | cutClassNameList>]
[-width float]
-proximityBox float1 float2
-minEnclosure float
[-color colorName|any|same|diff]
[-isHard true|false]
[-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
cutLayer	cutLayer can be defined as a string or LPP.
metalLayer	metalLayer can be defined as a string or LPP.
spacingValue	Minimum distance between the enclosed cut shape on layer 1 and the shape on layer 2.
-cutClass	List of paired float values or a list of cutClass names. See cutClass and cutClassList . The constraint applies only in cases where the cuts comply with these specifications.
-width	Width of the enclosure shape on layer 2. Default: 0
-proximityBox	Dimensions of the proximity box (D2 D1).
-minEnclosure	Minimum distance between the cut and the edge of the shape on the metal layer.
-color	Color of the enclosure shape. Value values are colorName, 0 (any), 1 (same), and 2 (diff). The colorName must be defined in the Controls Section . Default: 0

Examples:

```
allowedEnclosedViaProximitySpacing vial metall ( (>=0.1 <=0.2) (0.5 0.6) (0.7
<=0.9))
    -proximityBox 0.2 0.4
    -minEnclosure 0.45

allowedEnclosedViaProximitySpacing vial metall 0.75 0.97
    -proximityBox 0.2 0.4
    -minEnclosure 0.45
    -cutClass ((0.6 0.8) (0.4 0.6))
    -width 0.26
    -color red
```

allowedViaClusters

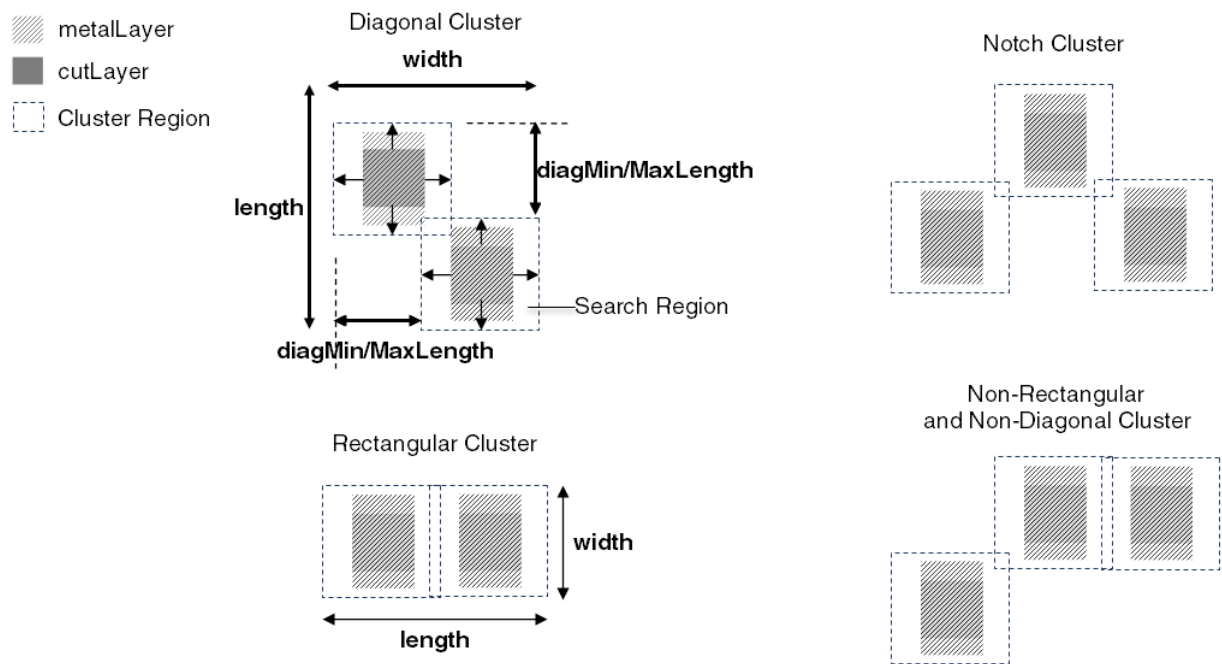
The `allowedViaClusters` constraint restricts the placement of self-aligned vias that are in a cluster.

A self-aligned via cluster is created when:

- A search region is defined by extending a certain distance from the edges of the via. The extensions are determined by `alignedEdgeUpsize` and `orthoEdgeUpsize`. See [selfAlignedViaExtension](#).
- When the search regions of neighboring vias overlap, a cluster is formed.

When the search regions of self-aligned vias overlap, restrictions determine the type of clustering:

Cluster Type	Description
Diagonal	The corners of two adjacent regions overlap completely, and the sides of the corners are not aligned.
Rectangular	When the corners two adjacent regions overlap partially, and the sides of the regions are aligned.
Notch	When the adjacent corners of a via's search region overlap partially with search region corners of neighboring vias. The sides of the different corners do not align.
Non-redtangular and non-diagonal	When search regions of three adjacent vias overlap partially, as shown.



Constraint Category	Spacing
Constraint Type	Complex
OA Rule	Custom Compiler Custom Layer Pair Constraint
Layer Symmetry	Asymmetric

Syntax:

```
allowedViaClusters
<string|LPP>metalLayer
<string|LPP>cutLayer
<float1 float2>width length
-diagMinEdgeLength float
-diagMaxEdgeLength float
[-uShapeAllowed true | false]
[-cutClass <cutClassArray | cutClassNameList>]
[-maxDiagonal float]
[-isHard true|false]
[-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
metalLayer	metalLayer can be defined as a string or a LPP.
cutLayer	cutLayer can be defined as a string or a LPP.

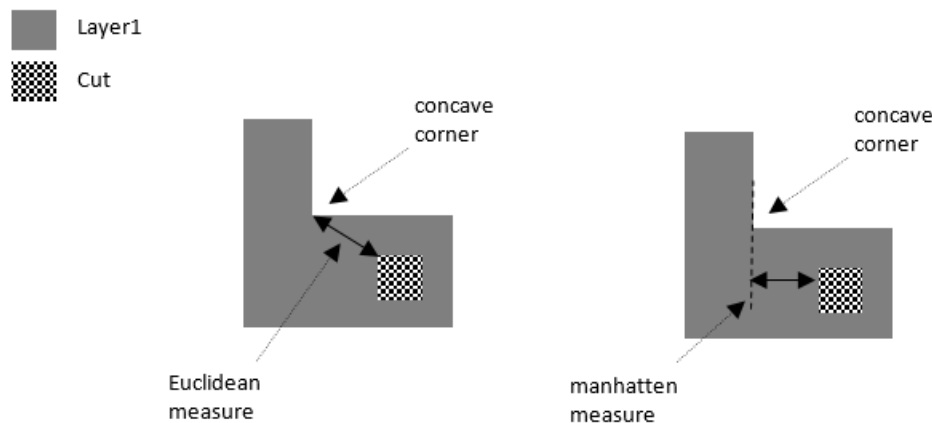
Argument	Description
width	Width, in microns, of the cluster region.
length	Minimum length, in microns, of the cluster region.
-diagMinEdgeLength	Minimum length, in microns, of the search regions that are not overlapping.
-diagMaxEdgeLength	Maximum length, in microns, of the search regions that are not overlapping.
-uShapeAllowed	Indicates whether a u-shaped cluster is permitted. Valid values are true and false. Default: false
-cutClass	List of paired float values or a list of cutClass names. See cutClass and cutClassList . The constraint applies only in cases where the cuts comply with these specifications.
-maxDiagonal	Maximum number of self-aligned vias that can be in a single-direction diagonal cluster.

For example:

```
allowedViaClusters metall vial 0.02 0.04
    -diagMinEdgeLength 0.05
    -diagMaxEdgeLength 0.07
    -uShapeAllowed true
    -cutClass 0.06 0.08
    -maxDiagonal 4
```

concaveCornerToCutSpacing

This layer pair constraint specifies the spacing requirements for cuts and shapes that have concave corners.



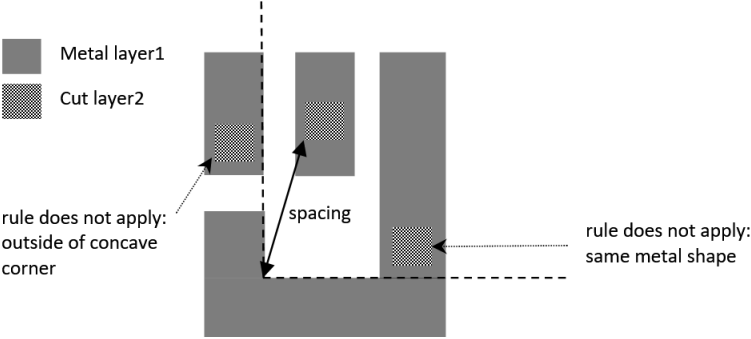
Constraint Category	Spacing
---------------------	---------

Constraint Type	Complex
OA Rule	Custom Compiler custom layer pair constraint
Layer Symmetry	Asymmetric

Syntax:

```
concaveCornerToCutSpacing <string|LPP>layer1
    <string|LPP>cutLayer
    <float>spacingValue
    [-cutClass <cutClassArray | cutClassNameList>]
    [-distanceMeasureType euclidean|manhattan]
    [-exceptConnectivityType
        anyConnectivityType|
        sameNetConnectivityType|
        contiguousShapesConnectivityType|
        directShapesConnectivityType|
        sameViaConnectivityType]
    [-otherLayerMaxExtension floatValue]
    [-isHard true|false]
    [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer1	Layer of the shape. Layer1 can be defined as a string or LPP.
cutLayer	Layer of the cut. cutLayer can be defined as a string or LPP.
spacingValue	Minimum distance between the cut shape and a concave corner of the shape on layer 1.
-cutClass	List of paired float values or a list of cutClass names. See cutClass and cutClassList . The constraint applies only in cases where the cuts comply with these specifications.
-distanceMeasureType	Type of measurement used to determine the distance between the cut shape and the concave corner. Valid values are euclidean and manhattan. Default: euclidean

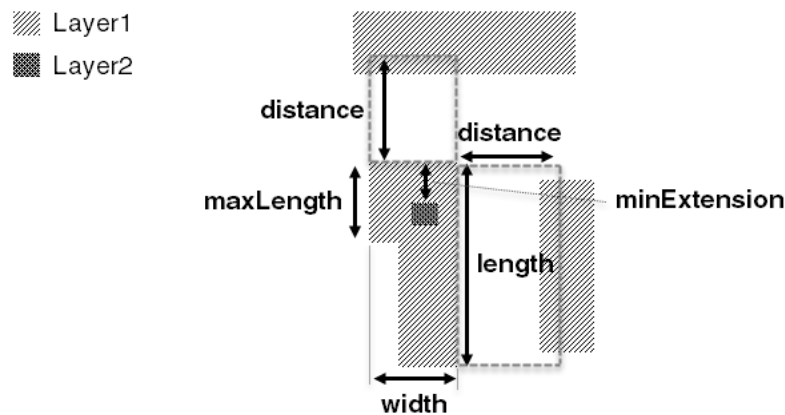
Argument	Description
-exceptConnectivityType	Optional. Specifies the type of connectivity between the two shapes that causes the constraint to not apply. Note: - If <code>exceptConnectivityType</code> is not specified, the <code>concaveCornerToCutSpacing</code> rule applies to all connectivity types. - If this parameter is set to <code>contiguousShapesConnectivityType</code>, the constraint only applies to a cut enclosed by a metal shape different from the metal shape forming the concave corner, as below.  The metal shapes can belong to the same net or two different nets. There is also an additional requirement for the cut shape to be at least partially located inside the concave corner.
-otherLayerMaxExtension	If given, the rule does not apply if extension of the other metal layer (metal above or metal below the cut layer) over the cut is greater than this value in database units (in all four directions).

For example,

```
concaveCornerToCutSpacing L1 vial 0.25
  -cutClass ((0.6 0.8) (0.6 0.8))
  -distanceMeasureType manhattan
  -exceptConnectivityType contiguousShapesConnectivityType
  -otherLayerMaxExtension 0.5
```

convexEndOfLineExtension

The `convexEndOfLineExtension` constraint requires a minimum extension at the end of a wire (or line) that has a jog. With the jog, the end of the line has three convex corners.



Constraint Category	Spacing
Constraint Type	Complex
OA Rule	Custom Compiler custom layer pair constraint
Layer Symmetry	Asymmetric

Syntax:

```
convexEndOfLineExtension
<string|LPP>metalLayer
<string|LPP>cutLayer
<float>minExtension
-width <float>
-length <float>
-maxLength <float>
-distance <float>
[-cutClass <cutClassArray | cutClassNameList>]
[-isHard true|false]
[-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
metalLayer	metalLayer can be defined as a string or LPP.
cutLayer	cutLayer can be defined as a string or LPP.
minExtension	Minimum distance between the cut shape and the end of the wire.
-width	Maximum width at the extended end of the wire. The constraint applies only if the width is less than or equal to this value.
-length	Minimum length of the longer side of the wire that is perpendicular to the end of wire. The constraint applies only if the length is greater than or equal to this value.

Argument	Description
-maxLength	Maximum length of the shorter side of the wire that is perpendicular to the end of wire. The constraint applies only if the length is less than or equal to this value.
-distance	Distance between the wire and two neighboring shapes. The constraint applies only if the distance is less than or equal to this value.
-cutClass	List of paired float values or a list of cutClass names. See cutClass and cutClassList . The constraint applies only in cases where the cuts comply with these specifications.

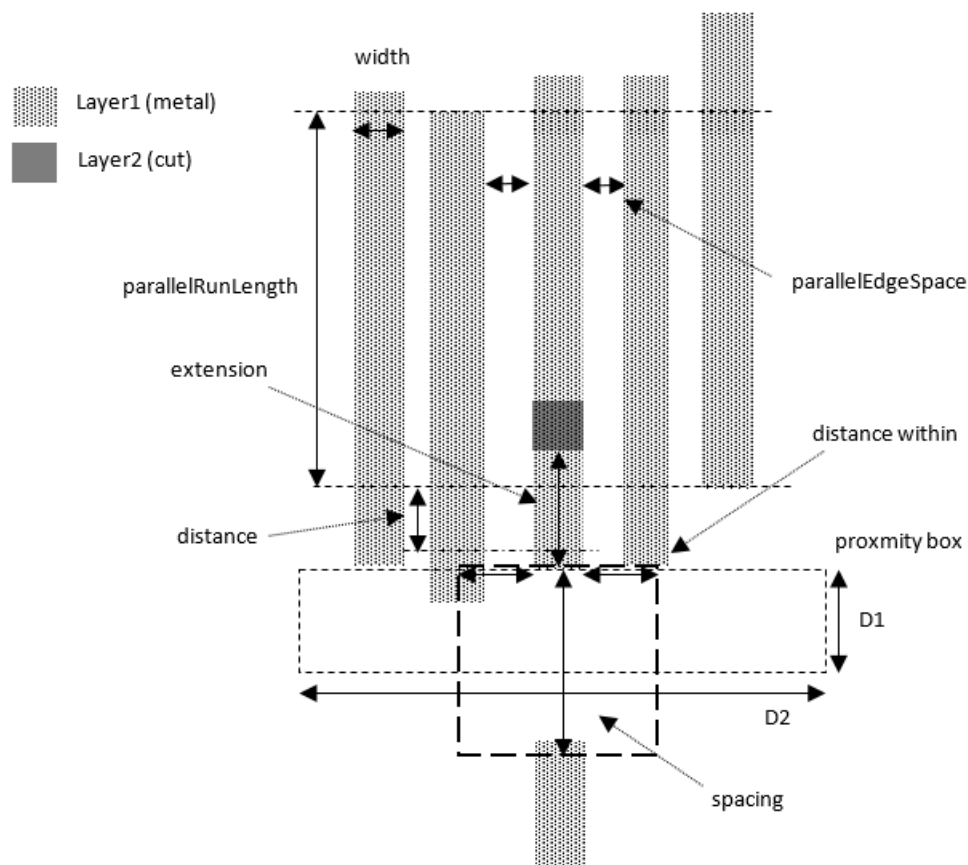
For example,

```
convecEndOfLineExtension M1 V1 0.012
                        -width 0.15
                        -length 0.13
                        -maxLength 0.08
                        -distance 0.06
                        -cutClass ((0.02 0.02) (0.6 0.8))
```

denseEndOfLineExtensionSpacing

This layer pair constraint specifies the requirement to extend a wire beyond the via (cut) when the following conditions are met:

- The wire with the cut is at the center of a group of five wires.
- The width of all five wires is less than or equal to a specified value (width).
- The distance between the wires are at most a specified value (parallelEdgeSpace).
- The distance between the edge of the cut and the end of the parallel run length must shorter than a specified value (distance).
- The wire with the cut does not go through the designated proximity box that has dimensions D2 x D1.



Constraint Category	Spacing
Constraint Type	Complex
OA Rule	Custom Compiler custom layer pair constraint
Layer Symmetry	Asymmetric

Syntax:

```
denseEndOfLineExtensionSpacing
<string|LPP>metalLayer
<string|LPP>cutLayer
<float float>extension spacing
-width float
-distance float
-proximityBox float1 float2
[-parallelRunLength float]
[-distanceWithin float]
-parallelEdgeSpace float
-cutClass <cutClassArray | cutClassNameList>
```

```
[-isHard true|false]
[-comment [<string rule_id>] <string rule_desc>]
```

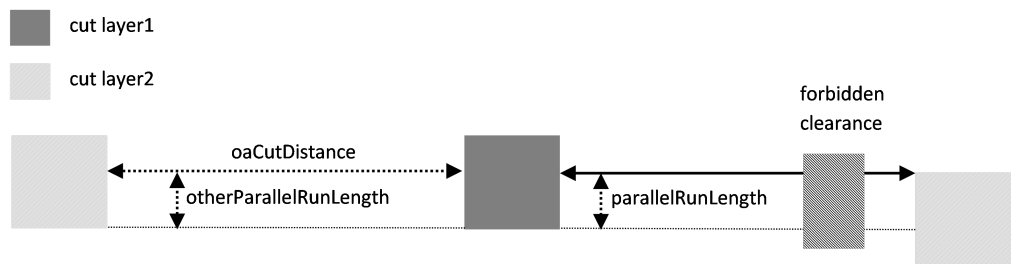
Argument	Description
mtalLayer	mtalLayer can be defined as a string or LPP.
cutLayer	cutLayer can be defined as a string or LPP.
extension	The length to extend the wire beyond the cut.
spacing	The distance between the end-to-end wires.
-width	Width of the wires.
-distance	Distance between the end of the common length and the border of the proximity box.
-proximityBox	Dimensions of the proximity box (D2 D1).
-parallelRunLength	Length that is common to all five wires. The common length must be greater than parallelRunLength.
-distanceWithin	Distance between the cut and the end of the parallelRunLength. The cut must be within this distance.
-parallelEdgeSpace	Distance between the parallel wires.
-cutClass	List of paired float values or a list of cutClass names. See cutClass and cutClassList . The constraint applies only in cases where the cuts comply with these specifications.

Example:

```
denseEndOfLineExtensionSpacing metall via1 0.75 0.45
    -width 0.25 -distance 0.78
    -proximityBox 0.23 0.56 -cutClass ((0.4 0.6) (0.6 0.8))
    -parallelEdgeSpace 0.35 -parallelRunLength 0.75
    -distanceWithin 0.26
```

forbiddenTwoSidedViaClearance

This constraint is a custom asymmetric layer pair constraint which is used when some processes impose additional requirements on clearance between via cuts like when a via cut on one layer is located between two via cuts on another layer.



Constraint Category	Clearance
Constraint Type	Complex
OA Rule	Custom Layer Constraint
Layer Symmetry	Asymmetric

Syntax:

```
forbiddenTwoSidedViaClearance layers | LPPs value params [-isHard
true|false]
[-comment ["rule id"] "string"]

layers::= layerName1 layerName2
LPPs::= (layerName1 purposeName1) (layerName2 purposeName2)
value::= rangeData

[-spacingDirection any|horizontal|vertical]
[-cutDistance floatValue]
[-otherParallelRunLength floatValue]
[-parallelRunLength floatValue]
```

Argument	Description
oaSpacingDirection	Optional parameter defining the direction in which the constraint applies. Values are 0 (any), 1 (horizontal direction), 2 (vertical direction).
oaCutDistance	Distance from the cut shape in the middle to the cut shape on the other side must be less than this value for the constraint to apply. The value is in database units.
otherParallelRunLength	Optional parameter requiring minimum parallel run length between the cut shape in the middle and the cut shape on the other side. The parallel run length must be greater than or equal to this value in database units for the constraint to apply.

Argument	Description
parallelRunLength	Optional parameter requiring minimum parallel run length between the cut shapes. The parallel run length must be greater than or equal to this value in database units for the constraint to apply.

For example:

```
forbiddenTwoSidedViaClearance VIA1 VIA2 (0.07 <=0.08)
                                -spacingDirection any
                                -cutDistance 0.03
                                -otherParallelRunLength
0.01
forbiddenTwoSidedViaClearance v1 v2 (0.07 <=0.08)
                                -parallelRunLength 0.02
                                -spacingDirection vertical
                                -cutDistance 0.03
                                -otherParallelRunLength
0.01
                                -parallelRunLength 0.02
```

generalCutClassClearance

Both [minCutClassClearance](#) and [oaMinViaClearance](#) constraints are commonly used to model cut clearance rules. [minCutClassClearance](#) is a table-based constraint which makes it hard to use while [oaMinViaClearance](#) constraint cannot be used when there are additional fixed clearance values allowed in addition to the minimum value. To overcome these limitations, this constraint is introduced which replaces the usage of [minCutClassClearance](#) and [oaMinViaClearance](#) in most cases.

Constraint Category	Clearance
Constraint Type	Complex
OA Rule	Custom Layer Constraint
Layer Symmetry	Symmetric

Syntax:

```
generalCutClassClearance layers | LPPs value params [-isHard
true|false]
[-comment ["rule id"] "string"]
layers::= layerName1 layerName2
LPPs::= (layerName1 purposeName1) (layerName2 purposeName2)
value::= FloatValue | rangeValue | rangeValueArray
[-cutSpacingType edgeX|edgeY |corner |center | edge]
[-cutClassList1 cutClassArray|cutClassNameList]
[-cutClassList2 cutClassArray|cutClassNameList]
```



```
[-connectivityType anyConnectivityType |
sameNetConnectivityType |
contiguousShapesConnectivityType |
directShapesConnectivityType |
sameViaConnectivityType]
[-exceptConnectivityType      anyConnectivityType |
sameNetConnectivityType |
contiguousShapesConnectivityType |
sameViaConnectivityType |
directShapesConnectivityType ]
[-xParallelRunLengthRange rangeValue]
[-yParallelRunLengthRange rangeValue]
[-selfAlignViaType1 [any | selfAlign| SA| nonSelfAlign |NSA ]
[-selfAlignViaType2 [any | selfAlign| SA| nonSelfAlign |NSA ]
[-selfAlignEdgeType1 [any | selfAlign| SA| nonSelfAlign|NSA ]
[-selfAlignEdgeType2 [any | selfAlign| SA| nonSelfAlign |NSA ]
[-exactAligned intValue]
[-exceptExactAligned true | false]
[-voltageRange frangeData]
[-stack true/false]
```

Argument	Description
cutSpacingType	Enumeration indicating how the clearance between the cuts is measured.
cutClassList1	Array of <code>oaDualInt</code> values, each defining width and height of a cut class in database units. If given, the constraint only applies to those cuts on the first layer (layer1) which cut class is from this cut class list.
cutClassList2	Array of <code>oaDualInt</code> values, each defining width and height of a cut class in database units. If given, the constraint only applies to those cuts on the second layer (layer2) which cut class is from this cut class list.
oaConnectivityType	Indicates whether the constraint applies only if the cuts are on the same net, same metal, same wire segment, same via, or any.
exceptConnectivityType	Specifies the type of connectivity between the two cuts that causes the constraint to not apply.
xParallelRunLengthRange	Optional parameters defining the x and y parallel run length range within which the constraint to apply. The default values of these two parameters depend on cutSpacingType: edgeX: yParallelRunLengthRange >=0. edgeY: xParallelRunLengthRange >=0. edge: yParallelRunLengthRange >= 0 for horizontal spacing, xParallelRunLengthRange >=0 for vertical spacing. corner: x yParallelRunLengthRange < 0
yParallelRunLengthRange	
selfAlignViaType1	Enumeration indicating the type of self-aligned vias on layer1 this constraint applies to.
selfAlignViaType2	Enumeration indicating the type of self-aligned vias on layer2 this constraint applies to.

Argument	Description
selfAlignEdgeType1	Enumeration indicating the type of edges on layer1 this constraint applies to. An edge can either be a self-aligned edge, which only exists for a self-aligned via, or a non-self-aligned edge, which could come from either a self-aligned or a non-self-aligned via.
selfAlignEdgeType2	Enumeration indicating the type of edges on layer2 this constraint applies to. An edge can either be a self-aligned edge, which only exists for a self-aligned via, or a non-self-aligned edge, which could come from either a self-aligned or a non-self-aligned via.
exactAligned	If given and value is '1', constraint applies only when two facing edges have the exact same size and also fully aligned. Ignore if value is not '1'. This parameter cannot be 1 when <code>exceptExactAlignedparameter</code> is true.
exceptExactAligned	If true, the constraint applies only between two edges which face each other (the parallel run length is greater than or equal to zero) but are not fully aligned (the length of the edges is different or the parallel run length is less than the edge length). This parameter cannot be true when <code>exactAligned</code> parameter is 1.
voltageRange	If given, the rule only applies if the difference between voltages of the cuts lies in this range in volts (see <code>Voltage Dependent Design Rules</code>).
oaStack	This parameter controls whether via cuts on layer1 and layer2 can be stacked, meaning, if they can overlap each other.

For example:

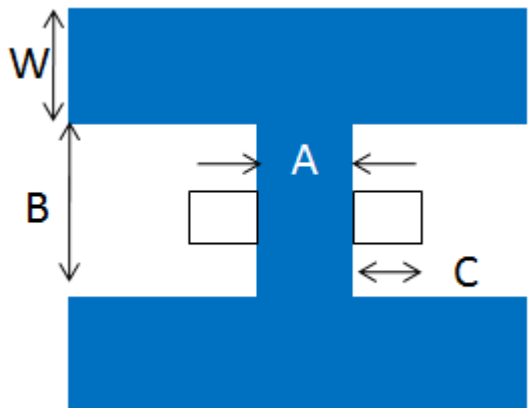
```
generalCutClassClearance VIA1 VIA2 0.06
                                -cutSpacingType edgeX
                                -connectivityType
    sameNetConnectivityType
                                -exceptConnectivityType sameVia
                                -exactAligned

// generalCutClassClearance constraint specified with cutClass
generalCutClassClearance v1 v2 0.08
                                -cutSpacingType corner
                                -cutClassList1 ( (0.02 0.04) (0.06
0.08) )
                                -cutClassList2 ( V2LRG V2BAR_40_40)
                                -xParallelRunLengthRange (0.07 <=0.08)
                                -yParallelRunLengthRange (0.08 <=0.09)
                                -voltageRange (1.5 2)
```

illegalHGatePattern

If a gate is connected to shapes on the same layer, it is illegal to form an H-shaped pattern in any of the following cases:

- The channel length (A)* of the gate is less than the specified length.
- The distance between the horizontal rails of poly (B)* is less than the specified distance.
- The width of the poly rails (W)* is less than the specified width.
- The extension of the poly rails (C)* from the connecting gate shape is greater than the given extension.



Constraint Category	Length and Width Process Rules
Constraint Type	Complex
OA Rule	oacIllegalHGatePattern
Layer Symmetry	Asymmetric

Syntax:

```
illegalHGatePattern
  <string|LPP>gateLayer
  <string|LPP>polyLayer
  <float>minWidth
  -width float
  -length float
  -interSpace float
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
gateLayer	gateLayer can be defined as a string or a LPP.
polyLayer	polyLayer can be defined as a string or a LPP.
minWidth	Specifies the minimum gate width.
-width	This parameter specifies the width(W) of the poly rails in microns.
-length	This parameter specifies the extension of the poly rails(C) in microns.
-interSpace	This parameter specifies the distance(B) between the horizontal poly rails.

For example:

```
illegalHGatePattern gate poly 0.07 -width 0.05  
                                -length 0.06  
                                -interSpace 0.10
```

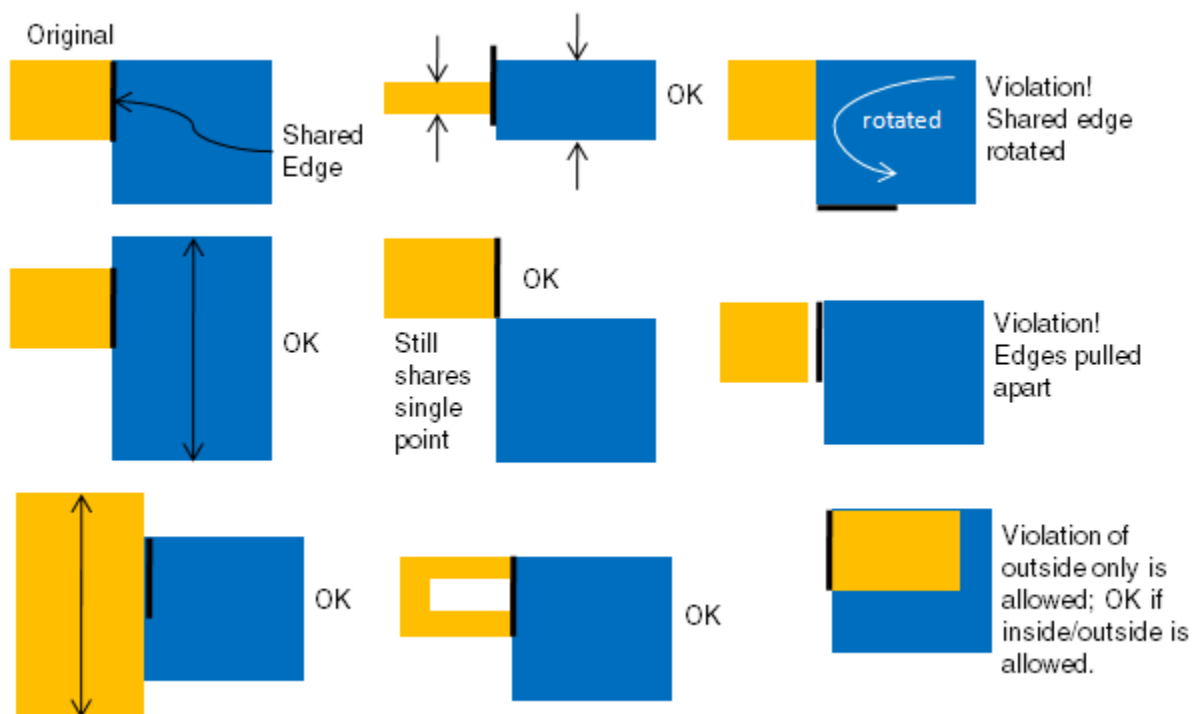
keepAlignedShapes

This layer pair constraint specifies that shapes with coincident edges on two layers must retain coincident edges.

The coincident edges of the shape on one layer can be inside, outside, or either inside or outside the shape on the second layer. This is defined using parameters. At least one shared point on the shared edge must be maintained on the shape. However, the length of the shared edge can change without any restrictions.

The constraint is also subject to the following considerations:

- If an application separates two shapes that have shared edges, other applications cannot enforce the constraint on those two shapes.
- If an application moves a shape on one layer so that it shares an edge with a shape on the other layer specified by the constraint, the constraint takes effect on those two shapes.
- This constraint can be applied to any two oaShapes that have a straight edge on the specified two layers.



Constraint Category	Placement and Alignment
Constraint Type	Complex
OA Rule	oacKeepAlignedShapes
Layer Symmetry	Symmetric

Syntax:

```
keepAlignedShapes
<string|LPP>layer1
<string|LPP>layer2
<bool>alignmentValue
[-alignmentType outside|inside|insideOutside]
[-isHard true|false]
[-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer1	layer1 can be defined as a string or a LPP.
layer2	layer2 can be defined as a string or a LPP.
alignmentValue	Specifies if the shapes with coincident edges on two layers must retain coincident edges.

Argument	Description
-alignmentType	• outside: Use this option to enforce outside alignment of shapes. • inside: Use this option to enforce inside alignment of shapes. • insideOutside: Use this option to enforce both inside and outside alignment of shapes (default).

For example:

```
keepAlignedShapes (m1 drawing) (m1 fill) true -alignmentType outside
```

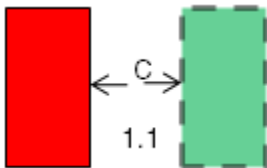
maxClearance

This layer pair constraint specifies the maximum distance separating two different shapes on two layers. The value is specified in microns. This constraint applies only to edges that have no intersections.

Clearances are specified from the outside edge of the shape on layer1 to the outside edge of the shape on layer2.

This constraint definition is symmetric, implying that the maximum clearance between layer1 and layer2 is the same as the maximum clearance between layer2 and layer1.

This constraint is made available to constrain design tools for layout strategy or floor planning purposes and not as a foundry constraint.



Constraint Category	Clearance
Constraint Type	Simple
OA Rule	oacMaxClearance
Layer Symmetry	Symmetric

Syntax:

```
maxClearance
  <string|LPP>layer1
  <string|LPP>layer2
  <float>spacingValue
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer1	layer1 can be defined as a string or LPP.
layer2	layer2 can be defined as a string or LPP.
spacingValue	Specifies the maximum spacing between shapes on layer1 and layer2.

Rule	Description	Value (in microns)
PO.CL.1	Maximum spacing between POLY and DIFF	1.1

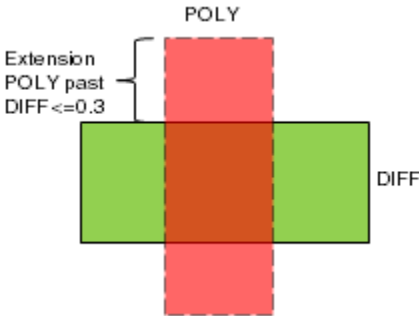
maxClearance POLY DIFF 1.1

maxExtension

This layer pair constraint specifies the maximum distance extension that a shape on one layer can have beyond a shape on another layer.

Extensions are measured from the outside edge of the shape on layer2 to the inside edge of the shape on layer1.

This constraint definition is not symmetric. This indicates that the maximum extension of layer1 past layer2 is not the same as the maximum extension of layer2 past layer1.



Constraint Category	Extension
Constraint Type	Complex
OA Rule	oacMaxExtension
Layer Symmetry	Asymmetric

Syntax:

```
maxExtension
  <string|LPP>layer1
  <string|LPP>layer2
  <float|float1DTable>extensionValue
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer1	layer1 can be defined as a string or a LPP.
layer2	layer2 can be defined as a string or a LPP.
extensionValue	• float: Specifies the maximum extension. • float1DTable: Specifies the width of the shape on the second layer and the maximum extension of that second shape past the first.

In this example, the maximum extension of POLY over DIFF (gate endcap) is 0.3

Rule	Description	Value (in microns)
PO.EX.4	POLY extension over DIFF	<= 0.3

```
maxExtension POLY DIFF 0.3 -rule "PO.EX.4"
```

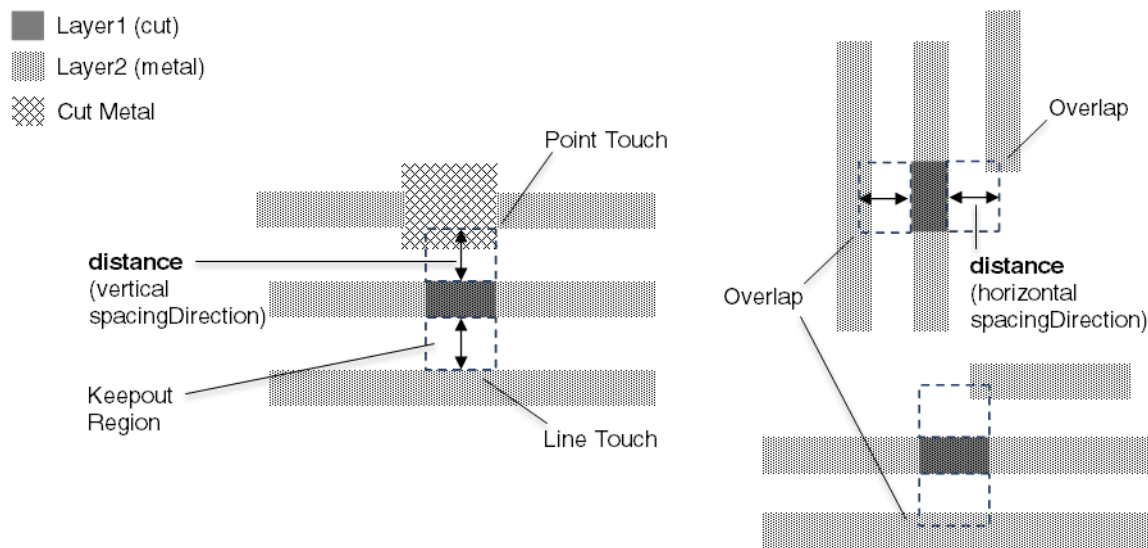
In this example, the maximum extension of POLY over DIFF (gate endcap) if the width of the DIFF is >= 3u is 0.5

Rule	Description	Value (in microns)
PO.EX.4.1	POLY extension over DIFF for DIFF width >= 3u	<= 0.5

```
maxExtension POLY DIFF 0.3 ( (3 0.5) ) -rule "PO.EX.4.1"
```

maxInteractCutKeepout

The maxInteractCutKeepout constraint limits the number of metal shapes that are permitted at the boundary of a cut shape’s keepout region.



Constraint Category	Spacing
Constraint Type	Complex
OA Rule	Custom Compiler Custom Layer Pair Constraint)
Layer Symmetry	Asymmetric

Syntax:

```
maxInteractCutKeepout
  <string|LPP>cutLayer
  <string|LPP>metalLayer
  <int>numInteractions
  -distance float
  [-spacingDirection any | horizontal | vertical]
  [-exceptLineTouch rangeData]
  [-exceptPointTouch true | false]
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
cutLayer	cutLayer can be defined as a string or a LPP.
metalLayer	metalLayer can be defined as a string or a LPP.
numInteractions	Maximum number of metal shapes permitted at the boundary of the cut shape keepout region.
-distance	Width of the cut's keepout region.

Argument	Description
-spacingDirection	Direction of distance. Valid values are any, horizontal, and vertical.
-exceptLineTouch	Length within which the cut and the metal shapes are interacting. You can specify a range of lengths. See Range Data Syntax .
-exceptPointTouch	Indicates whether point (or corner-to-corner) touches count toward the number of interactions in the keepout region. Valid values are true and false. Default: false

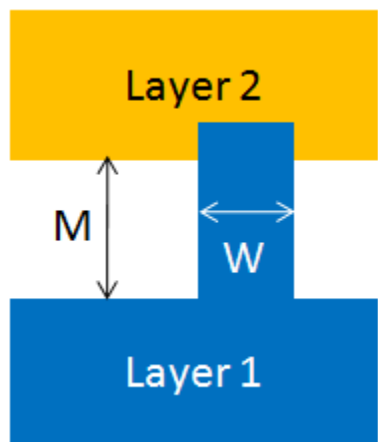
For example:

```
maxInteractCutKeepout via1 metall 2
-distance 0.09
-spacingDirection vertical

maxInteractCutKeepout via1 metall 2
-distance 0.11
-spacingDirection horizontal
-exceptPointTouch true
-exceptLineTouch (>= 0.12 <=0.3)
```

maxTouchingDirectionLength

This constraint limits the maximum length for certain shapes in a certain direction. The maximum length would depend on the width of the shape, which is on layer1. The shape on layer2 would indicate the direction in which the length is measured.



Constraint Category	Length and Width Process Rules
Constraint Type	Complex
OA Rule	oacMaxTouchingDirectionLength
Layer Symmetry	Asymmetric

Syntax:

```
maxTouchingDirectionLength
  <string|LPP>layer1
  <string|LPP>layer2
  <float1DTable>maxLength
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer1	layer1 can be defined as a string or a LPP.
layer2	layer2 can be defined as a string or a LPP.
maxLength	Specifies the width of layer1 shape on the second layer and the maximum length of that second shape past the first.

For example:

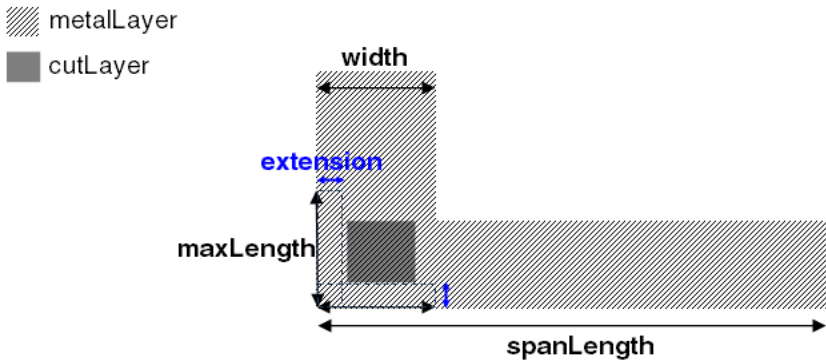
```
maxTouchingDirectionLength L1 L2 0.18 ((0.06 0.10) (0.08 0.12))
```

metalJointViaExtension

The `metalJointViaExtension` constraint requires a minimum enclosure around a via when it is at the joint of two perpendicular metal shapes. For the constraint to apply, the following conditions must be met:

- One of the metal shapes must be wider than the specified value (`width`).
- The narrower metal shape must be longer than `spanLength`.

The enclosure around the cut shape is determined by `extension` and `maxLength`.



Constraint Category	Extension
---------------------	-----------

Constraint Type	Complex
OA Rule	Custom Compiler Custom Layer Pair Constraint
Layer Symmetry	Asymmetric

Syntax:

```
metalJointViaExtension
  <string|LPP>metalLayer
  <string|LPP>cutLayer
  <float>extension
  -spanLength float
  [-width float]
  [-maxLength float]
  [-cutClass <cutClassArray | cutClassNameList>]
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
metalLayer	metalLayer can be defined as a string or a LPP.
cutLayer	cutLayer can be defined as a string or a LPP.
extension	Distance, in microns, between the edge of the via and the edge of the metal shape.
-spanLength	Length, in microns, of the narrower metal shapes.
-width	Width, in microns, of the wider metal shape.
-maxLength	Length, in microns, of the enclosure. The length runs parallel to the metal shapes.
-cutClass	List of paired float values or a list of cutClass names. See cutClass and cutClassList . The constraint applies only in cases where the cuts comply with these specifications.

For example:

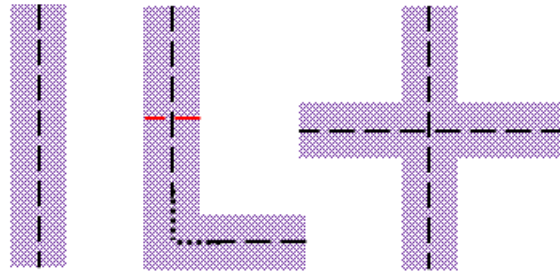
```
metalJointViaExtension M1 V1 0.02
                        -width 0.23 -spanLength 0.04
                        -maxLength 0.06 -cuClass ((0.06 0.08) (0.4 0.6))
```

minCenterlineExtension

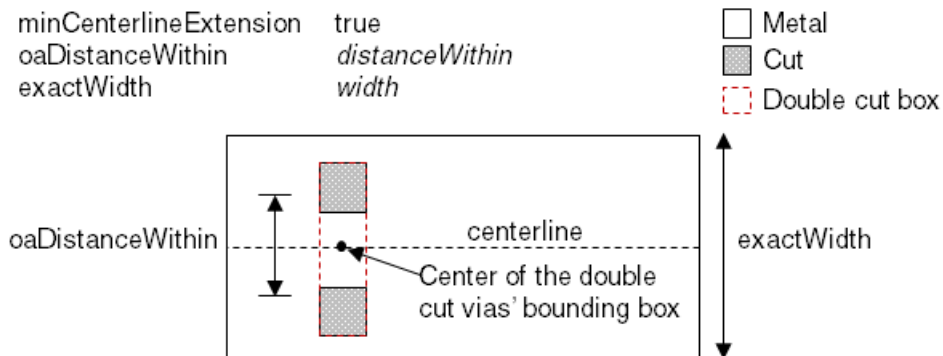
In some advanced processes, cut shapes must lie on the centerline of the shapes it connects for shapes less than a given width. Specifically, anchor points are defined for a cut shape. For a single cut via, the anchor point is the center of the cut shape. For a double cut vias within a particular cut distance, the anchor point is the center of the bounding box of the two cut shapes, and it must lie on the centerline. The two vias must

be within a given distance, must be aligned, and must be on shapes of at least a given width. The centerline must be pulled back from the end of the line when the end of the line falls in a specified width range. The definition of the centerline is straightforward in the case of a rectangle, but can be more complex in the case of an arbitrary polygon. The rule is specific to a given cut class.

Style 1 Centerline Calculation



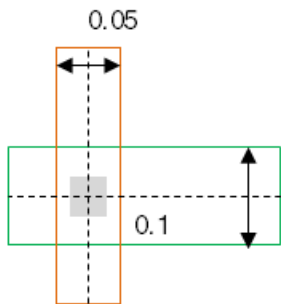
-  Regular Centerline Calculation
-  Extrapolated Centerlines (qualification 2)
-  Removed Centerlines (qualification 1)



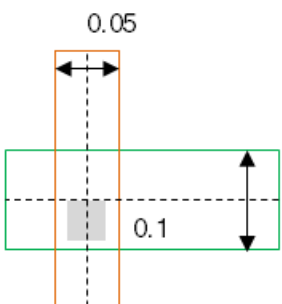
For double cut vias less than `oaDistanceWithin` apart, touching a shape that is exactly of `exactWidth` wide, the center of the bounding box for the cuts must be on the centerline

minCenterlineExtension true
centerLineDef 1
maxEffectWidth 0.13
cutClass {0.04 0.04} VA

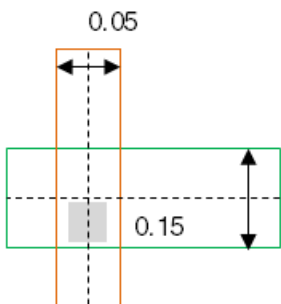
 Metal
 Metal
 Cut class VA



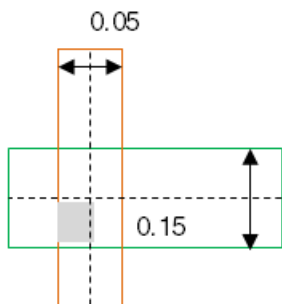
a) OK, the center of the cut is on the centerlines of both wires



b) Violation, the center of the cut is not on the centerline of the red wire



c) OK, the width of the green wire is 0.15 (≥ 0.13), so the constraint does not apply to that wire.



d) Violation, the center of the cut is not on the centerline of the red wire. The constraint does not apply to the green wire.

Constraint Category	Extension
Constraint Type	Complex
OA Rule	oacMinCenterlineExtension
Layer Symmetry	Asymmetric

Syntax:

```
minCenterlineExtension
  <string|LPP>metalLayer
  <string|LPP>cutLayer
  <bool>value
  [-cutClass <cutClassArray | cutClassNameList>]
  -centerLineDef int
```

```
-maxEffectiveWidth float
[-endOfLineWidthRange dualValue]
[-distanceWithin float]
[-exactWidth float]
[-isHard true|false]
[-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
metalLayer	metalLayer can be defined as a string or a LPP.
cutLayer	cutLayer can be defined as a string or a LPP.
value	The existence of this constraint, along with the various parameters which define how it is applied.
-cutClass	List of paired float values or a list of cutClass names. See cutClass and cutClassList . The constraint applies only in cases where the cuts comply with these specifications.
-centerLineDef	Style 1 of calculating centerline is only defined for orthogonal polygons. It is defined as the median line between two parallel internal edges of the polygon, subject to some qualifications. The two qualifications are: 1.The median line is not considered a centerline if the length of the median line is less than the distance between the two parallel edges, and the line touches two edges of the polygon. 2.The definition of the centerline for irregular regions of the polygon, where no median line is present, is derived by finding the center point of each rectangular region and extrapolating the centerlines from neighboring regions. If the extrapolated centerline of each neighboring region touches the center point of the irregular region, those extrapolated extensions are also considered centerlines of the polygon (See above diagrams).
-maxEffectiveWidth	The maxEffectiveWidth value is required as part of the centerline calculation if the centerLineDef parameter is 1. The rule does not apply if the shape is wider than the maxEffectiveWidth.
-endOfLineWidthRange	The centerline cannot intersect with the end of the line if the width of the end of the line falls in the range specified by this parameter.
-distanceWithin	For double cut vias less than this distance apart and touching a shape that is exactly of exactWidth in width, the center of the bounding box must lie on the centerline. This is a mandatory parameter only if exactWidth is specified.
-exactWidth	This is a mandatory parameter only if distanceWithin is specified.

For example:

```
minCenterlineExtension m1 v1 true -cutClass 0.06 0.08
                        -centerLineDef 1
                        -maxEffectiveWidth 0.13
```

```
-distanceWithin 0.18
-exactWidth 0.15
```

minClearance

This layer pair constraint specifies the minimum separation between two different shapes on two different layers.

Clearance constraints apply to edges that have no intersections. Clearances are measured from the outside edge of the shape on layer1 to the outside edge of the shape on layer2.

This constraint definition is symmetric. The minimum clearance between layer1 and layer2 is the same as the minimum clearance between layer2 and layer1.

Constraint Category	Clearance
Constraint Type	Complex
OA Rule	oacMinClearance
Layer Symmetry	Symmetric

Syntax:

```
minClearance
  <string|LPP>layer1
  <string|LPP>layer2
  <float>spacingValue
  [-distanceMeasureType euclidian|manhattan]
  [-spacingDirection any|horizontal|vertical]
  [-coincidentAllowed true|false]
  [-exceptSameNetConnectivity true|false]
  [-parallelRunLengthRange rangeData]
  [-widthRange rangeData]
  [-otherWidthRange rangeData]

  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Syntax:

```
minClearance
  <string|LPP>layer1
  <string|LPP>layer2
  <float2DTable>spacingValue
  [-distanceMeasureType euclidian|manhattan]
  [-widthLengthTableType
    widthWidthTableType|
```



```

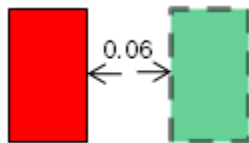
widthParallelRunLengthTableType |
twoWidthParallelRunLengthTableType]
[-spacingDirection horizontal|vertical|any]
[-coincidentAllowed true|false]
[-exceptSameNetConnectivity true|false]
[-parallelRunLengthRange rangeData]
[-widthRange rangeData]
[-otherWidthRange rangeData]
[-isHard true|false]
[-comment [<string rule_id>] <string rule_desc>]

```

Argument	Description
layer1	layer1 can be defined as a string or a LPP.
layer2	layer2 can be defined as a string or a LPP.
spacingValue	• float: Specifies the minimum spacing between the two shapes • float2DTable: The value of which depends on the widthLengthTableType argument. For widthLengthTableType: • widthWidthTableType: The rows and the columns represent the widths of the two shapes being analyzed, the float value represents the minimum spacing between the shapes. • widthParallelRunLengthTableType: The row ("width") represents the width of the wider of the two shapes, the column ("length") represents the parallel run length between the two shapes, and the float value represents the minimum spacing required between the two shapes. • twoWidthParallelRunLengthTableType: The rows represent the widths of the shapes and the columns represent the parallel run lengths between the two shapes, the float value represents the minimum spacing required between the two shapes.
-distanceMeasureType	This parameter determines whether the spacing values specified in the table values of the constraint are in either a Euclidian or Manhattan metric. Default value is "euclidean".
-widthLengthTableType	• This parameter determines how the value of the constraint is interpreted; whether the value is based on the widths of two shapes or is based on the width of the wider shape and the parallel run length of the other shape. • widthWidthTableType: Use this option to designate that the constraint applies to the widths of the two shapes • widthParallelRunLengthTableType: Use this option to designate that the constraint applies to the wider of two adjacent shapes and their parallel run length (default) • twoWidthParallelRunLengthTableType: Use this option to designate that the constraint applies to the width of both shapes and the parallel run length

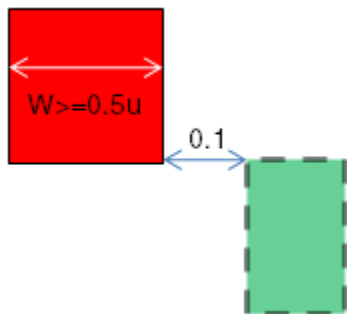
Argument	Description
-spacingDirection	- Specifies the spacing measure direction: horizontal: Specifies the horizontal spacing direction. vertical: Specifies the vertical spacing direction. any: Specifies the spacing direction that the constraint applies. Can be either horizontal or vertical (default).
-coincidentAllowed	Indicates if the edges of shapes can be coincident or not. Default is false.
-exceptSameNetConnectivity	If set to true, and the connectivityType is any, then the constraint applies only when the two shapes are on different nets.
-parallelRunLengthRange	It represents the range of parallel run length (PRL) in which the constraint applies. Default: none
-widthRange	Constraint applies only if at least one shape width is in the range. Default: none
-otherWidthRange	Constraint applies only if one shape width is in the range of -widthRange and the other is in the range of -otherWidthRange. Default: none

Rule	Description	Value (in microns)
PO.CL.1	Minimum POLY -DIFF spacing	0.06



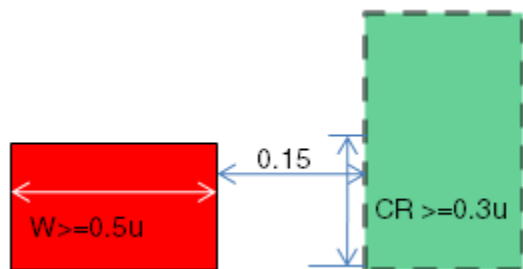
minClearance POLY DIFF 0.06

Rule	Description	Value (in microns)
PO.CL.1.1	Minimum POLY - DIFF spacing for POLY width $\geq 0.5u$	0.1



mminClearance POLY DIFF 0.06 ((0.5 0) 0.1))

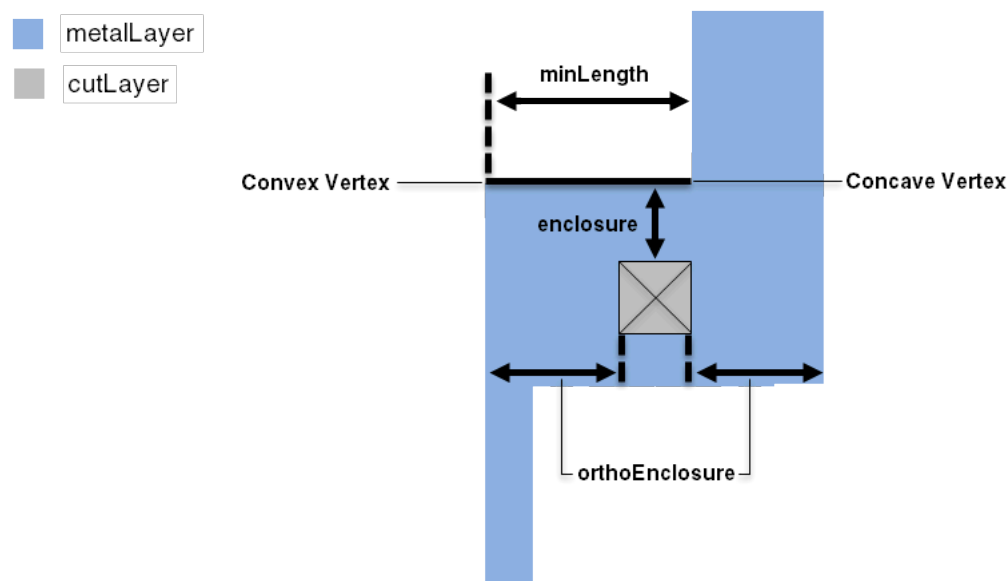
Rule	Description	Value (in microns)
PO.CL.1.2	Minimum POLY - DIFF spacing for POLY width >=0.5u and Common Run is >= 0.3u	0.15



minClearance POLY DIFF 0.06 ((0.5 0.3) 0.15))

minConcaveConvexEdgeExtension

The `minConcaveConvexEdgeExtension` constraint specifies the requirements for enclosing via cuts when the edges of the connecting layer consists of concave and convex vertices.



Constraint Category	Extension
Constraint Type	Complex
OA Rule	Custom Compiler custom layer pair constraint
Layer Symmetry	Asymmetric

Syntax:

```
minConcaveConvexEdgeExtension
<string|LPP>metalLayer
<string|LPP>cutLayer
-minLength float
[-orthoExtension float float]
[-cutClass <cutClassArray | cutClassNameList>]
[-isHard true|false]
[-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
metalLayer	metalLayer can be defined as a string or a layer-purpose pair.
cutLayer	cutLayer can be defined as a string or a layer-purpose pair.
-minLength	The minimum distance between the convex vertex and the concave vertex. The constraint applies when the distance is < the specified minLength value.

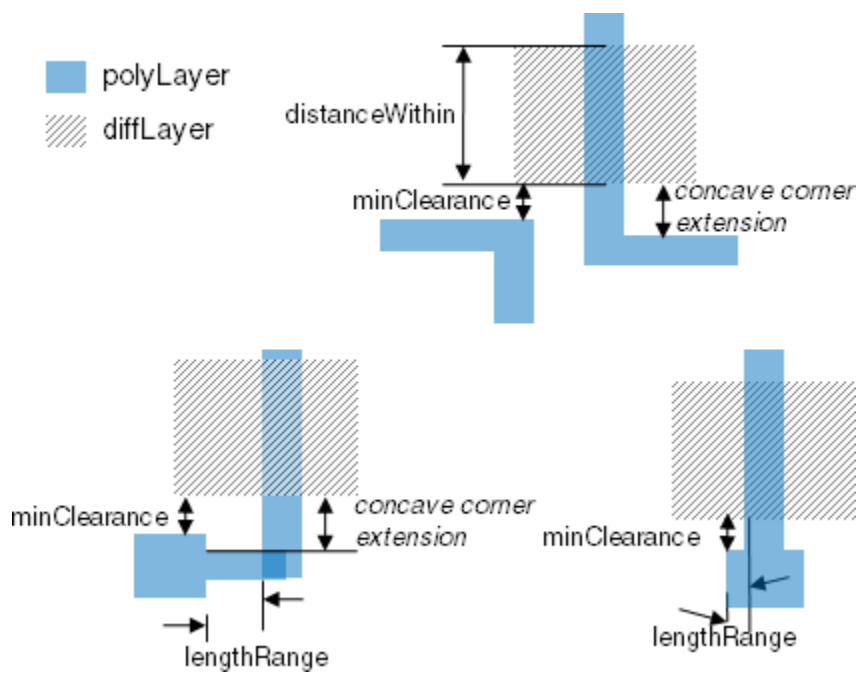
Argument	Description
-orthoExtension	The requirements for extending the connecting layer and enclosing the via. The requirements are based on the following two values, in the order shown: • enclosure • orthoEnclosure If <code>minLength</code> < the specified <code>enclosure</code> value, then the orthogonal edges (the edges opposite the concave and convex vertices) must be >= the specified <code>orthoEnclosure</code> value. If <code>minLength</code> >= the specified <code>enclosure</code> value, then the orthogonal edge requirement does not apply.
-cutClass	List of paired float values or a list of cutClass names. See cutClass and cutClassList. The constraint applies only in cases where the cuts comply with these specifications.

In the following example, the constraint applies to vias that are 0.4 x 0.6 when the `minLength` is less than 0.15. If the `minLength` < 0.02 (the `enclosure` value), then the orthogonal edges must be at least 0.06 or longer.

```
minConcaveConvexEdgeExtension metall vial 0.25
                                -minLength 0.15
                                -orthoExtension 0.02 0.06
                                -cutClass ((0.4 0.6) (0.6 0.8))
```

minConcaveCornerExtension

This constraint specifies a minimum extension of a shape on layer1 over a shape on layer 2, as measured from the facing edge of layer2 to an inside corner on the layer1 shape. This constraint also applies when the inside corner is 90-degrees.



Constraint Category	Extension
Constraint Type	Complex
OA Rule	oacMinConcaveCornerExtension
Layer Symmetry	Asymmetric

Syntax:

```
minConcaveCornerExtension
  <string|LPP>polyLayer
  <string|LPP>diffLayer
  <float>extensionValue
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
polyLayer	polyLayer can be defined as a string or a LPP.
diffLayer	deffLayer can be defined as a string or a LPP.
extensionValue	Minimum extension of layer1 LShape extends layer2 shape.

For example:

```
minConcaveCornerExtension poly gate 0.04
```

minConcaveCornerOverlap

This layer pair constraint specifies a minimum overlap from the inside corner of a shape on one layer to the edge of the layer that encloses it. This can apply to shapes with both 45 and 90 degree bends.

Constraint Category	Overlap
Constraint Type	Simple
OA Rule	oacMinConcaveCornerOverlap
Layer Symmetry	Asymmetric

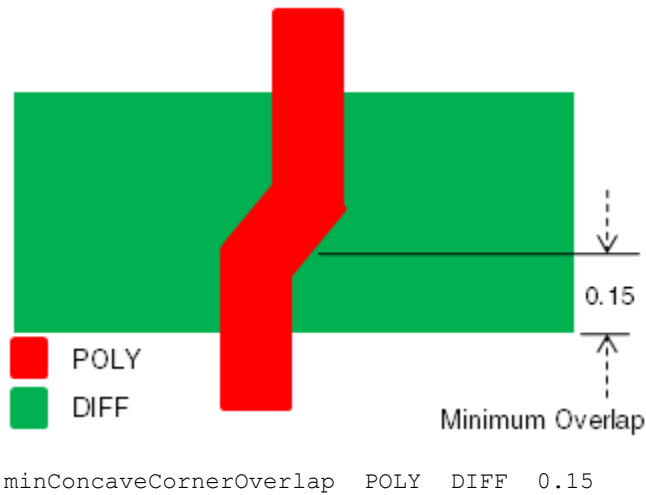
Syntax:

```
minConcaveCornerOverlap
  <string|LPP>polyLayer
  <string|LPP>diffLayer
  <float>voltageValue
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
polyLayer	polyLayer can be defined as a string or LPP.
diffLayer	diffLayer can be defined as a string or LPP.
overlapValue	Specifies a minimum overlap from the inside corner of a shape on one layer to the edge of the layer that encloses it.

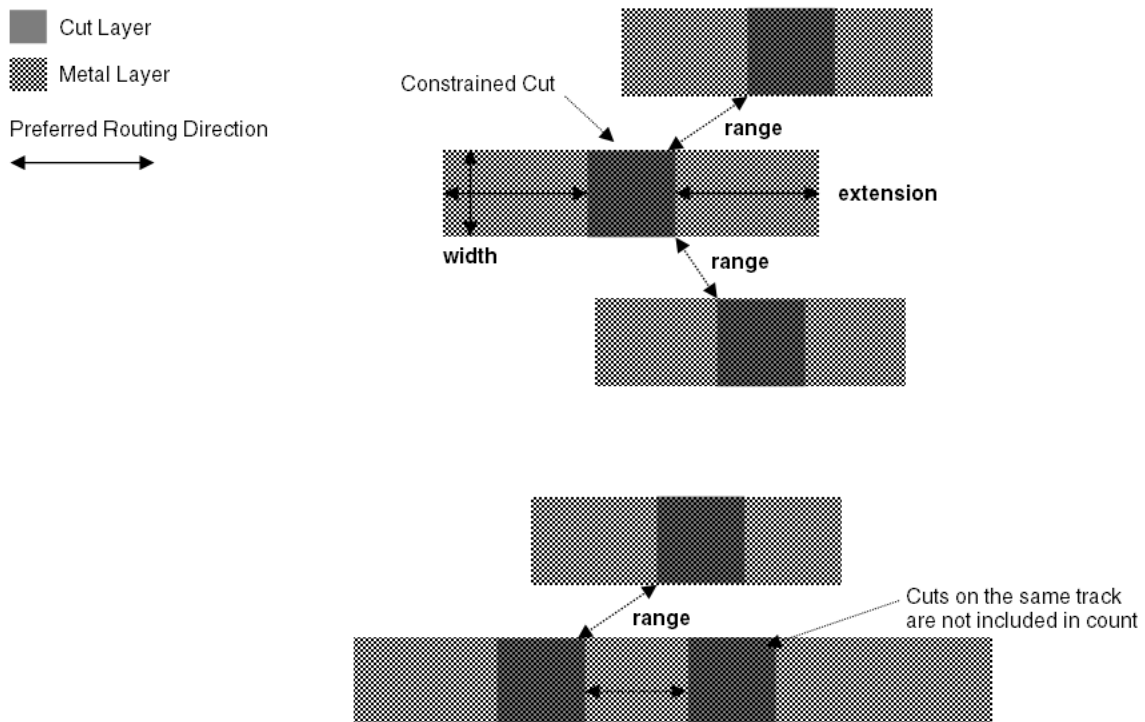
In this example, the minimum overlap of DIFF from the 45 degree POLY bend is mentioned:

Rule	Description	Value (in microns)
PO.OV.2	Minimum overlap of DIFF from the 45 degree bend of POLYU	0.15



minConstrainedViaExtension

Advanced processes can have special via extension rules when a cut is a certain distance from other cuts on the same layer. A constrained via cut is one that is a certain distance from another via cut on the same layer and mask. The extension applies in the preferred routing direction on both sides of the cut shape.



Constraint Category	Via Construction
Constraint Type	Complex
OA Rule	Custom Compiler Custom Layer Pair Constraint
Layer Symmetry	Asymmetric

Syntax:

```
minConstrainedViaExtension
<string|LPP>metalLayer
<string|LPP>cutLayer
<float>extensionValue
-width float
-range rangeData
[-cutClass <cutClassArray | cutClassNameList>]
-count int
    [-parallelRunLengthRange rangeData]
    [-distanceMeasureType euclidean| manhattan]
    [-color colorName | any | same]
[-isHard true|false]
[-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
metalLayer	metalLayer can be a string (layer name) or LPP.
cutLayer	cutLayer can be a string (layer name) or LPP.
extensionValue	Minimum distance (extension) of layer2 shape, in microns, required in the specified direction for a constrained via cut on layer1.
-cutClass	List of paired float values or a list of cutClass names. See cutClass and cutClassList . The constraint applies only in cases where the cuts comply with these specifications.
-width	Width of the wire, in microns. The wire width must be less than or equal the specified width value.
-range	Distance, in microns, between adjacent cuts. The distance must be within the specified range. See Range Data Syntax .
-count	Number of neighbor vias that are not on the same track within the <code>range</code> distance. The count must be satisfied for the constraint to apply.
-parallelRunLengthRange	If given, parallel run length between the constrained cut and the neighboring cut on the other track must lie in this range in database units for the neighboring cut to be considered.
-distanceMeasureType	Optional parameter defining how the distance between cuts is measured to check if a cut is constrained. If <code>manhattan</code> , the <code>parallelRunLengthRange</code> parameter must also be specified with a finite lower bound.

Argument	Description
-color	If positive number is specified, the constraint only applies if mask color of the metal shape enclosing the constrained cut matches this value. A value of zero means any color.

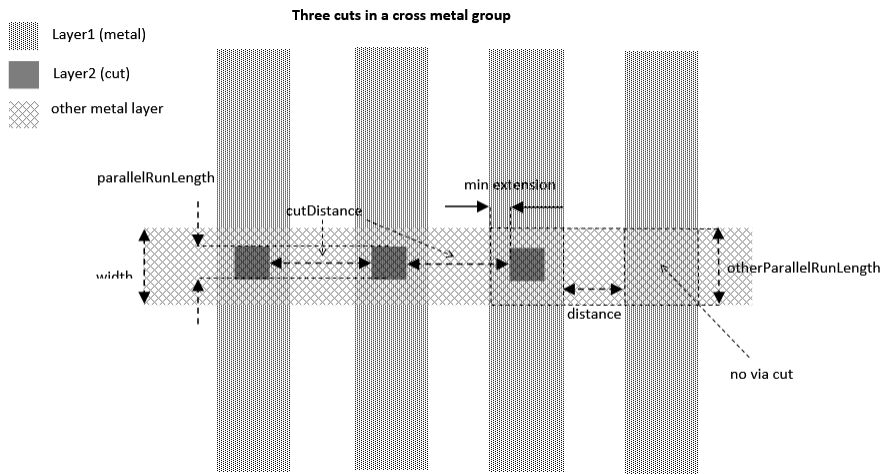
Example

```
minConstrainedViaExtension M1 V1 0.75 -width 0.23
    -range (>=0.25 <=0.38) -count 4
    -cutClass ((0.04 0.06) (0.6 0.8))
    -parallelRunLengthRange (>=0.30 <=0.35)
    -distanceMeasureType manhattan
    -color red
```

minCrossMetalViaGroupExtension

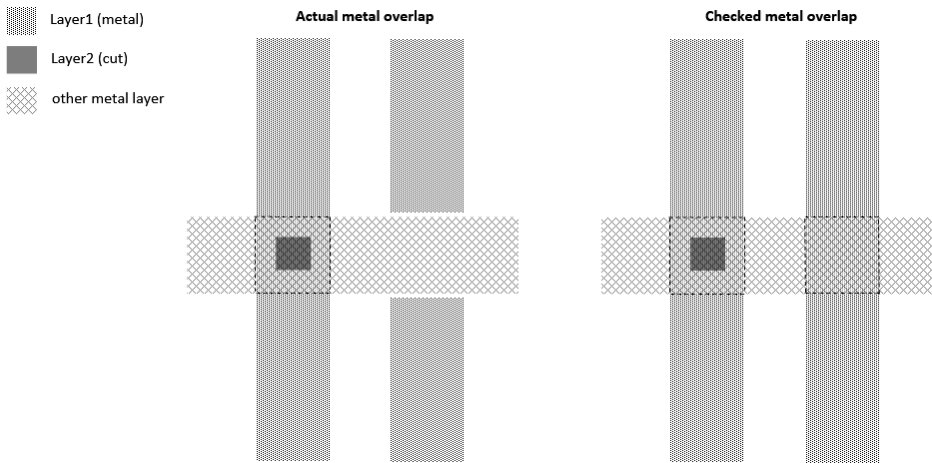
Advanced processes may require different metal extension over via cuts forming a group across metal wires when there is a metal shape nearby without a via. The rule only applies when there is a minimum number of cuts in the cross metal via group. It also only applies to via cuts located close enough to another metal shape without a via. The requirements for the nearby metal shape can be different for upper and lower metal layer extension.

In the following graphic, a cross metal via group is formed across lower metal wires, and specific lower metal layer1 extension is required when a nearby lower metal shape has an overlap with an upper metal shape but there is no via in the overlap region. The distance is measured between the metal overlap regions. The direction in which the cross metal via group is formed is specified with respect to layer1. In this situation, the direction parameter is non-preferred (default).



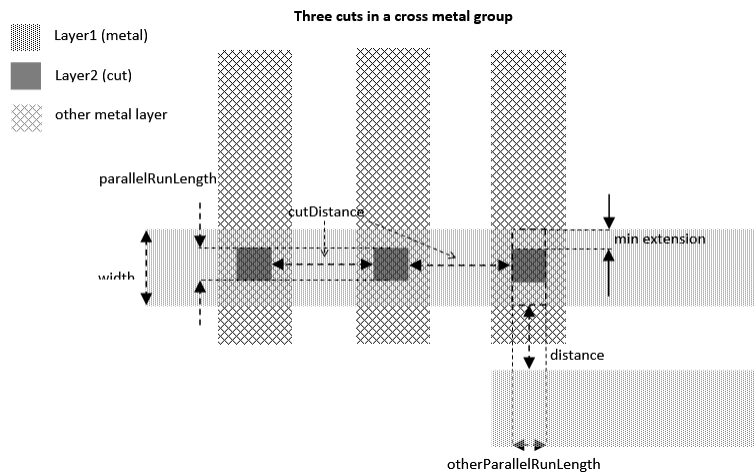
When determining lower and upper metal overlap regions, it is necessary to also consider minUncutEndToEndSpacing constraints for the metal layers. If the spacing between

metal line ends is less than the value of `minUncutEndToEndSpacing` constraint for this metal layer, it is necessary to merge the metal shapes into one wire before checking for conditions of the `minCrossMetalViaGroupExtension` constraint.



The uncut metal merging requirement for particular mask color can be optionally relaxed using `excludedUncutColor` and `otherExcludedUncutColor` parameters.

In the following figure, a cross metal via group is formed across lower metal wires, and specific upper metal layer1 extension is required when there is a nearby layer1 shape. In this case, there is no requirement for overlap with a lower metal shape, and distance to the nearby layer 1 shape is measured from a via cut extended to the edge of the layer1 shape covering it. The direction in which the cross metal via group is formed is specified with respect to layer1. In this situation, the direction parameter is `preferred`.



Constraint Category	Extension
Constraint Type	Complex

OA Rule	Custom Compiler Custom Layer Pair Constraint
Layer Symmetry	Asymmetric

Syntax:

```
minCrossMetalViaGroupExtension
  <string|LPP>metalLayer
  <string|LPP>cutLayer
  <float>extensionValue
  -numCuts int
  -cutDistance floatValue
  [-distance floatValue]
  [-parallelRunLength floatValue]
  [-width floatValue]
  [-direction preferred|nonpreferred]
  [-otherParallelRunLength floatValue]
  [-otherLayer layerName]
  [-excludedUncutColor colorName|any]
  [-otherExcludedUncutColor colorName|any]

  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
metalLayer	metalLayer can be a string (layer name) or LPP.
cutLayer	cutLayer can be a string (layer name) or LPP.
extensionValue	Minimum extension of layer1 shape in database units required in the non-preferred routing direction of the metal layer1 over a via cut on layer2.
-numCuts	The minimum number of cuts in a cross metal via group required for the rule to apply.
-cutDistance	Distance between two cuts in the direction specified by direction parameter must be less than or equal to this value for the cuts to be in the same cross metal group.
-distance	Distance between metal overlap regions (or between extended via cut and nearby layer1 shape) must be less than this value for the rule to apply. This parameter is optional if the -direction parameter is 1 (preferred direction). It is required when the -direction parameter is 2 (non-preferred direction). The distance is measured in non-preferred routing direction of layer1.
-parallelRunLength	Parallel run length between two cuts must be greater than this value for the cuts to be in the same cross metal group. Default: 0.
-width	Layer1 wire must have width less than or equal to this value for the rule to apply.

Argument	Description
-direction	The direction in which the cross metal via group is formed with respect to metal layer1. Valid values are 1 (preferred direction), 2 (non-preferred direction). Default : 2 (non-preferred).
-otherParallelRunLength	Parallel run length between metal overlap regions (or between extended via cut and nearby layer1 shape) must be greater than this value for the rule to apply. Default: 0.
-otherLayer	It indicates the upper metal layer for the cut layer2, if layer1 is the lower metal layer. It indicates the lower metal layer for the cut layer2, if layer1 is the upper metal layer. Only required when cannot be unambiguously determined from layer1 and layer2.
-excludedUncutColor	Uncut shapes on layer1 with this mask color are not required to be merged. A positive value means a specific mask color. A value of zero means any color.
-otherExcludedUncutColor	Uncut shapes on other metal layer with this mask color are not required to be merged. A positive value means a specific mask color. A value of zero means any color.

Note:

Report errors and stop importing when following scenarios occurs:

- When -direction parameter is any for this constraint, it is invalid value.

Example

```
minCrossMetalViaGroupExtension M1 VIA1 0.06
    -numCuts 3
    -cutDistance 0.12
    -distance 0.1
    -otherLayer M2
```

minCutClassClearance

The `minCutClassClearance` constraint specifies the spacing between cuts of different classes on different cut layers. The row key value in the constraint 2-D table specifies layer1 cut classes N. The column key value in the constraint 2-D table value specifies layer2 cut classes M. The size of table is 2N x 2M. The short edge spacing value is listed first and the long edge spacing value is listed second in each pair.

Constraint Category	Via Construction
Constraint Type	Complex
OA Rule	oacMinCutClassClearance
Layer Symmetry	Symmetric

Syntax:

```
minCutClassClearance
  <string|LPP>layer1
  <string|LPP>layer2
  <cutClass2DTableClearance>spacingValue
  [-connectivityType anyConnectivityType|
    sameNetConnectivityType|
    contiguousShapesConnectivityType|
    directShapesConnectivityType|
    sameViaConnectivityType]
  [-parallelRunLength float]
  [-exceptSameNetConnectivity true|false]
  [-exceptConnectivityType
    anyConnectivityType|
    sameNetConnectivityType|
    contiguousShapesConnectivityType|
    directShapesConnectivityType|
    sameViaConnectivityType]
  [-exactAlignedSpacing <float>spacingValue (
    <float>widthSpacing1
    <float>widthSpacing2) ]
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer1	layer1 can be defined as a string or LPP.
layer2	layer2 can be defined as a string or LPP.
spacingValue	A specialized two dimensional table that is 2Nx2N in size where N is the number of unique cut classes. The table holds both short and long edge spacing values. To know more about how spacing value is represented, see minCutClassClearance spacingValue Representation.
-connectivityType	anyConnectivityType: Use this option to designate that the constraint applies to any type of connectivity (default). sameNetConnectivityType: Use this option to designate that the constraint only applies to cuts on the same net. contiguousShapesConnectivityType: Use this option to designate that the constraint only applies to cuts on the same metal. directShapesConnectivityType: Use this option to designate that the constraint only applies when the two cuts are part of the same metal shape on the upper or lower metal and the metal shape covers the region between the two cuts. sameViaConnectivityType: This is similar to directShapesConnectivityType except that both lower and upper metal shapes must cover the cuts and the region between the cuts.

Argument	Description
-parallelRunLength	Specifies whether the cut edges in database units (dbu) must be parallel: -1: Cut edges must not be parallel. 0: Cuts must have ≥ 0 parallel run length. 1: Cuts must have > 0 parallel run length. - All other values: Minimum parallel run length between two end-of-line edges must be greater than or equal to the value for the constraint to apply. If the value is $> +1$ dbu, then the value is the threshold ($>$) parallel run required for the constraint to apply. The default is 0. $1 \text{ dbu} = 1 \text{ user unit} / \text{DBUperUU}$ (Database Units per User Unit). A user unit is usually 1um. For example, if dbuPerUU is 2000, then 1 dbu is calculated as $1\text{um}/2000 = 0.0005$.
-cutClassProfile	Optionally specifies an extension distance applied to cut shapes of certain cut classes before measuring cut spacing. The table layout is identical to the constraint value table. This parameter is only valid if there is no parallel overlap between cut shapes. The cutClassProfile is represented as: <code>(<float>shortEdgeExtension <float>longEdgeExtension*)</code>
-exceptSameNetConnectivity	If set to true, and the connectivityType is any, then the constraint applies only when the two shapes are on different net.
-exceptConnectivityType	If neither this nor the -connectivityType parameter is set, the constraint does not apply if the cuts are on the same net, equivalent to setting this parameter to sameNet. If it was set to directShapes, the constraint would not apply to cuts on the same shape. This parameter is mutually exclusive with the -connectivityType parameter. This parameter is represented same as the above -connectivityType parameter.
-exactAlignedSpacing	If this parameter is present, the first value in each oaDualIntValue is the width of a square cut class. The second value is the spacing (or clearance) in DBU required between exactly aligned horizontal or vertical cuts of the cut class.

minCutClassClearance spacingValue Representation

The spacingValue is represented as:

`(<float>cutClassKey <float>cutClassData)`

Where cutClassKey equals:

```
(shortEdge
  (cutEdge
    (squareCutEdge
      (<float>cutWidth <string>edgeFlag (-shortEdge|-longEdge))) |
    rectangleCutEdge
      ((<float>cutWidth <float>cutLength)
```

```

        <string>edgeFlag (-shortEdge|-longEdge)) )
longEdge
(cutEdge
  (squareCutEdge
    (<float>cutWidth <string>edgeFlag (-shortEdge|-longEdge)) ) |
  rectangleCutEdge
    ((<float>cutWidth <float>cutLength)
      <string>edgeFlag (-shortEdge|-longEdge)) )
)

```

And cutClassData equals:

```

<float>spacing|
spacingWithExtension(<float>spacing <float>extension)|
spacingWithMeasureType(<float>spacing <string>measureType(
    -edgeToEdge|
    -centerToCenter|
    -centerAndEdge|
    -centerAndEdgeNoPRL))|
spacingWithMeasureTypeAndExtension(<float>spacing <float>measureTypeAndExtension)

```

The measureType parameter should be used as specified by the enum entry for each cutclass combination:

- centerAndEdge: The maximum of all applicable values should be measured center-to-center. The minimum of all applicable values should be measured edge-to-edge.
- centerAndEdgeNoPRL: The spacing between two cuts should be the greater of the following two values: the maximum of all applicable values, measured center-to-center; and the minimum of all applicable values, measured edge-to-edge.

Note:

Since cutClassCenterToCenter and cutClassMeasureType parameters are represented by oaSpacingMeasureType enumeration, and cutClassMeasureType is the superset:

- When importing, always create both cutClassMeasureType and cutClassCenterToCenter parameters for backward compatibility. But only edgeToEdge and centerToCenter values go into the cutClassCenterToCenter table.
- When exporting, use the cutClassMeasureType table if it exists. If it does not exist, then use cutClassCenterToCenter table if it exists.

The warning message is displayed in the following situations:

- `-connectivityType` and `-exceptConnectivityType` are set to same value.
- `-exceptSameNetConnectivity` is set to true and `-connectivityType` is set to `sameNetConnectivityType`, `contiguousShapesConnectivityType`, `sameViaConnectivityType` or `directShapesConnectivityType` (only `anyConnectivityType` is allowed).

To define spacing center based between square cuts short edge:

```
(minCutClassClearance V0 V1
(
  ((0.032 -shortEdge) (0.032 -shortEdge) (0.107 -centerToCenter)
  ...
)
)
```

To define spacing edge based between rectangle cuts short edge:

```
(minCutClassClearance V0 V1
(
  ...
  (((0.032 0.064) -shortEdge) ((0.032 0.064) -shortEdge)) 0.112
  ...
)
)
```

To define spacing edge based with extension value between square cut short edge and rectangle cut short edge:

```
(minCutClassClearance V0 V1
(
  ...
  ((0.032 -shortEdge) ((0.032 0.064) -shortEdge)) (0.088 0.06)
  ...
)
)
```

To define spacing with `MeasureType=centerAndEdgeNoPRL` and extension value between rectangle cut long edge and square cut short edge:

```
(minCutClassClearance V0 V1
(
  ...
  (((0.032 0.064) -longEdge) (0.032 -shortEdge))
  (0.068 -centerAndEdgeNoPRL 0.06)
  ...
)
)
```

To define minCutClassClearance for layer A4 with cutClass (0.04 0.04) (0.04 0.08) (0.08 0.08) and layer W0 with cutClass (0.072 0.072)

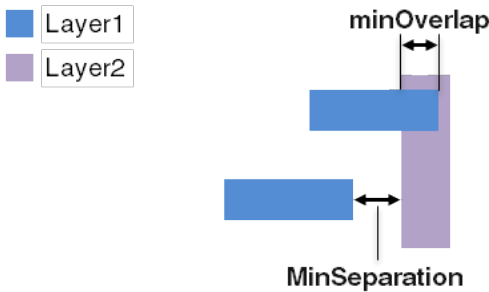
```
(minCutClassClearance "A4" "W0"
(
```

```
((0.04 -shortEdge) (0.072 -shortEdge)) (0.12 -centerToCenter)
((0.04 -shortEdge) (0.072 -longEdge)) (0.12 -centerToCenter)
((0.04 -longEdge) (0.072 -shortEdge)) (0.12 -centerToCenter)
((0.04 -longEdge) (0.072 -longEdge)) (0.12 -centerToCenter)
(((0.04 0.08) -shortEdge) (0.072 -shortEdge)) (0.14 -centerToCenter)
(((0.04 0.08) -shortEdge) (0.072 -longEdge)) (0.14 -centerToCenter)
(((0.04 0.08) -longEdge) (0.072 -shortEdge)) (0.14 -centerToCenter)
(((0.04 0.08) -longEdge) (0.072 -longEdge)) (0.14 -centerToCenter)
((0.08 -shortEdge) (0.072 -shortEdge)) (0.14 -centerToCenter)
((0.08 -shortEdge) (0.072 -longEdge)) (0.14 -centerToCenter)
((0.08 -longEdge) (0.072 -shortEdge)) (0.14 -centerToCenter)
((0.08 -longEdge) (0.072 -longEdge)) (0.14 -centerToCenter)
)
```

minCutMetalOverlap

This layer constraint specifies the following requirements for the erase layer operation:

- The minimum amount that the eraser and erasee shapes must overlap
- The minimum spacing between non-overlapping shapes



Constraint Category	Spacing
Constraint Type	Complex
OA Rule	Custom Compiler custom layer pair constraint
Layer Symmetry	Asymmetric

Syntax:

```
minCutMetalOverlap
<string|LPP>layer1
<string|LPP>layer2
<float>minOverlap
[-minSeparation <float>]
[-color <string>]
[-isHard true|false]
[-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer1	Layer of the erasee shape. Layer1 can be defined as a string or a LPP.
layer2	Layer of the eraser shape. Layer2 can be defined as a string or a LPP.
minOverlap	Minimum amount that the shapes must overlap for the erase layer operation to occur.
-minSeparation	Minimum distance between the non-overlapping shapes on layer2.
-color	The color mask (color name) for which this constraint applies. The color name must be defined in the Controls Section . If you specify 0, the constraint applies to all masks on this layer.

The following example specifies that erase layer can occur for overlapping shapes on layers L1 and L2. The overlapping shapes must overlap by 0.25 units, and the non-overlapping shapes must be 0.15 units apart. This constraint applies to all red masks.

```
minCutMetalOverlap L1 L2 0.25
                  -minSeparation 0.15
                  -color red
```

minCutRoutingClearance

This layer pair constraint defines the clearance between specific cut classes, poly, and metal shapes. To support this, a new asymmetric constraint is defined with a cut class parameter. In addition, for rectangular cut classes, the clearance might only need to be enforced between the short end of the cut and metal.

Note:

Both AutoVia and Router support poly in minCutRoutingClearance constraint.

This clearance constraint is always ignored for shapes on the same net in existing processes. In other cases, a clearance between a cut shape and the concave corner of the wire which contains the cut must be enforced.

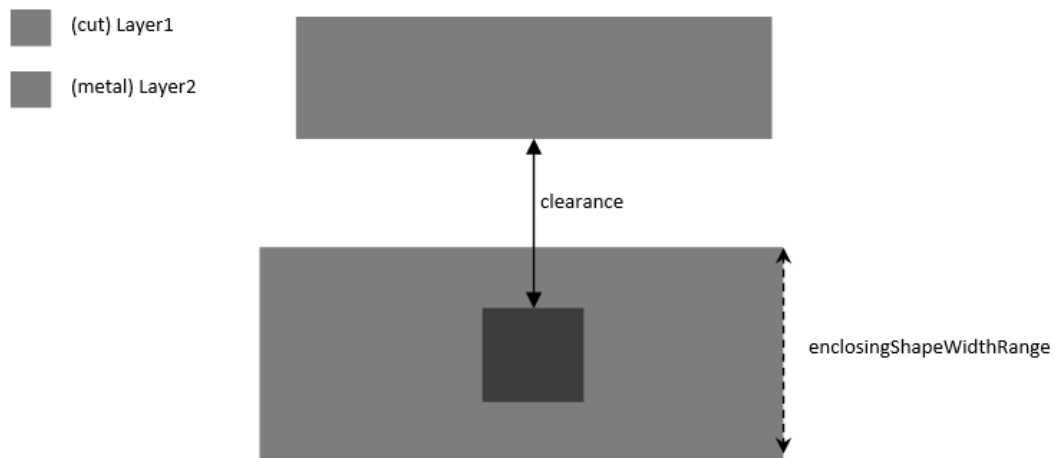


Figure 17 -otherLayerMaxExtension Rule

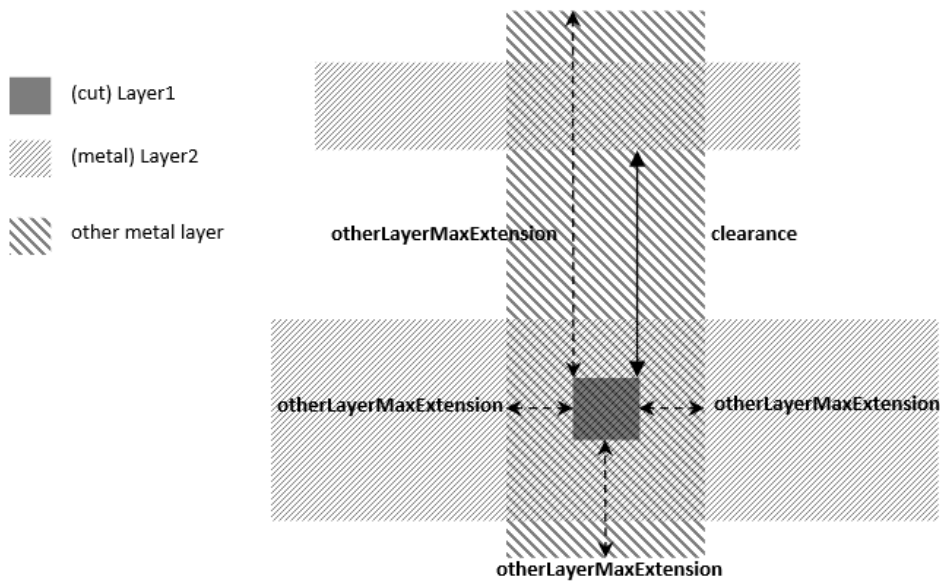
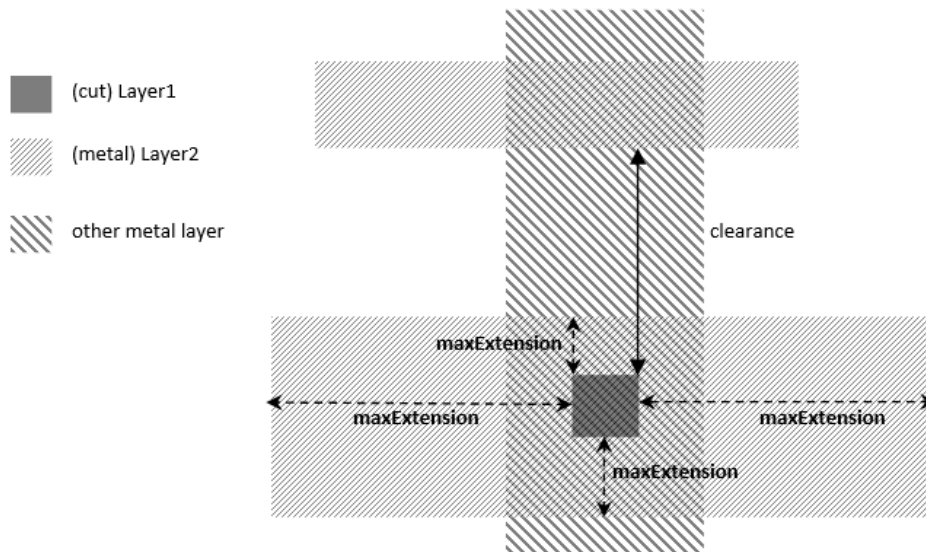
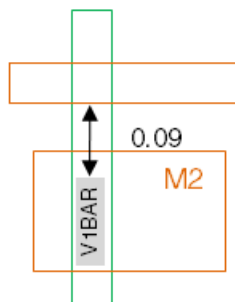


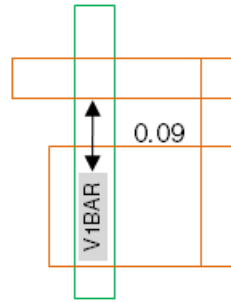
Figure 18 -maxExtension Rule



Example of minCutRoutingClearance Layer1=V1, Layer2=M2, Value=0.1, cutClass=V1BAR, shortEdgeOnly=true

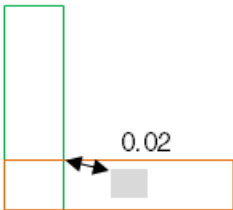


- a) Violation, the spacing between the short-end edge of a VXBAR to an edge of the above metal wire is 0.09 (<0.1)



- b) OK, same-net is exempted for the cut to metal rule

Example of minCutRoutingClearance Layer1=V1, Layer2=M2, Value=0.03, cutClass=VX, toConcaveCorner=true



a) Violation, the cut needs to be 0.03 away from concave corner

Constraint Category	Clearance
Constraint Type	Complex
OA Rule	oacMinCutRoutingClearance
Layer Symmetry	Asymmetric

Syntax:

```
minCutRoutingClearance
<string|LPP>cutLayer
<string|LPP>metalLayer
[-cutClass <cutClassArray | cutClassNameList>]
[-shortEdgeOnly true|false]
[-exceptConnectivityType
  anyConnectivityType|
  sameNetConectivityType|
  contiguousShapesConnectivityType|
  sameViaConnectivityType|
  directShapesConnectivityType]
[-toConcaveCorner true|false]
[-spacingDirection any|horizontal|vertical]
[-parallelRunLength floatValue]
  [-parallelRunLengthRange rangeData]
[-otherLayerMaxExtension floatValue]
[-maxExtension floatValue]
[-voltageRange frangeValue]
[-overLayer layerName]
[-enclosingShapeWidthRange rangeData]
  [-enclosingShapeDirection any | horizontal | vertical ]
[-isHard true|false]
[-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
cutLayer	cutLayer can be defined as a string or LPP.

Argument	Description
metalLayer	metalLayer can be defined as a string or LPP.
-cutClass	List of paired float values or a list of cutClass names. See cutClass and cutClassList . The constraint applies only in cases where the cuts comply with these specifications.
-shortEdgeOnly	This parameter applies only if a rectangular cut class was specified in the cutClass parameter. If this parameter is specified and is set to "true", the clearance value applies only between the metal shape and the short edge of the rectangular cut shapes. The default value is false.
-exceptConnectivityType	• anyConnectivityType: Use this option to designate that the constraint applies to any type of connectivity. • sameNetConnectivityType: Use this option to designate that the constraint only applies to cuts on the same net. • contiguousShapesConnectivityType: Use this option to designate that the constraint only applies to cuts on the same metal (default). • directShapesConnectivityType: Use this option to designate that the constraint only applies when the two cuts are part of the same metal shape on the upper or lower metal and the metal shape covers the region between the two cuts. • sameViaConnectivityType: This is similar to directShapesConnectivityType except that both lower and upper metal shapes must cover the cuts and the region between the cuts.
-toConcaveCorner	If this parameter is true, the spacing between the cut shape and a concave (inside) corner of the metal shape in which the cut shape is placed must be greater or equal to than the spacing specified in the constraint value. The default value is false.
-spacingDirection	Optional parameter defining the direction the clearance applies. The value is any, horizontal or vertical. The default value is any.
-parallelRunLength	If given, parallel run length threshold between the cut and the routing shape must be greater than or equal to this value for the constraint to apply. If the <code>oaSpacingDirection</code> parameter is given and its value is 1 (horizontal direction) or 2 (vertical direction), the default of the <code>parallelRunLength</code> parameter is 0. This parameter cannot be specified if the <code>parallelRunLengthRange</code> parameter is also given.
-parallelRunLengthRange	If given, parallel run length between the cut and the routing shape must lie in this range in database units for the constraint to apply. If the <code>oaSpacingDirection</code> parameter is 1 (horizontal) or 2 (vertical), the default of the <code>parallelRunLengthRange</code> parameter is (≥ 0). This parameter cannot be specified if the <code>parallelRunLength</code> parameter is also given.
-otherLayerMaxExtension	If given, the rule does not apply if the extension of the other metal layer (metal above or metal below the cut layer) over the cut is greater than this value in database units (in all four directions). See Figure 17 .

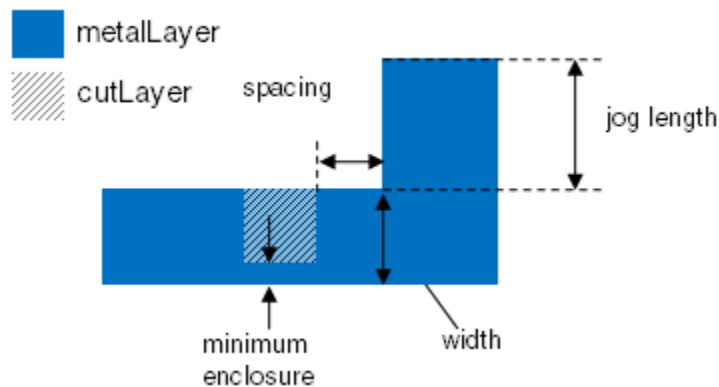
Argument	Description
-maxExtension	If given, the rule does not apply if the extension of the metal layer (layer2) over the cut is greater than this value in database units (in all four directions). See Figure 18 .
-voltageRange	If given, the rule only applies if the difference between voltages of the two shapes under consideration lies in this range in volts. To specify the ranges, see Range Data Syntax . See Voltage Dependent Design Rules .
-overLayer	If given, the rule only applies if both shapes interact with the same third shape in this specified layer.
-enclosingShapeWidthRange	If given, the rule only applies if the cut shape is enclosed by a metal layer2 shape with width in this range.
-enclosingShapeDirection	If given, the constraint only applies if the cut shape is enclosed by a metal layer2 wire oriented in this direction. Valid values are 0 (any), 1 (horizontal direction), 2 (vertical direction).

For example:

```
minCutRoutingClearance v1 m2 0.06 -cutClass 0.06 0.08
                                   -shortEdgeOnly true
                                   -toConcaveCorner true
                                   -spacingDirection horizontal
                                   -parallelRunLength 0.75
                                   -parallelRunLengthRange 2
                                   -otherLayerMaxExtension 0.5
                                   -maxExtension 0.6
                                   -voltageRange (1.5 2)
                                   -overLayer m3
                                   -enclosingShapeWidthRange (2.5 3.5)
                                   -enclosingShapeDirection vertical
```

minCutToMetalCornerLength

This constraint defines that a cut shape that is aligned with a metal shape must be either a minimum distance from a concave corner of a connecting metal shape or that the cut must have a minimum enclosure on the side opposite the aligned edge. The rule only applies if the jog has a certain height and the distance from the concave corner to the opposite edge is greater than or equal to a certain value.



Constraint Category	Spacing
Constraint Type	Complex
OA Rule	Custom Compiler custom layer pair constraint
Layer Symmetry	Asymmetric

Syntax:

```
minCutToMetalCornerLength
  <string|LPP>metalLayer
  <string|LPP>cutLayer
  <float>spacingValue
  -width float
  -minJogLength float
  -minEnclosure float
  [-cutClass <cutClassArray | cutClassNameList>]
  [-cutType <string>]
  [-includeArray true | false]
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
metalLayer	metalLayer can be defined as a string or layer-purpose pair (LPP).
cutLayer	cutLayer can be defined as a string or layer-purpose pair (LPP).
spacingValue	Minimum spacing of cut shape on layer1 to a concave corner defined by a shape on layer2.
-width	Distance from concave corner to opposite edge must be greater than or equal to this value for the constraint to apply.
-minJogLength	The length of the edge from the concave corner to end of line not containing the cut must be greater than or equal to this value for the constraint to apply.

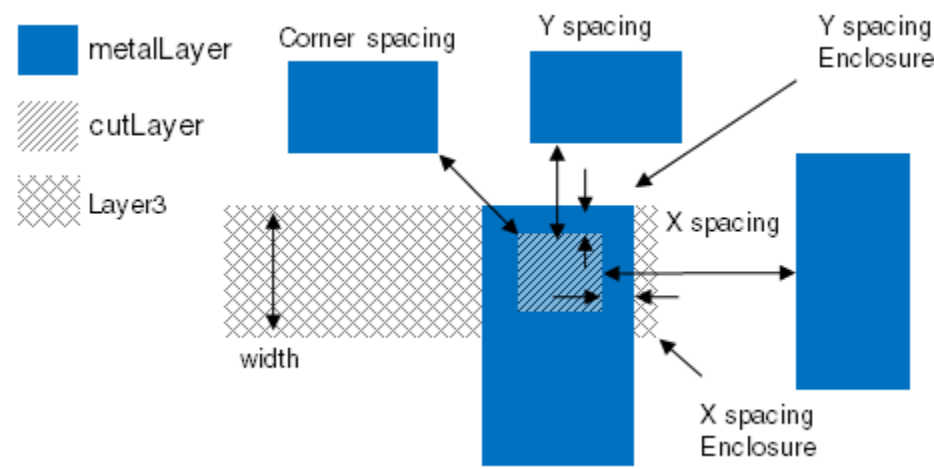
Argument	Description
-minEnclosure	The rule does not apply if the enclosure is greater than or equal to this value.
-cutClass	List of paired float values or a list of cutClass names. See cutClass and cutClassList . The constraint applies only in cases where the cuts comply with these specifications.
-cutType	Type of cut: - aligned: Self-aligned - unaligned: Not self-aligned - rectangular: Rectangular-shaped
-includeArray	Indicator of whether the constraint applies to via arrays. Default: false

For example,

```
minCutToMetalCornerLength m1 v1 0.06
                           -width 0.08
                           -minJogLength 0.06
                           -minEnclosure 0.04
                           -cutClass 0.06 0.08
                           -cutType unaligned
```

minCutToMetalSpacing

This constraint defines the spacings from a cut to a metal shape depending upon whether the cut is enclosed enough by that metal layer and optionally the other connecting metal layer. Spacings apply only to different nets, can be directional, and only apply if the upper metal shape is of a minimum width.



Constraint Category	Spacing
---------------------	---------

Constraint Type	Complex
OA Rule	Custom Compiler custom layer pair constraint
Layer Symmetry	Asymmetric

Syntax:

```
minCutToMetalSpacing
    <string|LPP>cutLayer
    <string|LPP>metalLayer
    <float>spacingValue
    [-lowerEnclosure float]
    [-width float]
    [-cutClass <cutClassArray | cutClassNameList>]
    [-isHard true|false]
    [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
cutLayer	cutLayer can be defined as a string or LPP.
metalLayer	metalLayer can be defined as a string or LPP.
spacingValue	Array of objects. If one value is given, it represents the X, Y and corner spacing in microns. If two values are given, the first value is the X spacing and the second value is the Y spacing both in microns and both using manhattan measure. If three values are given, the values are the X, Y, and corner spacing values in microns respectively.
-lowerEnclosure	Lower enclosures must be less than or equal to this value for the constraint to apply.
-width	If given, upper shape must be greater than or equal to this value for the constraint to apply.
-cutClass	List of paired float values or a list of cutClass names. See cutClass and cutClassList . The constraint applies only in cases where the cuts comply with these specifications.

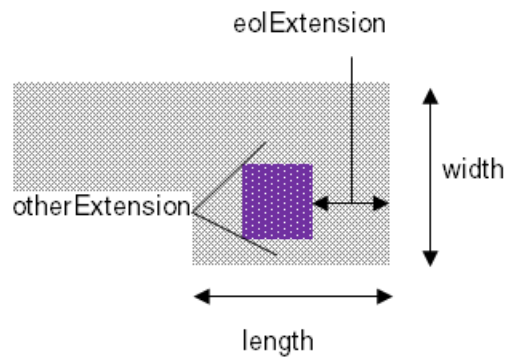
For example,

```
minCutToMetalSpacing via1 metall (0.06 0.06 0.10)
    -lowerEnclosure 0.04
    -upperEnclosure 0.06
    -width 0.10
    -cutClass 0.06 0.08
```

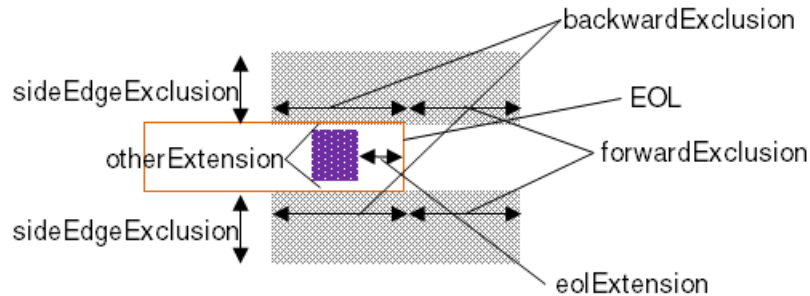
minDualEndOfLineExtension

Some designs must specify an end of line enclosure rule that only applies if no shapes on other nets lie in the rectangular regions around the sides and end of the line. This two-layer constraint defines how far a shape on the first layer should extend beyond a shape on the second layer. The second layer is expected to be a cut layer.

Example of minDualEndOfLineExtension

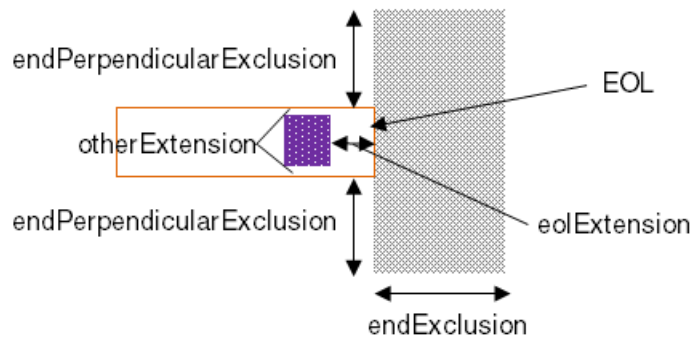


Definition of sideExclusionArea, consisting of sideEdgeExclusion, forwardExclusion, and backwardExclusion values below.

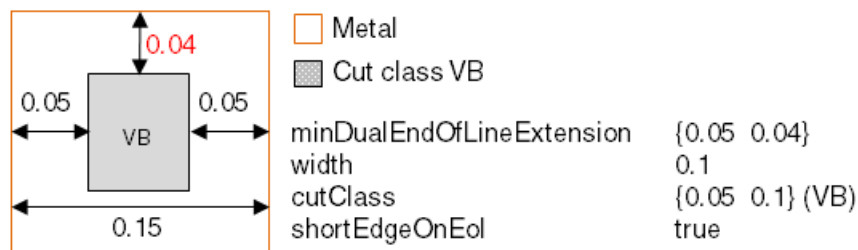


They should have no neighbor wires on the 2 side edge exclusion regions (dotted area) in order to apply the given enclosure.

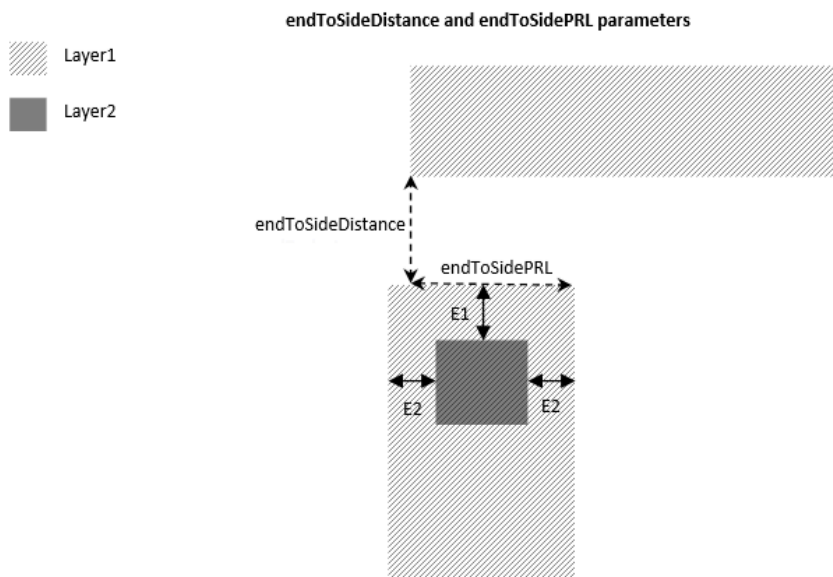
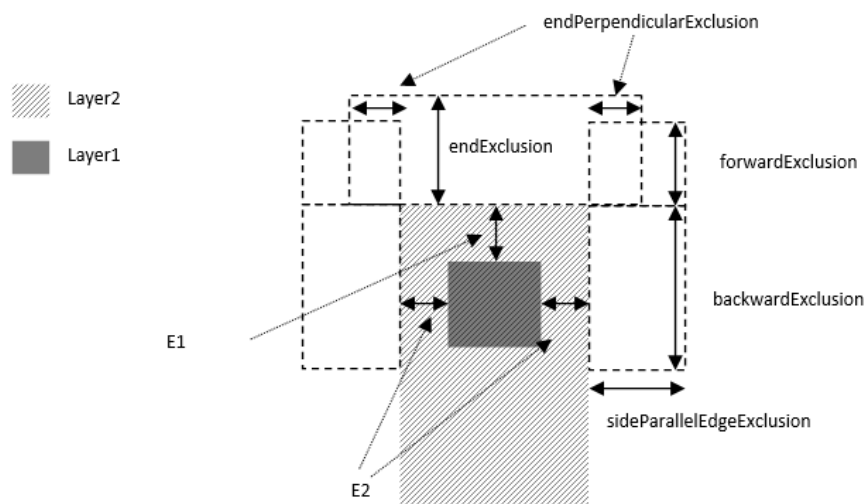
Definition of endPerpendicularExclusion and endExclusion.



They should have no neighbor wires in the exclusion region (dotted area) in order to apply the given enclosure.



- a) Violation, end/short cut edge has an overhang of $0.04 < \text{the required } 0.05$ overhang to an end-of-line edge.



Constraint Category	Extension
Constraint Type	Complex
OA Rule	oacMinDualEndOfLineExtension
Layer Symmetry	Asymmetric

Syntax:

```
minDualEndOfLineExtension
  <string|LPP>metalLayer
  <string|LPP>cutLayer
```

```
<firstFloat secondFloat>extensionValue
-width float
[-length float]
[-endOfLineOnly true|false]
[-endExclusionArea <dual>float1 float2]
[-sideExclusionArea (float1 float2 float3)]
[-shortEdgeOnEol true|false]
[-cutClass <cutClassArray | cutClassNameList>]
[-color colorName]
[-maxLength float]
[-widthRange rangeData]
[-preferredDirectionOnly true|false]
[-enclosingShapeDirection any|horizontal|vertical]
[-endToSideDistance float]
[-endToSidePRL float]
[-isHard true|false]
[-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
metalLayer	The layer which extends the layer2. layer1 can be defined as a string or LPP.
cutLayer	The layer which should be enclosed in layer1. layer2 can be defined as a string or LPP.
extensionValue	The first integer value is the extension in database units of layer1 beyond layer2 (E1). The second value is the extension that either adjacent side must extend beyond layer2 (E2).
-width	Specifies that the rule applies only if the width of the end-of-line is less than the given width.
-length	Specifies that the rule applies only if the length of the end-of-line is greater than or equal to the given length.
-endOfLineOnly	If it is true for a given constraint, it should be true for all minDualEndOfLineExtension constraints for that cutClass and layer pair. If only one constraint has the endOfLineOnly parameter, that constraint should be observed and the others should be ignored. If this parameter is true, then only minDualEndOfLineExtension constraints must be satisfied for all end of line edges. Any other enclosure type rules for that layer pair/cut class such as minDualExtension can be ignored.
-endExclusionArea	If specified, this rule applies only when there are no different-net neighbors in the rectangular area at the end of the line. The first value is the distance on either side of the line end and the second value is the endExclusion. This is an optional parameter.
-sideExclusionArea	Specifies an exclusion area at either side of the line. The area is defined by sideparallelEdgeExclusion, backwardExclusion and forwardExclusion parameters. The rule only applies if there is no different-net neighbor shape in that region. This is an optional parameter.

Argument	Description
-shortEdgeOnEol	It should only be present when a rectangular cut class parameter is also present. If this parameter is present and set to true, the short edges of the cut should be enclosed by the first value of the constraint's <code>eolExtension</code> value. The long sides of the cut should be enclosed by the 2 nd value in the pair <code>otherExtension</code> .
-cutClass	List of paired float values or a list of cutClass names. See cutClass and cutClassList . The constraint applies only in cases where the cuts comply with these specifications.
-color	Color of the metal layer for which the constraint applies. The <code>colorName</code> must be defined in the Controls Section , and the color mask value cannot be greater than the <code>numberOfColorMasks</code> value defined in the functions section. Default: 0
-maxLength	Maximum wire length. This argument applies only if <code>length</code> is specified, and <code>maxLength</code> must be <code><= length</code> .
-widthRange	Range of widths of the metal layer that is extending beyond the cut layer. To specify a range of values, see Range Data Syntax . Specify only <code>width</code> or <code>widthRange</code> , but not both.
-preferredDirectionOnly	Specifies whether the constraint applies only to metal shapes that are in the preferred direction. Default: false
-enclosingShapeDirection	If given, the constraint applies only to wires (enclosing shape) oriented in this direction. Valid values are 0 (any), 1 (horizontal direction), 2 (vertical direction). Default: any A non-default value can be specified only if <code>-preferredDirectionOnly</code> parameter value is false.
-endToSideDistance	If given, the constraint applies only if there is another wire side on the metal layer1 next to the end of line which is located at a distance less than this value in database units.
-endToSidePRL	This parameter can be given only if the <code>-endToSideDistance</code> parameter is also specified. If given, the constraint applies only if the parallel run length between the end of line and the neighboring metal layer1 wire side is greater than or equal to this value in database units. Default: 0

Note:

The error message is displayed in the following situations:

- When `-shortEdgeOnEol` is set to true and `-cutClass` is specified, `-cutClass` is specified as square cut class.
- When `-maxLength` and `-length` are specified, `-maxLength` is smaller than or equal to `-length`.

Report errors and stop importing when following scenarios occurs:

- If `-enclosingShapeDirection` is specified as horizontal or vertical and `-preferredDirectionOnly` is specified as true (mutually exclusive).
- If `-endToSidePRL` is specified, but `-endToSideDistance` is not specified.

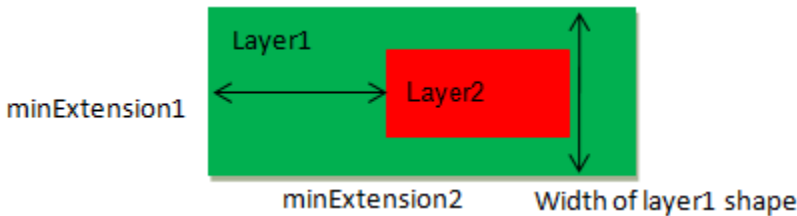
For example:

```
minDualEndOfLineExtension m1 v1 0.05 0.04 -width 0.1
                                -cutClass ((0.05 0.1) (0.6 0.8))
                                -shortEdgeOnEol true
                                -enclosingShapeDirection vertical
                                -endToSideDistance 0.03
```

minDualExtension

This layer pair constraint specifies the minimum extension a shape must have beyond a shape on another layer.

Extensions are measured from the outside edge of the shape on layer2 to the inside edge of the shape on layer1.



This constraint specifies two values: the first value applies to extensions in one direction (horizontal or vertical), the second applies to extensions in the other direction. Each value can apply to a different direction for each shape on a layer.

This constraint definition is not symmetric. That indicates that the minimum extension of layer1 past layer2 is not the same as the minimum extension of layer2 past layer1.

Note:

If you specify the `minDualExtension` constraint for a layer pair, you should not specify an `minExtension` constraint.

Constraint Category	Extension
Constraint Type	Complex
OA Rule	oacMinDualExtension
Layer Symmetry	Asymmetric

Syntax:

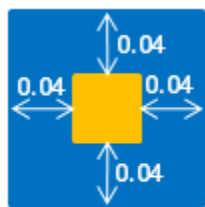
```
minDualExtension
  <string|LPP>layer1
  <string|LPP>layer2
  <dualFloat|dualfloat1DTable|
    dualfloatArrayTable>extensionValue
  [-coincidentAllowed true|false]
  [-length <float>]
  [-cutDistance <float>]
  [-noSharedEdge true|false]
  [-extensionType
    horizontalVertical|
    verticalHorizontal|
    twoSides|
    threeSides]
  [-cutClass <cutClassArray | cutClassNameList>]
  [-endSideOverhang true|false]
  [-numCuts int]
  [-redundantCutDistance float]
  [-hasParallelRunLength true|false]
  [-exceptCutMetalEdgeExtension true|false]
  [-offCenterline true|false]
  [-enclosingShapeDirection any|horizontal|vertical]
  [-otherLayer layerName]
  [-otherLayerDirection any|horizontal|vertical]
  [-selfAlignViaType any|selfAlign|SA|nonSelfAlign|NSA]
  [-voltageRange frangeValue]
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
  [-color colorName]
```

Argument	Description
layer1	The layer which extends the layer2. layer1 can be defined as a string or LPP
layer2	The layer which should be enclosed in layer1. layer2 can be defined as a string or LPP.
extensionValue	• dualFloat: Specifies the extension values of layer1 over layer2, used to define simple symmetrical/asymmetrical extension values. • dualfloat1DTable: List of a list of two values, where the first value is the width of the shape on layer 1 and the second value is the symmetrical extension value of layer1 over layer2, used to define width dependent symmetrical extension values. • dualfloatArrayTable: Specifies an array of dualfloat1DTable value pairs for a given width. The value pairs are used to define the asymmetrical/symmetrical extension values of layer1 over layer2, used to define width dependent symmetrical/asymmetrical extension rules.

Argument	Description
-coincidentAllowed	If specified as “true”, the shapes can either meet the minimum extension or their edges can be coincident else the edges must meet the minimum extension value specified and cannot be coincident.
-length	Specifies the total length of the enclosure required for the extension pair.
-cutDistance	Specifies a distance with that via cuts are considered.
-noSharedEdge	Specifies if vias on a shared edge that are within the cutDistance are counted as redundant. If set to “true”, then only those vias that are not along the same wire edge can be counted as being inside the cutDistance else via cuts along the same wire edge are counted.
-extensionType	- horizontalVertical: The first value of dual extension needs to be applied to horizontal, the second value of dual extension needs to be applied to vertical. - verticalHorizontal: The first value of dual extension needs to be applied to vertical, the second value of dual extension needs to be applied to horizontal. - twoSides: horizontalVertical or verticalHorizontal. (default) - threeSides: The three sides could be one value, another side is another value.
-cutClass	List of paired float values or a list of cutClass names. See cutClass and cutClassList . The constraint applies only in cases where the cuts comply with these specifications.
-endSideOverhang	If the value is false, then the pair of extensions can apply to either set of opposite sides. If the value is true, then the first value applies to the short edges and the second value to the long edges.
-numCuts	If the value is 2 and there are two or more cuts present within redundantCutDistance then the constraint does not apply.
-redundantCutDistance	The MinDualExtension constraint with this parameter has smaller enclosure requirements. It applies only if there is a redundant cut within cutWithin distance (with the same metal shape above and below) which has the required enclosures specified in a MinDualExtension rule without the redundant parameter.
-hasParallelRunLength	If this parameter is set to true, a common parallel run length > 0 exist on the opposite edges for all of the failing edges of a cut.
-exceptCutMetalEdgeExtension	If this parameter is true, and a via metal shape overlaps an edge of a wide wire, then the constraint would not apply to that edge.
-offCenterline	The constraint only applies to rectangular cuts that do not collide with the centerline of the wire. Must be used together with the cutClass parameter. The default is <i>false</i> .

Argument	Description
-enclosingShapeDirection	If given, the constraint applies only to wires oriented in this direction. Valid values are 0 (any), 1 (horizontal direction), and 2 (vertical direction). Default: any
-otherLayer	If given, the cut shape must interact with material on the given layer, otherwise the constraint does not apply.
-otherLayerDirection	If given along with <code>-otherLayer</code> , the interacting material must be on the specified direction, otherwise the constraint does not apply. Valid values are 0 (any), 1 (horizontal direction), and 2 (vertical direction). This parameter should only be defined when <code>-otherLayer</code> is also defined.
-selfAlignViaType	It could be one cut or both cuts, depending on whether <code>-SelfAlignViaType2</code> is also defined. Valid values are 0 (any), 1 (selfAligned), and 2 (nonSelfAligned). <code>-SelfAlignViaType</code> is determined by <code>-selfAlignedViaExtension</code> .
-voltageRange	If given, the rule only applies if the difference between voltages of the two shapes under consideration lies in this range in volts. To specify the ranges, see Range Data Syntax . See Voltage Dependent Design Rules .
-color	If given, this constraint only applies to enclosing layer1 shapes on this color mask (positive value). A value of zero means any color

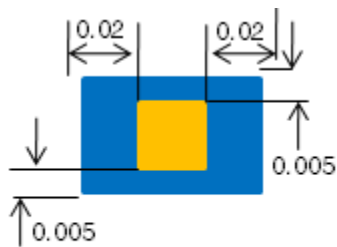
Rule	Description	Value (in microns)
M1.ENC.1	Minimum M1 extension over CA	0.04



Symmetrical extension
value of 0.04

minDualExtension M1 CA 0.04 0.04

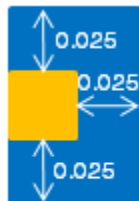
Rule	Description	Value (in microns)
M1.ENC.1.1	Minimum M1 extension over CA on any two sides, with the other two sides extension ≥ 0.005	0.02



Meets minDualExtension
values of 0.02, 0.005

```
minDualExtension M1 CA ((0.02 0.005) (0.04 0.04)) ((0.0 ((0.02 0.005) (0.04 0.04))))
```

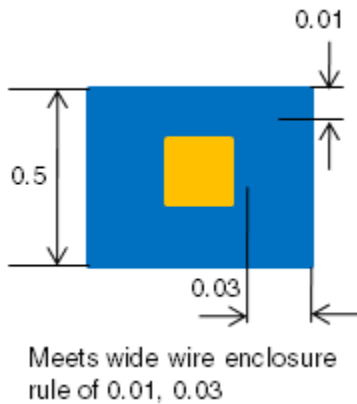
Rule	Description	Value (in microns)
M1.ENC.1.2	Minimum M1 extension over CA on any three sides, with the other side extension >= 0	0.025



Meets minDualExtension values of
0.025, 0.025, 0.025, 0.00

```
minDualExtension M1 CA 0.025 0.0 -extensionType threeSides
```

Rule	Description	Value (in microns)
M1.ENC.2	Minimum M1 extension over CA on any two sides, with the other two sides extension >= 0.01 for M1 width >= 0.5	0.03



```
minDualExtension M1 CA ((0.02 0.005) (0.04 0.04)) ((0.0((0.02 0.005) (0.04
0.04))) (0.5((0.01 0.03))))
```

For more information on defining 1-D/2-D tables, see [Understanding and Interpreting 1-D, 2-D, and 3-D Table Values](#) section.

Note:

If `cutClass` parameter is specified with `cutClassName` in ASCII file, technology importer looks-up the first `cutClass` constraint specified in the same hierarchical constraint group to convert to cut class size. If a `cutClass` constraint does not exist then an error message is displayed.

The error message is displayed in the following situations:

- The indexes of two constraint value tables are not aligned.
- When `-endSideOverhang` is set to true and `-cutClass` is specified, `-cutClass` is specified as square cutclass.
- `-otherLayerDirection` is defined if `-otherLayer` is not defined.
- `colorName` should be specified in controls colors section and the color mask value cannot bigger than the value specified in the layerRules functions `numberOfColorMasks` value.

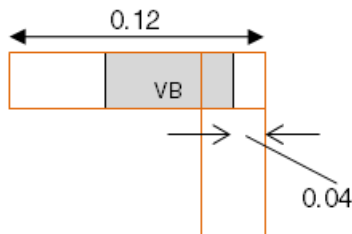
minEndOfLineEdgeExtension

In some advanced processes, cut shapes which lie near the end of a line have an additional extension requirement for the cut edge at the end of the line. The extension might be expressed as a simple minimum extension, although optionally a smaller `exactExtension` might also be allowed. The rule might also only apply to the extension of the end of line edge over the long edge of a non-square cut class. If the cut layer in question has cut classes, then a cut class must be specified in the constraint, and must be a non-square cut class if the long edge parameter is given. The rule only applies if the line

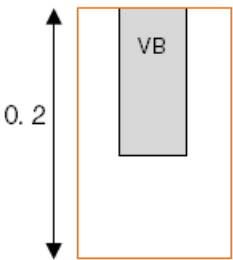
is less than a given `eolWidth`. However, the rule might not apply to edges with length less than `eolWidth` if those edges are not equal to the wire width (are actually side edges).

`minEndOfLineEdgeExtension` 0.04
`Width` 0.15
`equalRectWidth` true
`longEdgeOnly` true
`cutClass` {0.06 0.1} (VB)

□ Metal
■ Cut class VA



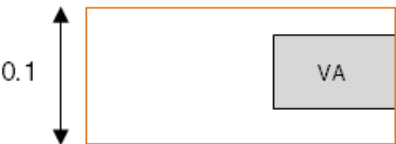
a) Violation, 0.04 overhang must be on the side/long edge when the cut is on an end-of-line edge.



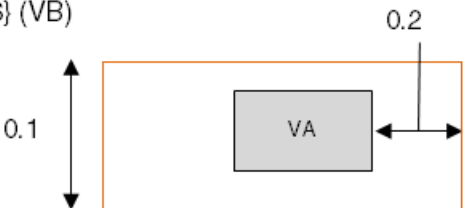
b) OK, the side/long edge of the cut is not on an end-of-line edge.

`minEndOfLineEdgeExtension` 0.05
`width` 0.15
`equalRectWidth` true
`longEdgeOnly` true
`exactExtension` 0.0
`cutClass` {0.06 0.6} (VB)

□ Metal
■ Cut class VA

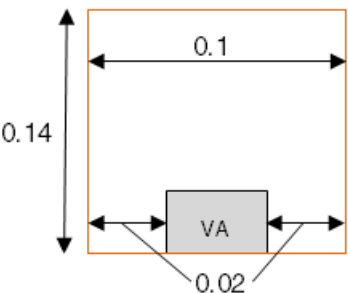


a) OK, zero overhang is allowed



b) Violation overhang should be either zero or ≥ 0.05

c) OK, the vertical edges are not EOL edges with length equal to wire width, and the rule does not apply on them. The bottom edge has zero overhang, which is allowed. It is a violation if `equalRectWidth` is not specified.



Constraint Category	Extension
Constraint Type	Complex

OA Rule	oacMinEndOfLineEdgeExtension
Layer Symmetry	Asymmetric

Syntax:

```
minEndOfLineEdgeExtension
  <string|LPP>metalLayer
  <string|LPP>cutLayer
  [-cutClass <cutClassArray | cutClassNameList>]
  -width float
  [-exactExtension float]
  [-longEdgeOnly true|false]
  [-equalRectWidth true|false]
  [-minLength float]
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
metalLayer	metalLayer can be defined as a string or LPP. metalLayer must enclose cutLayer.
cutLayer	cutLayer can be defined as a string or LPP. cutLayer must be enclosed by metalLayer.
-cutClass	List of paired float values or a list of cutClass names. See cutClass and cutClassList . The constraint applies only in cases where the cuts comply with these specifications.
-width	The end of line width. The rule does not apply if the shape width is >= than the width.
-exactExtension	If specified, the overhang might exactly match this value, instead of being >= the constraint's extension value.
-longEdgeOnly	If specified, the rule only applies to long edge of a rectangular (non square) cut at the end of the line. This parameter is mutually exclusive with the exactExtension parameter.
-equalRectWidth	If specified, the rule does not apply to edges with length less than eolWidth if those edges are not equal to the wire width, and are actually side edges.
-minLength	Minimum length of the edge adjacent to the line end.

For example:

```
minEndOfLineEdgeExtension m1 v1 0.15 -cutClass ((0.06 0.1) (0.06 0.1))
                                     -width 0.15
                                     -equalRectWidth true
                                     -longEdgeOnly true
```


minEndOfLineExtension

This layer pair constraint specifies the minimum extension of wire required on a metal layer with respect to a cut layer. This constraint is applicable when the wire has adjacent, parallel edges on three sides within the specified regions.

If this constraint is not specified, or if it is specified but there are no parallel edges on three sides within the specified regions, a wire is extended beyond a cut shape dependent only on the [minExtension](#) constraint.

Constraint Category	Extension
Constraint Type	Complex
OA Rule	oacMinEndOfLineExtension
Layer Symmetry	Asymmetric

Syntax:

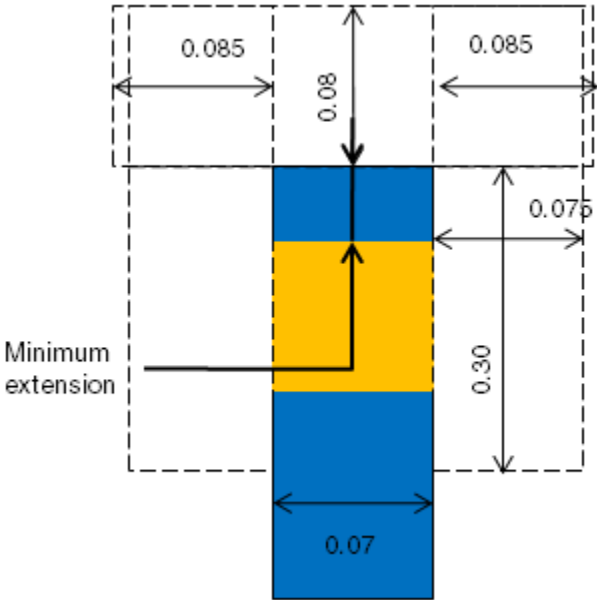
```
minEndOfLineExtension
  <string|LPP>metalLayer
  <string|LPP>cutLayer
  <float>minExtension
  <-width float>
  <-distance float>
  <-endOfLineSpacing float>
  <-parallelEdgeSpace float>
  <-parallelEdgeWithin float>
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
metalLayer	metalLayer can be defined as a string or LPP. metalLayer encloses the cutLater.
cutLater	cutLater can be defined as a string or LPP. cutLater should be enclosed by metalLayer.
minExtension	Specifies the minimum end-of-line extension that must be added when adjacent shapes satisfy the criteria defined by the following parameters: • -width • -distance • -endOfLineSpacing • -parallelEdgeSpace • -parallelEdgeWithin
-width	Specifies the width of line end. The constraint applies if the wire width is less than this value.

Argument	Description
-distance	Specifies the distance from line-end edge in the opposite direction of line-end. This together with the endOfLineSpace parameter defines the lateral verification region. This constraint applies, if there is a parallel edge in the lateral verification region in addition to two parallel edges in the region defined by the parallelEdgeSpace and parallelEdgeWidth parameters described next.
-endOfLineSpace	Specifies the spacing between line-end and shapes in the line-end direction.
-parallelEdgeSpace	Specifies the spacing between two parallel lines. Parallel wires must lie within this distance on both sides of the end-of-line in order for this constraint to apply.
-parallelEdgeWithin	Specifies the distance of two line-end on parallel lines. If there are parallel wires within this distance on both sides of the wire, the constraint applies.

In this example, the METAL1 extension over CONT depends on the proximity of other METAL1 near the end of line:

Rule	Description	Value (in microns)
M1.EX.6	End of line extension when width ≤ 0.07 , parallel run length is ≥ 0.3 , and parallel spacing ≤ 0.075 , and end of line spacing ≤ 0.08	≥ 0.200

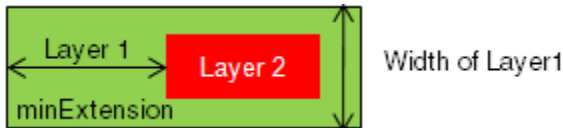


```
minEndOfLineExtension METAL1 CONT 0.200 -width 0.07 -distance 0.085
-endOfLineSpace 0.080 -parallelEdgeSpace 0.075 -parallelEdgeWithin 0.30
```

minExtension

This layer pair constraint specifies the minimum distance a shape on one layer must extend past a shape on a second layer.

Extensions are specified from the outside edge of the shape on layer2 to the inside edge of the shape on layer1



This constraint definition is not symmetric. This indicates that the minimum extension of layer1 past layer2 is not the same as the minimum extension of layer2 past layer1.

Note:

When the minExtension constraint is specified for a layer pair, the [minDualExtension](#) constraint should not be specified.

Constraint Category	Extension
Constraint Type	Complex
OA Rule	oacMinExtension
Layer Symmetry	Asymmetric

Syntax:

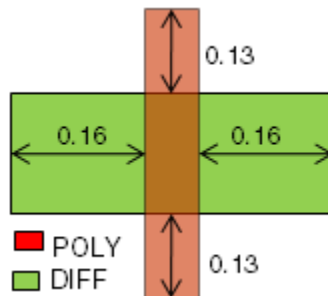
```
minExtension
<string|LPP>layer1
<string|LPP>layer2>
<float|float1DTable>minExtension
[-distanceMeasureType euclidian|manhattan]
[-coincidentAllowed true|false]
[-spacingDirection any|horizontal|vertical]
[-isHard true|false]
[-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer1	The layer which extends the layer2. layer1 can be defined as a string or LPP.
layer2	The layer which should be enclosed in layer1. layer2 can be defined as a string or LPP.
minExtension	float: Specifies the minimum extension. float1DTable: Specifies the width of the shape on the second layer and the minimum extension of that second shape past the first.

Argument	Description
-distanceMeasureType	Specify if spacing values are in Euclidian or Manhattan value. Default value is "euclidean".
-coincidentAllowed	If specified as "true", the shapes can either meet the minimum extension or their edges can be coincident else the edges must meet the minimum extension value specified and cannot be coincident.
-spacingDirection	Direction for which this constraint applies. Default: any

In this example, the minimum extension of POLY over DIFF (gate endcap) is 0.13 and the minimum extension of DIFF over POLY is 0.16

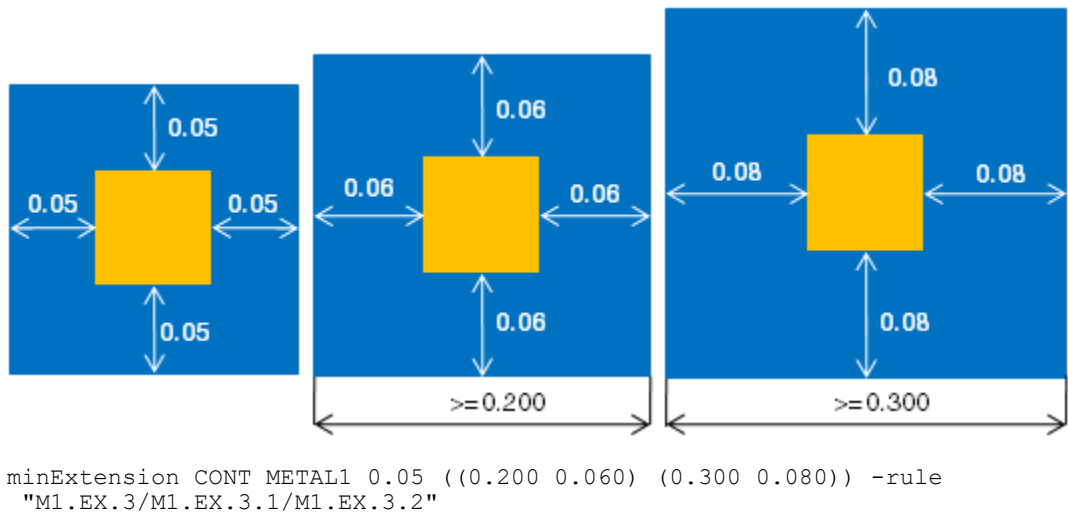
Rule	Description	Value (in microns)
PO.EX.1	POLY extension over DIFF	>= 0.13
DIFF.EX.1	DIFF extension over POLY	>= 0.16



```
minExtension POLY DIFF 0.13 -rule "PO.EX.1"
minExtension DIFF POLY 0.16 -rule "DIFF.EX.1"
```

In this example, the minimum METAL1 extension over the CONT layer depends on the width of the METAL1 shape.

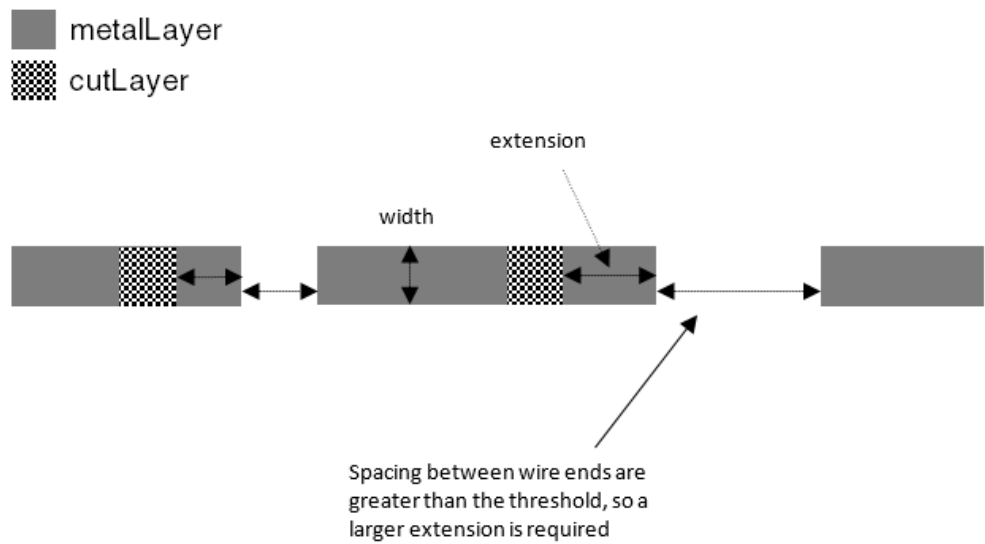
Rule	Description	Value (in microns)
M1.EX.3	M1 extension over CONT	>= 0.05
M1.EX.3.1	M1 extension over CONT for METAL1 width >=0.200	>= 0.06
M1.EX.3.2	M1 extension over CONT for METAL1 width >=0.300	>= 0.08



minExtensionAlignedProximity

This layer pair constraint specifies the extension requirements when the spacing between end-to-end wires are greater than a threshold.

This constraint definition is not symmetric, because it requires a minimum extension of layer1 but not layer 2.



Constraint Category	Extension
Constraint Type	Complex

OA Rule	Custom Compiler custom layer pair constraint
Layer Symmetry	Asymmetric

Syntax:

```
minExtensionAlignedProximity
  <string|LPP>metalLayer
  <string|LPP>cutLayer
  <float>minExtension
  [-width float|-widthRange rangeData]
  [-minLength float]
  [-color colorName|any|same|diff]
  -edgeRange rangeData
  -cutClass <cutClassArray | cutClassNameList>
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
metalLayer	metalLayer can be defined as a string or LPP.
cutLayer	cutLayer can be defined as a string or LPP.
minExtension	Minimum length to extend the wire beyond the cut.
-width	Maximum width of the wire. The wire must be narrower than the specified width for the constraint to apply.
-minLength	Minimum length of the wire, as measured from the end.
-edgeRange	Range of distances between the two wire ends. If the end-to-end distance is greater than edgeRange, then the extension is added. To specify ranges, see Range Data Syntax .
-cutClass	List of paired float values or a list of cutClass names. See cutClass and cutClassList . The constraint applies only in cases where the cuts comply with these specifications.
-color	Defines the specific color (positive value) of the Layer1 shapes. A value of zero means any color. A value of -1 means same color and a value of -2 means different color. The colorName must be defined in the Controls Section , and the color mask value cannot be greater than the numberOfColorMasks value defined in the functions section. Default is any.
-widthRange	If given, this constraint only applies to shapes with width in this range. This parameter or the width parameter should not be given at the same time.

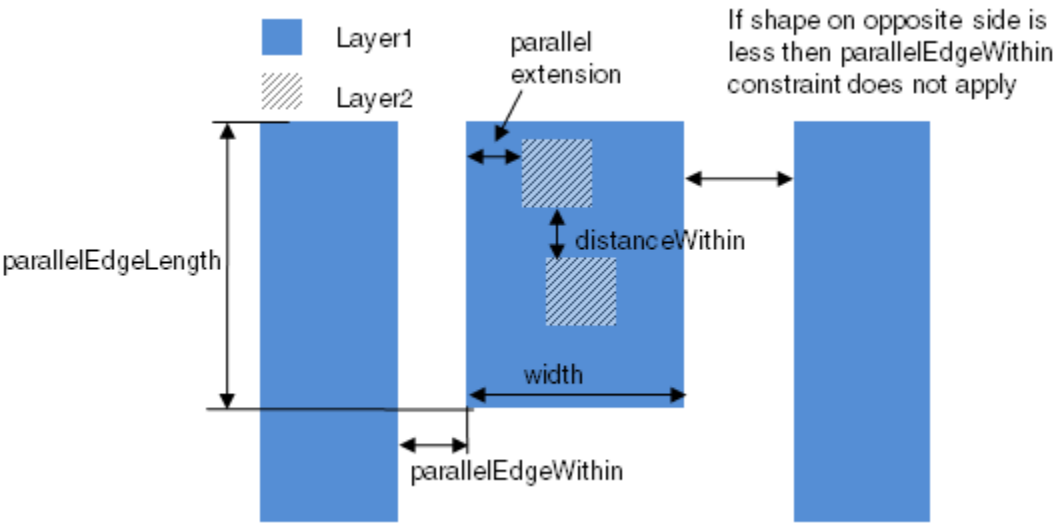
Example:

```
minExtensionAlignedProximity M1 V1 0.75
  -width 0.25 -minLength 0.78
```

```
-cutClass ((0.4 0.6) (0.6 0.8))  
-edgeRange (>=0.05 <=0.08)
```

minExtensionEdge

This constraint specifies the extension of one layer (typically, a metal layer) past another layer (typically a cut layer). The specified extension is typically larger than the minExtension or minDualExtension values.



Constraint Category	Extension
Constraint Type	Complex
OA Rule	oacMinExtensionEdge
Layer Symmetry	Asymmetric

Syntax:

```
minExtensionEdge  
  <string|LPP>layer1  
  <string|LPP>layer2>  
  <float>minExtension  
  -width float  
  -parallelEdgeLength float  
  -parallelEdgeWithin float  
  [-cutClass <cutClassArray | cutClassNameList>]  
  [-exceptExtraCut true|false]  
  [-distanceWithin float]  
  [-exceptTwoEdges true|false]  
  [-maxWidth float]  
  [-perpendicularWidth true|false]
```

```
[ -minParallelEdgeWithin float]
[ -minLength Value]
[ -endOfLineWidth float]
[ -exceptConcaveCornerWithin float]
[ -twoSides true|false]
[ -cutToMetalSpacing float]
[ -isHard true|false]
[ -comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer1	The layer which extends the layer2. layer1 can be defined as a string or LPP.
layer2	The layer which should be enclosed in layer1. layer2 can be defined as a string or LPP.
minExtension	Minimum extension of layer1 shape over layer2 shape.
-width	The minimum width of the extension shape for this constraint to be applied.
-parallelEdgeLength	Parallel run length must be greater than this value for the constraint to be applied.
-parallelEdgeWithin	Distance of parallel shape must be less than this value for the constraint to be applied.
-cutClass	List of paired float values or a list of cutClass names. See cutClass and cutClassList . The constraint applies only in cases where the cuts comply with these specifications.
-exceptExtraCut	If true, an extra cut within the distanceWithin parameter value causes this constraint to not apply.
-distanceWithin	Distance in which a second cut causes the constraint to not apply if exceptExtraCut is true.
-exceptTwoEdges	If true, then constraint does not apply if a shape on opposite side is less than distanceWithin distance and greater than parallelEdgeLength parallel run.
-maxWidth	If given, defines maximum width for which this constraint applies.
-perpendicularWidth	If true, width is measured perpendicular to extension edge, otherwise width is smaller dimension.
-minParallelEdgeWithin	If given, defines minimum spacing nearby shape must be for constraint to apply.
-minLength	Specifies the minimum length of the other adjacent edge which must have a parallel wire within parWithin.
-endOfLineWidth	An edge must be greater than or equal to endOfLineWidth to be considered a side edge. This parameter is optional, but if used, it must appear in conjunction with the spacingDirection parameter.

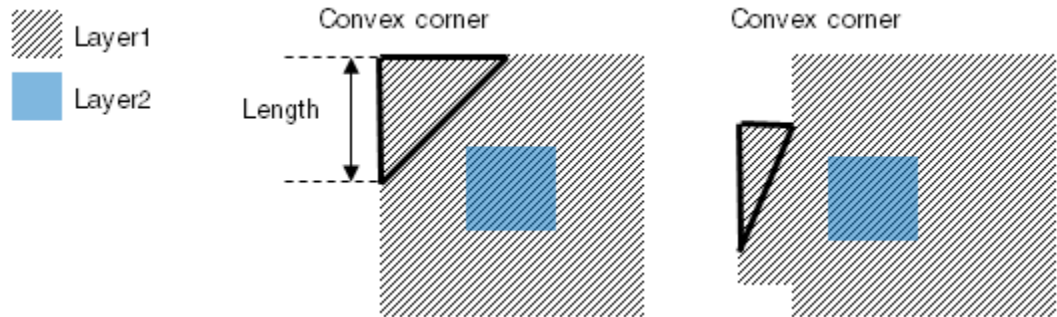
Argument	Description
-exceptConcaveCornerWithin	If present and set to within, then in the <code>TwoSides</code> case there must not be a concave corner on the above metal layer with within distance of the opposite overhang edges.
-twoSides	If <code>twoSides</code> is true, the line must be greater than or equal to <code>minLength</code> on both sides else this constraint does not apply.
-cutToMetalSpacing	The constraint only applies if the cut-edge to metal-edge space beyond the end-of-line edge is less than this value (<code>cutToMetalSpace</code>).

For example:

```
minExtensionEdge m1 v1 0.02 -width 0.12
                             -parallelEdgeLength 0.12
                             -parallelEdgeWithin 0.08
                             -cutClass 0.08 0.08
```

minExtensionToCorner

The minExtensionToCorner constraint requires that shapes being overlapped must be away from the enclosing shape's corner. The enclosed shape must not overlap a right triangle originating at a convex corner with the length of the right angle sides being a certain value or the complete edge length, which ever is smaller.



Constraint Category	Extension
Constraint Type	Complex
OA Rule	Custom Compiler custom layer pair constraint
Layer Symmetry	Asymmetric

Syntax:

```
minExtensionToCorner
<string|LPP>layer1
```

```
<string|LPP>layer2>  
<float>minEdgeLength  
[-toConvexCorner true|false]  
[-endOfLineWidth float]  
[-minLength float]  
[-cutClass <cutClassArray | cutClassNameList>]  
[-isHard true|false]  
[-comment [<string rule_id>] <string rule_desc>]
```

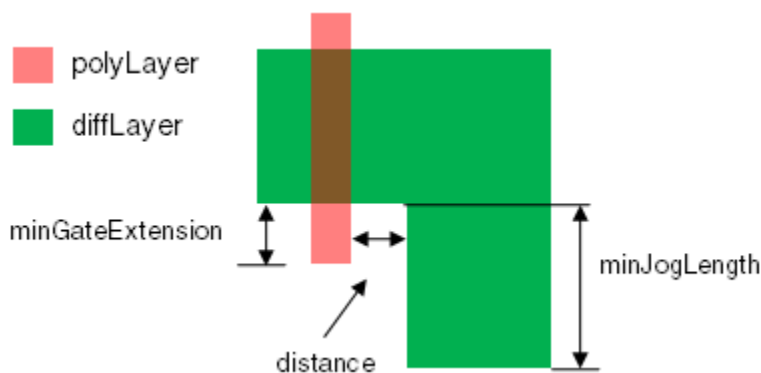
Argument	Description
layer1	The layer which extends the layer2. layer1 can be defined as a string or LPP.
layer2	The layer which should be enclosed in layer1. layer2 can be defined as a string or LPP.
minEdgeLength	Minimum edge length of layer1 shape over layer2 shape.
-toConvexCorner	If <code>toConvexCorner</code> is specified, the constraint's value specifies a triangular keep-out region in each convex corner of shapes on the first layer. Shapes on the second layer which overlap this region violate the constraint. The default is false.
-endOfLineWidth	An edge must be greater than or equal to <code>endOfLineWidth</code> to be considered a side edge. The default is 0.
-minLength	This is an optional parameter, but is required when <code>convexLength</code> is set. This parameter specifies the minimum length of the other adjacent edge which must have a parallel wire within <code>parWithin</code> .
-cutClass	List of paired float values or a list of cutClass names. See cutClass and cutClassList . The constraint applies only in cases where the cuts comply with these specifications.

For example:

```
minExtensionToCorner m1 v1 0.06 -toConvexCorner true
```

minGateExtension

This constraint specifies the gate extension if a layer1 shape is within specific distance of an L-shape of the same layer2 shape.



Constraint Category	Extension
Constraint Type	Complex
OA Rule	oacMinGateExtension
Layer Symmetry	Asymmetric

Syntax:

```
minGateExtension
<string|LPP>polyLayer
<string|LPP>diffLayer
<float>minExtension
[-distance float]
[-minJogLength float]
[-isHard true|false]
[-comment [<string rule_id>] <string rule_desc>]
```

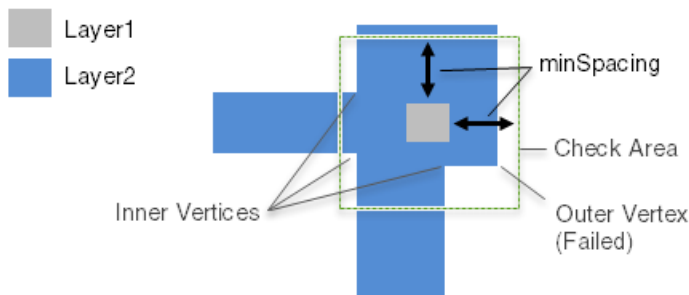
Argument	Description
polyLayer	The layer which extends beyond the diffLayer. polyLayer can be defined as a string or LPP.
diffLayer	The layer which should be enclosed in polyLayer. diffLayer can be defined as a string or LPP.
minExtension	Minimum extension of layer1 shape extends layer2 LShape.
-distance	The distance threshold Layer1 shapes greater than or equal to this value do not apply.
-minJogLength	The minimum jog length of layer2 shape for which constraint is applied.

For example:

```
minGateExtension poly diff 0.02 -distance 0.08
-minJogLength 0.06
```

minOuterVertexSpacing

This constraint specifies a square check area around shapes on layer 1. Any outer vertices on layer 2 (enclosing) shapes must be separated by a minimum distance from the edges of layer 1 shapes. This layer pair constraint is not symmetric.



Constraint Category	Spacing
Constraint Type	Simple
OA Rule	oacMinOuterVertexSpacing
Layer Symmetry	Asymmetric

Syntax:

```
minOuterVertexSpacing
<string|LPP>layer1
<string|LPP>layer2
<float>minSpacing
[-isHard true|false]
[-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer1	layer1 can be defined as a string or LPP.
layer2	layer2 can be defined as a string or LPP.
minSpacing	This value specifies the required minimum spacing in microns.

For example:

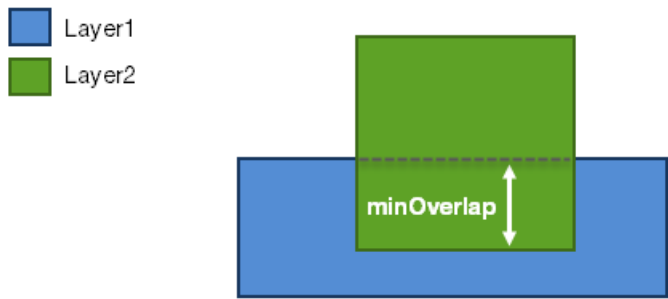
```
minOuterVertexSpacing (poly drawing) (diff drawing) 0.20
```

minOverlap

This layer pair constraint specifies the minimum distance a shape on one layer must overlap a shape on a second layer in microns.

Overlaps are specified from the inside edge of the shape on layer2 to the inside edge of the shape on layer1.

This constraint definition is not symmetric. The minimum overlap of layer1 past layer2 is not the same as the minimum overlap of layer2 past layer1.



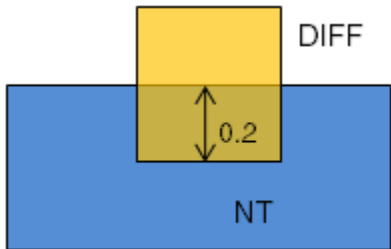
Constraint Category	Overlap
Constraint Type	Simple
OA Rule	oacMinOverlap
Layer Symmetry	Asymmetric

Syntax:

```
minOverlap
  <string|LPP>layer1
  <string|LPP>layer2
  <float>overlapValue
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer1	The layer which overlaps the layer2. layer1 can be defined as a string or LPP.
layer2	The layer2 is overlapped by layer1. layer2 can be defined as a string or LPP.
overlapValue	Specifies by how much layer1 overlaps layer2.

Rule	Description	Value (in microns)
NT.OV.1	Minimum overlap of NT over DIFF	0.2



minOverlap NT DIFF 0.2

minSameNetClearance

This layer pair constraint specifies the minimum clearance required between two geometries on different layers when the geometries are electrically equivalent.

This rule is required in the following situations:

- If the same-net clearance is smaller than the different-net clearance specified with the [minClearance](#) constraint
- If vias on different layers have special stacking rules.

This is a symmetric constraint definition. The minimum clearance between layer1 and layer2 is the same as the minimum clearance between layer2 and layer1.

Constraint Category	Clearance
Constraint Type	Simple
OA Rule	oaMinSameNetClearance
Layer Symmetry	Symmetric

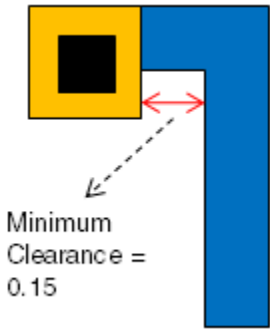
Syntax:

```
minSameNetClearance
  <string|LPP>layer1
  <string|LPP>layer2
  <float>clearanceValue
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer1	The layer which overlaps the layer2. layer1 can be defined as a string or LPP.
layer2	The layer2 is overlapped by layer1. layer2 can be defined as a string or LPP.
clearanceValue	Specifies the minimum clearance required between two geometries on different layers.

In this example, the minimum spacing between M1 and M2 when on the same net is mentioned:

Rule	Description	Value (in microns)
M1.SP.2	Minimum M1 - M2 spacing when on the same net	0.15

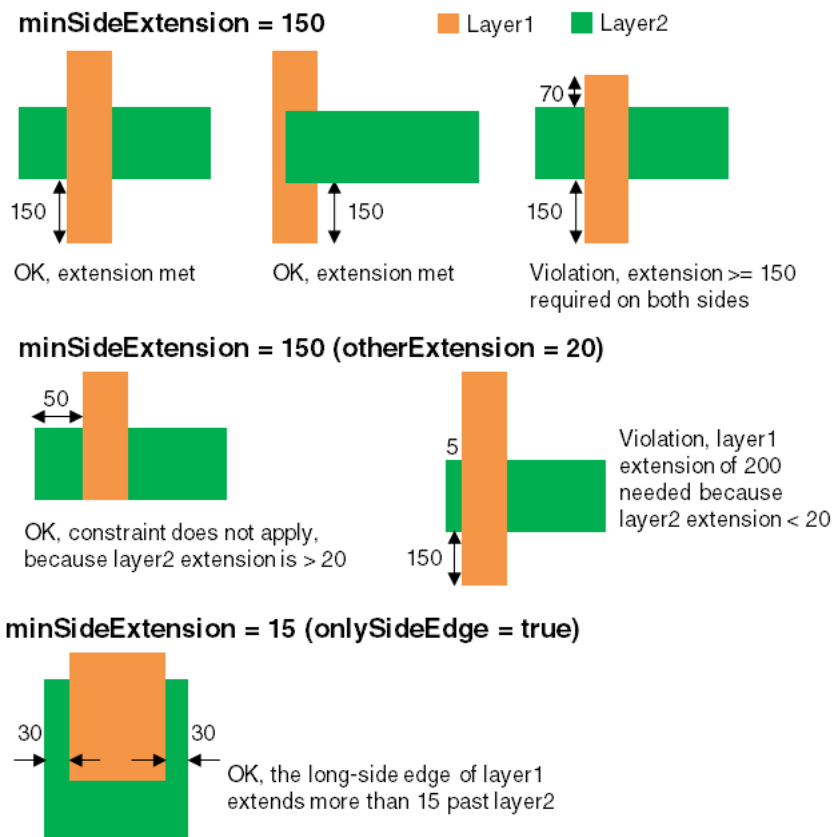


minSameNetClearance M1 M2 0.15

minSideExtension

The extension that either the short or long side edge of a layer1 shape has to extend past a layer2 shape.

Optionally, the constraint might only apply if the layer2 extension is $\leq$ a certain value. The constraint might not apply to all layer1 or layer2 widths. Two parameters indicate the set of width ranges for each layer when the constraint applies.



Constraint Category	Clearance
Constraint Type	Simple
OA Rule	oaMinSideExtension
Layer Symmetry	Asymmetric

Syntax:

```
minSideExtension
<string|LPP>layer1
<string|LPP>layer2
[-otherExtension float]
[-longSideEdge true|false]
[-widthRangeArray rangeValue]
[-otherWidthRangeArray rangeValue]
[-isHard true|false]
[-comment [<string rule_id>] <string rule_desc>]
```


Argument	Description
layer1	The layer which overlaps the layer2. layer1 can be defined as a string or LPP.
layer2	The layer2 is overlapped by layer1. layer2 can be defined as a string or LPP.
-otherExtension	The <code>otherExtension</code> constraint applies only if layer2 extension past layer1 is $\leq$ this value.
-longSideEdge	If true, the constraint applies to the long side edge of layer1, otherwise to the short side edge.
-widthRangeArray	rangeValue: If given, the constraint applies only if layer1 width is in one of the ranges. To specify the ranges, see Range Data Syntax .
-otherWidthRangeArray	rangeValue: If given, the constraint applies only if layer2 width is in one of the ranges. To specify the ranges, see Range Data Syntax .

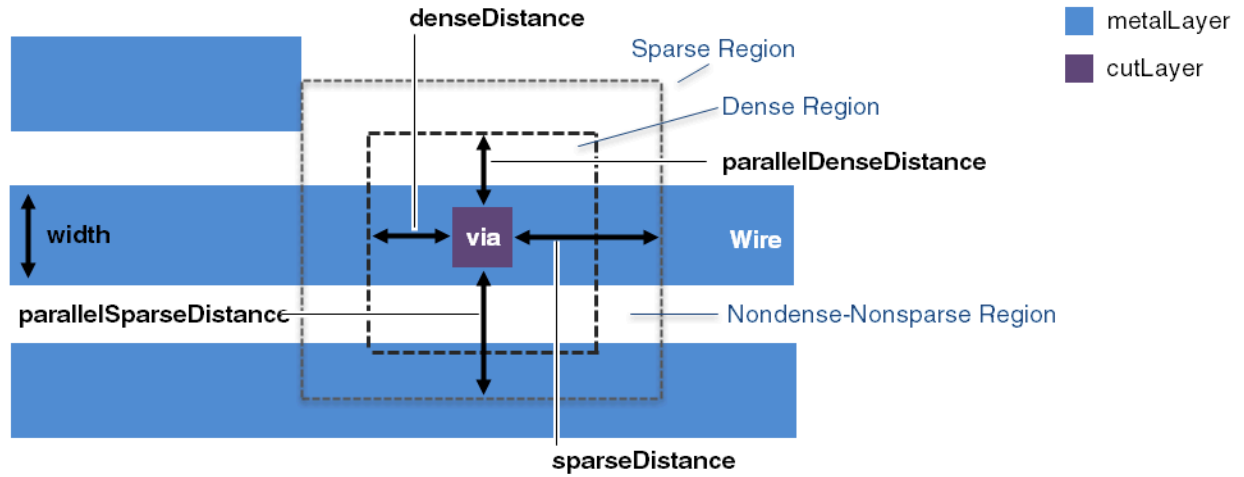
For example:

```
minSideExtension m1 m2 0.15
                 -otherExtension 0.25
                 -longSideEdge true
                 -widthRangeArray( (>=0.1 <=0.2) (0.5 0.6) (0.7 <=0.9) )
```

minSparseDenseExtension

In advanced node designs, metal layer enclosures for cuts are required when:

- The `width` of the upper level or lower level metal that is enclosing the cut is within the specified range.
- Of the two neighboring metal shapes on the opposing sides of the wire, at least one is located within the nondense-nonsparse region.



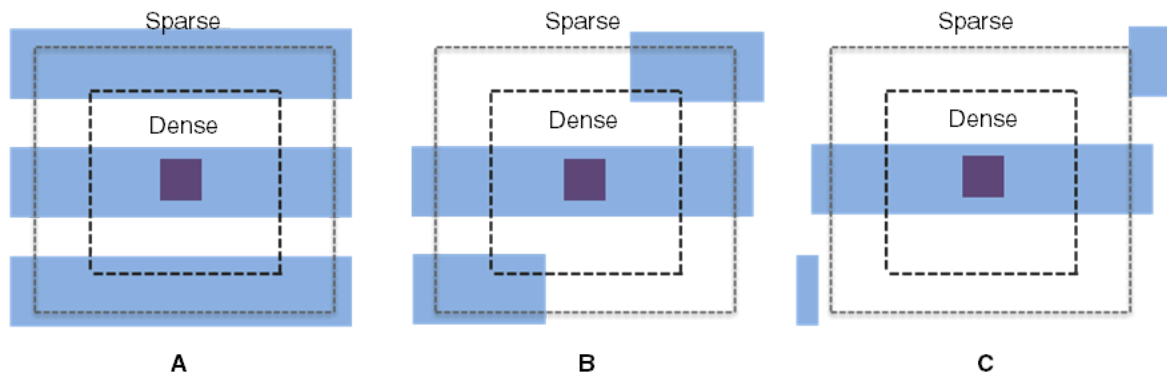
The constraint does not applied when:

- Both neighbors are within the dense region (A and B).
- Both neighbors are only in the sparse region (C).

Note:

Metal shapes that touch a region border is not considered to be within that region, as shown in example C.

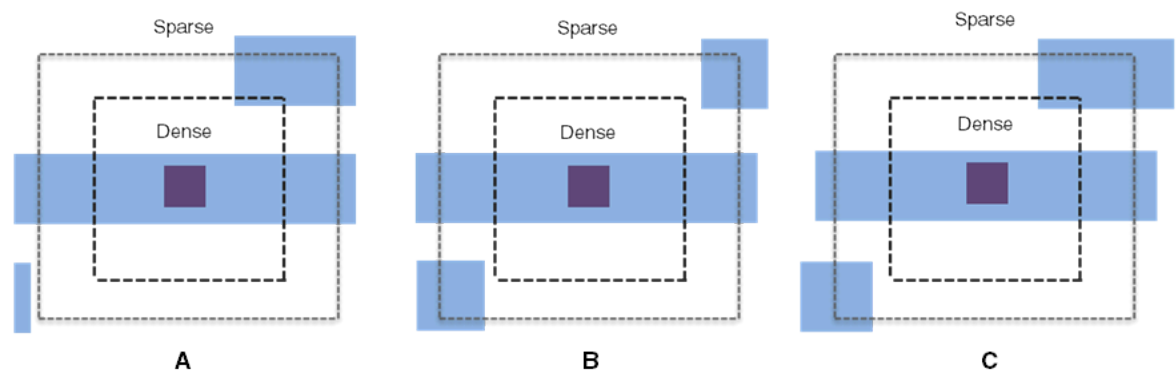
Conditions under which the constraint does not apply



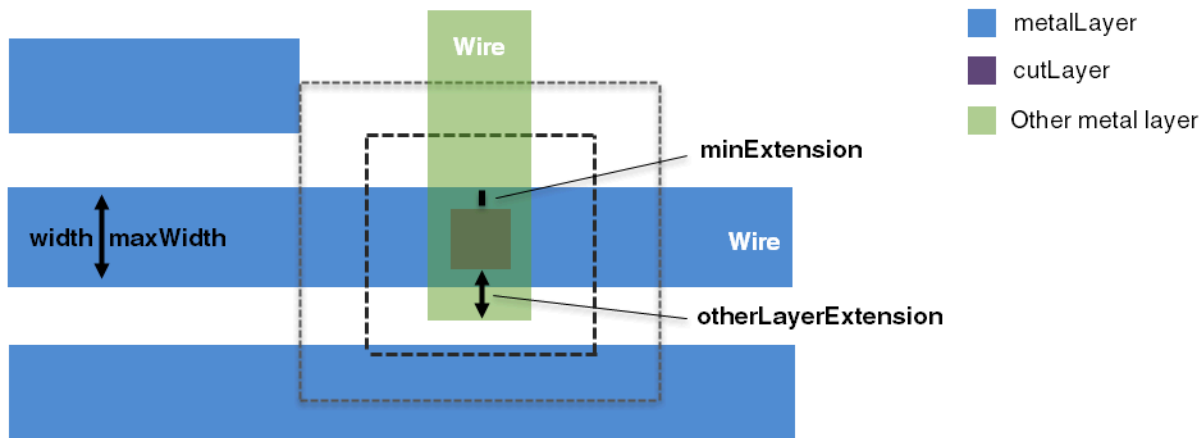
The following examples show when the constraint applies:

- **A:** One neighbor is within the dense region. The neighbor on the opposing side is within the sparse region only.
- **B:** Both neighbors are within the nondense-nonsparse region.
- **C:** One neighbor is within the dense region. The neighbor on the opposing side is within the nondense-nonsparse region.

Conditions under which the constraint applies



The upper and lower metal layers that enclose the cut must meet the `minExtension` and `otherLayerExtension` requirements, as the following figure shows.



Constraint Category	Extension
Constraint Type	Complex
OA Rule	Custom Compiler Custom Layer Pair Constraint
Layer Symmetry	Asymmetric

Syntax:

```
minSparseDenseExtension
  <string|LPP>metalLayer
  <string|LPP>cutLayer
  <float>minExtension
  -width float
  [-maxWidth float]
  -denseDistance float
  -parallelDenseDistance float
  [-sparseDistance float]
  [-cutClass <cutClassArray | cutClassNameList>]
  [-parallelSparseDistance float]
  [-otherLayerExtension float]
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
metalLayer	metalLayer can be a string (layer name) or LPP.
cutLayer	cutLayer can be a string (layer name) or LPP.
-width	Minimum width, in microns, of the enclosing layer for the constraint to apply. The enclosing layer must be greater than or equal to this width value.
-maxWidth	(Optional) Maximum width, in microns, of the enclosing layer for the constraint to apply.
-denseDistance	Distance measured from the edge of the cut in the direction that is perpendicular to the width direction. denseDistance and parallelDenseDistance are dimensions that define the dense region.
-parallelDenseDistance	Distance measured from the edge of the cut in the direction that is parallel to the width direction. denseDistance and parallelDenseDistance are dimensions that define the dense region.
-sparseDistance	(Optional) Distance measured from the edge of the cut in the direction that is perpendicular to the width direction. sparseDistance and parallelSparseDistance are dimensions that define the nondense-nonsparse region. Default: denseDistance
-parallelSparseDistance	(Optional) Distance measured from the edge of the cut in the direction that is parallel to the width direction. sparseDistance and parallelSparseDistance are dimensions that define the nondense-nonsparse region. Default: parallelDenseDistance
-cutClass	(Optional) List of paired float values or a list of cutClass names. See cutClass and cutClassList . The constraint applies only in cases where the cuts comply with these specifications.
-otherLayerExtension	(Optional) Minimum distance between the edge of the cut and the end of the other metal layer. The distance measured must be perpendicular to the width measurement.

Example:

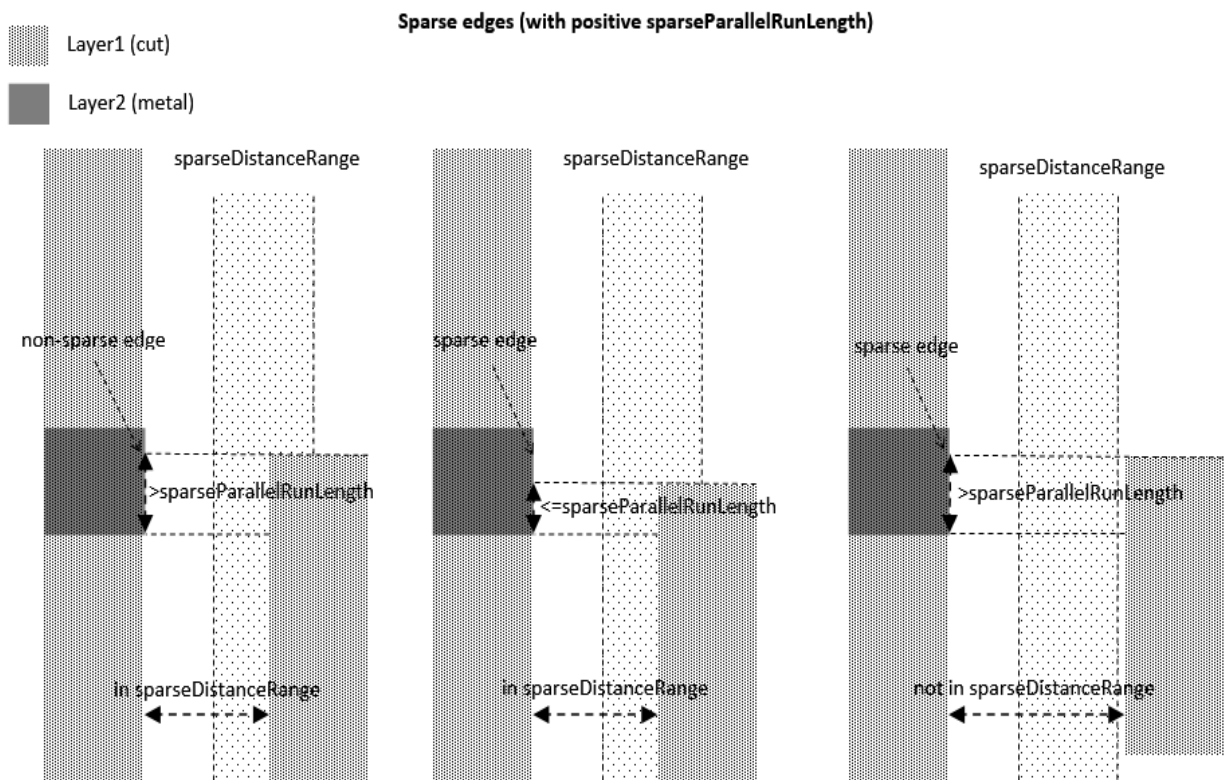
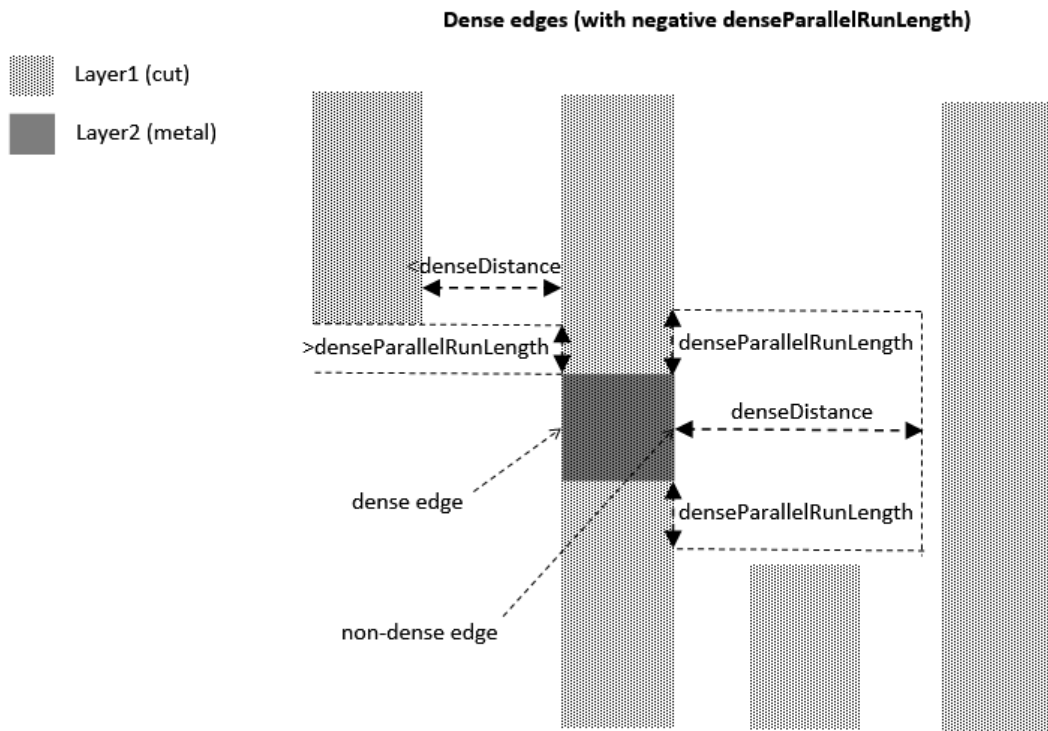
```
minSparseDenseExtension M1 V1 0.75
    -width 0.25
    -denseDistance 0.78
    -ParallelDenseDistance 0.23
    -cutClass ((0.4 0.6) (0.6 0.8))
    -sparseDistance 0.35
    -maxWidth 0.88
    -parallelSparseDistance 0.55
    -otherLayerExtension 0.35
```

minSparseDenseViaSpacing

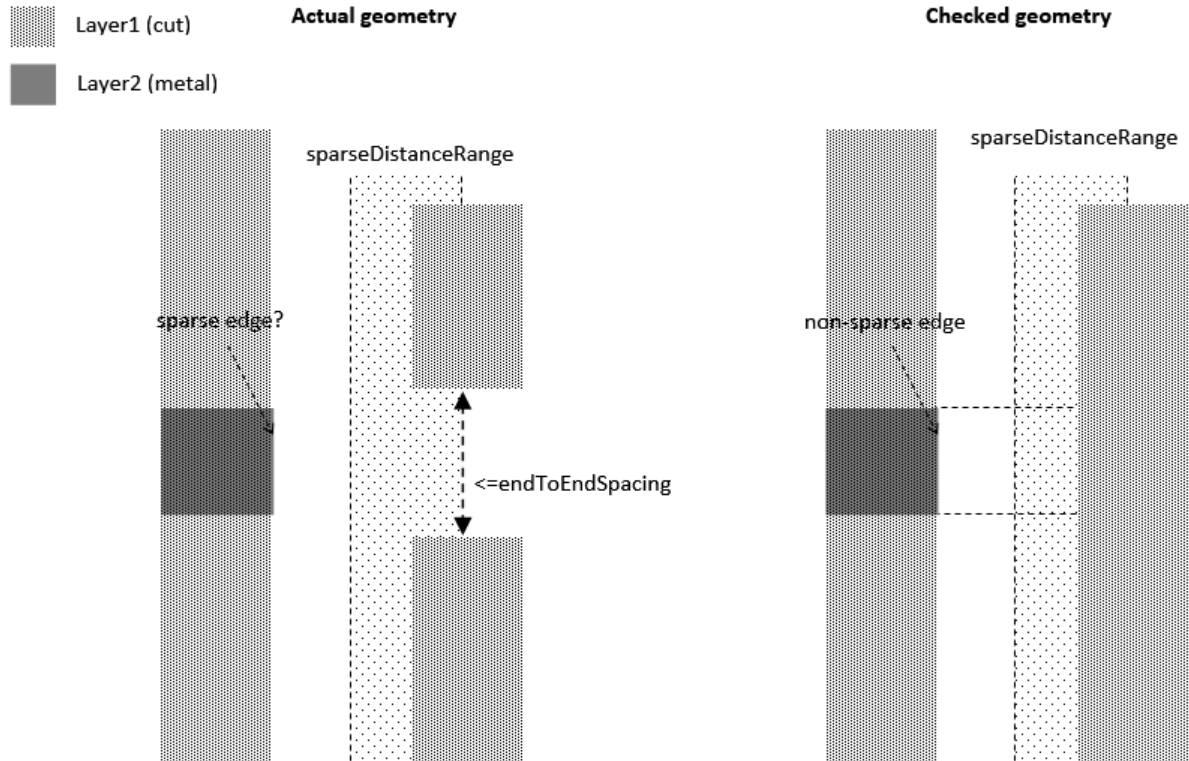
Advanced processes may require different via spacing depending on the presence of nearby metal shapes on one side of the via cut and the absence of nearby metal shapes on the opposite side.

The rule defines the following cut shape edge categories depending on the presence of nearby metal:

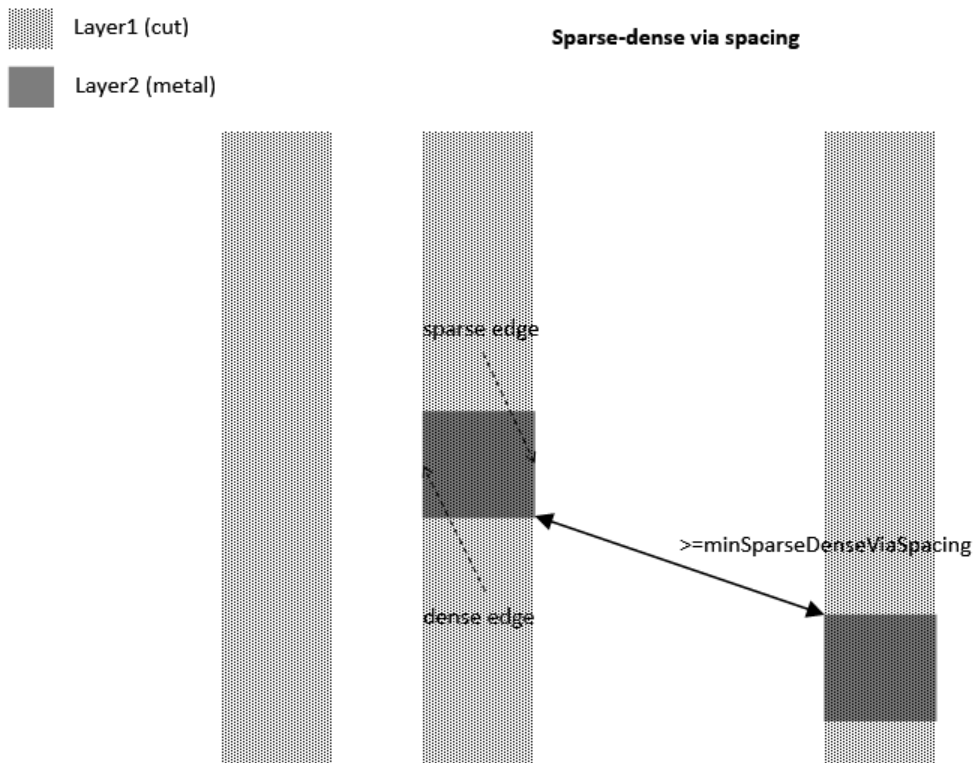
- The dense via edge is a via layer1 edge oriented in the preferred routing direction of metal layer2 which has a nearby metal layer2 shape within particular distance and with particular minimum parallel run length.
- The sparse via edge is a via layer1 edge oriented in the preferred routing direction of metal layer2 which does not have a nearby metal layer2 shape within particular distance range and with particular minimum parallel run length.
- The sparse-dense via cut is a via cut with both a sparse edge and a dense edge. By default, the sparse edge and the dense edge must be located on the opposite sides of the via cut. Optionally, the dense edge and the sparse edge can be located on the same side of the via cut. In this case, the same edge is both sparse and dense.



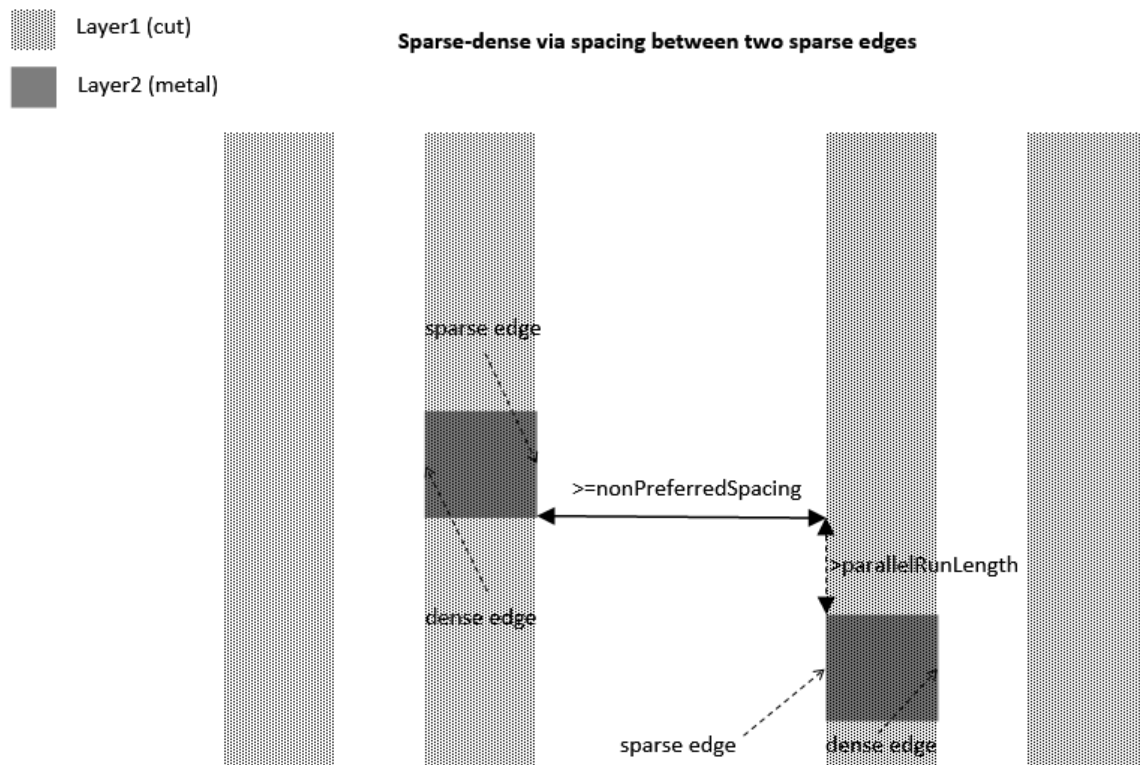
When determining sparse edges, it is also necessary to consider the `endToEndSpacing` parameter, if the `endToEndSpacing` parameter is given. If spacing between nearby metal line ends is less than or equal to the value of the `endToEndSpacing` parameter, it is necessary to merge the metal shapes into one wire before checking for conditions of the sparse edge.



The rule requires minimum spacing between the sparse edge of a sparse-dense via cut and any other via cut shape on layer1.



Optionally, the rule can require a bigger minimum spacing in non-preferred routing direction of metal layer2 between two sparse edges of two sparse-dense via cuts with minimum parallel run length.



Constraint Category	Via Construction
Constraint Type	Complex
OA Rule	Custom Compiler Custom Layer Pair Constraint
Layer Symmetry	Asymmetric

Syntax:

```
minSparseDenseViaSpacing
<string|LPP>layer1
<string|LPP>layer2
<float>spacingValue
-denseDistance floatValue
-sparseDistanceRange rangeData
[-denseParallelRunLength floatValue]
[-sparseParallelRunLength floatValue]
[-sameSide boolValue]
[-nonPreferredSpacing floatValue]
[-parallelRunLength floatValue]
[-endToEndSpacing floatValue]
[-color colorName|any|same]
[-cutClass dualValue | cutClassName]
```

```
[ -cutClassList cutClassArray|cutClassNameList]]
[ -isHard true|false]
[ -comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer1	layer1 can be defined as a string or LPP.
layer2	layer2 can be defined as a string or LPP.
spacingValue	Minimum spacing in database units from sparse edge of sparse-dense layer1 cut shape to any other layer1 cut shape.
-denseDistance	Defines a dense edge. Distance from the dense edge to a metal shape must be less than this value in database.
-sparseDistanceRange	Defines a sparse edge. Distance from the sparse edge to a metal shape cannot be in this range in database units.
-denseParallelRunLength	Defines a dense edge. Parallel run length between the dense edge and a metal shape must be greater than this value in database units. Default: 0.
-sparseParallelRunLength	Defines a sparse edge. Parallel run length between the sparse edge and a metal shape cannot be greater than this value in database units. Default: 0.
-sameSide	If false, the dense edge and the sparse edge must be located on the opposite sides of the sparse-dense via cut. If true, the dense edge must be located on the same side of the via cut as the sparse edge. The same edge is both sparse and dense. Default: false.
-nonPreferredSpacing	If specified, spacing in non-preferred metal layer direction between two sparse edges of two sparse-dense via cuts with parallel run length greater than the <code>parallelRunLength</code> parameter value must be greater than or equal to this value.
-parallelRunLength	Only used with the <code>nonPreferredSpacing</code> parameter. Default: 0.
-endToEndSpacing	Two metal wires with end-to-end spacing less than or equal to this value in database units are merged together.
-color	The constraint only applies when all cut shapes under consideration have this mask color. A value of zero means any color. A value of -1 means same color. Default: 0.
-cutClass	The constraint only applies to sparse-dense via cut of this cut class.
-cutClassList	Array of <code>oaDualInt</code> values, each defining width and height of a cut class. If given, the constraint only applies to sparse-dense via cuts from these cut classes. This parameter cannot be given if the <code>cutClass</code> parameter is also given.

Note:

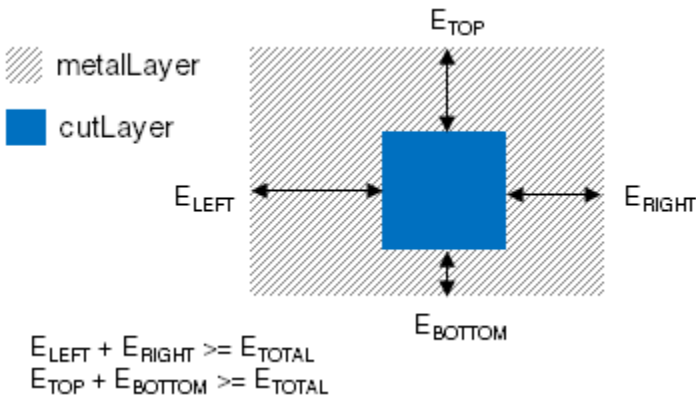
- The `colorName` must be defined in the [Controls Section](#), and the color mask value cannot be greater than the `numberOfColorMasks` value defined in the [functions](#) section.
- If `cutClass` parameter is specified with `cutClassName` in ASCII file, technology importer looks-up the first `cutClass` constraint specified in the same hierarchical constraint group to convert to cut class size. If a `cutClass` constraint does not exist then an error message is displayed.
- The error message is displayed in the following situations:
 - `-parallelRunLength` parameter is specified and `-nonPreferredSpacing` is not specified.

Example

```
minSparseDenseViaSpacing VIA1 M1 0.12
    -denseDistance 0.07
    -sparseDistanceRange (MINUS_INFINITY <0.08)
    -cutClassList ( (0.02 0.04) (0.06 0.08) )
```

minTotalEnclosure

This layer pair constraint specifies that the sum of both enclosures, left/right or top/bottom, must be a minimum value, but only if the total enclosure of the other direction is less than a certain value.



Constraint Category	Spacing
Constraint Type	Complex
OA Rule	Custom Compiler Custom layer pair constraint
Layer Symmetry	Asymmetric

Syntax:

```
minTotalEnclosure
  <string|LPP>metalLayer
  <string|LPP>cutLayer
  <float> enclosureValue
  -minEnclosure <float>
  [-cutClass <cutClassArray | cutClassNameList>]
  [-widthRange rangeValue]
  [-spacingDirection any|horizontal|vertical]
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
metalLayer	metalLayer can be defined as a string or LPP.
cutLayer	cutLayer can be defined as a string or LPP.
enclosureValue	Specifies the minimum total enclosure needed in one direction.
-minEnclosure	The rule applies if the total enclosure in one direction is less than this value. The other direction must then have a minimum enclosure greater than or equal to the constraint value.
-cutClass	List of paired float values or a list of cutClass names. See cutClass and cutClassList . The constraint applies only in cases where the cuts comply with these specifications.
-widthRange	rangeValue: If given, upper metal width must be in this range for the constraint to apply. To specify the ranges, see Range Data Syntax .
-spacingDirection	Direction of the enclosure. Valid values are any, horizontal, and vertical. Default: any

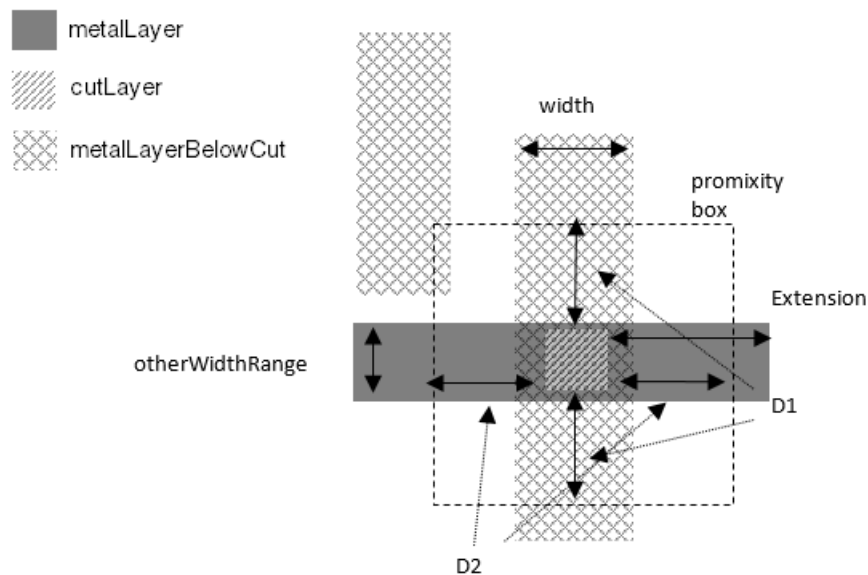
For example,

```
minTotalEnclosure M1 V1 0.10
  -minEnclosure 0.12
  -cutClass ((0.06 0.06) (0.06 0.08))
  -widthRange (>=0.09 <=0.15)
  -spacingDirection vertical
```

minUpperMetalExtension

This layer pair constraint specifies the requirements for extending the metal layer that is on top of a via under the following circumstances:

- The shape below the via has a certain color.
- The shape below the via is adjacent to another shape on the same layer, but that shape has a different color.



Constraint Category	Spacing
Constraint Type	Complex
OA Rule	Custom Compiler custom layer pair constraint
Layer Symmetry	Asymmetric

Syntax:

```
minUpperMetalExtension
  <string|LPP>metalLayer
  <string|LPP>cutLayer
  <float> enclosureValue
  -width float
  -minExtension float
  -lowerColor colorName
  -cutClass <cutClassArray | cutClassNameList>
  -otherColor colorName
  -otherWidthRange rangeValue
  -proximityBox float float
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
metalLayer	metalLayer can be defined as a string or LPP.
cutLayer	cutLayer can be defined as a string or LPP.
enclosureValue	Specifies the minimum total enclosure needed in one direction.

Argument	Description
-width	Maximum width of the wire below the via.
-minExtension	Minimum length by which the layer 1 wire must extend beyond the via.
-lowerColor	Color of the metalLayerBelowCut wire for which this constraint applies. The colorName must be defined in the Controls Section .
-cutClass	List of paired float values or a list of cutClass names. See cutClass and cutClassList . The constraint applies only in cases where the cuts comply with these specifications.
-otherColor	Color of the metalLayerBelowCut shape that is adjacent to the metalLayerBelowCut wire. The colorName must be defined in the Controls Section .
-otherWidthRange	Range of widths of the metalLayer wire. To specify the ranges, see Range Data Syntax .
-proximityBox	Dimensions of the proximity box (D2 D1).

Example:

```
minUpperMetalExtension M1 V1 0.75
    -width 0.25 -minExtension 0.78
    -lowerColor red -cutClass 0.4 0.6
    -otherColor green -otherWidthRange (>=0.05 <=0.08)
    -proximityBox 0.45 0.87
```

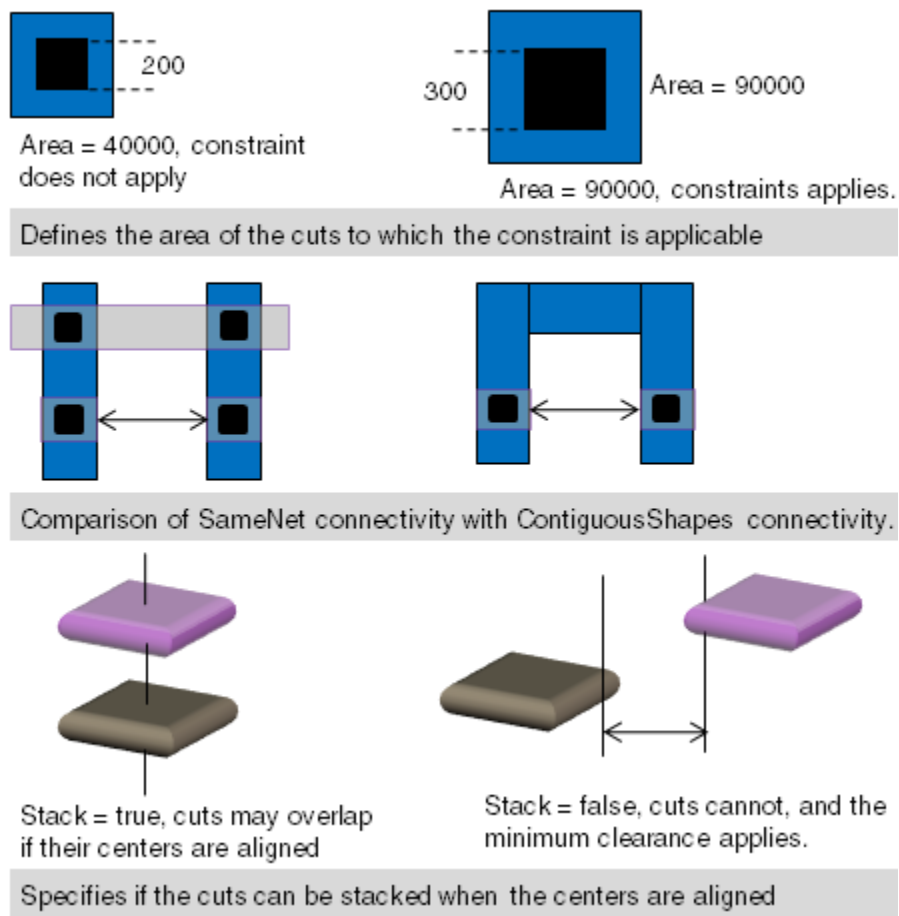
minViaClearance

This layer pair constraint specifies the via cut clearance between cuts on different layers.

Optional parameters can be used to specify the following:

- Whether clearance must be measured from center-to-center or from edge-to-edge
- Whether the connectivity type for this constraint should be any, same net, or same metal
- Cuts with an area equal to or greater than a specified value must meet the specified clearance

Optionally, the rule can be applied only to cuts of particular cut classes.



Constraint Category	Clearance
Constraint Type	Complex
OA Rule	oacMinViaClearance
Layer Symmetry	Asymmetric

Syntax:

```
minViaClearance
<string|LPP>layer1
<string|LPP>layer2
<float|float1DTblValue|float2DTblValue>spacingValue

[-connectivityType
    anyConnectivityType|
    sameNetConnectivityType|
    contiguousShapesConnectivityType]
```

```
[-area <float>]
[-stack true|false]
[-exceptConnectivityType sameNetConnectivityType |
                           contiguousShapesConnectivityType |
                           sameViaConnectivityType |
                           directShapesConnectivityType]
[-cutSpacingType edgeX|edgeY|corner|center|edge]
[-voltageRange frangeData]
[-cutClassList1 cutClassArray|cutClassNameList]
[-cutClassList2 cutClassArray|cutClassNameList]

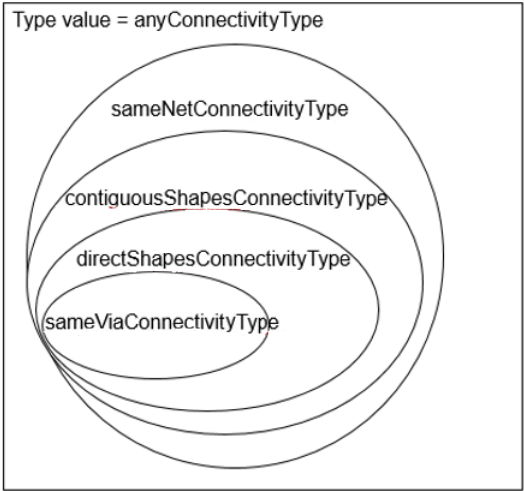
[-isHard true|false]
[-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer1	layer1 can be defined as a string or LPP. It should be a cut layer.
layer2	layer2 can be defined as a string or LPP. It should be a cut layer.
spacingValue	float: specifies the minimum spacing required between the two cut shapes. float1DTblValue: Specifies the width dependent spacing value between the two cuts based on the width of the wider of the two shapes. float2DTblValue: Specifies the minimum spacing between the cuts based on the width of both the shapes.
-connectivityType	anyConnectivityType: Use this option to designate that the constraint applies to any type of connectivity (default). sameNetConnectivityType: Use this option to designate that the constraint only applies to cuts on the same net. contiguousShapesConnectivityType: Use this option to designate that the constraint only applies to cuts on the same metal.
-exceptConnectivityType	Specifies the type of connectivity between the two cuts that causes the constraint to not apply.
-area	Specifies the cut area, which when exceeded, requires the specified clearance. Default: 0.
-stack	Specifies if via cuts on the two different layers can be stacked if the cuts are exactly aligned. Default is "false"
-cutSpacingType	If specified, the constraint applies only to the given type of spacing. See -cutSpacingType in generalCutClassSpacing . This parameter supersedes the <code>oaCenterToCenter</code> parameter.
-voltageRange	If given, the rule only applies if the difference between voltages of the cuts lies in this range in volts. See Voltage Dependent Design Rules .
-cutClassList1	Array of <code>oaDualInt</code> values, each defining width and height of a cut class in database units. If given, the constraint only applies to those cuts on the first layer (layer1) which cut class is from this cut class list. See cutClass and cutClassList .

Argument	Description
-cutClassList2	Array of oaDualInt values, each defining width and height of a cut class in database units. If given, the constraint only applies to those cuts on the second layer (layer2) which cut class is from this cut class list. See cutClass and cutClassList .

Note:

- If `cutClassList1` or `cutClassList2` parameter is specified with `cutClassName` in the ASCII file, the tech importer looks up the first `cutClass` constraint specified in the same hierarchical constraint group to convert to cut class size ; if no `cutClass` constraint existing, report the error (dual value format is not be reported). In exporting, the tech exporter looks up the first `cutClass` constraint specified in the same hierarchical constraint group to convert cut class size to cut class name ; if no `cutClass` constraint existing, to export `cutClass` parameter with cut class size in dual value.
- Report errors when following scenarios occurs:
 - When `-exceptConnectivityType` has a larger scope than `-connectivityType`, `-connectivityType` parameters conflict with `-exceptConnectivityType` parameters.



For example, when `-connectivityType = any` and `-exceptConnectivityType = sameVia`, no error is reported.

When `-connectivityType = sameVia` and `-exceptConnectivityType = sameNet`, errors are reported.

When `-connectivityType = sameNet` and `-exceptConnectivityType = sameNet`, errors are reported.

Rule	Description	Value (in microns)
V1.CL.1	Min center to center V1 - V2 Spacing	0.08
V1.CL.1.1	Min V1 - V2 spacing when V1/V2 width $\geq$ 0.1	0.12
V1.CL.1.2	Min V1-V2 spacing when V1 and V2 width $\geq$ 0.12	0.15

Examples

Example 1

```
//minViaClearance constraint specified with simple value
minViaClearance V1 V2 0.08
```

Example 2

```
//minViaClearance constraint specified with simple value and parameters
minViaClearance v1 v2 0.08 -cutSpacingType edge
                           -connectivityType sameNetConnectivityType
                           -exceptConnectivityType sameVia
```

Example 3

```
//minViaClearance constraint specified with width depending value
minViaClearance v1 v2 0.06 ((0.12 0.08) (0.15 0.10))
```

Example 4

```
//minViaClearance constraint specified with two width depending values
minViaClearance v1 v2 0.06
    (((0.12 0.14) 0.08)
     ((0.12 0.18) 0.08)
     ((0.15 0.14) 0.08)
     ((0.15 0.18) 0.10) )
-area 0.6
```

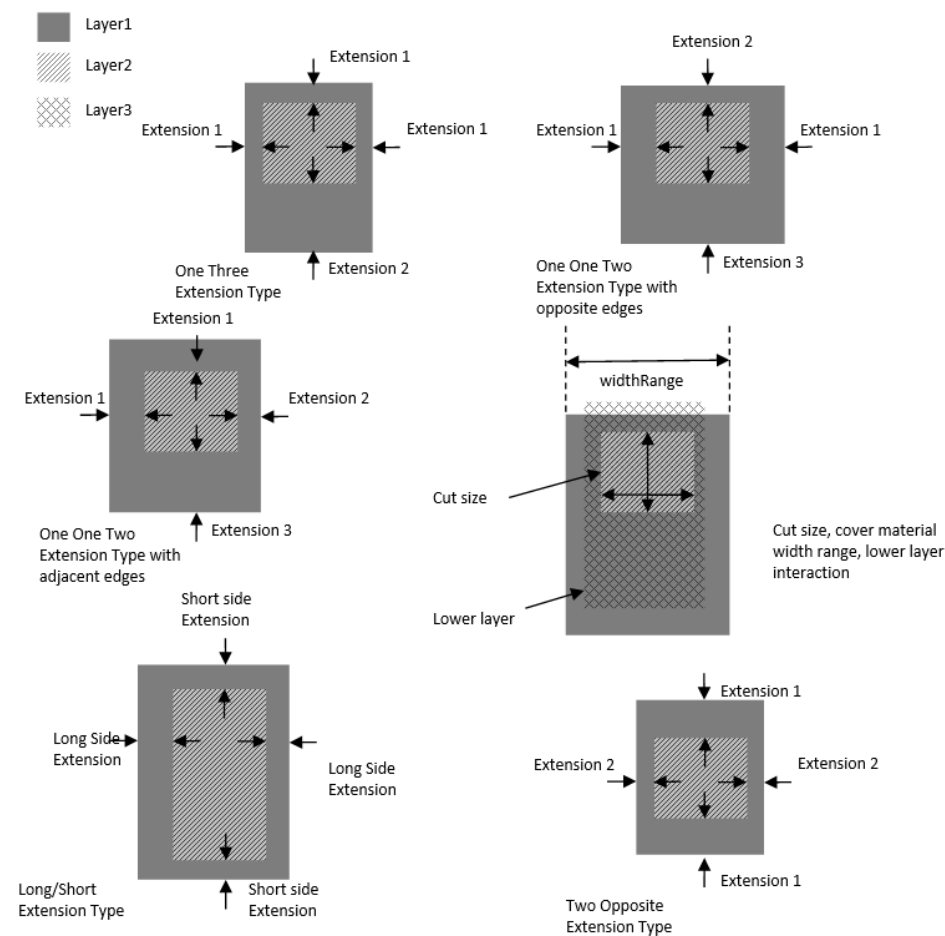
Example 5

```
//minViaClearance constraint specified with cutClass
minViaClearance v1 v2 0.06
    -area 0.6
    -cutClassList1 ((0.02 0.04) (0.06 0.08))
    -cutClassList2 (V2LRG V2BAR_40_40)
```

minViaExtension

The minViaExtension constraint specifies the required metal extension for vias. It supports the following configurations:

- On three sides, the extension should be greater than or equal to a certain value X as long as one side is greater than or equal to a certain value Y.
- The extension on two opposite sides needs to be greater than or equal to a certain value X as long as one side is greater than or equal to a certain value Y and another side is greater than or equal to a certain value Z.
- In addition to the normal extension X, a larger minimum extension y is required to a diagonal edge.



Constraint Category

Via Construction

Constraint Type

Complex

OA Rule	oacMinViaExtension
Layer Symmetry	Asymmetric

Syntax:

```
minViaExtension
  <string|LPP>metalLayer
  <string|LPP>cutLayer
  <floatArray>extensionValue
  -viaExtensionType <string oneThree|oneOneTwo|diagonal|twoOpposite
                        |oneOneTwoAdjacent | longShort
                        horizontalVertical| verticalHorizontal>

  [-widthRange <rangeValue>]
  [-cutClass <cutClassArray | cutClassNameList>]
  [-minLength <float>]
  [-otherLayer <string>layerName]
  [-otherLayerWidth <rangeValue>]
  [-otherColor colorName]
  [-color any|colorName]
  [-enclosingShapeDirection any|horizontal|vertical]
  [-exceptCutDistanceRange RangeData]
    [-cutDistanceRange RangeData]
    [-cutPRLRange RangeData]
    colorName ::= string
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
metalLayer	metalLayer can be defined as a string or LPP.
cutLayer	cutLayer can be defined as a string or LPP.
extensionValue	An array of float values that specify the amount of the extension. The extensionType specifies how to interpret the extensionValue.

Argument	Description
-viaExtensionType	OneThree: First array value used for three sides, second value used for fourth side. oneOneTwo: First array value used for two opposite sides, second value used for one side, third used for other side. diagonalVia: The first value is used for non-diagonal edges and the second value is for diagonal edges. The first two are identical to <i>oneThree</i> type. The third is the required extension to a diagonal edge. The first three are identical to <i>oneOneTwo</i> type. The fourth is the required extension to a diagonal edge. twoOpposite: Two values are given where each define the extension for opposite sides. • If two extension values are given, the first value applies in one direction (horizontal or vertical), the second value applies in the orthogonal direction. • If four extension values are given, the first and the second values apply in one direction (horizontal or vertical), the third and the fourth value apply in the orthogonal direction. oneOneTwoAdjacent: Three values are given where the first value is for two adjacent sides and the second and third values are for the remaining sides. longShort: Two values are given where the first value is the extension for the long side and the second value is for the short side of a rectangular shape. • If two extension values are given, the first value is the extension for the long sides and the second value is for the short sides of a rectangular shape. • If four extension values are given, the first and the second values apply to the long sides, the third and the fourth values apply to the short sides. horizontalVertical: The first value of the extension applies to the horizontal direction, the second value of extension applies to the vertical direction. • If two extension values are given, the first value applies in horizontal direction, the second value applies in vertical direction. • If four extension values are given, the first and the second values apply in horizontal direction, the third and the fourth value apply in vertical direction. verticalHorizontal: The first value of the extension applies to the vertical direction, the second value of the extension applies to the horizontal direction. • If two extension values are given, the first value applies in vertical direction, the second value applies in horizontal direction. • If four extension values are given, the first and the second values apply in vertical direction, the third and the fourth value apply in horizontal direction.
-widthRange	The constraint applies only if the cover material width is in this range. To specify the ranges, see Range Data Syntax .
-cutClass	List of paired float values or a list of cutClass names. See cutClass and cutClassList . The constraint applies only in cases where the cuts comply with these specifications.
-minLength	Minimum wire length.

Argument	Description
-otherLayer	The layer with which the cut shape must interact for the constraint to apply. <code>-otherLayer</code> needs to be specified along with <code>-otherColor</code> .
-otherLayerWidth	The width of the <code>otherLayer</code> . The width must be within the specified range for the constraint to apply. To specify the ranges, see Range Data Syntax .
-otherColor	The color of the <code>otherLayer</code> for this constraint to apply. The color name must be defined in the Controls Section . Default: any
-color	The constraint applies only if the cover material color matches. A value of zero means any color. Default is any.
-enclosingShapeDirection	If given, the constraint applies only to wires (enclosing shapes on layer1) oriented in this direction. Valid Values: 0 (any), 1 (horizontal direction), and 2 (vertical direction). Default: any
exceptCutDistanceRange	If specified, the constraint applies only if there is no cut nearby such that the distance between the cuts lies in the given range in database units. This range must have a finite upper bound.
cutDistanceRange	If specified, the constraint applies only if there is another cut nearby and the distance between the cuts lies in the given range in database units. This range must have a finite upper bound.
cutPRLRange	Specifies an additional condition to be used with the <code>exceptCutDistanceRange</code> and <code>cutDistanceRange</code> parameters. Only nearby cuts with parallel run length in the given range in database units are considered. Can be specified only if the <code>exceptCutDistanceRange</code> or <code>cutDistanceRange</code> parameter is also given.

Examples

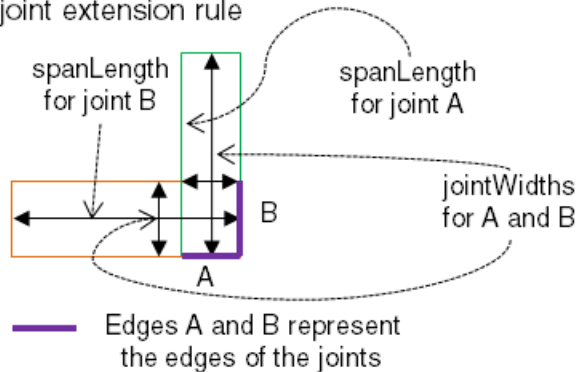
```
//specified with one cutClass
minViaExtension m1 v1 (0.06 0.08 0.06)
    -viaExtensionType oneOneTwo
    -widthRange(>=0.09 <=0.15)
    -cutClass 0.06 0.08 // or -cutClass via12
    -otherLayer poly
    -otherLayerWidth(0.10 0.20)
    -cutDistanceRange (0.07 0.08)
    -cutPRLRange (0.10 0.20)

//specified with multiple cutClass
minViaExtension m1 v1 (0.06 0.08 0.06)
    -viaExtensionType oneOneTwo
    -widthRange(>=0.09 <=0.15)
    -cutClass((0.06 0.06) (0.06 0.08))//or -cutClass(V12A V12B)
    -otherLayer poly
    -otherLayerWidth(0.10 0.20)
```

minViaJointExtension

Some processes require cut shapes must have different enclosure for joint edges than for regular edges. These rules are specific to a given cut class:

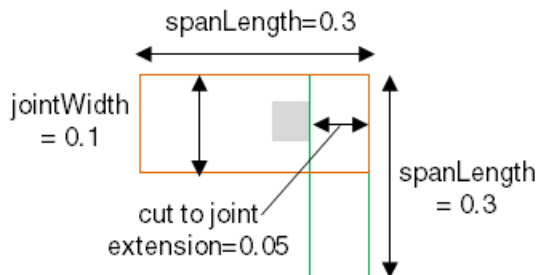
Illustration of jointWidth and spanLength parameters used to identify a joint for via to joint extension rule



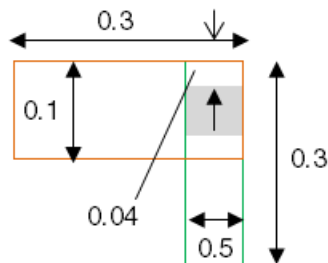
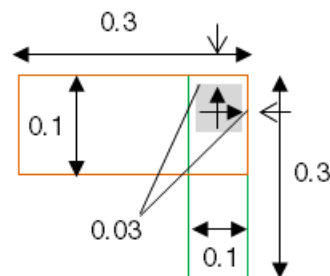
Example of minJointExtension dualIntValue = {0.5 0.3} (either one overhang ≥ 0.5 or both ≥ 0.3)

jointWidth parameter = 0.15, spanLength parameter = 0.1
(example given is user units, not DBU, for clarity)

a) OK, having 0.05 overhang to one of the joint edges



b) OK, having 0.03 overhang to both of the joint edges



c) Violation, having 0.04 and 0.0 overhang to the joints would not meet either of the requirements. Must have one overhang of 0.5 or two of 0.3

Constraint Category

Extension

Constraint Type	Complex
OA Rule	oacMinViaJointExtension
Layer Symmetry	Asymmetric

Syntax:

```
minViaJointExtension
  <string|LPP>metalLayer
  <string|LPP>cutLayer
  [-cutClass <cutClassArray | cutClassNameList>]
  -jointWidth float
  -spanLength float [-eolMinLength float]
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

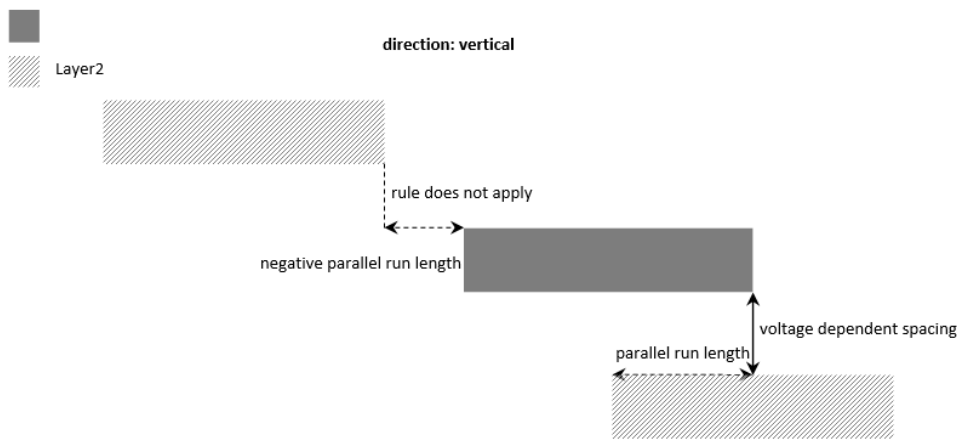
Argument	Description
metalLayer	metalLayer can be defined as a string or LPP.
cutLayer	cutLayer can be defined as a string or LPP.
-cutClass	List of paired float values or a list of cutClass names. See cutClass and cutClassList . The constraint applies only in cases where the cuts comply with these specifications.
-jointWidth	Specifies the width of a joint. This is required.
-spanLength	Used to identify a joint.
-eolMinLength	Indicates that the joint must be an end of line edge with length >= this value along both sides.

For example:

```
minViaJointExtension m1 v2 0.06 -cutClass 0.06 0.08
                                -jointWidth 0.3
                                -spanLength 0.5
```


minVoltageClearance

Some processes require different minimum clearance values in horizontal and vertical directions depending on the voltage difference between shapes on two different layers. This rule is modeled by built-in OA constraint `minVoltageClearance` with new parameter `-spacingDirection`. By default, clearance is measured as Euclidian, and corner-to-corner clearance is enforced. However, if only horizontal or vertical direction is specified, the clearance is enforced only when the shapes are facing each other, that is, the parallel run length (in the orthogonal direction) between the shapes is greater than or equal to zero.



The constraint is used when you check for voltage-dependent spacing violations as you edit the design layout. The Custom Router also uses the constraint to determine violations.

Constraint Category	Clearance
Constraint Type	Complex
OA Rule	oacMinVoltageClearance
Layer Symmetry	Symmetric

Syntax:

```
minVoltageClearance
  <string|LPP>layer1
  <string|LPP>layer2
  <float1DTable>clearanceValue
  [-spacingDirection any|horizontal|vertical]
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

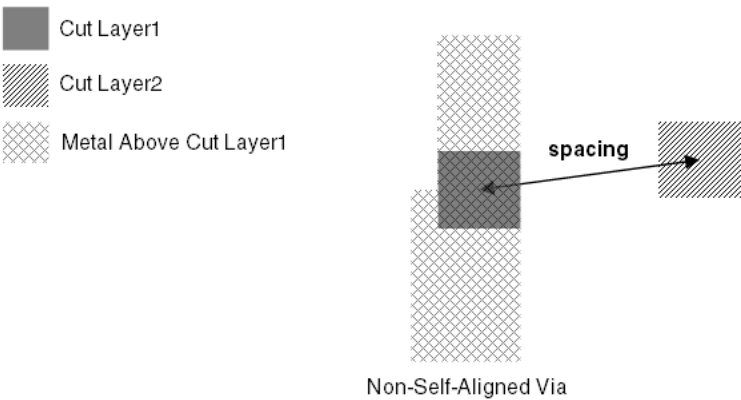
Argument	Description
layer1	layer1 can be defined as a string or LPP.
layer2	layer2 can be defined as a string or LPP.
clearanceValue	The constraint value consists of float-float pairs. The first float represents the voltage. The second float represents the clearance. The voltage index represents the maximum voltage difference between the two shapes. The spacingValue is represented as: (<float>voltageValue <float>clearanceValue) The voltage value has a >= semantic, that specifies the required clearance which applies to all voltage differences greater than or equal to it.
-spacingDirection	Defines the spacing direction in which this constraint applies: only horizontal (1), only vertical (2) or any (0). Default: any

For example:

```
minVoltageClearance m1 v1 ((1.5 0.1) (1.8 0.2)) -spacingDirection vertical
// If the voltage >=1.5 then spacing must be >= 0.1, the voltage >=1.8 then
   spacing must be >= 0.2.
```

nonSelfAlignedViaClearance

Some processes have different spacing requirements for cut shapes in vias which are not self-aligned. The nonSelfAlignedViaClearance constraint requires a minimum center-to-center spacing between a non-self-aligned via cut and another cut on a different cut layer. The neighboring via can be either a self-aligned via or a non-self-aligned via. The constraint applies only if the vias belong to different nets. The required minimum center-to-center spacing is more restrictive than spacing allowed for self-aligned vias.



Constraint Category	Spacing
---------------------	---------

Constraint Type	Complex
OA Rule	Custom Compiler Custom Layer Pair Constraint
Layer Symmetry	Asymmetric

Syntax:

```
nonSelfAlignedViaClearance
  <string|LPP>layer1
  <string|LPP>layer2
  <float>spacingValue
  [-cutClass float1 float2|<string>cutClassName]
  [-cutClass2 float1 float2|<string>cutClassName]
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

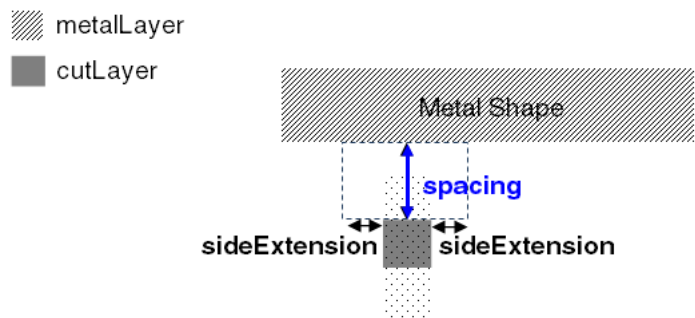
Argument	Description
layer1	Layer on which cut1 is placed. <code>layer1</code> can be a string (layer name) or LPP.
layer2	Layer on which cut2 is placed. <code>layer2</code> can be a string (layer name) or LPP.
spacingValue	Minimum center-to-center distance, in microns, between the cuts.
-cutClass	(Optional) Cut class of the non-self-aligned via cut shape.
-cutClass2	(Optional) Cut class of the neighboring via cut shape.

Example:

```
nonSelfAlignedViaClearance L1 L2 0.75
                             -cutClass 0.04 0.06 -cutClass2 0.12 0.16
```

nonSelfAlignedViaEdgeToMetalSpacing

The `nonSelfAlignedViaEdgeToMetalSpacing` constraint requires a minimum distance between the edge of a cut in a self-aligned via and the edge of a metal shape. That distance must be maintained up to a distance (`sideExtension`) beyond both sides of the cut.



Constraint Category	Spacing
Constraint Type	Complex
OA Rule	Custom Compiler Custom Layer Pair Constraint
Layer Symmetry	Asymmetric

Syntax:

```
nonSelfAlignedViaEdgeToMetalSpacing
<string|LPP>metalLayer
<string|LPP>cutLayer
<float>spacingValue
[-sideExtension float]
[-isHard true|false]
[-comment [<string rule_id>] <string rule_desc>]
```

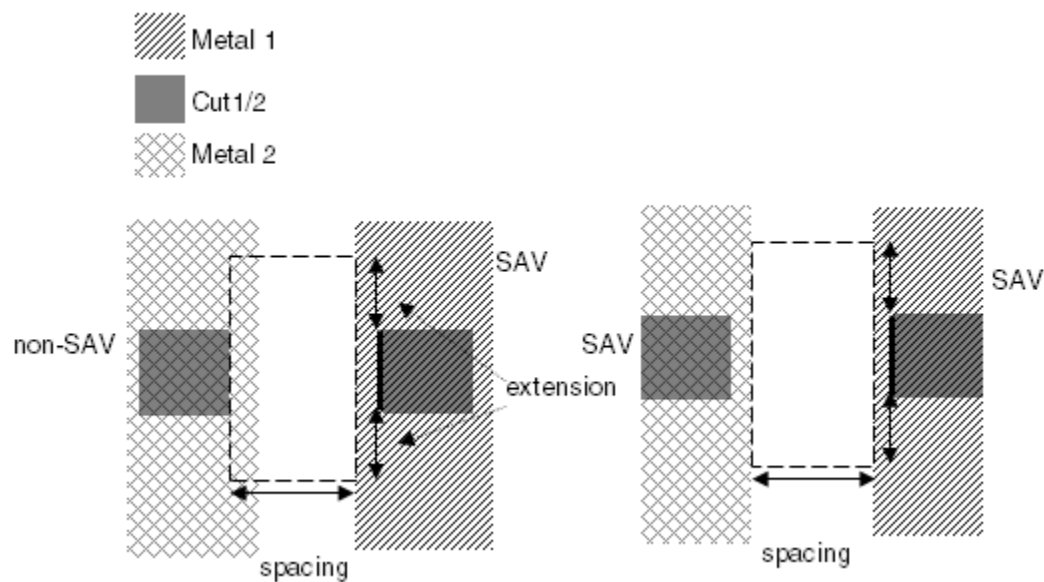
Argument	Description
metalLayer	metalLayer can be a string (layer name) or LPP.
cutLayer	cutLayer can be a string (layer name) or LPP.
spacingValue	Minimum distance, in microns, between the edge of the cut to the edge of the metal shape.
-sideExtension	Distance beyond the sides of the cut. The spacing must be maintained up to sideExtension on both sides of the cut.

Example:

```
nonSelfAlignedViaEdgeToMetalSpacing M1 V1 0.02
-sideExtension 0.23
```

selfAlignedViaEdgeClearance

Advanced processes require a minimum spacing from the lower layer edge from a SAV to a neighbor shape on the upper metal layer if an SAV cut enclosed by shape has parallel overlap by the spacing region. The spacing is required along the extended SAV cut edge. If the neighbor shape edge is a non-SAV edge, the spacing requirement is to the cut edge.



Constraint Category	Via Construction
Constraint Type	Complex
OA Rule	Custom Compiler custom layer pair constraint
Layer Symmetry	Asymmetric

Syntax:

```
selfAlignedViaEdgeClearance
<string|LPP>layer1
<string|LPP>layer2
[-extension float]
[-cutClass float1 float2|<string>cutClassName]
[-cutClass2 float1 float2|<string>cutClassName]
[-isHard true|false]
[-comment [<string rule_id>] <string rule_desc>]
```

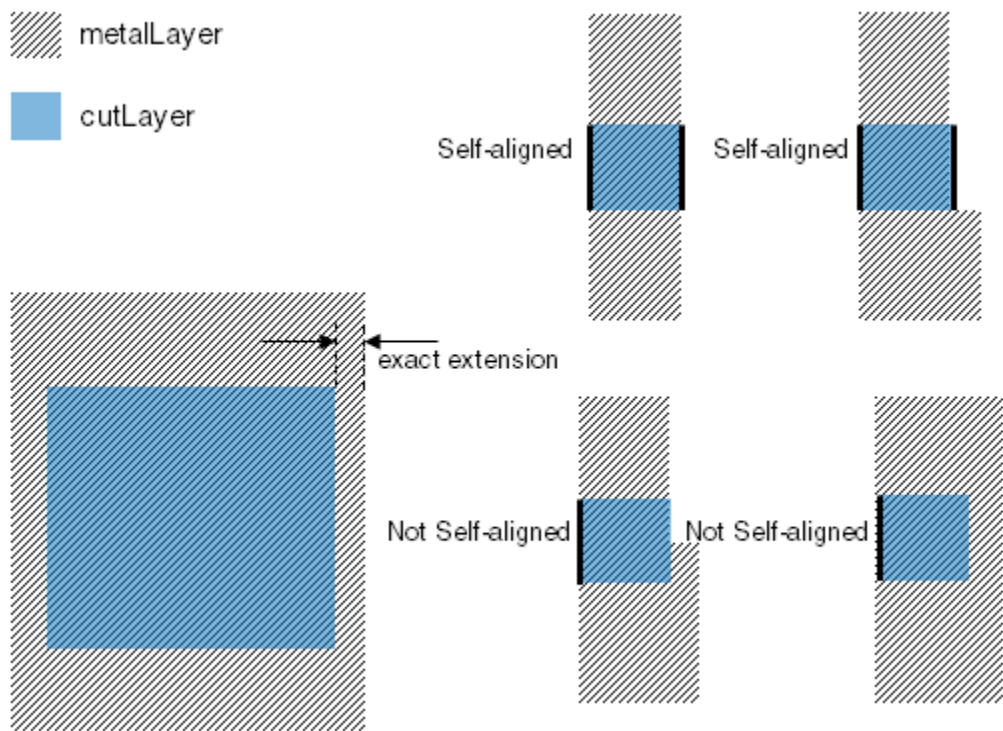
Argument	Description
layer1	layer1 can be defined as a string or LPP.
layer2	layer2 can be defined as a string or LPP.
spacingValue	Minimum spacing between layer1 SAV upper metal edge and layer2 SAV upper metal edge or layer2 non-SAV edge.
-extension	Defines the amount of extension to apply to the cut edge for determining the spacing region. The default value is 0.
-cutClass	If given, only layer1 shapes matching this geometry apply. Note: If <code>cutClass</code> parameter is specified with <code>cutClassName</code> in the ASCII file, the tech importer looks up the first <code>cutClass</code> constraint specified in the same hierarchical constraint group to convert to cut class size; if no <code>cutClass</code> constraint exists; an error message is displayed. While exporting, the tech exporter looks up the first <code>cutClass</code> constraint specified in the same hierarchical constraint group to exists; convert the cut class size to cut class name. If no <code>cutClass</code> constraint exists; the <code>cutClass</code> parameter is exported with cut class size in dual value.
-cutClass2	If given, only layer2 shapes matching this geometry apply. Note: If <code>cutClass</code> parameter is specified with <code>cutClassName</code> in the ASCII file, the tech importer looks up the first <code>cutClass</code> constraint specified in the same hierarchical constraint group to convert to cut class size; if no <code>cutClass</code> constraint exists; an error message is displayed. While exporting, the tech exporter looks up the first <code>cutClass</code> constraint specified in the same hierarchical constraint group to exists; convert the cut class size to cut class name. If no <code>cutClass</code> constraint exists; the <code>cutClass</code> parameter is exported with cut class size in dual value.

For example:

```
selfAlignedViaEdgeClearance L1 L2 0.25 -extension 0.15
                                     -cutClass 0.4 0.6
                                     -cutClass2 0.3 0.7
```

selfAlignedViaExtension

The `selfAlignedViaExtension` constraint defines an array of exact opposite extensions that identify a cut shape as being a self-aligned via.



This constraint is based upon whether the edge has an exact overlap or extension from an upper metal shape edge. An edge is aligned if the upper metal shape edge has one of the given exact extension values. The enclosing edge from the upper metal layer must also extend to the end of the cut edge to be considered an aligned edge. If two opposite edges of a cut shape are considered aligned, then the cut shape is considered self-aligned. If a cut shape with a single aligned edge is considered to be a violation.

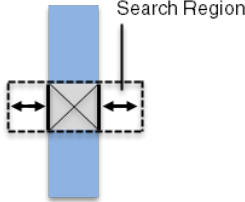
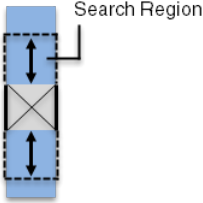
Constraint Category	Via Construction
Constraint Type	Complex
OA Rule	Custom Compiler custom layer pair constraint
Layer Symmetry	Asymmetric

Syntax:

```
selfAlignedViaExtension
<string|LPP>metalLayer
<string|LPP>cutLayer
<float|dualValue|strictRangeValue>value
[-cutClass <cutClassArray | cutClassNameList>]
[-singleEdgeForbidden true|false]
[-singleSided true|false]
[-alignedEdgeUpsize float]
```

```
[-orthoEdgeUpsize float]
[-edgeGridX float]
[-edgeGridY float]
[-edgeXUpsize float]
[-edgeYUpsize float]
[-isHard true|false]
[-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description								
metalLayer	metalLayer can be defined as a string or LPP.								
cutLayer	cutLayer can be defined as a string or LPP.								
value	Defines an array of exact opposite extensions that identify a cut shape as being a self aligned via. Valid value types for the array are <code>oaIntValue</code>, <code>oaDualIntValue</code> and <code>oaIntRangeValue</code>. <code>oaIntValue</code>: Represents an exact extension applicable to either opposite side. <code>floatValue</code> will be stored as <code>oaIntValue</code> <code>oaDualIntValue</code>: Represents exact extensions for each opposite side. <code>dualValue</code> will be stored as <code>oaDualIntValue</code> <code>oaIntRangeValue</code>: Represents a range of extension values applicable to either opposite side. <code>strictRangeData</code> will be stored as <code>oaIntRangeValue</code>. The syntax is similar to the range syntax format Range Data Syntax but with a restriction that range sign (<code>></code>, <code><</code>, <code>>=</code>, <code><=</code>) must be fully specified. If range sign is not fully specified for range value, tech importer will report errors and stop importing. Following table shows some valid/invalid examples. <table border="1"> <thead> <tr> <th>Valid Range</th><th>Invalid Range</th></tr> </thead> <tbody> <tr> <td><code>(>0.7 <=0.8)</code></td><td><code>(0.7 <=0.8)</code></td></tr> <tr> <td><code>(>=0.7 <0.8)</code></td><td><code>(>=0.7 0.8)</code></td></tr> <tr> <td><code>(>0.9 INFINITY)</code></td><td><code>(0.9 INFINITY)</code></td></tr> </tbody> </table>	Valid Range	Invalid Range	<code>(>0.7 <=0.8)</code>	<code>(0.7 <=0.8)</code>	<code>(>=0.7 <0.8)</code>	<code>(>=0.7 0.8)</code>	<code>(>0.9 INFINITY)</code>	<code>(0.9 INFINITY)</code>
Valid Range	Invalid Range								
<code>(>0.7 <=0.8)</code>	<code>(0.7 <=0.8)</code>								
<code>(>=0.7 <0.8)</code>	<code>(>=0.7 0.8)</code>								
<code>(>0.9 INFINITY)</code>	<code>(0.9 INFINITY)</code>								
-cutClass	List of paired float values or a list of cutClass names. See cutClass and cutClassList . The constraint applies only in cases where the cuts comply with these specifications.								
-singleEdgeForbidden	Indicates whether a single edge matching an extension is allowed. If true, then a case where just one of the sides matches one of the exact extensions is a violation. The default is false.								
-singleSided	When it is true, an extension matching on one side is also applicable. This applies to all the <code>oaIntValue</code> from the <code>oaValueArrayValue</code> .								

Argument	Description
-alignedEdgeUpsize	(Optional) Specifies the distance from the self-aligned edge and opposite edge to the boundary of the cluster search region. 
-orthoEdgeUpsize	(Optional) Specifies the distance from non-self-aligned edges to the boundary of the cluster search region. 
-edgeGridX	Defines the x-axis grid that is used to determine whether the upsized via is on grid. If the x-axis of the edges are a multiple of the edgeGridX value, then the via is on grid, and it is upsized again by edgeXUpsize.
-edgeGridY	Defines the y-axis grid that is used to determine if the upsized via is on grid. If the y-axis of the edges are a multiple of the edgeGridY value, then the via is on grid, and it is upsized again by edgeYUpsize.
-edgeXUpsize	Adds the specified distance along the an x-axis to on-grid vias.
-edgeYUpsize	Adds the specified distance along the an y-axis to on-grid vias.

Note:

Tech importer reports error when `-singleEdgeForbidden` and `-singleSided` are set to true at the same time.

Examples:

```
selfAlignedViaExtension    m1 v1 (0.10 (0.08 0.06))
                           -cutClass ((0.06 0.08) (0.06 0.08))
                           -singleEdgeForbidden true

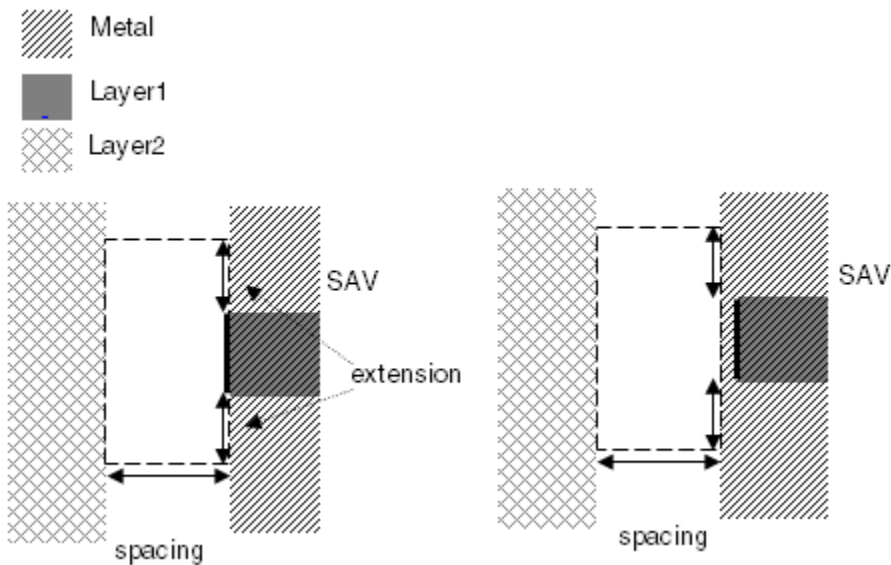
selfAlignedViaExtension    m1 v1 ( (0.10 0.12) (0.14 0.16) (0.17 0.18))
                           -cutClass 0.06 0.08
                           -singleEdgeForbidden true
```

In the following example, 0.1 will be stored as `oaIntValue`, (0.14, 0.16) will be stored as `oaDualIntValue`, ($\geq 0.17 \leq 0.18$) will be stored as `oaIntRangeValue`.

```
selfAlignedViaExtension    m1 v1 ( 0.1 (0.14 0.16) (>=0.17 <=0.18))
                           -cutClass 0.06 0.08
                           -singleEdgeForbidden    true
```

selfAlignedViaMetalClearance

Advanced processes require a minimum spacing from the upper layer edge from a SAV to a neighbor shape on the lower metal layer. The spacing is required along the extended SAV cut edge.



Constraint Category	Via Construction
Constraint Type	Complex
OA Rule	Custom Compiler Custom layer pair constraint
Layer Symmetry	Asymmetric

Syntax:

```
selfAlignedViaMetalClearance
<string|LPP>layer1
<string|LPP>layer2
[<float>spacingValue]
[-extension float]
[-isHard true|false]
[-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer1	layer1 can be defined as a string or LPP.
layer2	layer2 can be defined as a string or LPP.
spacingValue	Minimum spacing between the upper metal edge from a SAV from layer1 to a parallel edge from a shape on layer2. The spacing is required along the edge in addition to a given extension on both sides of the SAV cut edge.
-extension	Defines the amount of extension to apply to the cut edge for determining the spacing region. The default value is 0.

For example:

```
selfAlignedViaMetalClearance L1 L2 0.25 -extension 0.15
```

vertexInsideNotAllowed

This layer pair constraint specifies that only vertices completely inside the shape are not allowed. A vertex can lie on the edge or vertex of the shape of layer2.

Constraint Category	Miscellaneous
Constraint Type	Complex
OA Rule	Custom Compiler custom layer pair constraint
Layer Symmetry	Asymmetric

Syntax:

```
vertexInsideNotAllowed
  <string|LPP>layer1
  <string|LPP>layer2
  <bool>allowedValue
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer1	The layer which overlaps the layer2. layer1 can be defined as a string or LPP
layer2	The layer2 is overlapped by layer1. layer2 can be defined as a string or LPP
allowedValue	If true, shapes on the layer1 must not have a vertex inside a shape on layer2.

For example,

```
vertexInsideNotAllowed L1 L2 true
```

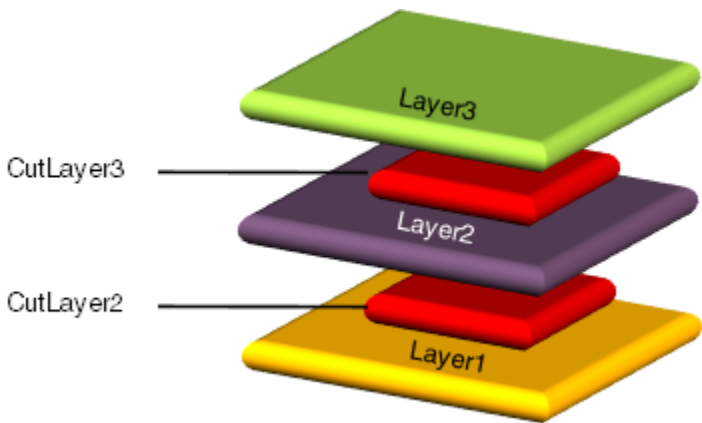
viaStackingAllowed

This layer pair constraint specifies whether stacking of two cut layers is allowed.

Note:

A via is considered to be in a stack with another via if the cut of one via overlaps with any part of the cut of the other via.

This constraint definition is symmetric. This indicates that when via stacking is allowed for layer1and layer2, it is also allowed for layer2 and layer1.



Constraint Category	Via Construction
Constraint Type	Simple
OA Rule	oacViaStackingAllowed
Layer Symmetry	Symmetric

Syntax:

```
viaStackingAllowed
  <string|LPP>layer1
  <string|LPP>layer2
  <bool>allowedValue
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer1	The layer which overlaps the layer2. layer1 can be defined as a string or LPP.
layer2	The layer2 is overlapped by layer1. layer2 can be defined as a string or LPP.
allowedValue	Specifies whether stacking of two cut layers is allowed.

In this example, the VIA1 and VIA2 are allowed to be stacked:

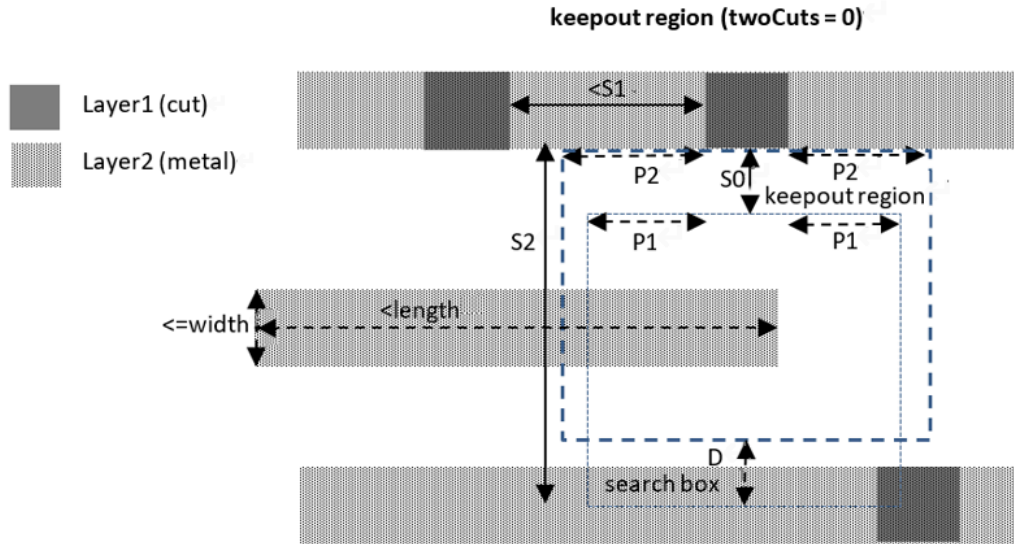
Rule	Description	Value (in microns)
VIA2.R.1	VIA1 allowed stacking	N/A

```
viaStackingAllowed VIA1 VIA2 true
```

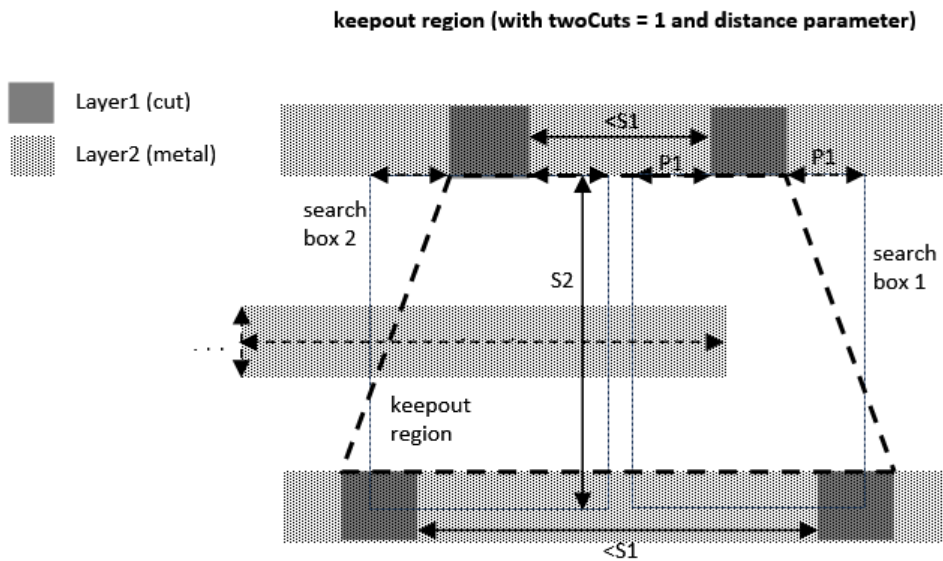
viaSurroundingMetalKeepout

Some processes do not allow metal shapes to go in a specific region defined by three or four via cuts. The keepout region can be formed in many different ways depending on number of via cuts and values of the constraint parameters. The rule considers one or two pairs of via cuts fully aligned in the preferred routing direction of the metal layer when distance between the cuts is less than the rule value (S1).

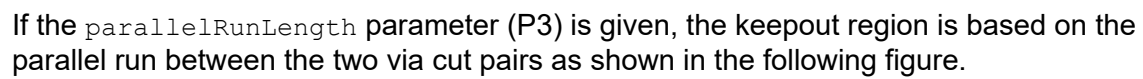
In the case of three via cuts (one pair of aligned cuts), the `twoCuts` parameter value is set to 0, and the search for the third via cut is based on the rule value (S2) and the value of distance parameter (P1). The keepout region is formed using values of `enclosedDistance` (D) and `otherExtension` (P2) parameters.



In the case of four cuts (two pairs of aligned cuts), the `twoCuts` parameter value is set to 1, and the search for the second via pair is based on the rule value ($S2$) and the value of distance parameter ($P1$) or the value of `parallelRunLength` parameter ($P3$). If the distance parameter ($P1$) is given, the keepout region is a trapezoid formed by the via corners as shown in the following figure.

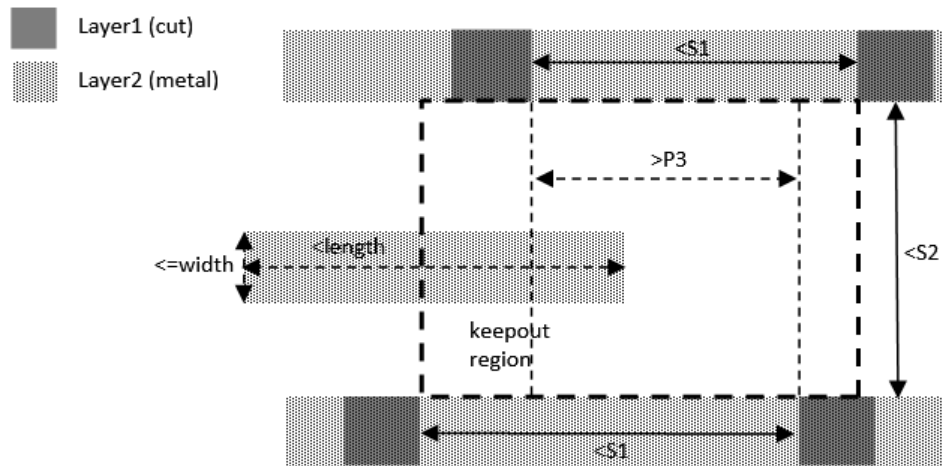


Optionally, the rule can be relaxed by forming a smaller keepout trapezoid when the `checkInnerEdge` parameter value is true.



Optionally, the rectangular keepout region can be extended as shown in the following figure when the `extendProjection` parameter value is true.

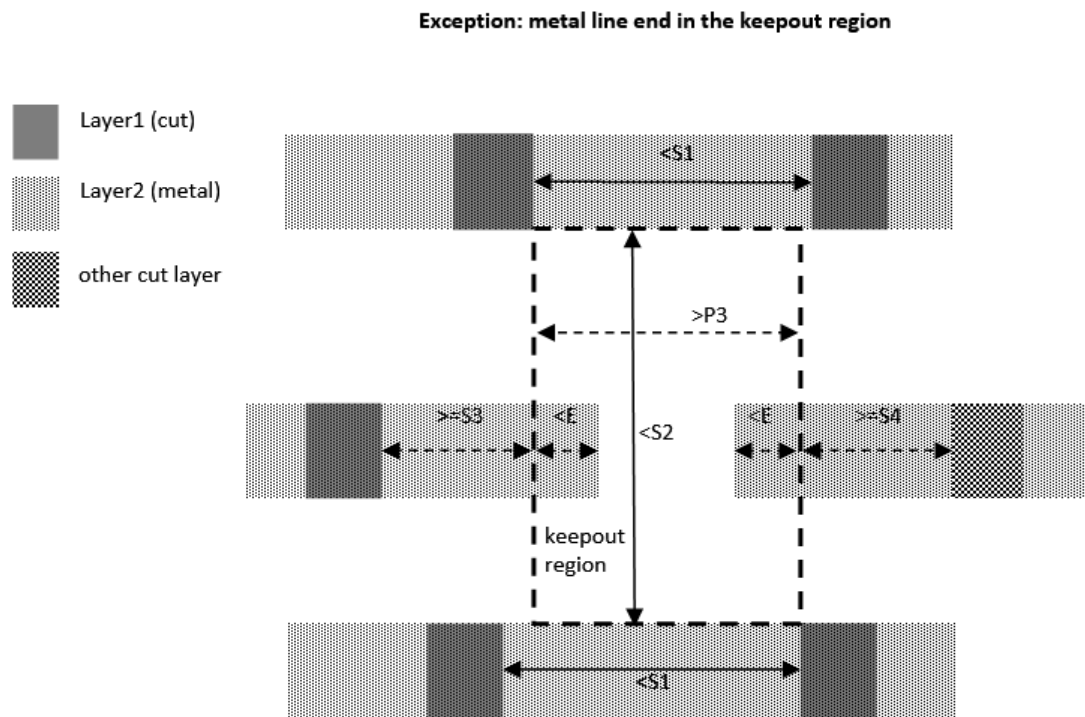
keepout region (with twoCuts = 1, parallelRunLength parameter and extendProjection = true)



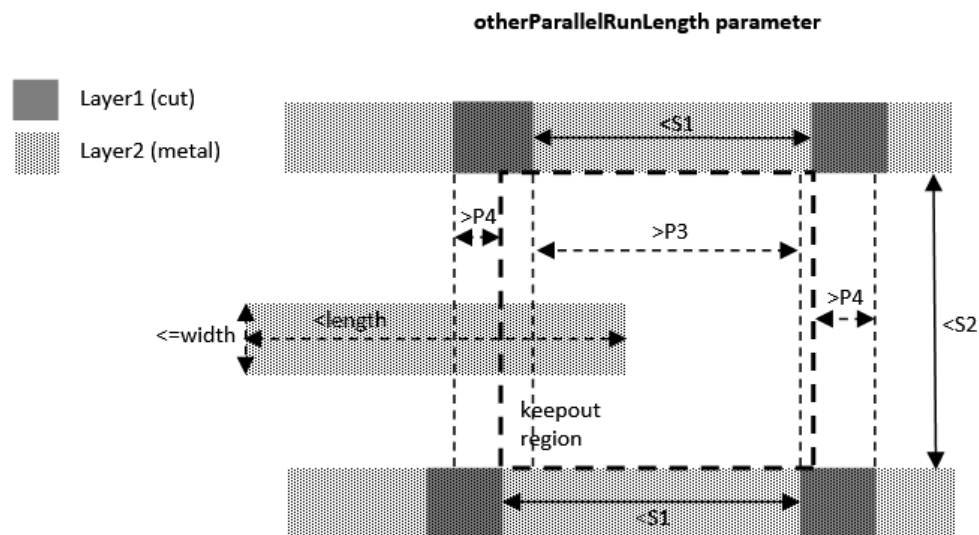
There are also exceptions from the rule depending on complex geometrical conditions.

Metal line end in the keepout region doesn't violate the rule if the following conditions are met:

- The line end overlap with the keepout region is less than a particular threshold (E).
- Via shapes on the cut layer1 do not overlap or touch the keepout region.
- Spacing between via shapes on the cut layer1 and the keepout region is greater than or equal to a particular threshold (S3).
- Via shapes on the other cut layer (above or below the metal layer2) don't overlap or touch the keepout region.
- Spacing between via shapes on the other cut layer and the keepout region is greater than or equal to a particular threshold (S4).



The rule can also be waived depending on the parallel run length between corresponding via cuts from different aligned cut pairs. If the `otherParallelRunLength` parameter is given, the parallel run length must be greater than the parameter value (P4) for the constraint to apply.



Constraint Category	Spacing
---------------------	---------

Constraint Type	Complex
OA Rule	Custom Compiler custom layer pair constraint
Layer Symmetry	Asymmetric

Syntax:

```
viaSurroundingMetalKeepout
<string|LPP>cutLayer
<string|LPP>metalLayer
<float1 float2>spacing1 spacing2
-width float
[-distance float][-parallelRunLength floatValue [-extendProjection
booleanValue]]
[-enclosedDistance float]
[-twoCuts int]
[-otherExtension float]
[-exceptDiffSegs true|false]
[-checkInnerEdge true|false]
[-exceptConnectedRange rangeData]
[-length floatValue]
[-otherParallelRunLength floatValue]
[-extension floatValue [-cutDistance floatValue][-otherCutDistance
floatValue]]
[-minDistanceWithin floatValue]
[-cutClass dualValue | cutClassName]

[-isHard true|false]
[-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
cutLayer	cutLayer can be defined as a string or LPP.
metalLayer	metalLayer can be defined as a string or LPP.
spacing1	Maximum distance (S1) between two consecutive vias along the same wire.
spacing2	Maximum distance (S2) between the edge of the wire that hosts the two consecutive vias and the centerline of the parallel wire that has one or two vias.
-width	Maximum wire width.
-distance	Distance (P1) between the edges of the via and the vertical boundary of search box. Default value of -distance is 0 and default value of -extendProjection is false.
-enclosedDistance	Distance (D) between the keepout region boundary and the search box boundary.
-twoCuts	Defines how the keepout region is formed. 0 means the keepout region is a rectangle defined by P2 and D. 1 means the keepout region is a trapezoid or a rectangle defined by the corner points of the four cut shapes.

Argument	Description
-otherExtension	Distance (P2) between the edge of the via and vertical boundary of keepout region.
-exceptDiffSeqs	If true, at least one pair of aligned vias must be on the same wire segment for the constraint to apply. A metal shape which is cut by a cut-metal shape constitutes two metal segments.
-checkInnerEdge	If true, adjusts the corners of the keepout region based on the parallel overlap of via edges. Default: false. This can only be used when the value of the <code>twoCuts</code> parameter is 1.
-exceptConnectedRange	Defines the range of widths for a metal shape connected to the middle shape. When a shape connected to the middle shape with a via that is also in the region has the correct width and has the correct <code>iccRouteType</code> , this constraint does not apply. See Range Data Syntax .
-length	If given, the rule only forbids metal wires with length less than this value in database units. If the wire length is greater than or equal to this value, the wire can go into the keepout region.
-minDistanceWithin	The distance (S0) between the search box and the metal shape enclosing the aligned via cuts. Default: 0
-parallelRunLength	If given, the search for nearby cut pairs is not based on the box extension by P1 from the aligned cut edge but on parallel run between two aligned via pairs. The parallel run length must be greater than this value in database units (P3) for the constraint to apply. This cannot be specified together with the <code>distance</code> parameter.
-extendProjection	If true, the keepout region is extended to the bounding box of inner corners of the four via cuts. This can only be specified together with the <code>parallelRunLength</code> parameter.
-otherParallelRunLength	If given, the parallel run length between corresponding via cuts from different aligned cut pairs must be greater than this value in database units (P4) for the constraint to apply.
-extension	If given, metal line end with overlap less than this value in database units (E) is not a violation if other conditions of the metal line end exception are met.
-cutDistance	If given, defines additional condition for metal line end exception: spacing from cut layer1 shapes to the keepout region must be greater than or equal this value in database units (S3). This can only be specified together with the <code>extension</code> parameter.
-otherCutDistance	If given, defines additional condition for metal line end exception: spacing from shapes on other cut layer (above or below the metal layer2) to the keepout region must be greater than or equal this value in database units (S4). This can only be specified together with the <code>extension</code> parameter.
-cutClass	If given, the constraint only applies if the aligned cuts are from this cut class. The third via cut can be from any cut class.

Note:

The warning message is displayed in the following situations:

- `-distance` and `-parallelRunLength` are specified together.
- `-extendProjection` is specified but `-parallelRunLength` is not specified.
- `-cutDistance` is specified but `-extension` is not specified.
- `-otherCutDistance` is specified but `-extension` is not specified.
- `-checkInnerEdge` is set to `true` but `-twoCuts` is not set to 1.
- `-twoCuts` is larger than 1.

Example:

```
viaSurroundingMetalKeepout L1 L2 0.75 0.97
                             -width 0.25 -enclosedDistance 0.4
                             -distance 0.45 -twoCuts 0
                             -cutClass 0.4 0.6 -minDistanceWithin 0.06
```

9

Layer Array Constraints

Describes how constraints are used to define design rules and provides the syntax with examples for all the supported constraints.

About Layer Array Constraints

Defined for a layer array, simple layer array constraints take a single value or a preferred value.

General Syntax

```
ruleName layers|LPPs value
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Value	Description
layers	stringArrayValue	Specify the name of layers.
LPPs	stringArrayValue	Specify the name of layers and their purposes.
value	float int bool	Specify a value.

List of Layer Pair Constraints

The following table lists the layer pair constraints you can define in the technology. Click a constraint name to view the details. For general information about technology constraints, see [Constraints Overview](#).

Constraint / ruleName	Constraint Category	Constraint Type
dummyPolyExtension	Extension	Complex
interLayerExtension	Extension	Complex
maxPolyLength	Length and Width Process Rules	Complex
minClearanceOverLayer	Clearance	Simple
minInnerVertexSpacing	Spacing	Complex

Constraint / ruleName	Constraint Category	Constraint Type
minOneDArrayStructure	Miscellaneous	Complex
minParallelViaClearance	Via Construction	Simple
minTouchingDirectionClearance	Clearance	Complex
minTouchingDirectionExtension	Extension	Complex
validRoutingLPPs	Routing	Complex

dummyPolyExtension

This layer array constraint specifies the required extension of dummy poly shapes beyond the adjacent shapes on the active layer.

The constraint applies only if no gate is already joined to another gate above or below.

The first layer is the gate layer, the second is the dummy gate layer and the third is the active layer. This constraint uses a parameter to specify the maximum ghost poly width at which it applies.

Constraint Category	Extension
Constraint Type	Complex
OA Rule	oacDummyPolyExtension

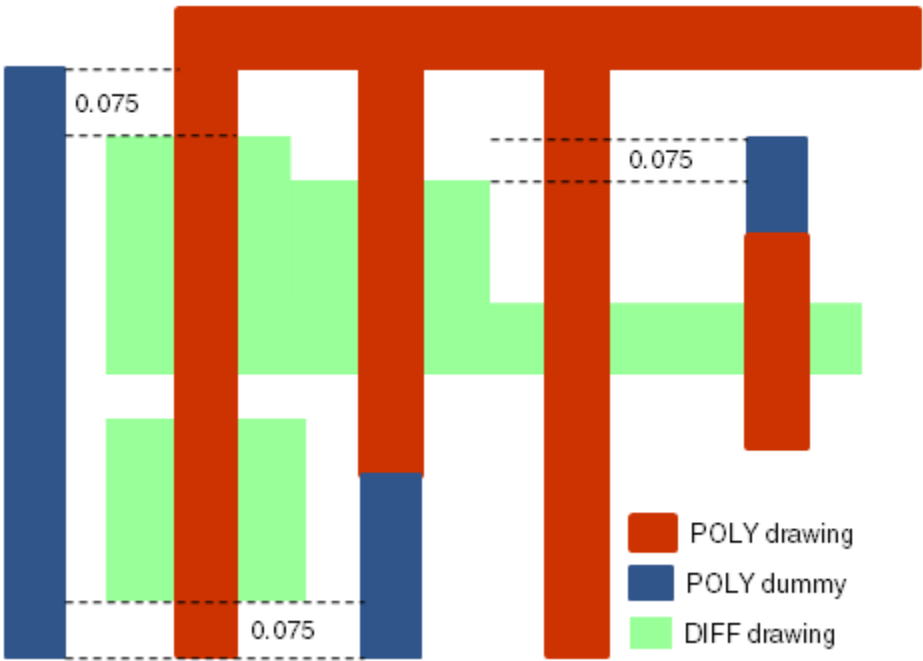
Syntax:

```
dummyPolyExtension
<List of strings|LPPs>layers
<float>polyExtension
[-width float]
[-isHard true|false]
[-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
Layers	Layers can be defined as a list of strings or a LPPs.
polyExtension	Specifies the minimum extension from active to dummy poly layer.
-width	Specifies the maximum width that a poly can be extended.

In this example, the dummy poly must extend the adjacent gate by 0.075:

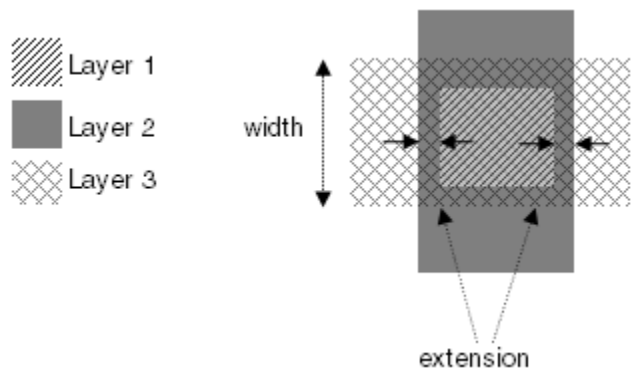
Rule	Description	Value (in microns)
PO.EX.2	Extension of dummy poly	≥ 0.075



```
dummyPolyExtension ((POLY drawing) (POLY dummy) (DIFF drawing)) 0.075
```

interLayerExtension

Advanced processes have enclosures dependency based on the width of the upper metal used over the cut. This rule requires three layers where layer1 is the enclosing layer beyond layer2 depending on the width of layer3 directly over the shape from layer2.



Constraint Category	Extension
Constraint Type	Complex
OA Rule	Custom Compiler Custom Constraint

Syntax:

```
interLayerExtension
  <List of strings|LPPs>layers
  <float>spacingValue
  -width float
  [-cutClass <cutClassArray | cutClassNameList>]
  [-proximityBox float float]
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

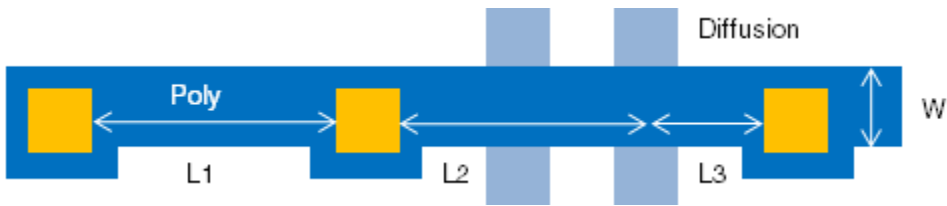
Argument	Description
layers	List of three layers or LPPs.
spacingValue	Minimum enclosure of layer1 beyond layer2 required over the layer3 shape which is greater than or equal to a certain width.
-width	Minimum width required for the layer3 shape. If the layer3 shape is less than this value, the constraint does not apply.
-cutClass	List of paired float values or a list of cutClass names. See cutClass and cutClassList . The constraint applies only in cases where the cuts comply with these specifications.
-proximityBox	(Optional) Region in which a shape on metal layer1 must be placed. Two values define the region: perpendicular distance from the cut edge, parallel distance from the cut corner.

For example:

```
interLayerExtension (m1 v1 m2) 0.8
                        -width 0.2
                        -cutClass 0.6 0.8
                        -proximityBox 0.25 0.88
```

maxPolyLength

The constraint defines the length of a run of poly material when the width of the run is less than a specific threshold.



Constraint Category	Length and Width process Rules
Constraint Type	Complex
OA Rule	oacMaxPolyLength

Syntax:

```
maxPolyLength
<List of strings|LPPs>layers
<float>spacingValue
[-width float]
[-isHard true|false]
[-comment [<string rule_id>] <string rule_desc>]
```

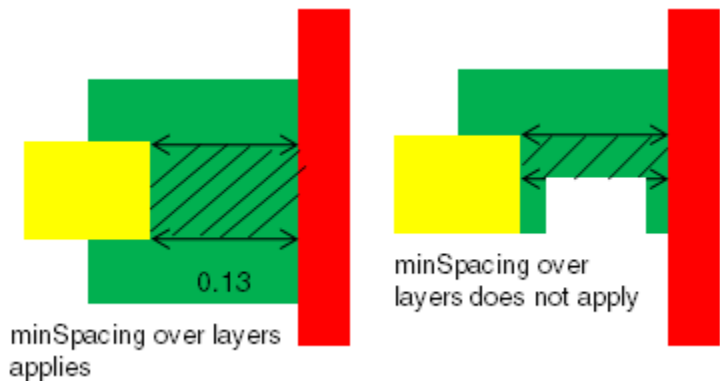
Argument	Description
layers	Specifies the list of layers. This list must contain three layers or LPPs.
spacingValue	The max length from active to dummy poly.
-width	Width threshold. Poly shapes less than this value do not apply.

For example:

```
maxPolyLength (poly dummy_poly diff) 0.8 -width 0.10
```

minClearanceOverLayer

This layer array constraint specifies a clearance between layer1 and layer2. This constraint applies when the space between the two layers is completely filled by another layer (layer3).



Constraint Category	Clearance
Constraint Type	Simple
OA Rule	oacMinClearanceOverLayer

Syntax:

```
minClearanceOverLayer
<List of strings|LPPs>layers
<float>spacingValue
[-isHard true|false]
[-comment [<string rule_id>] <string rule_desc>]
```

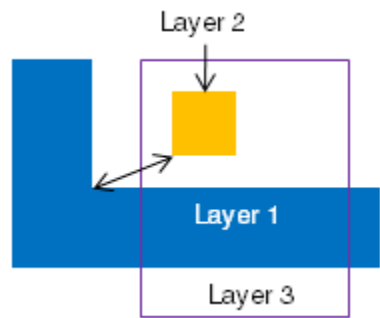
Argument	Description
layers	Specifies the list of layers. This list must contain three layers or LPPs.
spacingValue	Specifies the minimum spacing between shapes on layer1 and layer2 when the space between them is completely filled by layer3.

Rule	Description	Value (in microns)
PP.CL.1	PPLUS over DIFF space to GATE	0.13

```
minClearanceOverLayer (PPLUS GATE DIFF) 0.13
```

minInnerVertexSpacing

This constraint specifies the minimum spacing between contact shapes and shapes forming an inner vertex.



Constraint Category	Spacing
Constraint Type	Simple
OA Rule	oacMinInnerVertexSpacing

Syntax:

```
minInnerVertexSpacing
  <List of strings|LPPs>layers
  <float>spacingValue
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layers	Specifies the list of three layers. This list must contain three layers or LPPs.
spacingValue	Spacing between contact shapes and shapes forming an inner vertex.

For example:

```
minInnerVertexSpacing (PPLUS GATE DIFF) 0.23
```

minOneDArrayStructure

In some advanced processes, perfectly aligned cuts in a 1-D array in the preferred routing direction with more than a certain number of cuts have specific spacing requirements to each other and to other cuts both on the same layer and on different cut layers. They also

have enclosure rules that might differ from single or double cut vias on the same layer. The rules differ based on which cut class the cuts belong to.

Constraint Category	Miscellaneous
Constraint Type	Complex
OA Rule	oacMinOneDArrayStructure

Syntax:

```
minOneDArrayStructure
  <List of strings|LPPs>layers
  <float>structureValue
  [-cutClass <cutClassArray | cutClassNameList>]
  -numCuts int
  -arraySpace float
  -interViaSpace float
  -dualExtension float
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
Layers	layers can be defined as a list of strings or a LPPs.
structureValue	Specifies the required exact edge-to-edge spacing between cut shapes in a 1-D array.
-cutClass	List of paired float values or a list of cutClass names. See cutClass and cutClassList . The constraint applies only in cases where the cuts comply with these specifications.
-numCuts	Specifies the minimum number of cuts in perfectly aligned row in the preferred routing direction for this structure to qualify as a 1-D array.
-arraySpace	Specifies the spacing between this 1-D array and other cut classes, or another 1-D array if the parallel run length is > 0. The 1-D array should be merged into a single rectangular box (with a grow/shrink) before the check is applied
-interViaSpace	Specifies the spacing between this 1-D array and any different net cuts on all cut classes in the 3 rd layer specified in the constraint's layer array.
-dualExtension	Specifies the required extension of the metal layer specified in the 2 nd layer of this constraint beyond the cut layer (1 st layer in this constraint).

```
minOneDArrayStructure (v1 v2 m2) 0.8 -cutClass ((0.6 0.8) (0.6 0.8))
  -numCuts 4
  -arraySpace 0.5
  -interViaSpace 0.3
  -dualExtension 0.4 0.8
```

minParallelViaClearance

This layer array constraint specifies the clearance value between cut shapes on one layer in relation to cut shapes on an adjacent layer that have parallel edges with a positive overlap.

This constraint applies only when the cuts do not share the same metal shape on the layer between them.

This constraint is an `oaLayerArrayConstraint Type`, where the first two layers in the array are the cut layers, and the third is the layer between the two cut layers.

Constraint Category	Via Construction
Constraint Type	Simple
OA Rule	oacMinParallelViaClearance

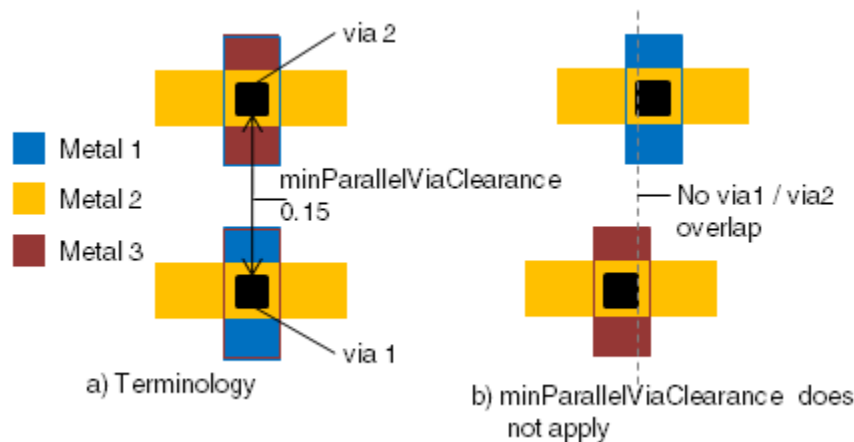
Syntax:

```
minParallelViaClearance
  <List of strings|LPPs>layers
  <float>viaClearance
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
Layers	layers can be defined as a list of strings or a LPPs.
viaClearance	Specifies the clearance value between cut shapes on one layer in relation to cut shapes on an adjacent layer that have parallel edges with a positive overlap.

In this example, the minimum VIA1 - VIA2 spacing is mentioned for VIAs on different net and having common run length > 0:

Rule	Description	Value (in microns)
VIA1.SP.5	VIA1 - VIA2 spacing for VIA1s on different net and when parallel run length > 0	0.15



minParallelViaSpacing (VIA1 VIA2 M2) 0.15

minTouchingDirectionClearance

This layer array constraint specifies a clearance between layer1 and layer2. This applies only in the direction normal to the layer1 edge that is touched by a third layer (layer3).

Touching can either be layer3 abutting layer1 or layer3 overlapping layer 1.

The clearance is between the layer1 edge that touches the third layer shape and adjacent layer2 shapes. The figure shows an illustration of how this clearance is defined with respect to the shapes on three layers.

Note:

If a dualInt is used to specify the clearance, the distance must be measured as Manhattan. In this case, the first number of the pair describes the required clearance normal to the touching edge, and the second number describes the required clearance as parallel to the touching edge.

Constraint Category	Clearance
Constraint Type	Complex
OA Rule	oacMinTouchingDirectionClearance

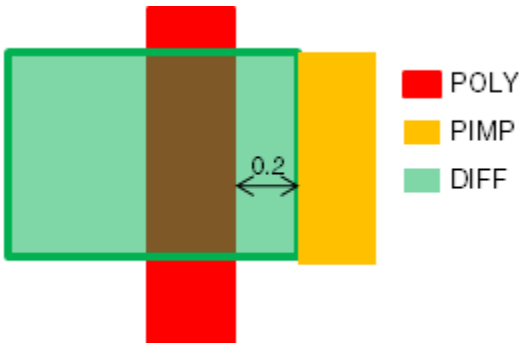
Syntax:

```
minTouchingDirectionClearance
<List of strings|LPPs>layers
<float|dualFloat>minClearance
<distanceMeasureType euclidean|manhattan>
[-isHard true|false]
[-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
Layers	layers can be defined as a list of strings or a LPPs.
minClearance	• float: Specifies the minimum clearance required distance between layer1 and layer2 in the direction perpendicular to the layer1 edge that is touching by a third layer. • dualFloat: Specifies that the first value is the clearance distance perpendicular between layer1 and layer2 shapes and the second value is total parallel to the touching edge.
-distanceMeasureType	Specifies if values are in Euclidian or Manhattan. Default value is "euclidean".

In this example, the Pimplant edge on diffusion spacing to nmos gate is mentioned:

Rule	Description	Value (in microns)
PIMP.SP.1	PIMP edge on DIFF spacing to GATE	0.2



`minTouchingDirectionClearance (GATE PIMP DIFF) 0.15`

minTouchingDirectionExtension

This layer array constraint specifies an enclosure for the field end of the gate. This enclosure is different from the enclosure for the channel.

The direction of field end enclosure depends on where the top and bottom edges of the gate touch poly. The direction of the channel enclosure depends on where the side edges of the gate touch diffusion.

This constraint is defined among three layers and specifies whether the enclosure must be measured in Euclidean or Manhattan distances. The enclosure direction is in the normal direction (at 90 degrees) to the perimeter edges of layer2 that are touching layer3 (where touching means the edges are either enclosed or abutting layer3), where layer3 extends beyond layer2 in the same direction.

Constraint Category	Extension
Constraint Type	Complex
OA Rule	oacMinTouchingDirectionExtension

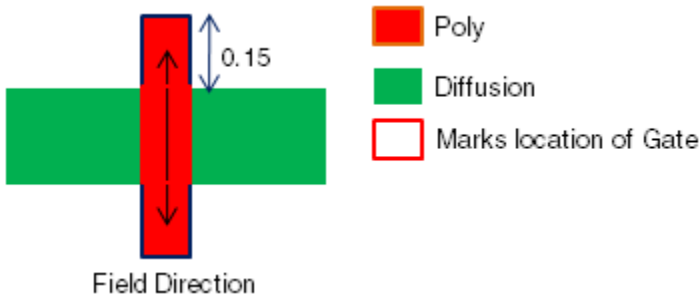
Syntax:

```
minTouchingDirectionExtension
  <List of strings|LPPs>layers
  <float>extension
  <-distanceMeasureType euclidian|manhattan>
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
Layers	Layers can be defined as a list of strings or a LPPs.
extension	Specifies the enclosure for the field end of the gate.
-distanceMeasureType	Specifies if values are in Euclidian or Manhattan. Default value is “euclidean”.

In this example, the POLY endcap extension over diffusion is specified:

Rule	Description	Value (in microns)
PO.EX.1	Endcap Extension of POLY over DIFF	0.15



```
minTouchingDirectionExtension (GATE DIFF POLY) 0.15
```


validRoutingLPPs

This constraint specifies a list of valid layer-purpose-pairs to be used in routing. It is intended to be used in a purpose-specific layerArray constraint. This implies that the constraint is created with an array of layers and an array of purposes.

Constraint Category	Routing
Constraint Type	Complex
OA Rule	oacValidRoutingLPPs

Syntax:

```
validRoutingLPPs  
  <List of strings|LPPs>layers  
  [-isHard true|false]  
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layers	Specifies the list of layers. This list must contain three layers or LPPs.

For example:

```
validRoutingLPPs((m1 drawing) (m2 drawing))
```

10

DRD Assist Constraints

Describes constraints for DRD Assist and provides the syntax with examples.

To allow to be configured with possible values for specific types of process rules, a set of preferred constraint definitions exists. These constraints are used only for DRD setup or configuration purposes.

The following table lists the DRD Assist constraints you can define in the technology. Click a constraint name to view the details. For general information about technology constraints, see [Constraints Overview](#).

Constraint / ruleName	Constraint Category
preferredClearance	Clearance
preferredExtension	Extension
preferredNotchSpacing	Spacing
preferredSpacing	Spacing
preferredSpanLength	Length and Width Process Rules

preferredClearance

This constraint specifies preferred clearance values between shapes on specified layers.

Syntax:

```
preferredClearance
  <string|LPP>layer1
  <string|LPP>layer2
    <arrayValue>clearanceValue
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer1	Name of the layer or LPP.

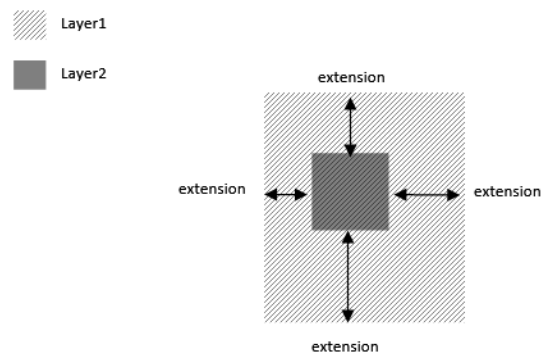
Argument	Description
layer2	Name of the layer or LPP.
clearanceValue	Specifies an array of preferred clearance values. Clearance between shapes must equal to the values in the array or is greater than or equal to the largest value in the array.

For example:

```
preferredClearance M1 M2 (0.01 0.03 0.05)
```

preferredExtension

This constraint specifies preferred extension values between shapes on specified layers.



Syntax:

```
preferredExtension
  <string|LPP>layer1
  <string|LPP>layer2
    <arrayValue>extensionValue
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

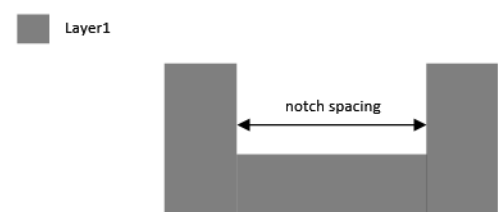
Argument	Description
layer1	Name of the layer or LPP.
layer2	Name of the layer or LPP.
extensionValue	Specifies an array of preferred extension values. Extension between shapes must equal to the values in the array or is greater than or equal to the largest value in the array.

For example:

```
preferredExtension M1 VIA1 (0.01 0.03 0.05)
```

preferredNotchSpacing

This constraint specifies preferred notch spacing values for the shape.



Syntax:

```
preferredNotchSpacing
    <string|LPP>layer
    <arrayValue>spacing
    [-isHard true|false]
    [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Name of the layer or LPP.
spacing	Specifies an array of preferred notch spacing values. Spacing between legs of a notch must equal to the values in the array or >= the largest value in the array.

For example:

```
preferredNotchSpacing m1 (0.01 0.03 0.05)
```

preferredSpacing

This constraint specifies preferred spacing values between shapes on the same layer.

Syntax:

```
preferredSpacing
    <string|LPP>layer
    <arrayValue>spacing
    [-isHard true|false]
    [-comment [<string rule_id>] <string rule_desc>]
```

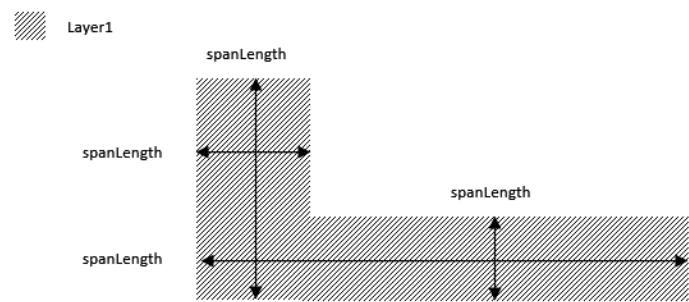
Argument	Description
layer	Name of the layer or LPP.
spacing	Specifies an array of preferred spacing values. Distance between shapes must equal to the values in the array or >= the largest value in the array.

For example:

```
preferredSpacing    m1 (0.01 0.03 0.05)
```

preferredSpanLength

This constraint specifies preferred span length inside the shape.



Syntax:

```
preferredSpanLength
  <string|LPP>layer
  <arrayValue>lengthValue
  [-isHard true|false]
  [-comment [<string rule_id>] <string rule_desc>]
```

Argument	Description
layer	Name of the layer or LPP.
lengthValue	Specifies an array of preferred span length values. Span length between edges inside the shape must equal to the values in the array or is greater than or equal to the largest value in the array.

For example:

```
preferredSpanLength    m1 (0.01 0.03 0.05)
```

11

Design Rule Verification Tcl

This chapter lists the vr:: Tcl commands that are used only in DRD rule deck files. Rule deck files are required for design rule verification and works with the db::checkDRD command. The dimension check filters and information are listed at the end of the chapter.

Note:

You cannot use the vr:: Tcl commands in the Console.

This chapter describes the following:

- [Commands](#)
- [Dimension Check Filters](#)
- [Information, Error, and Warning Messages](#)

See also [Sample Rule Deck File](#).

Commands

The commands covered are:

- [vr::acute](#)
- [vr::and](#)
- [vr::angle](#)
- [vr::area](#)
- [vr::attach](#)
- [vr::coinEdge](#)
- [vr::connect](#)
- [vr::convexEdge](#)
- [vr::copy](#)

- [vr::cut](#)
- [vr::density](#)
- [vr::donut](#)
- [vr::dfmText](#)
- [vr::dpt](#)
- [vr::exclude](#)
- [vr::extent](#)
- [vr::extents](#)
- [vr::extentCell](#)
- [vr::extendDrawn](#)
- [vr::expandEdge](#)
- [vr::expandText](#)
- [vr::enclose](#)
- [vr::encloseRectangle](#)
- [vr::enclosure](#)
- [vr::external](#)
- [vr::group](#)
- [vr::grow](#)
- [vr::holes](#)
- [vr::include](#)
- [vr::input](#)
- [vr::inside](#)
- [vr::insideCell](#)
- [vr::insideEdge](#)
- [vr::interact](#)
- [vr::internal](#)
- [vr::layer](#)

- [vr::layerMap](#)
- [vr::length](#)
- [vr::netAreaRatio](#)
- [vr::not](#)
- [vr::offGrid](#)
- [vr::or](#)
- [vr::orEdge](#)
- [vr::outside](#)
- [vr::outsideEdge](#)
- [vr::pathLength](#)
- [vr::perimeter](#)
- [vr::polygon](#)
- [vr::renameText](#)
- [vr::rectangle](#)
- [vr::rectangles](#)
- [vr::rectEnclosure](#)
- [vr::ruleSet](#)
- [vr::sconnect](#)
- [vr::selectRule](#)
- [vr::selectRulebyLayer](#)
- [vr::set](#)
- [vr::shift](#)
- [vr::shrink](#)
- [vr::size](#)
- [vr::skew](#)
- [vr::stamp](#)
- [vr::textLayer](#)

- [vr::textOrder](#)
- [vr::touch](#)
- [vr::touchEdge](#)
- [vr::tpt](#)
- [vr::unselectRule](#)
- [vr::unselectRuleByLayer](#)
- [vr::vertex](#)
- [vr::virtualConnect](#)
- [vr::withCopy](#)
- [vr::withEdge](#)
- [vr::withNeighbor](#)
- [vr::withNet](#)
- [vr::withProperty](#)
- [vr::withSpace](#)
- [vr::withText](#)
- [vr::withWidth](#)
- [vr::xor](#)

vr::acute

Description

Selects acute angles of shapes on the input layer.

Syntax

```
vr::acute -in <string> [-out <string>]
```

Arguments

-in

Specifies the input layer.

-out

Specifies the output layer.

Examples

```
vr::acute -in { M1 }
```

vr::and

Description

Boolean and operation.

Syntax

```
vr::and -in <string> [-connected | -notConnected] [-out <string>]
```

Arguments

-in

Specifies input layers {layer1 layer2}.

-connected

Specifies that polygons must have the same net id for their intersection to be output.

-notConnected

Specifies that polygons must have the different net id for their intersection to be output.

-out

Specifies the output layer.

Examples

```
vr::and -in { M1 M2 }
```

vr::angle

Description

Selects all layer edges having a smaller angle magnitude with respect to the x-axis of the coordinate system, which conform to the constraint.

Syntax

```
vr::angle -in <string> -value <constraint> [-not] [-out <string>]
```

Arguments

-in

Specifies the input layer.

-value

Specifies the constraint.

-not

The complement of the operation is output.

-out

Specifies the output layer.

Examples

```
vr::angle -in M1_edge -value {==33} -out Edge3
```

vr::area

Description

Selects all inLayer polygons having areas that conform to the constraint.

Syntax

```
vr::area -in <string> -value <constraint> [-not] [-out <string>]
```

Arguments

-in

Specifies the input layer.

-value

Specifies the constraint.

-not

The complement of the operation is output.

-out

Specifies the output layer.

Examples

```
vr::area -in Layer1 -value {>12.5} -out L1Big
```

vr::attach

Description

Assigns names to the extracted nets or ports. It attaches the connectivity information from textLayer objects onto connLayer objects.

Syntax

```
vr::attach -in <string> -to <string>
```

Arguments

-in

Specifies the connectivity information of the text layer.

-to

Attaches the connectivity information onto this layer.

Examples

```
vr::attach -in M1 -to M2
```

vr::coinEdge

Description

Selects all layer1 edges or edge portions coincident with layer2 edges.

Syntax

```
vr::coinEdge -in <string> [-inside | -outside] [-not] [-out <string>]
```

Arguments

-in

Specifies the input layers {layer1 layer2}.

-inside

Select coincident inside edge.

-outside

Select coincident outside edge.

-not

The complement of the operation is output.

-out

Specifies the output layer.

vr::connect

Description

Specifies electrical connections between objects on input layers.

Syntax

```
vr::connect -in <string>  
vr::connect -in <string> -by <string>
```

Arguments

-in

Specifies the input layers {inLayer1 [inLayer2] ... } or {inLayer1 inLayer2 [inLayerN] ...}.

-by

Specifies the contact layer {inLayer3}.

Examples

```
vr::connect -in { M1 M2 }
```

vr::convexEdge

Description

Selects edges based upon the number of convex endpoints the edges have.

Syntax

Syntax I:

```
vr::convexEdge -in <string> -value <constraint> [-withLength  
  <constraint>] [-out <string>]
```

Syntax II:

```
vr::convexEdge -in <string> -angle1 <constraint> [-length1 <constraint>]  
  -angle2 <constraint> [-length2 <constraint>] [-withLength <constraint>]  
  [-out <string>]
```

Arguments

`-in`

Specifies the input layer.

`-value`

Specifies the number of endpoints.

`-withLength`

Specifies that output edges are limited in length to the length constraint.

`-angle1`

Specifies the angle constraint of abutting edges.

`-angle2`

Specifies the angle constraint of interior-facing sides.

`-length1`

Specifies the length of the abutting edge.

`-length2`

Specifies the length of the interior-facing side.

`-out`

Specifies the output layer.

Examples

```
vr::convexEdge -in M1_edge -value "==0" -out M1edge_cvx  
vr::convexEdge -in M1 -angle1 {==90} -angle2 {==90} -  
withLength {<=0.09}
```

vr::copy

Description

Selects all inLayer shapes and dumps them to the outLayer shapes.

Syntax

```
vr::copy -in <string> [-out <string>]
```

Arguments

`-in`

Specifies the input layer.

-out

Specifies the output layer.

vr::cut

Description

Selects all layer1 polygons that share some (but not all) of their area with layer2 polygon.

Syntax

```
vr::cut -in <string> -value <constraint> [-byNet] [-even | -odd] [-out  
  <string>]
```

Arguments

-in

Specifies the input layers {layer1 layer2}.

-value

Specifies the constraint.

-byNet

layer1 polygon is selected when a number of distinct nets in the set of layer2 polygons, which share some of their area with the layer1 polygon, meets the specified constraint.

-even

The number of layer2 polygons that meet the constraint is also an even number.

-odd

The number of layer2 polygons that meet the constraint is also an odd number.

-out

Specifies the output layer.

Examples

```
vr::cut -in {layer1 layer2} -value >=1 -out cutout1  
vr::cut -in {layer2 layer1} -value >=1 -out notcutout2
```

vr::density

Description

Constructs a derived polygon layer by measuring the density of its input layer over a user-specified area within a user-specified window.

Syntax

```
vr::density -in <string> -value <constraint> [-window <float> [-step  
<float>]] [[-insideOf <string>] | [-insideOfExtent] | [-insideOfLayer  
<string>]] [-backUp | -truncate | -ignore] [-gradient <constraint>  
[-relative | -absolute] [-corner]] [-expr <string>] [-out <string>]
```

Arguments

-in

Specifies the input layers {inLayer [inLayerN]...}.

-value

Specifies the density constraint.

-window

Specifies the dimensions {wxy | wx wy} of data capture window.

wxy indicates the square window.

wx is the dimension in x-axis and wy is the dimension in y-axis.

-step

Specifies the step size {sxy | sx sy} for the sliding window.

sxy indicates the incremental size in the x and y direction.

sx indicates the incremental size in x direction and sy indicates the incremental size in y direction.

-insideOf

Specifies the rectangle area {x1 y1 x2 y2} for the data capture extent.

-insideOfExtent

Specifies the rectangular extent of the input layer for the data capture extent.

-insideOfLayer

Specifies the rectangular extent of polygons from inLayerM for the data capture extents.

`-backup`

Specifies if a window overlaps the right-hand or top edges of the data capture extent, the window is shifted left or down until it is no longer overlapping the data capture extent.

`-truncate`

Specifies a data capture window that is truncated at the limits of the data capture extent.

`-ignore`

Specifies to ignore the data capture window if the window overlaps the right-hand or top edges of the data capture extent.

`-gradient`

Specifies the gradient value a window must satisfy in order to be output where the gradient is a measure of density value change from one window to the adjacent non-overlapping windows.

`-relative`

Specifies the gradient function $G(A, B) = \frac{||V_a| - |V_b||}{\max(|V_a|, |V_b|)}$.

`-absolute`

Specifies the gradient function $G(A, B) = ||V_a| - |V_b||$.

`-corner`

Specifies the gradient calculation including the four corners.

`-expr`

Specifies the numeric expression to customizable control over the operation computations.

`-out`

Specifies the output layer.

Examples

```
db::density -in {F M1 CHIP} -value <$M1_DN_1 \  
-window $M1_DN_1_W -step $M1_DN_1_S -backUp \  
-expr {!AREA(F)+AREA(M1)/AREA(CHIP)} -out m1_dn_1
```

vr::donut

Description

Selects all inlayer polygons having interior cycles (holes) that are fully enclosed within a inlayer polygon.

Syntax

```
vr::donut -in <string> [-value <constraint>] [-not] [-out <string>]
```

Arguments

-in

Specifies the input layer.

-value

Specifies the constraint.

-not

The complement of the command.

-out

Specifies the output layer.

vr::dfmText

Description

Converts text objects to rectangles with design-for-manufacturing (DFM) properties.

Syntax

```
vr::dfmText -in <string> -propName <string> [-out <string>]
```

Arguments

-in

Specifies the input layer.

-propName

Specifies the property name of the converted text objects.

-out

Specifies the output layer.

Examples

```
vr::dfmText -in M1_Text -propName Hv -out M1_Hv_
```

vr::dpt

Description

Checks if the input layers can pass the double-patterning (DPT) criteria.

Syntax

```
vr::dpt -in <string> -link <string> [-sameMask <string>] [-anchorA  
<string>] [-anchorB <string>] [-out <string>] [-maskA <string>] [-maskB  
<string>] [-collision <string>]
```

Arguments

-in

Specifies the input layer.

-link

Specifies the link layers.

-sameMask

Specifies the same mask layers.

-anchorA

Specifies layer with color A.

-anchorB

Specifies layer with color B.

-out

Output the conflict layer.

-maskA

Output layer assigned to be color A in this operation.

-maskB

Output layer assigned to be color B in this operation.

-collision

The collision layer.

Examples

```
vr::dpt -in M1 -link {m1_link} -out m1_conflict \  
-maskA m1_maskA -maskB m1_maskB -collision m1_collision \  
-anchorA M1CA -anchorB M1CB
```

vr::exclude

Description

Specifies a single polygon window that defines the exclusion of input shapes and text to be used in subsequent rule decks with syntax I. It also excludes shapes and text objects for each specified cells with syntax II.

Syntax

Syntax I:

```
vr::exclude -window <string>
```

Syntax II:

```
vr::exclude -cell <string>
```

Arguments

-window

Specifies the single polygon window {x1 y1 x2 y2 [xN yN]...}.

-cell

Specifies the cell name {cell1 [cellN]...}.

Examples

```
vr::exclude -window { 0 0 0.2 0.3 }
```

vr::extent

Description

Outputs the data extent of the current cell.

Syntax

```
vr::extent [-in <string>] [-out <string>]
```

Arguments

-in

Specify the input layer.

`-out`

Specify the output layer.

Examples

```
vr::extent -out cell_bbox
```

vr::extents

Description

Generates a derived polygon layer consisting of the merged minimum bounding boxes of each polygon on inlayer.

Syntax

```
vr::extents -in <string> [-centers [<int>] | -squares] [-out <string>]
```

Arguments

`-in`

Specify the input layer.

`-centers`

Generates marker squares at the center of each bounding box instead of the bounding boxes themselves. The default value is 1.

`-squares`

Squares are output rather than minimum bounding box.

`-out`

Specify the output layer.

Examples

```
vr::extents -in M1 -out M1_bbox
```

vr::extentCell

Description

Generates a derived polygon layer consisting of rectangles that represent the extents of cells in the cell list.

Syntax

```
vr::extentCell -cell <string>
```

Arguments

`-cell`

Specifies cell names {cell [cell 2] ...}.

vr::extendDrawn

Description

Generates a derived polygon layer consisting of the extent of shapes on the specified input layers.

Syntax

```
vr::extentDrawn [-in <string>] [-out outLayer]
```

Arguments

`-in`

Specifies the input layers {layer [layer]...}.

`-out`

Specifies output layer.

vr::expandEdge

Description

Expands edges into rectangles in the specified directions and amounts.

Syntax

```
vr::expandEdge -in <string> {-insideBy <float>| -outsideBy <float> | -by  
<float>} [-extendBy <float>] [-cornerFill] [-out <string>]
```

Arguments

`-in`

Specifies the input layer.

`-insideBy`

Expands an edge towards the inside of the polygon it originated.

`-outsideBy`

Expands an edge towards the outside of the polygon it originated.

`-by`

Shorthand notation for specifying both keywords `insideBy` and `outsideBy`.

`-extendBy`

Set that directs an operation to extend or retract the length of all inlayer edges prior to expanding them into rectangles.

`-cornerFill`

Fill gaps between rectangles formed by the operation at corners of the input layer.

`-out`

Specifies the output layer.

Examples

```
vr::expandEdge -in inLayer1 -insideBy 0.5
vr::expandEdge -in inLayer1 -outsideBy 0.5
vr::expandEdge -in M1 -outsideBy 0.5 -extendBy 0.5 -out M1S
```

vr::expandText

Description

Constructs the merged squares center on the positions of text objects having the specified text name.

Syntax

```
vr::expandText -name <string> -by <float> [-text <string>] [-primaryOnly]
               [-caseSensitive] [-out <string>]
```

Arguments

`-name`

Specifies the text name.

`-by`

Specifies the size of marker squares.

`-text`

Specifies the layer of the text layer.

`-primaryOnly`

Specifies only the text in the top-cell is used.

`-caseSensitive`

Specifies the text name that is case sensitive.

`-out`

Specifies the output layer.

Examples

```
vr::expandText -name VDD -by 0.5 -text L7
```

vr::enclose

Description

Selects all layer1 polygons that contain any layer2 polygons as a subset

Syntax

```
vr::enclose -in <string> [-value <constraint> [-byNet] [-even | -odd]]  
[-not] [-out <string>]
```

Arguments

`-in`

Specifies the input layers {layer1 layer2}.

`-value`

Specifies the constraint.

`-byNet`

Specifies a layer1 polygon is selected when the number of distinct nets (not polygons) on layer2, enclosed by the layer1 polygon, meets the specified constraint.

`-even`

The number of layer2 polygons that meet the constraint is also an even number.

`-odd`

The number of layer2 polygons that meet the constraint is also an odd number.

`-not`

The complement of the operation.

`-out`

Specifies the output layer.

Examples

```
vr::enclose -in {layer1 layer2} -value >=1 -out enclose_out1
```

vr::encloseRectangle

Description

Selects all polygons on the specified inLayer that can enclose a rectangle of the specified width and length.

Syntax

```
vr::encloseRect -in <string> -rect <float> [-orthogonalOnly] [-not] [-out  
  <string>]
```

Arguments

-in

Specifies the input layer.

-rect

Specifies the width and length of rectangle.

-orthogonalOnly

Limits selection to polygons that enclose rectangles having sides that are parallel (respectively) to the coordinate axes of the database.

-not

The complement of the command.

-out

Specifies the output layer.

Examples

```
vr::encloseRect -in M1 -rect {0.2 0.4} -not 1
```

vr::enclosure

Description

The layer-pair enclosure and extension operations are performed by checking from the sea side of enclosed edges to the land side of enclosing edges.

For more information on filter commands, see [Dimension Check Filters](#).

Syntax

```
vr::enclosure -in <string> -value <constraint>  
[metric] [polygon_containment] [connectivity_filter]  
[orientation_filter] [projection_filter] [angled_filter]  
[corner_filter] [intersection_filter] [reversal_filter] [exclude_false]  
[output_filter]
```

vr::external

Description

The single-layer external check (spacing) operation is performed by checking from sea side to sea side of edges. The layer-pair external check (clearance) operation is performed by checking from sea side to sea side of edges.

For more information on filter commands, see [Dimension Check Filters](#).

Syntax

One-Layer:

```
vr::external -in <string> -value <constraint>  
[metric] [polygon_filter] [connectivity_filter]  
[orientation_filter] [projection_filter] [angled_filter]  
[corner_filter] [intersection_filter] [reversal_filter]  
[exclude_false] [output_filter]
```

Two-Layer:

```
vr::external -in <string> -value <constraint>  
[metric] [polygon_filter] [polygon_containment]  
[connectivity_filter] [orientation_filter]  
[projection_filter] [angled_filter] [corner_filter]  
[intersection_filter] [reversal_filter] [exclude_false] [output_filter]
```

vr::group

Description

Assembles one or more rules in a group.

Syntax

```
vr::group -name <string> -rule <string> [-exclude <string>]
```

Arguments

-name

Specifies the group name.

`-rule`

Specifies one or more rules.

`-excludeRule`

Specifies the exclude rules.

Examples

```
vr::group -name rule12 -rule {rule1 rule2}
```

vr::grow

Description

Expands input layer in the specified direction.

Syntax

```
vr::grow -in <string> [-rightBy <float>] [-topBy <float>] [-leftBy  
<float>] [-bottomBy <float>] [-out <string>]
```

Arguments

`-in`

Specifies the input layer.

`-rightBy`

Expand the right edges of objects on the input layer by the specified value.

`-topBy`

Expand the top edges of objects on the input layer by the specified value.

`-leftBy`

Expand the left edges of objects on the input layer by the specified value.

`-bottomBy`

Expand the bottom edges of objects on the input layer by the specified value.

`-out`

Specifies the output layer.

Examples

```
vr::grow -in M1 -rightBy 0.005
```

vr::holes

Description

Forms a layer consisting of all polygons that fit exactly inside of layer polygon holes.

Syntax

```
vr::holes -in <string> [-value <constraint>] [-inner] [-empty] [-out  
  <string>]
```

Arguments

-in

Specifies the input layer.

-value

Specifies the constraint.

-inner

Prevents holes containing other holes from being output.

-empty

Prevents output of holes that are not outside a layer polygon.

-out

Specifies the output layer.

Examples

```
vr::holes -in layer1 -value >1 -out L1holeGT1
```

vr::include

Description

Specifies a single polygon window that defines the inclusion of input shapes and text with syntax I. It also includes shapes and text objects for each specified cells with syntax II.

Syntax

Syntax I:

```
vr::include -window <string>
```

Syntax II:

```
vr::include -cell <string>
```

Arguments

`-window`

Specifies the single polygon window {x1 y1 x2 y2 [xN yN]...}.

`-cell`

Specifies the cell names.

Examples

```
vr::include -window { 0 0 0.2 0.3 }
```

vr::input

Description

Specifies the important parameters for input before running other commands.

Arguments `-lib` and `-cell` must be specified with either `db::checkDRD` or `vr::input` commands in the rule deck file.

Syntax

```
vr::input -lib <string> -cell <string> [-view <string>] [-map <string>]  
[-objectMap <string>]
```

Arguments

`-lib`

A library name.

`-cell`

A cell name.

`-view`

A view type.

`-map`

Layer mapping file.

`-objectMap`

Object mapping file that maps boundaries and layer blockages between Stream shapes and OA databases. For information about object mapping files, see the *Custom Compiler Environment User Guide*.

Examples

```
vr::input -lib drc -cell spacing
```

vr::inside

Description

Selects all layer1 polygons that share their entire areas with layer2 polygons.

Syntax

```
vr::inside -in <string> [-not] [-out <string>]
```

Arguments

-in

Specifies the input layers.

-not

The complement of the command.

-out

Specifies the output layer.

Examples

```
vr::inside -in {layer1 layer2} -out inside_out1
```

vr::insideCell

Description

Selects the polygons from inlayer in the specified cell.

Syntax

```
vr::insideCell -in <string> -cell <string> [-primaryOnly] [-not] [-out  
<string>]
```

Arguments

-in

Specifies the input layer.

-cell

Specifies the cell name or cell name list.

-primaryOnly

Output geometry only from the top level of the specified cells.

`-not`

The complement of the operation is output.

`-out`

Specifies the output layer.

Examples

```
vr::insideCell -in M2 -cell EDFQD1BWP
```

vr::insideEdge

Description

Selects all layer1 edges or edge segments that lie completely inside layer2 polygons.

Syntax

```
vr::insideEdge -in <string> [-not] [-out <string>]
```

Arguments

`-in`

Specifies the input layers.

`-not`

The complement of the operation is output.

`-out`

Specifies the output layer.

Examples

```
vr::insideEdge -in {layer1 layer2} -out M1E_in_M2
```

vr::interact

Description

Selects layer1 polygons based upon the type and number of intersections they have with layer2 polygons.

Syntax

```
vr::interact -in <string> [-value <constraint> [-byNet] [-even | -odd]]  
[-singularAlso | -singularOnly] [-not] [-out <string>]
```

Arguments

`-in`

Specifies the input layers.

`-value`

Specifies the constraint.

`-byNet`

Specifies the selection of a layer1 polygon based upon the number of distinct nets it interacts with on layer2.

`-even (odd)`

A layer1 polygon is selected if the number of layer2 polygons that meet the constraint is also an even (odd) number.

`-singularAlso`

Including outside point-to-point and point-to-edge interaction.

`-singularOnly`

Selects all polygons from layer1 that intersect polygons on layer2 by outside point-to-point and point-to-edge interaction, regardless of any other Interactions.

`-not`

The complement of the command.

`-out`

Specifies the output layer.

Examples

```
vr::interact -in {layer1 layer2} -value >=3 -out interact_out1
```

vr::internal

Description

The single-layer internal check (width) operation is performed by checking from land side to land side of edges. For the two-layer syntax, measures the separations between the interior-facing sides of layer1 edges and the interior-facing sides of layer2 edges.

For more information on filter commands, see [Dimension Check Filters](#).

Syntax

One-layer:

```
vr::internal -in <string> -value <constraint>  
[metric][orientation_filter][projection_filter]  
[angled_filter][corner_filter][intersection_filter] [exclude_false]  
[output_filter]
```

Two-layer:

```
vr::internal -in <string> -value <constraint>  
[metric][polygon_containment] [connectivity_filter][orientation_filter]  
[projection_filter][angled_filter][corner_filter]  
[intersection_filter][exclude_false] [output_filter]
```

vr::layer

Description

Assigns a layer number for the specified layer name.

Syntax

```
vr::layer -in <string> -layer <int>
```

Arguments

-in

Specifies the unique layer name.

-layer

Specifies the corresponding layer number of the specified layer.

Examples

```
vr::layer -in M1 -layer 31
```

vr::layerMap

Description

Groups the mapping layers and the range type into a specified target layer number.

Syntax

```
vr::layerMap -layer <constraint> [-text] -type <constraint> -toLayer  
<int>
```

Arguments

`-layer`

Specifies a range of mapping layers.

`-text`

Indicates whether to use the mapping for text layers.

`-type`

Specifies the type of mapping layers.

`-toLayer`

Layer number of the target layer.

Examples

```
vr::layerMap -layer 31 -type { >=120<=121} -toLayer 120121
```

vr::length

Description

Selects all edges of layer having length that conforms to the constraint.

Syntax

```
vr::length -in <string> -value <constraint> [-not] [-out <string>]
```

Arguments

`-in`

Specifies the input layers.

`-value`

Specifies the length constraint.

`-not`

Complements of the operation is output.

`-out`

Specifies the output layer.

Examples

```
vr::insideEdge -in {layer1 layer2} -out M1E_in_M2  
vr::length -in M1E_in_M2 -value {>1.55<3.15} -out EdgeLong
```

vr::netAreaRatio

Description

Selects polygons from the specific input layer. An expression value is produced to meet the constraint.

Syntax

```
vr::netAreaRatio -in <string> [-connect <string>] [-cumulated <string>]  
[-expr <string>] -value <constraint> [-out <string>]
```

Arguments

-in

Specifies the input layers.

-connect

Specifies the denominator input layers.

-cumulated

Specifies the derived layers from a previous netAreaRatio operation whose value is attached to the output layer.

-expr

Specifies the numeric expression to customizable control the operation computations.

-value

Specifies the constraining internal of values.

-out

Specifies the output layer.

Examples

```
vr::netAreaRatio -in {M1 GATE} -expr {AREA(M1)/AREA(GATE)} -value {>10}
```

vr::not

Description

Boolean not operation.

Syntax

```
vr::not -in <string> [-out <string>]
```

Arguments

`-in`

Specifies the input layers.

`-out`

Specifies the output layer.

Examples

```
vr::not -in { M1 M2 } -out notM1M2
```

vr::offGrid

Description

Selects the off-grid objects in the input layer.

Syntax

```
vr::offGrid -in <string> -resolution <int> [-insideOf <string> [-relative  
| -absolute]] [-centers] [-edge] [-region [<float>]] [[-offset [<int>]  
[-insideBy <int> | -outsideBy <int>]]] [-hint] [-out <string>]
```

Arguments

`-in`

Specifies the input layer.

`-resolution`

Specifies the resolution {xy-resolution | {x-resolution y-resolution}}.

`-insideOf`

Specifies only objects of layer1 that lie inside layer2 are used.

`-relative`

Specifies that the grid origin is determined by the total rectangular extent of layer1 objects inside of layer2.

`-absolute`

Specifies that the grid origin is determined by the unmerged extents of polygons from layer2.

`-centers`

Specifies the centers of input objects are checked against the grid.

`-edge`

Specifies the output of the operation is a derived edge layer.

`-region`

Specifies the output is a derived polygon layer consisting of squares that mark the locations of offgrid vertices. The default value is 1.

`-offset`

An offset displacement is applied to objects before checking them against the resolution parameters.

`-insideBy`

Each input edge is offset in the inside direction by value.

`-outsideBy`

Each input edge is offset in the outside direction by value.

`-hint`

Specifies the output objects provide locations of nearby on-grid points.

`-out`

Specifies the output layer.

Examples

```
vr::offGrid -in M1 -resolution 2 -region
```

vr::or

Description

Boolean or operation.

Syntax

```
vr::or -in <string> [-out <string>]
```

Arguments

`-in`

Specifies the input layers.

`-out`

Specify the output layer.

Examples

```
vr::or -in { M1 M2 } -out orM1M2
```

vr::orEdge

Description

Performs a Boolean or on edges from the input layers.

Syntax

```
vr::orEdge -in <string>
```

Arguments

-in

Specifies the input layers.

vr::outside

Description

Selects all layer1 polygons with areas that lie completely outside all layer2 polygons.

Syntax

```
vr::outside -in <string> [-not] [-out <string>]
```

Arguments

-in

Specifies the input layers.

-not

The complement of the operation.

-out

Specifies the output layer.

Examples

```
vr::outside -in {layer1 layer2} -out outside_out1
```

vr::outsideEdge

Description

Selects all layer1 edges or edge segments that lie completely outside layer2 polygons.

Syntax

```
vr::outsideEdge -in <string> [-not] [-out <string>]
```

Arguments

-in

Specifies the input layers.

-not

The complement of the operation is output.

-out

Specifies the output layer.

Examples

```
vr::outsideEdge -in {layer1 layer2} -out M1E_out_M2
```

vr::pathLength

Description

Selects a maximum-length contiguous path of distinct edges from a single polygon on the input layer. The path has a total length that satisfies the constraint.

Syntax

```
vr::pathLength -in <string> -value <constraint> [-out <string>]
```

Arguments

-in

Specifies the input layer.

-value

Specifies the path length constraint.

-out

Specifies the output layer.

Examples

```
vr::insideEdge -in {M1 M2} -out inMlinsideEdge  
vr::pathLength -in inMlinsideEdge -value {>=0.35}
```

vr::perimeter

Description

Selects all inLayer polygons with perimeters conforming to the constraint.

Syntax

```
vr::perimeter -in <string> -value <constraint> [-out <string>]
```

Arguments

-in

Specifies the input layers.

-value

Specifies the constraint.

-out

Specifies the output layer.

Examples

```
vr::perimeter -in Layer 1 -value {>14<=26} -out Llperim
```

vr::polygon

Description

Constructs a polygon having the given (x, y) coordinates on the specified layer.

Syntax

```
vr::polygon -in <string> -point <float> [-out <string>]
```

Arguments

-in

Specifies the input layer.

-point

Specifies the coordinates of the polygon.

`-out`

Specifies the output layer.

vr::renameText

Description

Enables the runtime editing of layout text values.

Syntax

```
vr::renameText [-all] [-nocase] -regExp <string> -to <string>
```

Arguments

`-all`

Specifies to find all text values matching regular expression and perform substitution for each text value.

`-nocase`

Specifies to convert the text values to lowercase and then perform matching and substitution.

`-regExp`

Specifies the regular expression.

`-to`

Specifies the pattern string for replacement when the matching is occurred.

vr::rectangle

Description

Selects rectangles based on geometric properties.

Syntax

```
vr::rectangle -in <string> [-value <constraint> [-by <constraint>]]  
[-aspect <constraint>] [-measureExtents] [-not] [-out <string>]
```

Arguments

`-in`

Specifies the input layer.

`-value`

Specifies the constraint.

`-by`

Specifies the constraint.

`-aspect`

Specifies the ratio of the longer side to the shorter side.

`-measureExtents`

Selects polygons having individual rectangular extents that meet the constraint.

`-not`

Complement of the operation is output.

`-out`

Specifies the output layer.

Examples

```
vr::rectangle -in inLayer -value {==2} -by {>4}
```

vr::rectangles

Description

Constructs an array of rectangles with the specified dimensions and spacing.

Syntax

```
vr::rectangles -width <float> -length <float> -space <float> | -insideOf  
<float> [-offset <float>] [-maintainSpacing] [-out <string>]
```

Arguments

`-width`

Specifies the width (x-axis) of the rectangle.

`-length`

Specifies the width (y-axis) of the rectangle.

`-space`

Specifies the spacing (x-axis, y-axis) between two adjacent rectangles.

`-offset`

Specifies the offset between two adjacent rectangles.

`-insideOf`

Specifies the region or layer to construct rectangles.

`-maintainSpacing`

Specifies no rectangles are constructed in the area specified in the `-space` argument.

`-out`

Specifies the output layer.

vr::rectEnclosure

Description

Constructs polygons from rectangles in `inLayer1` enclosed by rectangles in `inLayer2`.

Syntax

```
vr::rectEnclosure -in <string> [-abut [<constraint>]] [-singular]
  [-outsideAlso] [-orthogonalOnly] -rules <string>
```

Arguments

`-in`

Specifies the input layers.

`-abut`

Specifies to measure the separation between intersecting edges. The default value is $\geq 0 < 180$.

`-singular`

Measure the separation between intersecting edges at points of polygon singularity.

`-outsideAlso`

Specifies to output the rectangles from `inLayer1` outside or coincident outside with `inLayer2`.

`-orthogonalOnly`

Only deal with orthogonal rectangles.

`-rules`

Specifies rectangle rules, {{{GOOD|BAD} value1 [metric1] value2 [metric2]...} ...}. GOOD-type rectangle rules do not output rectangles that satisfy rules.

valueN

Non-negative, floating-point numbers used to check edge extensions of enclosed rectangles.

-out

Specifies the output layer.

Examples

```
vr::rectEnclosure -in {M1 M2} -outsideAlso -rules { {GOOD {0} {0.05  
  OPPOSITE} {0} {0.05 OPPOSITE} }
```

vr::ruleSet

Description

A RuleSet is a set of core commands that describes a specific design rule. DRD executes the commands in a RuleSet and outputs the results.

Syntax

```
vr::ruleSet -opSet {  
  ...  
} [-id <string>] [-description <string>]
```

Arguments

-id

Rule Id.

-description

Description of the rule set.

Examples

```
vr::ruleSet -opSet {  
  ...  
} -id M1.EOL -description {M1 end of line checking}
```

vr::sconnect

Description

Passes connectivity from the upperLayer polygons to lowerLayer polygons.

Syntax

```
vr::sconnect -in <string> [-abutAlso]  
vr::sconnect -in <string> -by <string>
```

Arguments

-in

Specifies the input layers.

-by

Specifies the contact layer.

-abutAlso

Specifies that abutment is considered to constitute overlap.

vr::selectRule

Description

Selects rules to be applied.

Syntax

```
vr::selectRule -rule <string>
```

Arguments

-rule

Specifies one or more rules to be applied

Examples

```
vr::selectRule -rule { rule1 rule2 }
```

vr::selectRulebyLayer

Description

Selects rules by one or more layers.

Syntax

```
vr::selectRulebyLayer -in <string>
```

Arguments

-in

Specifies one or more layers to be applied.

Examples

```
vr::selectRulebyLayer -in { M1 M2 }
```

vr::set

Description

Specifies the level of rule checking results to report and performs color balancing for a design. During color balancing, the DRD engine ensures that the color distribution meets specified requirements.

Note:

Color balancing is only applicable during viewport-level design checking, and if you selected **Color Generation** in the **DRD Options** dialog box. For information about the **DRD Options** dialog box, see Setting DRD Options in the *Custom Compiler™ Design Checking and Physical Verification User Guide*.

Syntax

```
vr::set [-maxResults <string>][-depth <int>][-textDepth  
<int>][-colorBalance 0|count|area] [-colorDensity {layer1 diffDensity1  
densityWindow1 stepWindow1 model [layer2 diffDensity2 densityWindow2  
stepWindow2 mode2] ...}]
```

Arguments

`-maxResults`

Specifies the maximum check results to report: `all` or the number of results to report. The default is 1000.

`-depth`

Specifies the hierarchy depth of the input layer.

`-textDepth`

Specifies the hierarchy depth of the text layer.

`-colorBalance`

Balances color distribution in the design using one of the following methods:

- `0`: Do not perform color balancing using the `-colorBalance` argument.
- `count`: Uses shape count to assign and distribute color.

In the following double patterning example, shapes are assigned the colors blue and red. When the number of shapes is odd, the color distribution is uneven. In example shown, the number of blue shapes exceeds the number of red shapes by 1.

If the number of shapes is even, the blue and red colors are assigned evenly. In the example shown, two shapes are blue, and two shapes are red.



- `area`: Uses the total area to assign and distribute the colors.

For example, in a double patterning design, where an even number of identical shapes are placed, 50% of the area will contain shapes of one color, and 50% of the area will contain shapes of the second color.

`-colorDensity`

Balances colors based on the color density analysis. If you specify both `-colorBalance` and `-colorDensity`, the DRD engine will use `-colorDensity`.

`-layer`

The name of the layer on which the color balancing or color density check is performed. You must specify the layers, because not every layer is colorable and require balancing.

`-diffDensity`

The maximum percent difference between the color densities. For example, if you specify 10, then the colors in a balanced double-patterning design should remain approximately 45% and 55%.

`-densityWindow`

The size of the density window within the specified layer. The value is in microns.

The metal and color density analyzer divides a design into subsections, or windows, and checks each window during analysis. For more information about the density windows, see the *Custom Compiler* Design Checking and Physical Verification User Guide.

`-stepWindow`

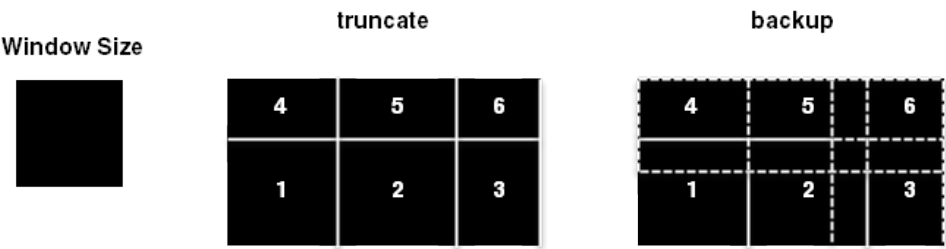
The size of the step that the metal and color density analyzer takes towards the next density window as it progresses forward during analysis. The value is in microns.

For more information about the density window steps, see the *Custom Compiler* Design Checking and Physical Verification User Guide.

-moden

If the design cannot be divided evenly into full-sized density windows, you can specify how to determine the windows for the remaining design:

- **truncate**: The remaining density windows will be truncated such that they do not comply with the specified density window size. If the window size and the step size are identical, then the windows do not overlap. In the example shown, windows 1 and 2 comply with the specified window size. However, windows 3, 4, 5, and 6 are truncated.
- **backup**: The analyzer will back up from the edge of the design so that all windows comply with the density window size specification. If the window size and the step size are identical, then the windows overlap. In the example shown, all six windows comply with the specified density window size. However, each of the six windows overlap with at least one other window.



Examples

In the following example, the `vr::set` command will balance colors based on the color density analysis. Color balancing is performed on layer `m1` and `m2`. The percent difference in the color coverage must remain within 5% on each layer. On both layers, the density analysis window size is 40 mm, and the step size is 20 mm. The density analysis window mode is `backup`.

```
vr::set -colorDensity {m1 5 40 20 backup m2 5 40 20 backup}
```

vr::shift

Description

Constructs polygons from `inLayer` by shifting the object in the x and y direction.

Syntax

```
vr::shift -in <string> -by <float> [-out <string>]
```


Arguments

`-in`

Specifies the input layer.

`-by`

Specifies the distance to shift the inlayer polygons.

`wxy` indicates the square window.

`wx` is the dimension in x-axis and `wy` is the dimension in y-axis.

`-out`

Specifies the output layer.

Examples

```
vr::shift -in M1 -by {0.05 0.1}
```

vr::shrink

Description

Specifies inside contraction of objects on the input layer in the direction of the x-axis, y-axis, or both.

Syntax

```
vr::shrink -in <string> [-rightBy <float>] [-topBy <float>] [-leftBy  
<float>] [-bottomBy <float>] [-out <string>]
```

Arguments

`-in`

Specifies the input layer.

`-rightBy`

Contract toward the interior, the right edges of objects on the input layer by the specified value.

`-topBy`

Contract toward the interior, the top edges of objects on the input layer by the specified value.

`-leftBy`

Contract toward the interior, the left edges of objects on the input layer by the specified value.

`-bottomBy`

Contract toward the interior, the bottom edges of objects on the input layer by the specified value.

`-out`

Specifies the output layer.

Examples

```
vr::shrink -in M1 -bottomBy 0.005 -out M1BS
vr::shrink -in M1 -rightBy 0.005 -out M1RS
```

vr::size

Description

Expands or shrinks polygons on the input layer by the specified amount.

Syntax

```
vr::size -in <string> -value <float> [-overUnder | -underOver]
        [[-insideOf <string> [-step <float>]] | [-outsideOf <string> [-step
        <float>]] [-truncate <float>] [-out <string>]
```

Arguments

`-in`

Specifies input layer.

`-value`

Specifies how much to expand or shrink. Positive values expand polygons and negative values shrink them.

`-overUnder`

First increased in size, then decreased in size.

`-underOver`

First decreased in size, then increased in size.

`-insideOf`

inlayer to expand inside of inLayer2.

`-outsideOf`

inlayer to expand outside of inLayer2.

`-step`

Specifies the step size of the sizing process.

`-truncate`

Controls spike truncation.

`-out`

Specifies the output layer.

Examples

```
vr::size -in M1 -value 100 -insideOf nwell -step 2.3
```

vr::skew

Description

Selects skew edges on the input layer.

Syntax

```
vr::skew -in <string> [-out <string>]
```

Arguments

`-in`

Specifies the input layer.

`-out`

Specifies the output layer.

Examples

```
vr::skew -in M1 -out M2
```

vr::stamp

Description

Constructs a layer which is the inLayer1 polygons overlapped with inLayer2 polygons. The connectivity info is transferred from inLayer2 to inLayer1.

Syntax

```
vr::stamp -in <string> -by <string> [-abutAlso] [-out <string>]
```

Arguments

`-in`

Specifies the input layer.

`-by`

Specifies the overlapped polygons that transfers the connectivity info.

`-abutAlso`

Specifies the abutment which is considered to constitute overlap in the operation.

`-out`

Specifies the output layer.

vr::textLayer

Description

Specifies the layers of free-floating text for connectivity extraction.

Syntax

```
vr::textLayer -in <string> [-depth <int>]
```

Arguments

`-in`

Specifies the text layers for connectivity extraction.

`-depth`

Specifies the text depth.

Examples

```
vr::textLayer -in { M1 M2 }
```

vr::textOrder

Description

Defines the order in which connectivity extraction searches layers for objects that intersect another object owning a text location.

Syntax

```
vr::textOrder -in <string>
```

Arguments

`-in`

Specifies the input layers.

vr::touch

Description

Selects all layer1 polygons that lie completely outside all layer2 polygons, but share a coincident edge, or segment, with a polygon from layer2.

Syntax

```
vr::touch -in <string> [-value <constraint> [-byNet] [-even | -odd]]  
          [-not] [-out <string>]
```

Arguments

-in

Specifies the input layers.

-value

Specifies the number of layer2 polygons a layer1 polygon must touch in order to be selected.

-byNet

Specifies that a layer1 polygon is selected when a number of distinct nets in the set of layer2 polygons, which have a coincident outside edge with the layer1 polygon, meets the specified constraint.

-even

A layer1 polygon is selected if the number of layer2 polygons that meet the constraint is also an even number.

-odd

A layer1 polygon is selected if the number of layer2 polygons that meet the constraint is also an odd number.

-not

The complement of the operation.

-out

Specifies the output layer.

Examples

```
vr::touch -in {Layer1 Layer2} -value >=2
```

vr::touchEdge

Description

Selects all complete layer1 edges that are coincident with any portion of layer edges.

Syntax

```
vr::touchEdge -in <string> [-inside | -outside] [-not] [-out <string>]
```

Arguments

-in

Specifies the input layers.

-inside

Share the interior sides.

-outside

Share the exterior sides.

-not

Complement of the operation is output.

-out

Specifies the output layer.

Examples

```
vr::touchEdge -in {M1 M2} -out M1_touch_M2
vr::touchEdge -in {M1E M2E} -inside 1 -out M1_touch_M2
vr::touchEdge -in {M1E M2E} -outside 1 -out M1_touch_M2
```

vr::tpt

Description

Checks for triple-patterning conflicts.

Syntax

```
vr::tpt -in <string> -link <string> [-anchor1 <string>] [-anchor2  
<string>] [-anchor3 <string>] [-mask1 <string>] [-mask2 <string>] [-mask3  
<string>] [-out <string>]
```

Arguments

`-in`

Specifies the input layer.

`-link`

Specifies the link layer.

`-anchor1`

Specifies the color1 layer.

`-anchor2`

Specifies the color2 layer.

`-anchor3`

Specifies the color3 layer.

`-mask1`

Output layer that is assigned color1 during this operation.

`-mask2`

Output layer that is assigned color2 during this operation.

`-mask3`

Output layer that is assigned color3 during this operation.

`-out`

Outputs the layers that have triple-patterning conflicts.

Examples

```
vr::tpt -in M1 -link M1_link -anchor1 M1C1 -anchor2 M1C2 -anchor3 M1C3  
-mask1 M1_color1 -mask2 M1_color2 -mask3 M1_color3 -out M1_conflict
```

vr::unselectRule

Description

Selects rules to be excluded.

Syntax

```
vr::unselectRule -rule <string>
```

Arguments

`-rule`

Specifies one or more rules to be excluded.

Examples

```
vr::unselectRule -rule { rule1 rule2 }
```

vr::unselectRuleByLayer

Description

Selects rules to be excluded by one or more layers.

Syntax

```
vr::unselectRuleByLayer -in <string>
```

Arguments

`-in`

Specifies one or more layers to be excluded.

Examples

```
vr::unselectRuleByLayer -in { M1 M2 }
```

vr::vertex

Description

Selects all inLayer polygons having vertex counts conforming to the constraint.

Syntax

```
vr::vertex -in <string> -value <constraint> [-out <string>]
```

Arguments

`-in`

Specifies the input layer.

`-value`

Specifies the constraint.

`-out`

Specifies the output layer.

Examples

```
vr::vertex -in layer1 -value {>4} -out L1vertex4
```

vr::virtualConnect

Description

Specifies virtual connections for identical net names.

Syntax

```
vr::virtualConnect -name <string>
```

Arguments

-name

Specifies net names.

vr::withCopy

Description

Copies the specified layer to a new layer.

Syntax

```
vr::withCopy -in <string> [-region | -edge | -centers | -bridge] [-out  
  <string>]
```

Arguments

-in

Specifies the input layer.

-region

Polygon layers are output.

-edge

Edge layers are output.

-centers

Output an edge layer that is the centerline between the error cluster.

-bridge

Output an edge layer that connects the two edges in the cluster.

-output

Specifies output layer.

vr::withEdge

Description

Selects all layer1 polygons that have edges or edge segments on layer2.

Syntax

```
vr::withEdge -in <string> [-value <constraint>] [-not] [-out <string>]
```

Arguments

-in

Specifies the input layers.

-value

Specifies the number of edges or segments that the layer1 polygon has in layer2.

-not

Complement of the operation is output.

-out

Specifies the output layer.

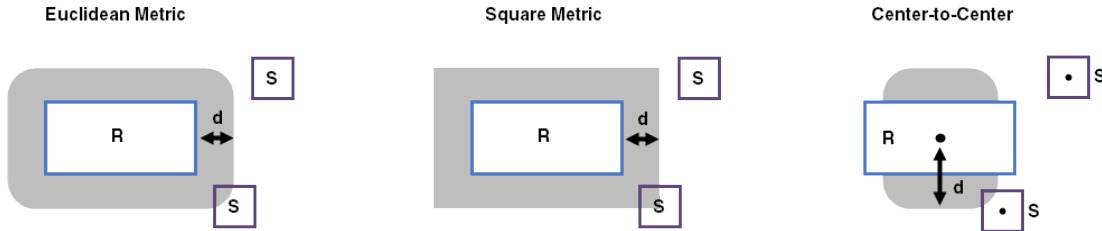
Examples

```
vr::external -in M1 -value 0.040 -out M1_edge  
vr::withEdge -in {M1 M1_edge} -value ==2 -out M1_2neighbor
```

vr::withNeighbor

Description

Selects a rectangle (R) if the specified number of neighboring rectangles (S) are partially or wholly within its halo region. The halo region is calculated using the distance (d). By default, the halo region is determined using the Euclidean metric. To determine the halo region using the square metric, add the -square argument. To determine the halo region using the rectangle's center point, add the -center argument.



Syntax

```
vr::withNeighbor -in <string> -value <constraint> -space <constraint>
  [-square] [-centers [-octagonalOnly | -orthogonalOnly]] [-not] [-out
  <string>]
```

Arguments

`-in`

Specifies the input layers.

`-value`

Specifies the number of neighboring rectangles that must be partially or wholly within the halo region.

`-space`

Specifies distance between the edge of the main rectangle and the edge of the halo. This value is used to calculate the halo region. By default, the Euclidean metric is used to calculate the halo region.

`-square`

Uses the square metric instead of the Euclidean metric to determine the halo region.

`-centers`

Uses the center point of the rectangle instead of the Euclidean metric to determine the halo region.

`-octagonalOnly`

Specifies the center points of rectangles must be aligned at multiples of 45°.

`-orthogonalOnly`

Specifies the center points of rectangles must be aligned orthogonally.

`-not`

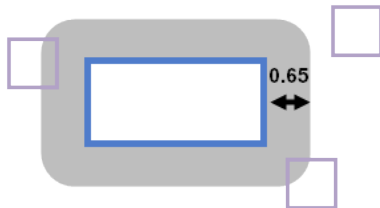
The complement of the operation.

`-out`

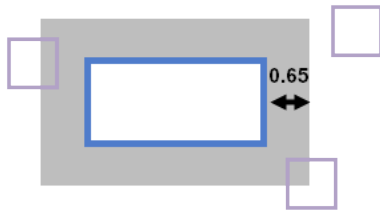
Specifies the output layer.

Examples

```
vr::withNeighbor -in {Layer1} -value {==2} -space {<=0.65}
```



```
vr::withNeighbor -in {Layer1} -value {==2} -space {<=0.65} -square
```



vr::withNet

Description

Selects all inLayer polygons that belong to the net having the specified name.

Syntax

```
vr::withNet -in <string> -name <string> [-not] [-out <string>]
```

Arguments

`-in`

Specifies the input layer.

`-name`

Specifies the name of nets.

`-not`

Complement of the operation is output.

`-out`

Specifies the output layer.

vr::withProperty

Description

Creates a new layer by copying some shapes and associated properties from another layer.

Syntax

```
vr::withProperty -in <string> [property_definition] [-nodal] [-out  
  <string>]
```

where property_definition is:

```
-def {{-expr <string> [-not] [-value <string>]}}  
  {-expr <string> [-not] [-value <string>]}}...
```

Arguments

-in

Specifies one or more input layers.

-expr

Specifies the property values through expressions.

-not

Negates the constraint specified by -value.

-nodal

Specifies the cluster method. A cluster is a set of layout objects with the same node number on primary and secondary layers. The spatial relations between the objects are ignored.

-value

Specifies the constraints that must be met.

-out

Specifies the output layer.

Examples

In the following example, the ratio of the M1 shapes and M2 shapes is used as the criteria for determining which M1 and M2 shapes will be copied to the newly created layer. If the ratio is greater than 1, then the M1 and M2 shapes and their properties will be copied.

```
vr::withProperty -in {M1 M2} -def { -expr {==AREA(M1)/AREA(M2)} -value  
  >1}
```

vr::withSpace

Description

Performs measurement and generates results that are similar to dimension check commands.

Syntax

```
vr::withSpace -in <string> -value <constraint> {-byExt | -byInt | -byEnc  
| -all} [-posEdgeA | -posEdgeB] [-horizontal] [-vertical] [-byLayer  
<string>] [-measureAll] [-connected | -notConnected] [-grid step] [-out  
<string>]
```

Arguments

-in

Specifies the input layers.

-value

Specifies the distance constraint.

-byExt -byInt -byEnc -all

Specifies the type of measurement.

-posEdgeA -posEdgeB

Positive edge output.

-horizontal

The distances between edges are measured horizontally.

-vertical

The distances between edges are measured vertically.

-byLayer

Edges on the specified layer3 are considered as shielding edges.

-measureAll

Ignore polygon containment criteria when two input layers are specified.

-connected

Restricts the output to edges that are on the same net.

-notConnected

Restricts the output to edges that are on the different net.

`-grid`

Vertical or horizontal edge pairs separated by a multiple of step are not output.

`-out`

Specifies the output layer.

vr::withText

Description

Selects polygons from inLayer intersecting the positions of text objects having the specified name.

Syntax

```
vr::withText -in <string> -name <string> [-text <string>] [-primaryOnly]
             [-caseSensitive] [-out <string>]
```

Arguments

`-in`

Specifies the input layers.

`-name`

Specifies the required name of a free-standing text object.

`-text`

Specifies the text layer where text objects are found.

`-primaryOnly`

Specifies only top-cell text is used.

`-caseSensitive`

Specifies the text names that are case sensitive.

`-out`

Specifies the output layer.

Examples

```
vr::withText -in L7 -name VDD
```

vr::withWidth

Description

Selects all regions of inlayer polygons that satisfy the width constraint.

Syntax

```
vr:: withWidth -in <string> -value <constraint> [-not] [-out <string>]
```

Arguments

-in

Specifies the input layer.

-value

Specifies constraint.

-not

The complement of the operation.

-out

Specifies output layer.

Examples

```
vr::withWidth -in M1 -value <=0.045 -out M1S  
vr::withWidth -in M1 -value >=0.045<=0.090 -out M1M
```

vr::xor

Description

Boolean xor operation.

Syntax

```
vr::xor -in <string> [-out <string>]
```

Arguments

-in

Specifies the input layer.

-out

Specifies the output layer.

Examples

```
vr::xor -in { M1 M2 }
```

Dimension Check Filters

This section lists filter commands used in dimension check operations.

The commands covered are:

- [Edge Directed](#)
- [Metric](#)
- [Polygon Containment](#)
- [Polygon Filter](#)
- [Connectivity Filter](#)
- [Orientation Filter](#)
- [Projection Filter](#)
- [Angled Filter](#)
- [Corner Filter](#)
- [Intersection Filter](#)
- [Exclude False](#)
- [Output Filter](#)
- [Reversal Filter](#)

Edge Directed

Description

Defines the constraint criteria based on specified edges.

Syntax

```
[-posEdgeA | -negEdgeA | -posEdgeB | -negEdgeB]
```

Arguments

-posEdgeA

Edges that meet the constraint from layer1 are output.

`-negEdgeA`

Edges that meet the constraint from layer2 are output.

`-posEdgeB`

Edges that do not meet the constraint from layer1 are output.

`-negEdgeB`

Edges that do not meet the constraint from layer2 are output.

Metric

Description

Defines the metric.

Syntax

`[-euclidean | -opposite | -oppositeExt <float> | -square]`

Arguments

`-euclidean`

Euclidean.

`-opposite`

Opposite.

`-oppositeExt`

Opposite extension.

`-square`

Square.

Polygon Containment

Description

Used for polygon containment.

Syntax

`[-measureAll | -measureCoin]`

Arguments

`-measureAll`

Ignore polygon containment criteria.

`-measureCoin`

Measure also coincident edges.

Polygon Filter

Description

Used for the measuring polygons.

Syntax

`[-notch | -space]`

Arguments

`-notch`

Measures the separation between the exterior-facing edges only from the same polygon.

`-space`

Measures the separation between the exterior-facing edges only from different polygons.

Connectivity Filter

Description

Used for measuring connectivity.

Syntax

`[-connected | -notConnected]`

Arguments

`-connected`

Measures only edges from polygons that belong to the same net.

`-connected`

Measures only edges from polygons that belong to the different net.

Orientation Filter

Description

Used for orientation.

Syntax

```
[-acuteAlso | -acuteOnly | -notAcute] [-parallelAlso | -parallelOnly  
| -notParallel] [-perpAlso | -perpOnly | -notPerp] [-obtuseAlso |  
-obtuseOnly | -notObtuse]
```

Arguments

-acuteAlso

Measures edges with acute ($0 < \text{angle} < 90$ degrees) angle.

-acuteOnly

Measures only edges with acute angle.

-notAcute

Not to measure edges with acute angle.

-parallelAlso

Specifies to measure parallel edges in addition to non-parallel edges.

-parallelOnly

Specifies to measure only parallel edges.

-notParallel

Specifies not to measure parallel edges.

-perpAlso

Specifies to measure perpendicular edges in addition to other orientations.

-perpOnly

Specifies to measure only perpendicular edges.

-notPerp

Specifies not to measure perpendicular edges.

-obtuseAlso

Specifies to measure edges with obtuse ($0 < \text{angle} < 180$ degrees) angle.

-obtuseOnly

Specifies to only measure edges with obtuse angle.

`-notObtuse`

Specifies not to measure edges with obtuse angle.

Projection Filter

Description

Used for projection.

Syntax

```
[-proj [<constraint>] | -notProj]
```

Arguments

`-proj`

Specifies the projection constraint. The default value is ≥ 0 .

`-notProj`

Specifies to measure the separation between two edges only when neither edge projects onto the other edge.

Angled Filter

Description

Used for angled shapes.

Syntax

```
[-angled [<constraint>]]
```

Arguments

`-angled`

Specifies to measure the two edges only when the number of non-orthogonal edges in the pair meets the given constraint. The default value is > 0 .

Corner Filter

Description

Used for corner.

Syntax

```
[-cornerToCorner | -cornerToEdge | -corner | -notCorner]
```

Arguments

`-cornerToCorner`

Specifies to measure the edge clusters only if they are in a corner-to-corner configuration.

`-cornerToEdge`

Specifies to measure the edge clusters only if they are in a corner-to-edge configuration.

`-corner`

Specifies to measure the edge clusters only if they are in a corner-to-corner or corner-to-edge configuration.

`-notCorner`

Specifies to measure the edge clusters only if they are not in a corner-to-corner or corner-to-edge configuration.

Intersection Filter

Description

Used for intersection.

Syntax

```
[-abut [<constraint>]] [-overlap] [-singular] [-intersectingOnly]
```

Arguments

`-abut`

Specifies that separation between intersecting edges should also be checked. The default value is $\geq 0 < 180$.

`-overlap`

Specifies that measurement between intersecting edges at points where a polygon from one input layer crosses a polygon from the other input layer should also occur.

`-singular`

Specifies that the measurement between the corresponding sides of intersecting edges at points of polygon singularity should also be considered.

`-intersectingOnly`

Specifies to measure only between intersecting edges and to ignore non-intersecting edges.

Exclude False

Description

Excludes false errors and specifies the level of shield checking.

Syntax

```
[-excludeFalse] [-excludeShielded [1|2|3|4]]
```

Arguments

-excludeFalse

Excludes rare false errors caused by the absence of endpoint blocking edges at the correct hierarchical level.

-excludeShielded

Specifies the level of shield checking. Valid values are:

- 0—Skips shield checking. You can specify 0 only if you specify **-measureAll**.
- 1—Performs shield checking throughout design and requires that shields without violation intersect with a shield that has a violation.



- 2—Performs shield checking throughout the design without the additional requirement specified in the level-1 check.
- 3—Performs the same level of shield checking as 2.
- 4—Performs the same level of shield checking as 2.

Default: 1 if you do not specify **-excludeShield**, 4 if you specify **-excludeShield** without a value.

Output Filter

Description

Used for output.

Syntax

```
[-region | -regionExt | -regionCenterLine [<int>]]
```

Arguments

`-region`

Derived polygons are output.

`-regionExt`

Output rectangular extents of polygons generated by `-region` option.

`-regionCenterLine`

Generates derived polygons use the centerline of polygons generated by `region`.
And have width of value user units. The default value is 8.

Reversal Filter

Description

Used for reversal.

Syntax

```
[-insideAlso | -outsideAlso]
```

Arguments

`-insideAlso`

For external command, output edges that are enclosed by either `layer1` or `layer2`. For enclosure commands, outputs edges from `layer2` that are enclosed by `layer1`.

`-outsideAlso`

Only for enclosure command. Outputs edges from `layer1` that are not enclosed by `layer2`.

Information, Error, and Warning Messages

[Table 2](#) lists the information, and error or warning messages printed on console if any error occurs during execution.

Table 2 Information and Error/Warning Messages Printed on Console

Name	Syntax	Type	Description
CADE-001	%s	Information	Information
CADE-002	%s	Warning	Warning
CADE-003	%s	Error	Error
CADE-004	Invalid or extra arguments in command %s %s	Error	Reports invalid or extra arguments
CADE-005	Input: %s %s	Warning	Reports current input command
CADE-006	Usage: %s %s	Warning	Command usage
CADE-007	Exception in command \n%s\n	Warning	Exception in command occurs
CADE-008	Tcl_Eval failure : %s	Error	Tcl evaluation failure
CADE-009	Tcl_Eval failure in %s:%d	Error	Tcl evaluation failure

A

Sample Technology File

Presents a sample technology file to demonstrate the file syntax. The sample is written in Tcl.

This appendix provides a [System Technology File Sample](#).

- [System Technology File Sample](#)

System Technology File Sample

```
;CONTROLS
;*****
controls(

    ;techArray
    techArray(
        lppLib
        deviceLib
    );techArray

;techParams
techParams(
; ( parameter          value )
; ( -----          - )
    ( maskGrid          0.005 )
    ( cadGrid           0.005 )
    ( drcGrid           0.005 )
    ( mfgGrid           0.005 )
    ( scale             1.0 )
    ( MPPS              (list (list nil (quote cwW) 0.09 (quote cwL) 0.09 (quote
cwSp) 0.10 (quote cwRowSp) 0.15 (quote Cw) "cont" (quote M1) "m1" (quote
M1CwEnc) 0.005 (quote breakLayer) "psub") (list nil (quote name) "nwell" (quote
pathLayer) "diff" (quote pathWidth) 0.09 (quote pathCwEnc) 0.05 (quote
enclosureLayers) (list) (quote spacings) (list nil)) (list nil (quote name)
"sub" (quote pathLayer) "diff" (quote pathWidth) 0.09 (quote pathCwEnc) 0.05
(quote enclosureLayers) (list) (quote spacings) (list nil))) )
) ;techParams

;techPermissions
techPermissions(
; ( class              (read-only users)  (read & write users)      )
; ( -----          - )
) ;techPermissions
```

Appendix A: Sample Technology File

System Technology File Sample

```

;viewTypeUnits
viewTypeUnits(
; ( viewType          userUnit          dbuperuu          )
; ( -----          -----          -----          )
  ( maskLayout      "micron"          1000          )
  ( schematic       "inch"           160           )
  ( schematicSymbol "inch"           160           )
  ( netlist         "inch"           160           )
) ;viewTypeUnits

;mfgGridResolution
mfgGridResolution(
  ( 0.005000 )
) ;mfgGridResolution

;leMPPControls
leMPPControls (

;leMPPDefinition
; P type guard ring      name      obj-list      spacings master-index
leMPPDefinition (psubGR ("ptypeGuardRing")      (1)      0
) ; MPP Definition
; N type guard ring      name      obj-list      spacings master-index
leMPPDefinition (nwellGR ("ntypeGuardRing")      (1)      0
) ; MPP Definition

;leMPPRingObject
; P type inside Pwell name      MasterPathName      EnclosedPathNames
OffsetPathNames      SubRectangleNames      EncShapeNames
leMPPRingObject ( ptypeGuardRing "defaultPath"      (diffEncPath
pplusEncPath ) nil (defaultContacts) nil
; net name      ExposedParameters
vss userParams (name netName)
) ; MPP Ring Object
; N type inside Nwell name      MasterPathName      EnclosedPathNames      OffsetPathNames
SubRectangleNames      EncShapeNames
leMPPRingObject ( ntypeGuardRing "defaultPath"      (diffEncPath nplusEncPath
nwellEncPath) nil (defaultContacts) nil
; net name      ExposedParameters
vdd userParams (name netName)
) ; MPP Ring Object

;leMPPMasterPath
; name layer      purpose      width      path-style      conn      chop
leMPPMasterPath ("defaultPath"      "m1"      drawing 0.1      extend      t
tuserParams (name width layer purpose conn chop)
) ; Master Path

;leMPPEnclosedPath
;name layer      purpose      enclosure      path-style      conn      chop
leMPPEnclosedPath ("diffEncPath "      "diff "drawing 0.045      extend      t
nil userParams (name layer purpose enclosure conn chop)) ; Enclosed Path
;name layer      purpose      enclosure      path-style      conn      chop
leMPPEnclosedPath ("pplusEncPath"      "pplus "drawing 0.065      extend      t
nil userParams (name layer purpose enclosure conn chop)
) ; Enclosed Path
;name layer      purpose      enclosure      path-style      conn      chop
leMPPEnclosedPath ("nplusEncPath"      "nplus "drawing 0.065      extend
t nil userParams (name layer purpose enclosure conn chop)
) ; Enclosed Path
;name layer      purpose      enclosure      path-style      conn      chop

```

Appendix A: Sample Technology File

System Technology File Sample

```

leMPPEnclosedPath ("nwellEncPath" "nwell" "drawing" 0.275 extend t
  nil userParams (name layer purpose enclosure conn chop)
) ; Enclosed Path

;leMppOffsetPath
leMppOffsetPath(

); leMppOffsetPath

;leMPPSubRect - contacts
;name layer purpose width height conn chop enc EOL spa spaType numRows
leMPPSubRect (defaultContacts cont drawing 0.09 0.09 t t 0.005 0.005 0.1 fit
  1 userParams (name layer purpose width height conn chop minEnclosure eol
    spacing spaceType numRows)
); leMPPSubRect

;leMPPEnclosedShape
;name layer purpose leESEnclosure conn-flag leESEExposedParameters
;leMPPEnclosedShape (encNWEll nwell drawing 0.275 t
); leMPPEnclosedShape
); leMPPControls
); controls

;LAYER DEFINITION
;*****
layerDefinitions(

;techPurposes
techPurposes(
; ( PurposeName           Purpose#   Abbreviation )
; ( -----             -)
;User-Defined Purposes:
( dmy0                   0           dmy0         )
( dmy1                   1           dmy1         )
( dmy2                   2           dmy2         )
( dmy3                   3           dmy3         )
( dmy4                   4           dmy4         )
( dmy5                   5           dmy5         )
( dmy6                   6           dmy6         )
( dmy7                   7           dmy7         )
( dmy8                   8           dmy8         )
( dmy9                   9           dmy9         )
( waterMark              63          waterMark    )
( dummy                  127          dummy        )
( subnode                 221         subnode      )
( extract                 222         extract      )
;System-Reserved Purposes:
( redundant              -8           red          )
( gapFill                -7           gap          )
( annotation             -6           ann          )
( OPCAntiSerif           -5           opa          )
( OPCSerif               -4           ops          )
( slot                   -3           slt          )
( fill                   -2           fil          )
( drawing                -1           drw          )
( fatal                  223          fat          )
( critical               224          crt          )
( soCritical             225          scr          )
( soError                226          ser          )
( ackWarn                227          ack          )
( info                   228          inf          )

```

Appendix A: Sample Technology File System Technology File Sample

```

( track                229      trk          )
( blockage             230      blk          )
( warning              234      wng          )
( tool1                235      tl1          )
( tool0                236      tl0          )
( label                237      lbl          )
( flight               238      flt          )
( error                239      err          )
( annotate             240      ant          )
( drawing1             241      dr1          )
( drawing2             242      dr2          )
( drawing3             243      dr3          )
( drawing4             244      dr4          )
( drawing5             245      dr5          )
( drawing6             246      dr6          )
( drawing7             247      dr7          )
( drawing8             248      dr8          )
( drawing9             249      dr9          )
( boundary             250      bnd          )
( pin                  251      pin          )
( net                  253      net          )
( cell                 254      cel          )
( all                  255      all          )
( steiner              -842194943    stn          )
( dielectricFill       -842194946      )
( activeFill          -842194947      )
( object               -842194949      )
) ;techPurposes

;techLayers
techLayers(
; ( LayerName          Layer#      Abbreviation )
; ( -----          -)
;User-Defined Layers:
( nwell                1          nwell      )
( diff                 2          diff       )
( poly                 3          poly       )
( pplus                4          pplus     )
( nplus                5          nplus     )
( rpoly                6          rpoly     )
( cont                 7          cont      )
( m1                   8          m1       )
( m1text               9          m1text   )
( m2                   10         m2       )
( m2text               11         m2text   )
( m3                   12         m3       )
( m3text               13         m3text   )
( m4                   14         m4       )
( m4text               15         m4text   )
( v1                   16         v1       )
( v2                   17         v2       )
( v3                   18         v3       )
( exclude              19         exclude  )
( mtcap                20         mtcap    )
( mbcap                21         mbcap    )
( dnw                  22         dnw      )
( nwdmy                23         nwdmy    )
( rpdmy                24         rpdmy    )
( diode                25         diode    )
( nat                  26         nat      )
( diff25               27         diff25   )

```

Appendix A: Sample Technology File

System Technology File Sample

(rmarker	28	rmarker	)
(hrimp	29	hrimp	)
(inddmy	30	inddmy	)
(varmarker	31	varmarker	)
(rmdmy	32	rmdmy	)
(medvtn	33	medvtn	)
(medvtp	34	medvtp	)
(diff18	35	diff18	)
(diff33	36	diff33	)
(lowvtn	37	lowvtn	)
(lowvtp	38	lowvtp	)
(highvtn	39	highvtn	)
(highvtp	40	highvtp	)
(polytext	41	polytext	)
(bjtdummy	49	bjtdummy	)
(iplayer	63	iplayer	)
(psub	64	psub	)
(pindummy	65	pindummy	)
(rtermmarker	66	rtermmarker	)
(ctmdmy	67	ctmdmy	)
(momdmy	68	momdmy	)
(rfdmy	69	rfdmy	)
(boundary	127	boundary	)
(m5	70	m5	)
(m5text	71	m5text	)
(m6	72	m6	)
(m6text	73	m6text	)
(m7	74	m7	)
(m7text	75	m7text	)
(m8	76	m8	)
(m8text	77	m8text	)
(m9	78	m9	)
(m9text	79	m9text	)
(v4	80	v4	)
(v5	81	v5	)
(v6	82	v6	)
(v7	83	v7	)
(v8	84	v8	)
;System-Reserved Layers:			
(Unrouted	200	Unroute	)
(Row	201	Row	)
(Group	202	Group	)
(Cannotoccupy	203	noOccupy	)
(Canplace	204	Canplac	)
(hardFence	205	hardFnc	)
(softFence	206	softFnc	)
(y0	207	y0	)
(y1	208	y1	)
(y2	209	y2	)
(y3	210	y3	)
(y4	211	y4	)
(y5	212	y5	)
(y6	213	y6	)
(y7	214	y7	)
(y8	215	y8	)
(y9	216	y9	)
(designFlow	217	dsnFlow	)
(stretch	218	stretch	)
(edgeLayer	219	edgeLyr	)
(changedLayer	220	chngLyr	)
(unset	221	unset	)

Appendix A: Sample Technology File System Technology File Sample

```

( unknown          222      unknown      )
( spike            223      spike         )
( hiz              224      hiz           )
( resist           225      resist        )
( drive            226      drive         )
( supply           227      supply        )
( wire             228      wire          )
( pin              229      pin           )
( text             230      text          )
( device           231      device        )
( border           232      border        )
( snap             233      snap         )
( align            234      align        )
( prBoundary       235      prBndry      )
( instance         236      instnce      )
( annotate         237      anotate      )
( marker           238      marker      )
( select           239      select      )
( grid             251      grid         )
( axis             252      axis         )
( hilite           253      hilite        )
( background       254      bkground      )
) ;techLayers

;techLayerPurposePriorities
techLayerPurposePriorities(
;layers are ordered from lowest to highest priority
;( LayerName      Purpose      )
;( -----      -)
( background      drawing      )
( grid            drawing      )
( grid            drawing1     )
( grid            object       )
( exclude         blockage     )
( nwell           subnode      )
( diff            subnode      )
( poly            subnode      )
( cont            subnode      )
( nplus           subnode      )
( pplus           subnode      )
( m1              subnode      )
( v1              subnode      )
( m2              subnode      )
( v2              subnode      )
( m3              subnode      )
( v3              subnode      )
( m4              subnode      )
( v4              subnode      )
( m5              subnode      )
( v5              subnode      )
( m6              subnode      )
( v6              subnode      )
( m7              subnode      )
( v7              subnode      )
( m8              subnode      )
( v8              subnode      )
( m9              subnode      )
( psub            drawing      )
( psub            subnode      )
( dnw             drawing      )
( nwell           drawing      )

```

Appendix A: Sample Technology File System Technology File Sample

```

( nwell          net          )
( diff          drawing      )
( diff          net         )
( diff25        drawing      )
( diff18        drawing      )
( diff33        drawing      )
( pplus         drawing      )
( pplus         net         )
( nplus         drawing      )
( nplus         net         )
( nat           drawing      )
( poly          drawing      )
( poly          net         )
( poly          pin         )
( cont          drawing      )
( cont          net         )
( pindummy      drawing      )
( pindummy      pin         )
( m1            drawing      )
( m1            net         )
( m1            pin         )
( v1            drawing      )
( v1            net         )
( m2            drawing      )
( m2            net         )
( m2            pin         )
( v2            drawing      )
( v2            net         )
( m3            drawing      )
( m3            net         )
( m3            pin         )
( v3            drawing      )
( v3            net         )
( m4            drawing      )
( m4            net         )
( m4            pin         )
( v4            drawing      )
( v4            net         )
( m5            drawing      )
( m5            net         )
( m5            pin         )
( v5            drawing      )
( v5            net         )
( m6            drawing      )
( m6            net         )
( m6            pin         )
( v6            drawing      )
( v6            net         )
( m7            drawing      )
( m7            net         )
( m7            pin         )
( v7            drawing      )
( v7            net         )
( m8            drawing      )
( m8            net         )
( m8            pin         )
( v8            drawing      )
( v8            net         )
( m9            drawing      )
( m9            net         )
( m9            pin         )

```


Appendix A: Sample Technology File

System Technology File Sample

```

( diff          dummy      )
( poly          dummy      )
( m1            dummy      )
( m2            dummy      )
( m3            dummy      )
( m4            dummy      )
( m5            dummy      )
( m6            dummy      )
( m7            dummy      )
( m8            dummy      )
( m9            dummy      )
( m1            blockage   )
( m2            blockage   )
( m3            blockage   )
( m4            blockage   )
( m5            blockage   )
( m6            blockage   )
( m7            blockage   )
( m8            blockage   )
( m9            blockage   )
( m1            slot       )
( m2            slot       )
( m3            slot       )
( m4            slot       )
( m5            slot       )
( m6            slot       )
( m7            slot       )
( m8            slot       )
( m9            slot       )
( cont          boundary   )
( medvtp        drawing    )
( medvtn        drawing    )
( lowvtp        drawing    )
( lowvtn        drawing    )
( highvtp       drawing    )
( highvtn       drawing    )
( rmarker       drawing    )
( rtermmarker   drawing    )
( rpoly         drawing    )
( hrimp         drawing    )
( mtcap         drawing    )
( mbcap         drawing    )
( varmarker     drawing    )
( inddmy        drawing    )
( bjtdummy      drawing    )
( nwdmy         drawing    )
( rmdmy         drawing    )
( ctmdmy        drawing    )
( rfdmy         drawing    )
( diode         drawing    )
( momdmy        dmy1       )
( momdmy        dmy2       )
( momdmy        dmy3       )
( momdmy        dmy4       )
( momdmy        dmy5       )
( momdmy        dmy6       )
( momdmy        dmy7       )
( momdmy        dmy8       )
( momdmy        dmy9       )
( m1text        drawing    )
( m2text        drawing    )

```

Appendix A: Sample Technology File System Technology File Sample

```
( m3text          drawing )
( m4text          drawing )
( m5text          drawing )
( m6text          drawing )
( m7text          drawing )
( m8text          drawing )
( m9text          drawing )
( polytext        drawing )
( iplayer         waterMark )
( annotate        drawing )
( annotate        drawing1 )
( annotate        drawing2 )
( annotate        drawing3 )
( annotate        drawing4 )
( annotate        drawing5 )
( annotate        drawing6 )
( annotate        drawing7 )
( annotate        drawing8 )
( annotate        drawing9 )
( instance        drawing )
( instance        label )
( prBoundary      drawing )
( prBoundary      boundary )
( prBoundary      label )
( align           drawing )
( hardFence       drawing )
( softFence       drawing )
( text            drawing )
( text            drawing1 )
( text            drawing2 )
( border          drawing )
( border          boundary )
( device          drawing )
( device          label )
( device          drawing1 )
( device          drawing2 )
( device          annotate )
( wire            drawing )
( wire            label )
( wire            flight )
( pin             label )
( pin             drawing )
( pin             annotate )
( axis            drawing )
( edgeLayer       drawing )
( edgeLayer       pin )
( snap            drawing )
( snap            boundary )
( stretch        drawing )
( y0              drawing )
( y1              drawing )
( y2              drawing )
( y3              drawing )
( y4              drawing )
( y5              drawing )
( y6              drawing )
( y7              drawing )
( y8              drawing )
( y9              drawing )
( y0              flight )
( y1              flight )
```

Appendix A: Sample Technology File

System Technology File Sample

```
( y2          flight      )
( y3          flight      )
( y4          flight      )
( y5          flight      )
( y6          flight      )
( y7          flight      )
( y8          flight      )
( y9          flight      )
( hilite      drawing     )
( hilite      drawing1    )
( hilite      drawing2    )
( hilite      drawing3    )
( hilite      drawing4    )
( hilite      drawing5    )
( hilite      drawing6    )
( hilite      drawing7    )
( hilite      drawing8    )
( hilite      drawing9    )
( select      drawing     )
( drive       drawing     )
( hiz         drawing     )
( resist      drawing     )
( spike       drawing     )
( supply      drawing     )
( unknown     drawing     )
( unset       drawing     )
( designFlow  drawing     )
( designFlow  drawing1    )
( designFlow  drawing2    )
( designFlow  drawing3    )
( designFlow  drawing4    )
( designFlow  drawing5    )
( designFlow  drawing6    )
( designFlow  drawing7    )
( designFlow  drawing8    )
( designFlow  drawing9    )
( changedLayer tool0      )
( changedLayer tool1      )
( marker      warning     )
( marker      error       )
( marker      annotate     )
( marker      info        )
( marker      ackWarn     )
( marker      soError     )
( marker      soCritical   )
( marker      critical     )
( marker      fatal       )
( Row         drawing     )
( Row         label       )
( Row         boundary    )
( Group       drawing     )
( Group       label       )
( Group       boundary    )
( Cannotoccupy drawing    )
( Cannotoccupy boundary   )
( Canplace    drawing     )
( Unrouted    drawing     )
( Unrouted    drawing1    )
( Unrouted    drawing2    )
( Unrouted    drawing3    )
( Unrouted    drawing4    )
```

Appendix A: Sample Technology File System Technology File Sample

```

( Unrouted          drawing5 )
( Unrouted          drawing6 )
( Unrouted          drawing7 )
( Unrouted          drawing8 )
( Unrouted          drawing9 )
( Unrouted          track    )
( PinBorder         substrate )
( PinBorder         blockage )
( PinBorder         control  )
( PinBorder         softBdry )
( PinBorder         warning  )
( PinBorder         tool     )
( PinBorder         label    )
( PinBorder         flight   )
( PinBorder         annotate  )
( PinBorder         boundary )
( PinBorder         pin      )
( PinBorder         drawing  )
( PinBorder         net      )
( PinBorder         cell     )
( border            track    )
) ;techLayerPurposePriorities

;techDisplays
techDisplays(
; ( LayerName      Purpose      Packet      Vis Sel Con2ChgLy DrgEnbl Valid )
; ( -----      -
( background      drawing      background  t nil t nil nil )
( grid            drawing      grid        t nil t nil nil )
( grid            drawing1     grid1       t nil t nil nil )
( grid            object       FinGrid     t nil t nil nil )
( annotate        drawing      annotate     t t t t nil )
( annotate        drawing1     annotate1    t t t t nil )
( annotate        drawing2     annotate2    t t t t nil )
( annotate        drawing3     annotate3    t t t t nil )
( annotate        drawing4     annotate4    t t t t nil )
( annotate        drawing5     annotate5    t t t t nil )
( annotate        drawing6     annotate6    t t t t nil )
( annotate        drawing7     annotate7    t t t t nil )
( annotate        drawing8     annotate8    t t t t nil )
( annotate        drawing9     annotate9    nil t t t nil )
( instance        drawing      instance    t t t t nil )
( instance        label        instanceLbl  t t t t nil )
( prBoundary      drawing      prBoundary  t t t t nil )
( prBoundary      boundary     prBoundaryBnd t t t t nil )
( prBoundary      label        prBoundaryLbl t t t t nil )
( prBoundary      object       prBoundaryObject t t nil nil nil )
( align           drawing      align       t t t t nil )
( hardFence       drawing      hardFence   t t t t nil )
( softFence       drawing      softFence   t t t t nil )
( dnw             drawing      dnw         t t nil t t )
( nwell           drawing      nwell       t t nil t t )
( diff            drawing      sldiff      t t nil t t )
( medvtp          drawing      medvtp      t t nil t t )
( medvtn          drawing      medvtn      t t nil t t )
( lowvtp          drawing      lowvtp      t t nil t t )
( lowvtn          drawing      lowvtn      t t nil t t )
( highvtp         drawing      highvtp     t t nil t t )
( highvtn         drawing      highvtn     t t nil t t )
( nat             drawing      nat         t t nil t t )
( diff25          drawing      diff25      t t nil t t )

```

Appendix A: Sample Technology File System Technology File Sample

```

( diff18      drawing      diff18      t t nil t t )
( diff33      drawing      diff33      t t nil t t )
( poly        drawing      poly        t t nil t t )
( poly        pin          polyPin     t t t   t t )
( pplus       drawing      pplus      t t nil t t )
( nplus       drawing      nplus      t t nil t t )
( rpoly       drawing      rpoly      t t nil t t )
( cont        drawing      cont       t t nil t t )
( cont        boundary     contb      t t nil t t )
( m1          drawing      m1        t t nil t t )
( pindummy    drawing      pindummy   nil nil nil nil nil )
( v1          drawing      v1        t t nil t t )
( m2          drawing      m2        t t nil t t )
( v2          drawing      v2        t t nil t t )
( m3          drawing      m3        t t nil t t )
( v3          drawing      v3        t t nil t t )
( m4          drawing      m4        t t nil t t )
( v4          drawing      v4        t t nil t t )
( m5          drawing      m5        t t nil t t )
( v5          drawing      v5        t t nil t t )
( m6          drawing      m6        t t nil t t )
( v6          drawing      v6        t t nil t t )
( m7          drawing      m7        t t nil t t )
( v7          drawing      v7        t t nil t t )
( m8          drawing      m8        t t nil t t )
( v8          drawing      v8        t t nil t t )
( m9          drawing      m9        t t nil t t )
( psub        drawing      psub      t t nil t t )
( psub        subnode     psub      t t nil t t )
( m1          pin          m1Pin      t t nil t t )
( pindummy    pin          pindummyPin nil nil nil nil nil )
( m2          pin          m2Pin      t t nil t t )
( m3          pin          m3Pin      t t nil t t )
( m4          pin          m4Pin      t t nil t t )
( m5          pin          m5Pin      t t nil t t )
( m6          pin          m6Pin      t t nil t t )
( m7          pin          m7Pin      t t nil t t )
( m8          pin          m8Pin      t t nil t t )
( m9          pin          m9Pin      t t nil t t )
( m1          blockage    m1Blk     t t nil t t )
( m2          blockage    m2Blk     t t nil t t )
( m3          blockage    m3Blk     t t nil t t )
( m4          blockage    m4Blk     t t nil t t )
( m5          blockage    m5Blk     t t nil t t )
( m6          blockage    m6Blk     t t nil t t )
( m7          blockage    m7Blk     t t nil t t )
( m8          blockage    m8Blk     t t nil t t )
( m9          blockage    m9Blk     t t nil t t )
( m1          boundary    m1b       t t nil t nil )
( m2          boundary    m2b       t t nil t nil )
( m3          boundary    m3b       t t nil t nil )
( m4          boundary    m4b       t t nil t nil )
( m5          boundary    m5b       t t nil t nil )
( m6          boundary    m6b       t t nil t nil )
( m7          boundary    m7b       t t nil t nil )
( m8          boundary    m8b       t t nil t nil )
( m9          boundary    m9b       t t nil t nil )
( rmarker     drawing      rpdmy    t t nil t t )
( rtermmarker drawing      rtdmy    t t nil t t )
( v1          boundary    v1b       t t nil t nil )
( v2          boundary    v2b       t t nil t nil )

```

Appendix A: Sample Technology File System Technology File Sample

```
( v3          boundary      v3b          t t nil t nil )
( v4          boundary      v4b          t t nil t nil )
( v5          boundary      v5b          t t nil t nil )
( v6          boundary      v6b          t t nil t nil )
( v7          boundary      v7b          t t nil t nil )
( v8          boundary      v8b          t t nil t nil )
( hrimp       drawing       hrimp        t t nil t t )
( mtcap       drawing       mtcap        t t nil t t )
( mbcap       drawing       mbcap        t t nil t t )
( varmarker   drawing       varmarker    t t nil t t )
( inddmy      drawing       inddmy       t t nil t t )
( bjtdummy    drawing       bjtdummy     t t nil t t )
( nwdmy       drawing       nwdmy        t t nil t t )
( rmdmy       drawing       rmdmy        t t nil t t )
( ctmdmy      drawing       ctmdmy       t t nil t t )
( rfdmy       drawing       rfdmy        t t nil t t )
( diode       drawing       diode        t t nil t t )
( exclude     blockage     exclude     t t nil t t )
( diff        dummy        diffDmy      t t nil t t )
( poly        dummy        polyDmy      t t nil t t )
( m1          dummy        m1Pin        t t nil t t )
( m2          dummy        m2Pin        t t nil t t )
( m3          dummy        m3Pin        t t nil t t )
( m4          dummy        m4Pin        t t nil t t )
( m5          dummy        m5Pin        t t nil t t )
( m6          dummy        m6Pin        t t nil t t )
( m7          dummy        m7Pin        t t nil t t )
( m8          dummy        m8Pin        t t nil t t )
( m9          dummy        m9Pin        t t nil t t )
( momdmy      dmy1         momDmy1   t t nil t t )
( momdmy      dmy2         momDmy2   t t nil t t )
( momdmy      dmy3         momDmy3   t t nil t t )
( momdmy      dmy4         momDmy4   t t nil t t )
( momdmy      dmy5         momDmy5   t t nil t t )
( momdmy      dmy6         momDmy6   t t nil t t )
( momdmy      dmy7         momDmy7   t t nil t t )
( momdmy      dmy8         momDmy8   t t nil t t )
( momdmy      dmy9         momDmy9   t t nil t t )
( iplayer     waterMark    ipLayer   t t nil t t )
( m1          slot        m1slot      t t nil t t )
( m2          slot        m2slot      t t nil t t )
( m3          slot        m3slot      t t nil t t )
( m4          slot        m4slot      t t nil t t )
( m5          slot        m5slot      t t nil t t )
( m6          slot        m6slot      t t nil t t )
( m7          slot        m7slot      t t nil t t )
( m8          slot        m8slot      t t nil t t )
( m9          slot        m9slot      t t nil t t )
( m1text      drawing      m1txt      t t nil t t )
( m2text      drawing      m2txt      t t nil t t )
( m3text      drawing      m3txt      t t nil t t )
( m4text      drawing      m4txt      t t nil t t )
( m5text      drawing      m5txt      t t nil t t )
( m6text      drawing      m6txt      t t nil t t )
( m7text      drawing      m7txt      t t nil t t )
( m8text      drawing      m8txt      t t nil t t )
( m9text      drawing      m9txt      t t nil t t )
( polytext    drawing      polytxt    t t nil t t )
( text        drawing      text        t t t t t )
( text        drawing1     text1      t t t t nil )
( text        drawing2     text2      t t t t nil )
```

Appendix A: Sample Technology File

System Technology File Sample

```

( border      drawing      border      t t t t nil )
( device      drawing      device      t t t t nil )
( device      label        deviceLbl   t t t t nil )
( device      drawing1     device1    t t t t nil )
( device      drawing2     device2    t t t t nil )
( device      annotate     deviceAnt   t t t t nil )
( wire        drawing      wire        t t t t nil )
( wire        label        wireLbl     t t t t nil )
( wire        flight       wireFlt     t t t t nil )
( pin         label        pinLbl      t t t t nil )
( pin         drawing      pin         t t t t nil )
( pin         annotate     pinAnt      t t t t nil )
( axis        drawing      axis        t nil t t nil )
( edgeLayer   drawing      edgeLayer   t t t t nil )
( edgeLayer   pin         edgeLayerPin t t t t nil )
( snap        drawing      snap        t t t t nil )
( stretch    drawing      stretch    t t t t nil )
( y0          drawing      y0          t t t t nil )
( y1          drawing      y1          t t t t nil )
( y2          drawing      y2          t t t t nil )
( y3          drawing      y3          t t t t nil )
( y4          drawing      y4          t t t t nil )
( y5          drawing      y5          t t t t nil )
( y6          drawing      y6          t t t t nil )
( y7          drawing      y7          t t t t nil )
( y8          drawing      y8          t t t t nil )
( y9          drawing      y9          t t t t nil )
( hilite      drawing      hilite      t t t t nil )
( hilite      drawing1     hilite1    t t t t nil )
( hilite      drawing2     hilite2    t t t t nil )
( hilite      drawing3     hilite3    t t t t nil )
( hilite      drawing4     hilite4    t t t t nil )
( hilite      drawing5     hilite5    t t t t nil )
( hilite      drawing6     hilite6    t t t t nil )
( hilite      drawing7     hilite7    t t t t nil )
( hilite      drawing8     hilite8    t t t t nil )
( hilite      drawing9     hilite9    t t t t nil )
( select      drawing      select     t t t t nil )
( drive       drawing      drive       t t t t nil )
( hiz         drawing      hiz         t t t t nil )
( resist      drawing      resist     t t t t nil )
( spike       drawing      spike      t t t t nil )
( supply      drawing      supply     t t t t nil )
( unknown     drawing      unknown    t t t t nil )
( unset       drawing      unset      t t t t nil )
( designFlow  drawing      designFlow t t t nil nil )
( designFlow  drawing1     designFlow1 t t t nil nil )
( designFlow  drawing2     designFlow2 t t t nil nil )
( designFlow  drawing3     designFlow3 t t t nil nil )
( designFlow  drawing4     designFlow4 t t t nil nil )
( designFlow  drawing5     designFlow5 t t t nil nil )
( designFlow  drawing6     designFlow6 t t t nil nil )
( designFlow  drawing7     designFlow7 t t t nil nil )
( designFlow  drawing8     designFlow8 t t t nil nil )
( designFlow  drawing9     designFlow9 t t t nil nil )
( changedLayer tool0       changedLayerTl0 nil nil t nil nil )
( changedLayer tool1       changedLayerTl1 nil nil t nil nil )
( marker      warning      markerWarn t t t t nil )
( marker      error        markerErr  t t t t nil )
( Row         drawing      Row         t t t t nil )
( Row         label        RowLbl      t nil t t nil )

```

Appendix A: Sample Technology File

System Technology File Sample

(Group	drawing	Group	t t t t nil)
(Group	label	GroupLbl	t nil t t nil)
(Cannotoccupy	drawing	Cannotoccupy	t t t t nil)
(Cannotoccupy	boundary	CannotoccupyBnd	t t t t nil)
(Canplace	drawing	Canplace	t t t t nil)
(Unrouted	drawing	Unrouted	t t t t nil)
(Unrouted	drawing1	Unrouted1	t t t t nil)
(Unrouted	drawing2	Unrouted2	t t t t nil)
(Unrouted	drawing3	Unrouted3	t t t t nil)
(Unrouted	drawing4	Unrouted4	t t t t nil)
(Unrouted	drawing5	Unrouted5	t t t t nil)
(Unrouted	drawing6	Unrouted6	t t t t nil)
(Unrouted	drawing7	Unrouted7	t t t t nil)
(Unrouted	drawing8	Unrouted8	t t t t nil)
(Unrouted	drawing9	Unrouted9	t t t t nil)
(marker	annotate	markerAno	t t t t nil)
(marker	info	markerInf	t t t t nil)
(marker	ackWarn	markerAck	t t t t nil)
(marker	soError	markerSer	t t t t nil)
(marker	soCritical	markerScr	t t t t nil)
(marker	critical	markerCrt	t t t t nil)
(marker	fatal	markerFat	t t t t nil)
(snap	boundary	snap	t nil t t nil)
(Row	boundary	RowBnd	t t t t nil)
(Unrouted	track	UnroutedTrk	t t t t nil)
(Group	boundary	GroupBnd	t nil t t nil)
(y0	flight	y0Flt	t t t t nil)
(y1	flight	y1Flt	t t t t nil)
(y2	flight	y2Flt	t t t t nil)
(y3	flight	y3Flt	t t t t nil)
(y4	flight	y4Flt	t t t t nil)
(y5	flight	y5Flt	t t t t nil)
(y6	flight	y6Flt	t t t t nil)
(y7	flight	y7Flt	t t t t nil)
(y8	flight	y8Flt	t t t t nil)
(y9	flight	y9Flt	t t t t nil)
(border	boundary	area	t nil t t nil)
(border	track	track	t t t t nil)
(snpsTrunk	drawing	routeTrunk	t t nil nil nil)
(snpsTrunk	drawing1	routeTrunk1	t t nil nil nil)
(snpsTrunk	drawing2	routeTrunk2	t t nil nil nil)
(snpsTrunk	drawing3	routeTrunk3	t t nil nil nil)
(snpsTrunk	drawing4	routeTrunk4	t t nil nil nil)
(snpsTrunk	drawing5	routeTrunk5	t t nil nil nil)
(snpsTrunk	drawing6	routeTrunk6	t t nil nil nil)
(snpsTrunk	drawing7	routeTrunk7	t t nil nil nil)
(snpsTrunk	drawing8	routeTrunk8	t t nil nil nil)
(snpsTrunk	drawing9	routeTrunk9	t t nil nil nil)
(snpsTapoff	drawing	routeTapoff	t t nil nil nil)
(snpsTapoff	drawing1	routeTapoff1	t t nil nil nil)
(snpsTapoff	drawing2	routeTapoff2	t t nil nil nil)
(snpsTapoff	drawing3	routeTapoff3	t t nil nil nil)
(snpsTapoff	drawing4	routeTapoff4	t t nil nil nil)
(snpsTapoff	drawing5	routeTapoff5	t t nil nil nil)
(snpsTapoff	drawing6	routeTapoff6	t t nil nil nil)
(snpsTapoff	drawing7	routeTapoff7	t t nil nil nil)
(snpsTapoff	drawing8	routeTapoff8	t t nil nil nil)
(snpsTapoff	drawing9	routeTapoff9	t t nil nil nil)
(track	boundary	track	t t nil nil nil)
(nwell	net	nwell	t t nil t nil)
(nwell	subnode	nwell	t t nil t nil)

Appendix A: Sample Technology File System Technology File Sample

```

( diff      net      sldiff      t t nil t nil )
( diff      subnode  sldiff      t t nil t nil )
( poly      net      polynet     t t nil t nil )
( poly      subnode  poly       t t nil t nil )
( cont      net      contnet     t t nil t nil )
( cont      subnode  cont       t t nil t nil )
( nplus     net      nplus      t t nil t nil )
( nplus     subnode  nplus      t t nil t nil )
( pplus     net      pplus      t t nil t nil )
( pplus     subnode  pplus      t t nil t nil )
( m1        net      m1net      t t nil t nil )
( m1        subnode  m1         t t nil t nil )
( v1        net      v1net      t t nil t nil )
( v1        subnode  v1         t t nil t nil )
( m2        net      m2net      t t nil t nil )
( m2        subnode  m2         t t nil t nil )
( v2        net      v2net      t t nil t nil )
( v2        subnode  v2         t t nil t nil )
( m3        net      m3net      t t nil t nil )
( m3        subnode  m3         t t nil t nil )
( v3        net      v3net      t t nil t nil )
( v3        subnode  v3         t t nil t nil )
( m4        net      m4net      t t nil t nil )
( m4        subnode  m4         t t nil t nil )
( v4        net      v4net      t t nil t nil )
( v4        subnode  v4         t t nil t nil )
( m5        net      m5         t t nil t nil )
( m5        subnode  m5         t t nil t nil )
( v5        net      v5net      t t nil t nil )
( v5        subnode  v5         t t nil t nil )
( m6        net      m6         t t nil t nil )
( m6        subnode  m6         t t nil t nil )
( v6        net      v6net      t t nil t nil )
( v6        subnode  v6         t t nil t nil )
( m7        net      m7         t t nil t nil )
( m7        subnode  m7         t t nil t nil )
( v7        net      v7net      t t nil t nil )
( v7        subnode  v7         t t nil t nil )
( m8        net      m8         t t nil t nil )
( m8        subnode  m8         t t nil t nil )
( v8        net      v8net      t t nil t nil )
( v8        subnode  v8         t t nil t nil )
( m9        net      m9         t t nil t nil )
( m9        subnode  m9         t t nil t nil )
) ;techDisplays

;techLayerProperties
techLayerProperties (
; ( PropName      Layer1 [ Layer2 ]      PropValue )
; ( -----      ----- )
( sheetResistance nwell      550.0 )
( sheetResistance poly       6.34 )
( sheetResistance diff       5.525 )
( sheetResistance v1         0.0016 )
( sheetResistance v2         0.0016 )
( sheetResistance v3         0.0016 )
( sheetResistance m1         0.115 )
( sheetResistance m2         0.086 )
( sheetResistance m3         0.086 )
( sheetResistance m4         0.086 )
( sheetResistance m5         0.086 )

```

Appendix A: Sample Technology File

System Technology File Sample

```

( sheetResistance      m6              0.086 )
( sheetResistance      m7              0.086 )
( sheetResistance      m8              0.086 )
( sheetResistance      m9              0.04  )
( areaCapacitance      poly            6.950000e-05 )
( edgeCapacitance      poly            4.503000e-11 )
( areaCapacitance      m1              2.620000e-05 )
( edgeCapacitance      m1              3.163000e-11 )
( areaCapacitance      m2              1.180000e-05 )
( edgeCapacitance      m2              2.345000e-11 )
( areaCapacitance      m3              7.870000e-06 )
( edgeCapacitance      m3              1.028000e-11 )
( areaCapacitance      m4              7.870000e-06 )
( edgeCapacitance      m4              1.028000e-11 )
( areaCapacitance      m5              7.870000e-06 )
( edgeCapacitance      m5              1.028000e-11 )
( areaCapacitance      m6              7.870000e-06 )
( edgeCapacitance      m6              1.028000e-11 )
( areaCapacitance      m7              7.870000e-06 )
( edgeCapacitance      m7              1.028000e-11 )
( areaCapacitance      m8              7.870000e-06 )
( edgeCapacitance      m8              1.028000e-11 )
( areaCapacitance      m9              4.9980000e-06 )
( edgeCapacitance      m8              1.110000e-11 )
) ;techLayerProperties

;techDerivedLayers
techDerivedLayers(
; ( DerivedLayerName      #      composition )
; ( -----            ----- )
) ;techDerivedLayers
) ;layerDefinitions

;LAYER RULES
;*****
layerRules(

;functions
functions(
; ( layer      function      [maskNumber])
; ( -----            ----- )
( psub      "pWell"      1      )
( nwell     "nWell"      2      )
( diff      "diffusion"   3      )
( diff18    "diffusion"   4      )
( diff25    "diffusion"   5      )
( diff33    "diffusion"   6      )
( pplus     "pImplant"    7      )
( nplus     "nImplant"    8      )
( rpoly     "other"       9      )
( poly      "poly"        10     )
( cont      "cut"         11     )
( m1        "metal"       12     )
( v1        "cut"         13     )
( m2        "metal"       14     )
( v2        "cut"         15     )
( m3        "metal"       16     )
( v3        "cut"         17     )
( m4        "metal"       18     )
( v4        "cut"         19     )
( m5        "metal"       20     )

```

Appendix A: Sample Technology File System Technology File Sample

```

( v5          "cut"      21          )
( m6          "metal"    22          )
( v6          "cut"      23          )
( m7          "metal"    24          )
( v7          "cut"      25          )
( m8          "metal"    26          )
( v8          "cut"      27          )
( m9          "metal"    28          )
( mtcap       "other"    29          )
( mbcap       "other"    30          )
( rmarker     "other"    31          )
( rmdmy       "other"    32          )
( ctmdmy      "other"    33          )
( rfdmy       "other"    34          )
( iplayer     "other"    35          )
) ;functions

;mfgResolutions
mfgResolutions(
; ( layer          mfgResolution )
; ( -----          ----- )
( dnw             0.005 )
( nwell           0.005 )
( diff            0.005 )
( medvtp          0.005 )
( medvtn          0.005 )
( lowvtp          0.005 )
( lowvtn          0.005 )
( highvtp         0.005 )
( highvtn         0.005 )
( nat             0.005 )
( diff25          0.005 )
( diff18          0.005 )
( diff33          0.005 )
( poly            0.005 )
( pplus           0.005 )
( nplus           0.005 )
( rpoly           0.005 )
( cont            0.005 )
( m1              0.005 )
( v1              0.005 )
( m2              0.005 )
( v2              0.005 )
( m3              0.005 )
( v3              0.005 )
( m4              0.005 )
( v4              0.005 )
( m5              0.005 )
( v5              0.005 )
( m6              0.005 )
( v6              0.005 )
( m7              0.005 )
( v7              0.005 )
( m8              0.005 )
( v8              0.005 )
( m9              0.005 )
( rmarker         0.005 )
( rtermmarker     0.005 )
( hrimp           0.005 )
( mtcap           0.005 )
( varmarker       0.005 )

```

```

( inddmy                0.005 )
( bjtdummy              0.005 )
( nwdmy                 0.005 )
( rmdmy                 0.005 )
( ctmdmy                0.005 )
( rfdmy                 0.005 )
( diode                 0.005 )
( momdmy                0.005 )
( exclude               0.005 )
( psub                  0.005 )
( iplayer               0.005 )
) ;mfgResolutions

;routingDirections
routingDirections( ;( layer                direction      )
; ( ----- )
( m1 horizontal )
( m2 vertical   )
( m3 horizontal )
( m4 vertical   )
) ;routingDirections

;VIADEFS
;*****
viaDefs(

;standardViaDefs
standardViaDefs(
; ( viaDefName layer1 layer2 (cutLayer cutWidth cutHeight [resistancePerCut])
;   (cutRows cutCol (cutSpace))
;   (layer1Enc) (layer2Enc) (layer1Offset) (layer2Offset) (origOffset)
;   [implant1 (implant1Enc) [implant2 (implant2Enc)])])
; ( ----- )
( via89      m8      m9      ("v8" 0.13 0.13 0.0016)      (1 1 (0.15 0.15))
(0.01 0.04) (0.04 0.01) (0.0 0.0) (0.0 0.0) (0.0 0.0) )
( via78      m7      m8      ("v7" 0.13 0.13 0.0016)      (1 1 (0.15 0.15))
(0.04 0.01) (0.01 0.04) (0.0 0.0) (0.0 0.0) (0.0 0.0) )
( via67      m6      m7      ("v6" 0.13 0.13 0.0016)      (1 1 (0.15 0.15))
(0.01 0.04) (0.04 0.01) (0.0 0.0) (0.0 0.0) (0.0 0.0) )
( via56      m5      m6      ("v5" 0.13 0.13 0.0016)      (1 1 (0.15 0.15))
(0.04 0.01) (0.01 0.04) (0.0 0.0) (0.0 0.0) (0.0 0.0) )
( via45      m4      m5      ("v4" 0.13 0.13 0.0016)      (1 1 (0.15 0.15))
(0.01 0.04) (0.04 0.01) (0.0 0.0) (0.0 0.0) (0.0 0.0) )
( via34      m3      m4      ("v3" 0.13 0.13 0.0016)      (1 1 (0.15 0.15))
(0.04 0.01) (0.01 0.04) (0.0 0.0) (0.0 0.0) (0.0 0.0) )
( via23      m2      m3      ("v2" 0.13 0.13 0.0016)      (1 1 (0.15 0.15))
(0.01 0.04) (0.04 0.01) (0.0 0.0) (0.0 0.0) (0.0 0.0) )
( via12      m1      m2      ("v1" 0.13 0.13 0.0016)      (1 1 (0.15 0.15))
(0.04 0.01) (0.01 0.04) (0.0 0.0) (0.0 0.0) (0.0 0.0) )
( via89_swap m8      m9      ("v8" 0.13 0.13 0.0016)      (1 1 (0.15
0.15)) (0.04 0.01) (0.01 0.04) (0.0 0.0) (0.0 0.0) (0.0 0.0) )
( via78_swap m7      m8      ("v7" 0.13 0.13 0.0016)      (1 1 (0.15
0.15)) (0.01 0.04) (0.04 0.01) (0.0 0.0) (0.0 0.0) (0.0 0.0) )
( via67_swap m6      m7      ("v6" 0.13 0.13 0.0016)      (1 1 (0.15
0.15)) (0.04 0.01) (0.01 0.04) (0.0 0.0) (0.0 0.0) (0.0 0.0) )
( via56_swap m5      m6      ("v5" 0.13 0.13 0.0016)      (1 1 (0.15
0.15)) (0.01 0.04) (0.04 0.01) (0.0 0.0) (0.0 0.0) (0.0 0.0) )
( via45_swap m4      m5      ("v4" 0.13 0.13 0.0016)      (1 1 (0.15
0.15)) (0.04 0.01) (0.01 0.04) (0.0 0.0) (0.0 0.0) (0.0 0.0) )
( via34_swap m3      m4      ("v3" 0.13 0.13 0.0016)      (1 1 (0.15
0.15)) (0.01 0.04) (0.04 0.01) (0.0 0.0) (0.0 0.0) (0.0 0.0) )

```

```

( via23_swap      m2      m3      ("v2" 0.13 0.13 0.0016)      (1 1 (0.15
0.15))      (0.04 0.01) (0.01 0.04) (0.0 0.0) (0.0 0.0) (0.0 0.0) )
( via12_swap      m1      m2      ("v1" 0.13 0.13 0.0016)      (1 1 (0.15
0.15))      (0.01 0.04) (0.04 0.01) (0.0 0.0) (0.0 0.0) (0.0 0.0) )
( diffCont      diff      m1      ("cont" 0.09 0.09)      (1 1 (0.1 0.1))
(0.05 0.05) (0.04 0.005) (0.0 0.0) (0.0 0.0) (0.0 0.0) )
( polyCont      poly      m1      ("cont" 0.09 0.09)      (1 1 (0.1 0.1))
(0.05 0.07) (0.04 0.005) (0.0 0.0) (0.0 0.0) (0.0 0.0) )
( pdiff      diff      m1      ("cont" 0.09 0.09)      (1 1 (0.1 0.1))      (0.05
0.05) (0.005 0.04) (0.0 0.0) (0.0 0.0) (0.0 0.0)      pplus      (0.02 0.02)
)
( ndiff      diff      m1      ("cont" 0.09 0.09)      (1 1 (0.1 0.1))      (0.05
0.05) (0.005 0.04) (0.0 0.0) (0.0 0.0) (0.0 0.0)      nplus      (0.02 0.02)
)
( diffWellCont diff      m1      ("cont" 0.09 0.09)      (1 1 (0.1 0.1))
(0.05 0.05) (0.005 0.04) (0.0 0.0) (0.0 0.0) (0.0 0.0)      nplus      (0.02
0.02) nwell (0.275 0.24) )
; Line vias definition
( via89Line      m8      m9      ("v8" 0.13 0.13 0.0016)      (1 1 (0.15
0.15))      (0.04 0.01) (0.04 0.01) (0.0 0.0) (0.0 0.0) (0.0 0.0) )
( via78Line      m7      m8      ("v7" 0.13 0.13 0.0016)      (1 1 (0.15
0.15))      (0.04 0.01) (0.04 0.01) (0.0 0.0) (0.0 0.0) (0.0 0.0) )
( via67Line      m6      m7      ("v6" 0.13 0.13 0.0016)      (1 1 (0.15
0.15))      (0.04 0.01) (0.04 0.01) (0.0 0.0) (0.0 0.0) (0.0 0.0) )
( via56Line      m5      m6      ("v5" 0.13 0.13 0.0016)      (1 1 (0.15
0.15))      (0.04 0.01) (0.04 0.01) (0.0 0.0) (0.0 0.0) (0.0 0.0) )
( via45Line      m4      m5      ("v4" 0.13 0.13 0.0016)      (1 1 (0.15
0.15))      (0.04 0.01) (0.04 0.01) (0.0 0.0) (0.0 0.0) (0.0 0.0) )
( via34Line      m3      m4      ("v3" 0.13 0.13 0.0016)      (1 1 (0.15
0.15))      (0.04 0.01) (0.04 0.01) (0.0 0.0) (0.0 0.0) (0.0 0.0) )
( via23Line      m2      m3      ("v2" 0.13 0.13 0.0016)      (1 1 (0.15
0.15))      (0.04 0.01) (0.04 0.01) (0.0 0.0) (0.0 0.0) (0.0 0.0) )
( via12Line      m1      m2      ("v1" 0.13 0.13 0.0016)      (1 1 (0.15
0.15))      (0.04 0.01) (0.04 0.01) (0.0 0.0) (0.0 0.0) (0.0 0.0) )
) ;standardViaDefs

;customViaDefs
customViaDefs( ;( viaDefName libName cellName viewName layer1 layer2
resistancePerCut)
;( -----)
( via12_cust reference90RF via12_cust layout m1 m2 1.0)
( via23_cust reference90RF via23_cust layout m2 m3 1.0)
( via34_cust reference90RF via34_cust layout m3 m4 1.0)
( via45_cust reference90RF via45_cust layout m4 m5 1.0)
( via56_cust reference90RF via56_cust layout m5 m6 1.0)
( via67_cust reference90RF via67_cust layout m6 m7 1.0)
( via78_cust reference90RF via78_cust layout m7 m8 1.0)
( via89_cust reference90RF via89_cust layout m8 m9 1.0)
( pdiff_cust reference90RF pdiff_cust layout diff m1 1.0)
( ndiff_cust reference90RF ndiff_cust layout diff m1 1.0)
( diff_cust reference90RF diff_cust layout diff m1 1.0)
( poly_cust reference90RF poly_cust layout poly m1 1.0)
) ;customViaDefs
) ;viaDefs

;CONSTRAINT GROUPS
;*****
constraintGroups(
; ( group [override] )
; ( ----- )

```

Appendix A: Sample Technology File System Technology File Sample

```
( "orionDefaultExtractorSetup "nil
) ;orionDefaultExtractorSetup

;foundry
;( group [override] )
;( ----- )
( "foundry "nil

;spacings
spacings(
  ( minWidth "nwell "0.55 )
  ( minSpacing "nwell "0.55 )
  ( minArea "nwell "1.2 )
  ( minWidth "diff "0.09 )
  ( minSpacing "diff "0.09 )
  ( minArea "diff "0.05 )
  ( minWidth "poly "0.09 )
  ( minSpacing "poly "0.09 )
  ( minArea "poly "0.05 )
  ( minWidth "pplus" 0.2 )
  ( minSpacing "pplus" 0.1 )
  ( minArea "pplus" 0.08 )
  ( minWidth "nplus" 0.2 )
  ( minSpacing "nplus" 0.1 )
  ( minArea "nplus" 0.08 )
  ( minWidth "rpoly" 0.34 )
  ( minSpacing "rpoly" 0.34 )
  ( minArea "rpoly" 0.85 )
  ( minWidth "cont "0.09 )
  ( minSpacing "cont "0.1 )
  ( minWidth "m1 " 0.1 )
  ( minSpacing "m1 " 0.1 )
  ( minArea "m1 " 0.04 )
  ( minWidth "v1 "0.13 )
  ( minSpacing "v1 "0.15 )
  ( minWidth "m2 "0.15 )
  ( minSpacing "m2 "0.15 )
  ( minArea "m2 "0.06 )
  ( minWidth "v2 "0.13 )
  ( minSpacing "v2 "0.15 )
  ( minWidth "m3 "0.15 )
  ( minSpacing "m3 "0.15 )
  ( minArea "m3 "0.06 )

  ( minWidth "v3 "0.13 )
  ( minSpacing "v3 "0.15 )
  ( minWidth "m4 "0.15 )
  ( minSpacing "m4 "0.15 )
  ( minArea "m4 "0.06 )
  ( minWidth "v4 "0.13 )
  ( minSpacing "v4 "0.15 )
  ( minWidth "m5 "0.15 )
  ( minSpacing "m5 "0.15 )
  ( minArea "m5 "0.06 )
  ( minWidth "v5 "0.13 )
  ( minSpacing "v5 "0.15 )
  ( minWidth "m6 "0.15 )
  ( minSpacing "m6 "0.15 )
  ( minArea "m6 "0.06 )
  ( minWidth "v6 "0.13 )
  ( minSpacing "v6 "0.15 )
```

Appendix A: Sample Technology File System Technology File Sample

```

( minWidth          "m7"  "0.15 )
( minSpacing        "m7"  "0.15 )
( minArea           "m7"  "0.06 )
( minWidth          "v7"  "0.13 )
( minSpacing        "v7"  "0.15 )
( minWidth          "m8"  "0.15 )
( minSpacing        "m8"  "0.15 )
( minArea           "m8"  "0.06 )
( minWidth          "v8"  "0.13 )
( minSpacing        "v8"  "0.15 )
( minWidth          "m9"  "0.15 )
( minSpacing        "m9"  "0.15 )
( minArea           "m9"  "0.06 )
( minWidth          "mtcap" 1.00 )
( minWidth          "dnw"  "2.8 )
( minSpacing        "dnw"  "4.4 )
( minSpacing        "nwell ""diff "0.22 )
( minSpacing        "pplus ""diff "0.13 )
( minSpacing        "nplus ""diff "0.13 )
( minSpacing        "rpoly ""diff "0.22 )
( minSpacing        "cont ""poly "0.08 )
( minSpacing        "cont ""diff "0.1 )
( minSpacing        "diff ""poly "0.045 )
( minSpacing        "rpoly ""cont "0.22 )
( minSpacing        "rpoly ""poly "0.22 )
( minSpacing        "nat  ""diff "0.20 )
( minSpacing        "nplus ""pplus "0.00 )
) ;spacings

;orderedSpacings
orderedSpacings(
...
) ;orderedSpacings

;electrical
electrical(
; ( constraint layer1 [layer2] value )
; ( ----- )
( contactRes "nplus" 0.363776)
( contactRes "pplus" 0.367872)
( contactRes "poly" 0.28416)
( contactRes "v1" 1)
( contactRes "v2" 1)
( contactRes "v3" 1)
( contactRes "v4" 1)
( contactRes "v5" 1)
( contactRes "v6" 1)
( contactRes "v7" 1)
( contactRes "v8" 1)
) ;electrical

;orderedElectrical
orderedElectrical(
; ( constraint layer1 layer2 value )
; ( ----- )
( areaCap "m1" "diff "3.83e-05)
( areaCap "m1" "poly "4.97e-05)
( areaCap "m2" "diff "1.34e-05)
( areaCap "m2" "poly "1.56e-05)
( areaCap "m2" "m1 "3.11e-05)
( areaCap "m3" "diff "7.99e-06)

```

Appendix A: Sample Technology File System Technology File Sample

```

( areaCap          "m3"      "poly" "0.77e-05)
( areaCap          "m3"      "m1"   "1.17e-05)
( areaCap          "m3"      "m2"   "3.67e-05)
( areaCap          "m4"      "diff" "5.81e-06)
( areaCap          "m4"      "poly" "6.1e-06)
( areaCap          "m4"      "m1"   "7.91e-06)
( areaCap          "m4"      "m2"   "1.16e-05)
( areaCap          "m4"      "m3"   "2.94e-05)
( edgeCapacitance "m1"      "diff" "4.448e-11)
( edgeCapacitance "m2"      "diff" "2.188e-11)
( edgeCapacitance "m2"      "poly" "2.271e-11)
( edgeCapacitance "m2"      "m1"   "4.156e-11)
( edgeCapacitance "m3"      "diff" "1.432e-11)
( edgeCapacitance "m3"      "poly" "1.456e-11)
( edgeCapacitance "m3"      "m1"   "2.34e-11)
( edgeCapacitance "m3"      "m2"   "3.862e-11)
( edgeCapacitance "m4"      "diff" "1.278e-11)
( edgeCapacitance "m4"      "poly" "1.409e-11)
( edgeCapacitance "m4"      "m1"   "1.642e-11)
( edgeCapacitance "m4"      "m2"   "2.473e-11)
( edgeCapacitance "m4"      "m3"   "4.116e-11)
( sheetRes        "nwell"   "diff" "472)
( contactRes      "poly"    "nplus" "0.31454)
( contactRes      "poly"    "pplus" "0.32416)
) ;orderedElectrical

;interconnect
interconnect(
  ( validRoutingLayers ( v8 v7 v6 v5 v4 v3 v2 v1 m9 m8 m7 m6 m5
m4 m3 m2 m1 cont poly diff25 diff18 diff33 diff ) )
) ;interconnect
) ;foundry

;LEFDefaultRouteSpec
( "LEFDefaultRouteSpec" nil
  ( minWidth          cont          0.09 )
  ( horizontalRouteGridPitch m1      0.2 )
  ( verticalRouteGridPitch m1        0.2 )
  ( horizontalRouteGridOffset m1     0.1 )
  ( verticalRouteGridOffset m1       0.1 )
  ( minWidth          m1            0.1 )
  ( minWireExtension m1            0.105 )
  ( minWidth          v1            0.13 )
  ( horizontalRouteGridPitch m2      0.3 )
  ( verticalRouteGridPitch m2        0.3 )
  ( horizontalRouteGridOffset m2     0.15 )
  ( verticalRouteGridOffset m2       0.15 )
  ( minWidth          m2            0.15 )
  ( minWireExtension m2            0.105 )
  ( minWidth          v2            0.13 ) ( horizontalRouteGridPitch
m3 0.3 )
  ( verticalRouteGridPitch m3        0.3 )
  ( horizontalRouteGridOffset m3     0.15 )
  ( verticalRouteGridOffset m3       0.15 )
  ( minWidth          m3            0.15 )
  ( minWireExtension m3            0.105 )
  ( minWidth          v3            0.13 )
  ( horizontalRouteGridPitch m4      0.3 )
  ( verticalRouteGridPitch m4        0.3 )
  ( horizontalRouteGridOffset m4     0.15 )
  ( verticalRouteGridOffset m4       0.15 )

```


Appendix A: Sample Technology File

System Technology File Sample

```

( minWidth          m4          0.15 )
( minWireExtension  m4          0.105 )
( minWidth          v4          0.13 )
( horizontalRouteGridPitch m5          0.3 )
( verticalRouteGridPitch m5          0.3 )
( horizontalRouteGridOffset m5          0.15 )
( verticalRouteGridOffset m5          0.15 )
( minWidth          m5          0.15 )
( minWireExtension  m5          0.105 )
( minWidth          v5          0.13 )
( horizontalRouteGridPitch m6          0.3 )
( verticalRouteGridPitch m6          0.3 )
( horizontalRouteGridOffset m6          0.15 )
( verticalRouteGridOffset m6          0.15 )
( minWidth          m6          0.15 )
( minWireExtension  m6          0.105 )
( minWidth          v6          0.13 )
( horizontalRouteGridPitch m7          0.3 )
( verticalRouteGridPitch m7          0.3 )
( horizontalRouteGridOffset m7          0.15 )
( verticalRouteGridOffset m7          0.15 )
( minWidth          m7          0.15 )
( minWireExtension  m7          0.105 )
( minWidth          v7          0.13 )
( horizontalRouteGridPitch m8          0.3 )
( verticalRouteGridPitch m8          0.3 )
( horizontalRouteGridOffset m8          0.15 )
( verticalRouteGridOffset m8          0.15 )
( minWidth          m8          0.15 )
( minWireExtension  m8          0.105 )
( minWidth          v8          0.13 )
( horizontalRouteGridPitch m9          0.3 )
( verticalRouteGridPitch m9          0.3 )
( horizontalRouteGridOffset m9          0.15 )
( verticalRouteGridOffset m9          0.15 )
( minWidth          m9          0.15 )
( minWireExtension  m9          0.105 )
( validRoutingLayers (m1 m2 m3 m4 m5 m6 m7 m8 m9))
( validRoutingVias (via12 via23 via34 via45 via56 via67 via78 via89
  polyCont diffCont vial2_swap via23_swap via34_swap via45_swap
  via56_swap via67_swap via78_swap via89_swap ))
);LEFDefaultRouteSpec
) ;constraintGroups

```

B

Sample Display Resource File

Presents a sample display resource file to demonstrate the file syntax. The sample shown here is in Tcl.

The Display Resource File (.tcl) contains the following sections in the order listed here:

1. #Display Resource Tcl
2. #Colors Definition
3. #Stipples Definition
4. #Line Styles Definition
5. #Packets Definition
6. #Packet Aliases Definition
7. #Text Packets Definition

Display Resource File Sample

```
#Display Resource Tcl

set display_object [db::getNext [dr::getDisplays -filter "%name == {display}"]]
if {$display_object == ""} {
    set display_object [dr::createDisplay display -maxColors
        256 -maxStipples 256 -maxLineStyles 256 ]
}

#Colors Definition
dr::createColor white -display $display_object -red 255 -green 255 -blue 255
dr::createColor yellow -display $display_object -red 255 -green 255 -blue 0
dr::createColor silver -display $display_object -red 217 -green 230 -blue 255
dr::createColor cream -display $display_object -red 255 -green 255 -blue 204
dr::createColor pink -display $display_object -red 255 -green 191 -blue 242
dr::createColor magenta -display $display_object -red 255 -green 0 -blue 255
dr::createColor lime -display $display_object -red 0 -green 255 -blue 0
dr::createColor tan -display $display_object -red 255 -green 230 -blue 191
dr::createColor cyan -display $display_object -red 0 -green 255 -blue 255
dr::createColor cadetBlue -display $display_object -red 57 -green 191 -blue 255
dr::createColor orange -display $display_object -red 255 -green 128 -blue 0
dr::createColor red -display $display_object -red 255 -green 0 -blue 0
dr::createColor purple -display $display_object -red 153 -green 0 -blue 230
dr::createColor green -display $display_object -red 0 -green 204 -blue 102
```

Appendix B: Sample Display Resource File Display Resource File Sample

```

dr::createColor brown -display $display_object -red 191 -green 64 -blue 38
dr::createColor blue -display $display_object -red 0 -green 0 -blue 255
dr::createColor slate -display $display_object -red 140 -green 140 -blue 166
dr::createColor gold -display $display_object -red 217 -green 204 -blue 0
dr::createColor maroon -display $display_object -red 230 -green 31 -blue 13
dr::createColor violet -display $display_object -red 94 -green 0 -blue 230
dr::createColor forest -display $display_object -red 38 -green 140 -blue 107
dr::createColor chocolate -display $display_object -red 128 -green 38 -blue 38
dr::createColor navy -display $display_object -red 51 -green 51 -blue 153
dr::createColor black -display $display_object -red 0 -green 0 -blue 0
dr::createColor gray -display $display_object -red 204 -green 204 -blue 217
dr::createColor lightWhite -display $display_object -red 127 -green 127 -blue 127
dr::createColor lightYellow -display $display_object -red 127 -green 127 -blue 0
dr::createColor lightRed -display $display_object -red 127 -green 0 -blue 0
dr::createColor lightPurple -display $display_object -red 76 -green 0 -blue 115
dr::createColor lightGreen -display $display_object -red 0 -green 102 -blue 51
dr::createColor lightGray -display $display_object -red 102 -green 102 -blue 108
dr::createColor lightGold -display $display_object -red 108 -green 102 -blue 0
dr::createColor lightMaroon -display $display_object -red 115 -green 15 -blue 6
dr::createColor lightCyan -display $display_object -red 0 -green 127 -blue 127
dr::createColor lightBlue -display $display_object -red 0 -green 0 -blue 127
dr::createColor winColor1 -display $display_object -red 166 -green 166 -blue 166
dr::createColor winColor2 -display $display_object -red 115 -green 115 -blue 115
dr::createColor winColor3 -display $display_object -red 189 -green 204 -blue 204
dr::createColor winColor4 -display $display_object -red 204 -green 204 -blue 204
dr::createColor winColor5 -display $display_object -red 199 -green 199 -blue 199
dr::createColor blinkRed -display $display_object -red 255 -green 0 -blue 0 -blink
true
dr::createColor blinkYellow -display $display_object -red 255 -green 255 -blue 0
-blink true
dr::createColor blinkWhite -display $display_object -red 255 -green 255 -blue 255
-blink true
dr::createColor TrueColorBright -display $display_object -red 166 -green 176 -blue 186
dr::createColor TrueColorGhost -display $display_object -red 160 -green 200 -blue 240
dr::createColor violet -display $display_object -red 94 -green 0 -blue 230
dr::createColor winBack -display $display_object -red 224 -green 224 -blue 224
dr::createColor winFore -display $display_object -red 128 -green 0 -blue 0
dr::createColor winText -display $display_object -red 51 -green 51 -blue 51
dr::createColor paula -display $display_object -red 32 -green 232 -blue 239
dr::createColor whiteB -display $display_object -red 255 -green 255 -blue 255 -blink
true
dr::createColor grayB -display $display_object -red 204 -green 204 -blue 217 -blink
true
dr::createColor yellowB -display $display_object -red 255 -green 255 -blue 0 -blink
true
dr::createColor whiteBlink -display $display_object -red 255 -green 255 -blue 255
-blink true
dr::createColor gray8 -display $display_object -red 225 -green 225 -blue 225
dr::createColor clay -display $display_object -red 128 -green 0 -blue 0
dr::createColor charcoal -display $display_object -red 41 -green 41 -blue 41
dr::createColor gray4 -display $display_object -red 166 -green 166 -blue 166
dr::createColor gray3 -display $display_object -red 115 -green 115 -blue 115
dr::createColor gray5 -display $display_object -red 189 -green 204 -blue 204
dr::createColor gray7 -display $display_object -red 204 -green 204 -blue 204
dr::createColor gray6 -display $display_object -red 199 -green 199 -blue 199
dr::createColor grayBlink -display $display_object -red 199 -green 199 -blue 199
-blink true
dr::createColor limeB -display $display_object -red 0 -green 255 -blue 0 -blink true
dr::createColor cyanB -display $display_object -red 0 -green 255 -blue 255 -blink true
dr::createColor joy1 -display $display_object -red 0 -green 204 -blue 242

#Stipples Definitiondr::createStipple blank -display $display_object -pattern {
    {0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
    {0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}

```

```

{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
}

dr::createStipple solid -display $display_object -pattern {
{1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1}
{1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1}
{1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1}
{1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1}
{1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1}
{1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1}
{1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1}
{1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1}
{1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1}
{1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1}
{1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1}
{1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1}
{1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1}
{1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1}
{1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1}
{1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1}
{1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1}
}

dr::createStipple dots -display $display_object -pattern {
{0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
}

dr::createStipple dots1 -display $display_object -pattern {
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0}
}

```

Appendix B: Sample Display Resource File

Display Resource File Sample

[illegible]

Appendix B: Sample Display Resource File Display Resource File Sample

```

                                {0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
                                {0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
                                {0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0} }
dr::createStipple grid -display $display_object -pattern {
{0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1}
{0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1}
{0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1}
{1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1}
{0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1}
{0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1}
{0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1}
{1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1}
{0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1}
{0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1}
{0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1}
{1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1}
{0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1}
{0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1}
{0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1}
{1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1}
{0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1}
{0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1}
{0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1} }
dr::createStipple slash -display $display_object -pattern {
{0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1}
{0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0}
{0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0}
{1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0}
{0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1}
{0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0}
{0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0}
{1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0}
{0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1}
{0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0}
{0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0}
{1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0}
{0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1}
{0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0}
{0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0}
{1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0} }
dr::createStipple backSlash -display $display_object -pattern [ list \
{ 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 } \
{ 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 } \
{ 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 } \
{ 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 } \
{ 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 } \
{ 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 } \
{ 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 } \
{ 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 } \
{ 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 } \
{ 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 } \
{ 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 } \
{ 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 } \
{ 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 } \
{ 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 } \
{ 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 } \
{ 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 } ]
dr::createStipple hZigZag -display $display_object -pattern {
{1 1 0 0 0 0 0 0 0 1 1 0 0 0 0 0}
{0 0 1 0 0 0 0 0 0 0 0 1 0 0 0 0}
{0 0 0 1 1 0 0 0 0 0 0 0 1 1 0 0}
{0 0 0 0 0 1 0 0 0 0 0 0 0 0 1 0}
{0 0 0 0 0 0 1 1 0 0 0 0 0 0 1 1}
{1 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0}
{0 1 1 0 0 0 0 0 0 1 1 0 0 0 0 0}
{0 0 0 1 0 0 0 0 0 0 0 1 0 0 0 0}
{0 0 0 0 1 1 0 0 0 0 0 0 1 1 0 0}
{0 0 0 0 0 0 1 0 0 0 0 0 0 0 1 0}
}
```

Appendix B: Sample Display Resource File Display Resource File Sample

```

{1 0 0 0 0 0 0 1 1 0 0 0 0 0 0 1}
{0 1 0 0 0 0 0 0 0 1 0 0 0 0 0 0}
{0 0 1 1 0 0 0 0 0 0 1 1 0 0 0 0}
{0 0 0 0 1 0 0 0 0 0 0 0 1 0 0 0}
{0 0 0 0 0 1 1 0 0 0 0 0 0 1 1 0}
{0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 1} }
dr::createStipple vZigZag -display $display_object -pattern {
{1 0 0 0 0 1 0 0 0 0 0 1 0 0 0 0}
{1 0 0 0 0 0 1 0 0 0 0 0 1 0 0 0}
{0 1 0 0 0 0 1 0 0 0 0 0 0 1 0 0}
{0 0 1 0 0 0 0 1 0 0 0 0 0 1 0 0}
{0 0 1 0 0 0 0 0 1 0 0 0 0 0 1 0}
{0 0 0 1 0 0 0 0 1 0 0 0 0 0 1 0}
{0 0 0 0 1 0 0 0 0 1 0 0 0 0 0 1}
{0 0 0 0 1 0 0 0 0 0 1 0 0 0 0 1}
{1 0 0 0 0 1 0 0 0 0 0 1 0 0 0 0}
{1 0 0 0 0 0 1 0 0 0 0 0 1 0 0 0}
{0 1 0 0 0 0 1 0 0 0 0 0 0 1 0 0}
{0 0 1 0 0 0 0 1 0 0 0 0 0 1 0 0}
{0 0 1 0 0 0 0 0 1 0 0 0 0 0 1 0}
{0 0 0 1 0 0 0 0 1 0 0 0 0 0 1 0}
{0 0 0 0 1 0 0 0 0 1 0 0 0 0 1 0}
{0 0 0 0 1 0 0 0 0 0 1 0 0 0 0 1} }
dr::createStipple hCurb -display $display_object -pattern {
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 1 1 1 1 1 0 0 0 0 1 1 1 1}
{0 0 0 1 0 0 0 1 0 0 0 0 1 0 0 0}
{0 0 0 1 0 0 0 1 0 0 0 0 1 0 0 0}
{1 1 1 1 0 0 0 0 1 1 1 1 1 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 1 1 1 1 1 0 0 0 0 1 1 1 1}
{0 0 0 1 0 0 0 1 0 0 0 0 1 0 0 0}
{0 0 0 1 0 0 0 1 0 0 0 0 1 0 0 0}
{1 1 1 1 0 0 0 0 1 1 1 1 1 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0} }
dr::createStipple vCurb -display $display_object -pattern {
{0 0 0 0 0 1 0 0 0 0 0 0 0 0 1 0}
{0 0 0 0 0 1 0 0 0 0 0 0 0 0 1 0}
{0 0 0 0 0 1 0 0 0 0 0 0 0 0 1 0}
{0 0 1 1 1 1 0 0 0 0 0 1 1 1 1 0}
{0 0 1 0 0 0 0 0 0 0 0 1 0 0 0 0}
{0 0 1 0 0 0 0 0 0 0 0 1 0 0 0 0}
{0 0 1 0 0 0 0 0 0 0 0 1 0 0 0 0}
{0 0 1 1 1 1 0 0 0 0 0 1 1 1 1 0}
{0 0 0 0 0 1 0 0 0 0 0 0 0 0 1 0}
{0 0 0 0 0 1 0 0 0 0 0 0 0 0 1 0}
{0 0 0 0 0 1 0 0 0 0 0 0 0 0 1 0}
{0 0 1 1 1 1 0 0 0 0 0 1 1 1 1 0}
{0 0 1 0 0 0 0 0 0 0 0 1 0 0 0 0}
{0 0 1 0 0 0 0 0 0 0 0 1 0 0 0 0}
{0 0 1 0 0 0 0 0 0 0 0 1 0 0 0 0}
{0 0 1 1 1 1 0 0 0 0 0 1 1 1 1 0} }
dr::createStipple brick -display $display_object -pattern {
{1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1}
{0 0 1 0 0 0 0 0 0 0 0 1 0 0 0 0}
{0 0 1 0 0 0 0 0 0 0 0 1 0 0 0 0}
{0 0 1 0 0 0 0 0 0 0 0 1 0 0 0 0}
{1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1}
{0 0 0 0 0 0 1 0 0 0 0 0 0 0 1 0}
{0 0 0 0 0 0 1 0 0 0 0 0 0 0 1 0}

```

Appendix B: Sample Display Resource File Display Resource File Sample

```

{0 0 0 0 0 0 1 0 0 0 0 0 0 0 1 0}
{1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1}
{0 0 1 0 0 0 0 0 0 0 1 0 0 0 0 0}
{0 0 1 0 0 0 0 0 0 0 1 0 0 0 0 0}
{0 0 1 0 0 0 0 0 0 0 1 0 0 0 0 0}
{1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1}
{0 0 0 0 0 0 1 0 0 0 0 0 0 0 1 0}
{0 0 0 0 0 0 1 0 0 0 0 0 0 0 1 0}
{0 0 0 0 0 0 1 0 0 0 0 0 0 0 1 0} }
dr::createStipple dagger -display $display_object -pattern {
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 1 0 0 0 0 0 0 0 1 0 0 0 0 0}
{0 0 1 0 0 0 0 0 0 0 1 0 0 0 0 0}
{0 0 1 0 0 0 0 0 0 0 1 0 0 0 0 0}
{1 1 1 1 1 0 0 0 1 1 1 1 1 0 0 0}
{0 0 1 0 0 0 0 0 0 0 1 0 0 0 0 0}
{0 0 1 0 0 0 0 0 0 0 1 0 0 0 0 0}
{0 0 1 0 0 0 0 0 0 0 1 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 1 0 0 0 0 0 0 0 0 1 0}
{0 0 0 0 0 1 0 0 0 0 0 0 0 0 1 0}
{0 0 0 0 0 1 0 0 0 0 0 0 0 0 1 0}
{0 0 0 1 1 1 1 1 0 0 0 0 1 1 1 1}
{0 0 0 0 0 1 0 0 0 0 0 0 0 0 1 0}
{0 0 0 0 0 1 0 0 0 0 0 0 0 0 1 0} }
dr::createStipple triangle -display $display_object -pattern {
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 1 1 1 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0} }
dr::createStipple X -display $display_object -pattern {
{1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0}
{0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1}
{0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0}
{0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1}
{1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0}
{0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1}
{0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0}
{0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1}
{1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0}
{0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1}
{0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0}
{0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1}
{1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0}
{0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1}
{0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0}
{0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1} }
dr::createStipple x -display $display_object -pattern {
{1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0}
{0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1}
{0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0}
{0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1}

```



```

        {1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0}
        {0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1}
        {0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0}
        {0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1}
        {1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0}
        {0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1}
        {0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0}
        {0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1}
        {1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0}
        {0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1}
        {0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0}
        {0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1} }
dr::createStipple medslash -display $display_object -pattern {
        {0 0 0 0 1 0 0 0 0 0 0 0 0 1 0 0 0 0}
        {0 0 1 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0}
        {0 1 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0}
        {1 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0}
        {0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 1}
        {0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 1 0}
        {0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 1 0 0}
        {0 0 0 0 1 0 0 0 0 0 0 0 0 1 0 0 0 0}
        {0 0 0 1 0 0 0 0 0 0 0 0 0 1 0 0 0 0}
        {0 0 1 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0}
        {0 1 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0}
        {1 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0}
        {0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 1}
        {0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 1 0}
        {0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 1 0 0}
        {0 0 0 0 1 0 0 0 0 0 0 0 0 1 0 0 0 0} }
dr::createStipple dotsa -display $display_object -pattern {
        {0 0 0 0 0 1 0 0 0 0 0 0 0 0 1 0 0}
        {0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
        {0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
        {0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
        {0 1 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0}
        {0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
        {0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
        {0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
        {0 0 0 0 0 1 0 0 0 0 0 0 0 0 1 0 0}
        {0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
        {0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
        {0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
        {0 1 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0}
        {0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
        {0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
        {0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0} }
dr::createStipple medbackSlash -display $display_object -pattern
{
        {0 0 0 0 0 1 0 0 0 0 0 0 0 0 1 0 0 0}
        {0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 1 0 0}
        {0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 1 0}
        {0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 1}
        {1 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0}
        {0 1 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0}
        {0 0 1 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0}
        {0 0 0 1 0 0 0 0 0 0 0 0 1 0 0 0 0 0}
        {0 0 0 0 1 0 0 0 0 0 0 0 0 1 0 0 0 0}
        {0 0 0 0 0 1 0 0 0 0 0 0 0 0 1 0 0 0}
        {0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 1 0 0}
        {0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 1 0}
        {1 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0}
        {0 1 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0}
        {0 0 1 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0}
        {0 0 0 1 0 0 0 0 0 0 0 0 0 1 0 0 0 0} }
dr::createStipple dots2 -display $display_object -pattern {

```

Appendix B: Sample Display Resource File Display Resource File Sample

```

{0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0} }
dr::createStipple stipple1 -display $display_object -pattern {
    {1 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
    {0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
    {0 0 1 0 0 0 0 0 0 0 0 0 0 0 0}
    {0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
    {0 0 0 0 1 0 0 0 0 0 0 0 0 0 0}
    {0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
    {0 0 0 0 0 0 1 0 0 0 0 0 0 0 0}
    {0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
    {0 0 0 0 0 0 0 0 1 0 0 0 0 0 0}
    {0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
    {0 0 0 0 0 0 0 0 0 0 1 0 0 0 0}
    {0 0 0 0 0 0 0 0 0 0 0 0 0 1 0}
    {0 0 0 0 0 0 0 0 0 0 0 0 0 0 1} }
dr::createStipple stipple2 -display $display_object -pattern {
    {0 0 0 0 0 0 0 0 0 0 0 0 0 0 1}
    {0 0 0 0 0 0 0 0 0 0 0 0 0 0 1}
    {0 0 0 0 0 0 0 0 0 0 0 0 0 1 0}
    {0 0 0 0 0 0 0 0 0 0 0 0 1 0 0}
    {0 0 0 0 0 0 0 0 0 0 0 1 0 0 0}
    {0 0 0 0 0 0 0 0 0 0 1 0 0 0 0}
    {0 0 0 0 0 0 0 0 0 1 0 0 0 0 0}
    {0 0 0 0 0 0 0 0 1 0 0 0 0 0 0}
    {0 0 0 0 0 0 0 1 0 0 0 0 0 0 0}
    {0 0 0 0 0 0 1 0 0 0 0 0 0 0 0}
    {0 0 0 1 0 0 0 0 0 0 0 0 0 0 0}
    {0 0 1 0 0 0 0 0 0 0 0 0 0 0 0}
    {0 1 0 0 0 0 0 0 0 0 0 0 0 0 0}
    {1 0 0 0 0 0 0 0 0 0 0 0 0 0 0} }
dr::createStipple stipple3 -display $display_object -pattern {
    {1 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
    {0 1 0 0 0 0 0 0 0 0 0 0 0 0 0}
    {0 0 1 0 0 0 0 0 0 0 0 0 0 0 0}
    {0 0 0 1 0 0 0 0 0 0 0 0 0 0 0}
    {0 0 0 0 1 0 0 0 0 0 0 0 0 0 0}
    {0 0 0 0 0 1 0 0 0 0 0 0 0 0 0}
    {0 0 0 0 0 0 1 0 0 0 0 0 0 0 0}
    {0 0 0 0 0 0 0 1 0 0 0 0 0 0 0}
    {0 0 0 0 0 0 0 0 1 0 0 0 0 0 0}
    {0 0 0 0 0 0 0 0 0 1 0 0 0 0 0}
    {0 0 0 0 0 0 0 0 0 0 1 0 0 0 0}
    {0 0 0 0 0 0 0 0 0 0 0 1 0 0 0}
    {0 0 0 0 0 0 0 0 0 0 0 0 1 0 0}
    {0 0 0 0 0 0 0 0 0 0 0 0 0 1 0}
    {0 0 0 0 0 0 0 0 0 0 0 0 0 0 1} }
dr::createStipple stipple4 -display $display_object -pattern {

```

Appendix B: Sample Display Resource File Display Resource File Sample

```

{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0} }
dr::createStipple stipple5 -display $display_object -pattern {
{1 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0}
{0 1 0 0 0 0 0 0 0 1 0 0 0 0 0 0}
{0 0 1 0 0 0 0 0 0 0 0 1 0 0 0 0}
{0 0 0 1 0 0 0 0 0 0 0 0 1 0 0 0}
{0 0 0 0 1 0 0 0 0 0 0 0 0 1 0 0}
{0 0 0 0 0 1 0 0 0 0 0 0 0 0 1 0}
{0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 1}
{1 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0}
{0 1 0 0 0 0 0 0 0 1 0 0 0 0 0 0}
{0 0 1 0 0 0 0 0 0 0 0 1 0 0 0 0}
{0 0 0 1 0 0 0 0 0 0 0 0 1 0 0 0}
{0 0 0 0 1 0 0 0 0 0 0 0 0 1 0 0}
{0 0 0 0 0 1 0 0 0 0 0 0 0 0 1 0}
{0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 1} }
dr::createStipple stipple6 -display $display_object -pattern {
{0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1}
{0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0} }
dr::createStipple stipple7 -display $display_object -pattern {
{0 0 0 1 0 0 0 0 1 0 0 0 0 1 0 0}
{0 0 1 0 0 0 0 1 0 0 0 0 1 0 0 0}
{0 1 0 0 0 0 1 0 0 0 0 1 0 0 0 0}
{1 0 0 0 0 1 0 0 0 0 1 0 0 0 0 0}
{0 0 0 1 0 0 0 0 1 0 0 0 0 1 0 0}
{0 0 1 0 0 0 0 1 0 0 0 0 1 0 0 0}
{0 1 0 0 0 0 1 0 0 0 0 1 0 0 0 0}
{1 0 0 0 0 1 0 0 0 0 1 0 0 0 0 0}
{0 0 0 1 0 0 0 0 1 0 0 0 0 1 0 0}
{0 0 1 0 0 0 0 1 0 0 0 0 1 0 0 0}
{0 1 0 0 0 0 1 0 0 0 0 1 0 0 0 0}
{1 0 0 0 0 1 0 0 0 0 1 0 0 0 0 0}
{0 0 0 1 0 0 0 0 1 0 0 0 0 1 0 0}
{0 0 1 0 0 0 0 1 0 0 0 0 1 0 0 0}
{0 1 0 0 0 0 1 0 0 0 0 1 0 0 0 0}
{1 0 0 0 0 1 0 0 0 0 1 0 0 0 0 0}
{0 0 0 1 0 0 0 0 1 0 0 0 0 1 0 0}
{0 0 1 0 0 0 0 1 0 0 0 0 1 0 0 0}

```

Appendix B: Sample Display Resource File Display Resource File Sample

```

{0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0}
{1 0 0 0 1 0 0 0 1 0 0 0 1 0 0} }
dr::createStipple stipple8 -display $display_object -pattern {
{1 0 0 0 1 0 0 0 1 0 0 0 1 0 0}
{0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0}
{0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0}
{0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1}
{1 0 0 0 1 0 0 0 1 0 0 0 1 0 0}
{0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0}
{0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0}
{0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1}
{1 0 0 0 1 0 0 0 1 0 0 0 1 0 0}
{0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0}
{0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0}
{0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1}
{1 0 0 0 1 0 0 0 1 0 0 0 1 0 0}
{0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0}
{0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0}
{0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1} }
dr::createStipple stipple9 -display $display_object -pattern {
{0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0} }
dr::createStipple sldiff -display $display_object -pattern {
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0}
{0 0 0 1 0 0 0 0 1 0 0 0 0 0 0 0}
{0 0 1 0 0 0 0 0 0 1 0 0 0 0 0 0}
{0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0}
{0 0 1 0 0 0 0 0 0 0 0 0 0 1 0 0}
{0 0 0 1 0 0 0 0 0 0 0 1 0 0 0 0}
{0 0 0 0 1 0 0 0 0 0 1 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0} }
dr::createStipple zigZag1 -display $display_object -pattern {
{0 1 0 0 0 0 0 0 0 0 1 0 0 0 0 0}
{0 0 1 0 0 0 0 0 0 0 0 1 0 0 0 0}
{0 0 0 1 0 0 0 0 0 0 0 0 1 0 0 0}
{0 0 0 0 1 0 0 0 0 0 0 0 0 1 0 0}
{0 0 0 0 0 1 0 0 0 0 0 0 0 0 1 0}
{0 0 0 0 1 0 0 0 0 0 0 0 0 1 0 0}
{0 0 1 0 0 0 0 0 0 0 0 1 0 0 0 0}
{0 1 0 0 0 0 0 0 0 0 1 0 0 0 0 0}
{0 0 1 0 0 0 0 0 0 0 0 1 0 0 0 0}
{0 0 0 1 0 0 0 0 0 0 0 0 1 0 0 0}

```

Appendix B: Sample Display Resource File Display Resource File Sample

```

        {0 0 0 0 1 0 0 0 0 0 0 0 1 0 0 0}
        {0 0 0 0 0 1 0 0 0 0 0 0 0 1 0 0}
        {0 0 0 0 1 0 0 0 0 0 0 0 0 1 0 0}
        {0 0 0 1 0 0 0 0 0 0 0 0 1 0 0 0}
        {0 0 1 0 0 0 0 0 0 0 0 1 0 0 0 0} }
dr::createStipple bt_solid -display $display_object -pattern {
        {1 1 1 1 1 1 1 1 1 1}
        {1 1 1 1 1 1 1 1 1 1}
        {1 1 1 1 1 1 1 1 1 1}
        {1 1 1 1 1 1 1 1 1 1}
        {1 1 1 1 1 1 1 1 1 1}
        {1 1 1 1 1 1 1 1 1 1}
        {1 1 1 1 1 1 1 1 1 1}
        {1 1 1 1 1 1 1 1 1 1} }
dr::createStipple empty -display $display_object -pattern {
        {0 0 0 0 0 0 0 0 0 0}
        {0 0 0 0 0 0 0 0 0 0}
        {0 0 0 0 0 0 0 0 0 0}
        {0 0 0 0 0 0 0 0 0 0}
        {0 0 0 0 0 0 0 0 0 0}
        {0 0 0 0 0 0 0 0 0 0}
        {0 0 0 0 0 0 0 0 0 0}
        {0 0 0 0 0 0 0 0 0 0} }
dr::createStipple met1 -display $display_object -pattern {
        {1 0 0 0 0 0 0 0 1}
        {0 0 0 0 0 0 0 0 0}
        {0 0 0 0 0 0 0 0 0}
        {0 0 0 1 0 0 0 0 0}
        {0 0 0 0 1 0 0 0 0}
        {0 0 0 0 0 0 0 0 0}
        {0 0 0 0 0 0 0 0 0}
        {1 0 0 0 0 0 0 0 1} }
dr::createStipple os -display $display_object -pattern {
        {1 0 0 0 1 0 0 0 1 0 1 0 1 0 0 0}
        {0 1 0 1 0 1 0 0 0 1 0 0 0 1 0 1}
        {0 0 1 0 0 0 1 0 1 0 1 0 0 0 1 0}
        {0 1 0 1 0 0 0 1 0 0 0 1 0 1 0 1}
        {1 0 0 0 1 0 1 0 1 0 0 0 1 0 0 0}
        {0 1 0 0 0 1 0 0 0 1 0 1 0 1 0 0}
        {0 0 1 0 1 0 1 0 0 0 1 0 0 0 1 0}
        {0 0 0 1 0 0 0 1 0 1 0 1 0 0 0 1}
        {1 0 1 0 1 0 0 0 1 0 0 0 1 0 1 0}
        {0 1 0 0 0 1 0 1 0 1 0 0 0 1 0 0}
        {1 0 1 0 0 0 1 0 0 0 1 0 1 0 1 0}
        {0 0 0 1 0 1 0 1 0 0 0 1 0 0 0 1}
        {1 0 0 0 1 0 0 0 1 0 1 0 1 0 0 0}
        {0 1 0 1 0 1 0 0 0 1 0 0 0 1 0 1}
        {0 0 1 0 0 0 1 0 1 0 1 0 0 0 1 0}
        {0 1 0 1 0 0 0 1 0 0 0 1 0 1 0 1} }
dr::createStipple dots3 -display $display_object -pattern {
        {0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
        {0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
        {0 0 1 0 0 0 0 0 0 1 0 0 0 0 0 1}
        {0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
        {0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
        {0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
        {0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
        {0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
        {0 0 1 0 0 0 0 0 0 1 0 0 0 0 0 1}
        {0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
        {0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
        {0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
        {0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
        {0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
        {0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
        {0 0 1 0 0 0 0 0 0 1 0 0 0 0 0 1} }

```

Custom Compiler™ Technology and Display Resource File Reference Manual
W-2024.09

Appendix B: Sample Display Resource File Display Resource File Sample

```

{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0} }
dr::createStipple backslash2 -display $display_object -pattern {
{0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0} }
dr::createStipple dots_rare -display $display_object -pattern {
{0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0} }
dr::createStipple thickdots -display $display_object -pattern {
{1 1 0 0 0 0 0 0 0 1 1 0 0 0 0 0}
{1 1 0 0 0 0 0 0 0 1 1 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 1 1 0 0 0 0 0 0 0 1 1 0}
{0 0 0 0 1 1 0 0 0 0 0 0 0 1 1 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{1 1 0 0 0 0 0 0 0 1 1 0 0 0 0 0}
{1 1 0 0 0 0 0 0 0 1 1 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 1 1 0 0 0 0 0 0 0 1 1 0}
{0 0 0 0 1 1 0 0 0 0 0 0 0 1 1 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0} }
dr::createStipple cross1 -display $display_object -pattern {
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0}
{0 0 0 0 0 0 1 1 1 1 1 0 0 0 0 0}
{0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0}

```

Appendix B: Sample Display Resource File Display Resource File Sample

```

{0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0} }
dr::createStipple fineCross -display $display_object -pattern {
{1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0}
{0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1}
{1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0}
{0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1}
{1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0}
{0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1}
{1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0}
{0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1}
{1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0}
{0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1}
{1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0}
{0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1}
{1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0}
{0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1}
{1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0}
{0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1}
{0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1} }
dr::createStipple checker -display $display_object -pattern {
{1 1 1 1 1 1 1 1 1 0 0 0 0 0 0 0 0 0}
{1 1 1 1 1 1 1 1 1 0 0 0 0 0 0 0 0 0}
{1 1 1 1 1 1 1 1 1 0 0 0 0 0 0 0 0 0}
{1 1 1 1 1 1 1 1 1 0 0 0 0 0 0 0 0 0}
{1 1 1 1 1 1 1 1 1 0 0 0 0 0 0 0 0 0}
{1 1 1 1 1 1 1 1 1 0 0 0 0 0 0 0 0 0}
{1 1 1 1 1 1 1 1 1 0 0 0 0 0 0 0 0 0}
{1 1 1 1 1 1 1 1 1 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1 1}
{0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1 1}
{0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1 1}
{0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1 1}
{0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1 1}
{0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1 1}
{0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1 1}
{0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1 1} }
dr::createStipple zigZag -display $display_object -pattern {
{1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0}
{0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0}
{0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0}
{0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1}
{0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0}
{0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0}
{0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0}
{0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1}
{0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0}
{0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0}
{0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0}
{0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1}
{0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0}
{0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0}
{0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0}
{0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1} }
dr::createStipple zigZag2 -display $display_object -pattern {
{1 1 0 0 0 0 0 0 0 1 1 0 0 0 0 0 0}
{0 1 1 0 0 0 0 0 0 0 1 1 0 0 0 0 0}
{0 0 1 1 0 0 0 0 0 0 0 1 1 0 0 0 0}
{0 0 0 1 1 0 0 0 0 0 0 0 1 1 0 0 0}
{0 0 0 0 1 1 0 0 0 0 0 0 0 1 1 0 0}
{0 0 0 1 1 0 0 0 0 0 0 0 0 1 1 0 0}

```


Appendix B: Sample Display Resource File

Display Resource File Sample

[illegible]

Appendix B: Sample Display Resource File

Display Resource File Sample

[illegible]

Appendix B: Sample Display Resource File Display Resource File Sample

```

{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0} }
dr::createStipple viap -display $display_object -pattern {
{0 0 1 0 0 0 1 0 0 0 1 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1} }
dr::createStipple metal1S -display $display_object -pattern {
{1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 1 1 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 1 1 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 1 1 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 1 1 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 1 1 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 1 1 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 1 1 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 1 1 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 1 1 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 1 1 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 1 1 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1} }
dr::createStipple metal2S -display $display_object -pattern {
{1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0}
{0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0}

```

Appendix B: Sample Display Resource File

Display Resource File Sample

[illegible]

Appendix B: Sample Display Resource File

Display Resource File Sample

[illegible]

Appendix B: Sample Display Resource File

Display Resource File Sample

[illegible]

Appendix B: Sample Display Resource File Display Resource File Sample

```

{0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1}
{0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0}
{0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0}
{1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0}
{0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1}
{0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0}
{0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0}
{1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0} }
dr::createStipple stipple13 -display $display_object -pattern {
{1 1 0 0 1 1 0 0 1 1 0 0 1 1 0 0}
{1 1 0 0 1 1 0 0 1 1 0 0 1 1 0 0}
{0 0 1 1 0 0 1 1 0 0 1 1 0 0 1 1}
{0 0 1 1 0 0 1 1 0 0 1 1 0 0 1 1}
{1 1 0 0 1 1 0 0 1 1 0 0 1 1 0 0}
{1 1 0 0 1 1 0 0 1 1 0 0 1 1 0 0}
{0 0 1 1 0 0 1 1 0 0 1 1 0 0 1 1}
{0 0 1 1 0 0 1 1 0 0 1 1 0 0 1 1}
{0 0 1 1 0 0 1 1 0 0 1 1 0 0 1 1}
{1 1 0 0 1 1 0 0 1 1 0 0 1 1 0 0}
{1 1 0 0 1 1 0 0 1 1 0 0 1 1 0 0}
{0 0 1 1 0 0 1 1 0 0 1 1 0 0 1 1}
{0 0 1 1 0 0 1 1 0 0 1 1 0 0 1 1}
{0 0 1 1 0 0 1 1 0 0 1 1 0 0 1 1}
{1 1 0 0 1 1 0 0 1 1 0 0 1 1 0 0}
{1 1 0 0 1 1 0 0 1 1 0 0 1 1 0 0}
{0 0 1 1 0 0 1 1 0 0 1 1 0 0 1 1}
{0 0 1 1 0 0 1 1 0 0 1 1 0 0 1 1} }
dr::createStipple stipple14 -display $display_object -pattern {
{0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 1}
{0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 1}
{0 0 0 0 0 1 0 0 0 0 0 0 0 0 1 0}
{0 0 0 0 1 0 0 0 0 0 0 0 0 1 0 0}
{0 0 0 1 0 0 0 0 0 0 0 0 1 0 0 0}
{0 0 1 0 0 0 0 0 0 0 0 0 1 0 0 0}
{0 1 0 0 0 0 0 0 0 0 1 0 0 0 0 0}
{1 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0}
{0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 1}
{0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 1}
{0 0 0 0 0 1 0 0 0 0 0 0 0 1 0 0}
{0 0 0 0 1 0 0 0 0 0 0 0 0 1 0 0}
{0 0 0 1 0 0 0 0 0 0 0 0 1 0 0 0}
{0 0 1 0 0 0 0 0 0 0 0 1 0 0 0 0}
{0 1 0 0 0 0 0 0 0 0 1 0 0 0 0 0}
{1 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0} }
dr::createStipple stipple15 -display $display_object -pattern {
{0 0 1 1 0 0 1 1 0 0 1 1 0 0 1 1}
{0 0 1 1 0 0 1 1 0 0 1 1 0 0 1 1}
{1 1 0 0 1 1 0 0 1 1 0 0 1 1 0 0}
{1 1 0 0 1 1 0 0 1 1 0 0 1 1 0 0}
{0 0 1 1 0 0 1 1 0 0 1 1 0 0 1 1}
{0 0 1 1 0 0 1 1 0 0 1 1 0 0 1 1}
{1 1 0 0 1 1 0 0 1 1 0 0 1 1 0 0}
{1 1 0 0 1 1 0 0 1 1 0 0 1 1 0 0}
{0 0 1 1 0 0 1 1 0 0 1 1 0 0 1 1}
{0 0 1 1 0 0 1 1 0 0 1 1 0 0 1 1}
{1 1 0 0 1 1 0 0 1 1 0 0 1 1 0 0}
{1 1 0 0 1 1 0 0 1 1 0 0 1 1 0 0}
{0 0 1 1 0 0 1 1 0 0 1 1 0 0 1 1}
{0 0 1 1 0 0 1 1 0 0 1 1 0 0 1 1}
{1 1 0 0 1 1 0 0 1 1 0 0 1 1 0 0}
{1 1 0 0 1 1 0 0 1 1 0 0 1 1 0 0} }
dr::createStipple stipple16 -display $display_object -pattern {
{1 1 0 0 0 0 0 0 0 1 1 0 0 0 0 0}
{1 1 0 0 0 0 0 0 0 1 1 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
{0 0 0 0 1 1 0 0 0 0 0 0 0 1 1 0}

```

Appendix B: Sample Display Resource File Display Resource File Sample

```

                                {0 0 0 0 1 1 0 0 0 0 0 0 1 1 0 0}
                                {0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
                                {0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
                                {1 1 0 0 0 0 0 0 0 1 1 0 0 0 0 0}
                                {1 1 0 0 0 0 0 0 0 1 1 0 0 0 0 0}
                                {0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
                                {0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
                                {0 0 0 0 1 1 0 0 0 0 0 0 0 1 1 0}
                                {0 0 0 0 1 1 0 0 0 0 0 0 0 1 1 0}
                                {0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
                                {0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0} }
dr::createStipple dot4 -display $display_object -pattern {
                                {0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
                                {0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0}
                                {0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
                                {0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
                                {0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
                                {0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0}
                                {0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
                                {0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
                                {0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
                                {0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0}
                                {0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
                                {0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
                                {0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
                                {0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0}
                                {0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
                                {0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0} }
dr::createStipple stipple0 -display $display_object -pattern {
                                {0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1}
                                {0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
                                {0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0}
                                {0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
                                {0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0}
                                {0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
                                {0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0}
                                {0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
                                {0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0}
                                {0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
                                {0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0}
                                {0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0}
                                {1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0} }

#Line Styles Definition
dr::createLineStyle solid -display $display_object -width 1 -pattern [ list 1 1 1 ]
dr::createLineStyle dashed -display $display_object -width 1 -pattern [ list 1 1 1 1 0
0 ]
dr::createLineStyle dots -display $display_object -width 1 -pattern [ list 1 0 0 ]
dr::createLineStyle dashDot -display $display_object -width 1 -pattern [ list 1 1 1 0
0 1 0 0 ]
dr::createLineStyle shortDash -display $display_object -width 1 -pattern [ list 1 1 0
0 ]
dr::createLineStyle doubleDash -display $display_object -width 1 -pattern [ list 1 1 1
1 0 0 1 1 0 0 ]
dr::createLineStyle hidden -display $display_object -width 1 -pattern [ list 1 0 0 0 ]
dr::createLineStyle mediumLine -display $display_object -width 2 -pattern [list 1 1
1 ]
dr::createLineStyle thickLine -display $display_object -width 3 -pattern [ list 1 1
1 ]
dr::createLineStyle thickDash -display $display_object -width 3 -pattern [ list 1 1 1
1 0 0 ]
dr::createLineStyle lineStyle0 -display $display_object -width 1 -pattern [ list 1 ]

```


Appendix B: Sample Display Resource File Display Resource File Sample

```

dr::createLineStyle lineStyle1 -display $display_object -width 1 -pattern [ list 1 1 1
1 0 0 0 0 1 1 1 1 0 0 0 0 ]
dr::createLineStyle lineStyle2 -display $display_object -width 1 -pattern [ list 0 1 0
1 0 1 0 1 0 1 0 1 0 1 1 ]
dr::createLineStyle lineStyle3 -display $display_object -width 1 -pattern [ list 1 0 1
0 1 0 1 0 1 0 1 0 1 0 1 0 ]
dr::createLineStyle lineStyle4 -display $display_object -width 1 -pattern [ list 1 1 1
1 1 0 0 0 0 0 0 1 1 1 1 1 ]
dr::createLineStyle lineStyle5 -display $display_object -width 1 -pattern [ list 0 0 0
1 1 0 0 0 0 0 0 1 1 0 0 0 ]
dr::createLineStyle mLine -display $display_object -width 2 -pattern [ list 1 1 1 ]
dr::createLineStyle thickLine2 -display $display_object -width 2 -pattern [ list 1 1
1 ]

#Packets Definition
dr::createPacket defaultPacket -display $display_object -stipple blank \ -lineStyle
solid -fill white -outline yellow
dr::createPacket background -display $display_object -stipple blank \ -lineStyle solid
-fill black -outline black
dr::createPacket grid -display $display_object -stipple blank \ -lineStyle solid -fill
slate -outline slate
dr::createPacket grid1 -display $display_object -stipple blank \ -lineStyle solid
-fill white -outline white
dr::createPacket axis -display $display_object -stipple blank \ -lineStyle solid -fill
white -outline white
dr::createPacket instance -display $display_object -stipple blank \ -lineStyle solid
-fill red -outline red
dr::createPacket instanceLbl -display $display_object -stipple blank \ -lineStyle
solid -fill gold -outline gold
dr::createPacket prBoundary -display $display_object -stipple blank \ -lineStyle solid
-fill purple -outline purple
dr::createPacket prBoundaryBnd -display $display_object -stipple blank \ -lineStyle
solid -fill cyan -outline cyan
dr::createPacket prBoundaryLbl -display $display_object -stipple blank \ -lineStyle
solid -fill purple -outline purple
dr::createPacket align -display $display_object -stipple blank \ -lineStyle solid
-fill tan -outline tan
dr::createPacket hardFence -display $display_object -stipple blank \ -lineStyle solid
-fill red -outline red
dr::createPacket softFence -display $display_object -stipple blank \ -lineStyle solid
-fill yellow -outline yellow
dr::createPacket text -display $display_object -stipple blank \ -lineStyle solid -fill
white -outline white
dr::createPacket text1 -display $display_object -stipple blank \ -lineStyle dashed
-fill white -outline white
dr::createPacket text2 -display $display_object -stipple solid \ -lineStyle solid
-fill white -outline white
dr::createPacket border -display $display_object -stipple blank \ -lineStyle solid
-fill tan -outline tan
dr::createPacket device -display $display_object -stipple blank \ -lineStyle solid
-fill green -outline green
dr::createPacket device2 -display $display_object -stipple blank \ -lineStyle dashed
-fill green -outline green
dr::createPacket device1 -display $display_object -stipple solid \ -lineStyle solid
-fill green -outline green
dr::createPacket wire -display $display_object -stipple solid \ -lineStyle solid -fill
cadetBlue -outline cadetBlue
dr::createPacket wireLbl -display $display_object -stipple solid \ -lineStyle solid
-fill cadetBlue -outline cadetBlue
dr::createPacket wireFlt -display $display_object -stipple blank \ -lineStyle dashed
-fill red -outline red
dr::createPacket highlightUnplaced -display $display_object -stipple X \ -lineStyle
solid -fill magenta -outline magenta

```

Appendix B: Sample Display Resource File Display Resource File Sample

```

dr::createPacket deviceAnt -display $display_object -stipple blank \ -lineStyle solid
  -fill yellow -outline yellow
dr::createPacket deviceLbl -display $display_object -stipple blank \ -lineStyle solid
  -fill green -outline green
dr::createPacket pinLbl -display $display_object -stipple blank \ -lineStyle solid
  -fill red -outline red
dr::createPacket pin -display $display_object -stipple solid \ -lineStyle solid -fill
  red -outline red
dr::createPacket pinAnt -display $display_object -stipple blank \ -lineStyle solid
  -fill red -outline red
dr::createPacket annotate -display $display_object -stipple blank \ -lineStyle solid
  -fill orange -outline orange
dr::createPacket annotate1 -display $display_object -stipple blank \ -lineStyle solid
  -fill pink -outline pink
dr::createPacket annotate2 -display $display_object -stipple blank \ -lineStyle solid
  -fill lime -outline lime
dr::createPacket annotate3 -display $display_object -stipple blank \ -lineStyle solid
  -fill cyan -outline cyan
dr::createPacket annotate4 -display $display_object -stipple blank \ -lineStyle solid
  -fill yellow -outline yellow
dr::createPacket annotate5 -display $display_object -stipple blank \ -lineStyle solid
  -fill white -outline white
dr::createPacket annotate6 -display $display_object -stipple blank \ -lineStyle solid
  -fill silver -outline silver
dr::createPacket annotate7 -display $display_object -stipple blank \ -lineStyle solid
  -fill red -outline red
dr::createPacket annotate8 -display $display_object -stipple blank \ -lineStyle solid
  -fill tan -outline tan
dr::createPacket annotate9 -display $display_object -stipple blank \ -lineStyle solid
  -fill green -outline green
dr::createPacket edgeLayer -display $display_object -stipple blank \ -lineStyle solid
  -fill winColor5 -outline winColor5
dr::createPacket edgeLayerPin -display $display_object -stipple blank \ -lineStyle
  solid -fill yellow -outline yellow
dr::createPacket snap -display $display_object -stipple blank \ -lineStyle solid -fill
  yellow -outline yellow
dr::createPacket stretch -display $display_object -stipple blank \ -lineStyle solid
  -fill yellow -outline yellow
dr::createPacket y0 -display $display_object -stipple blank \ -lineStyle solid -fill
  yellow -outline yellow
dr::createPacket y1 -display $display_object -stipple blank \ -lineStyle solid -fill
  red -outline red
dr::createPacket y2 -display $display_object -stipple blank \ -lineStyle solid -fill
  green -outline green
dr::createPacket y3 -display $display_object -stipple blank \ -lineStyle solid -fill
  magenta -outline magenta
dr::createPacket y4 -display $display_object -stipple blank \ -lineStyle solid -fill
  cyan -outline cyan
dr::createPacket y5 -display $display_object -stipple blank \ -lineStyle solid -fill
  purple -outline purple
dr::createPacket y6 -display $display_object -stipple blank \ -lineStyle solid -fill
  orange -outline orange
dr::createPacket y7 -display $display_object -stipple blank \ -lineStyle solid -fill
  gold -outline gold
dr::createPacket y8 -display $display_object -stipple blank \ -lineStyle solid -fill
  blue -outline blue
dr::createPacket y9 -display $display_object -stipple blank \ -lineStyle solid -fill
  silver -outline silver
dr::createPacket y0Flt -display $display_object -stipple blank \ -lineStyle solid
  -fill yellow -outline yellow
dr::createPacket y1Flt -display $display_object -stipple blank \ -lineStyle solid
  -fill red -outline red
dr::createPacket y2Flt -display $display_object -stipple blank \ -lineStyle solid
  -fill green -outline green

```

Appendix B: Sample Display Resource File Display Resource File Sample

```

dr::createPacket y3Flt -display $display_object -stipple blank \ -lineStyle solid
  -fill magenta -outline magenta
dr::createPacket y4Flt -display $display_object -stipple blank \ -lineStyle solid
  -fill cyan -outline cyan
dr::createPacket y5Flt -display $display_object -stipple blank \ -lineStyle solid
  -fill purple -outline purple
dr::createPacket y6Flt -display $display_object -stipple blank \ -lineStyle solid
  -fill orange -outline orange
dr::createPacket y7Flt -display $display_object -stipple blank \ -lineStyle solid
  -fill gold -outline gold
dr::createPacket y8Flt -display $display_object -stipple blank \ -lineStyle solid
  -fill blue -outline blue
dr::createPacket y9Flt -display $display_object -stipple blank \ -lineStyle solid
  -fill silver -outline silver
dr::createPacket hilite -display $display_object -stipple blank \ -lineStyle solid
  -fill white -outline white
dr::createPacket hilite1 -display $display_object -stipple blank \ -lineStyle
  thickDash -fill white -outline white
dr::createPacket hilite2 -display $display_object -stipple blank \ -lineStyle solid
  -fill tan -outline tan
dr::createPacket hilite3 -display $display_object -stipple blank \ -lineStyle solid
  -fill cyan -outline cyan
dr::createPacket hilite4 -display $display_object -stipple blank \ -lineStyle solid
  -fill orange -outline orange
dr::createPacket hilite5 -display $display_object -stipple blank \ -lineStyle solid
  -fill lime -outline lime
dr::createPacket hilite7 -display $display_object -stipple blank \ -lineStyle solid
  -fill cream -outline cream
dr::createPacket hilite6 -display $display_object -stipple blank \ -lineStyle solid
  -fill orange -outline orange
dr::createPacket hilite8 -display $display_object -stipple blank \ -lineStyle solid
  -fill magenta -outline magenta
dr::createPacket hilite9 -display $display_object -stipple blank \ -lineStyle solid
  -fill pink -outline pink
dr::createPacket select -display $display_object -stipple blank \ -lineStyle thickLine
  -fill tan -outline tan
dr::createPacket drive -display $display_object -stipple blank \ -lineStyle solid
  -fill blue -outline blue
dr::createPacket hiz -display $display_object -stipple blank \ -lineStyle solid -fill
  orange -outline orange
dr::createPacket resist -display $display_object -stipple blank \ -lineStyle solid
  -fill cyan -outline cyan
dr::createPacket spike -display $display_object -stipple blank \ -lineStyle solid
  -fill purple -outline purple
dr::createPacket supply -display $display_object -stipple blank \ -lineStyle solid
  -fill lime -outline lime
dr::createPacket unknown -display $display_object -stipple blank \ -lineStyle solid
  -fill yellow -outline yellow
dr::createPacket unset -display $display_object -stipple blank \ -lineStyle solid
  -fill forest -outline forest
dr::createPacket designFlow -display $display_object
  -stipple dots \ -lineStyle solid -fill blue -outline blue
dr::createPacket designFlow1 -display $display_object -stipple dots \ -lineStyle solid
  -fill purple -outline purple
dr::createPacket designFlow2 -display $display_object -stipple dots \ -lineStyle solid
  -fill magenta -outline magenta
dr::createPacket designFlow3 -display $display_object -stipple dots \ -lineStyle solid
  -fill cyan -outline cyan
dr::createPacket designFlow4 -display $display_object -stipple dots \ -lineStyle solid
  -fill forest -outline forest
dr::createPacket designFlow5 -display $display_object -stipple dots \ -lineStyle solid
  -fill green -outline green
dr::createPacket designFlow6 -display $display_object -stipple dots \ -lineStyle solid
  -fill orange -outline orange
dr::createPacket designFlow7 -display $display_object -stipple dots \ -lineStyle solid
  -fill yellow -outline yellow

```

Appendix B: Sample Display Resource File Display Resource File Sample

```

dr::createPacket designFlow8 -display $display_object -stipple dots \ -lineStyle solid
  -fill brown -outline brown
dr::createPacket designFlow9 -display $display_object -stipple dots \ -lineStyle solid
  -fill red -outline red
dr::createPacket changedLayerTl0 -display $display_object -stipple blank \ -lineStyle
  solid -fill red -outline red
dr::createPacket changedLayerTl1 -display $display_object -stipple blank \ -lineStyle
  solid -fill yellow -outline yellow
dr::createPacket markerAno -display $display_object -stipple X \ -lineStyle solid
  -fill blue -outline blue
dr::createPacket markerInf -display $display_object -stipple X \ -lineStyle solid
  -fill green -outline green
dr::createPacket markerAck -display $display_object -stipple X \ -lineStyle solid
  -fill yellow -outline yellow
dr::createPacket markerWarn -display $display_object -stipple X \ -lineStyle solid
  -fill blinkYellow -outline blinkYellow
dr::createPacket markerSer -display $display_object -stipple X \ -lineStyle solid
  -fill orange -outline orange
dr::createPacket markerErr -display $display_object -stipple X \ -lineStyle solid
  -fill blinkWhite -outline blinkWhite
dr::createPacket markerScr -display $display_object -stipple X \ -lineStyle solid
  -fill red -outline red
dr::createPacket markerCrt -display $display_object -stipple X \ -lineStyle solid
  -fill red -outline red
dr::createPacket markerFat -display $display_object -stipple X \ -lineStyle solid
  -fill blinkRed -outline blinkRed
dr::createPacket Row -display $display_object -stipple blank \ -lineStyle solid -fill
  cyan -outline cyan
dr::createPacket RowBnd -display $display_object -stipple blank \ -lineStyle solid
  -fill cyan -outline cyan
dr::createPacket RowLbl -display $display_object -stipple blank \ -lineStyle solid
  -fill cyan -outline cyan
dr::createPacket Group -display $display_object -stipple dots \ -lineStyle solid -fill
  green -outline green
dr::createPacket GroupLbl -display $display_object -stipple blank \ -lineStyle solid
  -fill green -outline green
dr::createPacket GroupBnd -display $display_object -stipple blank \ -lineStyle solid
  -fill green -outline green
dr::createPacket Cannotoccupy -display $display_object -stipple X \ -lineStyle solid
  -fill red -outline red
dr::createPacket CannotoccupyBnd -display $display_object -stipple blank \ -lineStyle
  solid -fill red -outline red
dr::createPacket Canplace -display $display_object -stipple blank \ -lineStyle solid
  -fill cyan -outline cyan
dr::createPacket Unrouted -display $display_object -stipple blank \ -lineStyle dashed
  -fill winColor5 -outline winColor5
dr::createPacket UnroutedTrk -display $display_object -stipple blank \ -lineStyle
  dashed -fill yellow -outline yellow
dr::createPacket Unrouted1 -display $display_object -stipple blank \ -lineStyle dashed
  -fill brown -outline brown
dr::createPacket Unrouted2 -display $display_object -stipple blank \ -lineStyle dashed
  -fill red -outline red
dr::createPacket Unrouted3 -display $display_object -stipple blank \ -lineStyle dashed
  -fill pink -outline pink
dr::createPacket Unrouted4 -display $display_object -stipple blank \ -lineStyle dashed
  -fill orange -outline orange
dr::createPacket Unrouted5 -display $display_object -stipple blank \ -lineStyle dashed
  -fill green -outline green
dr::createPacket Unrouted6 -display $display_object -stipple blank \ -lineStyle dashed
  -fill blue -outline blue
dr::createPacket Unrouted7 -display $display_object -stipple blank \ -lineStyle dashed
  -fill purple -outline purple
dr::createPacket Unrouted8 -display $display_object -stipple blank \ -lineStyle dashed
  -fill gold -outline gold

```

Appendix B: Sample Display Resource File Display Resource File Sample

```

dr::createPacket Unrouted9 -display $display_object -stipple blank \ -lineStyle dashed
  -fill silver -outline silver
dr::createPacket stroke white -display $display_object -stipple blank \ -lineStyle
  solid -fill white -outline white
dr::createPacket select1 yellow -display $display_object -stipple blank \ -lineStyle
  solid -fill yellow -outline yellow
dr::createPacket select2 red -display $display_object -stipple blank \ -lineStyle
  solid -fill red -outline red
dr::createPacket select3 blue -display $display_object -stipple blank \ -lineStyle
  solid -fill blue -outline blue
dr::createPacket select4 purple -display $display_object -stipple blank \ -lineStyle
  solid -fill purple -outline purple
dr::createPacket highlight1 yellow -display $display_object -stipple blank \
  -lineStyle solid -fill yellow -outline yellow
dr::createPacket highlight2 red -display $display_object -stipple blank \ -lineStyle
  solid -fill red -outline red
dr::createPacket highlight3 green -display $display_object -stipple blank \ -lineStyle
  solid -fill green -outline green
dr::createPacket highlight4 blue -display $display_object -stipple blank \ -lineStyle
  solid -fill blue -outline blue
dr::createPacket highlight5 purple -display $display_object -stipple blank \
  -lineStyle solid -fill purple -outline purple
dr::createPacket highlight6 cyan -display $display_object -stipple blank \ -lineStyle
  solid -fill cyan -outline cyan
dr::createPacket highlight7 white -display $display_object -stipple blank \ -lineStyle
  solid -fill white -outline white
dr::createPacket highlight8 gold -display $display_object -stipple blank \ -lineStyle
  solid -fill gold -outline gold
dr::createPacket highlight9 maroon -display $display_object -stipple blank \
  -lineStyle solid -fill maroon -outline maroon
dr::createPacket highlight10 gray -display $display_object -stipple blank \ -lineStyle
  solid -fill gray -outline gray
dr::createPacket highlight1 yellow_dim -display $display_object -stipple blank \
  -lineStyle solid -fill lightYellow -outline lightYellow
dr::createPacket highlight2 red_dim -display $display_object -stipple blank \
  -lineStyle solid -fill lightRed -outline lightRed
dr::createPacket highlight3 green_dim -display $display_object -stipple blank \
  -lineStyle solid -fill lightGreen -outline lightGreen
dr::createPacket highlight4 blue_dim -display $display_object -stipple blank \
  -lineStyle solid -fill lightBlue -outline lightBlue
dr::createPacket highlight5 purple_dim -display $display_object -stipple blank \
  -lineStyle solid -fill lightPurple -outline lightPurple
dr::createPacket highlight6 cyan_dim -display $display_object -stipple blank \
  -lineStyle solid -fill lightCyan -outline lightCyan
dr::createPacket highlight7 white_dim -display $display_object -stipple blank \
  -lineStyle solid -fill lightWhite -outline lightWhite
dr::createPacket highlight8 gold_dim -display $display_object -stipple blank \
  -lineStyle solid -fill lightGold -outline lightGold
dr::createPacket highlight9 maroon_dim -display $display_object -stipple blank \
  -lineStyle solid -fill lightMaroon -outline lightMaroon
dr::createPacket highlight10 gray_dim -display $display_object -stipple blank \
  -lineStyle solid -fill lightGray -outline lightGray
dr::createPacket ruler white -display $display_object -stipple blank \ -lineStyle
  solid -fill white -outline white
dr::createPacket active TrueColorBright -display $display_object -stipple blank \
  -lineStyle dashed -fill TrueColorBright -outline TrueColorBright
dr::createPacket runtime TrueColorGhost -display $display_object -stipple blank \
  -lineStyle solid -fill TrueColorGhost -outline TrueColorGhost
dr::createPacket cursor white -display $display_object -stipple blank \ -lineStyle
  solid -fill white -outline white
dr::createPacket major_grid -display $display_object -stipple blank \ -lineStyle
  shortDash -fill white -outline white
dr::createPacket minor_grid -display $display_object -stipple blank \ -lineStyle dots
  -fill slate -outline slate

```

Appendix B: Sample Display Resource File Display Resource File Sample

```

dr::createPacket yellow white -display $display_object -stipple blank \ -lineStyle
solid -fill yellow -outline white
dr::createPacket transparent white -display $display_object -stipple blank \
-lineStyle solid -fill yellow -outline white -fillStyle outline
dr::createPacket pinUnc -display $display_object -stipple X \ -lineStyle solid -fill
red -outline red
dr::createPacket wireUnrt -display $display_object -stipple blank \ -lineStyle dashed
-fill cadetBlue -outline cadetBlue
dr::createPacket inherited -display $display_object -stipple inherited \ -lineStyle
inherited -fill inherited -outline inherited -fillStyle inherited
dr::createPacket nwell -display $display_object -stipple dotsa \ -lineStyle solid
-fill magenta -outline magenta -fillStyle outline
dr::createPacket diff -display $display_object -stipple dots1 \ -lineStyle solid -fill
yellow -outline yellow
dr::createPacket sldiff -display $display_object -stipple backSlash \ -lineStyle solid
-fill red -outline red
dr::createPacket poly -display $display_object -stipple cross \ -lineStyle thickLine
-fill blue -outline blue
dr::createPacket polyPin -display $display_object -stipple X \ -lineStyle solid -fill
red -outline red -fillStyle X
dr::createPacket rpoly -display $display_object -stipple cross2 \ -lineStyle solid
-fill lime -outline lime
dr::createPacket nplus -display $display_object -stipple dots1 \ -lineStyle solid
-fill green -outline green -fillStyle outline
dr::createPacket pplus -display $display_object -stipple dots1 \ -lineStyle solid
-fill red -outline red -fillStyle outline
dr::createPacket cont -display $display_object -stipple cross \ -lineStyle solid -fill
green -outline green
dr::createPacket m1 -display $display_object -stipple dot4 \ -lineStyle solid -fill
cyan -outline cyan
dr::createPacket m1Pin -display $display_object -stipple X \ -lineStyle solid -fill
slate -outline slate -fillStyle X
dr::createPacket m1Blk -display $display_object -stipple X \ -lineStyle solid -fill
slate -outline slate -fillStyle X
dr::createPacket m2 -display $display_object -stipple dot4 \ -lineStyle solid -fill
yellow -outline yellow
dr::createPacket m2Pin -display $display_object -stipple X \ -lineStyle solid -fill
red -outline red -fillStyle X
dr::createPacket m2Blk -display $display_object -stipple X \ -lineStyle solid -fill
red -outline red -fillStyle X
dr::createPacket m3 -display $display_object -stipple dot4 \ -lineStyle solid -fill
blue -outline blue
dr::createPacket m3Pin -display $display_object -stipple X \ -lineStyle solid -fill
blue -outline blue -fillStyle X
dr::createPacket m3Blk -display $display_object -stipple X \ -lineStyle solid -fill
blue -outline blue -fillStyle X
dr::createPacket m4 -display $display_object -stipple dot4 \ -lineStyle solid -fill
navy -outline navy
dr::createPacket m4Pin -display $display_object -stipple X \ -lineStyle solid -fill
navy -outline navy -fillStyle X
dr::createPacket m4Blk -display $display_object -stipple X \ -lineStyle solid -fill
navy -outline navy -fillStyle X
dr::createPacket m5 -display $display_object -stipple dot4 \ -lineStyle solid -fill
pink -outline pink
dr::createPacket m5Pin -display $display_object -stipple X \ -lineStyle solid -fill
pink -outline pink -fillStyle X
dr::createPacket m5Blk -display $display_object -stipple X \ -lineStyle solid -fill
pink -outline pink -fillStyle X
dr::createPacket m6 -display $display_object -stipple dot4 \ -lineStyle solid -fill
cream -outline cream
dr::createPacket m6Pin -display $display_object -stipple X \ -lineStyle solid -fill
cream -outline cream -fillStyle X
dr::createPacket m6Blk -display $display_object -stipple X \ -lineStyle solid -fill
cream -outline cream -fillStyle X

```


Appendix B: Sample Display Resource File Display Resource File Sample

```

dr::createPacket m7 -display $display_object -stipple dot4 \ -lineStyle solid -fill
orange -outline orange
dr::createPacket m7Pin -display $display_object -stipple X \ -lineStyle solid -fill
orange -outline orange -fillStyle X
dr::createPacket m7Blk -display $display_object -stipple X \ -lineStyle solid -fill
orange -outline orange -fillStyle X
dr::createPacket m8 -display $display_object -stipple dot4 \ -lineStyle solid -fill
purple -outline purple
dr::createPacket m8Pin -display $display_object -stipple X \ -lineStyle solid -fill
purple -outline purple -fillStyle X
dr::createPacket m8Blk -display $display_object -stipple X \ -lineStyle solid -fill
purple -outline purple -fillStyle X
dr::createPacket m9 -display $display_object -stipple dot4 \ -lineStyle solid -fill
magenta -outline magenta
dr::createPacket m9Pin -display $display_object -stipple X \ -lineStyle solid -fill
magenta -outline magenta -fillStyle X
dr::createPacket m9Blk -display $display_object -stipple X \ -lineStyle solid -fill
magenta -outline magenta -fillStyle X
dr::createPacket v1 -display $display_object -stipple solid \ -lineStyle solid -fill
cadetBlue -outline cadetBlue -fillStyle X
dr::createPacket v2 -display $display_object -stipple solid \ -lineStyle solid -fill
orange -outline orange -fillStyle X
dr::createPacket v3 -display $display_object -stipple solid \ -lineStyle solid -fill
purple -outline purple -fillStyle X
dr::createPacket v4 -display $display_object -stipple solid \ -lineStyle solid -fill
magenta -outline magenta -fillStyle X
dr::createPacket v5 -display $display_object -stipple solid \ -lineStyle solid -fill
orange -outline orange -fillStyle X
dr::createPacket v6 -display $display_object -stipple solid \ -lineStyle solid -fill
navy -outline navy -fillStyle X
dr::createPacket v7 -display $display_object -stipple solid \ -lineStyle solid -fill
yellow -outline yellow -fillStyle X
dr::createPacket v8 -display $display_object -stipple solid \ -lineStyle solid -fill
cream -outline cream -fillStyle X
dr::createPacket mtcap -display $display_object -stipple blank \ -lineStyle thickLine2
-fill cadetBlue -outline cadetBlue -fillStyle outlinedr::createPacket dnw -display
$display_object -stipple dotsa \ -lineStyle solid -fill green -outline green
dr::createPacket rpdmy -display $display_object -stipple dots1 \ -lineStyle solid
-fill blue -outline blue -fillStyle outline
dr::createPacket rtdmy -display $display_object -stipple dots1 \ -lineStyle solid
-fill silver -outline blue
dr::createPacket y9 -display $display_object -stipple blank \ -lineStyle solid -fill
silver -outline silver -fillStyle outline
dr::createPacket area -display $display_object -stipple blank \ -lineStyle solid -fill
tan -outline tan -fillStyle outline
dr::createPacket magenta -display $display_object -stipple blank \ -lineStyle solid
-fill magenta -outline magenta -fillStyle outline
dr::createPacket pink -display $display_object -stipple blank \ -lineStyle solid -fill
pink -outline pink -fillStyle outline
dr::createPacket wireLbl -display $display_object -stipple solid \ -lineStyle solid
-fill cadetBlue -outline cadetBlue #extra packets
dr::createPacket medvtp -display $display_object -stipple cross1 \ -lineStyle solid
-fill blue -outline blue
dr::createPacket medvtn -display $display_object -stipple cross1 \ -lineStyle solid
-fill green -outline green
dr::createPacket lowvtp -display $display_object -stipple cross1 \ -lineStyle solid
-fill blue -outline blue
dr::createPacket lowvtn -display $display_object -stipple
cross1 \ -lineStyle solid -fill green -outline green
dr::createPacket highvtp -display $display_object -stipple cross1 \ -lineStyle solid
-fill pink -outline pink
dr::createPacket highvtn -display $display_object -stipple cross1 \ -lineStyle solid
-fill white -outline white
dr::createPacket nat -display $display_object -stipple triangle \ -lineStyle solid
-fill brown -outline brown

```

Appendix B: Sample Display Resource File Display Resource File Sample

```

dr::createPacket diff25 -display $display_object -stipple dots2 \ -lineStyle solid
  -fill orange -outline orange
dr::createPacket diff18 -display $display_object -stipple dots2 \ -lineStyle solid
  -fill green -outline green
dr::createPacket diff33 -display $display_object -stipple dots2 \ -lineStyle solid
  -fill magenta -outline magenta
dr::createPacket hrmp -display $display_object -stipple solid \ -lineStyle thickLine2
  -fill navy -outline navy -fillStyle outline
dr::createPacket varmarker -display $display_object -stipple cross1 \ -lineStyle
  thickLine2 -fill pink -outline pink -fillStyle outline
dr::createPacket dum ind -display $display_object -stipple blank \ -lineStyle
  thickLine2 -fill forest -outline forest -fillStyle outline
dr::createPacket bjtdummy -display $display_object -stipple blank \ -lineStyle
  thickLine2 -fill red -outline red -fillStyle outline
dr::createPacket nwdmy -display $display_object -stipple vZigZag \ -lineStyle
  thickLine2 -fill purple -outline purple -fillStyle outline
dr::createPacket rmdmy -display $display_object -stipple cross1 \ -lineStyle solid
  -fill silver -outline silver
dr::createPacket indmmy -display $display_object -stipple blank \ -lineStyle
  lineStyle0 -fill winFore -outline winFore -fillStyle outline
dr::createPacket ctm -display $display_object -stipple hZigZag \ -lineStyle solid
  -fill lime -outline lime
dr::createPacket mbcap -display $display_object -stipple blank \ -lineStyle thickLine2
  -fill green -outline green -fillStyle outline
dr::createPacket ctmdmy -display $display_object -stipple blank \ -lineStyle
  thickLine2 -fill orange -outline orange -fillStyle outline
dr::createPacket rfdmy -display $display_object -stipple blank \ -lineStyle thickLine2
  -fill yellow -outline yellow
dr::createPacket diode -display $display_object -stipple blank \ -lineStyle thickLine2
  -fill purple -outline purple -fillStyle outline
dr::createPacket exclude -display $display_object -stipple dots \ -lineStyle solid
  -fill pink -outline pink -fillStyle outline
dr::createPacket psub -display $display_object -stipple blank \ -lineStyle solid -fill
  green -outline green
dr::createPacket m1b -display $display_object -stipple slash \ -lineStyle solid -fill
  green -outline green
dr::createPacket m2b -display $display_object -stipple slash \ -lineStyle solid -fill
  yellow -outline yellow
dr::createPacket m3b -display $display_object -stipple slash \ -lineStyle solid -fill
  green -outline green
dr::createPacket m4b -display $display_object -stipple slash \ -lineStyle solid -fill
  pink -outline pink
dr::createPacket v1b -display $display_object -stipple solid \ -lineStyle solid -fill
  cadetBlue -outline cadetBlue -fillStyle X
dr::createPacket v2b -display $display_object -stipple solid \ -lineStyle solid -fill
  orange -outline orange -fillStyle X
dr::createPacket v3b -display $display_object -stipple solid \ -lineStyle solid -fill
  purple -outline purple -fillStyle X
dr::createPacket v4b -display $display_object -stipple solid \ -lineStyle solid -fill
  magenta -outline magenta -fillStyle X
dr::createPacket v5b -display $display_object -stipple solid \ -lineStyle solid -fill
  orange -outline orange -fillStyle X
dr::createPacket v6b -display $display_object -stipple solid \ -lineStyle solid -fill
  navy -outline navy -fillStyle X
dr::createPacket v7b -display $display_object -stipple solid \ -lineStyle solid -fill
  yellow -outline yellow -fillStyle X
dr::createPacket v8b -display $display_object -stipple solid \ -lineStyle solid -fill
  cream -outline cream -fillStyle X
dr::createPacket m1net -display $display_object -stipple slash \ -lineStyle solid
  -fill green -outline green
dr::createPacket m2net -display $display_object -stipple slash \ -lineStyle solid
  -fill yellow -outline yellow
dr::createPacket m3net -display $display_object -stipple slash \ -lineStyle solid
  -fill green -outline green

```


Appendix B: Sample Display Resource File Display Resource File Sample

```

dr::createPacket m4net -display $display_object -stipple slash \ -lineStyle solid
  -fill navy -outline navy
dr::createPacket v1net -display $display_object -stipple solid \ -lineStyle solid
  -fill cadetBlue -outline cadetBlue -fillStyle X
dr::createPacket v2net -display $display_object -stipple solid \ -lineStyle solid
  -fill orange -outline orange -fillStyle X
dr::createPacket v3net -display $display_object -stipple solid \ -lineStyle solid
  -fill purple -outline purple -fillStyle X
dr::createPacket v4net -display $display_object -stipple solid \ -lineStyle solid
  -fill magenta -outline magenta -fillStyle X
dr::createPacket v5net -display $display_object -stipple solid \ -lineStyle solid
  -fill orange -outline orange -fillStyle X
dr::createPacket v6net -display $display_object -stipple solid \ -lineStyle solid
  -fill navy -outline navy -fillStyle X
dr::createPacket v7net -display $display_object -stipple solid \ -lineStyle solid
  -fill yellow -outline yellow -fillStyle X
dr::createPacket v8net -display $display_object -stipple solid \ -lineStyle solid
  -fill cream -outline cream -fillStyle X
dr::createPacket m1slot -display $display_object -stipple slash \ -lineStyle solid
  -fill blue -outline blue
dr::createPacket m2slot -display $display_object -stipple slash \ -lineStyle solid
  -fill yellow -outline yellow
dr::createPacket m3slot -display $display_object -stipple slash \ -lineStyle solid
  -fill green -outline green
dr::createPacket m4slot -display $display_object -stipple slash \ -lineStyle solid
  -fill navy -outline navy
dr::createPacket m5slot -display $display_object -stipple slash \ -lineStyle solid
  -fill pink -outline pink
dr::createPacket m6slot -display $display_object -stipple slash \ -lineStyle solid
  -fill cream -outline cream
dr::createPacket m7slot -display $display_object -stipple slash \ -lineStyle solid
  -fill orange -outline orange
dr::createPacket m8slot -display $display_object -stipple slash \ -lineStyle solid
  -fill purple -outline purple
dr::createPacket m9slot -display $display_object -stipple slash \ -lineStyle solid
  -fill magenta -outline magenta
dr::createPacket bound -display $display_object -stipple blank \ -lineStyle solid
  -fill pink -outline pink -fillStyle outline
dr::createPacket momDmy1 -display $display_object -stipple blank \ -lineStyle
  thickLine2 -fill lime -outline lime -fillStyle outline
dr::createPacket momDmy2 -display $display_object -stipple blank \ -lineStyle
  thickLine2 -fill yellow -outline yellow -fillStyle outline
dr::createPacket momDmy3 -display $display_object -stipple blank \ -lineStyle
  thickLine2 -fill green -outline green -fillStyle outline
dr::createPacket momDmy4 -display $display_object -stipple blank \ -lineStyle
  thickLine2 -fill blue -outline blue -fillStyle outline
dr::createPacket momDmy5 -display $display_object -stipple blank \ -lineStyle
  thickLine2 -fill cream -outline cream -fillStyle outline
dr::createPacket momDmy6 -display $display_object -stipple blank \ -lineStyle
  thickLine2 -fill magenta -outline magenta -fillStyle outline
dr::createPacket momDmy7 -display $display_object -stipple blank \ -lineStyle
  thickLine2 -fill navy -outline navy -fillStyle outline
dr::createPacket momDmy8 -display $display_object -stipple blank \ -lineStyle
  thickLine2 -fill orange -outline orange -fillStyle outline
dr::createPacket momDmy9 -display $display_object -stipple blank \ -lineStyle
  thickLine2 -fill purple -outline purple -fillStyle outline
dr::createPacket nwellnet -display $display_object -stipple dotsa \ -lineStyle solid
  -fill magenta -outline magenta
dr::createPacket diffDmy -display $display_object -stipple dots1 \ -lineStyle solid
  -fill yellow -outline yellow
dr::createPacket polyDmy -display $display_object -stipple cross \ -lineStyle solid
  -fill maroon -outline maroon
dr::createPacket polynet -display $display_object -stipple cross \ -lineStyle solid
  -fill maroon -outline maroon

```

Appendix B: Sample Display Resource File Display Resource File Sample

```

dr::createPacket pplusDmy -display $display_object -stipple dots1 \ -lineStyle solid
  -fill red -outline red
dr::createPacket contb -display $display_object -stipple solid \ -lineStyle solid
  -fill white -outline white -fillStyle X
dr::createPacket contnet -display $display_object -stipple solid \ -lineStyle solid
  -fill white -outline white -fillStyle X
dr::createPacket exDmyd -display $display_object -stipple hZigZag \ -lineStyle solid
  -fill lime -outline lime
dr::createPacket exDmy0 -display $display_object -stipple hZigZag \ -lineStyle solid
  -fill lime -outline lime
dr::createPacket exDmy1 -display $display_object -stipple hZigZag \ -lineStyle solid
  -fill lime -outline lime
dr::createPacket exDmy2 -display $display_object -stipple hZigZag \ -lineStyle solid
  -fill lime -outline lime
dr::createPacket exDmy3 -display $display_object -stipple hZigZag \ -lineStyle solid
  -fill lime -outline lime
dr::createPacket exDmy4 -display $display_object -stipple hZigZag \ -lineStyle solid
  -fill lime -outline lime
dr::createPacket exDmy5 -display $display_object -stipple hZigZag \ -lineStyle solid
  -fill lime -outline lime
dr::createPacket exDmy6 -display $display_object -stipple hZigZag \ -lineStyle solid
  -fill lime -outline lime
dr::createPacket exDmy7 -display $display_object -stipple hZigZag \ -lineStyle solid
  -fill lime -outline lime
dr::createPacket exDmy8 -display $display_object -stipple hZigZag \ -lineStyle solid
  -fill lime -outline lime
dr::createPacket exDmy9 -display $display_object -stipple hZigZag \ -lineStyle solid
  -fill lime -outline lime
dr::createPacket ipLayer -display $display_object -stipple solid \ -lineStyle solid
  -fill white -outline white
dr::createPacket mltxt -display $display_object -stipple solid \ -lineStyle solid
  -fill white -outline white
dr::createPacket m2txt -display $display_object -stipple solid \ -lineStyle solid
  -fill yellow -outline yellow
dr::createPacket m3txt -display $display_object -stipple solid \ -lineStyle solid
  -fill orange -outline orange
dr::createPacket m4txt -display $display_object -stipple solid \ -lineStyle solid
  -fill pink -outline pink
dr::createPacket m5txt -display $display_object -stipple solid \ -lineStyle solid
  -fill green -outline green
dr::createPacket m6txt -display $display_object -stipple solid \ -lineStyle solid
  -fill purple -outline purple
dr::createPacket m7txt -display $display_object -stipple solid \ -lineStyle solid
  -fill yellow -outline yellow
dr::createPacket m8txt -display $display_object -stipple solid \ -lineStyle solid
  -fill cream -outline cream
dr::createPacket m9txt -display $display_object -stipple solid \ -lineStyle solid
  -fill yellow -outline yellow
dr::createPacket polytxt -display $display_object -stipple solid \ -lineStyle solid
  -fill gray -outline gray

#Packet Aliases Definition
dr::createPacketAlias stroke -packet stroke white -display $display_object
dr::createPacketAlias select1 -packet select1_yellow -display $display_object
dr::createPacketAlias select2 -packet select2_red -display $display_object
dr::createPacketAlias select3 -packet select3_blue -display $display_object
dr::createPacketAlias select4 -packet select4_purple -display $display_object
dr::createPacketAlias highlight1 -packet highlight1_yellow -display $display_object
dr::createPacketAlias highlight2 -packet highlight2_red -display $display_object
dr::createPacketAlias highlight3 -packet highlight3_green -display $display_object
dr::createPacketAlias highlight4 -packet highlight4_blue -display $display_object
dr::createPacketAlias highlight5 -packet highlight5_purple -display $display_object
dr::createPacketAlias highlight6 -packet highlight6_cyan -display $display_object
dr::createPacketAlias highlight7 -packet highlight7_white -display $display_object

```

Appendix B: Sample Display Resource File Display Resource File Sample

```

dr::createPacketAlias highlight8 -packet highlight8_gold -display $display_object
dr::createPacketAlias highlight9 -packet highlight9_maroon -display $display_object
dr::createPacketAlias highlight10 -packet highlight10_gray -display $display_object
dr::createPacketAlias highlight1dim -packet highlight1_yellow_dim -display
$display_object
dr::createPacketAlias highlight2dim -packet highlight2_red_dim -display
$display_object
dr::createPacketAlias highlight3dim -packet highlight3_green_dim -display
$display_object
dr::createPacketAlias highlight4dim -packet highlight4_blue_dim -display
$display_object
dr::createPacketAlias highlight5dim -packet highlight5_purple_dim -display
$display_object
dr::createPacketAlias highlight6dim -packet highlight6_cyan_dim -display
$display_object
dr::createPacketAlias highlight7dim -packet highlight7_white_dim -display
$display_object
dr::createPacketAlias highlight8dim -packet highlight8_gold_dim -display
$display_object
dr::createPacketAlias highlight9dim -packet highlight9_maroon_dim -display
$display_object
dr::createPacketAlias highlight10dim -packet highlight10_gray_dim -display
$display_object
dr::createPacketAlias coordinate_mark -packet yellow_white -display $display_object
dr::createPacketAlias ghost_mark -packet transparent_white -display $display_object
dr::createPacketAlias ruler -packet ruler_white -display $display_object
dr::createPacketAlias active -packet active_TrueColorBright -display $display_object
dr::createPacketAlias runtime -packet runtime_TrueColorGhost -display $display_object
dr::createPacketAlias axis_white -packet axis -display $display_object
dr::createPacketAlias cursor -packet cursor_white -display $display_object
dr::createPacketAlias display_major_grid -packet major_grid -display $display_object
dr::createPacketAlias display_minor_grid -packet minor_grid -display $display_object
dr::createPacketAlias SE_runtime -packet inherited -display $display_object
dr::createPacketAlias net_highlight -packet highlight2_red -display $display_object
dr::createPacketAlias net_highlight2 -packet annotate -display $display_object
dr::createPacketAlias textAttach -packet annotate5 -display $display_object

#Text Packets Definition
dr::createTextPacket active -display $display_object \
-color TrueColorBright
dr::createTextPacket block -display $display_object -color inherited \
-bold yes
dr::createTextPacket blockName -display $display_object -color green \
-bold yes
dr::createTextPacket blockPort -display $display_object -color red \
-italic yes
dr::createTextPacket deviceName -display $display_object -color green \
-bold yes
dr::createTextPacket instanceName -display $display_object -color green
dr::createTextPacket unboundInstanceName -display $display_object \
-color red -italic yes -underline yes
dr::createTextPacket instanceBlock -display $display_object -color green
dr::createTextPacket instanceSignal -display $display_object \
-color cadetBlue
dr::createTextPacket signal -display $display_object -color cadetBlue
dr::createTextPacket comment -display $display_object -color yellow \
-italic yes
dr::createTextPacket identifier -display $display_object -color brown
dr::createTextPacket includeFile -display $display_object -color red \
-underline yes
dr::createTextPacket keyword -display $display_object -color inherited \
-bold yes
dr::createTextPacket directive -display $display_object -color inherited
dr::createTextPacket whitespace -display $display_object -color inherited

```

Appendix B: Sample Display Resource File Display Resource File Sample

```
dr::createTextPacket dspfNet -display $display_object -color red
dr::createTextPacket other -display $display_object -color inherited
dr::createTextPacket annotate -display $display_object -color tan
dr::createTextPacket highlight1 -display $display_object -color black \
-backgroundColor yellow
dr::createTextPacket highlight2 -display $display_object -color black \
-backgroundColor red
dr::createTextPacket highlight3 -display $display_object -color black \
-backgroundColor green
dr::createTextPacket highlight4 -display $display_object -color black \
-backgroundColor blue
dr::createTextPacket highlight5 -display $display_object -color black \
-backgroundColor purple
dr::createTextPacket highlight6 -display $display_object -color black \
-backgroundColor cyan
dr::createTextPacket highlight7 -display $display_object -color black \
-backgroundColor white
dr::createTextPacket highlight8 -display $display_object -color black \
-backgroundColor gold
dr::createTextPacket highlight9 -display $display_object -color black \
-backgroundColor maroon
dr::createTextPacket highlight10 -display $display_object -color black \
-backgroundColor gray
```

C

Sample Rule Deck File

Presents a sample design rule deck file to demonstrate the file syntax. The sample shown here is in Tcl.

This section covers the following topics:

- [Rule Deck Sample](#)

Rule Deck Sample

```
vr::input -lib lib_name -cell cell_name           ;# specify the design for this check
vr::layer -in PO -layer 9                         ;# input layer PO (layer number 9)
vr::layer -in OD -layer 10
set w_1 0.4                                       ;# TCL variables; native TCL format
set w_2 0.6

vr::input -lib lib_name -cell cell_name           ;# specify the design for this check
vr::layer -in PO -layer 9                         ;# input layer PO (layer number 9)
vr::layer -in OD -layer 10

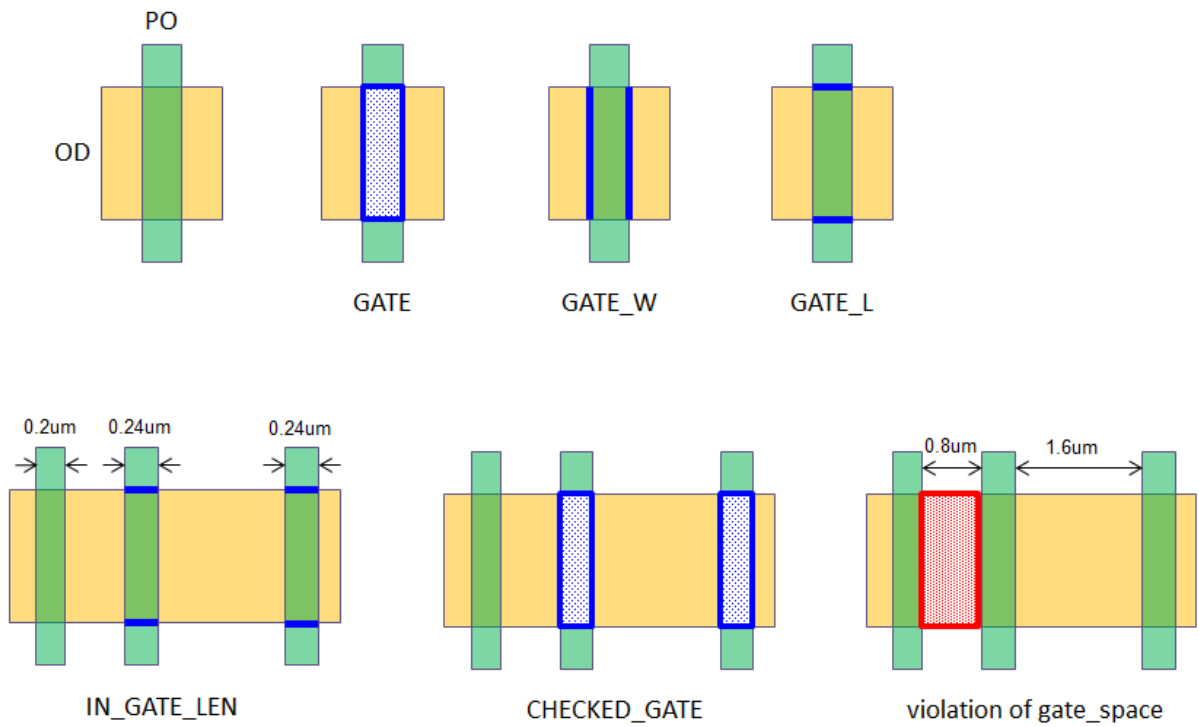
vr::and -in {PO OD} -out GATE                     ;# generate global derived
; layer GATE, GATE_L and GATE_W
vr::coinEdge -inside -in {PO GATE} -out GATE_W    ;# find PO edges which coincident
; with GATE edge
vr::touchEdge -not -in {GATE GATE_W} -out GATE_L  ;# find GATE edges which is
; not in GATE_W

vr::ruleSet -opSet {
  vr::length -in GATE_L -value >0.2<=0.26 -out IN_GATE_LEN ;# select length>0.2
; and <=0.26
  vr::withEdge -in {GATE IN_GATE_LEN} -out CHECKED_GATE
  vr::external -in {CHECKED_GATE GATE} -value <1.2 -opposite ;# report violation
; if space<1.2
} -id gate_space -description {Space to GATE (length>0.2um and <=0.26um)>=1.2 um}

vr::ruleSet -opSet {
  vr::length -in GATE_W -value <$w_1              ;# report violation
; for width != 0.4 and
; width != 0.6

  vr::length -in GATE_W -value >$w_1<$w_2
  vr::length -in GATE_W -value >$w_2
} -id gate_width -description {GATE width = 0.4um/0.6um}
```

Figure 19 *Derived Layers and Violation*



Glossary

constraint

A rule that must be enforced in a design. The Custom Compiler environment references the constraints when performing design rule checks as you lay out the design.

constraint group

A collection of design constraints that is assigned a name and various properties. Corresponds to the OpenAccess oaConstraintGroup object.

DBUPerUU

Database units per user unit.

display object

A hardware device such as a computer monitor or printer used to display the design. The display object is an argument to each display packet definition, and so each display packet can be associated with a specific display.

display packet

A color, line style, and stipple pattern that are associated together for a particular use (such as metal2 layer, or a flightline) and assigned a specific name. Each display packet is assigned to a particular display object, for example a specific printer model.

display resource file

An ASCII file that lists the colors, line styles, and stipples associated with the displays for this technology. It defines display packets and packet aliases that are associated with a particular hardware display such as a printer or computer display.

DRC

Design Rule Check: any check of the validity of the design against a design constraint.

guard ring

Layout structure consisting of paths and polygons. Represented in technology data as a multi-part path (MPP).

hard rule

A rule that has a hard flag set to true in its oaDesignConstraint representation or in the Custom Compiler technology file. This means that the rule has to be enforced, otherwise it is a DRC violation.

See [Constraints Overview](#) for information about supported constraints.

hierarchical technology

Technology data in an OpenAccess database that is organized in tree-like structure of technology libraries.

highly resistive layer

A layer in the `highlyResistiveLayers` constraint group. Connectivity can be directly assigned to objects on the highly resistive layers, and can be inherited from objects on these layers (propagated). Objects on these layers with no connectivity does not inherit connectivity from objects that physically touch them. By default, the Custom Compiler environment assumes layers with the function names `nWell`, `pWell`, `nDiff`, `pDiff`, `nImplant`, `pImplant`, and `diffusion` are highly resistive.

ISTFF

Industry-Standard Technology File Format.

lib.defs

An ASCII file that maps the design libraries to their physical locations on the computer disk. By default, this file is named `lib.defs`, but any legal UNIX file name can be used.

LineStyle

Line width and pattern associated with a LineStyle name.

LPP

Layer purpose pair: An OpenAccess layer (`oaLayer`) and its associated purpose (`oaPurpose`). For example, `metal1` (layer) and `drawing` (purpose).

LPP priorities

A technology file section listing layer purpose pairs in priority order. Layer purpose pairs listed first are drawn first on the canvas. Layer purpose pairs listed later are displayed on top of previously-displayed layers, and might obscure them.

MPP

Multi-part path: A complex layout object that consists of named paths placed relative to each other.

MPP definition

A definition used to create a multi-part path when generating a guard ring.

non-propagating layer

An interconnect layer that can have a net ID propagated to a shape, but that can't propagate connectivity to other shapes.

oaLayer

An OpenAccess database object that represents a layer in the design, such as `metal1`, `poly`, and so forth. The `oaLayer` has a name, number and properties associated with it.

oaPurpose

An OpenAccess database object that represents the purpose for a design layer. The `oaPurpose` has a type, number, and name associated with it.

OLP

Object Layer Panel: the interface for viewing and editing object and layer selectability and visibility.

packet

A combination of a name, fill-color, outline-color, stipple, line style, and fill style associated with the current display object. A display object is any hardware that displays the design.

packetAlias

Any alias for a packet name.

propagating layer

A layer that can propagate connectivity to other shapes.

purpose

The purpose for which a design layer is used, for example for design reasons such as laying out the circuit (“drawing”) or labeling an object in the design (“label”), or for system reasons such as highlighting objects (“highlight”) or showing object boundaries (“boundary”) in the display. Corresponds to the `oaPurpose` value in the OpenAccess database.

stipple

Bitmap pattern associated with a `StippleName` and display name in the technology file.

system layers

Layers reserved by the Custom Compiler environment that match the system-reserved layers in the industry-standard technology file (ISTF).

techArray

Incremental technology data that is created in memory as you reference technology data.

technology file

An ASCII representation of the technology data in a library, that defines layers, layer rules, vias, and constraints that define the design rules. A technology file by default has the extension `.tf`.

tf

Standard filename extension for the ASCII version of a Custom Compiler technology file.

valid routing layer

A layer included in the `validRoutingLayers` constraint group. By default, the Custom Compiler environment assumes layers with the function names `poly`, `cut`, `metal`, and `contactlessMetal` are valid routing layers.